AURIC ARTISAN · USER DOCUMENTATION

VS Code extensions

Every Auric Artisan extension for Visual Studio Code, what each one does, its Marketplace ID and minimum VS Code version, and where its documentation lives.

Updated 2026-09-25 · auricartisan.com

Contents

VS Code extensions

- Install and set up the Auric Artisan VS Code extensions
  - Install and setup guides
    - Install an extension from the Extensions view
    - Use an extension in Hindi
    - The two extension packs
    - Cursor, Windsurf, VSCodium and other editors
    - Get support
    - Install from the command line
    - Install from a VSIX file
    - Settings, sync and workspace trust
    - Update, disable and uninstall
    - Reference: extension IDs and requirements
    - Troubleshooting
    - FAQ
  - Supporting material
    - Shortcuts and commands
    - Glossary
    - Compatibility and limits
    - Privacy
    - Related extensions and pages
- Auric Artisan Studio
  - Auric Artisan Studio docs
    - Getting started
    - Appearance
    - Choose what to install
    - The CI gate
    - The Command Center
    - The Contrast Studio
    - Exports and health records
    - The Icon Studio bridge
    - The Overview view
    - The remediation plan
    - Rulebook and knowledge database
    - Scanning
    - Scores and grades
    - Accessibility of the Studio
    - The Studio
    - Suppression and branding
    - The Workstation
    - Reference
    - Troubleshooting
    - FAQ

- Auric Accessibility — WCAG 2.2 Linter
  - Auric Accessibility docs
    - Getting started with Auric Accessibility
    - Audit profiles and contrast standards
    - The Check space: Findings and Preview
    - Editor linting
    - Inspect
    - Manual evidence
    - The Project Issues view
    - Project scans
    - Rules
    - Auric Accessibility reference
    - Auric Accessibility troubleshooting
    - Auric Accessibility FAQ
  - Auric Accessibility: other resources
    - Auric Accessibility shortcuts
    - Auric Accessibility glossary
    - Auric Accessibility limits and accuracy
    - Auric Accessibility privacy
    - Auric Accessibility release notes
    - Related to Auric Accessibility
- Auric Code Health — Secrets, TODOs & Risk
  - Auric Code Health docs
    - Getting started with Auric Code Health
    - Categories and signatures
    - The Code Health Studio
    - Health records
    - Secrets and redaction
    - Auric Code Health reference
    - Auric Code Health troubleshooting
    - Auric Code Health FAQ
  - Auric Code Health: other resources
    - Auric Code Health shortcuts
    - Auric Code Health glossary
    - Auric Code Health limits and accuracy
    - Auric Code Health privacy
    - Auric Code Health release notes
    - Related to Auric Code Health
- Auric Color — Picker, Palettes & Contrast
  - Auric Color docs
    - Getting started with Auric Color
    - The brand audit
    - The colour pickers
    - Colour vision
    - Contrast diagnostics
    - Hover and swatches

- Palettes and shades
- Picker modes
- Auric Color reference
- Auric Color troubleshooting
- Auric Color FAQ
- Auric Color: other resources
- Auric Color shortcuts
- Auric Color glossary
- Auric Color limits and accuracy
- Auric Color privacy
- Auric Color release notes
- Related to Auric Color
- Auric Artisan Studio — Full Pack
- Full Pack docs
- Getting started with the Full Pack
- Manage the pack's members
- Full Pack reference
- Full Pack troubleshooting
- Full Pack FAQ
- Full Pack: other resources
- Full Pack glossary
- Full Pack privacy
- Full Pack release notes
- Related to the Full Pack
- Other resources
- Shortcuts
- Glossary
- Limits and accuracy
- Privacy
- Release notes
- Related
- Auric Artisan Theme
- Auric Artisan Theme guides
- Getting started with Auric Artisan Theme
- Check contrast
- Choose a theme
- Compare and preview
- Edit individual colours
- Interface language (English and हिन्दी)
- Looks and dials
- Save, share and export
- The Theme sidebar
- Auric Artisan Theme reference
- Troubleshooting
- Frequently asked questions
- Auric Artisan Theme: other information

- Shortcuts and commands
- Glossary
- Limits and accuracy
- Privacy
- Release notes
- Related extensions and tools

Auric Artisan File Icons

- Auric Artisan File Icons guides
  - Getting started with Auric Artisan File Icons
  - Assign icons to files and folders
  - Choose a style
  - Customise the Explorer
  - Insert icons into code
  - Modern and older Playful settings
  - Profiles
  - Smart detection
  - Auric Artisan File Icons reference
  - Troubleshooting
  - Frequently asked questions
- Auric Artisan File Icons: other information
  - Shortcuts and commands
  - Glossary
  - Limits and accuracy
  - Privacy
  - Release notes
  - Related extensions and comparison

Auric Artisan Icon Pack

- Auric Artisan Icon Pack guides
  - Getting started with the Icon Pack
  - Icon Pack reference
  - Troubleshooting
  - Frequently asked questions
- Icon Pack: other information
  - Glossary
  - Limits and accuracy
  - Privacy
  - Release notes
  - Related extensions and comparison

Auric Icons: Modern

- Auric Icons: Modern guides
  - Getting started with Auric Icons: Modern
  - Use the file icon theme
  - Browse the Icon Library
  - Insert, copy and save icons
  - The merged Icon Studio
  - Auric Icons: Modern reference

- Troubleshooting
- Frequently asked questions
- Auric Icons: Modern: other information
  - Shortcuts and commands
  - Glossary
  - Limits and accuracy
  - Privacy
  - Release notes
  - Related extensions and comparison
- Auric Icons: Drawn
- Auric Icons: Drawn guides
  - Getting started with Auric Icons: Drawn
  - Browse the Icon Library
  - Insert, copy and save icons
  - Use Drawn with Modern and Playful
  - Auric Icons: Drawn reference
  - Troubleshooting
  - Frequently asked questions
- Auric Icons: Drawn: other information
  - Shortcuts and commands
  - Glossary
  - Limits and accuracy
  - Privacy
  - Release notes
  - Related extensions and comparison
- Auric Icons: Playful
- Auric Icons: Playful guides
  - Getting started with Auric Icons: Playful
  - Browse the Icon Library
  - Insert, copy and save icons
  - Use Playful with Modern and Drawn
  - Auric Icons: Playful reference
  - Troubleshooting
  - Frequently asked questions
- Auric Icons: Playful: other information
  - Shortcuts and commands
  - Glossary
  - Limits and accuracy
  - Privacy
  - Release notes
  - Related extensions and comparison
- Auric Artisan Emoji
- Auric Artisan Emoji guides
  - Getting started with Auric Artisan Emoji
  - Use Emoji Studio
  - Insert from the editor

- Inspector, skin tones and output formats
- Interface language (English and हिन्दी)
- Compose a sequence
- Auric Artisan Emoji reference
- Troubleshooting
- Frequently asked questions
- Auric Artisan Emoji: other information
- Shortcuts and commands
- Glossary
- Limits and accuracy
- Privacy
- Release notes
- Related extensions and comparison
- Auric Artisan Icon Studio
- Icon Studio guides
- Getting started with Icon Studio
- Work with the Auric collection extensions
- Commands and keybindings
- Insert, copy and save
- Search and browse
- Icon Studio reference
- Troubleshooting
- Frequently asked questions
- Icon Studio: other information
- Shortcuts and commands
- Glossary
- Limits and accuracy
- Privacy
- Release notes
- Related extensions and comparison
- Auric Artisan Icon Hub
- Icon Hub guides
- Getting started with Icon Hub
- Browse and refine
- Catalog metadata updates
- Connect the icon CDN
- Collect and export
- Insert, copy and save
- Shape the output
- Icon Hub reference
- Troubleshooting
- Frequently asked questions
- Icon Hub: other information
- Shortcuts and commands
- Glossary
- Limits and accuracy

- Privacy
- Release notes
- Related extensions and comparison

Auric Artisan Live

- Auric Artisan Live guides
  - Getting started with Auric Artisan Live
  - Proxy an API, mock endpoints and a REST database
  - Use the in-page browser dashboard
  - Serve over HTTPS
  - Inspect requests
  - Simulate a slow network
  - Run PHP sites
  - Preview, devices and phone testing
  - Project configuration
  - Run a backend or framework dev server
  - Serve a folder with live reload
  - Reference: commands, keys and settings
  - Troubleshooting
  - FAQ
- Supporting material
  - Shortcuts
  - Glossary
  - Limits and safeguards
  - Privacy
  - Release notes
  - Related

Auric Artisan File System

- Auric Artisan File System guides
  - Getting started with Auric Artisan File System
  - Create files and structures
  - History, undo and the workspace trash
  - Tags, groups and profiles
  - Properties, compare and export
  - Related files
  - Rename, move and copy
  - Search, filters and duplicates
  - Templates and generators
  - Reference: commands, keys and settings
  - Troubleshooting
  - FAQ
- Supporting material
  - Shortcuts
  - Glossary
  - Limits and accuracy
  - Privacy
  - Release notes

- Related
- Auric Artisan Font Hub
  - Auric Artisan Font Hub guides
    - Getting started with Auric Artisan Font Hub
    - Add a font to your code
    - Check CDN and browser delivery
    - Find and compare fonts
    - Font details
    - Moving from Font Library
    - Offline use, cache and network
    - Export a self-hosting kit
    - Reference: commands, keys and settings
    - Troubleshooting
    - FAQ
  - Supporting material
    - Shortcuts
    - Glossary
    - Limits and accuracy
    - Privacy
    - Release notes
    - Related
- Auric Artisan Formatter
  - Auric Artisan Formatter guides
    - Getting started with Auric Artisan Formatter
    - Command line and CI
    - Configuration
    - Format files and format on save
    - Formatter Studio
    - Options reference
    - Presets
    - Preview formatting as a diff
    - The safety gate and formatting tiers
    - Supported languages
    - Scan the workspace
    - Reference: commands, keys and settings
    - Troubleshooting
    - FAQ
  - Supporting material
    - Shortcuts
    - Glossary
    - Limits and accuracy
    - Privacy
    - Release notes
    - Related

VS CODE EXTENSIONS

VS Code extensions

Every Auric Artisan extension for Visual Studio Code, what each one does, its Marketplace ID and minimum VS Code version, and where its documentation lives.

Auric Artisan publishes seventeen free extensions for Visual Studio Code under the publisher **auric-artisan**. They bring the same design system as the website into your editor: colour and accessibility linting, a theme with a live Theme Studio, icon and emoji libraries, file icon themes, a live server, a file workspace, a font hub and a formatter. None of them sends telemetry.

Two of them are packs that install a set in one step: the **Icon Pack** installs Modern, Drawn, Playful and Emoji, and the **Studio Full Pack** installs Color, Accessibility and Code Health. Every member stays its own extension that you can disable on its own.

New to installing extensions? Start with Install and set up.

Themes and file icons

Extension	Marketplace ID	VS Code	Documentation
Auric Artisan Theme	`auric-artisan.auric-artisan-theme`	1.75+	auric-artisan-theme
Auric Artisan File Icons	`auric-artisan.auric-artisan-file-icon`	1.85+	auric-artisan-file-icons

Icons and emoji

Extension	Marketplace ID	VS Code	Documentation
Auric Artisan Icon Pack	`auric-artisan.auric-artisan-icon-pack`	1.85+	auric-artisan-icon-pack
Auric Icons: Modern	`auric-artisan.auric-artisan-modern-icon`	1.85+	auric-icons-modern
Auric Icons: Drawn	`auric-artisan.auric-icons-drawn`	1.85+	auric-icons-drawn
Auric Icons: Playful	`auric-artisan.auric-icons-playful`	1.85+	auric-icons-playful
Auric Artisan Emoji	`auric-artisan.auric-artisan-emoji`	1.85+	auric-artisan-emoji
Auric Artisan Icon Studio	`auric-artisan.auric-artisan-icon-studio`	1.85+	auric-artisan-icon-studio
Auric Artisan Icon Hub	`auric-artisan.auric-artisan-icon-lib`	1.85+	auric-artisan-icon-hub

Colour, accessibility and code health — Auric Artisan Studio

All four are documented together in Auric Artisan Studio.

Extension	Marketplace ID	VS Code	Documentation
Auric Artisan Studio — Full Pack	`auric-artisan.auric-artisan`	1.74+	full-pack
Auric Color	`auric-artisan.auric-color`	1.74+	color
Auric Accessibility	`auric-artisan.auric-a11y`	1.74+	accessibility
Auric Code Health	`auric-artisan.auric-codehealth`	1.74+	code-health

Workspace tools

Extension	Marketplace ID	VS Code	Documentation
Auric Artisan Live	`auric-artisan.auric-artisan-live`	1.95+	auric-artisan-live
Auric Artisan File System	`auric-artisan.auric-artisan-file-system`	1.95+	auric-artisan-file-system
Auric Artisan Font Hub	`auric-artisan.auric-artisan-font-hub`	1.85+	auric-artisan-font-hub
Auric Artisan Formatter	`auric-artisan.auric-artisan-formatter`	1.74+	auric-artisan-formatter

Install in one line

Run this in a terminal, replacing the ID with any from the tables above:

```
code --install-extension auric-artisan.auric-artisan-icon-pack
code --install-extension auric-artisan.auric-artisan
```

Those two lines install both packs — five extensions. The install guide covers the Extensions view, VSIX files, forks such as Cursor, Windsurf and VSCodium, updating, and removing extensions.

Which extension do I need?

You want to…	Install
Give the whole editor a new look	Auric Artisan Theme
Put icons on files and folders in the Explorer	File Icons or Modern
Insert icons into your code	Icon Studio, a collection, or Icon Hub for open-source sets
Insert emoji by shortcode	Emoji
Check colour contrast as you type	Auric Color
Lint markup for WCAG 2.2	Auric Accessibility
Catch secrets and risky code before they ship	Auric Code Health
Preview a site with live reload	Live
Rename and organise files in bulk	File System
Add a web font to a project	Font Hub
Format code without surprises	Formatter

In this folder

- install-and-setup/
- auric-artisan-studio/
- auric-artisan-theme/
- auric-artisan-file-icons/
- auric-artisan-icon-pack/

- auric-icons-modern/
- auric-icons-drawn/
- auric-icons-playful/
- auric-artisan-emoji/
- auric-artisan-icon-studio/
- auric-artisan-icon-hub/
- auric-artisan-live/
- auric-artisan-file-system/
- auric-artisan-font-hub/
- auric-artisan-formatter/

VS CODE EXTENSIONS › INSTALL AND SETUP

Install and set up the Auric Artisan VS Code extensions

How to install, update, disable, remove and configure any of the seventeen Auric Artisan extensions for Visual Studio Code.

Auric Artisan publishes seventeen extensions for Visual Studio Code under the publisher name `auric-artisan`. They cover colour themes, file icons, icon and emoji libraries, colour and accessibility tooling, code health, a live development server, a file workspace, a font hub and a code formatter. Every extension is free and installs on its own. Two of them are *extension packs*: a pack contains no features of its own and simply installs a group of other extensions in one step.

This folder covers everything that is the same for all seventeen: finding and installing an extension, installing from the command line or from a VSIX file, using the packs, installing in editors built on VS Code (Cursor, Windsurf, VSCodium), updating, disabling and removing, where settings live, how Settings Sync treats them, switching the interface to Hindi where an extension supports it, and where to report a problem.

The guides for what each extension does live in that extension's own folder. This folder only gets you to the point where the extension is installed and configured the way you want.

Who it is for

- Anyone installing an Auric Artisan extension for the first time.
- Teams that want to install the same set of extensions on several machines, or share settings through a repository.
- People using Cursor, Windsurf, VSCodium or another editor based on VS Code.
- Anyone who needs to update, roll back, disable or remove an extension, or report a problem.

Where to find the extensions

- The Visual Studio Marketplace, publisher `auric-artisan`. Each extension has a page at `https://marketplace.visualstudio.com/items?itemName=<extension ID>`.
- The Auric Artisan VS Code page: https://auricartisan.com/vscode/
- Inside VS Code: the Extensions view (`Ctrl` + `Shift` + `X`).

The full list of extension IDs, with the minimum VS Code version each one needs, is in the reference.

To install from a terminal, pass the extension ID to the VS Code command-line launcher:

```
code --install-extension auric-artisan.auric-artisan-live
```

Quick start

1. Open VS Code and check its version under **Help** › **About** (on macOS, **Code** › **About Visual Studio Code**). Compare it with the minimum in the reference.
2. Open the Extensions view with `Ctrl` + `Shift` + `X`.
3. Type the exact ID with the `@id:` prefix, for example `@id:auric-artisan.auric-artisan-formatter`, and clear any other filter such as `@installed`.
4. Check that the publisher shown under the name is **Auric Artisan** (`auric-artisan`), then select **Install**.
5. Open the Command Palette (`Ctrl` + `Shift` + `P`) and type the extension's command prefix, for example `Auric Formatter`, to see its commands.

6. Open **File** › **Preferences** › **Settings** and search for the extension's setting prefix (for example `auricFormatter`) to review its options.
7. If VS Code asks whether you trust the folder you opened, read Settings, sync and workspace trust before you answer: several extensions limit what they do in an untrusted folder.

What you can do

- Install any extension from the Extensions view, the command line or a VSIX file.
- Install a whole group in one step with the Icon Pack or the Studio Full Pack.
- Install in Cursor, Windsurf, VSCodium and other editors based on VS Code.
- Update, roll back to an earlier version, disable or uninstall an extension.
- Choose between user settings and workspace settings, and share a configuration with a team.
- Keep extensions and settings in step across machines with VS Code Settings Sync.
- Use the extensions in Hindi where they support it: Auric Artisan Live, Auric Artisan Theme and Auric Artisan Emoji.
- Get support, with the information support needs.

In this folder

- docs/ — the guides.
  - Getting started: install from the Extensions view
  - Install from the command line
  - Install from a VSIX file
  - The two extension packs
  - Cursor, Windsurf, VSCodium and other editors
  - Update, disable and uninstall
  - Settings, sync and workspace trust
  - Use an extension in Hindi
  - Get support
  - Reference: all seventeen extension IDs
  - Troubleshooting
  - FAQ
- others/ — supporting material.
  - Glossary
  - Shortcuts and commands
  - Compatibility and limits
  - Privacy
  - Related extensions and pages

VS CODE EXTENSIONS › INSTALL AND SETUP

Install and setup guides

Contents of the installation and setup guides for the Auric Artisan VS Code extensions.

These guides apply to every Auric Artisan extension for VS Code. Start with the getting-started guide if you have never installed an extension before. The reference lists all seventeen extension IDs and the minimum VS Code version each one needs.

File	What it covers
getting-started.md	Find an extension in the Extensions view, check it is the right one and install it
install-from-command-line.md	Install, list and remove extensions with `code --install-extension` and related flags
install-from-vsix.md	Download a VSIX file and install it with **Extensions: Install from VSIX...**
extension-packs.md	The Icon Pack and the Studio Full Pack: what they install and how to trim them
forks-and-other-editors.md	Cursor, Windsurf, VSCodium and other editors based on VS Code
update-disable-uninstall.md	Automatic updates, installing a specific version, disabling and removing
settings-and-sync.md	User and workspace settings, project files, Settings Sync, workspace trust
display-language-hindi.md	Switch Auric Artisan Live, Theme and Emoji to Hindi
get-support.md	Where to get support and what to include
reference.md	All seventeen extension IDs, minimum VS Code versions and command-line flags
troubleshooting.md	Symptom, cause and fix for installation and setup problems
faq.md	Short answers to common questions

VS CODE EXTENSIONS › INSTALL AND SETUP

Install an extension from the Extensions view

Find the right Auric Artisan extension in VS Code, install it, and confirm that it is working.

The Extensions view is the panel inside VS Code where you search the Visual Studio Marketplace and install, update and manage extensions. This is the simplest way to install any Auric Artisan extension, and it needs a network connection to the Marketplace.

Before you start

- **Check your VS Code version.** Open **Help** › **About** (on macOS, **Code** › **About Visual Studio Code**). Each extension states the lowest version it runs on. The lowest version across the family is 1.74; Auric Artisan Live and Auric Artisan File System need 1.95. The reference lists every extension.
- **Open a folder if the extension works on a project.** Auric Artisan Live, File System and Formatter act on the folder you have open. The icon, theme and library extensions work without one.

Install

1. Open the Extensions view: select the Extensions icon in the Activity Bar, or press `Ctrl` + `Shift` + `X`.
2. In the search box, type the extension ID with the `@id:` prefix. For example:

```
@id:auric-artisan.auric-artisan-live
```

The `@id:` prefix returns exactly one extension. You can also type a name such as `Auric Artisan` and browse the results.

3. Remove any other filter from the search box, such as `@installed` or `@disabled`. A leftover filter hides extensions you have not installed yet.
4. Select the result. Check that the publisher shown under the name is **Auric Artisan** and that the identifier on the page matches the ID you searched for.
5. Select **Install**.
6. If VS Code shows a trust prompt for the publisher, read it and confirm.

When installation finishes, the **Install** button changes to **Disable** and **Uninstall** (or a settings gear), and the extension is active. Most Auric Artisan extensions do not need a restart.

Check it worked

- **Command Palette.** Press `Ctrl` + `Shift` + `P` and type the extension's command prefix. Each extension groups its commands under a prefix, shown in the table below.
- **Activity Bar or Explorer.** Some extensions add their own icon to the Activity Bar (for example **Auric Live** and **Auric Font Hub**) or a section to the Explorer (for example **Auric Files** and **Auric Formatter**).
- **Features tab.** On the extension's page in the Extensions view, the **Features** tab lists every command, setting and keyboard shortcut the extension contributes.

Extension	Command Palette prefix	Settings prefix
Auric Artisan Live	`Auric Live`	`auricLive.`
Auric Artisan File System	`Auric Files`	`auricFiles.`
Auric Artisan Font Hub	`Auric Artisan Font Hub` (and `Auric Font Hub` for **Show Logs**)	`auricFontLib.`
Auric Artisan Formatter	`Auric Formatter`	`auricFormatter.`
Auric Artisan Theme	`Auric Theme`	`auricTheme.`
Auric Artisan File Icons	`Auric Artisan File Icons`	`auricFileIcons.`
Auric Icons: Modern	`Auric Icons: Modern`	`auricIcons.`
Auric Icons: Drawn	`Auric Icons: Drawn`	`auricIconsDrawn.`
Auric Icons: Playful	`Auric Icons: Playful`	`auricIconsPlayful.`
Auric Artisan Emoji	`Auric Artisan Emoji`	`auricEmoji.`
Auric Artisan Icon Studio	`Auric Icon Studio`	`auricIconStudio.`
Auric Artisan Icon Hub	`Auric Icon Hub`	`auricIconLib.`
Auric Color	`Auric Artisan`	`auricColor.`
Auric Accessibility	`Auric Artisan`	`auricA11y.`
Auric Code Health	`Auric Artisan`	`auricCodeHealth.`

The two packs, Auric Artisan Icon Pack and Auric Artisan Studio — Full Pack, add no commands or settings of their own. See The two extension packs.

Install several at once

- Install a pack to get a set in one step. See The two extension packs.
- Run several `code --install-extension` lines in a terminal. See Install from the command line.
- Add the IDs to your repository's `.vscode/extensions.json` so VS Code recommends them to everyone who opens the project. See Settings, sync and workspace trust.

Next steps

- Configure the extension: Settings, sync and workspace trust.
- Use it in Hindi, where supported: Use an extension in Hindi.
- Learn the extension itself: open its folder from Related extensions and pages.

You should now see the extension listed under **Installed** in the Extensions view, and its commands in the Command Palette.

VS CODE EXTENSIONS › INSTALL AND SETUP

Use an extension in Hindi

Switch Auric Artisan Live, Auric Artisan Theme and Auric Artisan Emoji to Hindi, and understand which labels follow VS Code's own display language.

Three Auric Artisan extensions include Hindi in the extension itself, with no separate download and no translation service: **Auric Artisan Live**, **Auric Artisan Theme** and **Auric Artisan Emoji**. The other fourteen extensions use English.

Two separate things decide the language you see:

- **The extension's interface language.** This controls the extension's own panels, such as the Auric Live control panel, Theme Studio or Emoji Studio. Each of the three extensions has its own switch and setting.
- **VS Code's display language.** This controls native VS Code labels: Command Palette titles, sidebar view names, the Settings editor and VS Code's own notifications. Change it with the VS Code command **Configure Display Language**.

The interface-language setting

Each of the three extensions has a setting with the same three values.

Extension	Setting	Values	Default	What it changes
Auric Artisan Live	`auricLive.interfaceLanguage`	`auto`, `en`, `hi`	`auto`	The Auric Live control panel, quick settings, Phone QR guidance, preview toolbar and common messages
Auric Artisan Theme	`auricTheme.interfaceLanguage`	`auto`, `en`, `hi`	`auto`	Theme Studio and the Theme sidebar
Auric Artisan Emoji	`auricEmoji.interfaceLanguage`	`auto`, `en`, `hi`	`auto`	Emoji Studio

- `auto` follows VS Code's display language: Hindi locales, including regional Hindi locales, use Hindi; every other locale uses English.
- `en` always uses English.
- `hi` always uses Hindi.

The Theme and Emoji settings have **application** scope, so they can only be set in user settings. The Live setting can also be set per workspace.

Switch Auric Artisan Live to Hindi

1. Open the Auric Live control panel: run **Auric Live: Open Control Panel** (`Ctrl` + `Alt` + `Shift` + `I`).
2. In the panel header, choose **हिन्दी**. The other choices are **Auto** and **English**.

The choice is saved in `auricLive.interfaceLanguage`. Switching does not restart the server, and the panel keeps its current section and request filters.

What changes and what does not:

- The control panel, quick settings, request controls, inline Phone QR guidance and common extension messages appear in Hindi.
- New preview tabs use the selected language. Close and reopen an existing preview tab to update its toolbar.
- Technical setting descriptions, detailed diagnostics and messages without a Hindi translation stay in English.
- The in-page browser dashboard (opened with `Alt` + `Shift` + `D` inside a served page) and the website being served are not translated.
- URLs, request and response content, commands and configuration values are never rewritten.

Switch Auric Artisan Theme to Hindi

Use the **Auto · English · हिन्दी** control in Theme Studio or the Theme sidebar, or run **Auric Theme: Choose Interface Language...**. The switch changes both Theme panels at once, without a reload, and needs no language pack. It does not change VS Code's own display language.

Switch Auric Artisan Emoji to Hindi

Use the **Auto / English / हिन्दी** switch in Emoji Studio, run **Auric Artisan Emoji: Choose Interface Language...**, or set `auricEmoji.interfaceLanguage`. Switching keeps your search, filters and sequence. Emoji names, shortcodes and code points keep their catalogue spelling, but common Hindi words also find emoji when you search.

Command titles and view names in Hindi

Command Palette titles, sidebar view names and Settings descriptions come from VS Code, which shows them in its display language.

1. Open the Command Palette (`Ctrl` + `Shift` + `P`).
2. Run **Configure Display Language**.
3. Choose a Hindi display language if your VS Code offers one, and restart VS Code when asked.

With a Hindi display language:

- Auric Artisan Live shows all 54 command titles and its three sidebar view names (**Servers**, **Requests**, **Routes and Mocks**) in Hindi.
- Auric Artisan Theme and Auric Artisan Emoji show their command titles in Hindi.
- With `interfaceLanguage` set to `auto`, the three extensions' panels also switch to Hindi.

If Hindi is not available as a VS Code display language on your installation, set the extension's `interfaceLanguage` to `hi`. The extension's panels then use Hindi while VS Code's own labels stay in English.

Extensions without Hindi

Auric Artisan File System, Font Hub, Formatter, File Icons, Icon Studio, Icon Hub, Auric Icons: Modern, Drawn and Playful, Auric Color, Auric Accessibility and Auric Code Health currently use English only.

After switching, the panel's headings and buttons appear in Hindi straight away.

VS CODE EXTENSIONS › INSTALL AND SETUP

The two extension packs

What the Icon Pack and the Studio Full Pack install, how to install them, and how to keep only the parts you want.

An *extension pack* is an extension that contains no features of its own. It lists other extensions, and installing it installs all of them. Auric Artisan publishes two.

Pack	Extension ID	Installs	Minimum VS Code
Auric Artisan Icon Pack	`auric-artisan.auric-artisan-icon-pack`	Auric Icons: Modern, Auric Icons: Playful, Auric Artisan Emoji, Auric Icons: Drawn	1.85
Auric Artisan Studio — Full Pack	`auric-artisan.auric-artisan`	Auric Color, Auric Accessibility, Auric Code Health	1.74

Note the Full Pack's ID: it is the bare `auric-artisan.auric-artisan`, not `auric-artisan.auric-artisan-studio`.

Icon Pack

The Icon Pack installs the four creative libraries together:

- Auric Icons: Modern — `auric-artisan.auric-artisan-modern-icon`
- Auric Icons: Drawn — `auric-artisan.auric-icons-drawn`
- Auric Icons: Playful — `auric-artisan.auric-icons-playful`
- Auric Artisan Emoji — `auric-artisan.auric-artisan-emoji`

The pack adds nothing of its own. Each library keeps its own commands and settings. When more than one icon collection is installed, they merge into one Icon Studio instead of adding separate sidebars.

Studio Full Pack

The Full Pack installs the three focused Studio extensions:

- Auric Color — `auric-artisan.auric-color`
- Auric Accessibility — `auric-artisan.auric-a11y`
- Auric Code Health — `auric-artisan.auric-codehealth`

You can enable or disable each of the three independently.

Install a pack

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Search for the pack's ID with the `@id:` prefix, for example `@id:auric-artisan.auric-artisan-icon-pack`.
3. Select **Install**. VS Code installs the pack and every extension it lists.

Or, in a terminal:

```
code --install-extension auric-artisan.auric-artisan-icon-pack
code --install-extension auric-artisan.auric-artisan
```

The pack's page in the Extensions view has an **Extension Pack** tab that lists its members.

Keep only some members

- **Disable a member you do not want.** Find it in the Extensions view and select **Disable**. The rest of the pack keeps working.
- **Uninstall a member.** You can uninstall an individual member. VS Code may reinstall it if you later reinstall or update the pack.
- **Remove the whole set.** Uninstalling a pack offers to uninstall the extensions it installed as well. Read the prompt and choose accordingly.
- **Install members individually instead.** If you only want one or two, skip the pack and install those IDs directly.

Packs and VSIX files

A pack installed from a VSIX file only fetches its members if the editor can reach the Visual Studio Marketplace. In Cursor, Windsurf, VSCodium or an offline machine, install each member's VSIX. See Install from a VSIX file.

After installing a pack, the Extensions view lists the pack and each of its members under **Installed**.

VS CODE EXTENSIONS › INSTALL AND SETUP

Cursor, Windsurf, VSCodium and other editors

Install the Auric Artisan VS Code extensions in editors built on VS Code that use a different extension gallery.

Several editors are built from the VS Code source code. People call them *forks*. Cursor, Windsurf and VSCodium are common examples. They run most VS Code extensions, but many of them search a different extension gallery, usually Open VSX, instead of the Visual Studio Marketplace.

The Auric Artisan extensions are published only on the Visual Studio Marketplace. They are not on Open VSX, so a search inside one of these editors may find nothing. The extensions still work in forks that keep VS Code's extension, colour theme and icon theme interfaces; you install them from a VSIX file.

Install in a fork

1. **Find the VS Code version your editor is based on.** Open the editor's **About** dialog. Most forks show a VS Code version there. Compare it with the extension's minimum in the reference.
2. **Download the VSIX** of the extension you want from the Marketplace, as described in Install from a VSIX file. If you want a pack, download each member instead; see The two extension packs.

3. **Install it** in the editor:
   - Command Palette › **Extensions: Install from VSIX...**, or
   - the Extensions view's **...** menu › **Install from VSIX...**, or
   - the editor's own command-line launcher, if it has one, with `--install-extension <file>.vsix`. Cursor installs a `cursor` launcher, Windsurf a `windsurf` launcher and VSCodium a `codium` launcher; the flags are the same as VS Code's.
4. **Reload** the window if asked, then check the Command Palette for the extension's commands.

Keeping extensions up to date in a fork

An editor that cannot reach the Visual Studio Marketplace cannot tell when a new version is published. To update:

1. Check the extension's page on the Marketplace or its release notes (each extension's folder in these docs has release notes).
2. Download the new version's VSIX.
3. Install it over the old one. The new version replaces the old one.

What to expect

- **Themes and icon themes** work in forks that keep VS Code's colour theme and file icon theme support.
- **Webview panels** (for example Formatter Studio, the Font Hub studio or the Auric Live control panel) depend on the fork supporting VS Code webviews. Most do.
- **Minimum version.** If the fork's VS Code base is older than the extension's minimum, installation fails with a compatibility message. Use an older version of the extension, if one exists for your base version, or update the editor.
- **Keyboard shortcuts.** A fork may already use a shortcut that an Auric extension declares, for example `Ctrl` + `K` combinations or `Ctrl` + `Alt` combinations. Check **Preferences: Open Keyboard Shortcuts** for conflicts and rebind as needed.

Support

Auric Artisan describes the extensions as working in forks that preserve the VS Code interfaces they use; each fork is not separately certified. If something behaves differently in a fork than in VS Code, report it with the editor's name and version. See Get support.

When the install succeeds, the extension is listed as installed in the editor's Extensions view and its commands appear in the Command Palette.

VS CODE EXTENSIONS › INSTALL AND SETUP

Get support

Where to send a bug or question about an Auric Artisan VS Code extension, and how to gather the details that make it quick to solve.

Where to ask

- **Contact Auric Artisan:** https://auricartisan.com/company/contact/ — the support address listed on every extension's Marketplace page.

- **Frequently asked questions:** https://auricartisan.com/company/faq/ — linked from the Marketplace pages as the place for questions.

What to include

Collect these before you write. Each item saves a round of questions.

1. **The extension and its version.** Open the Extensions view, select the extension and read the version on its page, or run `code --list-extensions --show-versions` in a terminal.
2. **Your editor and its version.** In VS Code, **Help** › **About** has a **Copy** button. If you use Cursor, Windsurf, VSCodium or another fork, say so.
3. **Your operating system**, and whether you are working locally, over Remote SSH, in WSL, in a container or in a virtual workspace.
4. **What you did, what you expected and what happened.** A short numbered list of steps is best. Say which command, button or setting you used.
5. **The file language** (for example TypeScript or PHP) if the problem involves a file.
6. **The log or built-in summary** described below for your extension.

Before sending, remove private paths, secrets, tokens, customer data and the contents of private files. Reduce the example to the smallest file that still shows the problem.

Logs and built-in summaries

Auric Artisan Live

- **Auric Live: Show Log** opens the **Auric Artisan Live** output channel.
- **Auric Live: Diagnose Project and Runtime** opens a local diagnostic summary with credential fields redacted.
- **Auric Live: Show Dev Server Output** shows a run target's output.
- Set `auricLive.logLevel` to `debug` for more detail.

Auric Artisan File System

- **Auric Files: Show Log** opens the **Auric Artisan File System** output channel.
- Set `auricFiles.logLevel` to `debug` for per-step detail. Paths are logged relative to the workspace and file contents are never written.

Auric Artisan Font Hub

- **Auric Font Hub: Show Logs** opens the log.
- For fonts that do not load in a browser, run **Auric Artisan Font Hub: Check Font CDN and Browser CORS** and include the result.

Auric Artisan Formatter

- **Auric Formatter: Report a Formatting Problem** opens a Markdown document with the formatter version, VS Code version, language, tier and the explanation for the active file. Add what happened and attach a small input file.
- **Auric Formatter: Show the Formatter Log** shows the log.

For other extensions, use the extension's own log command if it has one (search the Command Palette for the extension's prefix and "log"), and see that extension's documentation folder.

Check before you send

- The Live diagnostic summary and log redact common credential fields, but free text can still contain private data. Read it first.
- The Formatter's problem template includes the explanation for your active file, not the file itself. Attach only a sample you are happy to share.
- Screenshots help with layout problems. Crop out anything private.

You should receive a reply through the contact channel you used.

VS CODE EXTENSIONS › INSTALL AND SETUP

Install from the command line

Use the VS Code command-line launcher to install, list, update and remove Auric Artisan extensions, including all seventeen at once.

VS Code ships a command-line launcher called `code`. It can install and remove extensions without opening the Extensions view, which is useful for setting up a new machine, scripting a team's setup, or installing all seventeen Auric Artisan extensions in one go. It downloads from the Visual Studio Marketplace, so it needs a network connection.

Make sure `code` is available

- **Windows and Linux.** The installer normally adds `code` to your `PATH`. Open a new terminal and run `code --version`.
- **macOS.** In VS Code, open the Command Palette (`Cmd` + `Shift` + `P`) and run **Shell Command: Install 'code' command in PATH**, then open a new terminal.
- **VS Code Insiders** uses `code-insiders` instead of `code`.

If `code --version` prints a version number, you are ready.

Install one extension

1. Open a terminal.
2. Run `code --install-extension` followed by the extension ID:

   ```
   code --install-extension auric-artisan.auric-artisan-formatter
   ```

3. Wait for the message that the extension was successfully installed.
4. If VS Code is already open, the extension appears under **Installed** in the Extensions view. Reload the window (**Developer: Reload Window**) if it does not show at once.

Install the two packs

```
code --install-extension auric-artisan.auric-artisan-icon-pack
code --install-extension auric-artisan.auric-artisan
```

The first installs Auric Icons: Modern, Drawn and Playful and Auric Artisan Emoji. The second installs Auric Color, Auric Accessibility and Auric Code Health. See The two extension packs.

Install all seventeen

This is the same grouped list the Auric Artisan VS Code page offers. You can paste the lines into a terminal one group at a time, or all at once.

```
# Themes and file icons
code --install-extension auric-artisan.auric-artisan-theme
code --install-extension auric-artisan.auric-artisan-file-icon

# Icons and emoji. The Icon Pack alone installs Modern, Drawn, Playful and Emoji.
code --install-extension auric-artisan.auric-artisan-icon-pack
code --install-extension auric-artisan.auric-artisan-modern-icon
code --install-extension auric-artisan.auric-icons-drawn
code --install-extension auric-artisan.auric-icons-playful
code --install-extension auric-artisan.auric-artisan-emoji
code --install-extension auric-artisan.auric-artisan-icon-studio
code --install-extension auric-artisan.auric-artisan-icon-lib

# Colour, accessibility and code health. The Full Pack alone installs all three.
code --install-extension auric-artisan.auric-artisan
code --install-extension auric-artisan.auric-color
code --install-extension auric-artisan.auric-a11y
code --install-extension auric-artisan.auric-codehealth

# Workspace tools
code --install-extension auric-artisan.auric-artisan-live
code --install-extension auric-artisan.auric-artisan-file-system
code --install-extension auric-artisan.auric-artisan-font-hub
code --install-extension auric-artisan.auric-artisan-formatter
```

Lines that begin with `#` are comments; most shells ignore them. In Windows Command Prompt, remove the comment lines before pasting.

Installing an extension that a pack has already installed does nothing harmful: `code` reports that it is already installed.

Other useful flags

These are standard VS Code launcher flags. They work the same for every extension.

Command	What it does
`code --install-extension <id>`	Install the latest version of an extension
`code --install-extension <id>@<version>`	Install a specific published version, for example `auric-artisan.auric-artisan-live@0.2.0`
`code --install-extension <id> --force`	Update an installed extension to the latest version without a prompt
`code --install-extension <path-to-file>.vsix`	Install from a downloaded VSIX file
`code --list-extensions`	List the IDs of every installed extension
`code --list-extensions --show-versions`	List installed extensions with their versions
`code --uninstall-extension <id>`	Remove an extension

To see only the Auric Artisan extensions you have installed, filter the list. In PowerShell:

```
code --list-extensions --show-versions | Select-String auric-artisan
```

In a POSIX shell such as bash or zsh:

```
code --list-extensions --show-versions | grep auric-artisan
```

Profiles

VS Code profiles keep separate sets of extensions and settings. The launcher installs into the default profile unless you name one with `--profile "<profile name>"`. If an extension seems missing after a command-line install, check which profile the VS Code window is using (the gear menu at the bottom of the Activity Bar shows it).

The IDs are listed with their minimum VS Code versions in the reference. When a command succeeds you see a line saying the extension was successfully installed, and the extension appears in the Extensions view.

VS CODE EXTENSIONS › INSTALL AND SETUP

Install from a VSIX file

Download an Auric Artisan extension as a VSIX package and install it in VS Code or an editor that cannot reach the Visual Studio Marketplace.

A *VSIX file* is the package format for VS Code extensions: a single file ending in `.vsix` that contains everything the extension needs. Installing from a VSIX is the route to use when your editor cannot reach the Visual Studio Marketplace, for example Cursor, Windsurf or VSCodium, or a machine with restricted network access.

When to use this route

- Your editor is based on VS Code but searches a different gallery (see Cursor, Windsurf, VSCodium and other editors). The Auric Artisan extensions are published only on the Visual Studio Marketplace; they are not on Open VSX.
- You need to install on a computer without direct Marketplace access.
- You want to keep a copy of a specific version.

A VSIX installed this way does not update itself from the Marketplace in an editor that cannot reach it. You repeat these steps for each new version.

Step 1: download the VSIX

Every published version of every Auric Artisan extension can be downloaded from the Marketplace at an address of this form:

```
https://marketplace.visualstudio.com/_apis/public/gallery/publishers/auric-artisan/vsextensions/<extension-name>/<version>/vspackage
```

- `<extension-name>` is the part of the ID after `auric-artisan.`, for example `auric-artisan-live`.
- `<version>` is the version number, for example `0.2.1`. The versions available at the time of writing are in the reference. The **Version History** tab of the extension's Marketplace page lists every version.

For example, to download Auric Artisan Live 0.2.1:

```
https://marketplace.visualstudio.com/_apis/public/gallery/publishers/auric-artisan/vsextensions/auric-artisan-live/0.2.1/vspackage
```

Open the address in a browser, or download it from a terminal. Save the file with a `.vsix` extension, for example `auric-artisan-live-0.2.1.vsix`. Some browsers save it without an extension; rename it if so.

> **Note:** An extension pack only lists other extensions. Installing a pack from a VSIX in an editor that cannot reach the Marketplace does not fetch its members. Download and install each member's VSIX instead. The members are listed in The two extension packs.

Step 2: install the file

Use any one of these three methods.

From the Command Palette

1. Open the Command Palette with `Ctrl` + `Shift` + `P`.
2. Run **Extensions: Install from VSIX...**.
3. Choose the `.vsix` file and select **Install**.

From the Extensions view

1. Open the Extensions view with `Ctrl` + `Shift` + `X`.
2. Select the **...** (Views and More Actions) menu at the top of the view.
3. Choose **Install from VSIX...** and pick the file.

From a terminal

```
code --install-extension auric-artisan-live-0.2.1.vsix
```

Use the launcher for your editor if it is not VS Code. See Cursor, Windsurf, VSCodium and other editors.

Step 3: confirm

1. Open the Extensions view and select **Installed**.
2. Find the extension and check the version on its page.
3. Open the Command Palette and type the extension's command prefix (listed in Getting started).

Things to know

- **Minimum editor version.** A VSIX still declares the lowest VS Code version it supports. If your editor reports an older VS Code version, installation fails with a message that the extension is not compatible. See Troubleshooting.
- **Replacing a version.** Installing a VSIX of a different version replaces the installed one. You may be asked to reload the window.
- **Only download from the Marketplace or Auric Artisan.** A VSIX runs code on your machine. Get it from the Marketplace address above, or from Auric Artisan support.

When this works, the extension is listed under **Installed** with the version you downloaded, and its commands appear in the Command Palette.

VS CODE EXTENSIONS › INSTALL AND SETUP

Settings, sync and workspace trust

Where Auric Artisan extension settings live, how to share them with a team, what Settings Sync carries between machines, and what each extension does in an untrusted folder.

Every Auric Artisan extension is configured through normal VS Code settings, and three of them also read a project file you can commit to a repository. This page explains where each kind of setting lives and how they combine.

User settings and workspace settings

VS Code keeps settings at several levels. A value at a more specific level wins.

Level	Where it is stored	Applies to
Default	Inside the extension	Everyone, until changed
User	Your personal `settings.json`	Every window you open, in the current profile
Workspace	`.vscode/settings.json` in the open folder, or the `.code-workspace` file	Only that folder or workspace
Folder	`.vscode/settings.json` in one folder of a multi-root workspace	Only that folder

To edit them:

1. Open **File** › **Preferences** › **Settings** (`Ctrl` + `,`).
2. Choose the **User** or **Workspace** tab at the top.
3. Search for the extension's settings prefix, for example `auricLive`, or type `@ext:auric-artisan.auric-artisan-live` to show only that extension's settings.
4. To edit the JSON directly, run **Preferences: Open User Settings (JSON)** or **Preferences: Open Workspace Settings (JSON)**.

Setting scope

Each setting declares a *scope*, which limits the levels where it can be set:

- **application** — user settings only. It applies to every window. Examples: all Auric Artisan Theme settings and the Emoji `auricEmoji.interfaceLanguage` setting.
- **window** — user or workspace settings. Most File System, Font Hub, icon-library and Studio settings use this scope, as do a few Live settings such as `auricLive.interfaceLanguage`.
- **resource** — user, workspace or folder settings, so each folder in a multi-root workspace can differ. Almost all Live settings and every Formatter setting use this scope.

The Settings editor only shows a setting under the tabs where it is allowed.

Project configuration files

Three extensions also read a file in the project, so a repository can carry its own setup. A project file wins over VS Code settings for that project.

Extension	Project files	Details
Auric Artisan Live	`auric.live.json` (name set by `auricLive.configFile`), `.auricignore-live`	Keys are the setting names without the `auricLive.` prefix; comments and trailing commas are allowed
Auric Artisan Formatter	`.auricformatrc` (several forms), `auric.format.json`, an `auricFormatter` key in `package.json`, `.auricformatignore`; also reads Prettier files and `.editorconfig`	See the Formatter's configuration guide
Auric Artisan File System	Template folders, `.auric/templates` and `.vscode/templates` by default	Relationship rules and generator profiles live in settings

Sharing a setup with a team

1. Commit `.vscode/settings.json` with the workspace-level values you agree on.
2. Commit the project files above where they apply.
3. Recommend the extensions: create `.vscode/extensions.json` with their IDs. VS Code then offers to install them when someone opens the folder.

```
{
  "recommendations": [
    "auric-artisan.auric-artisan-formatter",
    "auric-artisan.auric-artisan-live"
  ]
}
```

Settings with **application** scope cannot be shared this way; each person sets them in their own user settings.

Settings Sync

VS Code Settings Sync copies your user-level setup between machines through your Microsoft or GitHub account. It is part of VS Code, not of the Auric extensions.

1. Select the Accounts icon in the Activity Bar and choose **Backup and Sync Settings...** (older versions say **Turn on Settings Sync...**).
2. Tick what to sync. **Extensions** and **Settings** matter here.
3. Sign in.

What happens to Auric Artisan extensions:

- **Installed extensions** sync, including whether each one is enabled or disabled.
- **User settings** sync, including every `auricLive.*`, `auricFormatter.*` and other Auric user-level value.
- **Workspace settings and project files do not sync.** They travel with the repository instead.
- **Stored extension data does not sync.** Favourites, recents, selections, likes, tags, groups, operation history and similar data that an extension keeps in VS Code's extension storage stay on the machine where you created them. The Auric extensions do not register any of this data for Settings Sync.

To stop one Auric setting from syncing, add its ID to the VS Code setting `settingsSync.ignoredSettings`. To stop one extension from syncing, add its ID to `settingsSync.ignoredExtensions`.

Workspace trust

When you open a folder for the first time, VS Code may ask whether you trust its authors. In an untrusted folder, VS Code runs in *Restricted Mode*, and extensions that run code or read project configuration can limit themselves. You can change your answer later with **Workspaces: Manage Workspace Trust**.

- **Auric Artisan Live** — static serving, live reload and request inspection still work, on this machine only. Running commands, CGI (PHP), proxying, file-backed mocks, custom certificates and listening on your network need trust, even when a project file turns them on.
- **Auric Artisan File System** — browsing, search, filtering and related-file navigation still work. Creating, renaming, moving, duplicating and deleting files are disabled until you trust the folder.
- **Auric Artisan Formatter** — configuration files are read, but JavaScript configuration files and formatter plugins are not run.
- **Auric Artisan Font Hub** — fully supported. It only writes files after an explicit save or editor command.
- **Auric Artisan Theme, File Icons, Emoji, Icon Studio and Icon Hub** — fully supported.

For the other extensions, the **Manage Workspace Trust** page lists which installed extensions are limited or disabled in Restricted Mode.

Remote and virtual workspaces

- Auric Artisan Live needs a real file system and a local port. It does not support virtual workspaces (folders opened from a remote repository without a local copy).
- Auric Artisan File System works in virtual workspaces with limits: revealing in the operating system's file manager, content hashing for duplicate detection and the native trash are not available there.
- Auric Artisan Formatter supports virtual workspaces.

After configuring, your values appear in the Settings editor under the tab you chose, and the extension uses them the next time it acts.

VS CODE EXTENSIONS › INSTALL AND SETUP

Update, disable and uninstall

Keep Auric Artisan extensions up to date, install a specific version, switch an extension off, or remove it.

These controls are part of VS Code itself and work the same way for every Auric Artisan extension.

Updates

VS Code checks the Marketplace for new versions and, by default, updates installed extensions automatically.

- **See pending updates.** In the Extensions view, an extension with an update available shows an **Update** button. The view's **…** menu has **Check for Extension Updates**.
- **Control automatic updates.** The VS Code setting `extensions.autoUpdate` decides whether updates install on their own. You can also switch auto-update on or off for one extension: right-click it in the Extensions view and choose **Auto Update** (the exact wording depends on your VS Code version).
- **Read what changed.** The **Changelog** tab on an extension's page shows its release notes. Each extension's folder in these docs has a user-facing release-notes page.

Install a specific version

Use this to stay on a version you know, or to go back after an update.

1. In the Extensions view, right-click the extension (or use the gear on its page).
2. Choose **Install Specific Version...**.
3. Pick the version from the list.
4. Reload the window if asked.

From a terminal:

```
code --install-extension auric-artisan.auric-artisan-live@0.2.0
```

After installing an older version, turn off automatic updates for that extension, or VS Code will update it again.

Disable

Disabling keeps the extension installed but stops it running. Its settings are kept.

1. In the Extensions view, select the extension.
2. Select **Disable**, or open the drop-down next to it and choose **Disable (Workspace)** to switch it off only for the folder you have open.

A disabled extension contributes no commands, views, themes or icon themes until you select **Enable** again.

Two situations where disabling matters:

- **Moving from the old Font Library to Font Hub.** Font Hub replaces an earlier extension called Font Library, which has a different ID. Disable or uninstall the old one before enabling Font Hub, otherwise you see duplicate commands and views.
- **Keeping part of a pack.** Disable any pack member you do not use. See The two extension packs.

Uninstall

1. In the Extensions view, select the extension.
2. Select **Uninstall**.
3. Reload the window if VS Code asks.

From a terminal:

```
code --uninstall-extension auric-artisan.auric-artisan-live
```

What uninstalling leaves behind

Uninstalling removes the extension's code. Some things it created stay where they are, because they belong to you or to VS Code:

- **Your settings.** Values you set, such as `auricLive.port`, stay in your `settings.json` files. VS Code shows them as unknown settings; you can delete them.
- **Files in your project.** Files you created with an extension stay. For example: `auric.live.json`, `.auricignore-live` and the `.auric-live/` session folder from Auric Artisan Live; `.auric/templates/` and the `.auric/trash/` workspace trash

from Auric Artisan File System; `.auricformatrc` and `.auricformatignore` from Auric Artisan Formatter. Before uninstalling File System, restore anything you still need from its workspace trash.

- **Workspace settings an extension wrote for you.** For example, **Auric Formatter: Toggle Format on Save** writes `editor.defaultFormatter` and `editor.formatOnSave`, and File System's sort and exclusion commands write `explorer.sortOrder` and `files.exclude`. Review your workspace settings after removing those extensions.
- **Extension storage.** Favourites, recents and similar per-extension data live in VS Code's extension storage, which VS Code manages.

When you have finished, the extension no longer appears under **Installed**, and its commands are gone from the Command Palette.

VS CODE EXTENSIONS › INSTALL AND SETUP

Reference: extension IDs and requirements

Every Auric Artisan VS Code extension with its exact ID, minimum VS Code version, published version and Marketplace address, plus the install commands and flags.

All seventeen extensions

Every extension is published by `auric-artisan` on the Visual Studio Marketplace and is free. "Minimum VS Code" is the lowest VS Code version the published package accepts. "Version" is the latest published version when this page was written (24 September 2026).

Extension	Extension ID	Minimum VS Code	Version
Auric Artisan Studio — Full Pack	`auric-artisan.auric-artisan`	1.74	0.1.0
Auric Color — Picker, Palettes & Contrast	`auric-artisan.auric-color`	1.74	0.1.0
Auric Accessibility — WCAG 2.2 Linter	`auric-artisan.auric-a11y`	1.74	0.1.0
Auric Code Health — Secrets, TODOs & Risk	`auric-artisan.auric-codehealth`	1.74	0.1.0
Auric Artisan Theme	`auric-artisan.auric-artisan-theme`	1.75	0.2.0
Auric Artisan File Icons	`auric-artisan.auric-artisan-file-icon`	1.85	0.1.1
Auric Artisan Icon Pack	`auric-artisan.auric-artisan-icon-pack`	1.85	0.1.0
Auric Icons: Modern	`auric-artisan.auric-artisan-modern-icon`	1.85	0.12.1
Auric Icons: Drawn	`auric-artisan.auric-icons-drawn`	1.85	0.12.1
Auric Icons: Playful	`auric-artisan.auric-icons-playful`	1.85	0.12.1
Auric Artisan Emoji	`auric-artisan.auric-artisan-emoji`	1.85	0.2.0
Auric Artisan Icon Studio	`auric-artisan.auric-artisan-icon-studio`	1.85	0.1.0
Auric Artisan Icon Hub	`auric-artisan.auric-artisan-icon-lib`	1.85	0.1.0
Auric Artisan Live — Live Server & Preview	`auric-artisan.auric-artisan-live`	1.95	0.2.1
Auric Artisan File System	`auric-artisan.auric-artisan-file-system`	1.95	0.1.0
Auric Artisan Font Hub	`auric-artisan.auric-artisan-font-hub`	1.85	0.1.0
Auric Artisan Formatter	`auric-artisan.auric-artisan-formatter`	1.74	0.1.0

IDs that are easy to get wrong

- The Studio Full Pack is the bare `auric-artisan.auric-artisan`.
- Icon Hub's ID is `auric-artisan.auric-artisan-icon-lib`, not `…-icon-hub`.
- Auric Icons: Modern is `auric-artisan.auric-artisan-modern-icon`, while Drawn and Playful are `auric-artisan.auric-icons-drawn` and `auric-artisan.auric-icons-playful`.
- File Icons is `auric-artisan.auric-artisan-file-icon` (singular).
- Font Hub is `auric-artisan.auric-artisan-font-hub`. The older Font Library extension had a different ID.

What each one is

Extension	In one line
Studio — Full Pack	Installs Auric Color, Auric Accessibility and Auric Code Health together
Auric Color	Inline colour picker, swatches and hover readouts, WCAG and APCA contrast, palettes and OKLCH scales
Auric Accessibility	Accessibility linting against WCAG 2.0, 2.1 and 2.2 for HTML, JSX, Vue, Svelte, Astro, PHP and CSS
Auric Code Health	Scans for hardcoded secrets, TODO and FIXME markers, debug leftovers, risky APIs and oversized files
Theme	Colour themes with Theme Studio for customising them
File Icons	Facet, Playful and Modern file and folder icon themes for the Explorer
Icon Pack	Installs Auric Icons: Modern, Drawn and Playful and Auric Artisan Emoji together
Auric Icons: Modern	A file and folder icon theme plus an SVG icon library
Auric Icons: Drawn	A hand-drawn style SVG icon library
Auric Icons: Playful	An emoji-style SVG icon library in colour, outline, filled and small variants
Emoji	Emoji Studio for finding and inserting emoji, with English and Hindi
Icon Studio	One offline studio for every Auric icon collection, emoji and seven open-source icon sets
Icon Hub	Search open-source icons across seven collections, customise SVG and export sprites or ZIPs
Live	Local development server with live reload, preview, HTTPS, API proxy and mocks, and request inspection
File System	A file workspace in the Explorer: batch rename, templates, related files, tags, Properties and workspace trash
Font Hub	Discover and compare fonts, add them from the Auric font CDN, and export self-hosting kits
Formatter	Code formatter with native diff previews, Formatter Studio, workspace scans and a command-line check

---

Pack membership

Pack	Members
`auric-artisan.auric-artisan-icon-pack`	`auric-artisan.auric-artisan-modern-icon`, `auric-artisan.auric-icons-playful`, `auric-artisan.auric-artisan-emoji`, `auric-artisan.auric-icons-drawn`
`auric-artisan.auric-artisan`	`auric-artisan.auric-color`, `auric-artisan.auric-a11y`, `auric-artisan.auric-codehealth`

Addresses

What	Address pattern
Marketplace page	`https://marketplace.visualstudio.com/items?itemName=<extension ID>`
VSIX download	`https://marketplace.visualstudio.com/_apis/public/gallery/publishers/auric-artisan/vsextensions/<name>/<version>/vspackage`
Auric Artisan VS Code page	https://auricartisan.com/vscode/
Support	https://auricartisan.com/company/contact/

In the VSIX address, `<name>` is the part of the ID after `auric-artisan.`.

Command-line flags

Command	What it does
`code --install-extension <id>`	Install the latest version
`code --install-extension <id>@<version>`	Install a specific version
`code --install-extension <id> --force`	Update to the latest version without a prompt
`code --install-extension <file>.vsix`	Install from a VSIX file
`code --uninstall-extension <id>`	Remove an extension
`code --list-extensions --show-versions`	List installed extensions with versions
`code --profile "<name>" --install-extension <id>`	Install into a named VS Code profile

VS Code commands used in these guides

Command	What it does
Extensions: Install from VSIX...	Install an extension from a `.vsix` file
Extensions: Check for Extension Updates	Look for newer versions now
Developer: Reload Window	Reload the window after installing or updating
Configure Display Language	Change VS Code's own display language
Workspaces: Manage Workspace Trust	Trust or restrict the open folder
Preferences: Open User Settings (JSON)	Edit user settings as JSON
Preferences: Open Workspace Settings (JSON)	Edit workspace settings as JSON
Preferences: Open Keyboard Shortcuts	Review and change shortcuts
Shell Command: Install 'code' command in PATH	macOS: make the `code` launcher available in terminals

Settings that control installation

These are VS Code settings, not Auric settings.

Setting	What it does
`extensions.autoUpdate`	Whether VS Code installs extension updates automatically
`extensions.autoCheckUpdates`	Whether VS Code checks for updates automatically
`settingsSync.ignoredExtensions`	Extension IDs that Settings Sync should not install or remove on other machines
`settingsSync.ignoredSettings`	Setting IDs that Settings Sync should not copy

VS CODE EXTENSIONS › INSTALL AND SETUP

Troubleshooting

Fix common problems when finding, installing, updating, configuring or removing an Auric Artisan VS Code extension.

Each entry gives the symptom, the likely cause and what to do.

Finding and installing

The search finds nothing

- **Cause:** a filter such as `@installed`, `@disabled` or `@recommended` is still in the search box, or the ID is misspelt.
- **Fix:** clear the search box and type `@id:` followed by the exact ID from the reference. Watch the IDs that are easy to get wrong: the Full Pack is `auric-artisan.auric-artisan`, Icon Hub is `auric-artisan.auric-artisan-icon-lib`, File Icons is `auric-artisan.auric-artisan-file-icon`.

The search finds nothing in Cursor, Windsurf or VSCodium

- **Cause:** the editor searches Open VSX, and the Auric Artisan extensions are only on the Visual Studio Marketplace.
- **Fix:** install from a VSIX file. See Cursor, Windsurf, VSCodium and other editors.

"Not compatible" or "requires a newer version of VS Code"

- **Cause:** your editor is older than the extension's minimum version. Live and File System need 1.95; Font Hub, the icon libraries, File Icons, Emoji, Icon Studio, Icon Hub and the Icon Pack need 1.85; Theme needs 1.75; Color, Accessibility, Code Health, the Full Pack and Formatter need 1.74.
- **Fix:** update VS Code (**Help** › **Check for Updates** on Windows and Linux, **Code** › **Check for Updates** on macOS). In a fork, update the fork.

`code` is not recognised in the terminal

- **Cause:** the launcher is not on your `PATH`, or the terminal was opened before VS Code was installed.
- **Fix:** open a new terminal. On macOS, run **Shell Command: Install 'code' command in PATH** from the Command Palette. On Windows, reinstall VS Code with the option to add it to `PATH`, or use the full path to `code.cmd` in the VS Code `bin` folder.

The install fails with a network error

- **Cause:** no connection to the Marketplace, or a proxy or firewall blocks it.
- **Fix:** check VS Code's `http.proxy` setting and your network. If the machine cannot reach the Marketplace, download the VSIX on another machine and install from the file.

A downloaded file will not install

- **Cause:** the file was saved without the `.vsix` extension, or the download was incomplete.
- **Fix:** rename the file to end in `.vsix` and try again. If it still fails, download it again.

Installed with the command line, but the extension is missing

- **Cause:** the window uses a different VS Code profile, or the window has not refreshed.
- **Fix:** run **Developer: Reload Window**. Check the active profile and install with `--profile "<name>"` if needed.

Packs

A pack installed but its members did not

- **Cause:** the pack was installed from a VSIX in an editor that cannot reach the Marketplace, so the members could not be fetched.
- **Fix:** install each member's VSIX. Members are listed in The two extension packs.

An extension I uninstalled came back

- **Cause:** it is a member of a pack that was reinstalled or updated.
- **Fix:** disable the member instead of uninstalling it, or uninstall the pack and install only the members you want.

After installing

No commands appear in the Command Palette

- **Cause:** the extension is disabled, the window needs a reload, or the command only appears in certain situations (for example with a folder open or an editor focused).
- **Fix:** check the extension is enabled in the Extensions view, reload the window, open a folder, and type the command prefix from Getting started.

Duplicate Font Hub commands and views

- **Cause:** the older Font Library extension is still installed alongside Font Hub.
- **Fix:** disable or uninstall Font Library, then reload. Existing `auricFontLib.*` settings keep working in Font Hub.

Some features are missing or greyed out in a folder

- **Cause:** VS Code is in Restricted Mode because the folder is not trusted.
- **Fix:** run **Workspaces: Manage Workspace Trust** and trust the folder if you trust its authors. See Settings, sync and workspace trust for what each extension limits.

A keyboard shortcut does nothing

- **Cause:** another extension, your editor or your operating system uses the same keys, or the shortcut is off by default (File System's full shortcut deck needs `auricFiles.keyboard.enableDefaultShortcuts`).
- **Fix:** open **Preferences: Open Keyboard Shortcuts**, search for the command and check for conflicts. Assign different keys if needed.

Settings

A setting I changed has no effect

- **Cause:** a more specific level overrides it. A workspace setting beats a user setting, and a project file such as `auric.live.json` or `.auricformatrc` beats both for that project.
- **Fix:** check the **Workspace** tab and the project files. The Settings editor marks a setting that is modified elsewhere.

A setting will not save in workspace settings

- **Cause:** it has **application** scope (for example Theme settings or `auricEmoji.interfaceLanguage`), so it can only be set in user settings.
- **Fix:** set it on the **User** tab.

Favourites or history did not follow me to another machine

- **Cause:** Settings Sync copies settings and the list of extensions, not the data an extension keeps in its own storage.
- **Fix:** none needed; this data is kept per machine by design. See Settings, sync and workspace trust.

The panel is still in English after choosing Hindi

- **Cause:** only Live, Theme and Emoji include Hindi, and some content (technical details, the Live browser dashboard) stays in English. An open Live preview tab keeps the language it opened with.
- **Fix:** see Use an extension in Hindi. Reopen the Live preview tab.

If none of these helps, see Get support.

VS CODE EXTENSIONS › INSTALL AND SETUP

FAQ

Short answers to common questions about installing, updating and configuring the Auric Artisan VS Code extensions.

Do I have to install all seventeen extensions?

No. Each extension works on its own. Two packs install a set in one step: the Icon Pack (Modern, Drawn, Playful and Emoji) and the Studio Full Pack (Color, Accessibility and Code Health).

Are the extensions free?

Yes. Every Auric Artisan extension is listed as free on the Marketplace, and none needs an Auric Artisan account. Each has its own licence; read it on the extension's Marketplace page before redistributing anything.

Which VS Code version do I need?

It depends on the extension, from 1.74 up to 1.95. The reference lists each one. If you keep VS Code up to date, all seventeen work.

Do they work in Cursor, Windsurf or VSCodium?

Yes, in forks that keep VS Code's extension, colour theme and icon theme interfaces. Those editors usually search Open VSX, where the Auric extensions are not published, so you install from a VSIX file. See Cursor, Windsurf, VSCodium and other editors.

How do I install the same set on every machine?

Turn on VS Code Settings Sync, or run the `code --install-extension` lines from Install from the command line on each machine. For a project, list the IDs in `.vscode/extensions.json` so VS Code recommends them to everyone.

Do the extensions update themselves?

In VS Code, yes, unless you turn automatic updates off. In an editor that cannot reach the Marketplace, you install each new VSIX yourself.

How do I go back to an earlier version?

In the Extensions view, right-click the extension and choose **Install Specific Version...**, or run `code --install-extension <id>@<version>`. Then turn off automatic updates for that extension.

Can I share my settings with my team?

Yes, for settings with window or resource scope: commit them in `.vscode/settings.json`. Live, Formatter and File System also read project files you can commit. Settings with application scope stay personal. See Settings, sync and workspace trust.

Can I use the extensions in Hindi?

Auric Artisan Live, Theme and Emoji include Hindi. See Use an extension in Hindi.

Do the extensions send data anywhere?

Most work entirely on your machine. Font Hub and Icon Hub use the network to fetch their catalogues and assets, and Live runs a local server that other devices on your network can reach if you allow it. See Privacy.

What happens to my files if I uninstall?

Files you created stay. Uninstalling removes the extension's code, not your project files, your settings or any workspace trash. See Update, disable and uninstall.

Where do I report a bug?

Use https://auricartisan.com/company/contact/ and include the details listed in Get support.

VS CODE EXTENSIONS › INSTALL AND SETUP

Supporting material

Contents of the supporting pages for installing and setting up the Auric Artisan VS Code extensions.

These pages support the installation guides: the terms they use, the keys and commands involved, what the extensions need from your editor, what uses the network, and where to go next for each extension.

File	What it covers
glossary.md	Terms used in the installation and setup guides
shortcuts.md	Keys and VS Code commands for managing extensions
limits-and-accuracy.md	Editor versions, forks, remote and virtual workspaces, and other boundaries
privacy.md	Which extensions use the network and what stays on your machine
related.md	Links to each extension's own documentation and to Auric Artisan pages

VS CODE EXTENSIONS › INSTALL AND SETUP

Shortcuts and commands

The VS Code keys and commands you use to install, configure and manage the Auric Artisan extensions.

These are VS Code's own default keys on Windows and Linux. On macOS, use `Cmd` in place of `Ctrl`. Each extension's own shortcuts are listed in that extension's folder.

Keys

Keys	What they do
`Ctrl` + `Shift` + `X`	Open the Extensions view
`Ctrl` + `Shift` + `P`	Open the Command Palette
`Ctrl` + `,`	Open the Settings editor

Commands

Command	Use it to
Extensions: Install from VSIX...	Install from a downloaded `.vsix` file
Extensions: Check for Extension Updates	Check for new versions now
Developer: Reload Window	Reload after installing, updating or changing a language
Configure Display Language	Change VS Code's own display language
Workspaces: Manage Workspace Trust	Trust or restrict the open folder
Preferences: Open User Settings (JSON)	Edit user settings directly
Preferences: Open Workspace Settings (JSON)	Edit workspace settings directly
Preferences: Open Keyboard Shortcuts	Find and change shortcuts, and spot conflicts

Extension shortcuts at a glance

These are the default shortcuts the four workspace-tool extensions declare. The full lists are in each extension's folder.

Extension	Keys (Windows and Linux)	Command
Auric Artisan Live	`Ctrl` + `Alt` + `L`	**Auric Live: Go Live**
Auric Artisan Live	`Ctrl` + `Alt` + `K`	**Auric Live: Open Preview Beside**
Auric Artisan File System	`Alt` + `Enter` (Explorer focused)	**Auric Files: File Properties**
Auric Artisan Font Hub	`Ctrl` + `Alt` + `F`	**Auric Artisan Font Hub: Open Font Hub**
Auric Artisan Font Hub	`Ctrl` + `Alt` + `T`	**Auric Artisan Font Hub: Quick Pick Font**
Auric Artisan Formatter	`Ctrl` + `Alt` + `Shift` + `F` (editor focused)	**Auric Formatter: Preview Formatting as a Diff**
Auric Artisan Formatter	`Ctrl` + `Alt` + `Shift` + `P`	**Auric Formatter: Open Formatter Studio**

These four extensions document `Cmd` in place of `Ctrl` on macOS for the combinations above.

VS CODE EXTENSIONS › INSTALL AND SETUP

Glossary

Plain-language definitions of the terms used in the installation and setup guides.

Activity Bar — the vertical strip of icons at the side of the VS Code window. Some extensions add their own icon there.

Application scope — a setting that can only be set in user settings and applies to every window.

Command Palette — the searchable list of every command, opened with `Ctrl` + `Shift` + `P`.

Extension ID — the unique name of an extension, in the form `publisher.name`, for example `auric-artisan.auric-artisan-live`.

Extension pack — an extension with no features of its own that installs a list of other extensions.

Extensions view — the VS Code panel for searching, installing and managing extensions, opened with `Ctrl` + `Shift` + `X`.

Fork — an editor built from the VS Code source code, such as Cursor, Windsurf or VSCodium.

Launcher — the `code` command (or a fork's equivalent) that opens VS Code and manages extensions from a terminal.

Marketplace — the Visual Studio Marketplace, the gallery where the Auric Artisan extensions are published.

Open VSX — a separate, open extension gallery used by many forks. The Auric Artisan extensions are not published there.

Profile — a named VS Code setup with its own extensions and settings.

Project file — a configuration file kept in the repository, such as `auric.live.json` or `.auricformatrc`, that an extension reads for that project.

Resource scope — a setting that can be set for the user, the workspace, or a single folder of a multi-root workspace.

Restricted Mode — the state VS Code runs in when you have not trusted a folder. Extensions can limit themselves in it.

Settings Sync — the VS Code feature that copies settings and the list of installed extensions between machines through your account.

Virtual workspace — a folder opened from a remote source without a local copy on disk.

VSIX file — the package file for a VS Code extension, ending in `.vsix`.

Window scope — a setting that can be set in user or workspace settings.

Workspace settings — settings stored in the open folder's `.vscode/settings.json` or in a `.code-workspace` file, applying only to that workspace.

Workspace trust — your decision, per folder, about whether VS Code and extensions may run code and read configuration from it.

VS CODE EXTENSIONS › INSTALL AND SETUP

Compatibility and limits

What the Auric Artisan extensions need from your editor and environment, and the boundaries to know before you install.

Editor versions

- The extensions need VS Code 1.74 or newer; some need 1.85 or 1.95. The reference gives each minimum.
- A newer VS Code is always fine. The minimum is a floor, not a range.

Editors other than VS Code

- The extensions are published only on the Visual Studio Marketplace. They are not on Open VSX.
- They work in forks that preserve VS Code's extension, colour theme and icon theme interfaces. Each fork is not separately certified.
- In a fork that cannot reach the Marketplace, updates and pack members must be installed by hand from VSIX files.
- The extensions are for desktop VS Code and compatible editors. They are not browser extensions.

Where extensions run

VS Code can run an extension on your computer (the "UI" side) or next to the files (the "workspace" side, for example on a remote server over SSH or in a container).

- Font Hub, Icon Hub, Icon Studio, File Icons and Emoji can run on either side.
- Live, File System and Formatter run next to the workspace files.

Remote and virtual workspaces

- **Auric Artisan Live** needs a real file system and a local port. It does not support virtual workspaces.
- **Auric Artisan File System** supports virtual workspaces with limits: no revealing in the operating system's file manager, no content hashing for duplicate detection and no native trash there. Remote file providers expose different capabilities.
- **Auric Artisan Formatter**, **Font Hub**, **Theme**, **File Icons**, **Emoji**, **Icon Studio** and **Icon Hub** declare support for virtual workspaces.

Workspace trust

Live, File System and Formatter limit themselves in an untrusted folder. See Settings, sync and workspace trust.

Languages

- Auric Artisan Live, Theme and Emoji include Hindi. The other extensions use English.
- Even in Hindi, some technical text stays in English, and served web pages are never translated.

Settings Sync

- Settings Sync carries user settings and the list of installed extensions.
- It does not carry workspace settings, project files or data an extension keeps in its own storage (favourites, recents, tags, history and similar).

Versions in these pages

Version numbers in the reference were correct when written (24 September 2026). The Marketplace page and the **Version History** tab always show the current state.

VS CODE EXTENSIONS › INSTALL AND SETUP

Privacy

A summary of which Auric Artisan extensions use the network, what they send, and where their data is stored.

Installing

Installing, updating and searching for extensions happens between VS Code and the Visual Studio Marketplace. That is VS Code's own traffic, covered by Microsoft's terms, and it is the same for every extension.

Once installed

The Auric Artisan VS Code page states that none of the extensions contains telemetry. Most work entirely on your machine. Three use the network as part of their job:

Auric Artisan Live

- Runs a development server on your machine. With the default host `0.0.0.0`, other devices on your local network can reach it, subject to your firewall.
- Proxy routes you configure forward requests to the targets you name.
- The phone QR code is generated locally; no address goes to a QR service.

Auric Artisan Font Hub

- Refreshes its catalogue, fetches preview fonts and downloads self-hosting files from `fonts.auricartisan.com`. Switch this off with `auricFontLib.network.enabled`.
- Search queries, source code, favourites and usage history are not uploaded. The CDN still receives ordinary request information, such as your IP address.

Auric Artisan Icon Hub

- Checks `icons.auricartisan.com` for catalogue updates, and can be switched off in its settings.

- Your code, searches and favourites are not sent.

Everything else ships inside the extension. Opening a website link from a panel (for example **auricartisan.com** ↗) opens your browser and needs a network connection.

Where data is kept

- **Settings** live in your user or workspace `settings.json`, like any VS Code setting.
- **Favourites, recents, likes, tags, groups, history and similar** are kept in VS Code's extension storage on your machine. They are not copied by Settings Sync.
- **Project files** you create (such as `auric.live.json`, `.auricformatrc` or File System's `.auric/trash`) stay in your project and travel wherever you copy or commit it.

The privacy page in each extension's own folder describes that extension in detail.

VS CODE EXTENSIONS › INSTALL AND SETUP

Related extensions and pages

Where to find the documentation for each of the seventeen Auric Artisan extensions, and related Auric Artisan pages.

Documentation for each extension

Extension	Documentation
Auric Artisan Studio — Full Pack, Auric Color, Auric Accessibility, Auric Code Health	Auric Artisan Studio
Auric Artisan Theme	Auric Artisan Theme
Auric Artisan File Icons	Auric Artisan File Icons
Auric Artisan Icon Pack	Auric Artisan Icon Pack
Auric Icons: Modern	Auric Icons: Modern
Auric Icons: Drawn	Auric Icons: Drawn
Auric Icons: Playful	Auric Icons: Playful
Auric Artisan Emoji	Auric Artisan Emoji
Auric Artisan Icon Studio	Auric Artisan Icon Studio
Auric Artisan Icon Hub	Auric Artisan Icon Hub
Auric Artisan Live	Auric Artisan Live
Auric Artisan File System	Auric Artisan File System
Auric Artisan Font Hub	Auric Artisan Font Hub
Auric Artisan Formatter	Auric Artisan Formatter

Auric Artisan pages

- The VS Code extensions page: https://auricartisan.com/vscode/
- Contact and support: https://auricartisan.com/company/contact/

- Frequently asked questions: https://auricartisan.com/company/faq/
- The Auric Artisan browser extension, a separate product: browser extension documentation

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Auric Artisan Studio

What Auric Artisan Studio is, which of its four extensions to install, and how to get your first result in a few minutes.

Auric Artisan Studio is a family of Visual Studio Code extensions that check and improve the colour, accessibility and code hygiene of your project while you write it. It is published as four Marketplace extensions: three focused extensions that each do one job, and a Full Pack that installs all three at once.

- **Auric Color — Picker, Palettes & Contrast** reads every colour literal in your code, shows its values and contrast in a hover, lets you edit it with a picker, flags text that is too faint to read, and generates accessible palettes and shade scales.
- **Auric Accessibility — WCAG 2.2 Linter** checks your markup and CSS against 70+ static rules mapped to the Web Content Accessibility Guidelines (WCAG) 2.0, 2.1 and 2.2, grades the whole project, and lets you inspect any element's name, role and colours on hover.
- **Auric Code Health — Secrets, TODOs & Risk** looks in every file, in more than 40 languages, for hardcoded secrets, security footguns, risky APIs, debug leftovers, task markers, merge markers and oversized code, and writes machine-readable records for continuous integration (CI).
- **Auric Artisan Studio — Full Pack** installs the three extensions above in one step.

Each extension works on its own. When you install two or three of them, they cooperate as one **Workstation**: one sidebar, one Command Center and one Studio tab that shows every capability you installed, instead of three separate dashboards. Everything runs on your machine. The extensions make no network requests, need no account and have no paid tier.

Who it is for

- Front-end and full-stack developers who want contrast and accessibility problems caught in the editor, before review.
- Designers and design-system maintainers who manage colour tokens, palettes and brand colours in code.
- Accessibility specialists who want a searchable rulebook, manual test procedures and a prioritised worklist for a codebase.
- Teams that want a local, repeatable gate for pull requests, with SARIF output for code scanning.

Where to find it

Extension	Marketplace ID	Install command
Auric Artisan Studio — Full Pack	`auric-artisan.auric-artisan`	`code --install-extension auric-artisan.auric-artisan`
Auric Color — Picker, Palettes & Contrast	`auric-artisan.auric-color`	`code --install-extension auric-artisan.auric-color`
Auric Accessibility — WCAG 2.2 Linter	`auric-artisan.auric-a11y`	`code --install-extension auric-artisan.auric-a11y`
Auric Code Health — Secrets, TODOs & Risk	`auric-artisan.auric-codehealth`	`code --install-extension auric-artisan.auric-codehealth`

Marketplace pages:

- https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan
- https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-color
- https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-a11y
- https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-codehealth

All four need VS Code 1.74 or newer. The current version is 0.1.0. They work in single-folder and multi-root workspaces. Publisher website: https://auricartisan.com.

Which extension should I install?

If you mainly want to…	Install
Edit colours, check contrast as you type, build palettes and shade scales	Auric Color
Lint HTML, JSX, Vue, Svelte, Astro, PHP and CSS for WCAG problems and grade the project	Auric Accessibility
Find secrets, security footguns and debug leftovers in any language, and gate CI	Auric Code Health
All of the above in one Workstation	The Full Pack

The page Choose what to install compares the install shapes in detail, including which extension owns which feature when several are installed.

Quick start

These steps assume the Full Pack. With a single extension, the same screens appear with only that extension's tools.

1. Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Artisan Studio — Full Pack**, and select **Install**.
2. Open a project folder with **File › Open Folder…**. The extensions start automatically when VS Code finishes loading.
3. Select the Auric icon in the Activity Bar (the vertical bar at the far left). The **Overview** view opens with the Workstation card.
4. Select **Scan project** on the Overview. A progress notification appears while the Studio reads your stylesheets, templates and components. When it finishes, the score ring shows a 0–100 score and a letter grade.
5. Open a stylesheet or component. Problems are underlined with a violet wavy line. Hover one to see what is wrong, how urgent it is and how to fix it.
6. Press `Ctrl` + `Alt` + `.` (`Cmd` + `Alt` + `.` on macOS) to open the **Command Center**, a searchable list of suggested actions, tools, recent colours, findings and files the Studio has written.
7. Select **Studio** on the Overview (or the status-bar score) to open the Studio tab with the Home, Color, Check, Health and Brand spaces.
8. When you are ready to share results, run **Auric Artisan: Generate Project Health Report (Color, A11y & Code Health)**. It writes a Markdown, JSON and printable HTML report to the `.auric-artisan` folder in your workspace.

You should now see a score in the status bar, findings in the editor and a report you can open in your browser. The getting started guide walks through the same steps with a tour of every screen.

What you can do

- Hover any colour literal in CSS, JavaScript, TypeScript, HTML, Vue, Svelte, Python, Go, Rust, YAML and many more languages to read its RGB, HSL, OKLCH and Display-P3 values, its contrast on white and black, and how it looks to people with colour-vision deficiency.
- Edit colours in place with a picker in the hover, a full spectrum picker beside the editor, or a picker docked in the sidebar.
- Catch low-contrast text as you type, including text that uses CSS custom properties, CSS-in-JS and dark themes, and fix it with one click to the nearest passing colour of the same hue.
- Open any failing pair in the **Contrast Studio**, see where its token is used, and write the fix back to the token so every use follows.

- Lint markup and CSS against 70+ WCAG rules with an audit profile for WCAG 2.0, 2.1, 2.2 or the 3.0 draft, and apply safe fixes from the hover.
- Inspect any element or CSS rule for its accessible name, role, colours, contrast and colour-vision impact, with a before-and-after preview.
- Scan the whole project, browse findings grouped by type, file or severity, and see a prioritised list of files to fix first.
- Find hardcoded secrets (with the secret itself redacted everywhere), injection and TLS-bypass patterns, weak cryptography and debug leftovers in more than 40 languages.
- Write `health.json`, SARIF 2.1.0 and a trend history for CI, and gate pull requests on new issues only with the bundled command-line scanner.
- Generate an agent-ready remediation plan: a deterministic, local backlog of fixes and a prompt you can give to a coding agent you already use.
- Search a local rulebook and knowledge database covering every rule, all 86 WCAG 2.2 success criteria and eight manual test procedures, and add your team's own guidance.
- Mark deliberate brand colours as intentional branding so they stay visible but leave the score, while structural failures can never be waived.
- Re-theme the Studio with 13 accent presets or any custom colour.

In this folder

This folder documents the Studio as a whole and each extension in its own subfolder.

Folder or file	What it covers
docs/	Guides and reference shared by all four extensions: the Workstation, the Overview, the Command Center, the Studio, scanning, scores, reports, the CI gate and more.
others/	Glossary, shortcuts, limits and accuracy, related tools, release notes and privacy for the whole Studio.
full-pack/	The Full Pack: what it installs and how to enable, disable or remove its members.
color/	Auric Color: hover, swatches, pickers, contrast diagnostics, palettes, shades, colour vision and the brand audit.
accessibility/	Auric Accessibility: editor linting, Inspect, project scans, every rule by WCAG success criterion, profiles and manual evidence.
code-health/	Auric Code Health: every finding category and signature, redaction, health records, SARIF and baselines.

docs/

File	What it covers
README	Contents of the shared docs.
Getting started	Install, first scan and a tour of every screen.
Choose what to install	The four extensions compared and who owns which feature.
The Workstation	How co-installed extensions become one product.
The Overview view	The sidebar home in each of its three layouts.
The Command Center	The `Ctrl` + `Alt` + `.` list and the smart suggestions.
The Studio	The Studio tab, its spaces, Home, Findings and Settings.
The Contrast Studio	Tuning one pair and writing the fix back to its file or token.
Scanning	Project scans, live re-grading, the cache, `.auricignore` and the data folder.

File	What it covers
Scores and grades	How every score is calculated.
Suppression and branding	`auric-disable-*` and `auric-brand-*` comments.
Exports and health records	The project health report, `health.json`, SARIF, history and exports.
The remediation plan	The agent-ready fix plan.
Rulebook and knowledge database	Searching rules and adding team guidance.
The CI gate	The `auric-artisan` command-line scanner.
The Icon Studio bridge	Opening the combined Auric icon library.
Appearance	Accent colours, themes and high contrast.
Accessibility of the Studio	Using the Studio with a keyboard and screen reader.
Reference	Every command, shortcut, setting and file in one place.
Troubleshooting	Symptoms, causes and fixes.
FAQ	Short answers.

others/

File	What it covers
README	Contents of this folder.
Glossary	Every term used in these docs.
Shortcuts	Keyboard shortcuts and Command Palette titles.
Limits and accuracy	What the checks measure and what they cannot.
Related	Related extensions, website tools and articles.
Release notes	Version history from the changelog.
Privacy	What stays on your machine and what is written to disk.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Auric Artisan Studio docs

The shared guides and reference pages that apply to every Auric Artisan Studio extension.

This folder holds the guides that apply to the Studio as a whole: the screens every extension shares, how installed extensions combine into one Workstation, scanning, scores, reports and the CI gate. Features that belong to one extension are documented in that extension's folder: Auric Color, Auric Accessibility and Auric Code Health.

Start with Getting started. If you are deciding what to install, read Choose what to install first.

File	What it covers
Getting started	Install, run your first scan, and a tour of the Activity Bar views, status bar, hovers and the Studio.
Choose what to install	The four extensions compared, install shapes, and which extension owns which feature.
The Workstation	How co-installed extensions elect one host, hide duplicate views and share one Studio.
The Overview view	The sidebar home: workspace card, This file card, suggestions, swatches and navigation, in each layout.
The Command Center	The `Ctrl` + `Alt` + `.` list and the six deterministic suggestion rules.
The Studio	The Studio tab: the rail, its spaces, Home, Findings, Settings and keyboard use.
The Contrast Studio	Tune one text and background pair, see where its token is used, and write the fix back.
Scanning	Project scans, live re-grading, background scans, the scan cache, `.auricignore` and the `.auric-artisan` folder.
Scores and grades	How the overall, contrast, accessibility and code-health scores are calculated.
Suppression and branding	Silence a false positive or classify a brand colour, with comment directives or quick fixes.
Exports and health records	The project health report (Markdown, JSON, HTML), `health.json`, SARIF, history and Studio exports.
The remediation plan	The agent-ready, deterministic fix plan and what it contains.
Rulebook and knowledge database	Search rules, standards and procedures; build `knowledge.json`; add validated team overrides.
The CI gate	Run the `auric-artisan` command-line scanner, freeze a baseline and fail pull requests on new issues.
The Icon Studio bridge	Open the combined Auric icon library from any Studio extension.
Appearance	Change the accent colour and understand how the Studio follows your VS Code theme.
Accessibility of the Studio	Keyboard, screen reader, zoom, motion and high-contrast support in the Studio's own interface.
Reference	Every command, keybinding, view, setting, file and theme colour across the four extensions.
Troubleshooting	Symptoms, causes and what to do.
FAQ	Short answers to common questions.

Back to the Studio overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Getting started

Install the Studio, run your first project scan, and learn where every view, hover and status-bar item lives.

This guide installs Auric Artisan Studio, runs a first scan and tours every place the Studio shows information. It assumes the Full Pack, which installs Auric Color, Auric Accessibility and Auric Code Health together. If you install only one extension, the same screens appear with only that extension's parts; the differences are listed in Choose what to install.

Before you start

- You need Visual Studio Code 1.74 or newer.
- Open a folder or a multi-root workspace. Several features (the scan cache, reports, the remediation plan and the knowledge database) write to a `.auric-artisan` folder inside your workspace, so they need a folder to be open.
- No account, sign-in, API key or network connection is needed. Everything runs on your machine.

Install

1. Open the Extensions view with `Ctrl` + `Shift` + `X`.
2. Search for `auric-artisan.auric-artisan` (the Full Pack) and select **Install**. VS Code installs the pack and its three members: Auric Color, Auric Accessibility and Auric Code Health.
3. Alternatively, run this in a terminal:

```
code --install-extension auric-artisan.auric-artisan
```

The extensions start automatically once VS Code has finished loading. There is nothing to configure before the first use.

Your first scan

1. Open a project folder that contains stylesheets, templates or components.
2. Select the Auric icon in the Activity Bar. When all three extensions are installed, the sidebar is titled **Auric Accessibility** and shows three views: **Overview**, **Project Issues** and a collapsed **Color Picker**.
3. In the **Overview**, select **Scan project**. A notification titled **Auric Artisan: scanning project…** shows progress and has a **Cancel** button.
4. When the scan finishes, a message reports the result, for example `Accessibility 82/100 (B) — 12 contrast + 30 WCAG issue(s) across 214 files (scanned).` It offers **Open Studio**, **Show Issues** and **Force Rescan**.
5. The Overview's score ring now shows the overall score and grade, and the rows beside it show the accessibility, contrast and code-health results.

The scan result is saved in `.auric-artisan/scan-cache.json`. Next time you open the workspace, the Studio loads it instead of reading every file again, as long as it is less than 24 hours old (you can change this; see Scanning).

A tour of the screens

The Activity Bar views

View	What it shows
Overview	The Studio's home in the sidebar: the workspace score, the file you are editing, up to three suggestions, recent colours and buttons for the Studio, the Rulebook, the Icon Studio and the fix plan. See The Overview view.
Project Issues	Every finding from the last scan as a tree, grouped by type, file or severity, with search. Provided by Auric Accessibility. See The Project Issues view.

View	What it shows
Color Picker	A docked colour picker, collapsed by default. Provided by Auric Color. See The colour pickers.

The status bar

Two items appear at the right of the status bar:

- **The score item**, for example `A11y 82 B`. A database icon after it means the score came from the saved scan cache. Its background turns to the theme's warning colour below 90 and to the error colour below 70. Hover it for the pass rate of contrast pairs, the data source and a warning if a file limit cut the scan short. Select it to open the Studio.
- **Auric: N in this file** appears only when the active file has contrast or accessibility findings. Hover it for the breakdown; select it to open the Project Issues view (or, without Auric Accessibility, the Contrast Studio).

When Auric Code Health is installed on its own, the score item reads `Health` instead of `A11y` and shows the code-health score.

In the editor

- **Underlines.** Auric findings are marked with a violet wavy underline and a mark in the overview ruler at the right edge of the editor. They also appear in the **Problems** panel with the source `auric-artisan`.
- **Issue hovers.** Hover an underlined finding for a card that leads with what is wrong and how urgent it is, then shows the evidence, a before-and-after of the fix, and actions such as **Apply fix**, **Go to code**, **WCAG docs**, **Ignore here** and **Copy reference**.
- **Colour hovers.** Hover any colour literal, such as `#1d4ed8` or `oklch(0.6 0.15 250)`, for its values, contrast on white and black, a colour-vision preview and an inline picker. See Hover and swatches.
- **Inspect.** Hover an element or CSS rule (when you are not over a colour or a finding) for its accessible name and role, the colours that apply and a before-and-after preview. See Inspect.
- **Quick fixes.** Put the cursor on an underline and press `Ctrl` + `.` to see fixes such as **Fix contrast → #767676 (4.54:1)** or **Ignore this WCAG 1.1.1 finding here**.
- **Swatches.** A small coloured square appears before every colour literal. Clicking a literal opens the inline picker.

The Studio

Select **Studio** in the Overview, or the status-bar score, to open the Studio in an editor tab titled **Auric Artisan — Workstation Studio**. A rail on the left lists its spaces:

Space	Tools inside
Home	The workspace score, files to fix first, graphs, suggestions, the CI gate command and the project palette.
Color	**Palette**, **Shades** and **Vision**.
Check	**Findings**, **Contrast** (the Contrast Studio) and **Preview**.
Health	The code-health score, categories and findings.
Brand	A brand accessibility audit on real text.
Settings	Appearance and the settings for installed capabilities.

The **Rulebook** and **Icons** buttons at the foot of the rail open the knowledge database and the Icon Studio. See The Studio.

The Command Center

Press `Ctrl` + `Alt` + `.` (`Cmd` + `Alt` + `.` on macOS) anywhere to open **Auric Artisan — Command Center**, a searchable list with **Suggested**, **Actions**, **Recent colors**, **Findings** and **Artifacts** sections. See The Command Center.

Your first fix

1. Open a stylesheet with a violet underline under a colour value.
2. Hover the underline. The card leads with the verdict, for example `2.32 : 1 — fails WCAG AA for normal text`, and shows the nearest passing colour of the same hue.
3. Select the gold **Apply fix** button. The value is replaced in one edit you can undo with `Ctrl` + `Z`.
4. If the colour comes from a CSS custom property defined in the same file, the button reads **Apply to --token-name** instead and changes the definition, so every use follows.

The underline disappears and the status-bar count drops. To fix every failing pair in the file at once, run **Auric Artisan: Fix All Contrast Issues in File**; it shows a preview of every change and asks before applying.

Where to go next

- The Overview view and The Studio for the full screen reference.
- Scanning to control what is scanned and when.
- Auric Color, Auric Accessibility and Auric Code Health for each extension's features.
- Exports and health records and The CI gate to share results and gate pull requests.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Appearance

Change the Studio's accent colour and understand how the Studio follows your VS Code light, dark and high-contrast themes.

The Studio's own screens (the Overview, the Studio tab and the colour pickers) share one visual style that follows your VS Code theme. You can change one thing about it: the **accent colour** used for primary buttons, selected tabs and highlights.

How the Studio follows your theme

Your VS Code theme	What the Studio looks like
Dark	Warm near-black surfaces, light text, gold primary actions
Light	Paper-coloured surfaces, dark text, gold primary actions
High contrast (dark or light)	VS Code's own high-contrast colours and borders; the Studio does not repaint them

Headings and scores use the Fraunces typeface and other text uses Manrope. Both are bundled with the extensions as small Latin subsets, so nothing is downloaded.

What VS Code draws itself stays native and follows your theme: hovers, the Command Center list, the status bar, the Problems panel and the Project Issues tree. The gold **Apply fix** button in hovers always uses Auric gold.

Change the accent colour

The accent colour recolours the Studio, the pickers, buttons, tabs and highlights, in the same way as the accent setting on https://auricartisan.com.

In the Studio

1. Open the Studio and select **Settings** at the foot of the rail.
2. In the **Appearance** card, select one of the 13 swatches, or the **+** swatch to pick any colour with the full picker.
3. The whole Studio changes immediately, and the choice is saved; there is no Save button for this card. A toast confirms, for example **Accent → Sapphire**.
4. **Reset to Gold** returns to the default.

The card shows the current accent and a small live preview of tabs, buttons, a badge and a bar. Arrow keys move between the swatches.

With a command

1. Run **Auric Artisan: Change Accent Color (Theme the Studio)** (Auric Color only).
2. Pick a preset (the current one is marked **current**), **Custom color…** to type a hex value such as `#2563EB` (three or six digits), or **Open Studio Appearance settings**.
3. The status bar confirms, for example `Auric Artisan accent → Sapphire #2F5EAA`. Open Studio and picker views update straight away.

In settings.json

Set the `appearance.accent` key of the extension that hosts your Studio (`auricA11y`, `auricColor` or `auricCodeHealth`) to a preset ID or a hex colour:

```
{
  "auricA11y.appearance.accent": "sapphire",
  "auricColor.appearance.accent": "#2563EB"
}
```

Anything that is not a preset ID or a `#rgb` or `#rrggbb` value falls back to `gold`.

The presets

Swatch name	Setting value	Colour
Gold (default)	`gold`	`#D3AF37`
Champagne	`champagne`	`#DEC68A`
Rose Gold	`rose-gold`	`#B76E79`
Silver	`silver`	`#C4C7CE`
Platinum Grey	`platinum`	`#8A8F99`
Bronze	`bronze`	`#B08D57`
Copper	`copper`	`#B87333`
Brown	`espresso`	`#8B5A2B`

Swatch name	Setting value	Colour
Maroon	`maroon`	`#800020`
Pink	`rose`	`#E75480`
Emerald	`emerald`	`#3EA882`
Sapphire	`sapphire`	`#2F5EAA`
Amethyst	`amethyst`	`#9966CC`

Where the accent is stored

The accent is a personal preference, so the Studio always saves it to your **user** settings, not the workspace. It follows you to every workspace on that machine (and to others if you use Settings Sync).

Each Studio extension has its own `appearance.accent` key. The Studio you see reads the key of the extension that hosts it (see The Workstation), and the Appearance card and the command write to that key.

Readability of the accent

A light accent such as Champagne or Silver would be hard to read as text on the paper theme. The Studio adjusts the accent's text colour, darkening or lightening it only as much as needed, until it reaches at least 4.5:1 on every surface it is used on, including its own tinted highlight. Button labels on the accent use black or white, whichever reads better.

Branding colours

The pink colours used for the Branding group can be changed through VS Code's `workbench.colorCustomizations`. See Theme colours.

Related

- The Studio
- Accessibility of the Studio

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Choose what to install

Compare the four Studio extensions, see what each install shape gives you, and learn which extension owns which feature when several are installed.

Auric Artisan Studio is four Marketplace extensions. Three of them contain code and do one job each; the fourth is an extension pack that installs the other three. You can install any combination. This page explains what each shape gives you.

The four extensions at a glance

	Auric Color	Auric Accessibility	Auric Code Health	Full Pack
Marketplace ID	`auric-artisan.auric-color`	`auric-artisan.auric-a11y`	`auric-artisan.auric-codehealth`	`auric-artisan.auric-artisan`

	Auric Color	Auric Accessibility	Auric Code Health	Full Pack
Main job	Colour picking, palettes, contrast	WCAG linting and project audits	Secrets, security and hygiene checks	Installs the other three
Activity Bar views	Overview, Color Picker	Overview, Project Issues, Color Picker (when colour is installed)	Overview	None of its own
Studio spaces (on its own)	Color, Check (Contrast), Brand, Settings	Home, Check (Findings, Contrast, Preview), Settings	Health, Settings	All spaces
Keyboard shortcuts	Command Center, Pick / edit color	Command Center, Inspect	Command Center	Those of its members
Settings	16 under `auricColor.`	24 under `auricA11y.`	28 under `auricCodeHealth.`	None of its own

What each extension does when installed alone

Auric Color alone

- Colour hovers, swatches and pickers in more than 40 languages.
- Contrast diagnostics (underlines, quick fixes and **Fix All Contrast Issues in File**) in stylesheets, markup and components.
- The Studio opens on the **Color** space (Palette, Shades, Vision), with **Check › Contrast**, **Brand** and **Settings**.
- The Overview is a colour workbench: the file you are editing, suggestions about that file, a brand swatch row, your recent colours, **Open the color picker** and shortcuts to the Studio tools.
- There is no **Scan project** button. The status-bar score still appears once the Studio has graded the workspace in the background while you edit.

Auric Accessibility alone

- WCAG diagnostics for markup and CSS, Inspect cards on hover, contrast diagnostics and quick fixes.
- **Scan Project for Accessibility**, the **Project Issues** view and the status-bar score.
- The Studio opens on **Home**, with **Check** (Findings, Contrast, Preview) and **Settings**.
- The agent-ready remediation plan.
- Project scans also run the code-health checks, so code-health findings appear in the Project Issues view and in reports. Code-health underlines in the editor need Auric Code Health.

Auric Code Health alone

- Code-health underlines and quick fixes in every supported language.
- The Overview is a hygiene report: a code-health ring, seven category chips, **Report** and **health.json + SARIF**.
- The Studio opens on the **Health** space, with **Settings**.
- **Generate Project Health Report**, **Write Health Record**, the remediation plan and the command-line scanner.
- The status-bar item reads `Health` and shows the code-health score.

The Full Pack, or any two together

When two or three code-bearing extensions are installed, they combine into one Workstation. One of them hosts a single sidebar and one Studio that contains every installed capability; the others hide their own views. See The Workstation.

Who owns which feature

When several extensions are installed, each editor-wide feature has exactly one owner, so you never see a colour swatch drawn twice or the same problem listed three times in the Problems panel.

Feature	Owner
Colour swatches, colour hover and inline picker, click-to-pick, VS Code colour decorator handling	Auric Color only
Inspect cards on elements and CSS rules	Auric Accessibility only
WCAG diagnostics	Auric Accessibility only
Code-health diagnostics	Auric Code Health only
Contrast diagnostics	Auric Accessibility; Auric Color when Auric Accessibility is not installed
Status-bar score, live re-grading as you type, automatic background scans	The first installed of Auric Accessibility, then Auric Color, then Auric Code Health

Commands you run yourself, such as **Scan Project for Accessibility** or **Write Health Record**, are never restricted: any installed extension that has the command can run it.

Install commands

```
code --install-extension auric-artisan.auric-artisan
code --install-extension auric-artisan.auric-color
code --install-extension auric-artisan.auric-a11y
code --install-extension auric-artisan.auric-codehealth
```

Changing your mind later

You can add or remove any extension at any time from the Extensions view. The Workstation re-elects its host immediately, without reloading the window: views appear or hide, the Studio rebuilds its spaces and the Command Center lists the new set of actions. See Manage the pack's members.

Related

- The Workstation
- Full Pack overview
- Reference for every command and setting per extension.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The CI gate

Run the bundled auric-artisan command-line scanner, freeze a baseline, fail pull requests only on new issues, and upload SARIF to code scanning.

The Studio extensions include a command-line scanner, `auric-artisan`, that runs the same colour, accessibility and code-health engine as the editor, without VS Code and without any network access. Use it to fail a pull request when it introduces

new problems, and to send findings to your code-scanning dashboard as SARIF.

CI (continuous integration) is the automated build and test run on every pull request. A **baseline** is a saved record of the findings a project already has, so a gate can fail only on findings that are new.

Get the scanner

The scanner is the file `bin/auric-artisan.js` inside each installed Studio extension (Auric Color, Auric Accessibility and Auric Code Health). It is not published as a separate npm package. It needs Node.js; the GitHub Actions example below uses Node 20.

VS Code installs extensions in the `extensions` folder of your user profile's `.vscode` folder, in a folder named after the extension and its version, for example:

```
~/.vscode/extensions/auric-artisan.auric-codehealth-0.1.0/bin/auric-artisan.js
```

On Windows this is `%USERPROFILE%\.vscode\extensions\auric-artisan.auric-codehealth-0.1.0\bin\auric-artisan.js`.

Run it with Node:

```
node ~/.vscode/extensions/auric-artisan.auric-codehealth-0.1.0/bin/auric-artisan.js scan .
```

The examples on this page write `auric-artisan` for short. To make that work, create a shell alias or a small script on your path that runs the file with Node. The Studio's **Home › CI gate** card shows the same short form.

> **Important:** The Auric Artisan Free Use Licence 2.0 allows you to install and run the extensions on machines you own or control, but does not allow you to redistribute their files, including publishing them to a public repository. Do not commit the scanner's files to a public repository. In CI, download the extension from the Visual Studio Marketplace during the job, as shown below.

Usage

```
auric-artisan scan [dir] [options]
```

`dir` is the folder to scan; it defaults to the current folder.

Option	What it does
`--json <file>`	Write the `health.json` record to a file
`--sarif <file>`	Write SARIF 2.1.0 to a file, for GitHub code scanning
`--baseline <file>`	Compare with a saved record and report only new issues
`--write-baseline <file>`	Save the current record as a baseline, then exit
`--fail-on <severity>`	Exit with code 1 if any issue is at or above `high`, `medium` or `low` (`critical` and `info` also work)
`--max-new <n>`	Exit with code 1 if there are more than `n` new issues; needs `--baseline`
`--min-score <n>`	Exit with code 1 if the overall score is below `n` (0–100)

Option	What it does
`--profile <profile>`	WCAG profile: `2.0`, `2.1`, `2.2` (default) or `3.0-draft`
`--threshold <ratio>`	WCAG contrast ratio target (default 4.5)
`--no-code-health`	Skip the code-health checks
`--max-files <n>`	Most files to scan (default 20000)
`--quiet`	Do not print the console summary
`--version`	Print the version
`--help`	Print the usage

With `--baseline`, `--fail-on` counts only new issues.

Exit codes

Code	Meaning
`0`	Passed, or no policy was set
`1`	A policy failed (`--fail-on`, `--max-new` or `--min-score`)
`2`	A usage or runtime error: no command, an unknown command, a missing folder, no scannable files, or a baseline that cannot be read

Running the scanner with no command, or with `--help` alone, prints the usage and exits with code 2. `auric-artisan help` exits with 0.

What it scans

- Files with any extension the editor scan reads (see Scanning), plus `.xhtml`, `.svg` (as XML) and `.tf` (Terraform, code health only). A file named `.env` is read too.
- It skips folders named `node_modules`, `.git`, `.hg`, `.svn`, `dist`, `build`, `out`, `.next`, `.nuxt`, `.svelte-kit`, `coverage`, `vendor`, `.auric-artisan`, `.cache`, `__pycache__`, `.venv`, `venv`, `.idea` and `.vscode-test`, and every folder whose name starts with a dot.
- It reads the `.auricignore` file at the root of the scanned folder.
- It skips files larger than 1.5 MB.
- When `--max-files` stops the scan early, it prints `⚠ Scanned N of M+ files (capped — raise --max-files).` so a partial scan is never mistaken for a clean one.

The scanner does not read your VS Code settings. It uses its own options: contrast target 4.5 unless you pass `--threshold`, profile 2.2 unless you pass `--profile`, all code-health categories and no custom secret patterns. It always grades rules that set only a text colour against the page background, as the editor does with `contrast.checkPageBackground` on.

Console output

Without `--quiet`, the scanner prints a summary:

```
Auric Artisan — scanned 4 files
  score 72/100  C   a11y 3  contrast 2  cvd 0  code-health 3
  1 critical  4 high  1 medium  1 low  1 info   (8 total)
  1 new  0 fixed  7 pre-existing (vs baseline)

  high   contrast  src/a.css:3
         #AAAAAA|#FFFFFF

✗ FAILED: 1 new issue(s) > --max-new 0
```

It lists up to 50 issues, worst first (only the new ones when you compare with a baseline), then `… and N more`. When every policy passes, it prints `✓ PASSED policy checks.`

Adopt it on an existing codebase

A project that already has findings would fail a strict gate immediately. Freeze the current findings once, then fail only on new ones.

Step 1. Create the baseline and commit it:

```
auric-artisan scan . --write-baseline .auric/baseline.json
```

The scanner prints `Wrote baseline (N issues) → .auric/baseline.json`. Keep the baseline in a folder whose name starts with a dot, such as `.auric/`, so the scanner does not read the baseline itself as a source file.

Step 2. In CI, compare every run with the baseline:

```
auric-artisan scan . --baseline .auric/baseline.json --max-new 0 --sarif auric.sarif
```

Step 3. When you fix old findings, regenerate the baseline so they cannot quietly return.

Issues are matched by fingerprint, which includes the line number. A finding that moves to another line counts as new once; regenerate the baseline after large refactors. See Fingerprints.

> **Note:** Code-health findings are fingerprinted by detector version 1.2.0 in the current package. A baseline written by an earlier detector may list some existing code-health findings as new once; regenerate it.

GitHub Actions

This workflow downloads Auric Code Health from the Marketplace during the job, scans the pull request, uploads SARIF to GitHub code scanning and fails the check when a new issue appears.

```
name: Auric Artisan
on: [pull_request]
jobs:
  audit:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Download Auric Code Health from the Marketplace
        run: |
          curl -sSL --compressed -o auric.vsix \
            "https://marketplace.visualstudio.com/_apis/public/gallery/publishers/auric-artisan/vsextensions/auric-
codehealth/0.1.0/vspackage"
          unzip -q auric.vsix -d "$RUNNER_TEMP/auric"
      - name: Scan
        run: >
          node "$RUNNER_TEMP/auric/extension/bin/auric-artisan.js" scan .
          --baseline .auric/baseline.json --max-new 0
          --sarif auric.sarif --min-score 70
      - if: always()
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: auric.sarif
```

- The package is a ZIP file; `--compressed` is needed because the Marketplace sends it compressed.
- Replace `0.1.0` with the version you want. Pinning a version keeps results reproducible.
- `security-events: write` lets the job upload SARIF. Findings then appear in the repository's Security tab and inline on the pull request's diff.

Pre-commit hook

On your own machine, where the extension is installed, a Git pre-commit hook can block commits with high-severity issues:

```
auric-artisan scan src --fail-on high --quiet
```

Same engine, same findings

The scanner, project scans in the editor and live diagnostics share one engine, so the same file with the same settings produces the same findings and fingerprints everywhere. Differences come only from settings: the editor uses your VS Code settings, the scanner uses its options.

Related

- Exports and health records for the `health.json` and SARIF formats.
- Scanning and `.auricignore`.
- Scores and grades for `--min-score`.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The Command Center

Open the Command Center with one shortcut, understand each of its sections, and learn the six deterministic rules behind its suggestions.

The **Command Center** is a searchable list of everything the installed Studio extensions can do right now: suggested next steps, actions, recent colours, the worst findings and the files the Studio has written. It is a native VS Code quick pick, so it looks and behaves like the Command Palette.

Open it

- Press `Ctrl` + `Alt` + `.` (`Cmd` + `Alt` + `.` on macOS). The shortcut works anywhere in VS Code.
- Or run **Auric Artisan: Workstation: Command Center** from the Command Palette (`Ctrl` + `Shift` + `P`).

The list is titled **Auric Artisan — Command Center**. Type to filter; the filter matches titles and their descriptions. Press `Enter` to run the highlighted item, or `Escape` to close.

When several Studio extensions are installed, the shortcut opens the host's Command Center, which covers every installed capability. See The Workstation.

Sections

Sections appear in this order, and only when they have something in them.

Suggested

The same deterministic suggestions the Overview shows, up to three. Each shows its title and, as its description, the reason it fired. See The suggestion rules.

Actions

Only actions for installed capabilities are listed.

Action	Description shown	Needs
Scan project	WCAG + contrast + CVD (uses cache)	Auric Accessibility
Rescan project	force-refresh the scan cache	Auric Accessibility
Open Studio: …	none; one entry per Studio tool (listed below)	The tool's extension
Open rulebook & knowledge database	rules, standards, manual checks, policies, and articles	Any
Build workspace knowledge database	knowledge.json + rulebook.md + validated team overrides	Any
Open unified Icon Studio	how many icon collections are installed and through which provider, or install an Auric Icons provider to enable the combined library	Any
Pick / edit color	inline picker on the token under the cursor	Auric Color
Check contrast of selection		Auric Color
Fix all contrast in file	recolor every failing pair (one undo)	Auric Color

Action	Description shown	Needs
Project health report	Markdown + JSON, written locally	Auric Code Health
Write health record	health.json + SARIF for CI	Auric Code Health
Open data folder	.auric-artisan workspace store	Auric Code Health
Generate AI remediation plan	agent-ready fix plan with a design-token draft	Auric Accessibility or Auric Code Health

The **Open Studio** entries are, in order and for installed capabilities only: **Scan** (the Studio's **Home** space), **Code Health**, **Preview**, **Contrast**, **Brand**, **Palette**, **Shades**, **Vision** and **Settings**.

Recent colors

Up to eight colours you applied recently with a Studio picker (Auric Color). Select one to open the colour picker on that value. If an editor is open, a colour you then apply from the picker is inserted at the cursor.

Findings

Up to eight of the worst findings from the last scan, each with its file name and line. Select one to open the file at that line. Contrast rows read like `#999999 on #FFFFFF · 2.32:1`; other rows show the finding's message.

Artifacts

Files the Studio has written to the workspace's `.auric-artisan` folder. Only files that exist are listed.

File	Description shown
`health.json` (or the name set in `health.fileName`)	unified health record
`knowledge.json`	searchable rules and guidance database
`rulebook.md`	generated engineering rulebooks
`knowledge-overrides.json`	reviewed workspace knowledge overrides
`ai-remediation-plan.md`	agent-ready remediation plan
`project-health.md`	project health report

The suggestion rules

Suggestions are produced by a fixed set of six rules. They are **not** generated by AI or machine learning, and nothing is sent anywhere: each rule checks one fact from your last scan or your open file. Rules are checked in the order below, at most three suggestions are shown, and a suggestion you dismiss is never offered again.

#	Suggestion	Pill	Shown when	Action
1	**N possible secrets detected**	Critical	The last scan found at least one possible hardcoded secret	**Open health report** (with Auric Code Health), otherwise **Review in Studio**
2	**N fixable contrast pairs in this file**	High	Auric Color is installed and the active file has at least two contrast pairs that can be fixed automatically	**Fix all in file**, plus **Review**, which opens the first pair in the Contrast Studio
3	**"rule-id" appears in N files**	High	Auric Accessibility is installed and one WCAG rule fires in three or more files	**Open the findings** (Check › Findings)

#	Suggestion	Pill	Shown when	Action
4	**Scan is N hours old**	Workflow	Auric Accessibility is installed and the loaded scan is older than `scan.cache.maxAgeMinutes`	**Rescan**
5	**Scanned X of Y files**	Workflow	Auric Accessibility or Auric Color is installed and the last scan stopped at the `scan.maxFiles` limit	**Scan settings** (Studio › Settings)
6	**N color values repeat 4+ times**	Tip	At least six colour values are each used four or more times, and Auric Accessibility or Auric Code Health is installed	**Draft tokens** (generates the remediation plan, which includes a design-token draft)

Each suggestion states a detected fact in its title and gives a one-line reason, for example **the code-health scan found hardcoded credentials** or **one rule firing across files usually shares a single fix**.

Dismissing suggestions

- In the Overview, select ✕ on a suggestion card. It disappears and is not offered again, even after later scans.
- VS Code remembers the dismissal across sessions. The Studio keeps the 40 most recent dismissals.
- There is no command to bring a dismissed suggestion back.

The Colour workbench layout of the Overview shows only suggestion 2, because it has no project score. The Command Center itself always evaluates all six rules for the installed capabilities.

Related

- The Overview view
- The remediation plan
- Reference for every command.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The Contrast Studio

Tune one text and background pair, choose a passing colour that keeps the hue, see where its token is used, and write the fix back to your code.

The **Contrast Studio** is the Studio's **Check › Contrast** tool. It shows one text colour on one background, grades the pair against WCAG, lists the nearest passing colours that keep the same hue, and, when the pair came from your code, writes the fix back to the file or to the design token the colour comes from. It is available with Auric Accessibility or Auric Color.

Contrast ratio is WCAG 2.x's measure of how far apart two colours are in brightness, from 1:1 (identical) to 21:1 (black on white). WCAG asks for at least 4.5:1 for normal text (level AA), 3:1 for large text and user-interface parts, and 7:1 for level AAA.

Linked and unlinked pairs

A pair is **linked** when you open it from a place in your code. The Studio remembers the file and position, shows where the pair lives, and can write a fix back. A pair is **unlinked** when you type colours in yourself or open the tool from a colour selection; you can still tune it and copy the result.

Ways to open a **linked** pair:

- **Open in Contrast Studio** in a contrast hover.

- The quick fix **Open in Auric Artisan Studio** (or **Open in the Contrast Studio** for values that cannot be rewritten in one click).
- **Open Issue in Contrast Studio**, the inline action on a contrast or colour-vision row in the Project Issues view.
- **Contrast Studio** on the Overview's **This file** card.
- **Review** on the suggestion **N fixable contrast pairs in this file**.

Ways to open an **unlinked** pair:

- Select a colour in the editor and run **Auric Artisan: Check Contrast of Selection** (also in the editor's right-click menu when text is selected). The selected colour becomes the text colour; if the selection is not a colour, the Studio starts from `#1A1A1A`.
- Open **Check › Contrast** in the Studio and edit the colours.
- **Open in Contrast** → from the Preview tool, or **Tune the pair** in an issue hover.

Linked pairs come from stylesheets and markup files: CSS, SCSS, Sass, Less, PostCSS, Stylus, HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX and Markdown.

The header

The kicker reads **Contrast Studio**. The title is **Contrast checker** for an unlinked pair, or **Text on** followed by the rule's selector or the element, for example **Text on .card-meta**, for a linked pair. A linked pair also shows its location, for example `styles.css:42`, and a **Back to file** button that selects the colour in the editor.

The pair

- **Sample.** A **Sample button** and a line of body text drawn in the text colour on the background.
- **Ratio and APCA.** The contrast ratio, for example `3.54 : 1`, and the APCA lightness contrast, for example `APCA Lc 61 · large/medium text ≥ 18px`. See Audit profiles and contrast standards for what APCA means.
- **Verdict line.** The ratio and whether it passes your target, for example `3.54 : 1 — fails WCAG AA for normal text (passes for large text)`. The Studio cannot see the font size of rendered text, so the verdict names the check rather than a pixel size.
- **Four checks. AA text · 4.5**, **AA large · 3**, **UI parts · 3** and **AAA text · 7**, each marked ✓ **Pass** or ✕ **Fail**.
- **Text** and **Background** fields. Each has a swatch button that opens the Studio's colour picker, a text box that accepts any CSS colour, and a copy button. ⇄ swaps the two colours.
- **Where the colour comes from** (linked pairs): for example `The file's text color reads --muted (#8A8F99).` If the background was assumed from the page (the pair's rule sets only `color:`), a note asks you to check the real surface.
- **Adjust the text colour: Fix to AA (4.5:1)**, **AAA (7:1)** and **UI (3:1)** change the text colour's lightness until it reaches that ratio, keeping its hue and chroma. **Best text → black** (or **white**) picks whichever of pure black or white reads better on the background.

The target used for the verdict and the candidates is your `contrast.threshold` setting (4.5 by default). For a linked pair of large text (at least 24 px, or at least 18.66 px and bold), the target is 3:1, or 4.5:1 when your threshold is 7.

Passing colours that keep the hue

The candidates panel is titled **Passing colors that keep the hue**, or **Stronger colors that keep the hue** when the pair already passes (it then says **no change is needed**). It lists up to five colours, **sorted by ΔE**, the closest first.

Each candidate shows:

- a swatch and its hex value;
- its ratio on the background and its level: **AAA**, **AA** or **AA large**;
- **ΔE**, how different it looks from the original text colour (CIEDE2000; below about 1 is hard to see, larger numbers are bigger changes);
- **closest** and ◆ **Recommended** on the first candidate when a fix is needed;
- **reuse --token** when the candidate is exactly the value of a custom property already defined in the same file, so you can reuse your design system instead of adding a new colour.

How candidates are found:

1. The Studio walks the text colour's lightness up and down until it reaches each rung of a ladder of targets (3, 4.5, 5.5, 7, 9, 12, 15 and 18 to 1) at or above your threshold. Hue and chroma are preserved.
2. It adds any opaque colour token defined in the file that passes and keeps the hue: within 20 degrees of hue in OKLCH, and greys only match greys.
3. It sorts them by ΔE from the original.

If no colour of that hue can reach the target on that background, the panel says **No color with this hue reaches 4.5 : 1 on this background. Try another background or hue.**

Select a candidate to preview it. On a linked pair, the list stays computed from the file's original colour, so it does not reshuffle while you compare.

Where the token is used

When the linked text colour comes from a CSS custom property, for example `color: var(--muted)`, a **Where --muted is used** panel lists its definitions and uses, up to 40, with the file name, line number, **defined** on definition lines, and the line's code. Select a row to open it. The panel searches your open files and the files from the last scan; it does not read other files from disk. A note such as **Showing 40 of 63.** appears when there are more.

Apply the fix

The apply panel depends on the pair:

Situation	What you see
Unlinked pair	**Use #hex as the text color**, and the note **Open a pair from an editor hover to write a fix back to its file.**
Linked, but you changed the background	**The background differs from the file's #hex, so a fix can't be written back.** and **Restore the file's colors**.
Linked, colour from a token defined in the same file	**Apply #hex to --token** (changes the definition; the note says how many uses follow), and **Apply only at file:line**
Linked, any other colour	**Apply #hex at file:line**
Linked and failing	**Mark as intentional branding** as well

Every write is a single edit you can undo with `Ctrl` + `Z`. When you apply a reused token, the Studio writes `var(--token)` rather than a hex value. After a write, the Studio re-reads the pair from the file and shows the new verdict, and the status bar confirms the change, for example `Auric Artisan: --muted → #6B7079 (undo with Ctrl+Z)`.

When a token is defined in several places, **Apply to --token** changes the light theme's root definition (in `:root`, `html`, `body`, `*` or `:host`), not a later dark-theme override.

Mark as intentional branding adds an `auric-brand-line` comment to the pair's line, so the pair moves to the Branding group and leaves the score. Selecting it again removes the mark. See Suppression and branding.

At the bottom of the panel: **Copy text #hex**, **Copy bg #hex** and **Insert text color**, which replaces the editor selection with the text colour (or copies it to the clipboard when no editor is open). The panel ends with **Computed on this machine. Nothing leaves it.**

Safety checks on write-back

The Studio re-reads the file before every write, so a fix can never land on the wrong text:

- If the line changed after you opened the pair: **the source line changed since the Studio opened it — open the pair from the editor again.**
- If the token is defined in another file: **this token is not defined in the same file, so only this use can be changed.**
- If the link was lost (for example you opened a different pair meanwhile): **this pair is no longer linked to its file — open it from the editor again.**

Related

- Contrast diagnostics
- The Check space: Findings and Preview
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Exports and health records

Generate the project health report, write health.json, SARIF and trend history, and export Markdown summaries from the Studio.

The Studio can turn a scan into files you can read, print, share or feed to other tools. All of them are generated on your machine and written to your workspace; nothing is uploaded.

Output	Made by	Format	Written to
Project health report	**Generate Project Health Report (Color, A11y & Code Health)**	Markdown, JSON, HTML	`.auric-artisan/project-health.*`
Health record	**Write Health Record (health.json + SARIF)**, or automatically after each scan	JSON	`.auric-artisan/health.json`
SARIF export	Same as the health record	SARIF 2.1.0 (JSON)	`.auric-artisan/health.sarif`
Trend history	Same as the health record	JSON	`.auric-artisan/health-history.json`
Accessibility summary	Studio › Home › **Report** (without Auric Code Health)	Markdown	Where you choose
WCAG summary	Studio › Check › Findings › **Export WCAG**	Markdown	Where you choose
Remediation plan	**Generate Agent-Ready Remediation Plan**	Markdown, JSON	`.auric-artisan/ai-remediation-plan.*`
Palette, shade and brand token exports	Studio › Color and Brand tools	CSS, JSON, Tailwind config	Where you choose

The two commands that make the project health report and the health record come with Auric Code Health. The health record files are also written automatically after every project scan, whichever Studio extension ran it (see `health.writeOnScan` below). All outputs that go to `.auric-artisan` need a workspace folder to be open; without one you see, for example, **Auric Artisan: open a workspace folder before generating a Project Health report.**

The project health report

Generate it

1. Run **Auric Artisan: Generate Project Health Report (Color, A11y & Code Health)**, or select **Report** on the Overview, in the Studio's Home or Health space, or **Project health report** in the Command Center.
2. The Studio uses a fresh scan cache if there is one, or scans first.
3. It writes three files to `.auric-artisan` and opens the Markdown one.
4. A message confirms, for example **Auric Artisan: Project Health report ready — grade B, 14 code-health finding(s). Generated locally.**, with three buttons:
   - **Open in Browser** opens `project-health.html` in your default browser. The file is local; nothing is fetched.
   - **Open JSON** opens `project-health.json`.
   - **Open Data Folder** shows the folder in your file manager.

Although the command comes with Auric Code Health, the report covers colour, accessibility and code health even when Code Health is the only Studio extension installed.

project-health.md

Section	Contents
Heading	**Auric Artisan — Project Health Report**, the date and time, the WCAG profile, and **all analysis ran locally**
Overview	Overall grade and score, files and languages, unique colours, contrast pairs, accessibility score and issues, code-health score and findings
Languages	Up to 20 languages with their file counts
Code Health	Counts per category and the top 20 findings with file and line
Accessibility (WCAG 2.2)	Counts by impact and the top 15 findings with criterion, impact, message and location
Worst contrast pairs	Up to 15 pairs: foreground, background, ratio, ratio needed, location; large text and dark theme are marked
Recommendations	Up to 8 recommendations from the scan

project-health.json

A JSON file with `"schema": "auric-artisan.project-health"`, the time it was generated and the full scan result. Long lists are capped: up to 600 failing pairs, 300 colour-vision risks, 600 WCAG findings, 800 code-health findings, 600 branding findings, 96 colours and 20 file hotspots.

project-health.html

A printable page styled like the Studio's paper theme. It is self-contained: the fonts are embedded, there are no scripts, and it loads nothing from the network, so it looks the same opened from disk, attached to an email or printed. It contains:

- the workspace name, scan date, number of files, WCAG profile and contrast target;
- score cards for **Overall**, **Accessibility**, **Contrast** and **Code health**;

- **Fix these first**: up to five recommendations with their priority;
- **Contrast pairs below 4.5:1** (your target): up to 12 pairs with a sample, the pair, its location, its ratio and the **nearest passing, same hue** colour with its ratio;
- **Accessibility by criterion**: up to 12 success criteria with counts;
- **Code health by kind**: counts per category, with secrets highlighted;
- **Files to fix first**: up to eight files with contrast, WCAG and colour-vision counts;
- a footer: **Nothing in this report left the machine it was made on.**

The health record

The health record is a stable, versioned, machine-readable record of every issue in the project, for CI dashboards, code scanning and trend tracking.

Write it

- Run **Auric Artisan: Write Health Record (health.json + SARIF)**, or select **SARIF** (Overview), **health.json + SARIF** (Overview or Studio › Health) or **Write health.json + SARIF** (Studio › Home › CI gate).
- The Studio uses a fresh scan cache or scans first, writes the formats listed in `health.formats`, and opens `health.json`.
- A message lists what was written, for example **Auric Artisan: wrote health.json, health.sarif, health-history.json — grade B, 14 code-health + 3 contrast + 21 a11y issues. Generated locally.**, with **Open SARIF** and **Open Data Folder**.

With `health.writeOnScan` on (the default), the record is also rewritten after every project scan, including automatic scans. Live re-grades never write files. If `health.enabled` is off, the command says **Auric Artisan: health report is disabled (auricCodeHealth.health.enabled).**

Settings

Setting	Default	Effect
`auricCodeHealth.health.enabled`	`true`	Maintain the record at all
`auricCodeHealth.health.writeOnScan`	`true`	Rewrite it after each project scan
`auricCodeHealth.health.formats`	`["json","sarif","history"]`	Which files to write
`auricCodeHealth.health.fileName`	`health.json`	Name of the JSON file; the SARIF and history names follow it
`auricCodeHealth.health.maxIssues`	`5000`	Most issues recorded, worst first (100–100000)
`auricCodeHealth.health.historyLimit`	`200`	Most history entries kept (10–5000)

For example, with `health.fileName` set to `quality.json`, the files are `quality.json`, `quality.sarif` and `quality-history.json`.

health.json

The top-level fields:

Field	Contents
`$schema`, `schemaVersion`, `kind`	`https://auricartisan.com/schema/health-v1.json`, `1`, `auric-artisan.health`
`tool`	Name, version and website of the tool
`knowledge`	Version, content hash and domains of the knowledge database used, and `actConformant: false`

Field	Contents
`generatedAt`, `workspace`	When the scan ran, and the workspace name
`git`	The branch and commit, read from the `.git` folder when there is one
`profile`, `thresholds`	The WCAG profile, the contrast ratio target and the APCA target
`scores`	`overall`, `grade`, `contrast`, `accessibility`, `codeHealth`
`summary`	Files, languages, colours, pairs, total issues, counts by source and severity, branding, categories, WCAG impacts
`languages`, `colors`	Files per language, and up to 64 colours with their use and file counts
`issues`	Every issue, worst first, up to `health.maxIssues`

Each issue has:

Field	Contents
`id`	An 8-character fingerprint that identifies the issue across runs
`source`	`accessibility`, `contrast`, `color-vision`, `branding` or `code-health`
`ruleId`	For example `img-alt`, `low-contrast`, `low-contrast-inherited`, `cvd-risk`, `branding-contrast`, `ch-secret-jwt`
`severity`	`critical`, `high`, `medium`, `low` or `info`
`file`, `line`, `col`	Workspace-relative path with forward slashes, and position
`message`, `fix`	What is wrong and how to fix it
Other fields	`wcag`, `knowledgeRuleId`, `detectorVersion`, `confidence`, `automation`, `category`, `data`, `excludedFromScore`, `redacted`

`wcag` holds the criterion, title, level and version. `data` holds the colours and ratios of a colour issue. `excludedFromScore` marks branding. `redacted` marks secrets.

How severities are assigned:

Finding	Severity
WCAG critical, serious, moderate, minor	critical, high, medium, low
Contrast failure	high (low when the background was assumed from the page)
Colour-vision risk	medium
Branding	info
Code health: merge markers	high
Code health: secrets, security, risky APIs	medium
Code health: debug leftovers	low
Code health: task markers, oversized	info

Secrets are never written: their evidence is replaced by a note such as `[redacted secret; 40 characters]`. See Secrets and redaction.

Fingerprints

The fingerprint is calculated from the issue's source, rule, file, line and a key detail (the colour pair for colour issues, the message or evidence otherwise). It stays the same from run to run while those stay the same, which is what makes baselines work. If a finding moves to another line, for example because you added lines above it, it gets a new fingerprint and counts as new against a baseline.

health.sarif

SARIF (Static Analysis Results Interchange Format) 2.1.0 is the format GitHub code scanning and many CI tools read. The file has one run whose tool is **Auric Artisan**, with:

- a **rules** list: each rule's ID, a short description (for WCAG, the criterion and its title), a help link (the W3C Understanding page for WCAG rules, auricartisan.com otherwise) and a default level;
- a **result** per issue: rule, level, message with the fix, file and position, a `partialFingerprints.auricArtisanId` fingerprint, and properties such as source, severity, confidence and whether evidence was redacted;
- the commit and branch, when known.

Levels: critical and high issues are `error`, medium issues are `warning`, low and info issues are `note`.

health-history.json

An append-only trend log. Each recorded scan adds one entry with its time, commit and branch, grade, scores, number of files, total issues and counts by source and severity. When the log exceeds `health.historyLimit` entries, the oldest are dropped. The bundled `.gitignore` excludes it; remove that line if you want to commit the history.

Studio Markdown exports

Both open a save dialog in your first workspace folder, then open the saved file.

Accessibility summary (accessibility-report.md)

From Studio › Home › **Report** when Auric Code Health is not installed. It contains the score and grade, files scanned, unique colours, contrast pair counts and harmony, the contrast distribution, the recommendations, up to 20 file hotspots, up to 500 failing pairs and up to 200 colour-vision risk pairs.

WCAG summary (wcag-report.md)

From Studio › Check › Findings › **Export WCAG**. It contains the accessibility sub-score and overall score, issue counts by level, impact and WCAG version, a table by success criterion, and every WCAG finding with criterion, impact, message and location. It ends with **Automated checks cover only part of WCAG; manual testing is still required.**

Related

- The CI gate, which writes the same `health.json` and SARIF from the command line.
- The remediation plan
- Palettes and shades and The brand audit for the colour exports.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The Icon Studio bridge

Open the combined Auric icon library from any Studio extension, and understand which icon extension it opens.

Every Studio extension has a shortcut to the **Icon Studio**, the combined icon library provided by Auric Artisan's icon extensions. The Studio itself contains no icons. It finds a compatible icon extension you have installed and opens that extension's library, which brings the Modern, Drawn and Playful collections together in one place.

Open the Icon Studio

- **Auric Artisan: Open Unified Icon Studio** in the Command Palette.
- **Icon Studio** in the Overview's navigation.
- **Icons** at the foot of the Studio's rail.
- **Open unified Icon Studio** in the Command Center. Its description tells you how many icon collections are installed and through which extension, or says **install an Auric Icons provider to enable the combined library**.

What you need

At least one Auric icon extension that provides the library, for example:

- Auric Icons Modern
- Auric Icons Drawn
- Auric Icons Playful

See also the Auric Artisan Icon Studio documentation for the library itself.

If none is installed, the command shows **No compatible Auric Icons provider is installed.**

Which extension opens

Icon extensions describe themselves to the Studio in their manifest. When several are installed, the Studio chooses the one with the newest icon library version (ties are broken by extension ID), starts only that one, and opens its library. That library then shows every compatible collection you have installed.

If the chosen extension cannot start or open its library, the Studio tries the next one in the same order. If none succeeds, you see **Auric Icon Studio could not be opened from the installed icon providers.**

The bridge only asks VS Code to start an icon extension and run its public command. It never reads another extension's files.

Related

- The Studio
- The Command Center

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The Overview view

Every card, row and button in the Overview sidebar view, in its accessibility, colour and code-health layouts.

The **Overview** is the first view in the Studio's Activity Bar container. It is a compact home that shows how the workspace is doing, what is wrong in the file you are editing, and what to do next. Its layout depends on which extension hosts it:

Layout	Used when the host is	What it centres on
Accessibility dashboard	Auric Accessibility (alone or in a Workstation)	The workspace score and one row per installed capability
Colour workbench	Auric Color (alone, or hosting Code Health)	The file you are editing, swatches and colour tools
Code-health report	Auric Code Health (alone)	The code-health score and its seven categories

Every layout opens with the **Auric Studio** heading (a link to https://auricartisan.com) and an **Open the Studio** button at the top right, and ends with a footer that shows `Ctrl Alt .` **Command Center** (`⌘ ⌥ .` on macOS).

The Overview follows **one gold action per view**: the most useful action is drawn in gold, everything else in the theme's plain button style. When the open file has contrast pairs that can be fixed automatically, **Fix N in this file** is gold; otherwise the workspace card's main button is.

The accessibility dashboard

Workspace card

- **Workspace** heading with your workspace's name, and how long ago it was scanned, for example `scanned 3 days ago` or `not scanned yet`. The age refreshes every minute.
- **Score ring.** The overall score out of 100 and its grade, for example `82` and `grade B`. Before the first scan it reads `–` and `no scan`.
- **Accessibility** row: the overall score, its grade and the number of WCAG issues, or `Run a scan to grade` before the first scan.
- **Contrast issues** row: the number of text pairs below target, followed by colour-vision and branding counts when there are any.
- **Code health** row (only when Auric Code Health is installed): the code-health score and number of findings.
- **Scan project** button. After the first scan it reads **Rescan** and forces a fresh read of the workspace.
- **Report** (only with Auric Code Health): runs **Generate Project Health Report**.
- **SARIF** (only with Auric Code Health): runs **Write Health Record (health.json + SARIF)**.

All rows update as you type, from the live re-grade described in Scanning. You do not need to press a button to see the effect of an edit.

This file

The **This file** card shows the file in the active editor:

- Its workspace-relative name and a count, for example `3 issues`, or `clean`.
- Up to four findings, worst first. Contrast rows lead with the ratio, for example `2.32:1 #999999 on #FFFFFF`; other rows name their kind (`WCAG`, `APCA`, `code health`, `color vision`). Each row shows its line number; select a row to jump to that line.
- **Fix N in this file**: recolours every automatically fixable contrast pair in the file (needs Auric Color). It runs **Fix All Contrast Issues in File**, which previews the changes and asks first.
- **Contrast Studio**: opens the file's first failing pair in the Contrast Studio, linked to its location.

A file with no findings shows **No findings in this file.** When no editor is open, the card is hidden.

Suggested

Up to three cards from the deterministic suggestion engine, each with a priority pill (**Critical**, **High**, **Workflow** or **Tip**), a title that states what was detected, a one-line reason, and one or two buttons. Select ✕ to dismiss a suggestion for good. The six rules are listed in The Command Center. The strip is hidden when there is nothing to suggest.

Recent colors

Shown when Auric Color is installed and you have applied at least one colour. It lists up to 12 recently applied colours; select a swatch to copy its value. **Picker** opens the colour picker.

Navigation

Button	What it opens
Studio	The Studio tab.
Rulebook	The rulebook and knowledge database search.
Icon Studio	The combined Auric icon library, if an icon provider is installed.
Fix plan	Generates the agent-ready remediation plan.

The footer reads **Runs on this machine**.

The colour workbench

Used when Auric Color is the host: on its own, or with Auric Code Health but without Auric Accessibility. It has no project score. (With that pair, open the Studio's **Health** space for the code-health score.)

- **This file** card, as above. **Fix N in this file** is the gold action when the file has fixable pairs.
- **Suggested**: only the file-level suggestion **N fixable contrast pairs in this file**, when it applies.
- **Brand palette**: a fixed row of nine swatches. Select one to copy it (**click to copy**).
- **Recent colors**: up to 12 recently applied colours, with a **Picker** link. Before you pick anything it says **Pick a color and it lands here.**
- **Open the color picker** (gold unless the file has fixable pairs): runs **Pick / edit color**.
- Navigation: **Contrast**, **Palette**, **Shades**, **Vision** (each opens that Studio tool), **Rulebook** and **Icon Studio**.
- Footer: **Hover a color literal to edit it in place**, or **Native decorator mode — VS Code's picker owns the hover** when you have switched to VS Code's own picker.

The code-health report

Used by Auric Code Health on its own.

- Workspace card with a **Code health** ring and row. Before the first report the row reads **Run a health report to grade this repo**.
- Seven category chips with counts: **Secrets**, **Security**, **Risky APIs**, **Debug**, **TODOs**, **Conflicts** and **Large files**. Chips with a count of zero are shown in a muted style.
- **Report** (gold): runs **Generate Project Health Report**, which scans (or reuses a fresh cache) and writes the report.
- **health.json + SARIF**: runs **Write Health Record**.
- **This file** and **Suggested**, as above.
- Navigation: **Studio**, **Rulebook**, **Icon Studio** and **Fix plan**.

Keyboard and screen readers

Every control is a real button. Score changes and new suggestions are announced through a polite live region, for example `Score 82 of 100, 42 findings`. See Accessibility of the Studio.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The remediation plan

Generate the agent-ready remediation plan, understand every part of it, and use it with a coding agent you already have.

Generate Agent-Ready Remediation Plan turns your last scan into a structured backlog of fixes: exact contrast repairs with computed colours, WCAG tasks, security and secret tasks, repeated patterns worth fixing once, a draft design-token system, a validation checklist and a prompt you can hand to a coding agent.

It is **100% deterministic and local**. The plan is computed from your scan by fixed rules. No AI model is called, nothing is sent over the network, and the same scan always produces the same plan. The only part intended for AI is the prompt text in the plan, a fixed template written for an external coding agent that you choose to use (for example Claude Code, Copilot or Cursor).

The command is provided by Auric Accessibility and Auric Code Health.

Generate the plan

1. Open a workspace folder.
2. Start the command in any of these ways:
   - **Auric Artisan: Generate Agent-Ready Remediation Plan** in the Command Palette;
   - **Fix plan** in the Overview's navigation;
   - **Fix plan for an agent** in the Studio's Home, Findings or Health space;
   - the **…** menu of the Project Issues view;
   - **Generate AI remediation plan** in the Command Center;
   - **Draft tokens** on the suggestion **N color values repeat 4+ times**.
3. The Studio uses a fresh scan cache if there is one, or scans first.
4. It writes `.auric-artisan/ai-remediation-plan.md` and `.auric-artisan/ai-remediation-plan.json`, and opens the Markdown file.
5. A message confirms, for example **Auric Artisan: agent-ready remediation plan created with 37 task(s) — generated locally from the scan, nothing sent to any external service.**, with **Open JSON**, **Open Data Folder** and **Open Studio**.

Generating again overwrites both files. They are excluded from source control by the folder's `.gitignore`.

What the Markdown plan contains

Section	Contents
Auric Artisan AI Remediation Plan	The heading and the time it was generated
Scan Summary	Score and grade, files scanned, failing contrast pairs, WCAG findings, code-health findings, colour-vision risk pairs, unique colours
AI Prompt	The prompt text for an external coding agent

Section	Contents
Phases	Five phases, each with a goal and actions
Backlog	A table of tasks: ID, priority, type, task, location and suggested fix
Design Token Draft	A summary and a CSS block of suggested tokens
Validation Checklist	Checks to tick off before calling the work done
Automation Notes	Which task IDs are low-risk to attempt automatically, which need review, and a warning

The phases

1. **Baseline and guardrails**: freeze the current report as the baseline before edits; work in small commits grouped by contrast, WCAG semantics and token migration.
2. **Automated contrast pass**: resolve the failing contrast pairs with the computed candidates, preferring component or token fixes when a colour repeats.
3. **Semantic WCAG pass**: fix labels, accessible names, keyboard handlers and ARIA relationships with product context, without inventing misleading alt text or labels.
4. **Design-system extraction**: turn repeated colours into accessible roles and shade scales.
5. **Verification loop**: rescan after each batch, run your tests, visual checks and manual keyboard and zoom checks, and stop if the score regresses.

The backlog

The backlog holds up to 80 tasks, at most 40 of each kind, sorted by priority (critical, high, medium, low).

Task ID	Type	Created from	Priority	Suggested fix
`AA-CONTRAST-001` …	contrast	Each failing contrast pair	critical below 3:1, otherwise high	Replace the colour with a computed candidate of the same hue, with its expected ratio and APCA value
`AA-WCAG-001` …	wcag	Each WCAG finding	From the finding's impact	A source edit following the rule's fix
`AA-CODEHEALTH-001` …	codehealth	Each secret, security or risky-API finding	Secrets critical (high if low confidence); security high; risky APIs high or medium	Secret remediation or a security review
`AA-PATTERN-001` …	pattern	A WCAG rule that fires three or more times (up to eight patterns)	high when any instance is critical or serious, otherwise medium	Fix once at the component or template level

Task markers, debug leftovers and oversized-code findings are not turned into tasks; they stay in the health record and the project health report.

Each task in the JSON file also carries evidence, acceptance criteria and an automation assessment. For example, a contrast task is marked safe to attempt with low risk ("The scanner has the exact foreground token location and a deterministic contrast repair candidate."), while tasks for labels, names, alt text, language, titles, keyboard handlers and accessible authentication are marked medium risk because the meaning needs review. Secret tasks are never marked safe to automate.

The design-token draft

The draft picks roles from your project's most used colours and makes sure the text roles pass your contrast target:

```
:root {
  --aa-color-bg: …;
  --aa-color-surface: …;
  --aa-color-text: …;
  --aa-color-text-muted: …;
  --aa-color-primary: …;
  --aa-color-on-primary: …;
  --aa-color-link: …;
  --aa-color-border: …;
  --aa-color-focus: …;
  --aa-primary-50: …;
  /* … through --aa-primary-950 */
}
```

The primary shade stops run from 50 to 950. The plan advises using these as a starting point, reviewing names against your own vocabulary, introducing tokens in one global place, replacing repeated raw colours with role tokens before tuning components, and rescanning after each batch.

The validation checklist

- Auric Artisan force rescan shows fewer failing contrast pairs.
- WCAG finding count drops or every remaining finding is documented.
- Secret, security and risky-API findings are resolved or explicitly reviewed; exposed credentials are rotated.
- Project tests and build still pass.
- Keyboard navigation, visible focus, 200% zoom and screen-reader names are manually spot checked.
- No broad token migration changed brand meaning without design review.

The prompt for your agent

The **AI Prompt** section is a fixed template. It tells an agent to treat the scan and plan as its source of truth, and sets rules such as: start with critical and high tasks; prefer shared component or token fixes; use a contrast candidate only if it still fits the visual role; never add generic alt text, labels or ARIA just to silence the scanner; never print or invent credentials; add a regression test around security fixes; rescan after each batch; and keep edits scoped. It ends with the current score, the number of files and the backlog size.

To use it, open the plan in your agent's workspace and ask the agent to follow the **AI Prompt** section, or paste that section into your agent's chat. The Studio does not connect to any agent itself.

The JSON plan

`ai-remediation-plan.json` holds the same plan in machine-readable form, with `"schema": "auric-artisan.ai-remediation-plan"` and `"version": 1`, and the fields `generatedAt`, `scan`, `goals`, `phases`, `backlog`, `automation`, `designSystem`, `validationChecklist` and `prompt`.

Related

- The Command Center
- Exports and health records
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Rulebook and knowledge database

Search the local rulebook and knowledge database, build knowledge.json and rulebook.md for your workspace, and add validated team guidance.

Every Studio extension ships a local, versioned **knowledge database**: a searchable collection of the rules the Studio checks, the standards they come from, manual test procedures, policies, articles, glossary terms and rulebooks. It works offline and is filtered to the capabilities you installed.

What is in it

Kind	What it is	Accessibility	Color	Code Health
Automated rule	One entry per rule the engine runs, with severity, confidence and remediation	72 WCAG rules	none	68 signatures
Requirement	Every WCAG 2.2 success criterion	86	none	none
Manual check	A step-by-step manual test procedure	8	none	none
Rule category	The seven code-health categories	none	none	7
Policy	A governance policy	3 brand	3 brand	4 code health
Article	A short practical guide	3 accessibility, 1 brand	1 brand	2 code health
Rulebook	A structured rulebook	accessibility and brand	brand	code health
Glossary	Term definitions	yes	yes	yes
Official source	Links to the standards the entries cite	14 shared sources	same	same

The counts are for version 0.1.0. With all three extensions installed, the database holds 283 entries.

Rulebooks

- **Accessibility engineering rulebook**
- **Accessible brand governance rulebook**
- **Code-health balanced baseline**

Manual checks

- Confirm text alternatives communicate the same purpose
- Review media alternatives for accuracy and synchronization
- Complete error recovery and authentication flows
- Verify focus order preserves meaning and operation
- Operate every workflow with a keyboard
- Test zoom, reflow, and text spacing without loss
- Confirm name, role, value, and state in an accessibility tree
- Verify important status changes are announced without moving focus

See Manual evidence for what each one asks you to do.

Policies

- Brand exceptions are narrow, owned, and temporary
- Brand classification cannot suppress structural findings
- Every brand token has provenance and a semantic role
- Report severity and confidence independently
- Never persist a detected secret value
- Each detector signature has a stable identity
- Suppressions are scoped and reviewable

Articles

- Debug accessible names from the computed result backward
- Use automation as evidence, not as a conformance verdict
- Audit keyboard and focus as complete user workflows
- Govern brand exceptions without creating an accessibility loophole
- Triage code-health findings with confidence and evidence
- Treat a committed secret as an incident, not a text-edit task

Official sources

WCAG 2.2, Understanding WCAG 2.2, Techniques for WCAG 2.2, WAI-ARIA 1.2, HTML Accessibility API Mappings 1.0, the ACT Rules Format 1.1, ACT Rules, OWASP Top 10:2025, the OWASP Application Security Verification Standard, the OWASP Cheat Sheet Series, the Common Weakness Enumeration and CWE-798 (hard-coded credentials), NIST's Secure Software Development Framework, and the Design Tokens Format Module.

ACT alignment

The accessibility entries follow the structure of the W3C's Accessibility Conformance Testing (ACT) rules format: they record what each check applies to, what it expects, how it is assessed, the requirements it maps to and where it comes from. This makes the evidence consistent and reviewable. It is **not** a claim of ACT conformance, and every exported file says so (`actConformant: false`).

Search the database

1. Run **Auric Artisan: Open Rulebook & Knowledge Database**, or select **Rulebook** on the Overview or at the foot of the Studio's rail, or **Open rulebook & knowledge database** in the Command Center.
2. A searchable list opens, titled **Auric Artisan Knowledge Base · 283 entries** (the number depends on what is installed). Rulebooks are listed first, then manual checks, automated rules, policies, articles, requirements, glossary entries, sources and rule categories.
3. Type to filter. The search matches titles, descriptions (kind, domain and ID) and summaries, so you can search for a rule ID such as `img-alt`, a criterion such as `1.4.3`, a term such as `ARIA`, or words such as `secret`.
4. Select an entry. It opens as a Markdown document with its summary, domain, kind, severity, confidence, requirements, remediation, applicability, procedure, pass and fail conditions, evidence to record, controls and official sources, whichever apply.

Selecting a rulebook opens the whole rulebook.

Rule details in an issue hover opens the entry for that finding's rule directly.

Build the workspace knowledge database

Auric Artisan: Build Workspace Knowledge Database (JSON + Markdown) writes the database into your workspace so it can be read by other tools, reviewed or shared.

1. Open a workspace folder and run the command (or **Build workspace knowledge database** in the Command Center).
2. The Studio writes:
   - `.auric-artisan/knowledge.json`: every entry, with the schema version, knowledge version, content hash, alignment, `actConformant: false`, domains and statistics;
   - `.auric-artisan/rulebook.md`: every installed rulebook in full, with a note that the database makes no ACT conformance claim;
   - `.auric-artisan/knowledge-overrides.json`: an empty template for team additions, created only if the file does not exist yet. An existing file is never overwritten.
3. `rulebook.md` opens. A message confirms, for example **Auric Artisan: built 283-entry knowledge.json and 3 rulebook(s) locally. A team override template was also created.**, with **Open JSON**, **Open Overrides** and **Open Data Folder**.

`knowledge.json` and `rulebook.md` are regenerated each time and are excluded from source control. `knowledge-overrides.json` is meant to be committed.

Add team guidance with knowledge-overrides.json

`knowledge-overrides.json` lets your team add reviewed guidance: manual checks, articles, policies, glossary entries and rulebooks. Added entries appear in search and in the generated files alongside the built-in ones. They can never replace a built-in entry.

The template looks like this:

```
{
  "schemaVersion": 1,
  "kind": "auric-artisan.knowledge-overlay",
  "entries": []
}
```

An example entry:

```
{
  "id": "custom:a11y:modal-focus-return",
  "kind": "manual-check",
  "domain": "a11y",
  "title": "Return focus when a modal closes",
  "summary": "After any modal closes, focus returns to the control that opened it.",
  "sourceIds": ["w3c-wcag22"],
  "tags": ["focus", "modal"]
}
```

Rules for entries

Field	Rule
`id`	Required. Must start with `custom:`, then 3 to 128 lowercase letters, digits, `:`, `.`, `_` or `-`. Must be unique and must not match a built-in ID.
`kind`	Required. One of `manual-check`, `article`, `policy`, `glossary`, `rulebook`.
`domain`	Required. One of `shared`, `a11y`, `code-health`, `brand`, `color` (also accepted: `accessibility`, `codehealth`, `health`, `branding`, `colour`).
`title`, `summary`	Required, non-empty, up to 12,000 characters each.
`sourceIds`	Optional. Must name built-in sources, for example `w3c-wcag22`, `w3c-aria12`, `owasp-top10-2025`.
`sections`	Required for a rulebook: a list of sections, each with a `title` and optional `entryIds` that must exist.

A policy with `"policyType": "brand-exception"` must also have `scope`, `rationale`, `owner`, `approvedAt` and `expiresAt` (ISO dates, expiry after approval and not already passed), a non-empty `allowedFindingKinds` limited to `contrast`, `color-vision` and `branding`, and non-empty `ruleIds` limited to colour rules (`low-contrast…`, `cvd-risk…`, `branding-…`, `contrast:…`). A brand exception can never cover a structural accessibility rule.

The whole file must be under 256 KB and hold at most 500 entries. Entries whose domain is not installed are kept in the file but not shown.

When the file is invalid

- **Searching** falls back to the built-in database and shows a warning that starts **Auric Artisan: workspace knowledge overrides are invalid; using the built-in database.**, followed by the problems found.
- **Building** stops and shows **Auric Artisan: could not build the knowledge database —** followed by every problem, one per line, such as `overlay.entries[0].id must start with custom: and use stable lowercase characters`.

Fix the listed problems and run the command again.

Related

- Manual evidence
- The accessibility rules
- Code-health categories and signatures
- Suppression and branding

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Scanning

How project scans, live re-grading and background scans work, what is read, how the cache and .auricignore behave, and what the .auric-artisan folder contains.

The Studio checks your code in three ways. Knowing which is which explains when numbers change and when files are written.

Kind	What it covers	When it runs	Writes files?
Live diagnostics	The files open in editors	As you type, about 300 ms after you stop	No
Live re-grade	The whole project score, from files held in memory	As you type, after `scan.liveDelayMs` (700 ms by default)	No
Project scan	Every matching file in the workspace	When you ask, or automatically if you turn on auto scan	Yes: the scan cache, and health records if enabled

Live diagnostics

Underlines, hovers and Problems entries for open files update as you type. They are always on and do not depend on any scan setting. What is checked depends on the installed extensions and their switches: `contrast.enabled`, `a11y.enabled`, `codeHealth.enabled` and `codeHealth.diagnostics`. Files larger than 1 MB are not analysed in the editor.

Project scans

A project scan reads the workspace, grades it and saves the result.

Start a scan

How	Uses a fresh cache?
Scan project on the Overview, or **Auric Artisan: Scan Project for Accessibility (Use Cache)**	Yes
Rescan on the Overview or Studio, **Force rescan** in Studio Settings, or **Auric Artisan: Force Rescan Project and Refresh Cache**	No, always re-reads
Scan Code Health or **Rescan** in the Studio's Health space	No
Generate Project Health Report, **Write Health Record**, **Generate Agent-Ready Remediation Plan**	Yes; they scan first if there is no fresh cache

A scan started from a command shows a progress notification, **Auric Artisan: scanning project…**, with a **Cancel** button. If a scan fails, you see an error and your previous results are left untouched.

What a scan produces

- The overall score and grade, the contrast and accessibility sub-scores, and the code-health score. See Scores and grades.
- Every contrast pair the scanner could resolve, the failing pairs, and pairs at colour-vision risk.
- WCAG findings from markup and CSS files, under the audit profile you chose.
- Code-health findings from every supported file.
- Branding classifications, the project palette (every colour and how often it appears), a harmony analysis and pairs of hues that become confusable under colour-vision deficiency.
- A ranked list of files to fix first and up to six recommendations.

The result appears in the Overview, the status bar, the Project Issues view, the Studio and the Command Center.

What is read

The scan reads files with these extensions. Stylesheets, markup and script files get the contrast and WCAG checks; every file gets the code-health checks.

Group	Extensions
Stylesheets	`.css` `.scss` `.sass` `.less` `.styl` `.stylus` `.pcss` `.postcss`
Markup and components	`.html` `.htm` `.vue` `.svelte` `.astro` `.php` `.xml` `.md` `.markdown`
Scripts and data	`.js` `.cjs` `.mjs` `.jsx` `.ts` `.mts` `.cts` `.tsx` `.json` `.jsonc` `.json5`
Python, Go, Rust, Java, C#	`.py` `.pyw` `.pyi` `.go` `.rs` `.java` `.cs`
C, C++, Objective-C	`.c` `.h` `.cpp` `.cc` `.cxx` `.hpp` `.hh` `.m` `.mm`
Kotlin, Swift, Dart, Ruby, Lua, R, Julia	`.kt` `.kts` `.swift` `.dart` `.rb` `.lua` `.r` `.jl`
Scala, Groovy, Perl, Haskell	`.scala` `.groovy` `.gradle` `.pl` `.pm` `.hs`
Elixir, Erlang, Clojure, F#, Visual Basic	`.ex` `.exs` `.erl` `.clj` `.cljs` `.fs` `.fsx` `.vb`
Shell and query	`.sh` `.bash` `.zsh` `.ps1` `.psm1` `.coffee` `.sol` `.graphql` `.gql` `.sql`
Config and templates	`.yaml` `.yml` `.toml` `.ini` `.cfg` `.conf` `.properties` `.env` `.hbs` `.handlebars` `.pug` `.jade` `.ejs` `.twig` `.razor`

It skips:

- anything matched by `scan.exclude` (by default `**/{node_modules,dist,build,out,.git,vendor,coverage,.auric-artisan}/**`);
- anything matched by a `.auricignore` file (below);
- the `.auric-artisan` data folder;
- files larger than 1 MB, and files that cannot be read.

If no matching files are found, the scan falls back to the stylesheets and markup files that are open in editors.

> **Note:** A file named exactly `.env` (with nothing before the dot) is not read by the in-editor project scan, although files such as `production.env` are. The command-line scanner reads both.

The file limit

`scan.maxFiles` (default 4000) caps how many files one scan reads. When the cap cuts a scan short, the status-bar tooltip says so, for example `⚠ Scanned 4000 of 5230 files (capped at 4000 …)`, and the Command Center suggests **Scanned X of Y files**. Raise the limit in Studio › Settings › **Scope**, or narrow the scan with `scan.exclude` or `.auricignore`.

The scan cache

Each completed scan is saved to `.auric-artisan/scan-cache.json`. The next **Scan project** reuses it instead of reading every file again, as long as it is fresh.

- **Freshness.** `scan.cache.maxAgeMinutes` (default 1440, which is 24 hours). A value of `0` keeps the cache until you force a rescan or an automatic scan refreshes it.

- **At startup** the Studio loads a fresh cache so the score is there straight away. The status-bar score shows a database icon when it came from the cache.
- **Audit profile.** If you change the WCAG audit profile, Auric Accessibility treats a cache made under another profile as stale and scans again.
- **Turning it off.** Clear **Use cached scan data** in Studio Settings (`scan.cache.enabled`), and every scan reads the workspace.
- **Clearing it.** Run **Auric Artisan: Clear Workspace Scan Cache**, or **Clear cache** in Studio Settings. The file is deleted and the score resets until the next scan.

Live re-grade

With `scan.live` on (the default), the project score, the Project Issues view and an open Studio keep up with your edits without a scan. The Studio keeps the files from the last scan in memory and swaps in the text of the file you are editing. It never reads the disk on a keystroke and never writes the cache.

- The first edit after VS Code starts reads the workspace once to warm up.
- Only files that were part of the scan are re-graded. New or ignored files are skipped until the next scan.
- Very large workspaces, over 1,500 files or about 12 MB of text, are not re-graded live. Run a scan, or turn on automatic scans, to refresh them.
- `scan.liveDelayMs` sets how long the Studio waits after your last keystroke (200–5000 ms, default 700).

When several Studio extensions are installed, the one that owns the status-bar score runs the live re-grade. The others re-grade only while their own Studio tab is open.

Automatic background scans

With `scan.autoScan` on (off by default), the Studio runs a full scan, and rewrites the cache, after files are created, changed, deleted or saved. It waits `scan.autoScanDelayMs` (default 1800 ms, range 250–60000) after the last change. With `scan.autoScanOnStartup` also on, it scans shortly after the workspace opens. When a background scan finishes, the status bar briefly shows **Auric Artisan cache updated**. Background scans never interrupt you with errors.

When several Studio extensions are installed, only the one that owns the status-bar score runs automatic scans, so the shared cache has exactly one automatic writer. Scans you start yourself are never restricted.

.auricignore

Put a file named `.auricignore` at the root of a workspace folder to keep files and folders out of scans. It uses a subset of `.gitignore` syntax:

```
# comments and blank lines are ignored
generated/                 # a folder
legacy-theme.css           # an exact path
*.min.js                   # * and ? wildcards
**/__snapshots__/          # **/ matches at any depth, including the root
docs/**/*.html             # ** between folders
!generated/keep-this.css   # ! re-includes something an earlier line excluded
/anchored/path             # a leading / anchors to the folder root
```

Later lines win, so a `!` line can bring back a file an earlier pattern excluded. The common cases match `.gitignore`, but not every edge case of Git's rules is reproduced. `.auricignore` applies on top of `scan.exclude`. In a multi-root workspace each

folder can have its own `.auricignore`. The command-line scanner reads it too.

The .auric-artisan folder

The Studio keeps its workspace data in one folder, `.auric-artisan` by default, at the root of the workspace. In a multi-root workspace it is created in the first folder. All installed Studio extensions share it.

File	What it is
`scan-cache.json`	The last scan, reused while fresh.
`settings.json`	A copy of the Studio's scan, data and picker settings, kept up to date as the Studio starts, scans and saves settings.
`health.json`	The fingerprinted health record. See Exports and health records.
`health.sarif`	The same issues in SARIF 2.1.0.
`health-history.json`	A capped trend log, one entry per recorded scan.
`project-health.md`, `.json`, `.html`	The project health report.
`ai-remediation-plan.md`, `.json`	The agent-ready remediation plan.
`knowledge.json`, `rulebook.md`	The generated knowledge database and rulebooks.
`knowledge-overrides.json`	Your team's validated additions to the knowledge database.
`README.md`	A short note explaining the folder, written the first time it is created.
`.gitignore`	Keeps the regenerated files out of source control.

The bundled `.gitignore` excludes `scan-cache.json`, the remediation plan, the project health report, `health.json`, `health.sarif`, `health-history.json`, `knowledge.json`, `rulebook.md` and temporary files. It leaves `settings.json`, `knowledge-overrides.json`, `README.md` and `.gitignore` so you can commit team defaults and reviewed guidance. Edit it if you want to commit the health history.

You can delete the whole folder at any time; the Studio recreates it on the next scan or settings save. To rename it, set `data.folderName`. To stop the Studio writing workspace data at all, turn off `data.enabled` (Studio Settings › **Store workspace data locally**); scans then run without a cache.

What static scanning can and cannot see

The scanner reads source text. It does not run your code or render pages. It finds colour pairs and markup problems it can resolve from the text: CSS rules, `<style>` blocks, inline `style=` attributes, JSX style objects (`style={{…}}`, `sx={{…}}`, `css={{…}}`), styled-components, Emotion and Linaria templates, and Vue, Angular and Svelte style bindings. It cannot see colours set by JavaScript at runtime, class-driven styles applied from another file, the full cascade between different selectors, or component state. See Limits and accuracy for the details, and Contrast diagnostics for how contrast pairs are resolved.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Scores and grades

How the overall, contrast, accessibility and code-health scores and their letter grades are calculated, with a worked example.

Every score in the Studio is a number from 0 to 100, where 100 means nothing was found. The formulas are fixed, so two projects with the same score are comparable on what was scanned. Scores are a way to prioritise work and track progress. They are **not** an accessibility certification: static checks cover only part of WCAG, and manual testing is still needed.

The letter grade

Grade	Score
A+	95 or more
A	90 to 94
B	80 to 89
C	70 to 79
D	55 to 69
F	below 55

The same scale is used for the overall score, the code-health score and the brand audit.

The overall score

The overall score is the one shown in the status bar, the Overview ring and the Studio's Home.

- When the project has both resolvable contrast pairs and markup or CSS files, the overall score is the average of the **contrast** and **accessibility** sub-scores, rounded.
- When only one of them applies, the overall score is that one.
- When neither applies, there is no score and the grade shows `—`.

Code health is scored separately and does not change the overall score.

The contrast sub-score

Every contrast pair the scanner resolved earns credit, measured against its own target (3:1 for large text and 4.5:1 otherwise, or 4.5:1 and 7:1 at the AAA threshold):

The pair…	Credit
meets AAA for its text size	1.00
meets the selected target	0.95
is below the target but at least 3:1	0.55
is below 3:1	ratio ÷ 3 × 0.35

A pair graded against an **assumed page background** (the rule sets only `color:` and the background was taken from `body`, `html` or `:root`) counts half, because it is an inference rather than a reading. Every other pair counts fully.

The contrast sub-score is the weighted average credit × 100, minus a **colour-vision penalty**: the share of pairs that pass for typical vision but weaken under colour-vision deficiency, × 25, capped at 10 points. Pairs classified as intentional branding are left out entirely. If there are no resolvable pairs, the contrast sub-score is omitted.

The accessibility (WCAG) sub-score

Each WCAG finding adds a penalty:

Factor	Values
Impact	critical 6, serious 4, moderate 2, minor 0.7
Confidence	high × 1, medium × 0.65, low × 0.35
Automation	automated × 1, assisted × 0.55, manual × 0
Best-practice hint	× 0.5

The penalties are added up and divided by the square root of the number of markup and CSS files scanned, so a large project is not punished merely for its size. The sub-score is 100 minus that result, never below 0. A project with no findings scores 100. If no markup or CSS files were scanned, the sub-score is omitted.

Every finding counts, including repeats of the same success criterion. Suppressed findings are removed before scoring. Branding marks never remove WCAG findings.

The code-health score

Each code-health finding adds a penalty by category, scaled by confidence:

Category	Weight
Possible secrets	14
Security risks	12
Merge markers	10
Risky APIs	4
Debug leftovers	2
Oversized	0.5
Task markers	0.25

Confidence	Multiplier
high	1
medium	0.75
low	0.4

The total is divided by the number of files scanned, multiplied by 16, and subtracted from 100 (never below 0). The code-health score uses the same letter grades.

A worked example

A project has 10 contrast pairs and 5 markup files.

1. Contrast: 6 pairs meet AAA (6 × 1.00), 2 meet AA (2 × 0.95), 1 is 3.2:1 (0.55) and 1 is 2.1:1 (2.1 ÷ 3 × 0.35 = 0.245). The average credit is 8.695 ÷ 10, so 86.95. One pair of the ten is a colour-vision risk: 0.1 × 25 = 2.5 points off. The contrast

sub-score is 84.

2. Accessibility: one serious, high-confidence, automated finding (4), two minor findings (0.7 each) and one best-practice hint (0.7 × 0.35 × 0.55 × 0.5, about 0.07). The total is about 5.47. Divided by √5 (about 2.24) that is about 2.45, so the sub-score is 98.
3. Overall: (84 + 98) ÷ 2 = 91, grade **A**.

Colours used for scores

- **Status bar:** the score item uses the theme's warning background below 90 and its error background below 70.
- **Studio:** scores of 90 and above use the theme's pass colour, 70 to 89 the warning colour, and below 70 the fail colour. Every score is also written as a number and grade, so colour is never the only signal.

Priority in hovers

Issue hovers show **Priority N/100** with a segmented meter and one of four pieces of advice: **fix now** (78 or more), **fix soon** (55 to 77), **worth fixing** (30 to 54) or **low urgency** (below 30). Priority is not part of any score. It starts from the finding's impact (critical 94, serious 78, moderate 56, minor 32), is scaled by confidence, and for contrast findings by how far the pair falls short of its target. Branding findings always show a low priority because they are outside the score.

Related

- Scanning
- Suppression and branding
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Accessibility of the Studio

Use the Studio's own screens with a keyboard, a screen reader, zoom, reduced motion and high-contrast themes.

An accessibility tool should be accessible itself. The Studio's own screens (the Overview, the Studio tab and the colour pickers) are built to meet WCAG 2.2 level AA. This page describes how to use them without a mouse, with a screen reader, and with high-contrast, zoom and reduced-motion settings. If you meet a barrier, it is a bug: report it through https://auricartisan.com.

Keyboard

Every control can be reached and operated from the keyboard.

Where	Keys	What they do
Anywhere in a Studio screen	`Tab`, `Shift` + `Tab`	Move between controls
Studio rail	Arrow keys, `Home`, `End`	Move between spaces; the space opens as you move
A space's tab row (Color, Check)	Arrow keys, `Home`, `End`	Move between tools
Findings and Health tools	`/`	Jump to the search box and select its text

Where	Keys	What they do
Findings and Health tools	`Escape`	Close an open picker, menu or tooltip; if none is open, clear the search
Swatches, finding rows, file rows and other tiles	`Enter`, `Space`	Activate, as a click would
Dropdown menus	`Down` or `Up` on the button, then arrow keys, `Home`, `End`; `Escape`	Open the menu, move through options; close and return to the button
Accent swatches (Settings › Appearance)	Arrow keys, `Home`, `End`	Move and select; on the custom swatch, `Enter` opens the picker
Colour picker hue and opacity bars	Arrow keys (`Shift` for steps of 10), `Page Up`, `Page Down`, `Home`, `End`	Change the value
Colour picker spectrum	`Left`, `Right` for saturation; `Up`, `Down` for brightness; `Shift` for steps of 10; `Home`, `End`	Change the colour
Studio colour picker pop-up	`Tab`	Stays inside the picker while it is open
Studio colour picker pop-up	`Escape` or **Done**	Close it and return focus to the swatch that opened it
Standalone colour picker beside the editor	`Escape`	Close it

Keyboard changes in a picker are written to your file after you pause for about 0.6 seconds, so holding an arrow key does not create hundreds of edits.

VS Code's own keyboard support applies to everything VS Code draws: hovers (`Ctrl` + `K`, `Ctrl` + `I` shows the hover at the cursor), quick fixes (`Ctrl` + `.`), the Command Center list and the Project Issues tree.

Focus

- Every interactive element shows a visible focus ring, in every theme.
- When a Studio screen refreshes (a scan result arrives, a filter changes, you switch tools), keyboard focus and text selection stay where they were.
- Closing a picker or menu returns focus to the control that opened it.
- `Escape` dismisses a tooltip without moving focus.

Screen readers

- Controls have accessible names, and dropdowns report whether they are expanded and which option is selected.
- The rail and tab rows are real tab lists; the colour bars are real sliders that announce values such as `Hue 210 degrees` and `85 percent opaque`.
- Tooltips are attached to their control while visible, so their text is read as a description.
- Changes are announced politely without moving focus: tab changes (for example **Palette tab**), **Search focused**, **Search cleared**, new suggestions, updated scores (for example `Score 82 of 100, 42 findings`) and the results of actions such as copying a colour.
- The decorative before-and-after mock-ups in the Preview tool that deliberately show failing contrast are hidden from screen readers; the numeric verdicts next to them are read instead.
- In VS Code's own views, rows in the Project Issues tree and the **Auric: N in this file** status-bar item have full spoken labels, for example `serious WCAG 1.1.1 Non-text Content in src/index.html line 12`.

Colour and contrast

- Every text colour and status colour in the dark and paper themes has been checked against the surfaces it sits on to hold at least 4.5:1.
- Pass, warning and fail colours are defined per theme and checked to hold 4.5:1.
- A custom accent colour is adjusted until its text reaches 4.5:1 on every surface, including its own tint. See Appearance.
- Colour is never the only signal. Verdicts carry ✓ or ✕ and the words **Pass** or **Fail**; scores carry a number and a grade; severity carries a label; the Branding group carries a heart icon, the word **branding** and a border.

High contrast and forced colours

- In VS Code's high-contrast themes (dark and light), the Studio uses the theme's own colours and borders instead of its own palette.
- When your operating system forces colours, for example Windows contrast themes, the Studio's screens follow the forced colours.

Motion and zoom

- When your system asks for reduced motion, the Studio turns off its transitions and ambient animation.
- Interactive targets are at least 24 by 24 CSS pixels.
- The Studio reflows from a narrow sidebar to a full-width editor tab, and works with VS Code's zoom (`Ctrl` + `=` and `Ctrl` + `-`).

Related

- The Studio
- The Overview view
- Appearance

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The Studio

Open the Studio tab, move between its spaces, and use the Home and Settings spaces.

The **Studio** is an editor tab that holds the Studio's larger tools: the workspace score and its details, the full list of findings, the Contrast Studio, colour generators, the brand audit, code health and settings. It is laid out as a **rail** of spaces on the left and the selected space's content on the right.

Open the Studio

Any of these opens it:

- **Studio** in the Overview, or the arrow button at the top right of the Overview.
- The status-bar score item.
- **Auric Artisan: Open Color Studio**, **Auric Artisan: Open Accessibility Studio** or **Auric Artisan: Open Code Health Studio** from the Command Palette.
- An **Open Studio: …** entry in the Command Center.

- Links in hovers and quick fixes, such as **Open in Contrast Studio**, **Tune the pair** or **Open in Preview**. These open the Studio on the matching tool.
- **Generate Accessible Palette** or **Check Contrast of Selection**, which open the Palette or Contrast tool on the selected colour.

The tab title shows which Studio you have: **Auric Artisan — Color Studio**, **Auric Artisan — Accessibility Studio**, **Auric Artisan — Code Health Studio**, or **Auric Artisan — Workstation Studio** when the host covers more than one installed extension. Only one Studio tab exists at a time; opening it again brings the existing tab forward.

The rail

The rail lists the spaces this install carries. Spaces with nothing to show for your install are left out.

Space	Tools	Provided by	Documented in
Home	Workspace score, files to fix first, graphs, suggestions, CI gate, project palette	Auric Accessibility	Home space below
Color	**Palette**, **Shades**, **Vision**	Auric Color	Palettes and shades, Colour vision
Check	**Findings**, **Contrast**, **Preview**	Findings and Preview: Auric Accessibility; Contrast: Auric Accessibility or Auric Color	The Check space: Findings and Preview, The Contrast Studio
Health	Code-health score, categories and findings	Auric Code Health	The Code Health Studio
Brand	Brand accessibility audit	Auric Color	The brand audit
Settings	Appearance and settings for installed capabilities	All	Settings space below

At the foot of the rail:

- **Rulebook** opens the rulebook and knowledge database search. See Rulebook and knowledge database.
- **Icons** opens the combined Auric icon library. See The Icon Studio bridge.
- The Auric Artisan mark at the top of the rail links to https://auricartisan.com.

The Studio opens on the first space in the rail: **Home** when Auric Accessibility is installed, **Color** for Auric Color alone, and **Health** for Auric Code Health alone. Within a session, each space remembers the tool you last used in it.

When the Studio tab is narrow, the rail turns horizontal above the content.

The space header

Every space opens with a header:

- A **kicker** (small label) naming the space and tool, for example **Check · Accessibility findings**.
- A **title**. On Home it is your workspace's name; in the Contrast Studio it is **Contrast checker**, or **Text on** followed by the rule or element when a pair is linked to a file.
- A one-line **summary** of what the tool is for.
- A **scan chip** on Home, Findings and Health showing the workspace and how long ago it was scanned, for example `my-app · scanned 12 min ago`.
- The tool's **actions**. The primary action is gold; the others are plain.
- A **tab row** when the space holds more than one tool (Color and Check). The Findings tab shows a count.

A thin status line under the header reads **Scanning the workspace…** or **Refreshing the workspace scan…** while a scan runs. When results arrive, a toast reports them, for example `Scan complete · 82/100 · fresh+cached`.

The footer reads **Crafted by Auric Artisan. Everything here runs on this machine.** with a link to auricartisan.com.

The Home space

Before the first scan

The header shows **Scan project**. The body explains what a scan does and offers **Fix plan for an agent** (which scans first) and, when Auric Color is installed, **Audit a brand by hand** →.

After a scan

Header actions:

Button	What it does
Report	With Auric Code Health: generates the project health report. Without it: saves an accessibility report as Markdown (`accessibility-report.md`) through a save dialog.
Rescan (gold)	Re-reads the workspace and refreshes the scan cache.

Left column:

- **Score card.** A ring with the overall score and grade, and tiles for **Accessibility** (the WCAG sub-score and number of findings), **Contrast** (text pairs below target, or **every text pair passes**) and, with Auric Code Health, **Code health** (score, findings and number of secrets).
- **Files to fix first.** Up to eight files ranked by critical and serious findings, then contrast failures, then total issues, with columns **File**, **Contrast**, **WCAG** and **Color vision**. Select a file name to open it at its first finding. The card shows how many files were scanned.
- **Graphs & details** (collapsed at first): **Score split** (Overall, Contrast, WCAG a11y), **Contrast distribution** (AAA, AA, Large/UI only, Fail, CVD risk) and **WCAG impact distribution** (Critical, Serious, Moderate, Minor), each with counts and percentages.
- **Harmony & temperature** (collapsed at first): the detected colour scheme of your palette, the number of hue families with a confidence percentage, a chip for each hue, and a warm and cool balance bar.

Right column:

- **Suggested.** Up to three recommendations from the scan, each with a priority pill (**high**, **medium**, **low** or **good**), a title, detail and the recommended action. Examples: **Fix the lowest contrast text pairs first**, **Clear critical and serious WCAG findings**, **Reduce color-vision-dependent distinctions**, **Fix repeated 1.1.1 patterns**, **Consolidate repeated color literals into tokens**, or **No urgent automated findings**.
- **CI gate** (with Auric Code Health): the command to fail a pull request only on new issues, a **Copy command** button and **Write health.json + SARIF**. See The CI gate.
- **Project palette · N**: up to 14 of the most used colours. Select a swatch to copy it. **Audit as a brand** sends the palette to the Brand space.

Collapsible sections remember whether you left them open or closed, across sessions.

The Settings space

The Settings space shows only controls for installed capabilities. Its header reads **Your Settings — only controls for installed capabilities** and offers **Open data folder**, **Clear cache** and **Force rescan** when a scanning extension is installed.

Appearance

The **Appearance** card changes the Studio's accent colour and saves immediately; there is no Save button for it. See Appearance.

The other cards

Card and control	Values	Setting written
Project data › **Store workspace data locally**	checkbox	`data.enabled`
Project data › **Data folder**	folder name; the full path is shown below	`data.folderName`
Scan engine › **Continuous scan as you type**	checkbox	`scan.live`
Scan engine › **Use cached scan data**	checkbox	`scan.cache.enabled`
Scan engine › **Auto scan after file changes**	checkbox	`scan.autoScan`
Scan engine › **Auto scan on startup**	checkbox	`scan.autoScanOnStartup`
Scan engine › **Live scan delay (ms)**	200–5000	`scan.liveDelayMs`
Scan engine › **Cache max age (minutes)**	0 or more	`scan.cache.maxAgeMinutes`
Scan engine › **Auto scan delay (ms)**	250–60000	`scan.autoScanDelayMs`
Scope › **Maximum files**	1 or more	`scan.maxFiles`
Scope › **Exclude glob**	a glob pattern	`scan.exclude`
Accessibility audit › **Audit profile**	WCAG 2.0, WCAG 2.1, WCAG 2.2 (default), WCAG 3.0 draft (+APCA)	`a11y.profile`
Contrast › **Contrast standard**	WCAG 2.x ratios, APCA (WCAG 3 draft), WCAG 2.x + APCA	`contrast.standard`
Contrast › **WCAG 2.x ratio target**	1–21, in steps of 0.5	`contrast.threshold`
Contrast › **APCA min Lc**	15–106	`contrast.apcaThreshold`
Color picker › click action	Inline hover picker, Spectrum picker panel, Click disabled	`colorPicker.clickAction`

The **Scan engine** card also shows whether the cache is ready, how old it is and when it was saved.

Project data, Scan engine and Scope appear when Auric Accessibility or Auric Code Health is installed; Accessibility audit with Auric Accessibility; Contrast with Auric Accessibility or Auric Color; Color picker with Auric Color.

Select **Save settings** at the bottom to write your changes. They are saved to the workspace's VS Code settings (or your user settings when no folder is open) and mirrored into `.auric-artisan/settings.json`. Each value is written to the extension that owns it. The contrast card reminds you to **Force a rescan after changing**, because saved scans keep the old thresholds until you rescan.

Keyboard use

- `Tab` moves through controls; arrow keys, `Home` and `End` move between spaces on the rail and between tools in a tab row.
- `/` jumps to the search box on the Findings and Health tools; `Escape` closes an open picker, menu or tooltip first, then clears that search.
- `Enter` or `Space` activates swatches, finding rows and other clickable tiles.

See Accessibility of the Studio for the full list.

Related

- The Contrast Studio
- The Overview view
- Scanning

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Suppression and branding

Silence a false positive with auric-disable comments, or classify a deliberate brand colour with auric-brand comments, by hand or with quick fixes.

Two families of comment directives let you tell the Studio about decisions you have made:

- **Suppression** (`auric-disable-…`) removes a WCAG or code-health finding from the results and the score. Use it for false positives or rules you have consciously decided not to follow in one place.
- **Branding** (`auric-brand-…`) re-classifies a colour finding as intentional brand identity. The finding stays visible in its own **Branding** group but leaves the score. Use it for deliberate brand colours, such as a logo lock-up or a decorative accent.

A **directive** is a special comment the Studio reads, written in the comment syntax of the file's language.

Suppression directives

Directive	Silences
`auric-disable-line`	Findings on the same line
`auric-disable-next-line`	Findings on the following line
`auric-disable-file`	Findings anywhere in the file

Each directive can be followed by a **scope**: one or more tokens separated by commas or spaces, optionally after a colon. Without a scope, a directive silences every WCAG and code-health finding it covers.

Scopes for WCAG findings

Token	Example	Silences
A rule ID	`auric-disable-line img-alt`	That rule only
A success criterion	`auric-disable-line 1.1.1`	Every rule mapped to that criterion
`a11y`	`auric-disable-file a11y`	All WCAG findings

Token	Example	Silences
`all`	`auric-disable-line all`	All WCAG and code-health findings

Scopes for code-health findings

Token	Example	Silences
A category	`auric-disable-line secret,security`	Findings in those categories: `secret`, `security`, `risk`, `debug`, `todo`, `conflict`, `size`
A rule ID	`auric-disable-line ch-risk-eval`	That signature only (the ID shown on the diagnostic)
`codehealth`	`auric-disable-file codehealth`	All code-health findings
`all`	`auric-disable-line all`	All WCAG and code-health findings

Whatever rule ID or category the editor shows on a finding can be pasted into a directive and will work.

Examples

```
<!-- auric-disable-next-line img-alt -->
<img src="spacer.gif">
```

```
/* auric-disable-file a11y */
```

```
cache_key = hashlib.md5(data).hexdigest()  # auric-disable-line security
```

```
// auric-disable-next-line ch-risk-eval
const result = eval(trustedTemplate);
```

In JSX markup, put the directive in braces so it is not rendered as text: `{/* auric-disable-next-line img-alt */}`.

Rules the Studio applies

- **Code-health directives must be inside a real comment.** A directive that is only mentioned in prose or a string literal never silences anything. In Markdown, directives shown inside fenced code blocks are treated as examples and ignored.
- **Suppression does not apply to contrast.** Contrast, APCA and colour-vision findings are not silenced by `auric-disable-…` directives. To take a deliberate colour pair out of the score, mark it as branding (below), or change the colours.
- **Suppressed findings disappear** from the Problems panel, the Project Issues view, the Studio, reports, `health.json`, SARIF and the score.

Suppress with a quick fix or hover

You rarely need to type a directive. The Studio writes one for you in the right comment syntax:

Where	Action	What it inserts
Quick fix on a WCAG finding (`Ctrl` + `.`)	**Ignore this WCAG 1.1.1 finding here**	`auric-disable-next-line 1.1.1` on the line above, with the line's indentation

Where	Action	What it inserts
Quick fix on a WCAG finding	**Ignore all accessibility findings in this file**	`auric-disable-file a11y` at the top of the file
Quick fix on a code-health finding	**Ignore this secret finding here** (the category varies)	`auric-disable-next-line secret` on the line above
Quick fix on a code-health finding	**Ignore all secret findings in this file**	`auric-disable-file secret` at the top of the file
Ignore here in an issue hover		`auric-disable-next-line` followed by the rule ID, on the line above

After **Ignore here**, the status bar confirms, for example `Auric Artisan: img-alt ignored on this line`.

Comment syntax used

Languages	Comment written
JavaScript, TypeScript, JSX, TSX, JSON, Go, Rust, C, C++, C#, Java, Swift, Kotlin, Dart, PHP, Scala, Groovy, SCSS, Less	`// …`
Python, Ruby, YAML, TOML, shell, Perl, R, Elixir, Dockerfile, Makefile	`# …`
SQL, Lua, Haskell	`-- …`
HTML, XML, Vue, Svelte, Astro, Markdown, Handlebars	`<!-- … -->`
CSS, PostCSS, Sass, Stylus, and any other language	`/* … */`

> **Note:** Strict JSON does not allow comments. A directive inserted into a `.json` file makes it invalid for most JSON parsers. For JSON files, prefer `.auricignore` or leave the finding and review it.

Branding directives

Directive	Covers
`auric-brand-line`	Colour findings on the same line
`auric-brand-next-line`	Colour findings on the following line
`auric-brand-file`	Colour findings anywhere in the file

Optional scope tokens narrow the claim: `contrast`, `apca` or `cvd` (colour-vision risk). `color` or `all`, or no scope at all, cover every colour kind. Example: `/* auric-brand-line: contrast,cvd */`.

```
.logo-mark { color: #d3af37; background: #fff; }  /* auric-brand-line */
/* auric-brand-next-line */
.hero { color: var(--brand-gold); background: var(--brand-ink); }
```

What branding changes

- In the editor, the finding becomes an information-level diagnostic starting **Branding —**, and its hover is titled **Intentional branding**.
- In the Project Issues view it moves to a **Branding** group marked **excluded from score**, with a heart icon. In the Studio's Findings tool it moves to a **Branding** section.

- It is left out of the contrast sub-score, the failing-pair lists and the colour-vision penalty.
- In `health.json` and SARIF it is still recorded, as an informational issue from the source `branding`, marked as excluded from the score. The exclusion is always visible, so a reviewer can audit every excluded pair.

Every branding explanation carries the same reminder: **Classified as intentional branding — shown separately and excluded from the accessibility score. Brand colors still need a readable pairing wherever they carry text or UI meaning.**

Branding is not a loophole

Only colour findings (contrast, APCA and colour-vision risk) can be classified as branding. Structural WCAG findings, such as a logo image without alt text or an unlabelled button, keep counting on a marked line. Their message adds: **(A branding mark covers this line, but structural accessibility issues are not brandable — the finding still counts.)** The Findings tool reports how many such marks were ignored. Code-health findings, including secrets, are never brandable.

Mark a line with a command

1. Put the cursor on the line (or one cursor on each of several lines).
2. Run **Auric Artisan: Mark Line as Intentional Branding (Toggle)** from the Command Palette or the editor's right-click menu. It is on the menu in HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX, CSS, SCSS, Less, Sass, PostCSS, Stylus and Markdown files.
3. The command adds `auric-brand-line` at the end of the line in a comment, and the status bar says **Auric Artisan: line marked as intentional branding — color findings here move to the Branding group.**
4. Run it again on the same line to remove the mark: **Auric Artisan: branding mark removed — findings here are graded again.**

You can also select **Intentional branding** in a contrast hover, or **Mark as intentional branding** in the Contrast Studio.

The command writes `# …` in hash-comment languages, `<!-- … -->` in markup languages, `/* … */` in CSS dialects, and `// …` everywhere else. Inside a `<style>` block of an HTML, Vue or Svelte file you can change the comment to `/* auric-brand-line */`; it works the same.

The branding.enabled setting

`auricColor.branding.enabled` and `auricA11y.branding.enabled` (both on by default) control whether the editor honours branding marks. When off, marked pairs are shown as ordinary contrast findings in the editor. Project scans and reports always honour the marks.

Branding colours in your theme

The Branding group's colours can be changed with VS Code's `workbench.colorCustomizations`. See Theme colours.

Suppression or branding?

Situation	Use
The checker is wrong about this code	Suppression, scoped to the rule ID
You deliberately accept a rule's risk here, after review	Suppression, scoped to the rule ID, with a comment explaining why
A deliberate brand colour pair that fails contrast	Branding
A structural failure on a brand element	Fix it; neither directive can hide it from the score as branding

Related

- Scores and grades
- Rulebook and knowledge database, which includes the accessible brand governance rulebook and policies such as **Brand classification cannot suppress structural findings**.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

The Workstation

How co-installed Studio extensions elect one host, hide duplicate views and present one Command Center and one Studio.

A **Workstation** is what you get when two or three Studio extensions are installed together. Instead of three sidebars, three dashboards and three copies of every finding, the extensions agree on one **host** that presents a single sidebar, one Command Center and one Studio covering every installed capability. A **capability** here means one extension's area: colour, accessibility or code health.

How the host is chosen

The host is the first installed extension in this fixed order:

1. Auric Accessibility
2. Auric Color
3. Auric Code Health

Installed	Host	Sidebar title
All three (the Full Pack)	Auric Accessibility	Auric Accessibility
Accessibility and Color	Auric Accessibility	Auric Accessibility
Accessibility and Code Health	Auric Accessibility	Auric Accessibility
Color and Code Health	Auric Color	Auric Color
One extension	That extension	Its own name

Every extension works out the same answer from the list of installed extensions, so there is nothing to configure.

What the host shows

- **One sidebar.** The host's Activity Bar container keeps its views. When Auric Color is installed and Auric Accessibility hosts, the host's sidebar also shows the docked **Color Picker** view.
- **One Overview.** The host's Overview shows a row for every installed capability: for example, an Accessibility host with Code Health installed adds a **Code health** row and **Report** and **SARIF** buttons. See The Overview view.
- **One Studio.** The host's Studio shows the union of the installed capabilities' tools. Its tab title changes to **Auric Artisan — Workstation Studio**. See The Studio.
- **One Command Center.** `Ctrl` + `Alt` + `.` opens the host's Command Center, which lists actions for every installed capability and runs each one through the extension that owns it. See The Command Center.
- **One knowledge database.** The Rulebook combines the rules, policies and articles of every installed capability. See Rulebook and knowledge database.

What the other extensions do

The extensions that are not the host are **suppressed**:

- Their Activity Bar views are hidden.
- Their `Ctrl` + `Alt` + `.` shortcut is switched off, so the shortcut always reaches the host.
- Their Studio commands, such as **Open Color Studio**, open the host's Studio instead. Commands that target one tool, such as **Generate Accessible Palette**, open that tool in the host's Studio.
- They keep doing the work they own. For example, Auric Code Health still publishes code-health underlines and Auric Color still draws swatches and colour hovers. See Choose what to install for the ownership table.

Changes apply without a reload

When you install or uninstall a Studio extension, the Workstation re-elects its host straight away. Views appear or hide, an open Studio tab is rebuilt with the new set of spaces (or closed, if its extension is no longer the host), the Overview is redrawn, and diagnostics are handed to the new owner. You do not need to reload the window.

One shared data folder

All installed extensions read and write one `.auric-artisan` folder in your workspace. To keep that folder consistent:

- Only one extension runs automatic background scans: the one that owns the status-bar score.
- Settings that shape the shared folder are read the same way by every extension. See Shared settings.
- Writes are atomic, so two extensions can never leave a half-written file.

See Scanning for the folder's contents.

Things to know

- **Duplicate Command Palette titles.** With several extensions installed, some titles appear more than once in the Command Palette, one per extension, for example three entries named **Auric Artisan: Workstation: Command Center**. Each runs in its own extension. The keyboard shortcut always opens the host's version, which lists every capability.
- **Settings stay per extension.** Each extension has its own settings namespace (`auricColor.` , `auricA11y.` , `auricCodeHealth.`). The host's Studio **Settings** space writes each value to the extension that owns it.
- **Removing the host.** If you uninstall Auric Accessibility from a Full Pack install, Auric Color becomes the host and its views reappear.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Reference

Every command, keybinding, view, menu entry, setting, theme colour and workspace file contributed by the four Studio extensions, in one place.

This page lists everything the Studio extensions add to VS Code, across all four extensions. Each extension's own reference page describes its commands and settings in more detail:

- Auric Color reference
- Auric Accessibility reference
- Auric Code Health reference

- Full Pack reference

The facts on this page are for version 0.1.0.

Extensions

Display name	ID	Settings prefix	Categories
Auric Color — Picker, Palettes & Contrast	`auric-artisan.auric-color`	`auricColor.`	Visualization, Programming Languages, Other
Auric Accessibility — WCAG 2.2 Linter	`auric-artisan.auric-a11y`	`auricA11y.`	Linters, Programming Languages, Other
Auric Code Health — Secrets, TODOs & Risk	`auric-artisan.auric-codehealth`	`auricCodeHealth.`	Linters, Other
Auric Artisan Studio — Full Pack	`auric-artisan.auric-artisan`	none	Extension Packs, Linters, Visualization, Other

All need VS Code 1.74 or newer. The three code extensions start when VS Code finishes loading.

Keybindings

Keys (Windows, Linux)	Keys (macOS)	Command	Extensions	Active when
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center	All three	The extension is not suppressed by a Workstation host
`Ctrl` + `Alt` + `C`	`Cmd` + `Alt` + `C`	Pick / edit color	Auric Color	The editor has focus
`Ctrl` + `Alt` + `A`	`Cmd` + `Alt` + `A`	Inspect Accessibility of Element	Auric Accessibility	The editor has focus

To rebind them, open **File › Preferences › Keyboard Shortcuts** and search for the command title.

Commands

Every command appears in the Command Palette under the **Auric Artisan** category, so its full palette title starts with `Auric Artisan:`. The command ID's prefix depends on the extension: `auricColor.`, `auricA11y.` or `auricCodeHealth.`.

Palette title (after `Auric Artisan:`)	ID suffix	Color	A11y	Code Health
Workstation: Command Center	`commandCenter`	yes	yes	yes
Open Color Studio	`openStudio`	yes		
Open Accessibility Studio	`openStudio`		yes	
Open Code Health Studio	`openStudio`			yes
Open Rulebook & Knowledge Database	`openKnowledgeBase`	yes	yes	yes
Build Workspace Knowledge Database (JSON + Markdown)	`writeKnowledgeDatabase`	yes	yes	yes
Open Unified Icon Studio	`openIconStudio`	yes	yes	yes
Pick / edit color	`pickColor`	yes		

Palette title (after `Auric Artisan:`)	ID suffix	Color	A11y	Code Health
Generate Accessible Palette	`generatePalette`	yes		
Check Contrast of Selection	`checkContrast`	yes	yes	
Fix All Contrast Issues in File	`fixAllContrast`	yes		
Choose Color Picker Mode	`chooseColorPickerMode`	yes		
Disable native color picker (use Auric Artisan)	`disableNativeColorPicker`	yes		
Restore native color picker (use VS Code's)	`restoreNativeColorPicker`	yes		
Refresh Contrast Diagnostics	`refreshDiagnostics`	yes		
Change Accent Color (Theme the Studio)	`chooseAccent`	yes		
Mark Line as Intentional Branding (Toggle)	`markBranding`	yes	yes	
Inspect Accessibility of Element	`inspectAccessibility`		yes	
Scan Project for Accessibility (Use Cache)	`scanProject`		yes	
Force Rescan Project and Refresh Cache	`rescanProject`		yes	
Generate Agent-Ready Remediation Plan	`generateAiPlan`		yes	yes
Clear Workspace Scan Cache	`clearScanCache`		yes	yes
Search Accessibility Issues	`searchIssues`		yes	
Group Accessibility Issues (Type / File / Severity)	`groupIssues`		yes	
Clear Accessibility Issue Search	`clearIssueSearch`		yes	
Open WCAG Documentation	`openIssueDocs`		row action only	
Open Issue in Contrast Studio	`openIssueInStudio`		row action only	
Generate Project Health Report (Color, A11y & Code Health)	`projectHealthReport`			yes
Write Health Record (health.json + SARIF)	`writeHealthReport`			yes
Open Workspace Data Folder	`openDataFolder`			yes

Auric Color contributes 15 commands, Auric Accessibility 17 and Auric Code Health 10. **Open WCAG Documentation** and **Open Issue in Contrast Studio** are hidden from the Command Palette; they appear as buttons on rows of the Project Issues view.

What each command does:

Command	What it does
Workstation: Command Center	Opens the Command Center list. See The Command Center.
Open … Studio	Opens the Studio tab, or the Workstation host's Studio. See The Studio.
Open Rulebook & Knowledge Database	Searches the local knowledge database.
Build Workspace Knowledge Database (JSON + Markdown)	Writes `knowledge.json`, `rulebook.md` and an overrides template.
Open Unified Icon Studio	Opens the combined Auric icon library.
Pick / edit color	Opens the colour picker on the colour under the cursor, or inserts a new colour.
Generate Accessible Palette	Opens Studio › Color › Palette seeded with the selected colour.

Command	What it does
Check Contrast of Selection	Opens the Contrast Studio with the selected colour as the text colour.
Fix All Contrast Issues in File	Previews and applies a fix for every failing pair in the file, as one undoable edit.
Choose Color Picker Mode	Chooses between the Auric picker and VS Code's native picker.
Disable native color picker (use Auric Artisan)	Switches to the Auric picker and turns VS Code's colour decorators off.
Restore native color picker (use VS Code's)	Hands colour picking back to VS Code's native picker.
Refresh Contrast Diagnostics	Re-checks every open file.
Change Accent Color (Theme the Studio)	Picks the Studio's accent colour.
Mark Line as Intentional Branding (Toggle)	Adds or removes an `auric-brand-line` comment on the current lines.
Inspect Accessibility of Element	Shows the Inspect card for the element or CSS rule at the cursor.
Scan Project for Accessibility (Use Cache)	Scans the workspace, reusing a fresh cache.
Force Rescan Project and Refresh Cache	Scans the workspace from disk and rewrites the cache.
Generate Agent-Ready Remediation Plan	Writes the local, deterministic remediation plan.
Clear Workspace Scan Cache	Deletes `scan-cache.json`.
Search Accessibility Issues	Filters the Project Issues view by text.
Group Accessibility Issues (Type / File / Severity)	Changes how the Project Issues view is grouped.
Clear Accessibility Issue Search	Removes the Project Issues filter.
Open WCAG Documentation	Opens the W3C Understanding page for a finding's success criterion.
Open Issue in Contrast Studio	Opens a contrast or colour-vision finding in the Contrast Studio, linked to its line.
Generate Project Health Report (Color, A11y & Code Health)	Writes the Markdown, JSON and HTML project health report.
Write Health Record (health.json + SARIF)	Writes `health.json`, SARIF and history.
Open Workspace Data Folder	Shows the `.auric-artisan` folder in your file manager.

Views

View	View ID	Extension	Notes
Overview	`auricColorHome`	Auric Color	Activity Bar container **Auric Color**
Color Picker	`auricColorPickerView`	Auric Color	Collapsed by default
Overview	`auricA11yHome`	Auric Accessibility	Activity Bar container **Auric Accessibility**
Project Issues	`auricA11yIssues`	Auric Accessibility	Tree view with a badge count
Color Picker	`auricA11yPickerView`	Auric Accessibility	Shown only when Auric Color is installed; collapsed by default
Overview	`auricCodeHealthHome`	Auric Code Health	Activity Bar container **Auric Code Health**

Views of a suppressed extension are hidden while another extension hosts the Workstation. VS Code also creates a focus command for each view, for example `auricA11yIssues.focus`, which you can bind to a key.

Menus

Editor right-click menu

Entry	Extension	Shown when
Inspect Accessibility of Element	Auric Accessibility	HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JS, TS, JSX, TSX and CSS-family files
Pick / edit color	Auric Color	Always
Generate Accessible Palette	Auric Color	Text is selected
Check Contrast of Selection	Auric Color, Auric Accessibility	Text is selected
Choose Color Picker Mode	Auric Color	Always
Mark Line as Intentional Branding (Toggle)	Auric Color, Auric Accessibility	The Inspect languages above, plus Markdown

Project Issues view title bar

Search Accessibility Issues, **Group Accessibility Issues**, **Scan Project for Accessibility**, **Open Accessibility Studio** and **Collapse All**, with **Generate Agent-Ready Remediation Plan** in the **...** menu.

Project Issues rows

Open WCAG Documentation on WCAG rows; **Open Issue in Contrast Studio** on contrast and colour-vision rows.

Settings

The table lists each setting's key after its prefix, and which extensions declare it. Give the full key in `settings.json`, for example `auricA11y.contrast.threshold`.

Key	Type	Default	Color	A11y	Code Health
`appearance.accent`	string	`gold`	yes	yes	yes
`hover.enabled`	boolean	`true`	yes		
`colorPicker.mode`	`auric` or `decorators`	`auric`	yes		
`colorPicker.nativeHoverPicker`	boolean	`false`	yes		
`colorPicker.swatches`	boolean	`true`	yes		
`colorPicker.clickAction`	`inlinePicker`, `fullEditor`, `off`	`inlinePicker`	yes		
`colorPicker.autoOpen`	boolean	`false`	yes		
`colorPicker.inlineSwatches`	boolean	`false`	yes		
`inspect.enabled`	boolean	`true`		yes	
`inspect.onHover`	boolean	`true`		yes	
`contrast.enabled`	boolean	`true`	yes	yes	
`contrast.standard`	`wcag2`, `apca`, `both`	`wcag2`	yes	yes	

Key	Type	Default	Color	A11y	Code Health
`contrast.threshold`	number, 1–21	`4.5`	yes	yes	
`contrast.apcaThreshold`	number, 15–106	`60`	yes	yes	
`contrast.severity`	`error`, `warning`, `information`, `hint`	`warning`	yes	yes	
`contrast.flagCvdRisk`	boolean	`true`	yes	yes	
`contrast.checkPageBackground`	boolean	`true`	yes	yes	
`branding.enabled`	boolean	`true`	yes	yes	
`a11y.enabled`	boolean	`true`		yes	
`a11y.profile`	`2.0`, `2.1`, `2.2`, `3.0-draft`	`2.2`		yes	
`codeHealth.enabled`	boolean	`true`			yes
`codeHealth.diagnostics`	boolean	`true`			yes
`codeHealth.todo`	boolean	`true`			yes
`codeHealth.debug`	boolean	`true`			yes
`codeHealth.secret`	boolean	`true`			yes
`codeHealth.conflict`	boolean	`true`			yes
`codeHealth.risk`	boolean	`true`			yes
`codeHealth.security`	boolean	`true`			yes
`codeHealth.customSecretPatterns`	array of strings	`[]`			yes
`codeHealth.size`	boolean	`true`			yes
`health.enabled`	boolean	`true`			yes
`health.writeOnScan`	boolean	`true`			yes
`health.formats`	array of `json`, `sarif`, `history`	all three			yes
`health.fileName`	string	`health.json`			yes
`health.maxIssues`	number, 100–100000	`5000`			yes
`health.historyLimit`	number, 10–5000	`200`			yes
`data.enabled`	boolean	`true`		yes	yes
`data.folderName`	string	`.auric-artisan`		yes	yes
`scan.cache.enabled`	boolean	`true`		yes	yes
`scan.cache.maxAgeMinutes`	number, 0 or more	`1440`		yes	yes
`scan.live`	boolean	`true`		yes	yes
`scan.liveDelayMs`	number, 200–5000	`700`		yes	yes
`scan.autoScan`	boolean	`false`		yes	yes
`scan.autoScanOnStartup`	boolean	`false`		yes	yes
`scan.autoScanDelayMs`	number, 250–60000	`1800`		yes	yes
`scan.maxFiles`	number	`4000`		yes	yes

Key	Type	Default	Color	A11y	Code Health
`scan.exclude`	glob string	see below		yes	yes

The default of `scan.exclude` is `**/{node_modules,dist,build,out,.git,vendor,coverage,.auric-artisan}/**`. The default of `health.formats` is `["json","sarif","history"]`.

Auric Color declares 16 settings, Auric Accessibility 24 and Auric Code Health 28.

Shared settings

The installed extensions share one `.auric-artisan` folder, so settings that shape it must mean the same thing to every extension. For these keys, each extension reads the first value it finds set (in any scope: user, workspace or folder), looking first under `auricCodeHealth.`, then `auricA11y.`, then `auricColor.`:

- `data.enabled`, `data.folderName`
- `scan.cache.enabled`, `scan.cache.maxAgeMinutes`, `scan.autoScan`, `scan.autoScanOnStartup`, `scan.autoScanDelayMs`, `scan.maxFiles`, `scan.exclude`, `scan.live`
- every `codeHealth.` key
- `health.formats`, `health.fileName`, `health.historyLimit`, `health.writeOnScan`

The simplest approach is to set each of these once, under one prefix. The Studio's **Settings** space writes a shared key to wherever its current value is set, so its edits always take effect.

The other settings are read by the extension that owns the feature: colour picker and hover settings by Auric Color; Inspect and profile settings by Auric Accessibility; contrast settings by Auric Accessibility when it is installed, otherwise by Auric Color; and the accent by the extension that hosts the Studio.

In a multi-root workspace, scan and data settings can be set per workspace folder; the accent is always stored in your user settings.

Theme colours

Both Auric Color and Auric Accessibility contribute four colours for the Branding group. Override them in `workbench.colorCustomizations`, for example `"auricA11y.branding.icon": "#FF4FA0"`.

Colour ID suffix	Used for	Dark	Light	High contrast	High contrast light
`branding.foreground`	Foreground of Branding findings	`#F06595`	`#C2255C`	`#FF80AB`	`#A61E4D`
`branding.background`	Soft wash behind Branding rows and badges	`#F0659522`	`#C2255C1A`	transparent	transparent
`branding.border`	Border or underline for Branding findings	`#F0659580`	`#C2255C80`	`#FF80AB`	`#A61E4D`
`branding.icon`	Icon tint of the Branding group in the Issues view	`#F06595`	`#C2255C`	`#FF80AB`	`#A61E4D`

The full IDs are `auricColor.branding.foreground` and so on, and `auricA11y.branding.foreground` and so on.

Status-bar items

Item	Shown	Select to
Score, for example `A11y 82 B` (or `Health 90 A` for Auric Code Health alone)	Always, by one extension	Open the Studio
Auric: N in this file	When the active file has contrast or accessibility findings	Open the Project Issues view, or the Contrast Studio without Auric Accessibility

Diagnostics

All findings in the Problems panel have the source `auric-artisan`.

Finding	Problems severity	Code shown
Low contrast	`contrast.severity` (warning by default); information when the background was assumed	`low-contrast`
Low APCA contrast	Same as low contrast	`apca-low`
Colour-vision risk	Hint	`cvd-risk`
Branding	Information	`branding:contrast`, `branding:apca` or `branding:cvd`
WCAG, critical or serious	Warning	`WCAG 1.1.1` (linked to the Understanding page)
WCAG, moderate	Information	Same
WCAG, minor, and best-practice hints	Hint	Same
Code health	By category: merge markers error; secrets, security and risky APIs warning; debug information; task markers and oversized hint	`code-health:secret` and so on

Workspace files

See The .auric-artisan folder for every file the Studio writes, and Exports and health records for their formats.

Command-line scanner

See The CI gate for every option and exit code of `auric-artisan`.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Troubleshooting

Symptoms, causes and fixes for problems that affect the Studio as a whole, from missing views to scans that skip files.

This page covers problems shared by all Studio extensions. Problems specific to one extension are in its own troubleshooting page: Auric Color, Auric Accessibility, Auric Code Health and the Full Pack.

Views and screens

I installed several Studio extensions but see only one Auric icon

Why: This is intended. Installed together, the extensions form one Workstation. The host (Auric Accessibility, else Auric Color, else Auric Code Health) keeps its views; the others hide theirs.

What to do: Use the host's sidebar. Its Overview, Command Center and Studio include every installed extension. See The Workstation.

The Color Picker view is missing

Why: It is collapsed by default, and in a Workstation hosted by Auric Accessibility it appears only when Auric Color is installed.

What to do: Expand **Color Picker** at the bottom of the sidebar, or run **Auric Artisan: Pick / edit color**.

The Command Palette lists the same command two or three times

Why: Each installed extension contributes its own copy of shared commands such as **Workstation: Command Center** and **Open Rulebook & Knowledge Database**.

What to do: Either works. For the Command Center, use `Ctrl` + `Alt` + `.`, which always opens the host's version covering every capability.

Ctrl + Alt + . does nothing

Why: Another extension or your operating system uses the same keys, or your keyboard layout treats `Ctrl` + `Alt` as `AltGr` and types a character instead.

What to do: Open **File › Preferences › Keyboard Shortcuts**, search for **Workstation: Command Center**, and check for conflicts or assign different keys. You can always run the command from the Command Palette.

The Studio or Overview does not appear in VS Code for the Web

Why: The Studio extensions run as desktop (and remote workspace) extensions; they do not include a build for VS Code in the browser.

What to do: Use VS Code on the desktop.

Scores and scans

The status bar shows "A11y" with no number

Why: There is no scan result yet. The tooltip says **Auric Artisan — run a project scan**.

What to do: Select **Scan project** on the Overview or run **Auric Artisan: Scan Project for Accessibility (Use Cache)**. With Auric Color alone, the score appears after the Studio has re-graded the workspace in the background as you edit.

The scan says "no color/background pairs found to evaluate"

Why: The scan found no text and background colours it could pair and no markup or CSS files to grade, so there is no score.

What to do: Check that your stylesheets and templates use file extensions the scan reads, and are not excluded. See What is read.

I changed the threshold or audit profile but the score did not change

Why: The loaded score comes from the scan cache, which was graded with the old settings. (Changing the audit profile makes Auric Accessibility treat the old cache as stale, but other settings do not.)

What to do: Select **Rescan**, **Force rescan** in Studio Settings, or run **Auric Artisan: Force Rescan Project and Refresh Cache**. Editor underlines update immediately without a rescan.

Some files were not scanned

Why: One of these applies:

- the file's extension is not in the scan list;
- it matches `scan.exclude` or a `.auricignore` rule;
- it is larger than 1 MB;
- the scan hit the `scan.maxFiles` limit (the status-bar tooltip says **Scanned N of M files**);
- it is a file named exactly `.env`, which the in-editor scan does not read.

What to do: Adjust `scan.exclude`, `.auricignore` or `scan.maxFiles` in Studio › Settings › **Scope**, then rescan. See Scanning.

Test fixtures or generated files flood the results

What to do: Add them to `.auricignore` (for example `test/fixtures/` or `*.min.css`), or to `scan.exclude`, then rescan. For a single deliberate case, use a suppression directive. See Suppression and branding.

The score does not follow my edits

Why: One of these applies:

- `scan.live` is off;
- the workspace is too large for live re-grading (more than 1,500 files or about 12 MB of text);
- the file you are editing was not part of the last scan (for example a new file);
- another installed Studio extension owns live re-grading, and its views are the ones updating.

What to do: Turn on **Continuous scan as you type** in Studio Settings, or rescan after your edits. For large workspaces, consider **Auto scan after file changes**.

A scan failed

What you see: Auric Artisan: project scan failed — followed by a reason, and **Existing results are untouched.** In the Studio, a toast starting **Scan failed**.

What to do: Your previous results are kept. Check that the workspace folder is readable and try **Force Rescan**. If a particular file causes the failure, add it to `.auricignore`.

The scan takes a long time

Why: Large workspaces with many files take longer, and background scans after every save add work.

What to do: Keep `scan.cache.enabled` on so ordinary **Scan project** reuses the cache, exclude generated folders, lower `scan.maxFiles` if you only need part of the project, and leave `scan.autoScan` off unless you need it.

Files and folders

A command says "open a workspace folder before ..."

Why: Reports, health records, the remediation plan and the knowledge database are written to `.auric-artisan` inside a workspace folder.

What to do: Open a folder with **File › Open Folder...** and run the command again.

A .auric-artisan folder appeared in my project

Why: The Studio stores its scan cache and outputs there. It includes a `.gitignore` so regenerated files are not committed.

What to do: Keep it, rename it with `data.folderName`, or turn off **Store workspace data locally** (`data.enabled`) to stop writing workspace data. Deleting it is safe; it is recreated when needed.

Write Health Record says "health report is disabled"

Why: `auricCodeHealth.health.enabled` is off.

What to do: Turn it back on to use **Write Health Record**.

Knowledge overrides are reported as invalid

What you see: Auric Artisan: workspace knowledge overrides are invalid; using the built-in database. or **could not build the knowledge database —** with a list of problems.

What to do: Fix each listed problem in `.auric-artisan/knowledge-overrides.json`. The rules are in Rulebook and knowledge database.

Settings

Studio Settings changes do not stick

Why: The cards other than Appearance are saved only when you select **Save settings**. A value set in a narrower scope (for example a workspace folder setting) can also override what you changed.

What to do: Select **Save settings**, then check **File › Preferences › Settings** for the same key under **Workspace** or a folder.

I changed a scan setting in one extension but another ignores it

Why: Shared scan and data settings are read from the first extension that has a value, checking Auric Code Health, then Auric Accessibility, then Auric Color.

What to do: Set shared keys under one prefix only, or give them the same value everywhere. See Shared settings.

The accent colour did not change

Why: You changed the `appearance.accent` of an extension that is not hosting the Studio.

What to do: Use the Studio's **Settings › Appearance** card, which writes to the right extension. See Appearance.

Suggestions and the Contrast Studio

A dismissed suggestion never comes back

Why: Dismissals are permanent by design, and there is no command to undo them.

What to do: The same actions remain available in the Command Center's **Actions** section and in the Studio.

"This pair is no longer linked to its file"

Why: You opened another pair, or the Studio tab was closed and reopened, so the link to the original line was lost.

What to do: Open the pair again from its hover or quick fix. See The Contrast Studio.

"The source line changed since the Studio opened it"

Why: The line was edited after the pair was opened, so the Studio will not write to it.

What to do: Open the pair again from the editor.

Still stuck?

Check Limits and accuracy to see whether the behaviour is a known limit of static analysis, and the FAQ. You can report a problem through https://auricartisan.com.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

FAQ

Short answers to common questions about Auric Artisan Studio as a whole.

Is Auric Artisan Studio free?

Yes. There is no account, API key or paid tier; every feature is included. The extensions are licensed under the Auric Artisan Free Use Licence 2.0: free to use for any purpose, personal or commercial, but not to redistribute, republish or resell.

Does it send my code anywhere?

No. All analysis runs on your machine, the extensions make no network requests and collect no telemetry. The only links that leave VS Code are ones you choose to open, such as auricartisan.com or a W3C documentation page. See Privacy.

Do I need all three extensions?

No. Each works on its own. Install the Full Pack if you want colour, accessibility and code health together; otherwise pick the one you need. See Choose what to install.

Why is there only one sidebar when I install several?

The extensions combine into one Workstation, so you get one Overview, one Command Center and one Studio instead of three copies. See The Workstation.

Is the remediation plan made by AI?

No. It is generated by fixed rules from your scan, on your machine, with no model call. It contains a prompt you can give to a coding agent of your own, if you want to. See The remediation plan.

Are the suggestions AI-powered?

No. They come from six fixed rules, each checking one fact from your scan or open file. See The suggestion rules.

Does a score of 100 mean my site is accessible?

No. The score covers only what static analysis can see in source code. Many WCAG requirements need a person to test with a keyboard, a screen reader, zoom and real content. Use the score to prioritise, and the manual checks for the rest.

Does it run my code or render my pages?

No. It reads source text. It does not execute scripts, run a browser or compute the final CSS cascade. See Limits and accuracy.

Which languages are supported?

Colour hovers and code-health checks cover more than 40 languages. Contrast checks run in CSS-family, markup, JavaScript, TypeScript and Markdown files. WCAG rules run in HTML, JSX, TSX, Vue, Svelte, Astro, PHP, XML, template dialects and CSS-family files. See Scanning.

Can I use it in CI?

Yes. The bundled `auric-artisan` command-line scanner produces the same findings, writes `health.json` and SARIF, and fails a build on new issues against a baseline. See The CI gate.

Where does it store data?

In a `.auric-artisan` folder at the root of your workspace, plus your VS Code settings. See The .auric-artisan folder.

Should I commit the .auric-artisan folder?

Commit `settings.json` if you want to share team defaults and `knowledge-overrides.json` for reviewed team guidance. The bundled `.gitignore` keeps everything else out, because it is regenerated.

Does it work in multi-root workspaces?

Yes. Every folder is scanned and can have its own `.auricignore`. The `.auric-artisan` folder is created in the first folder of the workspace.

Does it work over Remote SSH, WSL or dev containers?

The extensions run wherever VS Code runs your workspace's extensions, so they can read the remote files. They do not run in VS Code for the Web.

Can I turn off one kind of check?

Yes: `contrast.enabled`, `a11y.enabled`, `codeHealth.enabled` and each `codeHealth.<category>` switch. The first two control the in-editor diagnostics; project scans still grade contrast and WCAG. See Reference.

Can I ignore a finding?

Yes, with an `auric-disable-…` comment or the **Ignore** quick fix. Deliberate brand colours can be marked with `auric-brand-…` instead. See Suppression and branding.

How do I change the gold colour?

Studio › Settings › **Appearance**, or **Auric Artisan: Change Accent Color (Theme the Studio)**. See Appearance.

Where are the release notes?

In Release notes, taken from the changelog that ships with the extensions.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility — WCAG 2.2 Linter

What Auric Accessibility does, who it is for, how to install it, and a quick start from first scan to first fix.

Auric Accessibility finds accessibility problems in your markup and CSS while you write it. It checks HTML, JSX and TSX, Vue, Svelte, Astro, PHP, XML, common template languages and every CSS dialect against 70+ static rules mapped to the Web Content Accessibility Guidelines (WCAG) 2.0, 2.1 and 2.2: missing alternative text, unlabelled fields, empty buttons and links, invalid ARIA, blocked zoom, invisible focus, keyboard traps, small targets, accessible authentication and more. ARIA checks use the WAI-ARIA 1.2 role matrix and the HTML autofill grammar, so role and attribute errors are judged against the specification rather than a word list.

WCAG is the W3C's standard for accessible web content. Each requirement is a **success criterion** with a level: A, AA or AAA. **ARIA** attributes such as `role` and `aria-label` describe elements to assistive technology such as screen readers.

Beyond the editor, Auric Accessibility scans the whole project, grades it from 0 to 100, lists every finding in a searchable tree, and shows a before-and-after preview of contrast fixes. **Inspect** shows any element's accessible name, role, colours and contrast when you hover it. A local knowledge database adds all 86 WCAG 2.2 success criteria and eight manual test procedures for what automation cannot decide. Everything runs on your machine.

Who it is for

- Front-end developers who want accessibility problems flagged as they type, with safe fixes.
- Accessibility specialists who want a prioritised worklist and manual test procedures for a codebase.
- Teams who want a repeatable accessibility score and a CI gate.

Where to find it

- Marketplace ID: `auric-artisan.auric-a11y`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-a11y
- Install: `code --install-extension auric-artisan.auric-a11y`
- Version 0.1.0; VS Code 1.74 or newer. Also installed by the Full Pack.

Quick start

1. Install Auric Accessibility from the Extensions view (`Ctrl` + `Shift` + `X`).
2. Open a project with HTML, JSX, Vue, Svelte or similar files.
3. Open a component. Problems are underlined with a violet wavy line and listed in the Problems panel.
4. Hover an underline to see the WCAG criterion, how urgent it is and how to fix it. Select **Apply fix** where one is offered.
5. Put the cursor inside an element and press `Ctrl` + `Alt` + `A` (`Cmd` + `Alt` + `A` on macOS) to **Inspect** its name, role and colours.
6. Select the **Auric Accessibility** icon in the Activity Bar and select **Scan project**. The score ring shows the project's score and grade.
7. Expand **Project Issues** to browse every finding by type, file or severity.
8. Run **Auric Artisan: Generate Agent-Ready Remediation Plan** for a local, prioritised backlog of fixes.

What you can do

- Lint markup and CSS as you type against 70+ WCAG rules, each with its criterion, level, WCAG version, impact and a one-line fix.
- Choose the audit profile: WCAG 2.0, 2.1, 2.2 (default) or the 3.0 draft (which adds APCA contrast).
- Apply safe fixes from the hover, such as adding `lang="en"` or replacing a positive `tabindex`.
- Silence a false positive for one line, one rule or one file, or classify a brand colour as intentional.
- Inspect any element or CSS rule: accessible name, role, colours, contrast, APCA, colour-vision impact, a before-and-after preview and the issues found there.
- Check contrast in stylesheets, markup and CSS-in-JS, with WCAG 2.x or APCA targets and colour-vision risk hints.
- Scan the project, see a 0–100 score and grade, and triage findings in the Project Issues view or the Studio.
- Preview failing contrast pairs next to their accessible repair before changing code.
- Export a WCAG summary as Markdown, and generate an agent-ready remediation plan.
- Search the rulebook: every rule, every WCAG 2.2 criterion, manual procedures, policies and articles.

In this folder

Folder or file	What it covers
docs/	Guides and reference for Auric Accessibility.
docs/getting-started.md	First run and a tour.
docs/editor-linting.md	Diagnostics, hovers, quick fixes and supported languages.
docs/inspect.md	The Inspect card.
docs/project-scans.md	Scanning the project and reading the score.
docs/project-issues-view.md	The Project Issues tree.
docs/check-space.md	The Studio's Findings and Preview tools.
docs/rules.md	Every rule, grouped by WCAG success criterion.
docs/audit-profiles-and-contrast.md	Audit profiles, WCAG 2.x and APCA contrast.
docs/manual-evidence.md	The manual test procedures and how to record evidence.
docs/reference.md	Every command, keybinding, setting, view and menu.
docs/troubleshooting.md	Symptoms and fixes.
docs/faq.md	Short answers.
others/	Glossary, shortcuts, limits and accuracy, related pages, release notes and privacy.

Shared Studio features are in the Studio docs.

Back to the Studio overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility docs

The guides and reference pages for Auric Accessibility.

These pages cover Auric Accessibility's own features. Start with Getting started. Features shared with the other Studio extensions (the Overview, the Command Center, the Studio, the Contrast Studio, scanning, scores, suppression, exports and the CI gate) are in the Studio docs.

File	What it covers
Getting started	Install, first diagnostics, first scan and a tour.
Editor linting	Supported languages, diagnostics, issue hovers, Apply fix and quick fixes.
Inspect	The Inspect card for elements and CSS rules.
Project scans	Scanning the project, the score, the status bar and what a scan covers.
The Project Issues view	The tree of findings: grouping, search, row actions and tooltips.
The Check space: Findings and Preview	The Studio's full findings list and the before-and-after preview.
Rules	Every rule, grouped by WCAG success criterion, with severity and what it catches.
Audit profiles and contrast	WCAG 2.0, 2.1, 2.2 and 3.0-draft profiles; WCAG 2.x ratios and APCA.
Manual evidence	The eight manual test procedures and how to use them.
Reference	All 17 commands, both keybindings, all 24 settings, views and menus.
Troubleshooting	Symptoms, causes and fixes.
FAQ	Short answers.

Back to the Auric Accessibility overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Getting started with Auric Accessibility

Install Auric Accessibility, read your first WCAG finding, inspect an element, scan the project and find everything it adds to VS Code.

Install

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Search for `auric-artisan.auric-a11y` and select **Install**. Or run `code --install-extension auric-artisan.auric-a11y`.

It starts when VS Code finishes loading. You need VS Code 1.74 or newer. Nothing needs configuring: the default audit profile is WCAG 2.2 and the default contrast target is 4.5:1.

Your first finding

1. Open an HTML file, or a JSX, TSX, Vue, Svelte, Astro or PHP component.
2. Problems are underlined with a violet wavy line and listed in the Problems panel (`Ctrl` + `Shift` + `M`) with the source `auric-artisan`. For example, an `<img>` without `alt` shows:

```
Image has no alt attribute.  —  WCAG 1.1.1 Non-text Content (Level A, WCAG 2.0). Add alt="description" (or alt="" for purely decorative images).
```

3. Hover the underline. The card shows the criterion, a severity pill such as **CRITICAL**, the level and WCAG version, a priority such as **Priority 94/100 · fix now**, the file and line, the problem, the fix, and a before-and-after when a safe rewrite exists.
4. Select **Apply fix** to apply the rewrite, or press `Ctrl` + `.` for **Ignore this WCAG 1.1.1 finding here** and **Ignore all accessibility findings in this file**.

See Editor linting.

Inspect an element

1. Put the cursor inside an element, such as a `<button>`, or inside a CSS rule.
2. Press `Ctrl` + `Alt` + `A` (`Cmd` + `Alt` + `A` on macOS), or hover it.
3. The **Inspect** card shows its role and accessible name (or **No accessible name**), the colours that apply with contrast, APCA and a colour-vision check, a before-and-after preview, and the WCAG issues found there.

See Inspect.

Scan the project

1. Select the **Auric Accessibility** icon in the Activity Bar.
2. In the **Overview**, select **Scan project**.
3. When the scan finishes, a message reports the score, for example `Accessibility 82/100 (B) — 12 contrast + 30 WCAG issue(s) across 214 files (scanned).`, with **Open Studio**, **Show Issues** and **Force Rescan**.
4. The status bar now shows `A11y 82 B`, and **Project Issues** lists every finding.

See Project scans and The Project Issues view.

Where Auric Accessibility appears

Place	What you get
Editor	WCAG and contrast underlines, issue hovers, Inspect cards, quick fixes
Activity Bar, **Auric Accessibility**	**Overview** (the accessibility dashboard), **Project Issues**, and **Color Picker** when Auric Color is installed
Status bar	The score item, and **Auric: N in this file**
Editor right-click menu	**Inspect Accessibility of Element**, **Check Contrast of Selection** (with a selection), **Mark Line as Intentional Branding (Toggle)**
Studio (**Auric Artisan: Open Accessibility Studio**)	**Home**, **Check** (Findings, Contrast, Preview) and **Settings**
Command Center (`Ctrl` + `Alt` + `.`)	Scan, rescan, Studio tools, rulebook, remediation plan, worst findings

With the other Studio extensions

Auric Accessibility has the highest priority, so when it is installed with Auric Color or Auric Code Health it hosts the shared Workstation: its sidebar, Overview, Studio and Command Center include every installed capability. See The Workstation.

Next steps

- Rules: what each rule checks.
- Audit profiles and contrast: choose WCAG 2.0, 2.1, 2.2 or the 3.0 draft.
- Manual evidence: what to test by hand.
- The remediation plan.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Audit profiles and contrast standards

Choose which WCAG version the audit targets, and choose between WCAG 2.x contrast ratios and APCA.

Audit profiles

WCAG has grown in versions: 2.1 and 2.2 each added success criteria to 2.0. An **audit profile** tells Auric Accessibility which version you are targeting, so it only reports criteria that apply.

Profile	Studio label	Criteria checked
`2.0`	WCAG 2.0	Only criteria introduced in WCAG 2.0
`2.1`	WCAG 2.1	2.0 plus 2.1 criteria, such as reflow, text spacing, label in name and character key shortcuts
`2.2` (default)	WCAG 2.2 (default)	Everything from 2.0 to 2.2, such as target size and accessible authentication
`3.0-draft`	WCAG 3.0 draft (+APCA)	All 2.x rules, plus APCA contrast on top of the WCAG 2.x ratio checks

Criteria that depend on the profile:

Introduced in	Criteria with rules
WCAG 2.1	1.3.5 Identify Input Purpose, 1.4.10 Reflow, 1.4.12 Text Spacing, 2.1.4 Character Key Shortcuts, 2.3.3 Animation from Interactions, 2.5.3 Label in Name
WCAG 2.2	2.5.8 Target Size (Minimum), 3.3.8 Accessible Authentication (Minimum)

All other rules belong to WCAG 2.0 criteria and run in every profile.

Change the profile

- Studio › Settings › **Accessibility audit** › **Audit profile**, then **Save settings**; or
- set `auricA11y.a11y.profile` in your settings.

Editor diagnostics update immediately. Rescan the project afterwards: a cache made under another profile is treated as stale, so the next **Scan project** reads the workspace again. The command-line scanner has its own `--profile` option.

Contrast standards

WCAG 2.x contrast ratio

The **contrast ratio** compares the relative brightness of two colours, from 1:1 (identical) to 21:1 (black on white). WCAG 2.x asks for:

Target	Applies to
4.5:1	Normal text, level AA (1.4.3)
3:1	Large text (24 px, or 18.66 px bold) at AA, and user-interface parts (1.4.11)
7:1	Normal text at AAA (1.4.6); large text at AAA needs 4.5:1

Set your target with `auricA11y.contrast.threshold` (default 4.5). Large text automatically gets the lower target.

APCA

APCA (Accessible Perceptual Contrast Algorithm) is the contrast model proposed for the WCAG 3 working draft. It reports **Lc** (lightness contrast), roughly 0 to 106, and accounts for which colour is the text and which the background, so dark-on-light and light-on-dark pairs are rated differently. The Studio shows what an Lc value is suitable for:

Lc	Suitable for
90 or more	Any text
75 or more	Body text, 16 px and up
60 or more	Large or medium text, 18 px and up
45 or more	Large bold text, 24 px and up
30 or more	Non-text and UI only
below 30	Decorative only

APCA uses APCA-W3 0.0.98G. WCAG 3 is an unfinished W3C working draft, so an APCA verdict means only "meets the configured Lc target". It is not a WCAG 3 conformance claim.

Choose the standard

`auricA11y.contrast.standard`	Studio label	What is flagged
`wcag2` (default)	WCAG 2.x ratios	Pairs below the ratio target
`apca`	APCA (WCAG 3 draft)	Pairs below `auricA11y.contrast.apcaThreshold` (default Lc 60, range 15–106)
`both`	WCAG 2.x + APCA	WCAG failures first, then APCA shortfalls on pairs that pass WCAG

With the **3.0-draft** profile, `wcag2` behaves like `both`.

APCA findings are shown as `Low APCA contrast Lc … [WCAG 3 draft]`, with the quick fix **Fix APCA contrast**. Project scores and the Studio's pass rates always use WCAG 2.x ratios.

Colour-vision risk

Whichever standard you use, `auricA11y.contrast.flagCvdRisk` (on by default) adds a hint for pairs that pass for typical vision but fall below the target under protanopia, deuteranopia or tritanopia (WCAG 1.4.1, Use of Color).

Related

- Contrast diagnostics
- Scores and grades
- Reference

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

The Check space: Findings and Preview

Filter every finding in the Studio's Findings tool, and preview failing contrast next to its accessible repair in the Preview tool.

The Studio's **Check** space holds three tools. **Findings** and **Preview** come with Auric Accessibility; **Contrast** is the Contrast Studio. Switch between them with the tab row under the space header.

Findings

Findings lists every WCAG finding, failing contrast pair and colour-vision risk from the last scan. Its tab shows the total count.

Before a scan it says **No scan yet** and offers **Scan project**.

Header actions

Button	What it does
Export WCAG	Saves `wcag-report.md`, a Markdown summary by criterion with every finding. See Exports and health records.
Fix plan for an agent (gold)	Generates the remediation plan

WCAG accessibility

A section titled **WCAG accessibility** with the number of issues and files. It shows:

- badges for the accessibility sub-score (`a11y 97`), counts by level (`A 3 · AA 1 · AAA 0`) and counts by WCAG version;
- a badge for every success criterion found, such as `1.1.1 ×4`; select one to show only that criterion, and again to clear it;
- impact filters: **All**, **Critical**, **Serious**, **Moderate** and **Minor**, each with its count;
- a search box, **Filter by text, file, SC…**, that matches the message, criterion title and number, file path, evidence and level;
- a count such as `12 of 40 shown`, and the list, up to 200 rows at a time.

Each row shows the criterion, its title, the message, and the file, line and level (or **best practice**). Hover a row for the fix; select it (or press `Enter`) to open the file at that line.

Press `/` to jump to the search box, and `Escape` to clear it.

Failing contrast

Every failing pair, worst first (up to 200 rows), with the pair, its ratio, what it needs (for example `needs 4.5:1`, or `large text, needs 3:1`), `dark theme` when it applies, and its location. The section's subtitle shows your target and the large-text target.

Color-vision risk

Pairs that pass for typical vision but weaken under colour-vision simulation (up to 80), shown as `2.9:1 → 1.8:1 CVD`. Collapsed at first.

Branding

Pairs classified as intentional branding, each with a ♥ **branding** badge, its ratio and kind. A note explains the classification and how many structural findings had a branding mark that was ignored. Collapsed at first.

Confusable hues

Pairs of project colours that look different normally but merge under colour-vision simulation, with the colour difference before and after. Select one to copy both hex values. Collapsed at first.

Sections remember whether you left them open or closed.

Preview

Preview draws the same content twice, with your original colours and with an accessible repair, so you can judge legibility on real text rather than a swatch.

Controls

Control	Values	What it does
Text, **Background**	Any colour; ⇄ swaps them	The pair to preview
WCAG target	AA Large / UI · 3:1, AA body · 4.5:1, AAA · 7:1	The ratio the repair aims for
Fix by	Recolor text, Recolor background	Which colour the repair changes
Vision	Typical vision, Protanopia, Deuteranopia, Tritanopia, Achromatopsia	Shows both surfaces through a colour-vision deficiency

Interactive pair

Two surfaces side by side, **Original** and **After WCAG**. Each has a heading, body text, a caption, a **Primary action** button and a sample link, and below it the WCAG ratio, APCA Lc and **AA**, **AAA** and **UI 3:1** badges. A strip shows what changed (`text #999999 → #767676`), the ratio before and after, and ✓ **meets 4.5:1** or **best effort** when the hue cannot reach the target.

Buttons: **Copy** the repaired colour, **Insert fix** (replaces the editor selection with it) and **Open in Contrast** → (loads the repaired pair into the Contrast Studio).

The Original surface deliberately shows failing contrast, so it is hidden from screen readers; its numeric verdict below is read instead.

Project failures

After a scan, every failing pair from the project appears as its own before-and-after, up to the first 24 (the first three open), each with **Open file:line**. Before a scan, the section offers **Scan project**.

Open Preview on a specific pair with **Open in Preview** in an Inspect card.

Related

- The Studio
- The Project Issues view
- Contrast diagnostics

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Editor linting

Where Auric Accessibility checks your code as you type, how to read its diagnostics and hovers, and which fixes it can apply for you.

Auric Accessibility checks every open file it supports about 300 ms after you stop typing, and shows problems as underlines, hovers and entries in the Problems panel. No scan is needed for this.

Supported languages

Rules	VS Code languages
Markup rules	HTML, JavaScript React (JSX), TypeScript React (TSX), Vue, Svelte, Astro, PHP, XML, Handlebars
Markup rules, template dialects	Twig, Liquid, Jinja, ERB, Razor, Blade, EJS, Nunjucks, Mustache, Edge (when VS Code recognises the file type)
CSS rules	CSS, SCSS, Sass, Less, PostCSS, Stylus
Contrast checks	CSS dialects, HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX, Markdown

Plain JavaScript and TypeScript files (`.js`, `.ts`) get contrast checks for CSS-in-JS but not the markup rules; JSX and TSX files get both. Files larger than 1 MB are not checked.

The markup rules understand framework syntax: JSX attribute names such as `htmlFor` and `tabIndex`, bound values such as `:attr`, `[attr.x]` and `attr={…}`, and template output such as `{{ }}`, `<%= %>` and `@expr`. Values supplied at runtime are treated as unknown rather than wrong. Components such as `<Button>` are not judged as the native `<button>`. See Limits and accuracy.

Diagnostics

Every WCAG finding appears in the Problems panel with the source `auric-artisan` and a code such as `WCAG 1.1.1`, which links to the W3C's *Understanding* page for that criterion. The message has this shape:

```
<problem>  —  WCAG <criterion> <title> (Level <level>, WCAG <version>). <how to fix>
```

Impact	Problems severity
Critical, serious	Warning
Moderate	Information
Minor	Hint
Best-practice hint (any impact)	Hint, with `Best practice (related to WCAG …)` instead of the level

In the editor, findings get a violet wavy underline (lighter in dark themes) and a mark in the overview ruler.

When a branding mark covers the line of a structural finding, the message ends with **(A branding mark covers this line, but structural accessibility issues are not brandable — the finding still counts.)**

Contrast findings are covered in Contrast diagnostics; when Auric Accessibility is installed, it publishes them with its own `auricA11y.contrast.*` settings.

The issue hover

Hover an underline (or anywhere on its line) for the finding's card. Reading from the top:

1. **Title**: the criterion's name, for example **Non-text Content**.
2. **Badges**: a severity pill (**CRITICAL**, **SERIOUS**, **MODERATE** or **MINOR**), **Accessibility**, **Level A** (or AA, AAA), **WCAG 2.0** (the version that introduced the criterion), and a confidence chip when the rule reports one.
3. **Priority**: a ten-segment meter with **Priority N/100** and advice: **fix now**, **fix soon**, **worth fixing** or **low urgency**. See Priority in hovers.
4. **Where**: the file and line (select it to go there) and a link to the WCAG criterion.
5. **The problem**, as a quotation.
6. **How to fix**: a `diff` of the line before and after, when a safe rewrite exists; otherwise **In your code** shows the line. Then the fix in words.
7. **Actions**: the gold **Apply fix** button (when a rewrite exists), **Go to code**, **Rule details** (opens the rule in the knowledge database), **WCAG docs**, **Ignore here** and **Copy reference** (copies a one-line reference such as `WCAG 1.1.1 Non-text Content — src/index.html:12`).
8. **Footer**: the rule ID and detector version, for example `img-alt v1.2.0`.

Apply fix

Apply fix replaces the flagged tag or declaration with a corrected version in one edit you can undo. It is offered only where a deterministic rewrite is safe:

Rule	What Apply fix does
`img-alt`	Adds `alt=""` (marks the image as decorative; write a real description if it carries meaning)
`iframe-title`	Adds `title="Describe this frame"`
`input-label`	Adds `aria-label="Describe this field"`
`select-textarea-label`	Adds `aria-label="Describe this field"`
`button-name`	Adds `aria-label="Describe this action"`
`link-name`	Adds `aria-label="Describe this link"`
`html-lang`	Adds `lang="en"`
`aria-hidden-focusable`	Adds `tabindex="-1"`
`positive-tabindex`	Replaces the attribute with `tabindex="0"`
`accesskey`	Removes the `accesskey` attribute
`text-spacing-important`	Removes `!important` from the text-spacing declaration
`viewport-zoom`, `viewport-fixed-width`	Sets the viewport's `content` to `width=device-width, initial-scale=1`

> **Important:** The placeholder text such as `Describe this field` must be replaced with a real, meaningful label. An empty `alt=""` is right only for purely decorative images. Judgement calls, such as alt-text quality or link purpose, are never applied automatically.

Quick fixes

Put the cursor on a WCAG underline and press `Ctrl` + `.`:

Quick fix	What it inserts
Ignore this WCAG 1.1.1 finding here	`auric-disable-next-line 1.1.1` on the line above
Ignore all accessibility findings in this file	`auric-disable-file a11y` at the top of the file

Ignore here in the hover inserts `auric-disable-next-line` with the rule ID instead, so only that rule is silenced. See Suppression and branding.

The status bar

Auric: N in this file shows how many contrast and accessibility findings the active file has, for example `Auric: 5 in this file`; its tooltip splits them into contrast and accessibility. Select it to open the Project Issues view. It hides when the file is clean.

Settings that affect linting

Setting	Default	Effect
`auricA11y.a11y.enabled`	`true`	Turns WCAG diagnostics in the editor on or off. Project scans still grade WCAG findings.
`auricA11y.a11y.profile`	`2.2`	Which WCAG versions' rules run. See Audit profiles and contrast.
`auricA11y.branding.enabled`	`true`	Honours branding marks in the editor.
`auricA11y.contrast.*`	see Reference	Contrast checks.

Related

- Rules
- Inspect
- Reference

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Inspect

Use Inspect to see any element's or CSS rule's accessible name, role, colours, contrast, colour-vision impact and WCAG issues, with a before-and-after preview.

Inspect shows, in a hover card, everything the file tells Auric Accessibility about one element or CSS rule: its accessible name and role, the colours that apply, the contrast verdict with APCA and a colour-vision check, a rendered before-and-

after preview, and the WCAG issues found there.

An element's **accessible name** is what a screen reader announces for it, such as a button's text or an image's alt text. Its **role** is what kind of thing it is, such as a button, link or heading.

Open the Inspect card

- **Hover** an element or a CSS rule (when you are not over a colour value or an Auric finding).
- **Keyboard:** put the cursor inside the element or rule and press `Ctrl` + `Alt` + `A` (`Cmd` + `Alt` + `A` on macOS), or run **Auric Artisan: Inspect Accessibility of Element**. It is also on the editor's right-click menu.

Inspect works in HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX and every CSS dialect (CSS, SCSS, Sass, Less, PostCSS, Stylus).

If the command cannot inspect, it says why:

Message	Meaning
open a file and place the cursor on an element or CSS rule to inspect.	No editor is active.
Inspect works in HTML, JSX/TSX, Vue, Svelte, Astro, PHP, XML, JS/TS, and every CSS dialect.	The file's language is not supported.
place the cursor inside an element, a CSS rule, or a style region to inspect its accessibility.	Nothing inspectable is at the cursor.

What it inspects

- **Elements**: the innermost element around the cursor. Inspect shows a card for text and interactive elements (links, buttons, form fields and labels, images, paragraphs, headings, list items, table cells, landmarks such as `nav` and `main`, dialogs and similar), for components, and for any element that has a `role`, an `aria-label`, an inline style or a finding.
- **CSS rules**: the innermost rule around the cursor in a stylesheet, a `<style>` block, or a styled-components, Emotion or `css` template.

In JavaScript and TypeScript, a `<tag>` counts as an element only when it is a real HTML tag or a capitalised component, so comparisons such as `a < b` and generics such as `Array<string>` are never mistaken for markup.

The card

Heading

- For an element: **Inspect `<button>` · button**, then either **Name "Save changes"** or **No accessible name — assistive tech can't announce this element.**
- For a rule: **Inspect rule `.card-title`**.

The name comes from `aria-label`, `title`, `alt`, `placeholder` or the element's text, in that order.

Colours and contrast

When colours apply, the card shows:

- the text colour on the background, the ratio with a pass or fail icon and level, and **APCA Lc** with its use (for example **large/medium text ≥ 18px**);

- where the colours came from: **inline style**, **nearby rule**, **this rule**, **rule text on file bg** or **file colors** (the file's most used pair, when the element has no colours of its own);
- a **Now** and a **Fixed** (or **Passing**) preview, each a small surface with a heading, body text and a button drawn in the colours;
- when it fails, the fix: **Text #999999 → #767676 (hue & chroma preserved) reaches 4.54:1 for a 4.5:1 target.**;
- a **CVD** row with the pair under protanopia, deuteranopia and tritanopia and the worst ratio, marked **relies on hue** when it falls below the target.

Headings `h1` to `h3` use a 3:1 target, because browsers draw them as large text by default; other elements use 4.5:1.

Issues here

Up to six WCAG findings inside the element or rule, each with a severity badge, a level badge, a link to the criterion, the message and the fix. With colours but no findings, the card says **No structural WCAG issues detected here.**

Actions

- **Open in Preview**: opens the pair in the Studio's before-and-after Preview.
- **Go to element** or **Go to rule**: moves the cursor to it.
- **WCAG docs**: opens the first finding's criterion.

Settings

Setting	Default	Effect
`auricA11y.inspect.enabled`	`true`	Turns Inspect on or off, including the command
`auricA11y.inspect.onHover`	`true`	Shows the card automatically on hover. When off, Inspect appears only through the command, the right-click menu or `Ctrl` + `Alt` + `A`.

The Inspect card's findings follow your audit profile. Its contrast targets are fixed at 4.5:1, or 3:1 for `h1` to `h3`, whatever your `contrast.threshold`.

Limits

Inspect reads the file you are in. It does not know about styles in other files, classes applied at runtime or the final rendered cascade, so the colours shown are the best the file itself reveals. The **via** label tells you how the colours were found.

Related

- Editor linting
- The Contrast Studio
- Rules

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Manual evidence

The eight manual test procedures in Auric Accessibility's knowledge database, what each asks you to do, and how to record evidence for what automation cannot decide.

Static checks can find a missing `alt` attribute, but not whether the alt text is accurate; they can find a click handler without a key handler, but not whether a whole task works from the keyboard. Auric Accessibility's knowledge database includes eight **manual checks**: step-by-step procedures for the parts of WCAG a person has to test. Each one says what to do, what counts as a pass or a fail, and what evidence to record.

Open a manual check

1. Run **Auric Artisan: Open Rulebook & Knowledge Database** (or **Rulebook** on the Overview or the Studio rail).
2. Manual checks are listed near the top, after the rulebooks, marked **Manual check**. Type part of a title, or a criterion such as `2.4.3`, to find one.
3. Select it. It opens as a Markdown document with its **Procedure**, **Pass**, **Fail**, **Evidence to record** and **Official sources**.

The **Accessibility engineering rulebook** brings the rules and procedures together. **Build Workspace Knowledge Database** writes all of them to `.auric-artisan/rulebook.md` and `knowledge.json` for your team. See Rulebook and knowledge database.

The eight procedures

Operate every workflow with a keyboard

WCAG 2.1.1 and 2.1.2. Static analysis can find suspicious handlers, but only interaction testing proves complete keyboard operation.

1. Start at the browser chrome and use only `Tab`, `Shift` + `Tab`, arrow keys, `Enter`, `Space`, `Escape` and documented shortcuts.
2. Complete each primary task, and open and close every transient surface.
3. Confirm focus never becomes trapped, except inside a modal that provides an escape action.
4. Confirm pointer-only operations have an equivalent single-pointer or keyboard path.

Pass: every task is operable, focus stays discoverable and you can always move away or dismiss. **Evidence:** workflow name, keystroke sequence, observed focus order, failure recording.

Verify focus order preserves meaning and operation

WCAG 2.4.3.

1. Traverse the page forward and backward at each supported responsive breakpoint.
2. Compare focus movement with reading order, visual grouping and task dependencies.
3. Open dynamic content and confirm focus moves to and returns from it predictably.
4. Record any jump that changes meaning or forces the user to rediscover context.

Evidence: viewport, ordered focus trace, DOM and visual comparison, dynamic-state notes.

Confirm text alternatives communicate the same purpose

WCAG 1.1.1. Presence checks cannot decide whether alternative text is accurate, concise, contextual or intentionally empty.

1. Identify the purpose the non-text content serves in its context.
2. Inspect the computed accessible name and any adjacent equivalent text.
3. Confirm informative content is equivalent and decorative content is ignored.
4. Repeat for each distinct use; the same asset can need different alternatives in different places.

Evidence: element locator, accessible name, visible context, reviewer rationale.

Review media alternatives for accuracy and synchronization

WCAG 1.2.2, 1.2.3 and 1.2.5. A `<track>` element proves only presence.

1. Play the complete media with captions on, and separately with the audio muted.
2. Confirm speech, speakers, meaningful sounds and timing are accurate.
3. Confirm important visual information is available in narration, audio description or an equivalent.
4. Check the controls and alternatives remain keyboard and screen-reader operable.

Evidence: media identifier, timecode, caption or transcript excerpt, reviewer notes.

Confirm name, role, value, and state in an accessibility tree

WCAG 4.1.2.

1. Inspect the element in the browser's accessibility tree.
2. Confirm its computed name, role, value, states and relationships match the visible control.
3. Operate it with at least one supported screen reader and browser combination.
4. Confirm state changes are exposed when they happen, without duplicate or stale announcements.

Evidence: browser and assistive technology, accessibility-tree snapshot, announcement transcript.

Test zoom, reflow, and text spacing without loss

WCAG 1.4.4, 1.4.10 and 1.4.12.

1. Test at 200% browser zoom and at a 320 CSS-pixel viewport where applicable.
2. Apply the WCAG text-spacing overrides for line, paragraph, letter and word spacing.
3. Exercise controls and reveal hidden, expanded, validation and error states.
4. Confirm content does not clip, overlap, disappear or require two-dimensional scrolling, except for essential layouts.

Evidence: viewport and zoom, spacing preset, screenshots, exception rationale.

Verify important status changes are announced without moving focus

WCAG 4.1.3. Live-region markup alone does not prove that a meaningful, timely announcement reaches users.

1. Trigger each asynchronous success, progress, warning and error state.
2. Listen with a supported screen reader while focus stays on the control that started it.
3. Confirm the announcement is timely, concise, unique and includes what is needed for the next action.
4. Confirm rapid updates do not flood or overwrite critical messages.

Evidence: trigger, announcement transcript, timing, focus location.

Complete error recovery and authentication flows

WCAG 3.3.1, 3.3.3, 3.3.7 and 3.3.8.

1. Trigger each validation and service-error state with realistic invalid input.
2. Confirm errors identify the field and explain how to recover, in text.
3. Confirm entered data remains available and repeated entry is not unnecessarily required.
4. Verify authentication offers a path that does not depend solely on memory, transcription or puzzle solving.

Evidence: scenario, input, error text, recovery path, authentication alternative.

Record your evidence

The procedures list what to record so that another reviewer can repeat the test. Keep it wherever your team tracks accessibility work. To make your own procedures part of the database, add them as `manual-check` entries in `.auric-artisan/knowledge-overrides.json`; accessibility manual checks you add are marked as ACT-aligned manual assessments automatically. See Add team guidance.

The manual checks follow the structure of the W3C's ACT rules format, but make no claim of ACT conformance.

Related

- Rules
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

The Project Issues view

Browse, group, search and act on every finding from the last scan in the Project Issues tree view.

Project Issues is a tree view in the **Auric Accessibility** sidebar. It lists every finding from the last scan (or the live re-grade), so you can work through a project without opening files one by one.

Before a scan

The view shows two rows: **Run a project scan** (*WCAG, contrast, CVD*) and **Open Auric Artisan Studio**. Select either.

The title bar

Button	Command
Search	**Search Accessibility Issues**: filter the tree
Tree	**Group Accessibility Issues (Type / File / Severity)**
Play	**Scan Project for Accessibility (Use Cache)**
Studio	**Open Accessibility Studio**
Collapse All	Collapses every group
... menu	**Generate Agent-Ready Remediation Plan**

The Activity Bar icon shows a **badge** with the total number of issues (WCAG, contrast, colour-vision risk and code health), and the view's description reads, for example, `WCAG 2.2 · score 82`.

The summary row

The first row reads **Project accessibility: 82/100 B** with the number of files scanned and a small score ring. Select it to open the Studio's Home. Hover it for a table of the score, audit profile, files scanned, WCAG issues, contrast failures, colour-

vision risks, branding (when any), counts by level (A, AA, AAA) and by WCAG version, and links to **Open full scan report**, **Group by...** and **Search issues**.

Group the tree

Run **Group Accessibility Issues** (or the tree button) and choose:

Option	Groups
By type (default)	**WCAG issues**, **Contrast failures**, **CVD risk**, **Branding** (only when present, marked **excluded from score**), **Code health**, **Confusable colors**
By file	One group per file, busiest first, described as `N issues - worst serious`; rows sorted by line
By severity	**Critical**, **Serious**, **Moderate**, **Minor**, and **Branding (N) — excluded from score**

Your choice is remembered. In type mode each group shows its count; groups list up to 500 rows (800 for code health). File groups show up to 200 rows, and severity groups up to 300.

Rows

Row type	Label	Description	Icon
WCAG	`1.1.1 Non-text Content`	`critical - src/index.html:12`	Impact icon: error, warning, info or circle
Contrast failure	`#999999 on #FFFFFF`	`2.85:1 - src/app.css:40`	A small sample of the pair
Colour-vision risk	`#hex on #hex`	Ratio `->` CVD ratio, and location	A sample of the pair
Code health	The message	Category and location	Category icon
Branding	`#D3AF37 on #FFFFFF`	Kind, ratio and location	A sample of the pair
Confusable colors	`#hex <-> #hex`	`DeltaE 24 -> 6 (deuteranopia)`	A sample of the pair

Select a row to open the file at that line. Hover a row for the full issue card: the problem, priority, before-and-after and fix, as in the editor. Code-health rows show the category, message, evidence (redacted for secrets), location and fix.

Row actions

Button	Shown on	What it does
Open WCAG Documentation	WCAG rows	Opens the W3C *Understanding* page for the criterion
Open Issue in Contrast Studio	Contrast and colour-vision rows	Opens the pair in the Contrast Studio, linked to its line

These two commands are not in the Command Palette; use them from the row.

Search

1. Run **Search Accessibility Issues** (or the search button).
2. Type any text: part of a message, a criterion number, an impact, a file path or a colour, for example `label`, `auth/settings`, `#777777` or `serious`.
3. The tree shows only matching rows, with counts such as `WCAG issues (3/40)`, and every group opens. A **Filtered by "label"** row appears at the top; select it (or run **Clear Accessibility Issue Search**) to clear the filter.

The search is a case-insensitive text match on each row's details.

Related

- Project scans
- The Check space: Findings and Preview
- Reference

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Project scans

Scan the whole project with Auric Accessibility, read the result, and keep the score up to date.

A project scan reads every supported file in the workspace, grades it and saves the result, so you can see the whole project's state rather than just the open file.

Run a scan

How	What happens
Scan project on the Overview	Scans, reusing a fresh cache. After the first scan the button reads **Rescan** and forces a fresh read.
Auric Artisan: Scan Project for Accessibility (Use Cache)	Scans, reusing a fresh cache. Also the play button in the Project Issues title bar, and **Scan project** in the Command Center.
Auric Artisan: Force Rescan Project and Refresh Cache	Reads every file again and rewrites the cache. Also **Rescan project** in the Command Center, and **Rescan** in the Studio.
Auric Artisan: Clear Workspace Scan Cache	Deletes the cache; the score resets until the next scan. The message reads **Auric Artisan: cleared .auric-artisan scan cache.**

A notification, **Auric Artisan: scanning project…**, shows progress and can be cancelled.

Read the result

When the scan finishes, a message reports it:

```
Accessibility 82/100 (B) — 12 contrast + 30 WCAG issue(s) across 214 files (scanned).
```

`(cached)` instead of `(scanned)` means the fresh cache was reused. The buttons are **Open Studio** (opens Home), **Show Issues** (focuses the Project Issues view) and **Force Rescan**. If no contrast pairs or markup were found, the message is **Auric Artisan: no color/background pairs found to evaluate.** After a forced rescan, the message is **Auric Artisan: refreshed scan cache for N file(s).**

The result then appears in:

- the status bar: `A11y 82 B`, with a database icon when it came from the cache; hover it for pass rates and the data source;
- the Overview's workspace card;
- the Project Issues view, with a badge counting all issues;
- the Studio's Home and Check spaces;

- the Command Center's **Findings** section.

What the scan grades

- **Contrast:** every text and background pair the scanner can resolve, graded at your threshold (large text at 3:1), plus colour-vision risk and confusable hues.
- **WCAG:** every markup and CSS rule allowed by your audit profile.
- **Code health:** secrets, security, risky APIs, debug, task markers, merge markers and size, in every file. These appear in the Project Issues view and reports even without Auric Code Health; code-health underlines in the editor need Auric Code Health.
- **Branding:** pairs you marked as intentional are listed separately and left out of the score.

See Scanning for which files are read, the cache, `.auricignore` and the file limit, and Scores and grades for the formulas.

Keep it current

- **As you type:** with `scan.live` on (the default), the score, the Project Issues view and an open Studio re-grade from your edits without a scan.
- **After file changes:** turn on `scan.autoScan` for a full background rescan after files change, and `scan.autoScanOnStartup` to refresh when the workspace opens.
- **After changing settings:** rescan after changing the contrast threshold or standard. Changing the audit profile makes the cache stale, so the next **Scan project** reads the workspace again.

After a scan

- The Project Issues view to browse findings.
- The Check space: Findings and Preview to filter findings and preview contrast repairs.
- The remediation plan for a prioritised backlog.
- Exports and health records to share results; the health record is written after every scan by default.
- The CI gate to run the same scan on pull requests.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Rules

Every WCAG rule Auric Accessibility checks, grouped by success criterion, with its impact, what it catches and how to fix it.

Auric Accessibility runs 70+ static rules (72 in the current package) mapped to 29 WCAG success criteria, on top of the contrast checks. This page lists every rule by success criterion, in the order of WCAG's four principles: perceivable, operable, understandable and robust.

How to read this page

- **Rule ID** is what the editor, the Project Issues view, `health.json` and SARIF show, and what you can put in a suppression comment, for example `auric-disable-next-line img-alt`.
- **Impact** is how much the problem affects users: **critical**, **serious**, **moderate** or **minor**. It sets the Problems severity (critical and serious are warnings, moderate is information, minor is a hint) and the weight in the score.

- **hint** marks a **best-practice** rule: a pattern worth fixing that does not by itself fail the criterion. Best-practice findings are shown as hints (`Best practice (related to WCAG …)`), have low confidence, and weigh half in the score.
- **Checks** says whether the rule reads markup (HTML, JSX, TSX, Vue, Svelte, Astro, PHP, XML and template dialects) or CSS (every CSS dialect).
- **Confidence** and **automation** are given where a rule reports them. Other rules are high-confidence and fully automated. **Assisted** means a person should confirm the finding.
- The level and WCAG version are those of the success criterion. The audit profile decides which criteria run; see Audit profiles and contrast.

Each finding links to the W3C's *Understanding* page for its criterion, and **Rule details** in the hover opens the rule's entry in the knowledge database.

Contrast criteria

Contrast is graded by the contrast checks rather than by a markup rule. See Contrast diagnostics.

Criterion	Level	What is checked
1.4.3 Contrast (Minimum)	AA	Text and background pairs against your threshold (4.5:1, large text 3:1)
1.4.6 Contrast (Enhanced)	AAA	The same, when your threshold is 7 or more
1.4.1 Use of Color	A	Pairs that pass normally but lose contrast under colour-vision deficiency (hint)
1.4.11 Non-text Contrast	AA	Use a 3:1 threshold for UI parts and large text

APCA is offered as an experimental alternative model. An APCA result means "meets the configured Lc target", not WCAG 3 conformance.

Perceivable

1.1.1 Non-text Content

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`img-alt`	critical	Markup	Image has no alt attribute.
`area-alt`	serious	Markup	Image-map `<area>` has no alt text.
`input-image-alt`	critical	Markup	Image button (`<input type="image">`) has no alt text.
`svg-img-name`	moderate	Markup	`<svg role="img">` has no accessible name.
`object-name`	moderate	Markup	`<object>` / `<embed>` has no accessible name.
`redundant-alt`	minor	Markup	Alt text starts with “image of / photo of …” — screen readers already announce it as an image.
`alt-filename`	moderate	Markup	Alt text looks like a file name, not a description.

How to fix:

- `img-alt`: Add alt="description" (or alt="" for purely decorative images). Apply fix adds alt="".
- `area-alt`: Add alt="link destination" to every `<area>` that has an href.
- `input-image-alt`: Add alt="action name" so the button has an accessible name.

- `svg-img-name`: Add aria-label / aria-labelledby, or a `<title>` as the first child of the `<svg>`.
- `object-name`: Add title="what it shows" (or aria-label) to the `<object>` / `<embed>`.
- `redundant-alt`: Describe the content directly; drop the “image of” prefix.
- `alt-filename`: Replace the file name with a real description of the image.

1.2.2 Captions (Prerecorded)

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`media-captions`	serious	Markup	`<video>` / `<audio>` has no captions or subtitles track.
`video-captions`	moderate	Markup	Playable `<video>` has no captions track.

How to fix:

- `media-captions`: Add a `<track kind="captions">` (or provide a transcript) for the audio content. Muted video is skipped.
- `video-captions`: Add `<track kind="captions" src="…" srclang="…">` inside the `<video>` (not needed for silent/decorative video).

1.3.1 Info and Relationships

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`fieldset-legend`	moderate	Markup	`<fieldset>` has no `<legend>` to name the group.
`duplicate-main`	moderate	Markup	More than one main landmark — a page should have exactly one.
`heading-skip`	minor, hint	Markup	Heading level skipped — breaks the document outline.
`aria-required-owned`	serious	Markup	Composite ARIA role is missing a required owned role.
`aria-required-context`	serious	Markup	ARIA role appears outside the context role it requires.
`table-headers`	moderate	Markup	Data table has no header cells (`<th>`).
`details-summary`	moderate	Markup	`<details>` has no `<summary>` label.
`landmark-unique-name`	moderate	Markup	Repeated landmarks of the same type need unique accessible names.
`content-text`	minor	CSS	Meaningful text inserted via CSS content is unreliable for assistive tech and can't be translated or selected.

How to fix:

- `fieldset-legend`: Add a `<legend>` as the first child describing the grouped controls.
- `duplicate-main`: Keep a single `<main>` / role="main"; remove or relabel the extras.
- `heading-skip`: Don't jump heading levels (e.g. h2 → h4); step one at a time. Best-practice hint.
- `aria-required-owned`: Add the required child role, or use the equivalent native HTML control. Uses the WAI-ARIA 1.2 role matrix. In a component or partial, reported only when the wrong parent is visible in the same file. Confidence high, automated.

- `aria-required-context`: Nest it in the required composite/container role, or use native HTML with equivalent semantics. Uses the WAI-ARIA 1.2 role matrix. In a component or partial, reported only when the wrong parent is visible in the same file. Confidence high, automated.
- `table-headers`: Mark header cells with `<th scope="col/row">` (or role="presentation" for layout tables).
- `details-summary`: Add a direct `<summary>` child that names the disclosure. Confidence high, automated.
- `landmark-unique-name`: Give each repeated landmark a short, unique aria-label or aria-labelledby reference. Needs review. Confidence medium, assisted.
- `content-text`: Move real words into the markup; keep CSS content for decoration only.

1.3.5 Identify Input Purpose

Level AA, introduced in WCAG 2.1. Runs in the 2.1, 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`input-autocomplete`	minor	Markup	Identity input has no autocomplete token.
`autocomplete-invalid`	serious	Markup	autocomplete value is not a valid autofill field name.

How to fix:

- `input-autocomplete`: Add an autocomplete token (e.g. email, tel, name, current-password) to help users autofill.
- `autocomplete-invalid`: Use a token from the HTML autofill vocabulary — an unrecognised value turns autofill off entirely. Checked against the HTML autofill vocabulary.

1.4.2 Audio Control

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`media-autoplay`	moderate	Markup	Media autoplays — provide a way to pause/stop or mute.

How to fix:

- `media-autoplay`: Avoid autoplay, or add muted (video) and visible media controls.

1.4.4 Resize Text

Level AA, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`viewport-zoom`	serious	Markup	Viewport meta disables zoom (user-scalable=no / maximum-scale below 2).
`tiny-font`	minor, hint	CSS	Font size below 10px is illegible for many readers even after zooming.

How to fix:

- `viewport-zoom`: Remove user-scalable=no and let maximum-scale be ≥ 2 so users can zoom. Flags user-scalable=no, or maximum-scale below 2. Apply fix rewrites the viewport.
- `tiny-font`: Use at least 12px (0.75rem) for any text users must read. Best-practice hint.

1.4.8 Visual Presentation

Level AAA, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`justify-text`	minor	CSS	Justified text creates uneven “rivers” of space that hurt dyslexic readers.
`line-height-tight`	minor	CSS	Line height under 1.2 squeezes lines together and clips ascenders/descenders.

How to fix:

- `justify-text`: Use text-align: left (or start) for body copy.
- `line-height-tight`: Use a line-height of at least 1.2 for text (1.5 for body copy). Unitless or em values only; a px line-height is not judged.

1.4.10 Reflow

Level AA, introduced in WCAG 2.1. Runs in the 2.1, 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`viewport-fixed-width`	moderate	Markup	Viewport meta locks a fixed pixel width — content can't reflow on small screens.

How to fix:

- `viewport-fixed-width`: Use content="width=device-width, initial-scale=1". Apply fix rewrites the viewport.

1.4.12 Text Spacing

Level AA, introduced in WCAG 2.1. Runs in the 2.1, 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`text-spacing-important`	minor, hint	CSS	!important text spacing blocks user style-sheet overrides (WCAG text-spacing test).

How to fix:

- `text-spacing-important`: Drop !important from line-height / letter-spacing / word-spacing so users can adapt spacing. Best-practice hint. Apply fix removes !important.

Operable

2.1.1 Keyboard

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`interactive-keyboard`	serious	Markup	Click handler on a non-interactive element is not keyboard accessible.

How to fix:

- `interactive-keyboard`: Use a `<button>` / `<a>`, or add role + tabindex="0" and a key handler. Understands tabIndex={0}; catches Vue, Angular and Svelte click handlers.

2.1.4 Character Key Shortcuts

Level A, introduced in WCAG 2.1. Runs in the 2.1, 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`accesskey`	minor, hint	Markup	accesskey creates a single-character shortcut users can't remap or turn off.

How to fix:

- `accesskey`: Remove accesskey, or provide a way to disable/remap the shortcut. Best-practice hint. Apply fix removes the attribute.

2.2.1 Timing Adjustable

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`meta-refresh`	serious	Markup	Timed `<meta refresh>` reloads/redirects the page without user control.

How to fix:

- `meta-refresh`: Remove the timed refresh (an instant 0-second server redirect is exempt).

2.2.2 Pause, Stop, Hide

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`blink-marquee`	serious	Markup	`<blink>` / `<marquee>` auto-moves content and can't be paused.

How to fix:

- `blink-marquee`: Remove it; use CSS animation gated behind prefers-reduced-motion if needed.

2.3.3 Animation from Interactions

Level AAA, introduced in WCAG 2.1. Runs in the 2.1, 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`smooth-scroll-motion`	minor	CSS	scroll-behavior: smooth without a prefers-reduced-motion fallback.
`reduced-motion`	minor	CSS	Infinite animation without a prefers-reduced-motion fallback.

How to fix:

- `smooth-scroll-motion`: Wrap it in @media (prefers-reduced-motion: no-preference) { … }.
- `reduced-motion`: Wrap or disable it inside @media (prefers-reduced-motion: reduce).

2.4.1 Bypass Blocks

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`bypass-blocks`	moderate	Markup	Page has navigation but no skip link, `<main>` landmark or headings to bypass it.

How to fix:

- `bypass-blocks`: Add `<main>` (or role="main"), headings, or a “skip to content” link as the first focusable element.

2.4.2 Page Titled

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`doc-title`	serious	Markup	Document has no non-empty `<title>`.

How to fix:

- `doc-title`: Add a descriptive `<title>` in `<head>`.

2.4.3 Focus Order

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`positive-tabindex`	minor, hint	Markup	Positive tabindex disturbs the natural focus order.

How to fix:

- `positive-tabindex`: Use tabindex="0" (focusable, natural order) or "-1" (programmatic) — never > 0. Best-practice hint. Apply fix sets tabindex="0".

2.4.4 Link Purpose (In Context)

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`link-name`	serious	Markup	Link has no discernible text.
`link-generic`	minor	Markup	Link text is not descriptive out of context.

How to fix:

- `link-name`: Give the link meaningful text, or an aria-label for icon links. Apply fix adds an aria-label placeholder to replace.
- `link-generic`: Replace generic text like “click here” / “read more” with the link's destination.

2.4.6 Headings and Labels

Level AA, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`empty-heading`	moderate	Markup	Heading is empty — it labels nothing.

How to fix:

- `empty-heading`: Give the heading text (or remove it; don't use headings for spacing).

2.4.7 Focus Visible

Level AA, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`focus-outline-none`	serious	CSS	Focus outline removed without a visible replacement.

How to fix:

- `focus-outline-none`: Keep a visible focus indicator — add box-shadow / border, or use :focus-visible. One :focus-visible rule elsewhere in the file does not exempt every rule.

2.5.3 Label in Name

Level A, introduced in WCAG 2.1. Runs in the 2.1, 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`label-in-name`	moderate	Markup	aria-label doesn't contain the visible text — speech-input users can't activate it by name.

How to fix:

- `label-in-name`: Start the aria-label with the visible text (or drop the aria-label).

2.5.8 Target Size (Minimum)

Level AA, introduced in WCAG 2.2. Runs in the 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`target-size`	minor	CSS	Interactive target may be smaller than 24×24 CSS px.

How to fix:

- `target-size`: Give buttons/links/icons a min target of 24×24px (padding or min-width/height). Flags a CSS rule for a button, link, form control, `summary`, a `.btn` or `.button` class, `[role=button]` or `.chip` whose width and height are both set in pixels below 24.

Understandable

3.1.1 Language of Page

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`html-lang`	serious	Markup	`<html>` element is missing a lang attribute.
`lang-invalid`	moderate	Markup	`<html lang>` value is not a valid language tag.

How to fix:

- `html-lang`: Add lang="en" (or the page's language) to `<html>`. Apply fix adds lang="en".
- `lang-invalid`: Use a BCP-47 tag like lang="en", "en-US", "hi", "pt-BR".

3.1.2 Language of Parts

Level AA, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`lang-part-invalid`	moderate	Markup	lang on an element is not a valid language tag.

How to fix:

- `lang-part-invalid`: Use a BCP-47 tag like lang="fr", "es-MX", "hi" — or drop the attribute. The html element is left to 3.1.1, so one problem is never reported twice; templated values are skipped. Confidence high, automated.

3.2.1 On Focus

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`autofocus`	minor, hint	Markup	autofocus moves focus on load — disorienting for screen-reader and zoom users.

How to fix:

- `autofocus`: Avoid autofocus, or reserve it for single-purpose pages (like a lone search box). Best-practice hint.

3.2.2 On Input

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`select-onchange-nav`	moderate	Markup	Changing this control submits or navigates automatically — a context change on input.

How to fix:

- `select-onchange-nav`: Trigger submit/navigation from an explicit button instead of onchange.

3.3.2 Labels or Instructions

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`input-placeholder-label`	minor	Markup	Field is labelled only by its placeholder, which disappears as soon as the user types.
`select-textarea-label`	serious	Markup	`<select>` / `<textarea>` has no label or accessible name.

How to fix:

- `input-placeholder-label`: Add a visible `<label>` (keep the placeholder as an example or hint, not the label). Advisory: the placeholder does give the field a name, but it disappears as soon as someone types. Confidence medium, assisted.
- `select-textarea-label`: Associate a `<label for>`, or add aria-label / title. Apply fix adds an aria-label placeholder to replace.

3.3.8 Accessible Authentication (Minimum)

Level AA, introduced in WCAG 2.2. Runs in the 2.2 and 3.0-draft profiles.

Rule ID	Impact	Checks	What it catches
`no-paste-auth`	serious	Markup	Blocking paste / autocomplete on a password field fails accessible authentication.

How to fix:

- `no-paste-auth`: Allow paste and autocomplete="current-password" so password managers work.

Robust

4.1.2 Name, Role, Value

Level A, introduced in WCAG 2.0. Runs in all profiles.

Rule ID	Impact	Checks	What it catches
`aria-attr-unknown`	minor	Markup	Unknown ARIA attribute — likely a typo.
`input-label`	serious	Markup	Form field has no associated label or accessible name.
`button-name`	serious	Markup	Button has no discernible text.
`duplicate-id`	moderate	Markup	Duplicate id — breaks label/aria references and scripting.
`aria-role-invalid`	moderate	Markup	Unknown ARIA role.
`aria-abstract-role`	moderate	Markup	Abstract ARIA role must not be used by authors.
`redundant-role`	minor	Markup	Redundant ARIA role duplicates the element's native semantics.
`aria-reference-missing`	moderate	Markup	ARIA reference points to a missing id.
`aria-attr-unsupported`	serious	Markup	ARIA attribute is not supported on this element's role.
`aria-attr-invalid-value`	serious	Markup	ARIA attribute has a value outside its allowed set.
`aria-required-attr`	critical	Markup	Role is missing an attribute it requires.
`aria-required-name`	serious	Markup	ARIA role requires an accessible name.
`aria-roledescription-name`	moderate	Markup	aria-roledescription is present on an unnamed element.
`aria-attr-deprecated`	minor	Markup	Deprecated ARIA attribute has no effect.
`aria-prohibited-name`	minor	Markup	This role does not support an accessible name.
`iframe-title`	serious	Markup	`<iframe>` has no accessible title.
`aria-hidden-focusable`	serious	Markup	aria-hidden="true" on a focusable element hides it from assistive tech but not the keyboard.
`role-presentation-interactive`	serious	Markup	role="presentation"/"none" on an interactive element strips its semantics but keeps it focusable.
`nested-interactive`	serious	Markup	Interactive element nested inside another interactive element.
`dialog-name`	serious	Markup	Dialog has no accessible name.
`summary-name`	serious	Markup	`<summary>` has no accessible name.
`aria-hidden-body`	critical	Markup	The document body is hidden from the accessibility tree.

How to fix:

- `aria-attr-unknown`: Use a valid WAI-ARIA attribute (e.g. aria-label, aria-describedby, aria-expanded). Checked against the WAI-ARIA 1.2 attribute list.
- `input-label`: Add a `<label for>` tied to the field's id, or an aria-label / title. Apply fix adds an aria-label placeholder to replace.
- `button-name`: Add visible text, or an aria-label / title for icon-only buttons. Apply fix adds an aria-label placeholder to replace.
- `duplicate-id`: Make every id unique within the document. An id nothing refers to is a best-practice hint (minor); one a label or ARIA attribute points at stays moderate.
- `aria-role-invalid`: Use a valid WAI-ARIA role (or remove it).
- `aria-abstract-role`: Replace it with a concrete role (e.g. button, navigation, region) or remove it.
- `redundant-role`: Remove the role — the native element already exposes it. Deliberately narrow: never flags a list or list item given its role again (the fix for lists that lose their semantics when styled), a header or footer inside article, aside, main, nav or section, or an unnamed section or form given a role.
- `aria-reference-missing`: Make the referenced id exist, or remove the broken ARIA reference. Templated and framework-bound references are skipped.
- `aria-attr-unsupported`: Remove the attribute, or change the role to one that supports it.
- `aria-attr-invalid-value`: Use one of the values the specification defines — anything else is treated as absent.
- `aria-required-attr`: Add the required state — without it the role is announced but its value is not.
- `aria-required-name`: Provide visible text, aria-label, or aria-labelledby as appropriate for the role. Confidence high, automated.
- `aria-roledescription-name`: Give the element an accessible name, or remove aria-roledescription. Confidence high, automated.
- `aria-attr-deprecated`: Remove it — no current browser or screen reader implements it.
- `aria-prohibited-name`: Move the label to an element with a naming role, or give this one a role that accepts a name.
- `iframe-title`: Add title="what the frame contains" to the `<iframe>`. Apply fix adds a title placeholder to replace.
- `aria-hidden-focusable`: Remove aria-hidden, or also remove it from the tab order (tabindex="-1" / disabled). Apply fix adds tabindex="-1".
- `role-presentation-interactive`: Remove the role — interactive elements must expose their role to assistive tech. Applies to native controls and to any element with both a tab stop and a handler; purely decorative markup is not flagged.
- `nested-interactive`: Un-nest them — a link/button must not contain another link, button, or form control.
- `dialog-name`: Add aria-label or aria-labelledby pointing at the dialog's heading.
- `summary-name`: Put concise visible text in `<summary>`, or provide an accessible name. Confidence high, automated.
- `aria-hidden-body`: Remove aria-hidden="true" from `<body>`; hide only the specific inactive subtree. Confidence high, automated.

Rules that offer Apply fix

`img-alt`, `iframe-title`, `input-label`, `select-textarea-label`, `button-name`, `link-name`, `html-lang`, `aria-hidden-focusable`, `positive-tabindex`, `accesskey`, `text-spacing-important`, `viewport-zoom` and `viewport-fixed-width`. See Apply fix for what each rewrite does. Every other rule suggests a fix for you to make.

What the rules cannot judge

These rules read source text. They do not run your code, compute the final CSS cascade, expand component state or see markup that JavaScript generates. Several rules flag things that need human judgement, such as alt-text quality, link

purpose, dialog naming, field labels and authentication; for those, the fix is a suggestion to review, never an automatic repair. Automated checks cover only part of WCAG; use the manual checks for the rest. See Limits and accuracy.

Related

- Editor linting
- Suppression and branding
- Scores and grades

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility reference

Every command, keybinding, setting, view, menu entry and theme colour Auric Accessibility contributes.

Facts for Auric Accessibility 0.1.0 (`auric-artisan.auric-a11y`). It needs VS Code 1.74 or newer and starts when VS Code finishes loading. Categories: Linters, Programming Languages, Other.

Keybindings

Keys (Windows, Linux)	Keys (macOS)	Command	Active when
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center	Auric Accessibility is not suppressed (it is always the host when installed)
`Ctrl` + `Alt` + `A`	`Cmd` + `Alt` + `A`	Inspect Accessibility of Element	The editor has focus

Commands

All titles appear in the Command Palette after `Auric Artisan:`, except the two marked row action.

Title	Command ID	What it does
Open Accessibility Studio	`auricA11y.openStudio`	Opens the Studio on Home
Workstation: Command Center	`auricA11y.commandCenter`	Opens the Command Center list
Open Rulebook & Knowledge Database	`auricA11y.openKnowledgeBase`	Searches rules, WCAG criteria, manual checks, policies and articles
Build Workspace Knowledge Database (JSON + Markdown)	`auricA11y.writeKnowledgeDatabase`	Writes `knowledge.json`, `rulebook.md` and an overrides template
Open Unified Icon Studio	`auricA11y.openIconStudio`	Opens the combined Auric icon library
Scan Project for Accessibility (Use Cache)	`auricA11y.scanProject`	Scans the project, reusing a fresh cache
Force Rescan Project and Refresh Cache	`auricA11y.rescanProject`	Re-reads the project and rewrites the cache
Generate Agent-Ready Remediation Plan	`auricA11y.generateAiPlan`	Writes the local remediation plan
Clear Workspace Scan Cache	`auricA11y.clearScanCache`	Deletes `scan-cache.json`
Search Accessibility Issues	`auricA11y.searchIssues`	Filters the Project Issues view

Title	Command ID	What it does
Group Accessibility Issues (Type / File / Severity)	`auricA11y.groupIssues`	Groups the Project Issues view
Open WCAG Documentation	`auricA11y.openIssueDocs`	Row action: opens the criterion's Understanding page
Open Issue in Contrast Studio	`auricA11y.openIssueInStudio`	Row action: opens a contrast or colour-vision finding in the Contrast Studio
Clear Accessibility Issue Search	`auricA11y.clearIssueSearch`	Removes the Project Issues filter
Check Contrast of Selection	`auricA11y.checkContrast`	Opens the Contrast Studio with the selected colour as text
Inspect Accessibility of Element	`auricA11y.inspectAccessibility`	Shows the Inspect card at the cursor
Mark Line as Intentional Branding (Toggle)	`auricA11y.markBranding`	Adds or removes `auric-brand-line` on the current lines

Views

View	ID	Notes
Overview	`auricA11yHome`	In the **Auric Accessibility** Activity Bar container; the accessibility dashboard
Project Issues	`auricA11yIssues`	The findings tree, with a badge
Color Picker	`auricA11yPickerView`	Only when Auric Color is installed; collapsed by default

Menus

Menu	Entries
Editor right-click	**Inspect Accessibility of Element** (Inspect languages); **Check Contrast of Selection** (with a selection); **Mark Line as Intentional Branding (Toggle)** (Inspect languages and Markdown)
Project Issues title bar	**Search**, **Group**, **Scan**, **Open Accessibility Studio**, **Collapse All**; **Generate Agent-Ready Remediation Plan** in **...**
Project Issues rows	**Open WCAG Documentation** (WCAG rows); **Open Issue in Contrast Studio** (contrast and colour-vision rows)

Inspect languages: HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX, CSS, SCSS, Less, Sass, PostCSS and Stylus.

Settings

Auric Accessibility has 24 settings. Search `auricA11y` in **File › Preferences › Settings**. Many can also be changed in Studio › Settings.

Setting	Type	Default	What it does
`auricA11y.appearance.accent`	string	`gold`	The Studio's accent: a preset ID or a hex colour. Stored in user settings. See Appearance.
`auricA11y.inspect.enabled`	boolean	`true`	Turns Inspect (hover card and command) on or off.

Setting	Type	Default	What it does
`auricA11y.inspect.onHover`	boolean	`true`	Shows the Inspect card automatically on hover; when off, only through the command.
`auricA11y.contrast.enabled`	boolean	`true`	Flags text and background pairs below the target.
`auricA11y.contrast.standard`	`wcag2`, `apca`, `both`	`wcag2`	The contrast model.
`auricA11y.contrast.apcaThreshold`	number, 15–106	`60`	Minimum APCA Lc when APCA is used.
`auricA11y.contrast.threshold`	number, 1–21	`4.5`	Minimum WCAG 2.x ratio.
`auricA11y.contrast.severity`	`error`, `warning`, `information`, `hint`	`warning`	Problems severity for failing contrast.
`auricA11y.contrast.flagCvdRisk`	boolean	`true`	Adds colour-vision risk hints.
`auricA11y.contrast.checkPageBackground`	boolean	`true`	Grades colour-only rules against the page background, as softer hints.
`auricA11y.a11y.enabled`	boolean	`true`	Turns WCAG diagnostics in the editor on or off.
`auricA11y.a11y.profile`	`2.0`, `2.1`, `2.2`, `3.0-draft`	`2.2`	The audit profile. Rescan after changing it.
`auricA11y.branding.enabled`	boolean	`true`	Honours `auric-brand-…` marks in the editor.
`auricA11y.data.enabled`	boolean	`true`	Creates and uses the `.auric-artisan` folder. Shared.
`auricA11y.data.folderName`	string	`.auric-artisan`	The data folder's name, relative to the workspace. Shared.
`auricA11y.scan.cache.enabled`	boolean	`true`	Reuses the scan cache instead of rescanning. Shared.
`auricA11y.scan.cache.maxAgeMinutes`	number, 0 or more	`1440`	Maximum cache age; `0` keeps it until a forced or automatic scan. Shared.
`auricA11y.scan.live`	boolean	`true`	Re-grades the project as you type, from memory.
`auricA11y.scan.liveDelayMs`	number, 200–5000	`700`	Wait after the last keystroke before a live re-grade.
`auricA11y.scan.autoScan`	boolean	`false`	Rescans from disk after file changes. Shared.
`auricA11y.scan.autoScanOnStartup`	boolean	`false`	With auto scan on, also rescans soon after the workspace opens. Shared.
`auricA11y.scan.autoScanDelayMs`	number, 250–60000	`1800`	Wait after a file change before an automatic scan. Shared.
`auricA11y.scan.maxFiles`	number	`4000`	Most files one scan reads. Shared.
`auricA11y.scan.exclude`	glob	see below	Paths to skip in scans. Shared.

The default of `scan.exclude` is `**/{node_modules,dist,build,out,.git,vendor,coverage,.auric-artisan}/**`. "Shared" means the value is read suite-wide when other Studio extensions are installed; see Shared settings.

Theme colours

Colour ID	Dark	Light	High contrast	High contrast light
`auricA11y.branding.foreground`	`#F06595`	`#C2255C`	`#FF80AB`	`#A61E4D`

Colour ID	Dark	Light	High contrast	High contrast light
`auricA11y.branding.background`	`#F0659522`	`#C2255C1A`	transparent	transparent
`auricA11y.branding.border`	`#F0659580`	`#C2255C80`	`#FF80AB`	`#A61E4D`
`auricA11y.branding.icon`	`#F06595`	`#C2255C`	`#FF80AB`	`#A61E4D`

`auricA11y.branding.icon` colours the Branding group in the Project Issues view.

Studio spaces

On its own: **Home**, **Check** (Findings, Contrast, Preview) and **Settings** (Appearance, Project data, Scan engine, Scope, Accessibility audit, Contrast). As a Workstation host, it adds the spaces of the other installed extensions.

Diagnostics codes

Code	Meaning
`WCAG 1.1.1` and so on	A WCAG rule finding, linked to the criterion
`low-contrast`, `apca-low`, `cvd-risk`	Contrast findings
`branding:contrast`, `branding:apca`, `branding:cvd`	Classified as branding

Files it writes

`.auric-artisan/scan-cache.json`, `settings.json`, the health record (`health.json`, `health.sarif`, `health-history.json`) after scans, `ai-remediation-plan.md` and `.json`, and the knowledge files. See The .auric-artisan folder.

Related

- Rules
- Studio reference
- Shortcuts

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility troubleshooting

Symptoms, causes and fixes for Auric Accessibility's diagnostics, Inspect, scans and Project Issues view.

For problems shared by all Studio extensions (scans, the data folder, settings), see the Studio troubleshooting page.

Diagnostics

No WCAG underlines appear

Why: One of these applies:

- the file's language is not one the rules check; plain `.js` and `.ts` files get contrast checks only, not markup rules;

- VS Code does not recognise the file type (for example Svelte or Astro without their language extension), so the language is plain text;
- `auricA11y.a11y.enabled` is off;
- the file is larger than 1 MB;
- a suppression directive such as `auric-disable-file` covers it.

What to do: Check the language in the status bar's language picker, install the language's extension if needed, and check the setting and any `auric-disable` comments. See Editor linting.

A rule does not fire where I expect

Why: The profile excludes its criterion (for example target size needs the 2.2 profile), the value is supplied at runtime and is treated as unknown, the tag is a component rather than a native element, or the markup is generated by JavaScript.

What to do: Check your profile in Audit profiles and contrast, and see What the rules cannot judge.

A finding looks wrong

Why: Static checks cannot see everything, for example a label added by a component or text set at runtime.

What to do: If the markup is correct, silence that rule on that line with **Ignore here** or `auric-disable-next-line <rule-id>`, ideally with a comment saying why. See Suppression and branding.

A best-practice hint is not in the Problems list

Why: Best-practice findings are hints, which VS Code shows as dotted underlines and may hide from the Problems panel depending on your filters.

What to do: Check the Problems panel's filter, or view them in the Studio's Findings tool.

Apply fix added "Describe this field"

Why: Some fixes add a placeholder label so the control has a name at all.

What to do: Replace the placeholder with a real, specific label. See Apply fix.

The ignore comment appears as text on the page

Why: Inside JSX markup, a `//` comment inserted by the quick fix is JSX text.

What to do: Change it to `{/* auric-disable-next-line … */}`.

Inspect

Hovering shows nothing, or the colour card instead of Inspect

Why: The cursor is on a colour value (the colour card wins), on a line with a finding (the finding's card wins), on an element Inspect does not consider meaningful (such as a plain `div` with no role, label or finding), or `inspect.onHover` is off.

What to do: Press `Ctrl` + `Alt` + `A` with the cursor inside the element, or move to its tag name.

The command says "place the cursor inside an element, a CSS rule, or a style region"

Why: Nothing inspectable surrounds the cursor, for example in a CSS file outside any rule.

What to do: Move the cursor inside an element or a `{ … }` block.

Inspect shows "via file colors"

Why: The element has no colours of its own in this file, so Inspect shows the file's most used pair as an example.

What to do: Treat the contrast as indicative only; check the real styles in the browser.

Scans and the Project Issues view

"Scan Project" reports old results

Why: A fresh cache was reused (the message says `(cached)`).

What to do: Use **Force Rescan Project and Refresh Cache**.

The Project Issues view only says "Run a project scan"

Why: There is no scan result yet, or the cache was cleared.

What to do: Select **Run a project scan**.

The Project Issues view lists code-health findings, but there are no code-health underlines

Why: Project scans always include the code-health checks; editor underlines for them come from Auric Code Health.

What to do: Install Auric Code Health if you want them in the editor. With Auric Code Health installed, you can also leave them out of scans by turning off `auricCodeHealth.codeHealth.enabled`.

Search found nothing

Why: The search matches text in each row; it is not a pattern language.

What to do: Try a shorter word, a criterion number or a file name. Select **Filtered by "..."** to clear.

The badge number differs from the Problems panel

Why: The badge counts every finding in the project from the last scan; the Problems panel shows only open files.

Related

- FAQ
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility FAQ

Short answers about Auric Accessibility.

How many rules are there?

70+: the current package runs 72 static rules across 29 WCAG success criteria, plus the contrast checks. The Marketplace description gives the figure as 70 rules across 28 criteria. See Rules.

Will a clean result make my site WCAG compliant?

No. Static checks cover only part of WCAG, and many criteria need a person to test with a keyboard, a screen reader, zoom and real content. Use the manual checks for the rest. The score is for prioritising and tracking progress, not a certification.

Which WCAG version does it check?

WCAG 2.2 by default. You can choose 2.0, 2.1 or the 3.0 draft (which adds APCA). See Audit profiles and contrast.

Does it work with React, Vue, Svelte and Astro?

Yes. It reads JSX and TSX, Vue, Svelte and Astro files, understands framework attribute syntax and bound values, and does not judge components such as `<Button>` as native elements. It cannot see markup produced by JavaScript at runtime.

Does it check plain JavaScript files?

Plain `.js` and `.ts` files get contrast checks for CSS-in-JS, not the markup rules; put JSX in `.jsx` or `.tsx` files to get both. Inspect works in all four.

Why was a finding shown as a hint "Best practice"?

Six rules (`accesskey`, `autofocus`, `heading-skip`, `positive-tabindex`, `tiny-font`, `text-spacing-important`) and unreferenced duplicate IDs flag patterns worth fixing that do not by themselves fail WCAG. They are shown as hints and weigh half in the score.

Can it fix problems automatically?

Only where a rewrite is safe and deterministic, for 13 rules such as adding `lang="en"` or removing `accesskey`. Judgement calls such as alt-text quality are only suggested. See Apply fix.

How do I ignore a false positive?

Use **Ignore here** in the hover, the **Ignore this WCAG … finding here** quick fix, or an `auric-disable-next-line <rule-id>` comment. See Suppression and branding.

Can I mark a brand colour so it does not hurt the score?

Yes, with `auric-brand-line` or **Mark Line as Intentional Branding (Toggle)**. Only colour findings can be classified; structural findings always count.

Does it use axe-core?

No. It has its own engine. During development its results were checked against axe-core on rendered HTML; see Limits and accuracy.

Can I run the same checks in CI?

Yes, with the bundled `auric-artisan` command-line scanner. See The CI gate.

Does it send my code anywhere?

No. See Privacy.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility: other resources

Glossary, shortcuts, limits and accuracy, related pages, release notes and privacy for Auric Accessibility.

Supporting pages for Auric Accessibility. Each points to the fuller Studio page where one exists.

File	What it covers
Glossary	Accessibility terms used in these docs.
Shortcuts	Keyboard shortcuts and Command Palette titles.
Limits and accuracy	What the rules can and cannot judge, and how they were checked.
Related	Related extensions, tools and standards.
Release notes	Auric Accessibility's history from the changelog.
Privacy	What it stores and writes.

Back to the Auric Accessibility overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility shortcuts

Auric Accessibility's keyboard shortcuts, Command Palette titles and Studio keys.

Keyboard shortcuts

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `A`	`Cmd` + `Alt` + `A`	Inspect Accessibility of Element (editor focused)
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center
`Ctrl` + `.`	`Cmd` + `.`	VS Code's quick fixes, including **Ignore this WCAG ... finding here**
`Ctrl` + `Shift` + `M`	`Cmd` + `Shift` + `M`	VS Code's Problems panel

Command Palette titles

Type `Auric Artisan` in the Command Palette:

- Auric Artisan: Open Accessibility Studio
- Auric Artisan: Workstation: Command Center
- Auric Artisan: Open Rulebook & Knowledge Database
- Auric Artisan: Build Workspace Knowledge Database (JSON + Markdown)
- Auric Artisan: Open Unified Icon Studio
- Auric Artisan: Scan Project for Accessibility (Use Cache)
- Auric Artisan: Force Rescan Project and Refresh Cache
- Auric Artisan: Generate Agent-Ready Remediation Plan
- Auric Artisan: Clear Workspace Scan Cache
- Auric Artisan: Search Accessibility Issues

- Auric Artisan: Group Accessibility Issues (Type / File / Severity)
- Auric Artisan: Clear Accessibility Issue Search
- Auric Artisan: Check Contrast of Selection
- Auric Artisan: Inspect Accessibility of Element
- Auric Artisan: Mark Line as Intentional Branding (Toggle)

Open WCAG Documentation and **Open Issue in Contrast Studio** are row buttons in the Project Issues view, not palette commands.

In the Studio

Keys	Where	What it does
`/`	Findings	Focus the search box
`Escape`	Findings	Close pop-ups, then clear the search
`Enter`, `Space`	A finding row	Open the file at that line
Arrow keys	Rail and tab rows	Move between spaces and tools

For every Studio key, see the Studio shortcuts.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility glossary

Accessibility terms used in the Auric Accessibility documentation.

Accessibility tree. The structure the browser gives assistive technology: each element's role, name, state and relationships.

Accessible name. What assistive technology announces for an element, from its text, `aria-label`, `aria-labelledby`, `alt`, `title` or a `<label>`.

ACT-aligned. Structured like the W3C's Accessibility Conformance Testing rules format, without claiming ACT conformance.

APCA and Lc. The contrast model proposed for the WCAG 3 draft, and its lightness contrast value. Experimental in the Studio.

ARIA. Attributes such as `role`, `aria-label` and `aria-expanded` that describe elements to assistive technology. Checked against WAI-ARIA 1.2.

Assisted. A finding a person should confirm; it weighs less in the score.

Audit profile. The WCAG version targeted: 2.0, 2.1, 2.2 or 3.0 draft.

Autofill token. A value of the `autocomplete` attribute, such as `email` or `current-password`, from the HTML autofill vocabulary.

Best-practice hint. A rule that flags a pattern worth fixing that does not by itself fail WCAG.

Component. A capitalised, dotted or namespaced tag in JSX, Vue, Svelte or Astro, such as `<Button>`, which is not judged as a native element.

Confidence. How sure a rule is that its finding is real: high, medium or low.

Impact. How much a finding affects users: critical, serious, moderate or minor.

Inspect. The hover card that shows an element's or rule's name, role, colours, contrast and issues.

Landmark. A region such as `main`, `nav`, `header` or `footer` that assistive technology can jump between.

Large text. At least 24 CSS pixels, or 18.66 pixels and bold; it has a lower contrast target.

Level. A success criterion's conformance level: A, AA or AAA. Most policies require A and AA.

Manual check. A procedure a person carries out for what automation cannot decide.

Role. What kind of thing an element is, such as a button, link or heading, either from its tag or from `role`.

Success criterion. A single testable WCAG requirement, such as 1.1.1 Non-text Content.

Suppression. Removing a finding with an `auric-disable-…` comment.

WCAG. The Web Content Accessibility Guidelines, the W3C's accessibility standard.

For all other terms, see the Studio glossary.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility limits and accuracy

What Auric Accessibility's rules can judge, how their accuracy was measured, and what needs a person.

What it does well

- Reads markup with one shared tag reader that understands quoted values and JSX braces, so arrow functions and template expressions never cut a tag short, and matches attribute names whole.
- Tells components (`<Input>`, `<Form.Label>`, `<mat-form-field>`) apart from native elements, and treats a file without `<html>` or `<body>` as a component or partial rather than a whole page.
- Ignores comments, script and style contents, PHP blocks and strings that only look like markup, while keeping reported lines and columns exact.
- Treats values supplied at runtime (`{{id}}`, `${id}`, `:attr`, `[attr.x]`, `attr={…}`) as unknown, not wrong.
- Checks ARIA against the WAI-ARIA 1.2 role matrix, HTML-AAM implicit roles and the HTML autofill grammar.

Measured accuracy

During development, the rules were checked against axe-core on rendered HTML. On a regression suite of 264 cases, false positives fell from 132 to 0 and false negatives from 21 to 0. Of 83 rendered cases, 80 agree with axe; the three that differ are contested or best-practice cases.

What it cannot judge

- Anything decided at runtime: markup generated by JavaScript, class-driven semantics, component state, computed styles and the full CSS cascade.
- Relationships across files, such as a label defined in another component.
- Quality and meaning: whether alt text is accurate, a link's purpose is clear, a heading describes its section, or an authentication flow is usable. These rules suggest a fix to review, never an automatic repair.
- Most of WCAG's operable and understandable requirements in practice, such as focus order, keyboard traps and status announcements. See Manual evidence.

What the score is

A prioritisation and progress measure over what static analysis can see. It is not a certification, and a clean result does not mean the product meets WCAG.

Limits

- Files larger than 1 MB are not checked.
- The Inspect card sees only the current file's styles.
- Suppressed findings are removed from results and scores; branding marks never hide structural findings.

For contrast and scanning limits, see the Studio limits and accuracy page.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility privacy

What Auric Accessibility stores and writes, and when it uses the network.

Auric Accessibility analyses your code on your machine. It sends no code, results or telemetry anywhere and needs no account.

What it writes

Where	What
`.auric-artisan/` in your workspace	The scan cache, a settings copy, the health record, SARIF and history after scans, the remediation plan, and the knowledge files you build
Your settings	The accent colour (user settings) and values you save in Studio › Settings (workspace settings)
VS Code's storage for the extension	The Project Issues grouping, dismissed suggestions, collapsed Studio sections, and small colour icons for tree rows
Files you choose	The WCAG and accessibility Markdown exports

When the network is used

Only when you open a link yourself: **WCAG docs** and **Open WCAG Documentation** open the W3C's *Understanding* pages in your browser, and Auric Artisan marks open auricartisan.com.

See the Studio privacy page for details.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Auric Accessibility release notes

Auric Accessibility's version history, taken from the Studio changelog.

Auric Accessibility shares one version number and one changelog with the other Studio extensions. This page lists the entries that concern accessibility. The complete history is in the Studio release notes.

Unreleased

Listed under **Unreleased** in the changelog, as upcoming entries for the next numbered release (detector 1.2.0):

- The rules were checked against axe-core on rendered HTML: on a 264-case regression suite, false positives fell from 132 to 0 and false negatives from 21 to 0; 80 of 83 rendered cases agree with axe.
- One tag reader for every rule: a JSX `=>` or a Vue `v-if="n > 0"` no longer cuts a tag short; attributes are whole names; bound values are respected.
- Components such as `<Input>`, `<Button>` and `<Form>` are no longer judged as native elements.
- Correct markup no longer fails: required owned roles on `<ul role="list">`, radio groups and table rows; required context in component files; `aria-selected` on `option`; `lang` on `<style lang="scss">`; image alt with `aria-label` or spreads; Razor, Material and shadcn labels; unnamed forms; duplicate IDs inside `<template>`.
- Accessible names now come from labelled `svg` and `img` children, template output and `v-text`; `aria-hidden` text no longer counts.
- `focus-outline-none`: one `:focus-visible` rule no longer exempts a whole file.
- `interactive-keyboard` understands `tabIndex={0}` and catches Vue, Angular and Svelte click handlers.
- New rule `input-placeholder-label` (3.3.2, advisory).
- Six rules became best-practice hints, weighted at half: `accesskey`, `autofocus`, `heading-skip`, `positive-tabindex`, `tiny-font`, `text-spacing-important`.
- Performance: a case that never finished (60,000 links) now takes 0.28 s; 2,930 real markup files scan in 6.2 s instead of 7.6 s.
- The contrast scanner, hovers and Contrast Studio changes listed in the Studio notes also apply, because Auric Accessibility publishes contrast diagnostics.
- The Studio's new Home and Check spaces, and the **pattern-rule** suggestion now opening **Check › Findings**.

0.1.0 — Initial Studio release

- A specification-backed WCAG 2.2 engine with structural, semantic, ARIA, language, form, media, focus, motion and contrast checks.
- Inspect for element-level names, roles, colours, contrast, colour-vision impact, issues and fixes.
- Workspace scans, grouped findings, score summaries, suppressions, quick fixes, remediation plans and rule documentation.
- Framework and template awareness, so comments, runtime bindings and inert source are not reported.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC ACCESSIBILITY

Related to Auric Accessibility

Extensions, Studio pages, website tools and standards related to Auric Accessibility.

Other Studio extensions

- Auric Color: colour hovers, pickers, palettes and contrast tools.
- Auric Code Health: secrets, security and hygiene checks, health records and reports.
- The Full Pack: installs all three.

Shared Studio pages

- Scanning and Scores and grades
- The remediation plan
- Rulebook and knowledge database
- The CI gate

Website tools

- Analyzer
- Contrast Checker
- Accessibility Checklist
- Accessibility Compliance
- Vision Simulation
- The browser extension, for rendered pages.

Standards

- WCAG 2.2: https://www.w3.org/TR/WCAG22/
- Understanding WCAG 2.2: https://www.w3.org/WAI/WCAG22/Understanding/
- WAI-ARIA 1.2: https://www.w3.org/TR/wai-aria-1.2/
- HTML Accessibility API Mappings: https://www.w3.org/TR/html-aam-1.0/
- ACT Rules Format 1.1: https://www.w3.org/TR/act-rules-format/

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health — Secrets, TODOs & Risk

What Auric Code Health does, who it is for, how to install it, and a quick start from first finding to CI record.

Auric Code Health is a language-agnostic hygiene scanner for every file in your repository. It finds hardcoded secrets and credentials in URLs, security footguns such as disabled TLS verification, weak hashing, unsafe deserialization, command and SQL injection patterns and XSS sinks, risky APIs such as `eval`, forgotten debug statements, task markers such as `TODO` and `FIXME`, unresolved merge-conflict markers, and oversized lines and files. It works in more than 40 languages.

It reads source text with argument-aware analysis: it can tell a constant from a dynamic value, so `eval('use strict')` is downgraded while `eval(userInput)` is reported with high confidence. Every finding has a stable signature ID, a confidence level and a fix, and any secret it finds is redacted before it reaches the editor, the Studio, reports or files. Findings appear as you type, in a Code Health Studio with filters, and in `health.json` and SARIF 2.1.0 records for continuous integration (CI), with a bundled command-line scanner for pull-request gates.

Auric Code Health is one of the four Auric Artisan Studio extensions. It works on its own and joins the others in one Workstation. Everything runs on your machine.

Who it is for

- Developers in any language who want leftovers, secrets and risky patterns caught before they are committed.
- Reviewers and tech leads who want a repeatable hygiene score for a repository.
- Teams who want SARIF in GitHub code scanning and a CI gate that fails only on new issues.

It is a high-signal heuristic linter, not a full security scanner. Use it alongside, not instead of, dedicated security tools where compliance requires them.

Where to find it

- Marketplace ID: `auric-artisan.auric-codehealth`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-codehealth
- Install: `code --install-extension auric-artisan.auric-codehealth`
- Version 0.1.0; VS Code 1.74 or newer. Also installed by the Full Pack.

Quick start

1. Install Auric Code Health from the Extensions view (`Ctrl` + `Shift` + `X`).
2. Open a project. Findings in open files are underlined and listed in the Problems panel with the source `auric-artisan`.
3. Hover a finding for its title, category, confidence, priority and fix. Press `Ctrl` + `.` for **Ignore this … finding here**.
4. Select the **Auric Code Health** icon in the Activity Bar. The Overview shows the code-health ring and seven category chips.
5. Select **Report**. The Studio scans the project and writes the project health report; select **Open in Browser** to read the printable page.
6. Run **Auric Artisan: Open Code Health Studio** to filter findings by category, confidence and text.
7. Select **health.json + SARIF** to write the CI records to `.auric-artisan`.
8. Read The CI gate to fail pull requests on new issues.

What you can do

- Find 24 kinds of hardcoded secret, including AWS, GitHub, GitLab, Slack, Google, Stripe, OpenAI, Anthropic, npm, SendGrid and DigitalOcean credentials, private keys, JWTs, webhook URLs, credentials in URLs and connection strings, and secret-named assignments, and add your own patterns.
- Find disabled TLS verification, weak MD5 and SHA-1 hashing, unsafe deserialization, string timers, raw-HTML sinks, wildcard CORS, and command and SQL injection built from dynamic values.
- Find risky APIs, leftover debug statements, task markers, merge markers, lines over 240 characters and files over 800 lines.
- See confidence on every finding, and filter by it.
- Silence one finding, one category or one file with a comment, inserted for you in the right syntax.
- Score the repository from 0 to 100 with a letter grade.
- Write `health.json`, SARIF 2.1.0 and a trend history, and a Markdown, JSON and HTML project health report covering colour, accessibility and code health.
- Gate CI with the bundled scanner and a baseline, so only new issues fail a build.
- Generate a local remediation plan with secret and security tasks.
- Search the code-health rulebook, policies and official OWASP, CWE and NIST sources.

In this folder

Folder or file	What it covers
docs/	Guides and reference for Auric Code Health.
docs/getting-started.md	First run and a tour.
docs/categories-and-signatures.md	Every category and signature, what it detects and where.
docs/secrets-and-redaction.md	How secrets are found, filtered, scored and redacted.
docs/code-health-studio.md	The Health space in the Studio and the Overview.
docs/health-records.md	`health.json`, SARIF, history, fingerprints and baselines.
docs/reference.md	Every command, keybinding, setting and view.
docs/troubleshooting.md	Symptoms and fixes.
docs/faq.md	Short answers.
others/	Glossary, shortcuts, limits and accuracy, related pages, release notes and privacy.

Shared Studio features are in the Studio docs.

Back to the Studio overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health docs

The guides and reference pages for Auric Code Health.

These pages cover Auric Code Health's own features. Start with Getting started. Features shared with the other Studio extensions (the Overview, the Command Center, scanning, scores, suppression, exports, the remediation plan and the CI gate) are in the Studio docs.

File	What it covers
Getting started	Install, first finding, first report and a tour.
Categories and signatures	The seven categories and every signature: what it detects, in which languages, and its confidence.
Secrets and redaction	Secret detection, placeholder filtering, custom patterns, confidence and redaction.
The Code Health Studio	The Health space: score, categories, filters and findings.
Health records	`health.json`, SARIF and history for CI, fingerprints and baselines.
Reference	All 10 commands, the keybinding, all 28 settings and the views.
Troubleshooting	Symptoms, causes and fixes.
FAQ	Short answers.

Back to the Auric Code Health overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Getting started with Auric Code Health

Install Auric Code Health, read your first finding, grade the repository and write your first CI record.

Install

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Search for `auric-artisan.auric-codehealth` and select **Install**. Or run `code --install-extension auric-artisan.auric-codehealth`.

It starts when VS Code finishes loading. You need VS Code 1.74 or newer. All seven categories are on by default.

Your first finding

1. Open any source file, in any of the 40+ supported languages.
2. Findings are underlined and listed in the Problems panel with the source `auric-artisan` and a code such as `code-health:debug`. For example:

```
Leftover debug statement (console.).  —  Remove before committing, or route through a real logger.
```

3. Hover the underline. The card shows the signature's title (for example **Console debug output**), a severity pill, the category (for example **Debug leftovers**), the confidence, a priority, the location, requirement tags such as `CWE-798` or `OWASP-A05:2025` where they apply, the line of code, the fix, and actions: **Go to code**, **Rule details**, **Ignore here** and **Copy reference**. The footer shows the signature ID and detector version, such as `ch-debug-console v1.2.0`.

4. Press `Ctrl` + `.` for **Ignore this debug finding here** (inserts `auric-disable-next-line debug` above) or **Ignore all debug findings in this file**.

Secret findings never show the secret itself; the evidence reads, for example, `[redacted secret; 40 characters]`. See Secrets and redaction.

Grade the repository

1. Select the **Auric Code Health** icon in the Activity Bar. The **Overview** shows a **Code health** ring (empty until the first grade) and seven chips: **Secrets**, **Security**, **Risky APIs**, **Debug**, **TODOs**, **Conflicts** and **Large files**.
2. Select **Report**. The Studio scans the workspace (or reuses a fresh cache) and writes the project health report to `.auric-artisan`.
3. The ring shows the code-health score and grade, and each chip its count. The status bar shows, for example, `Health 88 B`.
4. In the message that appears, select **Open in Browser** to read the printable report.

See Scores and grades for how the score is calculated.

Explore the findings

Run **Auric Artisan: Open Code Health Studio** (or **Studio** on the Overview, or the status-bar item). The **Health** space shows the score, the seven categories as filters, confidence filters, a search box and every finding. See The Code Health Studio.

Write the CI records

Select **health.json + SARIF** on the Overview, or run **Auric Artisan: Write Health Record (health.json + SARIF)**. The Studio writes `health.json`, `health.sarif` and `health-history.json` to `.auric-artisan` and opens `health.json`. See Health records and The CI gate.

Where Auric Code Health appears

Place	What you get
Editor	Code-health underlines, hovers and ignore quick fixes
Activity Bar, **Auric Code Health**	**Overview** (the code-health layout)
Status bar	`Health` and the score, when it is the only Studio extension
Studio (**Open Code Health Studio**)	**Health** and **Settings**
Command Center (`Ctrl` + `Alt` + `.`)	Project health report, health record, data folder, remediation plan, worst findings, written files

With the other Studio extensions

When Auric Accessibility or Auric Color is installed, one of them hosts the Workstation. Auric Code Health hides its own views, adds its **Health** space, **Report**, **SARIF** and the CI gate card to the host, and keeps publishing code-health underlines. See The Workstation.

Next steps

- Categories and signatures
- Secrets and redaction
- The remediation plan

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Categories and signatures

The seven code-health categories and every signature Auric Code Health detects, with the languages it checks and its confidence.

Auric Code Health groups its checks into seven **categories**. Within each, a **signature** is one specific detector with a stable ID, such as `ch-secret-github-token`. The ID stays the same when the wording of a message improves, so records, baselines and suppression comments keep working. Every finding also records the detector version (1.2.0 in the current package).

The seven categories

Category	ID	Problems severity	Hover severity	Score weight	Setting
Possible secrets	`secret`	Warning	Critical	14	`codeHealth.secret`
Security risks	`security`	Warning	Serious	12	`codeHealth.security`
Merge markers	`conflict`	Error	Critical	10	`codeHealth.conflict`
Risky APIs	`risk`	Warning	Serious	4	`codeHealth.risk`
Debug leftovers	`debug`	Information	Minor	2	`codeHealth.debug`
Task markers	`todo`	Hint	Minor	0.25	`codeHealth.todo`
Oversized	`size`	Hint	Minor	0.5	`codeHealth.size`

Each setting is `auricCodeHealth.codeHealth.<category>` and is on by default. `auricCodeHealth.codeHealth.enabled` turns all of them off at once.

How the checks read code

- A small per-language reader marks comments and strings. Task markers must be inside comments; debug, risk and security signatures ignore commented-out or quoted code. Secrets are found everywhere, including comments, because a key in a comment is still leaked.
- Signatures run only in the languages where they mean something. In a language the Studio does not know, the checks behave language-agnostically.
- **Argument-aware** signatures read the call's argument or assigned value: a dynamic value (a variable, concatenation or template with `${}`) gives **high** confidence; a constant gives **low** confidence (or is skipped where a constant is harmless); an argument that cannot be read gives **medium**.
- A file reports at most 300 code-health findings.

In the tables, *web languages* means JavaScript, TypeScript, JSX, TSX, Vue, Svelte, Astro, HTML, EJS, Handlebars, Twig, Razor, PHP and CoffeeScript.

Possible secrets

All secret signatures map to CWE-798 (hard-coded credentials). Every match is redacted; see Secrets and redaction.

Signature	Title	Detects
`ch-secret-aws-access`	AWS access key	An AWS access key ID (`AKIA` followed by 16 characters)
`ch-secret-aws-temporary`	AWS temporary access key	A temporary AWS key (`ASIA` followed by 16 characters)
`ch-secret-private-key`	Embedded private key	A PEM private key block (RSA, EC, DSA, OpenSSH, encrypted or PGP)
`ch-secret-github-token`	GitHub token	A GitHub token starting `ghp_`, `gho_`, `ghu_`, `ghs_` or `ghr_`
`ch-secret-github-pat`	GitHub fine-grained token	A token starting `github_pat_`
`ch-secret-gitlab-token`	GitLab access token	A GitLab personal access token (`glpat-`)
`ch-secret-slack-token`	Slack token	A Slack token (`xoxb-`, `xoxp-` and similar, or `xapp-`)
`ch-secret-google-api-key`	Google API key	A Google API key (`AIza` followed by 35 characters)
`ch-secret-google-oauth`	Google OAuth client secret	A Google OAuth client secret (`GOCSPX-`)
`ch-secret-stripe-live`	Stripe live secret	A live Stripe secret key (`sk_live_`)
`ch-secret-stripe-key`	Stripe secret or restricted key	A Stripe test secret or restricted key (`sk_test_`, `rk_test_`, `rk_live_`)
`ch-secret-anthropic-key`	Anthropic API key	An Anthropic API key (`sk-ant-`)
`ch-secret-openai-key`	OpenAI API key	An OpenAI key (`sk-proj-`, `sk-svcacct-`, `sk-admin-`, or `sk-` and 32+ characters)
`ch-secret-npm-token`	npm access token	An npm token (`npm_` and 36 characters)
`ch-secret-sendgrid-key`	SendGrid API key	A SendGrid key (`SG.` with two dotted parts)
`ch-secret-digitalocean-token`	DigitalOcean token	A DigitalOcean token (`dop_v1_` and 64 hex characters)
`ch-secret-slack-webhook`	Slack webhook secret	A Slack incoming-webhook URL with its secret path
`ch-secret-discord-webhook`	Discord webhook secret	A Discord webhook URL with its token
`ch-secret-jwt`	JSON Web Token	A three-part token starting `eyJ` (medium confidence)
`ch-secret-credential-url`	Credentials embedded in URL	`scheme://user:password@host` for HTTP(S), Postgres, MySQL, MongoDB, Redis, AMQP, FTP, SSH, SMTP
`ch-secret-connection-string`	Password in a connection string	`Password=`, `Pwd=`, `AccountKey=` or `SharedAccessKey=` in a connection string (Azure, ADO.NET)
`ch-secret-auth-header`	Hard-coded Authorization header	A `Bearer` or `Basic` token written into a string
`ch-secret-assignment`	Hardcoded credential assignment	A credential-named key assigned a literal value (medium confidence; see below)
`ch-secret-custom`	Workspace custom secret signature	A match of one of your `codeHealth.customSecretPatterns` (medium confidence)

Credential assignments are keys whose name ends in a credential word, such as `password`, `passwd`, `pwd`, `passphrase`, `secret`, `token`, `bearer`, `api_key`, `api_secret`, `secret_key` or `private_key` (so `DB_PASSWORD` and `adminPassword` count,

but `password_hint` does not), assigned a value of 8 to 500 characters. Quoted and typed keys, `=`, `:`, `:=` and `=>` are all understood, as are unquoted values in YAML, INI, properties, `.env`, shell and TOML files, and XML settings such as `<add key="ApiKey" value="…"/>` and `<password>…</password>`.

The message for each is short, for example **Looks like a GitHub token.**, and the fix is **Move secrets to env vars / a secret manager and rotate any that were committed.**

Security risks

All security signatures map to OWASP Top 10:2025 A05 and have high confidence unless noted. The fix is **Harden or remove this — see the linked guidance.**

Signature	Title	Detects	Languages
`ch-sec-tls-node-option`	Node TLS verification disabled	`rejectUnauthorized: false`	Any
`ch-sec-tls-node-env`	Process-wide Node TLS bypass	`NODE_TLS_REJECT_UNAUTHORIZED` set to `0`	Any
`ch-sec-tls-python`	Python TLS verification disabled	`verify=False`, `ssl._create_unverified_context()`, `check_hostname=False`, `CERT_NONE`	Any, outside strings
`ch-sec-tls-go`	Go TLS verification disabled	`InsecureSkipVerify: true`	Any, outside strings
`ch-sec-tls-php`	PHP cURL TLS verification disabled	`CURLOPT_SSL_VERIFYPEER` or `CURLOPT_SSL_VERIFYHOST` set to false or 0	PHP
`ch-sec-weak-crypto-node`	Weak Node cryptographic hash	`createHash('md5')` or `createHash('sha1')`	Any, outside strings
`ch-sec-weak-crypto-python`	Weak Python cryptographic hash	`hashlib.md5`, `hashlib.sha1`, `hashlib.new('md5')`; not with `usedforsecurity=False`	Any, outside strings
`ch-sec-weak-crypto-java`	Weak Java / .NET cryptographic hash	`MessageDigest.getInstance("MD5")` or SHA-1, `DigestUtils.md5`, `MD5.Create()`	Java, Kotlin, Scala, Groovy, C#, F#, VB
`ch-sec-weak-crypto-php`	Weak PHP password hash	`md5()` or `sha1()` of a `$…pass`, `$…pwd` or `$…secret` variable	PHP
`ch-sec-deserialize-python`	Unsafe Python deserialization	`pickle`, `cPickle` or `marshal` `load` and `loads`	Any, outside strings
`ch-sec-yaml-load`	Unsafe YAML deserialization	`yaml.load`, `load_all`, `full_load`, `unsafe_load`; not with `SafeLoader`, not js-yaml	Any, outside strings
`ch-sec-php-unserialize`	Unsafe PHP deserialization	`unserialize()`, unless `allowed_classes` is false	PHP
`ch-sec-string-timer`	String-evaluated timer	`setTimeout` or `setInterval` given a string	Web languages
`ch-sec-vue-html`	Vue raw HTML sink	The `v-html` directive	Any
`ch-sec-insert-html`	insertAdjacentHTML sink	`insertAdjacentHTML` with dynamic, unsanitised markup	Web languages
`ch-sec-wildcard-cors`	Wildcard CORS policy	`Access-Control-Allow-Origin` set to `*`	Any
`ch-sec-command-injection`	Dynamic shell command	`child_process` `exec` with a dynamic command, or `spawn` and `execFile` with `shell: true`	JavaScript family

Signature	Title	Detects	Languages
`ch-sec-sql-template`	Interpolated SQL query	SQL built by interpolation: JS templates, Python f-strings, C# `$""`, PHP `"$x"`, Ruby `#{}`	Any
`ch-sec-sql-concat`	Concatenated SQL query	SQL built with `+`, PHP `.`, Python `%` or `.format`, or `sprintf`-style formatting	Any

Notes:

- **Command injection** follows how `child_process` was imported: `require`, `import`, aliases, destructuring and `promisify`. A constant command is not reported; an unreadable one is reported at medium confidence.
- **SQL** signatures look for upper-case SQL shapes (`SELECT … FROM`, `INSERT INTO`, `UPDATE … SET`, `DELETE FROM`, `DROP`, `CREATE` or `ALTER TABLE`, `UNION SELECT`, `MERGE INTO`), so ordinary prose is ignored. Tagged templates such as `sql`…`` are parameterised and not reported.
- **insertAdjacentHTML** is skipped when the markup is a constant or passes through a sanitiser (a function named like `sanitize…`, `escapeHtml`, `filterXSS`, `xss` or `purify`).

Risky APIs

Signatures with **contextual** confidence are argument-aware. The fix is **Review this call for safer alternatives.**

Signature	Title	Detects	Languages	Confidence
`ch-risk-eval`	Dynamic eval	`eval(…)`, including `window.eval` and `globalThis.eval`	Any, outside strings	contextual
`ch-risk-function-constructor`	Function constructor	`new Function(…)`	Web languages	contextual
`ch-risk-innerhtml`	Dynamic innerHTML assignment	Assigning a dynamic value to `innerHTML` or `outerHTML`; constants and sanitised values are skipped	Web languages	contextual
`ch-risk-react-html`	React raw HTML sink	`dangerouslySetInnerHTML`	JSX, TSX	medium
`ch-risk-document-write`	document.write usage	`document.write()` or `writeln()`	Web languages	medium
`ch-risk-shell`	Shell execution	`os.system`, `os.popen`, `subprocess.getoutput`, or `subprocess` calls with `shell=True`	Python	contextual

A constant argument adds **(constant argument — lower risk)** to the message and lowers the confidence.

Debug leftovers

The message is **Leftover debug statement (…).** and the fix **Remove before committing, or route through a real logger.** Normal output such as `print`, `echo` and `fmt.Println` is deliberately not reported. Commented-out and quoted code is ignored. Confidence is medium.

Signature	Title	Detects	Languages
`ch-debug-console`	Console debug output	`console.log`, `debug`, `trace`, `dir` or `table`	Web languages
`ch-debug-debugger`	Debugger statement	`debugger`	Web languages
`ch-debug-rust-dbg`	Rust dbg! output	`dbg!(…)`	Rust

Signature	Title	Detects	Languages
`ch-debug-php-dump`	PHP diagnostic dump	`var_dump`, `print_r`, `var_export`	PHP
`ch-debug-ruby`	Ruby debugger hook	`binding.pry`, `binding.irb`, `byebug`	Ruby
`ch-debug-python`	Python debugger hook	`pdb.set_trace()`, `ipdb.set_trace()`, `pudb.set_trace()`, `breakpoint()`	Python
`ch-debug-nslog`	NSLog debug output	`NSLog(…)`	Objective-C, Objective-C++, Swift
`ch-debug-java-system-out`	Java System.out debug output	`System.out.print` and `println`	Java, Groovy, Scala, Kotlin
`ch-debug-go`	Go debug print	`fmt.Print`, `Println` or `Printf` whose text starts with `DEBUG`	Go

Task markers

Signature	Marker
`ch-todo-todo`	TODO
`ch-todo-fixme`	FIXME
`ch-todo-hack`	HACK
`ch-todo-xxx`	XXX
`ch-todo-bug`	BUG
`ch-todo-optimize`	OPTIMIZE
`ch-todo-refactor`	REFACTOR
`ch-todo-deprecated`	DEPRECATED

- In languages the Studio can lex, an upper-case marker counts only inside a comment. Lower-case `@todo`, `todo:` and `fixme:` count inside comments too. Rust's `todo!()` and Kotlin's `TODO()` count in code.
- In markup files, a marker counts inside a comment, and `TODO`, `FIXME` and `HACK` count anywhere.
- The message quotes the text after the marker, for example `TODO: remove the legacy header`. The fix is **Resolve or convert into a tracked issue.** Confidence is medium.

Merge markers

Signature	Title	Detects
`ch-conflict`	Unresolved merge marker	A line starting with seven `<` or `>` characters, or seven equals signs or vertical bars inside a conflict

A lone `=======` line outside a conflict, such as a Markdown heading underline, is not reported. The message is **Unresolved merge-conflict marker.** and the fix **Resolve the conflict and remove the marker.**

Oversized

Signature	Title	Detects	Where
`ch-longline`	Oversized source line	A line longer than 240 characters, except lines that start with a URL	Editor and scans
`ch-bigfile`	Large file	A file with more than 800 lines, reported on line 1	Project scans and the command-line scanner

Messages: **Very long line (312 chars) — hard to read and review.** and **Large file (1204 lines) — consider splitting.**

Search the rulebook

Every signature, the seven categories, four policies (on confidence, redaction, stable identities and suppression review) and two articles are in the knowledge database. Run **Auric Artisan: Open Rulebook & Knowledge Database** and search for a signature ID or a word such as `secret`, or select **Rule details** in a finding's hover. The **Code-health balanced baseline** rulebook brings them together. See Rulebook and knowledge database.

Related

- Secrets and redaction
- Suppression and branding
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

The Code Health Studio

Use the Studio's Health space and the Overview to see the code-health score, filter findings by category and confidence, and act on them.

The Studio's **Health** space is Auric Code Health's full view of the repository: its score and grade, the seven categories, and every finding with filters. When Auric Code Health is installed alone, the Studio opens on it; in a Workstation, it is the **Health** space of the host's Studio.

Open it

- **Auric Artisan: Open Code Health Studio**, or **Studio** on the Overview, or the status-bar item.
- **Open Studio: Code Health** in the Command Center.

The tab is titled **Auric Artisan — Code Health Studio** (or **Workstation Studio** in a Workstation). The space header reads **Repository intelligence** with the summary **Find secrets, security footguns, debug leftovers, risky APIs, task debt, and oversized code.**, and a chip showing how long ago the workspace was scanned.

Before the first scan

A **Code Health Studio** card explains what will be checked and offers **Scan Code Health** (a fresh scan) and **Open data folder**.

After a scan

Header actions

Button	What it does
health.json + SARIF	Writes the health record. See Health records.
Report	Generates the project health report (Markdown, JSON and HTML).
Rescan (gold)	Scans the workspace again and refreshes the cache.

Repository hygiene and security

- **Data folder**: shows `.auric-artisan` in your file manager.
- **Fix plan for an agent**: generates the remediation plan, with a task for each secret, security and risky-API finding.
- A score ring with the code-health score and grade (A+ to F).
- Four figures: **files**, **findings**, **high confidence** and **secret + security**.

Seven health categories

A button for each category with its count and description, in this order:

Category	Description
Possible secrets	Hardcoded API keys, tokens or passwords.
Security risks	Weak crypto, TLS bypass, unsafe deserialization, XSS sinks.
Merge markers	Unresolved merge-conflict markers.
Risky APIs	eval, innerHTML and other footguns.
Debug leftovers	Forgotten debug / print statements.
Task markers	TODO / FIXME / HACK left in the code.
Oversized	Very long lines or large files.

Secrets, security and merge markers are shown in the fail colour; risky APIs and debug in the warning colour. Select a category to show only its findings; select **All** in the filter bar to show every category again.

Findings

- Filter chips: **All** (with the total), **Any confidence**, **High**, **Medium** and **Low** (each with its count).
- A search box, **Filter rule, message, file...**, matching the signature ID, category, message, fix, evidence, file, confidence and severity. Secret evidence is never searchable.
- A count such as `42 of 118 shown`. Up to 300 findings are listed at a time.
- Each row shows the category, the confidence, the signature ID, the message, **Fix:** with the fix, the evidence (or **Secret evidence redacted**), and the file and line. Select a row, or press `Enter`, to open the file at that line.

When nothing matches, the list says **No Code Health findings match the current filters.** A clean project shows **No Code Health findings. All seven enabled categories are clean in the current scan.**

Press `/` to jump to the search box and `Escape` to clear it.

The Overview

When Auric Code Health is alone, its Overview is the code-health layout: a **Code health** ring and row, seven chips (**Secrets**, **Security**, **Risky APIs**, **Debug**, **TODOs**, **Conflicts**, **Large files**), **Report** and **health.json + SARIF**, the **This file** card, suggestions and navigation to **Studio**, **Rulebook**, **Icon Studio** and **Fix plan**. See The Overview view.

With Auric Accessibility as host, the Overview gains a **Code health** row, **Report** and **SARIF**, and the Studio's Home gains a **Code health** tile and the **CI gate** card.

Settings space

With Auric Code Health alone, Studio › Settings shows **Appearance**, **Project data**, **Scan engine** and **Scope**. The category switches are in VS Code's settings (search `auricCodeHealth.codeHealth`). See Reference.

Related

- Categories and signatures
- The Studio
- Exports and health records

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Health records

Write health.json, SARIF 2.1.0 and a trend history for CI, understand how code-health findings are recorded and fingerprinted, and gate on new issues with a baseline.

Auric Code Health writes the project's findings to machine-readable files, so CI systems, dashboards and code-scanning tools can use them. The records cover every source the Studio scans: accessibility, contrast, colour vision, branding and code health.

The three files

File	Format	Purpose
`health.json`	JSON	A versioned record of scores, a summary and every issue, each with a stable fingerprint
`health.sarif`	SARIF 2.1.0	The same issues for GitHub code scanning and other SARIF tools
`health-history.json`	JSON	One entry per recorded scan, for tracking scores over time

They are written to `.auric-artisan/` in your workspace. The full formats are described in Exports and health records.

When they are written

- **Auric Artisan: Write Health Record (health.json + SARIF)**, **SARIF** or **health.json + SARIF** on the Overview, the button of the same name in the Health space, and **Write health.json + SARIF** in the Home space's CI gate card. The record opens when it is written, and a message offers **Open SARIF** and **Open Data Folder**.
- Automatically after every project scan, when `health.writeOnScan` is on (the default).
- Never during live re-grades while you type.

Settings

Setting	Default	Effect
`auricCodeHealth.health.enabled`	`true`	Maintain the record at all
`auricCodeHealth.health.writeOnScan`	`true`	Rewrite it after each project scan
`auricCodeHealth.health.formats`	`["json","sarif","history"]`	Which files to write
`auricCodeHealth.health.fileName`	`health.json`	Name of the JSON file; SARIF and history names follow it
`auricCodeHealth.health.maxIssues`	`5000`	Most issues recorded, worst first (100–100000)
`auricCodeHealth.health.historyLimit`	`200`	Most history entries kept (10–5000)

How code-health findings are recorded

Each code-health issue in `health.json` has:

- `source`: `code-health`;
- `ruleId`: the signature, for example `ch-sec-tls-python`, and `category`: `security`;
- `severity`: merge markers `high`; secrets, security and risky APIs `medium`; debug `low`; task markers and oversized `info`;
- `confidence`, `detectorVersion` (1.2.0) and `knowledgeRuleId` (the rule's entry in the knowledge database);
- `file`, `line`, `col`, `message` and `fix`;
- `match`: the evidence, which for secrets is only the redaction note; `evidenceType` (the kind of secret) and `redacted: true` for secrets.

In SARIF, each signature is a rule with its category, detector version and confidence as properties; results for secrets carry `evidenceRedacted`. Levels are `error` for high, `warning` for medium and `note` for low and info.

The `summary.codeHealthByCategory` object counts findings per category, and `scores.codeHealth` holds the code-health score.

Fingerprints and baselines

Every issue has an 8-character `id` fingerprint built from its source, signature, file, line and a key detail. For secrets, the key detail is the signature and column, never the secret. Fingerprints let a CI gate compare two runs:

1. Save the current state as a baseline once.
2. On each pull request, compare with it and fail only if there are new fingerprints.

Use the command-line scanner for this; see The CI gate. A finding that moves to another line gets a new fingerprint.

> **Note:** Code-health findings are fingerprinted with the detector version that produced them. A baseline written by an earlier detector (1.1.0) may list some existing findings as new once. Regenerate the baseline after updating.

Git information

When the workspace has a `.git` folder, the record includes the current branch and commit, read directly from the folder (no Git command is run). SARIF includes them as version-control provenance, so code scanning can link results to the

commit.

Related

- Exports and health records
- The CI gate
- Secrets and redaction

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Secrets and redaction

How Auric Code Health finds hardcoded secrets, avoids placeholders, scores confidence, supports your own patterns, and makes sure a secret is never stored.

A **hardcoded secret** is a credential, such as an API key, token, password or private key, written directly into source code or configuration. Anyone who can read the repository can use it. Auric Code Health looks for secrets in every supported file, including comments, and treats them as its most serious category.

What it finds

- **Provider credentials** with a recognisable shape: AWS access and temporary keys, GitHub and GitLab tokens, Slack tokens and webhooks, Discord webhooks, Google API keys and OAuth client secrets, Stripe secret and restricted keys, OpenAI and Anthropic API keys, npm, SendGrid and DigitalOcean tokens.
- **Private keys**: PEM blocks such as `-----BEGIN RSA PRIVATE KEY-----` followed by key data.
- **JSON Web Tokens**.
- **Credentials in URLs** (`postgres://user:password@host`) and **connection strings** (`Password=…;`, `AccountKey=…`).
- **Authorization headers** with a literal `Bearer` or `Basic` token.
- **Credential assignments**: a key named like a credential (`DB_PASSWORD`, `apiKey`, `client_secret`) given a literal value, in code, JSON, YAML, INI, `.env`, properties, TOML, shell and XML.
- **Your own patterns** from `codeHealth.customSecretPatterns`.

The full list with IDs is in Categories and signatures. When several detectors match the same text, only the most specific one is reported, so one credential is one finding.

What it deliberately ignores

To keep false positives rare, a value is not reported as a secret when it is clearly not one:

- placeholders such as `xxxxxx`, `****`, `<your-key>`, `CHANGE_ME`, `changeme`, `your_api_key`, `replace-me`, `example…`, `test…`, `dummy`, `fake`, `redacted`, `not-a-real…`;
- references to where the secret really lives: `process.env`, `import.meta`, `os.environ`, `getenv`, `${VAR}`, `$VAR`, `{{API_KEY}}`, `%VAR%`, SOPS `ENC[…]`, `vault:`, `op://`, Azure Key Vault references and ARNs;
- values that are labels, autocomplete tokens (`current-password`), validation rules, file paths to key files, or translation keys such as `auth.password`;
- sentences with spaces, and values shorter than 8 characters for name-based assignments;
- dynamic strings such as Python f-strings or C# interpolated strings with `{…}`.

Provider-shaped tokens only need to pass the obvious-placeholder check, because their shape is already specific.

Confidence

Each secret finding has a confidence:

- **high**: a provider token, or a value that looks random (high entropy, a measure of how unpredictable the characters are) or is a long unbroken token of 24 characters or more;
- **medium**: the default for name-based assignments, JWTs and custom patterns;
- **low**: a value shorter than 12 characters.

Confidence scales the score penalty (high × 1, medium × 0.75, low × 0.4) and can be filtered in the Code Health Studio. In the remediation plan, secrets are critical tasks unless their confidence is low.

Add your own patterns

For token formats specific to your organisation, add regular expressions (as strings) to `auricCodeHealth.codeHealth.customSecretPatterns`:

```
{
  "auricCodeHealth.codeHealth.customSecretPatterns": [
    "INTERNAL_[A-Z0-9]{20,}",
    "corp-tok-[a-f0-9]{32}"
  ]
}
```

- Matches are reported as `ch-secret-custom` at medium confidence, with the message **Matches a custom secret pattern.**
- Invalid regular expressions are ignored.
- Matches that look like placeholders are skipped.
- The patterns apply in the editor and in project scans. The command-line scanner does not read VS Code settings, so it does not use them.

Redaction

A detected secret is **never** stored or shown. Everywhere a finding's evidence would appear, the secret is replaced by a note giving only its length:

```
[redacted secret; 40 characters]
```

This applies to the editor hover, the Problems message, the Project Issues view, the Code Health Studio (which shows **Secret evidence redacted**), the scan cache, `health.json`, SARIF (marked `evidenceRedacted`), the history, the project health report and the remediation plan. Secret bytes are also scrubbed from other findings on the same text, such as a task marker or security finding next to the key, so they cannot leak through another category.

The underline still covers the secret's real position in the file, so you can find and remove it.

What to do about a secret

1. Remove the value from the source and load it at runtime from an environment variable or a secret manager.

2. **Rotate** the credential with its provider: a secret that was ever committed must be treated as exposed, even after you delete it.
3. Clean it from your repository's history if your policy requires it.

The knowledge database's article **Treat a committed secret as an incident, not a text-edit task** and the policy **Never persist a detected secret value** expand on this. Open them with **Open Rulebook & Knowledge Database**.

If a match is a false positive (for example a published test key), silence it with `auric-disable-line secret` or `auric-disable-next-line ch-secret-jwt` in a comment. See Suppression and branding.

Limits

Auric Code Health reads the current files only. It does not scan Git history, does not verify whether a credential is live, and is not a replacement for a dedicated secret scanner. On the labelled test set used in development, secret detection found 96% of the planted secrets with no false positives.

Related

- Categories and signatures
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health reference

Every command, keybinding, setting and view Auric Code Health contributes.

Facts for Auric Code Health 0.1.0 (`auric-artisan.auric-codehealth`). It needs VS Code 1.74 or newer and starts when VS Code finishes loading. Categories: Linters, Other.

Keybindings

Keys (Windows, Linux)	Keys (macOS)	Command	Active when
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center	Auric Code Health is not suppressed by a Workstation host

Commands

All titles appear in the Command Palette after `Auric Artisan:`.

Title	Command ID	What it does
Open Code Health Studio	`auricCodeHealth.openStudio`	Opens the Studio on Health, or the Workstation host's Studio
Workstation: Command Center	`auricCodeHealth.commandCenter`	Opens the Command Center list
Open Rulebook & Knowledge Database	`auricCodeHealth.openKnowledgeBase`	Searches signatures, categories, policies, articles and sources
Build Workspace Knowledge Database (JSON + Markdown)	`auricCodeHealth.writeKnowledgeDatabase`	Writes `knowledge.json`, `rulebook.md` and an overrides template

Title	Command ID	What it does
Open Unified Icon Studio	`auricCodeHealth.openIconStudio`	Opens the combined Auric icon library
Generate Agent-Ready Remediation Plan	`auricCodeHealth.generateAiPlan`	Writes the local remediation plan
Generate Project Health Report (Color, A11y & Code Health)	`auricCodeHealth.projectHealthReport`	Writes `project-health.md`, `.json` and `.html`
Write Health Record (health.json + SARIF)	`auricCodeHealth.writeHealthReport`	Writes `health.json`, SARIF and history
Clear Workspace Scan Cache	`auricCodeHealth.clearScanCache`	Deletes `scan-cache.json`
Open Workspace Data Folder	`auricCodeHealth.openDataFolder`	Shows `.auric-artisan` in your file manager, creating it if needed

Views

View	ID	Notes
Overview	`auricCodeHealthHome`	In the **Auric Code Health** Activity Bar container; hidden while another Studio extension hosts

Auric Code Health adds no editor right-click entries.

Settings

Auric Code Health has 28 settings. Search `auricCodeHealth` in **File › Preferences › Settings**. The data, scan, code-health and health settings are shared across installed Studio extensions; see Shared settings.

Appearance

Setting	Type	Default	What it does
`auricCodeHealth.appearance.accent`	string	`gold`	The Studio's accent: a preset ID or a hex colour. See Appearance.

Code health

Setting	Type	Default	What it does
`auricCodeHealth.codeHealth.enabled`	boolean	`true`	Runs the code-health checks at all
`auricCodeHealth.codeHealth.diagnostics`	boolean	`true`	Shows findings in the editor and Problems panel; when off, only in scans, the Studio and reports
`auricCodeHealth.codeHealth.todo`	boolean	`true`	Task markers: TODO, FIXME, HACK, XXX, BUG, OPTIMIZE, REFACTOR, DEPRECATED
`auricCodeHealth.codeHealth.debug`	boolean	`true`	Debug leftovers such as `console.log`, `debugger`, `dbg!`, `var_dump`, `breakpoint()`, `NSLog`
`auricCodeHealth.codeHealth.secret`	boolean	`true`	Possible hardcoded secrets
`auricCodeHealth.codeHealth.conflict`	boolean	`true`	Merge-conflict markers
`auricCodeHealth.codeHealth.risk`	boolean	`true`	Risky APIs such as `eval()`, `new Function()`, `innerHTML =`, `document.write()`
`auricCodeHealth.codeHealth.security`	boolean	`true`	Security footguns: TLS bypass, weak hashing, unsafe deserialization, XSS sinks, CORS, injection

Setting	Type	Default	What it does
`auricCodeHealth.codeHealth.customSecretPatterns`	array of strings	`[]`	Extra regular expressions reported as possible secrets (medium confidence)
`auricCodeHealth.codeHealth.size`	boolean	`true`	Very long lines and large files

Health record

Setting	Type	Default	What it does
`auricCodeHealth.health.enabled`	boolean	`true`	Maintains `health.json`, SARIF and history
`auricCodeHealth.health.writeOnScan`	boolean	`true`	Rewrites the record after each project scan
`auricCodeHealth.health.formats`	array: `json`, `sarif`, `history`	all three	Which files to write
`auricCodeHealth.health.fileName`	string	`health.json`	JSON file name; SARIF and history names follow it
`auricCodeHealth.health.maxIssues`	number, 100–100000	`5000`	Most issues recorded, worst first
`auricCodeHealth.health.historyLimit`	number, 10–5000	`200`	Most history entries kept

Data folder

Setting	Type	Default	What it does
`auricCodeHealth.data.enabled`	boolean	`true`	Creates and uses the `.auric-artisan` folder
`auricCodeHealth.data.folderName`	string	`.auric-artisan`	The folder's name, relative to the workspace

Scanning

Setting	Type	Default	What it does
`auricCodeHealth.scan.cache.enabled`	boolean	`true`	Reuses the scan cache
`auricCodeHealth.scan.cache.maxAgeMinutes`	number, 0 or more	`1440`	Maximum cache age; `0` keeps it until a forced or automatic scan
`auricCodeHealth.scan.live`	boolean	`true`	Re-grades the project as you type, from memory
`auricCodeHealth.scan.liveDelayMs`	number, 200–5000	`700`	Wait after the last keystroke before a live re-grade
`auricCodeHealth.scan.autoScan`	boolean	`false`	Rescans from disk after file changes
`auricCodeHealth.scan.autoScanOnStartup`	boolean	`false`	With auto scan on, also rescans soon after the workspace opens
`auricCodeHealth.scan.autoScanDelayMs`	number, 250–60000	`1800`	Wait after a file change before an automatic scan
`auricCodeHealth.scan.maxFiles`	number	`4000`	Most files one scan reads
`auricCodeHealth.scan.exclude`	glob	see below	Paths to skip

The default of `scan.exclude` is `**/{node_modules,dist,build,out,.git,vendor,coverage,.auric-artisan}/**`.

Diagnostics codes

Code	Category
`code-health:secret`	Possible secrets
`code-health:security`	Security risks
`code-health:conflict`	Merge markers
`code-health:risk`	Risky APIs
`code-health:debug`	Debug leftovers
`code-health:todo`	Task markers
`code-health:size`	Oversized

Quick fixes

Quick fix	What it inserts
Ignore this secret finding here (category varies)	`auric-disable-next-line <category>` on the line above
Ignore all secret findings in this file (category varies)	`auric-disable-file <category>` at the top

Studio spaces

On its own: **Health** and **Settings** (Appearance, Project data, Scan engine, Scope).

Files it writes

`.auric-artisan/` `scan-cache.json`, `settings.json`, `health.json`, `health.sarif`, `health-history.json`, `project-health.md`, `.json` and `.html`, `ai-remediation-plan.md` and `.json`, and the knowledge files. See The .auric-artisan folder.

Command-line scanner

The `auric-artisan` scanner ships inside the extension at `bin/auric-artisan.js`. See The CI gate.

Related

- Categories and signatures
- Studio reference
- Shortcuts

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health troubleshooting

Symptoms, causes and fixes for Auric Code Health's findings, secrets, Studio and records.

For problems shared by all Studio extensions (scans, the data folder, settings), see the Studio troubleshooting page.

Findings in the editor

No code-health underlines appear

Why: One of these applies:

- `codeHealth.enabled` or `codeHealth.diagnostics` is off;
- the category is turned off (for example `codeHealth.todo`);
- VS Code does not recognise the file's language, or the file is larger than 1 MB;
- a suppression comment covers the line or file.

What to do: Check the settings under `auricCodeHealth.codeHealth`, and look for `auric-disable` comments.

A TODO in my code is not reported

Why: In languages the Studio can lex, task markers count only inside comments, so a `TODO` inside a string or identifier is ignored on purpose. Lower-case markers need `@todo`, `todo:` or `fixme:`.

console.log is not reported in a Python or Go file

Why: Debug signatures are limited to the languages where they are debug hooks: `console.log` in web languages, `breakpoint()` in Python, a `DEBUG`-prefixed `fmt.Print` in Go, and so on. Normal output such as `print` or `fmt.Println` is never reported.

eval is reported with low confidence

Why: Its argument is a constant, so the risk is lower; the message ends **(constant argument — lower risk)**. A dynamic argument is reported with high confidence.

innerHTML is not reported

Why: The assigned value is a constant string or passes through a sanitiser function, which is safe.

A long line is not reported

Why: The line is 240 characters or fewer, or starts with a URL. Large files (over 800 lines) are reported only by project scans, not in the editor.

"Unresolved merge-conflict marker" in Markdown

Why: A line starts with seven `<` or `>` characters. A lone `=======` heading underline is not reported.

What to do: Resolve the conflict, or suppress the line if the marker is intentional example text.

Secrets

A secret is reported that is a placeholder or test value

Why: The value does not look like a placeholder to the filters, for example a realistic-looking test token.

What to do: Add `auric-disable-line secret` (or the signature ID) in a comment on that line. If it is a real credential, rotate it.

A real secret is not reported

Why: It does not match a provider shape, and its name does not end in a credential word, or its value looks like a placeholder or reference.

What to do: Add a pattern for your token format to `codeHealth.customSecretPatterns`. Auric Code Health is not a full secret scanner; use a dedicated tool where coverage matters.

My custom pattern does nothing

Why: The regular expression is invalid (it is ignored silently), the matches look like placeholders, or you are running the command-line scanner, which does not read VS Code settings.

What to do: Test the expression, and remember the JSON string needs escaped backslashes.

I cannot see which secret was found

Why: Secrets are always redacted; only their length is shown.

What to do: Use **Go to code**; the underline covers the secret's position.

Studio, reports and records

The Health space says "Code Health Studio" with a Scan button

Why: There is no scan result yet.

What to do: Select **Scan Code Health**.

The status bar shows no Health score

Why: The status-bar item shows `Health` only when Auric Code Health is the only Studio extension; otherwise it shows the accessibility score (`A11y`). Before a scan it has no number.

What to do: Select **Report** on the Overview, or use the Studio's **Health** space.

health.json was not updated after a scan

Why: `health.enabled` or `health.writeOnScan` is off, `health.formats` does not include `json`, or the scan was a live re-grade, which never writes files.

A baseline suddenly reports old findings as new

Why: Lines moved (fingerprints include the line), or the baseline was written by an earlier detector version.

What to do: Regenerate the baseline. See Health records.

Related

- FAQ
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health FAQ

Short answers about Auric Code Health.

Which languages does it check?

More than 40: JavaScript, TypeScript, Python, Go, Rust, Java, C#, C and C++, Kotlin, Swift, Ruby, PHP, shell, SQL, YAML, TOML, JSON, Markdown, HTML, CSS and many more. Some signatures apply only where they make sense; see Categories and signatures.

Is it a replacement for a secret scanner or SAST tool?

No. It is a fast, high-signal heuristic linter for the files in your workspace. It does not scan Git history, verify credentials or trace data flow. Use dedicated tools where compliance requires them.

Will it ever show or store my secrets?

No. Every secret is replaced by `[redacted secret; N characters]` everywhere it could appear. See Secrets and redaction.

How is the code-health score calculated?

Each finding costs points by category (secrets 14, security 12, merge markers 10, risky APIs 4, debug 2, oversized 0.5, task markers 0.25), scaled by confidence and divided by the number of files. See Scores and grades.

Does the code-health score affect the accessibility score?

No. They are separate. The overall score in the status bar and Overview combines contrast and accessibility only.

Can I turn off TODO findings?

Yes: set `auricCodeHealth.codeHealth.todo` to `false`. Every category has its own switch.

Can I silence one finding?

Yes: **Ignore here** in the hover, the **Ignore this … finding here** quick fix, or an `auric-disable-next-line <category>` comment. The comment must be a real comment; mentioning it in a string does nothing.

Why does the project health report mention accessibility and colour?

The report covers everything the Studio scans. Its command is provided by Auric Code Health, but the scan always includes contrast and WCAG checks.

How do I use it in CI?

Run the bundled `auric-artisan` command-line scanner with a baseline and upload the SARIF. See The CI gate.

Does it need the internet?

No. See Privacy.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health: other resources

Glossary, shortcuts, limits and accuracy, related pages, release notes and privacy for Auric Code Health.

Supporting pages for Auric Code Health. Each points to the fuller Studio page where one exists.

File	What it covers
Glossary	Code-health terms used in these docs.
Shortcuts	Keyboard shortcuts and Command Palette titles.
Limits and accuracy	What the checks can and cannot find, and how accuracy was measured.
Related	Related extensions, pages and standards.
Release notes	Auric Code Health's history from the changelog.
Privacy	What it stores and writes, and how secrets are protected.

Back to the Auric Code Health overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health shortcuts

Auric Code Health's keyboard shortcuts and Command Palette titles.

Keyboard shortcuts

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center
`Ctrl` + `.`	`Cmd` + `.`	VS Code's quick fixes, including **Ignore this ... finding here**

Command Palette titles

Type `Auric Artisan` in the Command Palette:

- Auric Artisan: Open Code Health Studio
- Auric Artisan: Workstation: Command Center
- Auric Artisan: Open Rulebook & Knowledge Database
- Auric Artisan: Build Workspace Knowledge Database (JSON + Markdown)
- Auric Artisan: Open Unified Icon Studio
- Auric Artisan: Generate Agent-Ready Remediation Plan
- Auric Artisan: Generate Project Health Report (Color, A11y & Code Health)
- Auric Artisan: Write Health Record (health.json + SARIF)
- Auric Artisan: Clear Workspace Scan Cache
- Auric Artisan: Open Workspace Data Folder

In the Studio's Health space

Keys	What it does
`/`	Focus the findings search box
`Escape`	Close pop-ups, then clear the search
`Enter`, `Space`	Open the selected finding's file at its line

For every Studio key, see the Studio shortcuts.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health glossary

Code-health terms used in the Auric Code Health documentation.

Argument-aware. A check that reads a call's argument or an assigned value to tell a constant (lower risk) from a dynamic value (higher risk).

Baseline. A saved health record of existing findings, so a CI gate fails only on new ones.

Category. One of the seven groups of checks: possible secrets, security risks, merge markers, risky APIs, debug leftovers, task markers and oversized.

Confidence. How sure a signature is that a finding is real: high, medium or low (contextual for argument-aware risky-API signatures). Reported separately from severity.

Credential assignment. A key named like a credential, such as `DB_PASSWORD`, given a literal value.

CWE-798. The Common Weakness Enumeration entry for hard-coded credentials, which every secret signature maps to.

Deserialization. Turning stored data back into objects. Unsafe deserialization of untrusted data can run attacker code.

Detector version. The version of the checking engine that produced a finding (1.2.0 in the current package).

Entropy. How unpredictable a value's characters are. Random-looking values are more likely to be real secrets.

Fingerprint. An 8-character ID that identifies a finding across runs.

Health record. `health.json`: every issue with scores and a summary, for CI.

Merge marker. A `<<<<<<<`, `=======` or `>>>>>>>` line left behind by an unresolved version-control conflict.

OWASP A05:2025. The OWASP Top 10 category the security signatures map to.

Placeholder. A stand-in value such as `CHANGE_ME` or `<your-key>`, not reported as a secret.

Redaction. Replacing a secret with `[redacted secret; N characters]` wherever it would appear.

SARIF. Static Analysis Results Interchange Format, the JSON format read by GitHub code scanning.

Secret. A credential such as an API key, token, password or private key.

Signature. One specific detector with a stable ID, such as `ch-secret-github-token`.

Task marker. A comment such as `TODO` or `FIXME` that marks unfinished work.

TLS verification. Checking a server's certificate on a secure connection. Turning it off exposes traffic to interception.

For all other terms, see the Studio glossary.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health limits and accuracy

What Auric Code Health's checks can and cannot find, and how their accuracy was measured.

What it is

A high-signal heuristic linter that reads the text of the files in your workspace. It favours precision: it reports the common, high-confidence cases and tells a constant from a dynamic value.

Measured accuracy

On a labelled set of examples used in development, including a held-out set, precision is 1.00 in every category (no false positives), and recall is 1.00 in every category except secrets, where 96% of the planted secrets were found.

What it does not do

- It does not build a syntax tree or trace data flow between functions, so it cannot tell whether a dynamic value is actually user-controlled.
- It does not scan Git history, so a secret removed from the current files but still in history is not found.
- It does not check whether a credential is live.
- It is not a full static application security testing tool or a dedicated secret scanner.

Behaviour to know

- Secrets are found everywhere, including comments; other categories ignore comments and, where it matters, strings.
- Signatures run only in the languages where they apply; see Categories and signatures.
- A file reports at most 300 findings.
- Lines over 240 characters are reported, except lines that start with a URL. Files over 800 lines are reported by project scans and the command-line scanner only.
- Files over 1 MB are skipped in the editor and in project scans (1.5 MB for the command-line scanner).
- Worst-case inputs, such as a 200 KB minified line or a 5,000-line file, are checked in milliseconds.

For scanning and scoring limits shared with the other extensions, see the Studio limits and accuracy page.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health privacy

What Auric Code Health stores and writes, how it protects secrets, and when it uses the network.

Auric Code Health analyses your code on your machine. It sends no code, findings or telemetry anywhere and needs no account. The bundled command-line scanner makes no network requests either.

Secrets are never stored

When it finds a secret, only a note with its length (`[redacted secret; 40 characters]`) is kept. The secret itself never reaches the editor hover, the Problems panel, the Studio, the scan cache, `health.json`, SARIF, the history, the project health report or the remediation plan. See Secrets and redaction.

What it writes

Where	What
`.auric-artisan/` in your workspace	The scan cache, a settings copy, `health.json`, `health.sarif`, `health-history.json`, the project health report, the remediation plan and the knowledge files
Your settings	The accent colour (user settings) and values you save in Studio › Settings (workspace settings)
VS Code's storage for the extension	Dismissed suggestions and collapsed Studio sections

The health record includes your current Git branch and commit, read from the `.git` folder. Review records before sharing them outside your team: they contain file paths and messages, never secret values.

When the network is used

Only when you open a link yourself, such as auricartisan.com or an official source in the knowledge database.

See the Studio privacy page for details.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Auric Code Health release notes

Auric Code Health's version history, taken from the Studio changelog.

Auric Code Health shares one version number and one changelog with the other Studio extensions. This page lists the entries that concern code health. The complete history is in the Studio release notes.

Unreleased

Listed under **Unreleased** in the changelog, as upcoming entries for the next numbered release (detector 1.2.0):

- Accuracy measured on a labelled set, including a held-out set: precision 1.00 in every category; recall 1.00 except secrets, 0.96 (up from 0.53).
- Secret detection now covers JSON, PHP, typed and Go assignments, prefixed names such as `MYSQL_PASSWORD`, unquoted `.env`, YAML and INI values, OpenAI `sk-proj-` keys, Google keys ending in `-`, and Azure and ADO.NET connection strings.
- False positives removed: placeholders such as `CHANGE_ME`, `{{API_KEY}}`, `$VAR` and SOPS `ENC[…]`; translation labels, autocomplete tokens and key-file paths; `redis.eval` and `model.eval()`; tagged SQL templates; phone masks read as `XXX` markers; Markdown underlines read as merge markers.
- One credential is one finding.
- Secret bytes no longer leak through task-marker messages or security evidence into SARIF or `health.json`.
- Calls whose arguments cannot be read are reported at medium confidence instead of being passed.

- A mention of `auric-disable-file` in prose or a string no longer silences a whole file.
- Task markers, `debugger` and breakpoints are recognised only where they belong in each language.
- The underline covers the finding's real span.
- Worst-case inputs scan in 5–30 ms.
- Findings are fingerprinted by detector 1.2.0, so a baseline written by 1.1.0 may list some findings as new once.
- **Generate Project Health Report** also writes a printable `project-health.html`, with **Open in Browser**.

0.1.0 — Initial Studio release

- Checks for exposed secrets, injection risks, unsafe APIs, merge markers, debug code, task markers, oversized files and maintainability problems.
- Secret evidence redacted before it reaches diagnostics, reports or the Studio.
- Project health records, JSON and SARIF 2.1.0 output, stable fingerprints, baselines, trend history and an offline command-line scanner.
- A versioned local knowledge base covering rules, policies, glossary entries, sources and remediation guidance.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC CODE HEALTH

Related to Auric Code Health

Extensions, Studio pages and standards related to Auric Code Health.

Other Studio extensions

- Auric Accessibility: WCAG linting, project scans and the Project Issues view.
- Auric Color: colour tools and contrast checks.
- The Full Pack: installs all three.

Shared Studio pages

- Exports and health records
- The CI gate
- The remediation plan
- Suppression and branding
- Rulebook and knowledge database

Standards and sources

The official sources the code-health entries in the knowledge database cite, and the SARIF specification:

- OWASP Top 10:2025: https://owasp.org/Top10/2025/0x00_2025-Introduction/
- OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/
- CWE-798, Use of Hard-coded Credentials: https://cwe.mitre.org/data/definitions/798.html
- NIST Secure Software Development Framework: https://csrc.nist.gov/Projects/ssdf
- SARIF 2.1.0: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color — Picker, Palettes & Contrast

What Auric Color does, who it is for, how to install it, and a quick start from hover to fix.

Auric Color turns every colour literal in your code into something you can read, understand and fix in place. Hover a colour such as `#1d4ed8`, `rgb(29 78 216)` or `oklch(0.49 0.22 264)` to see its RGB, HSL, OKLCH and Display-P3 values, its nearest CSS colour name, its contrast on white and black, and how it looks to people with colour-vision deficiency. The hover is also a picker: select a tile to change the colour, keeping the token's own syntax.

It also watches your stylesheets, markup and components for text that is too faint to read. Failing text and background pairs are underlined as you type, with a one-click fix to the nearest passing colour of the same hue, and a Contrast Studio for tuning a pair and writing the fix back to the design token it comes from. The Studio adds generators for accessible palettes, OKLCH shade scales, colour-vision previews and a brand audit on real text.

Auric Color is one of the four Auric Artisan Studio extensions. It works on its own, and joins the others in one Workstation when they are installed together. Everything runs on your machine.

Who it is for

- Front-end developers who edit colours in CSS, Sass, CSS-in-JS, Tailwind configs or component files.
- Designers and design-system maintainers who manage colour tokens and brand palettes in code.
- Developers in any language who keep colours in code, configuration or data files: Python, Go, Rust, Java, C#, Swift, Kotlin, Dart, YAML, TOML and more than 40 others.

Where to find it

- Marketplace ID: `auric-artisan.auric-color`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-color
- Install: `code --install-extension auric-artisan.auric-color`
- Version 0.1.0; VS Code 1.74 or newer. Also installed by the Full Pack.

Quick start

1. Install Auric Color from the Extensions view (`Ctrl` + `Shift` + `X`).
2. Open a stylesheet or component with colours in it. A small swatch appears before each colour literal.
3. Hover a colour. Read its values, contrast on white and black, and colour-vision preview.
4. Select a tile in the hover's **Spectrum**, **Hue**, **Alpha**, **Shades**, **Fix**, **Brand** or **Recent** rows. The colour in your code changes, in its own format, and the hover reopens on the new value.
5. Press `Ctrl` + `Alt` + `C` (`Cmd` + `Alt` + `C` on macOS) with the cursor on a colour to open the full picker beside the editor, with contrast checks, generators and an eyedropper.
6. Look for violet wavy underlines on text colours. Hover one and select **Apply fix** to reach the contrast target.
7. Run **Auric Artisan: Open Color Studio** to generate a palette, a shade scale or a colour-vision preview.

The first time you pick a colour, Auric Color may ask whether to replace VS Code's own colour picker. See Picker modes before you answer.

What you can do

- Read any colour's values, name, contrast and colour-vision preview in a hover, in more than 40 languages.
- Edit colours with an inline picker in the hover, a full spectrum picker beside the editor, or a picker docked in the sidebar, with HEX, RGB, HSL, HSV, OKLCH and LCH inputs and an eyedropper.
- Rewrite every occurrence of a colour in a file at once, each keeping its own format.
- Catch low-contrast text in CSS, Sass, Less, Stylus, HTML, Vue, Svelte, Astro, PHP, JSX, TSX, CSS-in-JS and Markdown, including colours from CSS custom properties and dark themes.
- Fix a pair with one click, or every failing pair in a file with one undoable edit after a preview.
- Flag pairs that pass for typical vision but weaken under colour-vision deficiency.
- Switch between WCAG 2.x ratios, APCA or both.
- Generate harmony palettes that meet a contrast target, OKLCH shade scales from 50 to 950, and export them as CSS, JSON or Tailwind.
- Preview any colour, and your project's palette, under eight kinds of colour-vision deficiency.
- Audit a brand's roles (background, surface, text, links, buttons, borders) on real text and export accessible tokens.
- Mark deliberate brand colours as intentional branding.
- Re-theme the Studio with 13 accent presets or any colour.

In this folder

Folder or file	What it covers
docs/	Guides and reference for Auric Color.
docs/getting-started.md	First run and a tour of what Auric Color adds.
docs/hover-and-swatches.md	The colour hover, swatches and supported languages and formats.
docs/color-picker.md	The inline, panel and docked pickers.
docs/picker-modes.md	Auric picker versus VS Code's native picker, and what each changes.
docs/contrast-diagnostics.md	Contrast underlines, hovers, quick fixes and Fix All.
docs/palettes-and-shades.md	The Palette and Shades tools and their exports.
docs/colour-vision.md	The Vision tool and colour-vision checks.
docs/brand-audit.md	The Brand tool and the brand rulebook.
docs/reference.md	Every command, keybinding, setting, view and menu.
docs/troubleshooting.md	Symptoms and fixes.
docs/faq.md	Short answers.
others/	Glossary, shortcuts, limits and accuracy, related pages, release notes and privacy.

Shared Studio features, such as the Command Center, the Contrast Studio and scanning, are in the Studio docs.

Back to the Studio overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color docs

The guides and reference pages for Auric Color.

These pages cover Auric Color's own features. Start with Getting started. Features Auric Color shares with the other Studio extensions (the Overview, the Command Center, the Studio, the Contrast Studio, suppression and branding) are in the Studio docs.

File	What it covers
Getting started	Install, first hover and pick, and a tour of what Auric Color adds.
Hover and swatches	Every part of the colour hover, the swatches, and the languages and formats supported.
The colour pickers	The inline hover picker, the full picker panel and the docked Color Picker view.
Picker modes	Auric picker or VS Code's native picker, and exactly which VS Code settings each changes.
Contrast diagnostics	How pairs are found, WCAG and APCA modes, hovers, quick fixes and Fix All.
Palettes and shades	The Palette and Shades tools and their CSS, JSON and Tailwind exports.
Colour vision	The Vision tool, colour-vision risk and confusable hues.
The brand audit	The Brand tool, token exports and the brand governance rulebook.
Reference	All 15 commands, both keybindings, all 16 settings, views and menus.
Troubleshooting	Symptoms, causes and fixes.
FAQ	Short answers.

Back to the Auric Color overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Getting started with Auric Color

Install Auric Color, read and edit your first colour, fix your first contrast problem, and find every place Auric Color shows up.

Install

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Search for `auric-artisan.auric-color` and select **Install**. Or run `code --install-extension auric-artisan.auric-color`.

Auric Color starts when VS Code finishes loading. You need VS Code 1.74 or newer.

> **Important:** In its default mode, Auric Color replaces VS Code's own colour decorators (the small squares VS Code draws before colours in CSS) with its own swatches and picker. To do this it sets `editor.colorDecorators` to `false` in your user settings as soon as it starts, and sets it back to `false` if it is turned on again while this mode is active. To keep VS Code's own picker instead, run **Auric Artisan: Restore native color picker (use VS Code's)**. See Picker modes.

Read a colour

1. Open a file that contains a colour, such as a CSS file with `color: #1d4ed8;`.
2. A small swatch appears before the colour value, and the value gets a faint tinted outline.
3. Hover the value. The hover shows the colour's hex value, its nearest CSS name, its RGB, HSL, OKLCH and Display-P3 values, its contrast on white and on black, and how it looks with protanopia, deuteranopia and tritanopia.

See Hover and swatches for every part of the hover.

Change a colour

1. Hover the colour. The top of the hover is a picker: a **Spectrum** of shades at the colour's hue, a **Hue** strip, an **Alpha** strip, **Shades**, **Fix** (versions that pass AA and AAA on white and black), **Brand** and **Recent**.
2. Select any tile. The value in your file changes, written in the same format it was in (hex stays hex, `oklch()` stays `oklch()`), and the hover reopens on the new colour.
3. To type an exact value, select **Type exact...** at the top of the hover.
4. For the full picker, select **Spectrum picker**, or press `Ctrl` + `Alt` + `C` (`Cmd` + `Alt` + `C` on macOS) with the cursor on the colour.

See The colour pickers.

Fix a contrast problem

1. In a stylesheet or component, a text colour that is too faint for its background gets a violet wavy underline.
2. Hover it. The card leads with the verdict, for example `2.32 : 1 — fails WCAG AA for normal text`, then shows the nearest passing colour of the same hue.
3. Select **Apply fix**, or press `Ctrl` + `.` and choose **Fix contrast → #767676 (4.54:1)**.

To fix every failing pair in the file, run **Auric Artisan: Fix All Contrast Issues in File**. See Contrast diagnostics.

Where Auric Color appears

Place	What you get
Editor	Swatches, colour hovers with a picker, contrast underlines, quick fixes
Activity Bar, **Auric Color**	**Overview** (the colour workbench) and a collapsed **Color Picker** view
Status bar	A score item, which shows a score once the Studio has graded the workspace; **Auric: N in this file** when the open file has contrast findings
Editor right-click menu	**Pick / edit color**, **Generate Accessible Palette** and **Check Contrast of Selection** (with a selection), **Choose Color Picker Mode**, **Mark Line as Intentional Branding (Toggle)**
Studio (**Auric Artisan: Open Color Studio**)	**Color** (Palette, Shades, Vision), **Check** (Contrast), **Brand** and **Settings**
Command Center (`Ctrl` + `Alt` + `.`)	Colour actions and your recent colours

The Overview (colour workbench)

The Overview shows the file you are editing and its findings, a suggestion when the file has two or more fixable pairs, a row of brand swatches, your recent colours, **Open the color picker**, and shortcuts to **Contrast**, **Palette**, **Shades**, **Vision**, **Rulebook** and **Icon Studio**. See The Overview view.

The Studio

Auric Artisan: Open Color Studio opens the Studio on **Color › Palette**. The other spaces are **Check › Contrast** (the Contrast Studio), **Brand** and **Settings**. See The Studio.

With the other Studio extensions

When Auric Accessibility is installed too, it hosts the shared Workstation: Auric Color's own views are hidden and its tools appear in Auric Accessibility's sidebar and Studio. Auric Color still draws the swatches, colour hovers and pickers. Contrast underlines are then published by Auric Accessibility instead. See The Workstation.

Next steps

- Palettes and shades
- Colour vision
- The brand audit
- Reference

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

The brand audit

Audit a brand's colour roles against real headings, body text, links and buttons, fix failing roles, export accessible tokens, and use the brand governance rulebook.

The Studio's **Brand** space checks a set of brand colours the way a reader meets them: as headings, body copy, captions, links, buttons and borders on your backgrounds. You can do it by hand, with no project, or start from the colours your last scan found.

Open it

- Select **Brand** on the Studio rail, or **Open Studio: Brand** in the Command Center.
- Select **Brand** in a colour hover's **Open in Studio** row. That colour becomes the **Primary** role.
- Select **Audit as a brand** on the Studio's Home (project palette), **Audit as brand** → in the Palette tool, or **Audit a brand by hand** → on Home before a scan.

The header

The header card is titled **Brand accessibility audit** (*real text · WCAG + APCA + CVD*) and shows:

- a score ring with the brand score and grade;
- **roles pass**, for example `6/8`; **text pass**, for example `5/6`; and **CVD risk**, the number of checks that fail under colour-vision simulation;

- **Auto-fix N roles** when any check fails, or ✓ **All roles pass**;
- **Load from scan** and **Reset**.

Brand colours

Eight roles, each with a colour field:

Role	Meaning	Starting value
Background	Page background	`#FFFFFF`
Surface	Cards, panels, raised areas	`#F4F6F8`
Body text	Primary reading text	`#1A1A1A`
Secondary text	Captions, hints, metadata	`#5B6470`
Primary	Brand or action colour	`#2563EB`
On primary	Text on the primary colour	`#FFFFFF`
Link	Inline hyperlink colour	`#1D4ED8`
Border	Dividers, outlines, inputs	`#D6DAE0`

Real-text preview

Two sample surfaces, **On background** and **On surface**, each with a headline, body text, a secondary caption, a **Primary action** button and a **Read the guidelines** link, drawn in your colours. Edit **Headline text** and **Body text** to preview your own copy. The vision dropdown shows the surfaces as seen with protanopia, deuteranopia, tritanopia or achromatopsia; a note reminds you when a simulation is on.

Role checklist

Check	Target
Body text on background	4.5:1
Body text on surface	4.5:1
Secondary text on background	4.5:1
Secondary text on surface	4.5:1
Link on background	4.5:1
Button label on primary	4.5:1
Primary as UI on background	3:1
Border / divider on background	3:1

Each row shows the pair, its hex values, **need 4.5:1** (or 3:1), the ratio, a **CVD** badge with the worst ratio under protanopia, deuteranopia or tritanopia, and ✓ or ✕. A failing row has a **fix** button that adjusts only that role.

Fix failing roles

- **fix** on a row changes that check's foreground role (for example **Secondary text**) until it reaches the target on its background, keeping its hue.
- **Auto-fix N roles** does the same for every failing check.

A toast confirms each change.

Load from scan

Load from scan fills the roles from your last project scan: the lightest colour becomes **Background**, the next lightest **Surface**, the darkest **Body text**, and the most used strongly coloured one **Primary**. **On primary** is set to black or white, and **Link**, **Secondary text** and **Border** are derived so they pass on the background. Without a scan, a toast says **Run a project scan first**.

The brand score

Each check earns credit: text checks count fully, UI checks (primary and border) count 0.6.

Text check	Credit
7:1 or more	1
4.5:1 or more	0.92
3:1 or more	0.5
below 3:1	ratio ÷ 3 × 0.3

UI check	Credit
4.5:1 or more	1
3:1 or more	0.9
below 3:1	ratio ÷ 3 × 0.4

A check that fails under colour-vision simulation has its credit multiplied by 0.85. The score is the weighted average × 100, with the usual letter grades. It is separate from the project score.

Export

Button	Output
Export CSS vars	`brand-tokens.css`: a comment with the score and each check's result, then `--brand-bg`, `--brand-surface`, `--brand-text`, `--brand-muted`, `--brand-primary`, `--brand-on-primary`, `--brand-link` and `--brand-border` in `:root`
Export JSON	`brand-tokens.json`: the score, the eight colours and every check with its ratio, target, result and CVD ratio
Copy hexes	All eight values, comma-separated

Exports open a save dialog in your first workspace folder. Your brand colours stay in the Brand space while the Studio tab is open; export them to keep them.

Brand governance in the knowledge database

Auric Color's knowledge database includes an **Accessible brand governance rulebook**, the article **Govern brand exceptions without creating an accessibility loophole**, and three policies:

- **Brand exceptions are narrow, owned, and temporary**
- **Brand classification cannot suppress structural findings**
- **Every brand token has provenance and a semantic role**

Open them with **Auric Artisan: Open Rulebook & Knowledge Database**. Teams can add their own reviewed brand exceptions in `knowledge-overrides.json`. See Rulebook and knowledge database.

Branding in your code

To keep a deliberate brand colour pair in your code visible but out of the score, mark it as intentional branding. See Suppression and branding.

Related

- Palettes and shades
- Colour vision

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

The colour pickers

Use Auric Color's three pickers (inline in the hover, the full panel beside the editor, and the docked sidebar view) to edit colours in your code.

Auric Color gives you three ways to edit a colour in your code. All three write the change straight into the file, in the colour's own format, as an edit you can undo.

Picker	Where	Best for
Inline hover picker	Inside the colour hover	Quick changes without leaving the code
Picker panel	A tab beside the editor	Precise editing, contrast checks, generating sets, rewriting every match
Color Picker view	Docked in the sidebar	Keeping a picker open while you move between colours

The inline hover picker

Point at a colour (or click it) and use the **Spectrum**, **Hue**, **Alpha**, **Shades**, **Fix**, **Brand** and **Recent** rows at the top of the hover. Each tile applies its colour and the hover reopens on the new value. **Type exact…** takes a typed value. See Hover and swatches.

Open the picker panel

- Put the cursor on a colour (or select one) and press `Ctrl` + `Alt` + `C` (`Cmd` + `Alt` + `C` on macOS), or run **Auric Artisan: Pick / edit color**. It is also on the editor's right-click menu.
- Select **Spectrum picker** in a colour hover.

- Click a colour literal when `auricColor.colorPicker.clickAction` is `fullEditor`, or when the colour hover is turned off.
- Select **Open the color picker** on the Overview, or **Open full** in the docked view.
- Select a colour in the Command Center's **Recent colors**.

The panel opens beside your editor, titled **Auric Artisan — Color Picker**, with the heading **Color picker** and the line **Edit, convert, test contrast, and generate accessible color systems.**

If the cursor is not on a colour, the picker starts from Auric gold, `#D3AF37`, and the colour you apply is inserted at the cursor.

The first time you pick a colour while VS Code's own colour decorators are still on, Auric Color may ask **Auric Artisan can replace VS Code's native color picker with its own. Turn off the native color decorators?** with **Use Auric Artisan picker**, **Keep native** and **Don't ask again**. See Picker modes.

The docked Color Picker view

Expand **Color Picker** in the sidebar (collapsed by default). It is the same picker in a narrower layout. While it is visible and no picker panel is open:

- **Pick / edit color** and clicking a colour load that colour into the docked picker instead of opening a panel;
- applying a colour writes to that token, or inserts at the cursor if no colour was targeted.

The ↗ button at the top and **Open full** at the bottom open the picker panel.

Picker controls

Control	What it does
Spectrum	Drag, or use the arrow keys, to set saturation (across) and brightness (down)
Hue bar	Drag or use the arrow keys to set the hue
Opacity bar	Drag or use the arrow keys to set transparency
Preview	The current colour
⊙ eyedropper	Picks a colour from anywhere on screen, where the system supports it; otherwise shows **Eyedropper unavailable here**
Format tabs	`hex`, `rgb`, `hsl`, `hsv`, `oklch`, `lch`: the input fields and the written format
Copy	Copies the current value
Value fields	Type values in the chosen format; the colour updates as you type
Apply to all N matches in file	Appears when the colour occurs more than once in the file; when ticked, every literal of the same colour is rewritten, each in its own format
Best text	Sets the colour to black or white, whichever reads better on it
Done (panel) or **Open full** (docked view)	Closes the panel, or opens the full panel

The picker starts in the token's own format when it is one of the six. **HSV** is not a CSS format, so choosing it writes hex.

Contrast section

Check the colour against a background:

- choose white or black with the chips, or type any background colour;
- read the ratio and APCA Lc, and the **AA**, **Large** and **AAA** badges (✓ or ✕);
- **Fix → AA** and **Fix → AAA** adjust the colour's lightness until it reaches 4.5:1 or 7:1 on that background, keeping its hue. A toast says **Nudged to passing**, or **Couldn't reach target** when no shade of that hue can get there.

Generate section

Choose **Shade ramp** or a harmony (**Complementary**, **Split complement**, **Analogous**, **Triadic**, **Tetradic**, **Square**, **Double split**, **Monochromatic**) to see a set of colours based on the current one. Select a tile to apply it. **Copy** copies the set as CSS custom properties, for example `--shade-500: #4581FF;` or `--triadic-2: …;`, and **Insert** types them at the editor cursor.

Accessibility section

Titled with the nearest colour name, for example **Accessibility · royalblue**. It shows the contrast on white and on black (coloured as pass, large-text pass or fail), APCA Lc on white and on black, relative luminance, and the colour difference to the named colour (**ΔE→name**).

Brand and recent section

A search box (**Search 140+ named colors or a hex…**) that shows up to 28 matching CSS colour names, or the colour you typed; the nine brand swatches; and your recent picks in this picker (up to 14). Select any swatch to apply it.

Each section can be collapsed with its heading; the picker remembers which you collapsed.

When changes are written

- Dragging writes when you release the pointer.
- Selecting a swatch, a tab or a fix writes immediately.
- Typing in a value field writes as the value becomes valid.
- Keyboard changes on the bars and spectrum write after a short pause (about 0.6 seconds).

Each write is a separate edit you can undo with `Ctrl` + `Z`. The picker never writes to a file outside your workspace or a file that is not open.

Pickers inside the Studio

Colour fields in the Studio (for example the Contrast Studio's **Text** and **Background**, or the Palette's **Seed**) have their own swatch button that opens a pop-up version of the picker: spectrum, hue and opacity bars, eyedropper, formats, copy, accessibility figures, brand and recent swatches, **Best text** and **Done**. It edits the Studio field, not your code.

Related

- Picker modes
- Hover and swatches
- Reference

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Colour vision

Preview colours and your project palette under colour-vision deficiencies, and understand colour-vision risk and confusable hues.

Colour-vision deficiency (CVD), often called colour blindness, makes some colours hard to tell apart. Colours that look clearly different to you can look alike to someone with a CVD. Auric Color shows you how your colours look and warns when a design relies on hue alone.

The simulations

Type	What it means
Protanopia	No red cones (about 1% of men)
Protanomaly	Weak red
Deuteranopia	No green cones (about 1% of men)
Deuteranomaly	Weak green; the most common form, about 5%
Tritanopia	No blue cones; rare
Tritanomaly	Weak blue
Achromatopsia	Total colour blindness
Achromatomaly	Partial colour blindness

Simulations use the Brettel 1997 method in linear light, which keeps white white and does not brighten reds. They are previews for design decisions, not a diagnosis or a medical measurement.

The Vision tool

Open the Studio's **Color › Vision**, select **Vision** in a colour hover's **Open in Studio** row or on the Overview, or choose **Open Studio: Vision** in the Command Center.

- **Color-vision simulation:** enter a colour to see it as a swatch next to all eight simulations. Hover a swatch for its simulated hex; select it to copy.
- **Project palette under color-blindness** (after a project scan): your 16 most used colours in four rows, for typical vision, protanopia, deuteranopia and tritanopia. Hover a cell to see the original and simulated hex.

Colour vision elsewhere

Where	What you see
Colour hover	The colour next to its protanopia, deuteranopia and tritanopia versions
Contrast diagnostics	A hint when a pair passes normally but its contrast falls below the target under protanopia, deuteranopia or tritanopia
Colour-vision risk hovers	The pair drawn for typical vision, protan, deutan and tritan, with each ratio
Palette and Brand tools	A **Preview as** or vision dropdown that shows every swatch or surface as seen with a CVD
Preview tool (Auric Accessibility)	A **Vision** dropdown for the before-and-after surfaces
Project scan	**Confusable colors** (Project Issues) and **Confusable hues** (Studio Findings)

Colour-vision risk

A pair is a colour-vision risk when it meets its contrast target for typical vision, but its worst ratio under protanopia, deuteranopia or tritanopia falls below the target. Such pairs are shown as hints (`[WCAG 1.4.1]`, Use of Color) and reduce the contrast sub-score by up to 10 points. The fix is to increase the difference in lightness, or to add a cue that is not colour, such as an icon, text or underline.

Turn the hints off with `auricColor.contrast.flagCvdRisk` (or `auricA11y.contrast.flagCvdRisk`).

Confusable hues

After a project scan, the Studio compares your most used chromatic colours in pairs. A pair is **confusable** when the two look clearly different to typical vision (colour difference of at least 12) but become hard to tell apart under protanopia, deuteranopia or tritanopia (difference below 9). Up to 12 such pairs are listed, most confusable first, with both colour differences and the simulation that merges them. Use stronger lightness contrast, shape, text or icons to tell those states apart.

Related

- Contrast diagnostics
- Palettes and shades
- Limits and accuracy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Contrast diagnostics

How Auric Color finds low-contrast text as you type, what its messages mean, and how to fix one pair or a whole file.

Auric Color checks the text and background colours in your stylesheets, markup and components as you type, and underlines text that does not meet your contrast target.

When Auric Accessibility is installed too, it publishes these contrast diagnostics instead, with the same behaviour and its own `auricA11y.contrast.*` settings. Only one extension ever publishes them, so you never see a pair twice.

Where contrast is checked

CSS, SCSS, Sass, Less, PostCSS, Stylus, HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX and Markdown files, up to 1 MB each.

The checker pairs a text colour with its background from:

- CSS rules and `<style>` blocks;
- inline `style="…"` attributes;
- JSX style objects: `style={{…}}`, `sx={{…}}`, `css={{…}}`;
- styled-components, Emotion and Linaria tagged templates (`styled.x`…``, `css`…``, `createGlobalStyle`);
- Vue, Angular and Svelte style bindings (`:style="{…}"`, `[ngStyle]="{…}"`, `style:color`).

It follows CSS custom properties (`color: var(--ink)` over `background: var(--surface)`), including fallbacks and chains, and grades dark themes separately. See Limits and accuracy for exactly how pairs are resolved and what cannot be seen.

Contrast standards

`auricColor.contrast.standard` chooses the model:

Value	Studio label	What is flagged
`wcag2` (default)	WCAG 2.x ratios	Pairs below `contrast.threshold`
`apca`	APCA (WCAG 3 draft)	Pairs whose APCA lightness contrast is below `contrast.apcaThreshold`
`both`	WCAG 2.x + APCA	WCAG failures first, then APCA shortfalls on pairs that pass WCAG

With Auric Accessibility installed and its audit profile set to **WCAG 3.0 draft**, `wcag2` behaves like `both`.

Targets

- `contrast.threshold` (default 4.5, range 1–21): 4.5 is WCAG AA for body text, 7 is AAA, 3 is AA for large text and UI parts.
- Large text (24 px, or 18.66 px and bold) gets a lower target: 3:1 when your threshold is 4.5, or 4.5:1 when it is 7.
- `contrast.apcaThreshold` (default 60, range 15–106): 90 is roughly AAA, 75 body text, 60 large or medium text, 45 large bold text, 30 non-text UI. APCA is experimental; meeting it is not a WCAG 3 conformance claim.

What the diagnostics say

All appear in the Problems panel with the source `auric-artisan` and a violet wavy underline in the editor.

Finding	Example message	Severity
Low contrast	`Low contrast 2.32:1 – #AAAAAA on #FFFFFF (need 4.5:1, APCA Lc 46). [WCAG 1.4.3]`	`contrast.severity` (warning)
Low contrast, large text	`… (need 3:1 (large text), …)`	Same
Low contrast, dark theme	`Low contrast in the dark theme 3.10:1 – …`	Same
Possible low contrast	`Possible low contrast 3.25:1 – #8A8F99 on the page background #FFFFFF … Verify the real background.`	Information
Low APCA contrast	`Low APCA contrast Lc 55 – #999999 on #FFFFFF (need Lc 60; large bold ≥ 24px). [WCAG 3 draft]`	`contrast.severity`
Colour-vision risk	`Contrast relies on hue – <text> on <background> drops to <ratio>:1 under color-vision deficiency. [WCAG 1.4.1]`	Hint
Branding	`Branding – #D3AF37 on #FFFFFF (2.11:1, APCA Lc 41). Classified as intentional branding …`	Information

The criterion is `1.4.3` when your threshold is below 7 and `1.4.6` (Contrast, Enhanced) when it is 7 or more.

- **Possible low contrast** appears when a rule sets only a text colour and the background was taken from the page background declared on `body`, `html` or `:root` in the same file (`contrast.checkPageBackground`, on by default). It is softer because the real surface might differ.
- **Colour-vision risk** appears when a pair passes for typical vision but its contrast under protanopia, deuteranopia or tritanopia falls below the target (`contrast.flagCvdRisk`, on by default).
- **Branding** replaces a finding on a line marked with `auric-brand-…`. See Suppression and branding.

The contrast hover

Hover an underlined colour for a card that reads, top to bottom:

1. **Auric Artisan · Contrast** (or **APCA contrast**).
2. The verdict, for example `2.32 : 1 — fails WCAG AA for normal text`, or for APCA `Lc 55 — below the APCA target of Lc 60`.
3. The pair as an **Aa** sample, its values, the other model's figure, and the token it reads, for example `from --muted · used in 12 places`.
4. A note when the background was assumed from the page.
5. **Nearest passing, same hue:** the colour, its ratio and level, and **reuse --token** when an existing token fits. If no colour of that hue can pass, it says so.
6. Actions: the gold **Apply to --token** (changes the token's definition, so every use follows) or **Apply fix**, **Open in Contrast Studio** and **Intentional branding**.
7. Smaller links: **Apply only here**, **Go to code**, the WCAG criterion, and **Copy reference** (copies a one-line description for a commit or ticket).

Apply to --token appears when the text colour comes from a custom property defined in the same file.

Quick fixes

Put the cursor on the underline and press `Ctrl` + `.`:

Quick fix	What it does
Fix contrast → #767676 (4.54:1) (preferred)	Replaces the colour with the nearest passing colour of the same hue
Fix APCA contrast → #hex (Lc N)	For APCA findings: raises contrast until the APCA target is met
Open in Auric Artisan Studio	Opens the pair in the Contrast Studio, linked to this line
Open in the Contrast Studio	Offered instead of a fix for mixes, relative colours, faded text and shared dark-theme values

Those values are left for the Contrast Studio because a one-click rewrite could change more than this text. A one-click fix rewrites a value only when the whole value is one colour, one `var()`, or one side of `light-dark()`.

Fix every pair in a file

1. Open the file and run **Auric Artisan: Fix All Contrast Issues in File** (or **Fix all contrast in file** in the Command Center, **Fix N in this file** on the Overview, or **Fix all in file** on the suggestion).
2. A dialog lists the changes, up to six as `before → after` and **… and N more**, and explains that this is a single undoable edit.
3. Select **Apply fixes**.
4. A message confirms **Auric Artisan: fixed N contrast pair(s). Undo with Ctrl+Z (Cmd+Z) to revert all at once.**

Only pairs that can be fixed with one rewrite are included; each keeps its hue and chroma and reaches its WCAG 2.x target. Other messages you may see:

- **no failing contrast pairs in this file.**
- **nothing to change — failing pairs are already at their best-effort fix.** (when no colour of that hue can pass)

Refresh

Auric Artisan: Refresh Contrast Diagnostics re-checks every open file. You rarely need it: files are re-checked as you type and whenever you change a setting.

Settings

Setting	Default	What it does
`auricColor.contrast.enabled`	`true`	Turns contrast diagnostics in the editor on or off
`auricColor.contrast.standard`	`wcag2`	`wcag2`, `apca` or `both`
`auricColor.contrast.threshold`	`4.5`	WCAG 2.x target (1–21)
`auricColor.contrast.apcaThreshold`	`60`	APCA target (15–106)
`auricColor.contrast.severity`	`warning`	`error`, `warning`, `information` or `hint` for failures
`auricColor.contrast.flagCvdRisk`	`true`	Adds colour-vision risk hints
`auricColor.contrast.checkPageBackground`	`true`	Grades colour-only rules against the page background
`auricColor.branding.enabled`	`true`	Honours branding marks in the editor

Related

- The Contrast Studio
- Scores and grades
- Colour vision

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Hover and swatches

Every part of Auric Color's colour hover and inline swatches, which colour formats are detected, and which languages are supported.

Auric Color finds colour literals in your code, draws a swatch before each one, and shows a detailed hover when you point at one.

Supported languages

The hover, swatches and pickers work in these VS Code languages. Some of them need a language extension installed so VS Code recognises the file type.

Group	Languages
Stylesheets	CSS, SCSS, Sass, Less, PostCSS, Stylus
Web and markup	HTML, Vue, Svelte, Astro, PHP, XML, Markdown
JavaScript family	JavaScript, TypeScript, JavaScript React (JSX), TypeScript React (TSX), CoffeeScript
Data and config	JSON, JSON with Comments, JSON5, HJSON, YAML, TOML, INI, Properties

Group	Languages
Templates	Handlebars, Pug, Jade, EJS, Twig, Razor, Blade
Systems and apps	C, C++, C#, Objective-C, Objective-C++, Go, Rust, Java, Kotlin, Swift, Dart, Scala, Groovy, F#, Visual Basic
Scripting	Python, Ruby, Lua, R, Julia, Perl, Shell script, PowerShell
Functional and others	Haskell, Elixir, Erlang, Clojure, Solidity, GraphQL, SQL, Dockerfile, Makefile

Detected colour formats

Format	Examples
Hex, 3, 4, 6 or 8 digits	`#abc`, `#abcd`, `#1d4ed8`, `#1d4ed880`
`rgb()` and `rgba()`	`rgb(29 78 216)`, `rgba(29, 78, 216, 0.5)`
`hsl()` and `hsla()`	`hsl(221 76% 48%)`
`hwb()`	`hwb(221 11% 15%)`
`lab()` and `lch()`	`lab(35 21 -71)`, `lch(35 74 287)`
`oklab()` and `oklch()`	`oklch(0.49 0.22 264)`
`color()`	`color(display-p3 0.2 0.3 0.85)`
CSS named colours (148)	`rebeccapurple`, `tomato`

Not detected as colours: integer literals such as `0xFF00FF`, custom property names such as `var(--brand)`, and `color-mix()` as a whole (the colours inside it are detected individually).

Swatches

In the default Auric mode, a small square in the colour appears before every colour literal, and the literal gets a faint tinted outline. Swatches appear when:

- the picker mode is **Auric** (`auricColor.colorPicker.mode` is `auric`);
- `auricColor.colorPicker.nativeHoverPicker` is off;
- `auricColor.colorPicker.swatches` is on (the default).

Swatches are not drawn in files larger than 1 MB, and at most 5,000 colours per file are decorated.

Clicking a colour literal (with a single cursor and no selection) does what `auricColor.colorPicker.clickAction` says: show the hover picker (default), open the full picker panel, or nothing. See The colour pickers.

The colour hover

Point at a colour literal to open the hover. In Auric Color alone, pointing anywhere else on a line that contains a colour shows the hover for the first colour on that line. With Auric Accessibility installed, pointing at an element or rule that is not a colour shows the Inspect card instead.

> **Note:** When the line has an Auric finding (for example a contrast failure), the hover shows the finding's card instead of the colour card. To see the colour card on such a line, use the full picker (`Ctrl` + `Alt` + `C`).

The hover has these parts, top to bottom.

Header

A swatch, the hex value, the original text when it differs (for example `oklch(0.49 0.22 264)`), and the nearest CSS colour name. The name is marked `≈` when it is only close, for example `≈ royalblue`.

Actions

Link	What it does
Type exact…	Asks for an exact value (hex, `rgb()`, `hsl()`, `oklch()`, `lch()` or a CSS colour name) and writes it back in the token's format. An invalid value shows **Not a recognized color value.**
Spectrum picker	Opens the full picker for this colour.
Picker mode	Runs **Choose Color Picker Mode**.

Picker rows

Every tile applies its colour to the token immediately, in the token's own format, and the hover reopens on the new value.

Row	Tiles
Spectrum	7 rows of 18 tiles: saturation across, brightness down, at the colour's hue. The tile nearest the current colour has a ring.
Hue	24 tiles around the colour wheel.
Alpha	12 tiles from fully opaque down to 8% opacity. Hex tokens become 8-digit hex; `rgb()` and `hsl()` get an alpha value.
Shades	The colour's 11-step OKLCH shade scale, 50 to 950.
Fix	Versions of the colour that reach AA (4.5:1) and AAA (7:1) on white, and AA and AAA on black, keeping the hue.
Brand	Nine fixed brand swatches.
Recent	Up to 12 colours you applied recently.

When VS Code's native picker is shown inside the hover (`colorPicker.nativeHoverPicker` on), the Spectrum, Hue and Alpha rows are left out because VS Code's sliders appear above the hover. In **VS Code native picker** mode, the hover shows only **Shades** and **Brand**.

Values

- `RGB` red, green and blue from 0 to 255;
- `HSL` hue in degrees, saturation and lightness in percent;
- `OKLCH` lightness, chroma and hue;
- `A` alpha, when the colour is translucent;
- `P3` the same colour as `color(display-p3 …)`, marked *(wide gamut)*.

When the literal is outside the sRGB range (for example a vivid `oklch()` or `color(display-p3 …)`), a warning explains that it is shown, and its contrast measured, as the gamut-mapped sRGB colour a standard screen paints.

Contrast

The colour's contrast on white and on black, each with a pass or fail icon, the ratio and the level (**AAA**, **AA**, **AA Large** or **Fail**), and **text → white** or **text → black**, whichever reads better on this colour as a background.

Colour vision

The colour next to how it appears with protanopia, deuteranopia and tritanopia (*protan · deutan · tritan*).

Open in Studio

Links that open the colour in a Studio tool: **Shades**, **Palette**, **Vision** and **Brand**.

Turn the hover off

Set `auricColor.hover.enabled` to `false`. Clicking a colour then opens the full picker panel instead of the hover. Contrast and issue hovers are not affected.

Recent colours

Colours you apply from the hover tiles, **Type exact…** and the pickers are added to your recent colours (up to 12, newest first). They appear in the hover's **Recent** row, the Overview, the Command Center and the pickers.

Related

- The colour pickers
- Picker modes
- Contrast diagnostics

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Palettes and shades

Generate harmony palettes that meet a contrast target and perceptually even OKLCH shade scales, preview them, and export them as CSS, JSON or Tailwind.

The Studio's **Color** space has two generators: **Palette**, for sets of related colours that all meet a contrast target, and **Shades**, for a tonal scale of one colour. Both are in Auric Color.

The Palette tool

Open it

- Select a colour in the editor and run **Auric Artisan: Generate Accessible Palette** (also on the right-click menu when text is selected). The selection becomes the seed; if it is not a colour, the seed is `#2563EB`.
- Select **Palette** in a colour hover's **Open in Studio** row, or on the Overview.
- Open the Studio and choose **Color › Palette**, or **Open Studio: Palette** in the Command Center.

Controls

Control	Values	What it does
Seed	Any colour	The starting colour
Background	Any colour (default white)	The surface every swatch must read on
Harmony	Complementary, Split complement, Analogous, Triadic, Tetradic, Square, Double split, Monochromatic	How the other colours relate to the seed on the colour wheel
Min contrast	3:1 · UI / large, 4.5:1 · AA body, 7:1 · AAA body	The ratio every swatch must reach on the background
Preview as	Typical vision, Protanopia · no red, Deuteranopia · no green, Tritanopia · no blue, Achromatopsia · no color	Shows the swatches as seen with that colour-vision deficiency

Each colour field has a swatch button that opens the Studio's pop-up picker, a text box and a copy button.

How the palette is built

1. The harmony gives a set of hues from the seed:

Harmony	Colours	Hue offsets from the seed
Complementary	2	0°, 180°
Split complement	3	0°, 150°, 210°
Analogous	5	−30°, −15°, 0°, 15°, 30°
Triadic	3	0°, 120°, 240°
Tetradic	4	0°, 60°, 180°, 240°
Square	4	0°, 90°, 180°, 270°
Double split	5	0°, 30°, −30°, 150°, 210°
Monochromatic	5	Same hue, five lightness steps

2. Each colour that does not reach the minimum contrast on the background has its lightness adjusted until it does, keeping its hue and chroma.

The strip shows each swatch's hex and ratio. A ✎ marks a swatch that was adjusted; hover it to see the original colour. Select a swatch to copy its hex (in a colour-vision preview, the real hex is copied, not the simulated one).

Export and share

Button	Output
Export CSS	`palette.css`: a `:root` block with `--brand-100`, `--brand-200` and so on, each with its ratio as a comment
Export JSON	`palette.json`: the harmony, background, minimum contrast, and each colour's hex, ratio and level
Export Tailwind	`palette.tailwind.js`: a Tailwind config that extends `colors` with a `brand` scale (`'100'`, `'200'` …)
Copy hex list	All hex values, comma-separated
Audit as brand →	Sends the seed and background to the Brand tool, with a readable text colour for the seed

Exports open a save dialog in your first workspace folder, then open the saved file.

The Shades tool

Open it

- Select **Shades** in a colour hover's **Open in Studio** row or on the Overview.
- Open the Studio and choose **Color › Shades**, or **Open Studio: Shades** in the Command Center.

What it shows

Enter a **Base color**. The tool builds an 11-step scale, 50, 100, 200 … 900 and 950, like Tailwind's naming. Each step has a fixed lightness in OKLCH, so the steps look evenly spaced, with the base colour's hue. Chroma is reduced towards the lightest and darkest steps so they do not look neon or muddy, and every step is kept inside the sRGB range.

Each cell shows the step and hex; hover it for its contrast on white. Select a cell to copy it.

Role legibility shows six common pairings and their ratios, green when they reach 4.5:1:

- 700 on 50
- 600 on 100
- white on 600
- 900 on 100
- white on 700
- 50 on 900

Export

Button	Output
Export CSS	`shades.css`: a `:root` block with `--shade-50` to `--shade-950`
Export Tailwind	`shades.tailwind.js`: a Tailwind config that extends `colors` with a `shade` scale
Copy hex list	All 11 hex values

The same scale appears in the colour hover's **Shades** row and in the picker's **Generate › Shade ramp**.

Related

- Colour vision
- The brand audit
- The colour pickers

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Picker modes

Choose between the Auric picker and VS Code's native colour picker, and learn exactly which VS Code settings each mode changes.

VS Code has its own colour picker: in CSS (and in other languages when an extension provides colours) it draws small **colour decorators** before colour values and opens a draggable picker when you hover or click them. VS Code offers no way for an extension to replace that picker, so two pickers would stack on top of each other. Auric Color solves this with a **picker mode**.

The three set-ups

Set-up	How to get it	Swatches	Clicking a colour	Hover
Auric picker (default)	`colorPicker.mode` = `auric`, `colorPicker.nativeHoverPicker` = `false`	Auric's swatches	Auric's inline picker in the hover (or per `colorPicker.clickAction`)	Full Auric hover with Spectrum, Hue and Alpha rows
Auric with VS Code's sliders	`colorPicker.mode` = `auric`, `colorPicker.nativeHoverPicker` = `true`	VS Code's decorators	The hover, per `colorPicker.clickAction`	VS Code's draggable picker above Auric's readouts (no Spectrum, Hue or Alpha rows)
VS Code native picker	`colorPicker.mode` = `decorators`	VS Code's decorators	VS Code's picker	Auric's readouts with only **Shades** and **Brand** rows

In the last two set-ups Auric Color supplies the colours it detects to VS Code's picker in non-CSS languages too, so VS Code's picker also works in JavaScript, TypeScript, HTML, Vue, Svelte, JSON, Markdown and the other supported languages. VS Code's picker then offers the colour in hex, `rgb()`, `hsl()`, `hwb()`, `oklch()`, `oklab()`, `lch()`, `lab()` and `color(display-p3 …)`, starting with the token's own format.

What Auric mode changes in your settings

> **Important:** In the default **Auric picker** set-up, Auric Color sets VS Code's `editor.colorDecorators` setting to `false` in your **user** settings, and also overrides a workspace or folder value that says otherwise. It does this when it starts, and again whenever the setting is switched back on while Auric mode is active (for example by Settings Sync or another extension), when you switch editors, and when the VS Code window regains focus.

This turns off VS Code's decorators in every language, for every workspace, while Auric Color is installed in this mode. Only Auric Color touches this setting; the other Studio extensions never do.

If you uninstall Auric Color, `editor.colorDecorators` stays `false`. Before uninstalling, run **Restore native color picker (use VS Code's)**, or afterwards set `"editor.colorDecorators": true` in your settings.

Switch mode

Choose Color Picker Mode

1. Run **Auric Artisan: Choose Color Picker Mode** (also on the editor's right-click menu, and **Picker mode** in the colour hover).
2. The list **Auric Artisan color picker mode** offers:
   - **Auric picker (default)**: Auric's own inline picker in the hover plus accessibility tools; VS Code's colour decorators turn off.
   - **VS Code native picker**: keeps `editor.colorDecorators` on so clicking a colour opens VS Code's draggable picker, with Auric's hover, scan, contrast and accessibility tools on top.
3. Your choice is saved to your user settings: `colorPicker.mode`, `colorPicker.swatches` and `colorPicker.inlineSwatches` are set to match, and `editor.colorDecorators` is switched off or on.

A message confirms the result:

Result	Message
Auric picker	**Auric Artisan: using the Auric spectrum picker. VS Code Color Decorators are off.**
Auric with VS Code's sliders	**Auric Artisan: hover any color for the draggable picker plus Auric's accessibility tools.**
VS Code native picker	**Auric Artisan: using VS Code Color Decorators with Auric hover, scan, and contrast tools.**

One-step commands

Command	What it sets
Disable native color picker (use Auric Artisan)	`nativeHoverPicker` off, Auric mode, VS Code's decorators off
Restore native color picker (use VS Code's)	`nativeHoverPicker` on, native mode, VS Code's decorators on; also stops the first-pick question below

The first-pick question

If VS Code's decorators are on when you first run **Pick / edit color** in Auric mode, Auric Color asks **Auric Artisan can replace VS Code's native color picker with its own. Turn off the native color decorators?**

Answer	Result
Use Auric Artisan picker	Switches to the Auric picker set-up
Keep native	Switches to the VS Code native picker set-up
Don't ask again	Changes nothing and does not ask again

The question is asked only once.

Click behaviour in Auric mode

`auricColor.colorPicker.clickAction` controls what a mouse click on a colour literal does (keyboard cursor moves never trigger it):

Value	Studio label	Click does
`inlinePicker` (default)	Inline hover picker	Shows the hover with the inline picker
`fullEditor`	Spectrum picker panel	Opens the picker panel beside the editor (or updates the docked picker when it is open)
`off`	Click disabled	Only moves the cursor

You can also set this in Studio › Settings › **Color picker**. The older setting `colorPicker.autoOpen` still works: when it is `true` and `clickAction` has never been set, clicks open the picker panel. Prefer `clickAction`.

Related

- The colour pickers
- Hover and swatches
- Troubleshooting

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color reference

Every command, keybinding, setting, view, menu entry and theme colour Auric Color contributes.

Facts for Auric Color 0.1.0 (`auric-artisan.auric-color`). It needs VS Code 1.74 or newer and starts when VS Code finishes loading. Categories: Visualization, Programming Languages, Other.

Keybindings

Keys (Windows, Linux)	Keys (macOS)	Command	Active when
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center	Auric Color is not suppressed by a Workstation host
`Ctrl` + `Alt` + `C`	`Cmd` + `Alt` + `C`	Pick / edit color	The editor has focus

Commands

All titles appear in the Command Palette after `Auric Artisan:`.

Title	Command ID	What it does
Open Color Studio	`auricColor.openStudio`	Opens the Studio on Color › Palette, or the Workstation host's Studio
Workstation: Command Center	`auricColor.commandCenter`	Opens the Command Center list
Open Rulebook & Knowledge Database	`auricColor.openKnowledgeBase`	Searches the brand rulebook, policies, articles, glossary and sources
Build Workspace Knowledge Database (JSON + Markdown)	`auricColor.writeKnowledgeDatabase`	Writes `knowledge.json`, `rulebook.md` and an overrides template
Open Unified Icon Studio	`auricColor.openIconStudio`	Opens the combined Auric icon library
Generate Accessible Palette	`auricColor.generatePalette`	Opens the Palette tool seeded with the selected colour (or `#2563EB`)
Check Contrast of Selection	`auricColor.checkContrast`	Opens the Contrast Studio with the selected colour as text (or `#1A1A1A`)
Fix All Contrast Issues in File	`auricColor.fixAllContrast`	Previews and applies fixes for every fixable failing pair, as one edit
Pick / edit color	`auricColor.pickColor`	Opens the picker on the colour at the cursor, or to insert a new one
Choose Color Picker Mode	`auricColor.chooseColorPickerMode`	Chooses Auric picker or VS Code native picker
Disable native color picker (use Auric Artisan)	`auricColor.disableNativeColorPicker`	Auric picker on, VS Code's decorators and hover sliders off
Restore native color picker (use VS Code's)	`auricColor.restoreNativeColorPicker`	VS Code's picker and decorators back on
Refresh Contrast Diagnostics	`auricColor.refreshDiagnostics`	Re-checks every open file
Change Accent Color (Theme the Studio)	`auricColor.chooseAccent`	Chooses the Studio's accent colour
Mark Line as Intentional Branding (Toggle)	`auricColor.markBranding`	Adds or removes `auric-brand-line` on the current lines

See Picker modes, Contrast diagnostics and Appearance for details.

Views

View	ID	Notes
Overview	`auricColorHome`	In the **Auric Color** Activity Bar container; the colour workbench layout
Color Picker	`auricColorPickerView`	Collapsed by default; the docked picker

Both are hidden while another Studio extension hosts the Workstation.

Editor right-click menu

Entry	Shown when
Pick / edit color	Always
Generate Accessible Palette	Text is selected
Check Contrast of Selection	Text is selected
Choose Color Picker Mode	Always
Mark Line as Intentional Branding (Toggle)	HTML, Vue, Svelte, Astro, PHP, XML, Handlebars, JavaScript, TypeScript, JSX, TSX, CSS, SCSS, Less, Sass, PostCSS, Stylus and Markdown files

Settings

Auric Color has 16 settings. Change them in **File › Preferences › Settings** (search `auricColor`), in `settings.json`, or for some of them in Studio › Settings.

Setting	Type	Default	What it does
`auricColor.appearance.accent`	string	`gold`	The Studio's accent: a preset ID or a hex such as `#2563EB` (see below). Stored in user settings.
`auricColor.hover.enabled`	boolean	`true`	Shows the colour hover (values, contrast, colour-vision preview, inline picker).
`auricColor.colorPicker.mode`	`auric` or `decorators`	`auric`	Auric's hover-first picker, or VS Code's native picker with Auric's tools on top.
`auricColor.colorPicker.nativeHoverPicker`	boolean	`false`	In Auric mode, shows VS Code's draggable picker inside the hover, with Auric's content beneath. VS Code's decorators stay on.
`auricColor.colorPicker.swatches`	boolean	`true`	Draws Auric's swatch before every colour literal (when VS Code's decorators are off).
`auricColor.colorPicker.clickAction`	`inlinePicker`, `fullEditor`, `off`	`inlinePicker`	What clicking a colour literal does in Auric mode.
`auricColor.colorPicker.autoOpen`	boolean	`false`	Legacy: when `true` and `clickAction` has never been set, clicking opens the picker panel.
`auricColor.colorPicker.inlineSwatches`	boolean	`false`	Contributes VS Code's native inline swatches and picker for colour literals. Set together with `mode` by **Choose Color Picker Mode**.
`auricColor.contrast.enabled`	boolean	`true`	Flags text and background pairs below the target.

Setting	Type	Default	What it does
`auricColor.contrast.standard`	`wcag2`, `apca`, `both`	`wcag2`	The contrast model for diagnostics.
`auricColor.contrast.apcaThreshold`	number, 15–106	`60`	Minimum APCA Lc when APCA is used (90 ≈ AAA, 75 body, 60 large or medium, 45 large bold, 30 non-text UI).
`auricColor.contrast.threshold`	number, 1–21	`4.5`	Minimum WCAG 2.x ratio (4.5 AA body, 7 AAA, 3 AA large and UI).
`auricColor.contrast.severity`	`error`, `warning`, `information`, `hint`	`warning`	Problems severity for failing contrast.
`auricColor.contrast.flagCvdRisk`	boolean	`true`	Adds a hint when a passing pair relies on hue and weakens under colour-vision deficiency.
`auricColor.contrast.checkPageBackground`	boolean	`true`	Grades colour-only rules against the page background, as softer "verify the real background" hints.
`auricColor.branding.enabled`	boolean	`true`	Honours `auric-brand-…` marks in the editor, moving marked colour findings to the Branding group.

Accent presets: `gold`, `champagne`, `rose-gold`, `silver`, `platinum`, `bronze`, `copper`, `espresso`, `maroon`, `rose`, `emerald`, `sapphire`, `amethyst`. See Appearance. The page background for `contrast.checkPageBackground` is the one declared on `body`, `html` or `:root` in the same file.

When Auric Accessibility is installed, it owns contrast diagnostics and the Studio's contrast settings, and reads its own `auricA11y.contrast.*` keys instead of these.

Theme colours

Colour ID	Dark	Light	High contrast	High contrast light
`auricColor.branding.foreground`	`#F06595`	`#C2255C`	`#FF80AB`	`#A61E4D`
`auricColor.branding.background`	`#F0659522`	`#C2255C1A`	transparent	transparent
`auricColor.branding.border`	`#F0659580`	`#C2255C80`	`#FF80AB`	`#A61E4D`
`auricColor.branding.icon`	`#F06595`	`#C2255C`	`#FF80AB`	`#A61E4D`

Studio spaces

When Auric Color is the host or runs alone: **Color** (Palette, Shades, Vision), **Check** (Contrast), **Brand** and **Settings** (Appearance, Contrast, Color picker).

Diagnostics

Code	Meaning
`low-contrast`	WCAG 2.x contrast below target
`apca-low`	APCA below target
`cvd-risk`	Colour-vision risk

Code	Meaning
`branding:contrast`, `branding:apca`, `branding:cvd`	Classified as intentional branding

Export files

File	From
`palette.css`, `palette.json`, `palette.tailwind.js`	Palette tool
`shades.css`, `shades.tailwind.js`	Shades tool
`brand-tokens.css`, `brand-tokens.json`	Brand tool

Related

- Studio reference, including shared settings.
- Shortcuts

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color troubleshooting

Symptoms, causes and fixes for Auric Color's hovers, swatches, pickers and contrast diagnostics.

For problems shared by all Studio extensions, see the Studio troubleshooting page.

Pickers and decorators

VS Code's own colour squares and picker disappeared everywhere

Why: In its default Auric mode, Auric Color turns off `editor.colorDecorators` in your user settings so two pickers do not stack.

What to do: If you prefer VS Code's picker, run **Auric Artisan: Restore native color picker (use VS Code's)**. See Picker modes.

I turned editor.colorDecorators back on, and it switched off again

Why: While Auric mode is active, Auric Color re-applies its choice whenever the setting drifts, including after Settings Sync or another extension changes it.

What to do: Change the mode instead of the setting: run **Choose Color Picker Mode** and pick **VS Code native picker**, or **Restore native color picker (use VS Code's)**.

VS Code's colour decorators are still off after uninstalling Auric Color

Why: Uninstalling does not reset the setting Auric Color changed.

What to do: Set `"editor.colorDecorators": true` in your user settings (and in any workspace settings Auric Color overrode).

Two pickers appear at once

Why: Another extension provides its own colour decorators or picker, or `colorPicker.nativeHoverPicker` is on while you expect Auric's own picker.

What to do: Turn off the other extension's colour feature, or run **Disable native color picker (use Auric Artisan)**.

No swatches appear

Why: One of these applies: the mode is **VS Code native picker**; `colorPicker.nativeHoverPicker` is on; `colorPicker.swatches` is off; the file is larger than 1 MB; or VS Code does not recognise the file's language (install a language extension for it).

What to do: Check the three settings in Reference, or run **Disable native color picker (use Auric Artisan)**.

Clicking a colour does nothing

Why: `colorPicker.clickAction` is `off`, the mode is **VS Code native picker**, or you clicked with several cursors or a selection.

What to do: Set **Color picker** to **Inline hover picker** or **Spectrum picker panel** in Studio › Settings, or press `Ctrl` + `Alt` + `C`.

The hover shows a finding instead of the colour card

Why: When a line has an Auric finding, the finding's card takes priority over the colour card.

What to do: Use **Pick / edit color** (`Ctrl` + `Alt` + `C`) to edit the colour, or fix the finding first.

My colour is not detected

Why: The format is not one Auric Color detects: for example an integer such as `0xFF00FF`, a custom property name, or a colour built at runtime.

What to do: Write the colour as hex, `rgb()`, `hsl()`, `hwb()`, `lab()`, `lch()`, `oklab()`, `oklch()`, `color()` or a CSS name. See Detected colour formats.

The eyedropper says "Eyedropper unavailable here"

Why: The screen colour picker is not available in this VS Code window on your system.

What to do: Type or paste the colour value instead.

The picker wrote a different format than I expected

Why: The picker writes in the tab you last chose. HSV is not a CSS format, so it writes hex.

What to do: Choose the format tab you want before applying. The hover's tiles and **Type exact…** always keep the token's own format.

Apply to all matches is missing

Why: The checkbox appears only when the same colour occurs more than once in the file.

Contrast diagnostics

No contrast underlines appear

Why: One of these applies:

- `contrast.enabled` is off;
- the file type is not checked (see Where contrast is checked);
- the text and background are in different files or set at runtime, so no pair can be resolved;
- Auric Accessibility is installed and its `auricA11y.contrast.enabled` is off (it owns contrast diagnostics then).

What to do: Check the setting of the extension that owns contrast, and see Limits and accuracy.

Changing auricColor.contrast settings has no effect

Why: Auric Accessibility is installed, so it owns contrast diagnostics and reads `auricA11y.contrast.*`.

What to do: Change the `auricA11y.contrast.*` keys, or use Studio › Settings › **Contrast**, which writes to the right extension.

Many "Possible low contrast" hints

Why: Rules that set only a text colour are graded against the page background declared in the same file, which may not be the surface they really sit on.

What to do: Check the real background. If the hints are not useful for your project, turn off `contrast.checkPageBackground`.

The quick fix only offers "Open in the Contrast Studio"

Why: The colour is a mix, a relative colour, text faded with `opacity`, or a dark-theme value shared with the light theme. A one-click rewrite could change more than this text.

What to do: Use the Contrast Studio to choose and apply a colour.

Fix All says "nothing to change"

Why: The failing pairs are already at their best-effort fix: no colour of that hue reaches the target on that background, or the remaining pairs cannot be fixed with a one-click rewrite.

What to do: Change the background or the hue in the Contrast Studio, or classify deliberate brand colours as branding.

Fix All did not fix APCA findings

Why: Fix All Contrast Issues in File raises pairs to their WCAG 2.x target.

What to do: Use the **Fix APCA contrast** quick fix on each APCA finding.

Studio tools

Load from scan says "Run a project scan first"

Why: The Studio has no scan result yet. Auric Color alone has no scan command; a result appears once the Studio has graded the workspace in the background while you edit.

What to do: Edit a stylesheet and try again, install Auric Accessibility or Auric Code Health and scan, or enter the brand colours by hand.

My brand colours were reset

Why: The Brand tool keeps its colours only while the Studio tab is open.

What to do: Use **Export CSS vars** or **Export JSON** to keep them.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color FAQ

Short answers about Auric Color.

Which languages does it work in?

More than 40: CSS and its dialects, HTML, Vue, Svelte, Astro, PHP, JavaScript, TypeScript, JSX, TSX, JSON, YAML, TOML, Markdown, Python, Go, Rust, Java, C#, C and C++, Swift, Kotlin, Dart, Ruby and many more. See Supported languages. Contrast checks run in stylesheets, markup, JavaScript, TypeScript and Markdown.

Why did VS Code's colour squares disappear?

Auric Color's default mode replaces them with its own swatches and picker, and turns `editor.colorDecorators` off. Run **Restore native color picker (use VS Code's)** to go back. See Picker modes.

Does editing keep my colour format?

Yes. Hover tiles and **Type exact…** write in the token's own syntax, using the fewest decimals that keep exactly the same colour. The picker panel writes the format tab you choose.

Can I change every use of a colour at once?

Within one file, tick **Apply to all N matches in file** in the picker. For a colour that comes from a CSS custom property, use **Apply to --token** in the contrast hover or the Contrast Studio, which changes the definition so every use follows.

What does "nearest passing, same hue" mean?

The colour with the same hue and chroma whose lightness has been moved just far enough to reach the contrast target. It is the smallest visible change that passes.

Is APCA the same as WCAG 3?

No. APCA is the contrast model proposed in the WCAG 3 working draft, which is not finished. Auric Color offers it as an experimental option; meeting its target is not a conformance claim.

Does it work with Tailwind?

It detects colour literals in Tailwind config files and exports palettes and shade scales as Tailwind config snippets. It does not resolve Tailwind utility classes such as `text-gray-500` to colours.

Does it support wide-gamut colours?

It reads `oklch()`, `lab()`, `lch()` and `color(display-p3 …)` and other `color()` spaces. Colours outside sRGB are measured as a standard screen shows them, and the hover says so.

Why does the status bar say "A11y" when I only installed Auric Color?

The score item shows the project's accessibility score (contrast and, where markup is present, WCAG). With Auric Color alone it appears once the Studio has graded the workspace in the background as you edit.

Do I need Auric Accessibility for contrast checks?

No. Auric Color publishes contrast diagnostics on its own. When Auric Accessibility is installed, it takes over contrast diagnostics, with the same behaviour.

Is anything sent to a server?

No. See Privacy.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color: other resources

Glossary, shortcuts, limits and accuracy, related pages, release notes and privacy for Auric Color.

Supporting pages for Auric Color. Each is short and points to the fuller Studio page where one exists.

File	What it covers
Glossary	Colour terms used in the Auric Color docs.
Shortcuts	Auric Color's keyboard shortcuts and Command Palette titles.
Limits and accuracy	What the colour engine and contrast checks can and cannot tell you.
Related	Related extensions, tools and pages.
Release notes	Auric Color's history from the changelog.
Privacy	What Auric Color stores and changes.

Back to the Auric Color overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color shortcuts

Auric Color's keyboard shortcuts, Command Palette titles and picker keys.

Keyboard shortcuts

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `C`	`Cmd` + `Alt` + `C`	Pick / edit color (editor focused)
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center
`Ctrl` + `.`	`Cmd` + `.`	VS Code's quick fixes, including **Fix contrast**

Command Palette titles

Type `Auric Artisan` in the Command Palette (`Ctrl` + `Shift` + `P`):

- Auric Artisan: Open Color Studio
- Auric Artisan: Workstation: Command Center
- Auric Artisan: Open Rulebook & Knowledge Database
- Auric Artisan: Build Workspace Knowledge Database (JSON + Markdown)
- Auric Artisan: Open Unified Icon Studio
- Auric Artisan: Generate Accessible Palette
- Auric Artisan: Check Contrast of Selection
- Auric Artisan: Fix All Contrast Issues in File
- Auric Artisan: Pick / edit color
- Auric Artisan: Choose Color Picker Mode
- Auric Artisan: Disable native color picker (use Auric Artisan)
- Auric Artisan: Restore native color picker (use VS Code's)

- Auric Artisan: Refresh Contrast Diagnostics
- Auric Artisan: Change Accent Color (Theme the Studio)
- Auric Artisan: Mark Line as Intentional Branding (Toggle)

Picker keys

Keys	What they do
Arrow keys on the hue or opacity bar	Change the value by 1 (`Shift` for 10)
`Page Up`, `Page Down`	Change the value by 10
`Home`, `End`	Jump to the minimum or maximum
`Left`, `Right` on the spectrum	Change saturation
`Up`, `Down` on the spectrum	Change brightness
`Escape`	Close the picker panel

For every Studio key, see the Studio shortcuts.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color glossary

Colour and contrast terms used in the Auric Color documentation.

Alpha. How opaque a colour is, from 0 (transparent) to 1 (solid). Written as the last two digits of 8-digit hex or the last value of `rgb()` and `hsl()`.

APCA and Lc. The Accessible Perceptual Contrast Algorithm from the WCAG 3 working draft. It reports lightness contrast as Lc; Auric Color's default target is Lc 60. Experimental.

Chroma. How colourful a colour is, as opposed to grey. The C in OKLCH.

Colour decorators. VS Code's own small colour squares and picker for colour values.

Colour literal. A colour written directly in code, such as `#1d4ed8` or `oklch(0.49 0.22 264)`.

Colour-vision deficiency (CVD). Reduced ability to tell some colours apart. Simulated types: protanopia, protanomaly, deuteranopia, deuteranomaly, tritanopia, tritanomaly, achromatopsia and achromatomaly.

Colour-vision risk. A pair that meets its contrast target for typical vision but not under a simulated deficiency.

Confusable hues. Two project colours that look distinct normally but alike under a simulated deficiency.

Contrast ratio. WCAG 2.x's brightness difference between two colours, 1:1 to 21:1. Targets: 4.5:1 for normal text, 3:1 for large text and UI, 7:1 for AAA.

Custom property. A CSS variable such as `--brand-ink`, also called a design token.

ΔE (CIEDE2000). How different two colours look. Around 1 is barely visible.

Display-P3. A wide-gamut colour space used by many modern screens.

Gamut mapping. Bringing a colour a screen cannot show to the nearest one it can.

Harmony. A rule for choosing related hues: complementary, split complement, analogous, triadic, tetradic, square, double split or monochromatic.

Hue. The colour's position on the colour wheel, in degrees.

Hue-preserving fix. A fix that changes only lightness, so the colour keeps its hue and chroma.

Large text. At least 24 CSS pixels, or 18.66 pixels and bold.

Nearest CSS name. The CSS named colour that looks closest, marked `≈` when not exact.

OKLCH. A colour model with lightness, chroma and hue designed so equal steps look equal. Used for shade scales.

Page background. The background declared on `body`, `html` or `:root`, used for rules that set only a text colour.

Picker mode. Whether Auric's picker or VS Code's native picker handles colours.

Shade scale. An 11-step tonal scale of one colour, 50 to 950.

Swatch. A small square of colour.

For all other terms, see the Studio glossary.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color limits and accuracy

What Auric Color's colour engine and contrast checks measure, how accurate they are, and what they cannot see.

What is accurate

- **Colour conversions** read every CSS Color 4 and 5 syntax and write back to exactly the same colour in the token's own format.
- **Contrast ratios** use the WCAG 2.x formula; verdicts use the exact ratio, and a ratio just under a target is never shown rounded up to it.
- **APCA** uses APCA-W3 0.0.98G and matches that algorithm's published test values.
- **Colour difference** (CIEDE2000) matches all 34 published Sharma test pairs.
- **Colour-vision simulation** uses the Brettel 1997 method in linear light.
- **Translucent colours** are blended the way browsers blend them (8-bit alpha).
- **Contrast scanning** agrees with Chromium's rendering in 62 of 68 cases on the audit set used in development; the other six are deliberately skipped.

What it cannot see

- Colours set by JavaScript at runtime, or by classes defined in another file.
- Which rule wins between two different selectors, inherited or computed values, pseudo-elements and component states.
- Text over images or gradients, gradient-clipped text and `currentColor` backgrounds.
- The rendered font size, so verdicts name the WCAG check rather than a size.
- Tailwind utility classes, integer colour literals such as `0xFF00FF`, and colours built from variables in other languages.

What it is not

- Colour-vision simulations are previews for design decisions, not a diagnosis.
- APCA results are experimental and not a WCAG 3 conformance claim.
- A clean contrast check does not mean the page is accessible; many criteria need manual review.

Size limits

- Files larger than 1 MB are not decorated or checked.
- Up to 5,000 colours per file get swatches.

For the full engine description, see the Studio limits and accuracy page.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color privacy

What Auric Color stores, which settings it changes, and when it uses the network.

Auric Color works entirely on your machine. It sends no code, colours or telemetry anywhere and needs no account.

What it changes or stores

Where	What
Your user settings	`auricColor.appearance.accent`; the picker mode settings when you switch mode; and VS Code's `editor.colorDecorators`, which it turns off in Auric mode (see Picker modes)
Workspace settings	Values you save in Studio › Settings
VS Code's storage for the extension	Your recent colours (up to 12), dismissed suggestions, whether you answered the picker question, and which Studio sections you collapsed
Files you choose	Palette, shade and brand exports, through a save dialog
`.auric-artisan/`	Only when you build the knowledge database, or when Auric Color is the host and scans run in the background

When the network is used

Only when you open a link yourself, such as auricartisan.com from the Studio. The eyedropper reads the screen locally.

See the Studio privacy page for the full picture.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Auric Color release notes

Auric Color's version history, taken from the Studio changelog.

Auric Color shares one version number and one changelog with the other Studio extensions. This page lists the entries that concern colour. The complete history is in the Studio release notes.

Unreleased

Listed under **Unreleased** in the changelog, as upcoming entries for the next numbered release:

- Every CSS Color 4 and 5 syntax is parsed, including `oklch()`, `lab()`, `lch()`, `hwb()`, `color()`, `color-mix()`, relative colour, `none` and `transparent`, with CSS gamut mapping for wide-gamut colours. Invalid hex such as `#12345g` is rejected.
- CIEDE2000 colour difference matches all 34 Sharma test pairs; colour naming uses it.
- Colour-vision simulation uses the Brettel 1997 method in linear light.
- The suggested fix is the nearest passing colour (`#888` on white becomes `#767676`, 4.54:1).
- Ratios just under a threshold never display as the threshold; translucent colours blend with 8-bit alpha.
- Writing colours back round-trips every CSS format exactly.
- The contrast scanner was rebuilt on a CSS rule tree: large text at 3:1, blended translucency and `opacity`, dark themes graded as themselves, `color-mix()`, relative colours and `light-dark()` graded, disabled controls and Markdown code samples exempt.
- The Quick Fix never corrupts source; values it cannot rewrite safely open the Contrast Studio.
- Contrast hovers lead with the ratio and verdict and offer **Apply to --token**, **Open in Contrast Studio** and **Intentional branding**.
- The Studio's Contrast tool became the Contrast Studio.
- Every Studio screen, including the picker, uses the new Auric look.

0.1.0 — Initial Studio release

- Colour detection and editable swatches across CSS and 40+ programming, template, data and markup languages.
- WCAG contrast, APCA, RGB, HSL, OKLCH, Display-P3, colour naming, accessible alternatives, palette generation, shade scales and colour-vision simulation.
- Colour editing inline, from the editor hover and in the docked picker.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › AURIC COLOR

Related to Auric Color

Extensions, Studio pages and website tools related to Auric Color.

Other Studio extensions

- Auric Accessibility: WCAG linting, Inspect and project scans; takes over contrast diagnostics when installed.
- Auric Code Health: secrets, security and hygiene checks.
- The Full Pack: installs all three.

Shared Studio pages

- The Contrast Studio
- Suppression and branding
- Appearance
- The Command Center

Website tools

- Contrast Checker
- Vision Simulation
- Basic Colour Tools
- Design Engine
- Tonal Steps

Elsewhere

- The browser extension, for colours on rendered web pages.
- The Studio's related page, with standards and more tools.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Auric Artisan Studio — Full Pack

What the Auric Artisan Studio Full Pack installs, who it is for, and how to get started with it.

The **Auric Artisan Studio — Full Pack** is a VS Code extension pack: an extension that contains no code of its own and exists only to install other extensions. Installing it installs the three Auric Artisan Studio extensions in one step: Auric Color, Auric Accessibility and Auric Code Health.

Each of the three stays a real, separate extension. You can update, disable or uninstall any of them from the Extensions view without affecting the others. When they are installed together, they cooperate as one Workstation: one sidebar, one Command Center and one Studio covering colour, accessibility and code health.

Like the rest of the Studio, everything runs locally, with no account, no telemetry and no paid tier. The pack does not install icon themes or icon libraries.

Who it is for

- Anyone who wants the complete Studio without choosing extensions one by one.
- Teams that want everyone on the same set of tools; one pack ID to recommend in a workspace.

Where to find it

- Marketplace ID: `auric-artisan.auric-artisan`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan
- Install from a terminal:

```
code --install-extension auric-artisan.auric-artisan
```

- Version 0.1.0; requires VS Code 1.74 or newer.

Quick start

1. Open the Extensions view (`Ctrl` + `Shift` + `X`) and search for **Auric Artisan Studio — Full Pack**.
2. Select **Install**. VS Code installs the pack and its three members.
3. Open a project folder.
4. Select the **Auric Accessibility** icon in the Activity Bar; it hosts the shared Workstation.
5. Select **Scan project** in the Overview and wait for the score.
6. Press `Ctrl` + `Alt` + `.` (`Cmd` + `Alt` + `.` on macOS) to open the Command Center.
7. Select **Studio** to open the Workstation Studio with the Home, Color, Check, Health and Brand spaces.

You now have colour hovers and pickers, contrast and WCAG diagnostics, code-health findings and a project score in one place.

What you can do

- Install or update all three Studio extensions with one pack.

- Keep each member independently manageable: disable Auric Code Health for one workspace, uninstall Auric Color, or update one member on its own.
- Use the combined Workstation: one Overview, one Command Center, one Studio and one knowledge database for every installed capability.
- Recommend one ID to your team, for example in a workspace's `.vscode/extensions.json`.

For what each member does, see Auric Color, Auric Accessibility and Auric Code Health.

In this folder

Folder or file	What it covers
docs/	Guides and reference for the pack.
docs/getting-started.md	Installing the pack and what happens next.
docs/managing-members.md	Enabling, disabling, updating and removing members.
docs/reference.md	The pack's manifest facts.
docs/troubleshooting.md	Pack-specific problems.
docs/faq.md	Short answers.
others/	Glossary, related pages, release notes and privacy.
others/glossary.md	Terms used for the pack.
others/related.md	Related pages.
others/release-notes.md	The pack's history.
others/privacy.md	Privacy for the pack.

Back to the Studio overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack docs

The guides and reference pages for the Auric Artisan Studio Full Pack.

These pages cover the pack itself: installing it, managing its three members and the facts in its manifest. For the features of the extensions it installs, see the Studio docs.

File	What it covers
Getting started	Install the pack, see what it installs and what appears afterwards.
Manage the pack's members	Enable, disable, update and uninstall members, and what the Workstation does when you do.
Reference	The pack's ID, members, version, requirements and categories.
Troubleshooting	Problems with installing or removing the pack.
FAQ	Short answers.

Back to the Full Pack overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Getting started with the Full Pack

Install the Auric Artisan Studio Full Pack, check what it installed, and find the Workstation it creates.

Install

1. Open the Extensions view with `Ctrl` + `Shift` + `X`.
2. Search for `auric-artisan.auric-artisan`, or **Auric Artisan Studio — Full Pack**.
3. Select **Install**.

Or run:

```
code --install-extension auric-artisan.auric-artisan
```

You need VS Code 1.74 or newer.

What gets installed

Extension	ID	What it adds
Auric Color — Picker, Palettes & Contrast	`auric-artisan.auric-color`	Colour hovers, swatches and pickers in 40+ languages, contrast diagnostics, palettes, shades, colour vision and the brand audit
Auric Accessibility — WCAG 2.2 Linter	`auric-artisan.auric-a11y`	70+ WCAG rules, Inspect, project scans, the Project Issues view and the remediation plan
Auric Code Health — Secrets, TODOs & Risk	`auric-artisan.auric-codehealth`	Secrets, security, risky APIs, debug leftovers, task markers, merge markers and size checks; health records, SARIF and reports

The pack itself adds no commands, views or settings.

To check what was installed, open the Extensions view and look under **Installed**, or open the pack's page in the Extensions view: VS Code lists the extensions it includes.

What you see afterwards

The three extensions start once VS Code has finished loading, and form one Workstation hosted by Auric Accessibility:

- **Activity Bar:** one Auric icon, **Auric Accessibility**, with the **Overview**, **Project Issues** and a collapsed **Color Picker** view. Auric Color's and Auric Code Health's own views are hidden.
- **Overview:** the workspace card with **Accessibility**, **Contrast issues** and **Code health** rows, **Scan project**, **Report** and **SARIF**.
- **Status bar:** the score item and, when the open file has findings, **Auric: N in this file**.
- **Studio:** titled **Auric Artisan — Workstation Studio**, with the Home, Color, Check, Health and Brand spaces and Settings.
- **Command Center:** `Ctrl` + `Alt` + `.` lists actions for all three capabilities.

See Getting started for a full tour, and The Workstation for how the three combine.

Recommend the pack to your team

To prompt everyone who opens a repository to install the pack, add it to the workspace's `.vscode/extensions.json`:

```
{
  "recommendations": ["auric-artisan.auric-artisan"]
}
```

VS Code then suggests the pack when a teammate opens the folder.

Next steps

- Manage the pack's members
- Choose what to install, if you decide you need only one member.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Manage the pack's members

Enable, disable, update and uninstall the extensions the Full Pack installs, and see what the Workstation does in each case.

The Full Pack is a convenience bundle, not a single product. Its three members are ordinary extensions, so VS Code's usual controls work on each one separately.

Disable a member

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Find the member, for example **Auric Code Health — Secrets, TODOs & Risk**.
3. Select the gear icon, then **Disable** (for all workspaces) or **Disable (Workspace)** (for this workspace only).
4. If VS Code asks you to restart extensions or reload the window, do so.

The other members keep working. To turn it back on, choose **Enable** or **Enable (Workspace)** in the same menu.

Uninstall a member

1. In the Extensions view, select the gear icon on the member.
2. Select **Uninstall**.

The pack and the other members stay installed. To get the member back, install it again by its ID, or reinstall the pack.

Update members

Members update like any other extension, automatically if you have auto-update on, or from the Extensions view. All Studio extensions share one version number, so keeping them on the same version avoids surprises in the shared `.auric-artisan` folder.

What the Workstation does when members change

The Workstation re-elects its host as soon as the set of installed or enabled members changes, without a reload. Views appear or hide, an open Studio tab rebuilds with the new spaces, and diagnostics move to their new owner.

Members left	Host	What you lose
All three	Auric Accessibility	Nothing
Accessibility and Color	Auric Accessibility	Code-health underlines, the Health space, **Report**, **SARIF**, the CI gate card; project scans still list code-health findings
Accessibility and Code Health	Auric Accessibility	Colour hovers, swatches and pickers, **Fix All Contrast Issues in File**, the Color and Brand spaces, recent colours
Color and Code Health	Auric Color	WCAG diagnostics, Inspect, the Project Issues view, **Scan project**, the Home space; the Overview becomes the colour workbench
Accessibility only	Auric Accessibility	Everything colour-specific and code-health underlines
Color only	Auric Color	Everything accessibility- and code-health-specific
Code Health only	Auric Code Health	Everything colour- and accessibility-specific

Contrast diagnostics move to Auric Color when Auric Accessibility is removed, and back when it returns.

Uninstall the pack

Select the gear icon on **Auric Artisan Studio — Full Pack** and choose **Uninstall**. Afterwards, check the **Installed** list in the Extensions view to see which members remain, and uninstall any you no longer want.

Uninstalling extensions does not delete the `.auric-artisan` folder from your workspaces. Delete it yourself if you no longer need it.

Related

- The Workstation

- Choose what to install
- Troubleshooting

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack reference

The Auric Artisan Studio Full Pack's ID, members, version, requirements and Marketplace metadata.

Property	Value
Display name	Auric Artisan Studio — Full Pack
Extension ID	`auric-artisan.auric-artisan`
Publisher	`auric-artisan` (Auric Artisan)
Version	0.1.0
VS Code required	1.74 or newer
Categories	Extension Packs, Linters, Visualization, Other
Keywords	color, accessibility, a11y, wcag, contrast, code health, extension pack
Licence	Auric Artisan Free Use Licence 2.0
Website	https://auricartisan.com
Marketplace	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan

Members

Order	Extension	ID
1	Auric Color — Picker, Palettes & Contrast	`auric-artisan.auric-color`
2	Auric Accessibility — WCAG 2.2 Linter	`auric-artisan.auric-a11y`
3	Auric Code Health — Secrets, TODOs & Risk	`auric-artisan.auric-codehealth`

What the pack contributes

Nothing beyond its members: no commands, keybindings, views, settings or files. Every command, shortcut and setting you get comes from a member. See the Studio reference.

Marketplace description

"The complete Auric Artisan Studio in one install: colour picking and palettes, 70-rule WCAG 2.2 accessibility linting, local rulebooks and team knowledge, and language-agnostic code-health scanning. Installs the three focused Studio extensions, which you can enable or disable independently."

The accessibility engine in the current package runs 72 rules; see the rule catalogue.

Commands

Command	Result
`code --install-extension auric-artisan.auric-artisan`	Installs the pack and its members
`code --uninstall-extension auric-artisan.auric-artisan`	Uninstalls the pack
`code --list-extensions`	Lists installed extensions, including the members

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack troubleshooting

Problems with installing, updating or removing the Auric Artisan Studio Full Pack, and what to do.

For problems with the Studio's features, see the Studio troubleshooting page.

The pack installed but I see only one Auric icon

Why: The three members form one Workstation hosted by Auric Accessibility; the other two hide their own views.

What to do: Nothing is wrong. Use the **Auric Accessibility** sidebar; it includes colour and code health. See The Workstation.

One member is missing after installing the pack

Why: The member failed to install (for example, a network interruption while VS Code downloaded it), or it was uninstalled or disabled later.

What to do: Open the Extensions view, search for the member's ID (`auric-artisan.auric-color`, `auric-artisan.auric-a11y` or `auric-artisan.auric-codehealth`), and install or enable it.

The pack will not install

Why: Your VS Code is older than 1.74, or VS Code cannot reach the Marketplace.

What to do: Update VS Code, check your connection or proxy, and try again. Offline, install each member's package file with **Extensions: Install from VSIX...** instead.

I only want some of the tools

What to do: Disable or uninstall the members you do not need; the others keep working. See Manage the pack's members. Or uninstall the pack and install only the extension you want.

Members show different versions

Why: One member updated before the others.

What to do: Update all three from the Extensions view so they share one version.

After uninstalling the pack, the tools are still there

Why: The members are separate extensions and may remain installed.

What to do: Uninstall the remaining members from the Extensions view if you no longer want them.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack FAQ

Short answers about the Auric Artisan Studio Full Pack.

What is the difference between the pack and installing the three extensions myself?

None in what you get. The pack installs the same three extensions in one step and gives you one ID to recommend or install.

Does the pack add anything of its own?

No. It has no commands, views, settings or code.

Does it install the Auric icon themes or icon libraries?

No. It installs only Auric Color, Auric Accessibility and Auric Code Health. The **Open Unified Icon Studio** command works when you install an Auric icon extension separately. See The Icon Studio bridge.

Can I disable one member for one project?

Yes. In the Extensions view, choose **Disable (Workspace)** on that member. See Manage the pack's members.

Will three extensions slow VS Code down?

They coordinate so that each piece of work is done once: one extension draws colour swatches, one publishes each kind of diagnostic, and one runs live re-grading and background scans. Heavy parts load only when first used.

Is it free?

Yes, like every Studio extension. No account, API key or paid tier.

Does it need the internet?

Only to download and update from the Marketplace. The extensions run entirely offline. See Privacy.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack: other resources

Glossary, related pages, release notes and privacy for the Auric Artisan Studio Full Pack.

Supporting pages for the pack. The pack has no shortcuts or settings of its own, so there is no shortcuts page; see the Studio shortcuts for its members'.

File	What it covers
Glossary	Terms used when talking about the pack.
Related	The member extensions and shared Studio pages.
Release notes	The pack's version history.
Privacy	What the pack and its members do with your data.

Back to the Full Pack overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack glossary

Terms used in the Full Pack documentation.

Extension pack. A VS Code extension that installs a list of other extensions and adds nothing of its own.

Member. One of the extensions an extension pack installs. The Full Pack's members are Auric Color, Auric Accessibility and Auric Code Health.

Workstation. Two or three Studio extensions installed together and presented as one product, with one sidebar, one Command Center and one Studio.

Host. The member that presents the shared Workstation: Auric Accessibility when installed, otherwise Auric Color, otherwise Auric Code Health.

Suppressed extension. A member that is not the host; it hides its own views but keeps doing the work it owns.

Capability. One member's area of work: colour, accessibility or code health.

Disable (Workspace). A VS Code option that turns an extension off for the current workspace only.

For every other term, see the Studio glossary.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack privacy

What the Full Pack and its members do with your code and data.

The pack contains no code, so it does nothing with your data by itself. VS Code downloads it and its members from the Visual Studio Marketplace.

Its three members analyse your code entirely on your machine: no network requests, no telemetry, no account. They write their results to a `.auric-artisan` folder in your workspace and keep a few preferences in VS Code's settings and storage.

For the details, see the Studio privacy page.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Full Pack release notes

The version history of the Auric Artisan Studio Full Pack.

The pack shares one version number with its members. The full history of what changed in the members is in the Studio release notes.

0.1.0 — Initial Studio release

- The first public release of the Full Pack.
- Installs only the three Studio extensions: Auric Color, Auric Accessibility and Auric Code Health. Icon themes and icon libraries are not part of the pack.
- Each member stays independently installable, updatable and removable.

Unreleased

The changelog lists no changes to the pack itself under **Unreleased**. The upcoming changes to its members are listed in the Studio release notes.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO › FULL PACK

Related to the Full Pack

The member extensions and Studio pages related to the Full Pack.

The members

- Auric Color: colour picking, palettes and contrast.
- Auric Accessibility: WCAG linting and project audits.
- Auric Code Health: secrets, security and hygiene checks.

Studio pages

- Studio overview
- Choose what to install
- The Workstation
- Getting started with the Studio
- Related tools and articles

Elsewhere

- Installing and setting up Auric Artisan's VS Code extensions
- Auric Artisan Icon Studio, which the pack does not install but its members can open.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Other resources

The glossary, shortcuts, limits, related tools, release notes and privacy information for Auric Artisan Studio.

This folder holds the supporting pages for the whole Studio: what the terms mean, every shortcut, what the checks can and cannot tell you, where else to look, what changed between versions, and what happens to your data. Each extension's folder has its own shorter versions of these pages that point back here.

File	What it covers
Glossary	Every term used in the Studio docs, from APCA to Workstation.
Shortcuts	Keyboard shortcuts, Command Palette titles and in-Studio keys.
Limits and accuracy	What each check measures, how precisely, and what it is not.
Related	Related extensions, website tools and Learn articles.
Release notes	The version history, from the changelog shipped with the extensions.
Privacy	What stays on your machine, what is written to disk, and what uses the network.

Back to the Studio overview.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Shortcuts

Every keyboard shortcut, Command Palette title and in-Studio key for Auric Artisan Studio.

Keyboard shortcuts

Windows and Linux	macOS	Command	Extension
`Ctrl` + `Alt` + `.`	`Cmd` + `Alt` + `.`	Workstation: Command Center	All three code extensions
`Ctrl` + `Alt` + `C`	`Cmd` + `Alt` + `C`	Pick / edit color (editor focused)	Auric Color
`Ctrl` + `Alt` + `A`	`Cmd` + `Alt` + `A`	Inspect Accessibility of Element (editor focused)	Auric Accessibility

These VS Code shortcuts are useful with the Studio:

Keys	What it does
`Ctrl` + `.`	Show quick fixes for the underline at the cursor
`Ctrl` + `Shift` + `P`	Open the Command Palette; type `Auric Artisan` to list every Studio command
`Ctrl` + `Shift` + `M`	Open the Problems panel
`Ctrl` + `Z`	Undo a fix, including a whole **Fix All Contrast Issues in File**

Command Palette titles

Every Studio command starts with `Auric Artisan:`.

Title	Extension
Workstation: Command Center	All three
Open Color Studio	Auric Color
Open Accessibility Studio	Auric Accessibility
Open Code Health Studio	Auric Code Health
Open Rulebook & Knowledge Database	All three
Build Workspace Knowledge Database (JSON + Markdown)	All three
Open Unified Icon Studio	All three
Pick / edit color	Auric Color
Generate Accessible Palette	Auric Color
Check Contrast of Selection	Auric Color, Auric Accessibility
Fix All Contrast Issues in File	Auric Color
Choose Color Picker Mode	Auric Color
Disable native color picker (use Auric Artisan)	Auric Color
Restore native color picker (use VS Code's)	Auric Color
Refresh Contrast Diagnostics	Auric Color
Change Accent Color (Theme the Studio)	Auric Color
Mark Line as Intentional Branding (Toggle)	Auric Color, Auric Accessibility
Inspect Accessibility of Element	Auric Accessibility
Scan Project for Accessibility (Use Cache)	Auric Accessibility
Force Rescan Project and Refresh Cache	Auric Accessibility
Generate Agent-Ready Remediation Plan	Auric Accessibility, Auric Code Health
Clear Workspace Scan Cache	Auric Accessibility, Auric Code Health
Search Accessibility Issues	Auric Accessibility
Group Accessibility Issues (Type / File / Severity)	Auric Accessibility
Clear Accessibility Issue Search	Auric Accessibility
Generate Project Health Report (Color, A11y & Code Health)	Auric Code Health
Write Health Record (health.json + SARIF)	Auric Code Health
Open Workspace Data Folder	Auric Code Health

Keys inside the Studio

Keys	Where	What it does
Arrow keys, `Home`, `End`	Studio rail and tab rows	Move between spaces or tools
`/`	Findings and Health	Focus the search box

Keys	Where	What it does
`Escape`	Studio	Close a picker, menu or tooltip, then clear the search
`Enter`, `Space`	Swatches, rows, tiles	Activate
Arrow keys, `Shift` + arrows, `Page Up`, `Page Down`, `Home`, `End`	Colour picker bars and spectrum	Adjust the colour
`Escape`	Colour picker	Close it

See Accessibility of the Studio for more.

Change a shortcut

1. Open **File › Preferences › Keyboard Shortcuts** (`Ctrl` + `K`, `Ctrl` + `S`).
2. Search for the command title, for example **Workstation: Command Center**.
3. Double-click the entry and press the new keys.

When several Studio extensions are installed, only the host's Command Center shortcut is active; the others are switched off automatically.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Glossary

Plain-language definitions of every term used in the Auric Artisan Studio documentation.

Accessible name. The text assistive technology announces for an element, such as a button's label or an image's alt text. It can come from visible text, `aria-label`, `aria-labelledby`, `alt`, `title` or a `<label>`.

ACT (Accessibility Conformance Testing). A W3C format for writing accessibility test rules consistently. The Studio's knowledge entries are **ACT-aligned**: they follow the format's structure but make no claim of ACT conformance.

Activity Bar. The vertical bar of icons at the far left of VS Code. The Studio adds an Auric icon there.

Alt text. The text alternative for an image, given with the `alt` attribute.

APCA (Accessible Perceptual Contrast Algorithm). A contrast model proposed for the WCAG 3 working draft. It reports lightness contrast as **Lc**, roughly 0 to 106 in either polarity, and takes into account which colour is the text. The Studio offers it as an experimental option; an APCA result is not a WCAG 3 conformance claim.

ARIA (Accessible Rich Internet Applications). W3C attributes such as `role` and `aria-label` that describe elements to assistive technology. The Studio checks them against WAI-ARIA 1.2.

Assisted (automation). A finding the engine detects but a person should confirm, such as a placeholder used as a label. Assisted findings weigh less in the score.

Assumed background. When a CSS rule sets only a text colour, the Studio can grade it against the page background declared on `body`, `html` or `:root`. Such pairs are marked as possible findings and weigh half in the score.

Audit profile. The WCAG version the accessibility checks target: 2.0, 2.1, 2.2 or the 3.0 draft.

Baseline. A saved health record of the findings a project already has, used by the CI gate to fail only on new findings.

Best-practice hint. A rule that flags a pattern worth fixing that does not by itself fail a WCAG success criterion, such as a positive `tabindex`. Shown as a hint and weighted at half.

Branding classification. Marking a colour finding as intentional brand identity with an `auric-brand-…` comment. The finding stays visible but leaves the score. Structural findings can never be classified as branding.

Capability. One extension's area of work: colour, accessibility or code health.

CI (continuous integration). The automated build and tests run for every change or pull request.

CIEDE2000 (ΔE00). A standard formula for how different two colours look. Around 1 is barely noticeable; larger numbers are bigger differences. The Studio uses it to rank colour suggestions and to name colours.

Code health. Hygiene and security checks on source code: secrets, security footguns, risky APIs, debug leftovers, task markers, merge markers and oversized code.

Colour-vision deficiency (CVD). Reduced ability to tell some colours apart, often called colour blindness. The Studio simulates protanopia and protanomaly (red), deuteranopia and deuteranomaly (green), tritanopia and tritanomaly (blue), and achromatopsia and achromatomaly (little or no colour).

Command Center. The Studio's searchable list of suggestions, actions, recent colours, findings and files, opened with `Ctrl` + `Alt` + `.`.

Confidence. How sure a detector is that a finding is real: high, medium or low (and contextual for some risky-API rules). Reported separately from severity.

Confusable hues. Two project colours that look clearly different to typical vision but become hard to tell apart under colour-vision deficiency.

Contrast ratio. WCAG 2.x's measure of the brightness difference between two colours, from 1:1 to 21:1. WCAG asks for 4.5:1 for normal text, 3:1 for large text and UI parts, and 7:1 for level AAA.

Contrast Studio. The Studio tool for tuning one text and background pair and writing the fix back to your code.

Custom property. A CSS variable such as `--brand-ink`, used with `var(--brand-ink)`. Often called a design token.

Design token. A named value in a design system, such as a colour role. In CSS, usually a custom property.

Detector version. The version of the checking engine that produced a finding. Findings are fingerprinted with it.

Diagnostic. A problem VS Code shows as an underline in the editor and a row in the Problems panel.

Directive. A special comment the Studio reads, such as `auric-disable-next-line` or `auric-brand-line`.

Display-P3. A colour space with a wider range of colours than standard sRGB, used by many modern screens.

Extension pack. A VS Code extension that only installs other extensions. The Full Pack is one.

Fingerprint. A short identifier calculated from a finding's source, rule, file, line and key detail, used to match findings between runs.

Gamut mapping. Bringing a colour that a screen cannot show (outside its gamut) to the nearest colour it can show. The Studio uses the CSS Color 4 method.

Grade. The letter for a score: A+ (95+), A (90+), B (80+), C (70+), D (55+), F (below 55).

Health record. The `health.json` file: a versioned, fingerprinted list of every issue with scores and a summary.

Host. In a Workstation, the one installed extension that presents the shared sidebar, Command Center and Studio.

Hue-preserving fix. A contrast fix that changes only a colour's lightness, keeping its hue and chroma, so it still looks like the same colour family.

Impact. How much a WCAG finding affects users: critical, serious, moderate or minor.

Inspect. The Auric Accessibility hover card that shows an element's or CSS rule's name, role, colours, contrast and WCAG findings.

Knowledge database. The local, searchable collection of rules, requirements, manual checks, policies, articles, glossary entries and sources.

Large text. Text of at least 24 CSS pixels, or at least 18.66 pixels and bold. WCAG lets it use a lower contrast target.

Lc. APCA's lightness contrast value. The Studio's default APCA target is Lc 60.

Live re-grade. Updating the project score as you type, from files held in memory, without a scan.

Manual check. A step-by-step test a person carries out, for what automation cannot decide.

OKLCH. A colour model with lightness (L), chroma (C, colourfulness) and hue (H) designed so equal steps look equally different. The Studio uses it for shade scales.

Overview. The Studio's home view in the sidebar.

Priority. A 0–100 number in issue hovers that ranks how soon to act on a finding, from **fix now** to **low urgency**.

Project scan. Reading and grading every matching file in the workspace.

Quick fix. A VS Code code action offered with `Ctrl` + `.` on an underline, such as **Fix contrast**.

Redaction. Replacing a detected secret with a placeholder such as `[redacted secret; 40 characters]` so the secret itself is never stored.

Remediation plan. The agent-ready backlog of fixes written by **Generate Agent-Ready Remediation Plan**.

Rulebook. A structured collection of rules and guidance in the knowledge database, such as the Accessibility engineering rulebook.

SARIF (Static Analysis Results Interchange Format). A standard JSON format for analysis results, read by GitHub code scanning and other tools.

Scan cache. The saved result of the last project scan, `scan-cache.json`.

Score. A 0–100 number where 100 means nothing was found.

Severity. How serious a finding is in a record or report: critical, high, medium, low or info.

Signature. One specific code-health detector, such as `ch-secret-github-token`.

Space. A section of the Studio tab listed on its rail, such as Home, Color or Check.

sRGB. The standard colour space of the web and most screens.

Static analysis. Checking code by reading its text, without running it.

Studio. The editor tab with the Studio's larger tools, organised in spaces.

Success criterion. A single testable requirement in WCAG, such as 1.4.3 Contrast (Minimum). Each has a level: A, AA or AAA.

Suppression. Removing a finding with an `auric-disable-…` comment.

Suppressed extension. In a Workstation, an installed extension that is not the host and hides its own views.

Swatch. A small square of colour. The Studio draws one before each colour literal and shows rows of them in pickers.

WCAG (Web Content Accessibility Guidelines). The W3C's accessibility standard. Versions 2.0, 2.1 and 2.2 are recommendations; WCAG 3 is a working draft.

Workstation. Two or three Studio extensions installed together and presented as one product.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Limits and accuracy

What the Studio's colour, contrast, accessibility and code-health checks measure, how they were checked for accuracy, and what they cannot tell you.

The Studio is a static analyser: it reads your source text and does not run your code or render your pages. That makes it fast, private and repeatable, and it also sets clear limits. This page explains both.

What the scores are and are not

- Scores are a **prioritisation and progress measure**, not a certification. A score of 100 means the static checks found nothing, not that the product is accessible or secure.
- Automated testing covers only part of WCAG. Many success criteria need a person: alt-text quality, focus order in practice, screen-reader announcements, media alternatives, error recovery and more. The knowledge database's manual checks cover these.
- Excluded, skipped and unscanned files neither help nor hurt a score.

Colour science

The colour engine is shared by hovers, pickers, the Studio, scans and the command-line scanner, so every surface gives the same numbers.

- **Parsing.** Hex (3, 4, 6 and 8 digits), `rgb()`, `hsl()` and `hwb()` with angle units and `none`, `lab()` and `lch()` (with CSS's D50 white point), `oklab()`, `oklch()`, `color()` in `srgb`, `srgb-linear`, `display-p3`, `a98-rgb`, `prophoto-rgb`, `rec2020` and `xyz`, `color-mix()`, relative colour syntax, the 148 CSS named colours and `transparent`. Invalid values such as `#12345g` are rejected.
- **Wide-gamut colours** outside sRGB are brought into sRGB with CSS Color 4 gamut mapping. Their contrast is measured as a standard screen shows them, and the hover says so.
- **Writing colours back** keeps the token's own syntax and uses the fewest decimals that parse back to exactly the same colour, so editing never drifts a colour.
- **Contrast** uses the WCAG 2.x formula. **APCA** uses APCA-W3 version 0.0.98G and matches that algorithm's published test values.
- **Colour difference** uses ΔE76 and CIEDE2000; CIEDE2000 matches all 34 of the published Sharma test pairs.
- **Colour-vision simulation** uses the Brettel 1997 method in linear light, with partial forms (anomalous trichromacy) by severity. Achromatopsia keeps relative luminance.
- **Translucent colours** are blended with 8-bit alpha, as browsers do, so a blended pixel matches what Chromium draws.
- **Verdicts** use the exact ratio. A ratio just under a threshold is never displayed as the threshold: 4.497 shows as 4.49, not 4.50.

- **Fixes** use the nearest passing colour of the same hue, found by adjusting lightness only. For example, `#888888` on white becomes `#767676` (4.54:1).

Contrast checks

What the scanner handles:

- It reads stylesheets as a tree of rules, including nesting, `@media`, `@supports` and `@layer`. Each rule's own declarations are graded; nested rules never leak into their parent, and two different rules are never paired. Rules with the same selector in the same context are combined, with the later declaration winning. Comments are ignored.
- A text colour is graded against its own background, otherwise its parent rule's (CSS nesting, styled-components) or its ancestor element's (inline styles), otherwise an explicit page background on exactly `body`, `html` or `:root`.
- Large text (24 px, or 18.66 px and bold, from `font-size`, `font-weight`, the `font` shorthand or the default sizes of `h1` to `h3`) is graded at 3:1, or 4.5:1 at the AAA threshold.
- Translucent text and backgrounds and `opacity` are blended. `-webkit-text-fill-color` counts as the text colour.
- Custom properties resolve by scope: the rule's own definitions, then enclosing rules', then the light theme's root values, then any other scope. Fallbacks may nest. A cycle between tokens makes each invalid, so the `var()` fallback applies. A token that is not a colour is not graded.
- Dark themes are graded as themselves: definitions under `prefers-color-scheme: dark` or a dark marker such as `.dark` or `[data-theme=dark]` form a dark theme, and every pair they change is graded again and reported **in the dark theme**. `light-dark()` grades each side in its own theme.
- A colour value is graded when the whole value is one colour, one `var()`, `light-dark()`, `color-mix()` or a relative colour. Anything else, such as a JavaScript expression, is left alone rather than guessed.

What it does not grade:

- text over an image or gradient, transparent (gradient-clipped) text and `currentColor` backgrounds;
- styles for print or forced-colours modes;
- disabled controls, which WCAG exempts;
- code samples in Markdown.

What it cannot see:

- colours set by JavaScript at runtime, or class-driven styles applied to markup in another file;
- which of two different selectors wins in the cascade, inherited and computed values, component state and pseudo-elements;
- the real rendered font size, so a verdict names the check ("fails WCAG AA for normal text") rather than a pixel size.

Measured accuracy. On the audit set used in development, the scanner's verdicts agree with Chromium's rendering in 62 of 68 cases; the remaining six are cases it deliberately skips.

One-click fixes rewrite a value only when the whole value is one colour or one side of `light-dark()`. Mixes, relative colours, faded text and dark-theme pairs that share their text with the light theme open the Contrast Studio instead, so a fix never corrupts source.

Accessibility (WCAG) checks

- 70+ static rules mapped to WCAG success criteria; see the rule catalogue.
- Markup rules read every tag with one shared reader that understands quoted values and JSX braces, so an arrow function or a Vue expression cannot cut a tag short. Attribute names are matched whole (`data-role` is not `role`).

- In JSX, TSX, Vue, Svelte and Astro, capitalised, dotted or namespaced tags (`<Input>`, `<Form.Label>`) are components, not native elements. A file without `<html>` or `<body>` is treated as a component or partial, not a whole page.
- Comments, `<script>` and `<style>` contents, PHP blocks and strings that only look like markup are not scanned.
- Values supplied at runtime (`{{id}}`, `${id}`, `:attr`, `[attr.x]`, `attr={…}`) are treated as absent, not wrong.
- Text inside `aria-hidden` does not count towards a name; template output such as `<%= %>`, `v-text` and `[innerHTML]` does.
- **Measured accuracy.** Checked against axe-core on rendered HTML: on a regression suite of 264 cases, false positives went from 132 to 0 and false negatives from 21 to 0; 80 of 83 rendered cases agree with axe, and the three that differ are contested or best-practice cases.
- Rules that need judgement (alt-text quality, link purpose, dialog naming, authentication) suggest fixes rather than applying them.

Code-health checks

- A high-signal heuristic linter, not a full static application security testing (SAST) tool or a history-scanning secret scanner. Use it alongside dedicated tools where compliance requires them.
- It reads source text with a small per-language lexer, so task markers must be in comments and debug, risk and security rules ignore commented-out or quoted code. It does not build a syntax tree and does not read Git history.
- It is argument-aware: a call with a constant argument is downgraded or ignored, and a call with a dynamic value is reported with high confidence.
- Secrets are scanned everywhere, including comments, and are always redacted.
- **Measured accuracy.** On a labelled set of examples, including a held-out set, precision is 1.00 in every category, and recall is 1.00 in every category except secrets, at 0.96.
- **Limits.** At most 300 findings per file; lines over 240 characters are reported (lines that start with a URL are not); files over 800 lines are reported as large; worst-case inputs such as a 200 KB minified line scan in milliseconds.

Size and scale limits

Limit	Value
Largest file analysed in the editor and in project scans	1 MB
Largest file read by the command-line scanner	1.5 MB
Files per project scan	`scan.maxFiles`, default 4000
Files per command-line scan	`--max-files`, default 20000
Live re-grade	Up to 1,500 files and about 12 MB of text
Issues in `health.json` and SARIF	`health.maxIssues`, default 5000
History entries	`health.historyLimit`, default 200
Recent colours kept	12 (8 shown in the Command Center)

Related

- Scores and grades
- Scanning
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Privacy

What Auric Artisan Studio does with your code and data, what it writes to disk, and when anything uses the network.

Auric Artisan Studio analyses your code entirely on your machine.

What never leaves your machine

- Your source code, colours, scan results, findings and reports.
- Telemetry: the extensions collect none.
- There is no account, sign-in, API key or licence server.

The extensions contain no network code. Their build is checked so that any code that could make a network request fails the build. The command-line scanner makes no network requests either.

When the network is used

Only when you choose to open a link:

- **auricartisan.com**, from the Auric Artisan marks and links in the Overview, the Studio and hovers. The Studio's screens will only open links to that one site.
- **W3C documentation**, from **WCAG docs** and **Open WCAG Documentation** links on findings, and links inside knowledge-database entries to official sources.

These open in your browser. Installing or updating the extensions from the Marketplace is handled by VS Code.

The remediation plan and "AI"

Generate Agent-Ready Remediation Plan calls no AI model and sends nothing anywhere. It writes a plan, including a prompt template, to your workspace. If you choose to give that prompt to a coding agent, that agent's own privacy terms apply.

What is written to your disk

Where	What	When
`.auric-artisan/` in your workspace	Scan cache, settings copy, health record, SARIF, history, reports, remediation plan, knowledge database and overrides, a README and a `.gitignore`	When you scan, generate outputs or save Studio settings; see The .auric-artisan folder
Your VS Code user settings	The accent colour; colour picker mode switches; `editor.colorDecorators` when Auric Color manages VS Code's colour decorators	When you change them
Your workspace's VS Code settings	Values saved from Studio › Settings	When you select **Save settings**
VS Code's storage for the extension	Recent colours, dismissed suggestions, the Project Issues grouping, which Studio sections you left open, whether you answered the colour picker prompt, and small colour icons for the Project Issues view	As you use the Studio
Files you choose	Palette, shade, brand and Markdown exports	When you export through a save dialog

To stop the Studio writing to your workspace, turn off **Store workspace data locally** (`data.enabled`). To remove everything, delete the `.auric-artisan` folder and uninstall the extensions.

Secrets

Code Health never stores a detected secret. The secret's text is replaced by a note such as `[redacted secret; 40 characters]` everywhere it could appear: diagnostics, the Studio, reports, `health.json`, SARIF, the history, the scan cache and remediation tasks. See Secrets and redaction.

Sharing reports

Reports and records are ordinary files. They contain file paths, line numbers, colour values and messages from your project, but never secret values. Review them before sharing outside your team.

Related

- Limits and accuracy
- FAQ

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Release notes

The version history of Auric Artisan Studio, taken from the changelog that ships with the extensions.

All four Studio extensions share one version number. These notes are taken from the changelog bundled with the extensions (the **Changelog** tab of each Marketplace listing), rewritten for readers of these docs.

Unreleased

The changelog lists the following changes under **Unreleased**, which means they are not yet recorded against a version number; they are the upcoming entries for the next numbered release. The rest of these docs describe the extensions as they behave in the package currently offered on the Marketplace.

Accuracy

Every engine was measured against an external reference, and each defect found was fixed with a permanent regression test: colour science against published reference values, contrast against Chromium's rendering, the WCAG rules against axe-core, and code health against a labelled set of examples.

Colour:

- Every CSS Color 4 and 5 syntax is parsed: `oklch()`, `lab()`, `lch()`, `hwb()`, `color()` with Display-P3, Rec. 2020 and other spaces, `color-mix()`, relative colour, `none`, angle units and `transparent`. Wide-gamut colours use CSS gamut mapping.
- Invalid hex values such as `#12345g` are rejected.
- CIEDE2000 colour difference was added and matches all 34 Sharma test pairs; colour naming now uses it.
- Colour-vision simulation now uses the Brettel 1997 method in linear light. The previous method brightened red for protanopes. Achromatopsia keeps relative luminance.
- The suggested fix is the nearest passing colour: `#888` on white now becomes `#767676` (4.54:1), not `#717171`.

- Ratios just under a threshold never display as the threshold.
- Translucent colours blend with 8-bit alpha, matching Chromium's pixels.
- Writing colours back round-trips every CSS format exactly. Previously `hsl()` output rounded to whole numbers and moved the colour, and `lch()` used the wrong white point.

Contrast scanning, rebuilt on a CSS rule tree (agreement with Chromium on the audit set rose from 14 to 62 of 68 cases; the six left are deliberate skips):

- Large text is graded at 3:1.
- Translucent text, translucent backgrounds and `opacity` are blended.
- Dark-mode token overrides no longer grade the light theme; dark themes are graded as themselves (**in the dark theme**).
- Only real `body`, `html` and `:root` rules set the page background.
- Nested and `@media` rules are graded once, never paired across rules; comments are ignored.
- `color-mix()`, relative colours and `light-dark()` are graded; cyclic tokens fall back as the CSS specification says.
- Inline colours sit on their ancestor element's background.
- Disabled controls and Markdown code samples are exempt; pass or fail uses the exact ratio.
- The Quick Fix never corrupts source: it rewrites a value only when the whole value is one colour (or one side of `light-dark()`); anything else opens the Contrast Studio.
- The Contrast Studio writes token fixes to the light theme's `:root` value, not to a later dark override.

Code health (detector 1.2.0; on a labelled set including a held-out set, precision is 1.00 in every category and recall is 1.00 except secrets at 0.96, up from 0.53):

- Secret detection now covers JSON, PHP, typed and Go assignments, prefixed names such as `MYSQL_PASSWORD`, unquoted `.env`, YAML and INI values, OpenAI `sk-proj-` keys, Google keys ending in `-`, and Azure and ADO.NET connection strings.
- False positives removed: placeholders (`CHANGE_ME`, `{{API_KEY}}`, `$VAR`, SOPS `ENC[…]`), translation labels, autocomplete tokens and key-file paths, `redis.eval` and `model.eval()`, tagged SQL templates, phone masks read as `XXX` markers, and Markdown underlines read as merge markers.
- One credential is one finding.
- Secret bytes no longer leak through task-marker messages or security evidence into SARIF or `health.json`.
- A call whose arguments cannot be read is now reported at medium confidence instead of being passed.
- A mention of `auric-disable-file` in prose or a string no longer silences a whole file.
- Task markers, `debugger` and breakpoints are recognised only where they belong in each language.
- The underline covers the finding's real span.
- Worst-case inputs (a 200 KB minified line, a 5,000-line file) scan in 5–30 ms.
- Because findings are fingerprinted by detector 1.2.0, a CI baseline written by detector 1.1.0 may list some findings as new once.

WCAG rules (detector 1.2.0; on a 264-case regression suite, false positives fell from 132 to 0 and false negatives from 21 to 0; 80 of 83 rendered cases agree with axe-core, and the three that differ are contested or best practice):

- One tag reader for every rule: a JSX `=>` or a Vue `v-if="n > 0"` no longer cuts a tag short; attributes are whole names; bound values are respected.
- Components such as `<Input>`, `<Button>` and `<Form>` are no longer judged as native elements.
- Correct markup no longer fails: required owned roles on `<ul role="list">`, radio groups and table rows; required context in component files; `aria-selected` on `option`; `lang` on `<style lang="scss">`; image alt with `aria-label` or spreads; Razor, Material and shadcn labels; unnamed forms as landmarks; duplicate IDs inside `<template>`.

- Accessible names now come from labelled `svg` and `img` children, template output and `v-text`; `aria-hidden` text no longer counts.
- `focus-outline-none`: one `:focus-visible` rule no longer exempts a whole file.
- `interactive-keyboard` understands `tabIndex={0}` and catches Vue, Angular and Svelte click handlers.
- New rule `input-placeholder-label` (3.3.2, advisory).
- Six rules are now marked best practice rather than WCAG failures: `accesskey`, `autofocus`, `heading-skip`, `positive-tabindex`, `tiny-font` and `text-spacing-important`. They are shown as hints and weigh half in the score.
- A case that never finished (60,000 links) now takes 0.28 s; scanning 2,930 real markup files went from 7.6 s to 6.2 s.

Studio redesign

- Every Studio screen now uses the Auric look shared with auricartisan.com and the browser extension: warm near-black on dark themes, paper on light themes, Fraunces headings and scores, Manrope text and gold primary actions. The fonts are bundled; nothing is fetched. High-contrast themes keep VS Code's own colours.
- The Studio is one tab with a rail of five spaces (Home, Color, Check, Health, Brand) plus Settings, Rulebook and Icon Studio at the foot. Each space opens with a header, and spaces with several tools have their own tab row. Each extension shows only the spaces it carries.
- A new Home space: the score ring with Accessibility, Contrast and Code-health tiles, files to fix first, suggestions, the CI gate command and the project palette. The findings list moved to **Check › Findings**, which the **pattern-rule** suggestion now opens.
- The Overview was rebuilt: a workspace card, a **This file** card with **Fix N in this file**, suggestions, recent colours and doors into the Studio, Rulebook, Icon Studio and fix plan, with one gold action per view.
- **Generate Project Health Report** also writes a printable, self-contained `project-health.html`, with **Open in Browser**.
- Contrast hovers lead with the ratio and verdict, show the pair, APCA Lc and the nearest passing colour of the same hue, and offer **Apply to --token**, **Open in Contrast Studio** and **Intentional branding**.
- New **Auric: N in this file** status-bar item.
- The Studio's Contrast tool is now the Contrast Studio: the pair on its background with four WCAG checks, passing colours that keep the hue, where the token is used, and write-back to the token or one use.
- The issue hover's **Apply fix** button is gold.

0.1.0 — Initial Studio release

The first public release of Auric Artisan Studio: a privacy-first workstation delivered as three focused extensions and one Full Pack.

Package	Focus
Auric Color	Colour picking, contrast, palettes, OKLCH scales and colour-vision previews
Auric Accessibility	WCAG 2.0, 2.1 and 2.2 auditing, APCA analysis, Inspect and project scans
Auric Code Health	Secrets, debug leftovers, task markers, risky APIs, size checks and SARIF
Full Pack	Installs Color, Accessibility and Code Health only

Studio experience:

- A Workstation that lets independently installed extensions behave as one product, without duplicate views, scans or diagnostics, with live host switching when extensions are installed or removed.
- The Full Pack installs only the three Studio extensions; icon themes and icon libraries are not included.
- A navigation rail, page summaries, action cards, consistent dialogs and a theme-aware settings experience.

- Custom dropdowns, switches, tooltips, focus management, keyboard navigation, visible focus, reduced-motion and forced-colours support throughout.
- Issue hovers built around priority, confidence, conformance, source context, fix guidance, ignore actions and rule references.
- Command Center shortcuts, capability-aware actions and deterministic suggestions.

Colour:

- Colour detection and editable swatches across CSS and 40+ programming, template, data and markup languages.
- WCAG contrast, APCA, RGB, HSL, OKLCH, Display-P3, colour naming, accessible alternatives, palette generation, shade scales and colour-vision simulation.
- Colour editing inline, from the hover and in the docked picker.

Accessibility:

- A WCAG 2.2 engine with structural, semantic, ARIA, language, form, media, focus, motion and contrast checks.
- Inspect for element-level names, roles, colours, contrast, colour-vision impact, issues and fixes.
- Workspace scans, grouped findings, score summaries, suppressions, quick fixes, remediation plans and rule documentation.
- Framework and template awareness, so comments, runtime bindings and inert source are not reported.

Code health and reporting:

- Checks for exposed secrets, injection risks, unsafe APIs, merge markers, debug code, task markers, oversized files and maintainability problems.
- Secret evidence redacted before it reaches diagnostics, reports or the Studio.
- Project health records, JSON and SARIF 2.1.0 output, stable fingerprints, baselines, trend history and an offline command-line scanner.
- A versioned local knowledge base of rules, requirements, procedures, policies, glossary entries, sources and remediation guidance.

Accessibility and trust:

- Every Studio screen designed for WCAG 2.2 AA and checked across light, dark, high-contrast, narrow and zoomed layouts.
- A no-network runtime, a strict content security policy for the Studio's screens, local-only analysis and deterministic builds.

VS CODE EXTENSIONS › AURIC ARTISAN STUDIO

Related

Other Auric Artisan extensions, website tools, articles and standards that work alongside Auric Artisan Studio.

Inside this documentation

- Auric Color, Auric Accessibility, Auric Code Health and the Full Pack.
- Installing and setting up Auric Artisan's VS Code extensions.

Other Auric Artisan VS Code extensions

- Auric Artisan Icon Studio: the combined icon library that **Open Unified Icon Studio** opens.

- Icon collections that provide it: Auric Icons Modern, Auric Icons Drawn and Auric Icons Playful.
- Auric Artisan Theme.

The browser extension

The Auric Artisan browser extension brings colour and accessibility tools to web pages in your browser, where you can check what is actually rendered rather than the source.

Tools on auricartisan.com

Tool	Use it to
Contrast Checker	Check a colour pair in the browser
Analyzer	Analyse a web page
Accessibility Checklist	Work through WCAG criteria by hand
Accessibility Compliance	Review accessibility compliance
Vision Simulation	Simulate colour-vision deficiencies
Basic Colour Tools	Everyday colour tools
Design Engine	Generate a design system
Tonal Steps	Build tonal scales

The Learn library has articles on colour, contrast and accessibility.

Standards the Studio refers to

- Web Content Accessibility Guidelines (WCAG) 2.2: https://www.w3.org/TR/WCAG22/
- Understanding WCAG 2.2 (linked from every WCAG finding): https://www.w3.org/WAI/WCAG22/Understanding/
- WAI-ARIA 1.2: https://www.w3.org/TR/wai-aria-1.2/
- HTML Accessibility API Mappings: https://www.w3.org/TR/html-aam-1.0/
- ACT Rules Format 1.1: https://www.w3.org/TR/act-rules-format/
- OWASP Top 10:2025: https://owasp.org/Top10/2025/0x00_2025-Introduction/
- CWE-798, Use of Hard-coded Credentials: https://cwe.mitre.org/data/definitions/798.html
- SARIF 2.1.0: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Auric Artisan Theme

What the Auric Artisan Theme extension for VS Code is, who it is for, how to install it and where to find each guide.

Auric Artisan Theme is a colour theme collection for Visual Studio Code. It ships 54 ready-made themes in six surface families (Obsidian, Midnight, Parchment, Porcelain, Dusk and Forest) plus two high-contrast builds, one dark and one light. Every theme is keyed to a single accent colour, so the whole workbench can be re-coloured in one step.

The extension also adds a live editor for the theme, called **Theme Studio**. Theme Studio has six tabs (**Looks**, **Dials**, **Colours**, **Audit**, **Share** and **Preview**). It gives you 11 complete looks, 37 theme dials and 107 individually editable colour slots. Every change applies to your editor straight away, and a contrast guard keeps text readable while you experiment.

A compact **Auric Theme** view in the Activity Bar gives you quick access to variants, accents, looks and the contrast status without opening the full Studio. Theme Studio and the sidebar can be shown in English or Hindi, independently of VS Code's own display language.

Who it is for

- Developers who want a warm, carefully balanced theme that works in dark, light and high-contrast settings.
- Designers and front-end developers who want to tune an editor theme precisely, down to a single colour slot.
- Accessibility-minded users who want every text colour checked against WCAG contrast ratios and APCA values.
- Hindi-speaking users who prefer a Hindi interface in the Studio and sidebar.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-theme
Extension ID	`auric-artisan.auric-artisan-theme`
Current version	0.2.0
Requires	Visual Studio Code 1.75 or later
Price	Free to install and use

Install from a terminal:

```
code --install-extension auric-artisan.auric-artisan-theme
```

Or open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Artisan Theme** and select **Install**.

Quick start

1. Install the extension.
2. Open the Command Palette (`Ctrl` + `Shift` + `P`) and run **Auric Theme: Switch Variant...**.
3. Pick a theme, for example **Auric Artisan — Obsidian Gold**. The editor changes immediately.
4. Run **Auric Theme: Open Theme Studio**, or press `Ctrl` + `Alt` + `;` (`Cmd` + `Alt` + `;` on macOS).
5. On the **Looks** tab, choose a starting look such as **Signature** or **Clarity**.
6. On the **Dials** tab, adjust the accent, surfaces, syntax palette and other dials. Changes apply live.

7. Open the **Audit** tab to check that every text colour still meets your contrast minimum.
8. On the **Share** tab, save a named preset so you can return to this set-up later.

What you can do

- Choose from 54 themes: 10 accents each for Obsidian, Midnight, Parchment and Porcelain, 6 each for Dusk and Forest, and 2 high-contrast builds.
- Re-key the whole theme onto any of 17 accents, or onto a custom hex colour.
- Start from one of 11 looks, then layer 37 dials on top: surfaces, text, syntax palette, italics and bold, cursor, selection, brackets, indent guides, workbench bars, terminal palette, shadows, focus ring and accessibility guard.
- Override any of 107 colour slots with a custom colour picker (hex, saturation and brightness, hue, opacity and RGB channels).
- Audit 81 foreground and background pairs with WCAG ratios and APCA Lc values, in the Studio or as a plain-text contrast report.
- Compare the shipped palette with your current palette and copy the result as CSS custom properties.
- Save, load and delete named presets; copy or paste your customisation as JSON; export a standalone VS Code colour theme file.
- Switch the Studio and sidebar between **Auto**, **English** and **हिन्दी** without reloading.

In this folder

docs/

- Contents of docs/
- Getting started — install, switch theme, the screen tour, your first customisation.
- Choose a theme — the six families, the high-contrast builds and every way to switch.
- Looks and dials — start from a look and tune the 37 dials.
- Edit individual colours — the Colours tab and the colour picker.
- Check contrast — the Audit tab, the contrast report and the contrast guard.
- Compare and preview — the Preview tab and CSS export.
- Save, share and export — presets, JSON customisation and theme files.
- The Theme sidebar — the Activity Bar view in detail.
- Interface language — English and Hindi.
- Reference — every command, setting, keybinding, dial, slot and option.
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related tools and extensions
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Auric Artisan Theme guides

Contents of the Auric Artisan Theme guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for the Auric Artisan Theme extension. Start with Getting started if you are new, then use the task guides for each workflow. The reference lists every command, setting, keybinding, dial, colour slot and option.

File	What it covers
getting-started.md	Install, switch to an Auric theme, tour Theme Studio and make a first change.
choose-a-theme.md	The six families, 54 variants, high-contrast builds and every way to switch.
looks-and-dials.md	The 11 looks and the 37 dials, with search, filters and resets.
edit-colours.md	Override any of the 107 colour slots with the custom colour picker.
check-contrast.md	Read the Audit tab and contrast report; tune the contrast guard.
compare-and-preview.md	Compare shipped and current palettes; copy colours as CSS.
save-share-and-export.md	Presets, customisation JSON, clipboard and standalone theme files.
theme-sidebar.md	Every part of the Auric Theme view in the Activity Bar.
interface-language.md	Switch Theme Studio and the sidebar between English and Hindi.
reference.md	Every command, setting, keybinding, dial, slot, look, accent and palette.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Getting started with Auric Artisan Theme

Install the theme, switch to an Auric variant, tour Theme Studio and make your first live customisation.

This guide takes you from installation to a first saved customisation in a few minutes.

Before you begin

- You need Visual Studio Code 1.75 or later.
- The extension works in untrusted and virtual workspaces.
- Customisations only apply while one of the Auric Artisan themes is active. If another theme is active, the customisation commands ask you to pick an Auric theme first.

1. Install the extension

1. Open the Extensions view with `Ctrl` + `Shift` + `X`.
2. Search for **Auric Artisan Theme** (publisher **auric-artisan**).
3. Select **Install**.

You can also run this in a terminal:

```
code --install-extension auric-artisan.auric-artisan-theme
```

After installation, VS Code may show the **Make it yours** walkthrough on its Welcome page. It has four steps: **Pick a variant**, **Change the gold**, **Open the Studio** and **Keep it readable**. Each step has a button that runs the matching command.

---

2. Switch to an Auric theme

1. Open the Command Palette with `Ctrl` + `Shift` + `P`.
2. Run **Auric Theme: Switch Variant...**.
3. The list shows all 54 themes. Each entry shows whether it is dark or light and the hex value of its canvas (the editor background). The theme you are using is marked **current**.
4. Select a theme. VS Code switches immediately.

You can also use VS Code's own **Preferences: Color Theme** picker. Every Auric theme starts with **Auric Artisan —**, for example **Auric Artisan — Midnight Sapphire**.

When an Auric theme is active, a status bar item appears on the right with a paint-can icon and the short theme name, for example **Obsidian Gold**. An asterisk after the name means you have customised the theme. Hover over it to see how many dials and colours you have changed; select it to open Theme Studio.

---

3. Open Theme Studio

Use any of these:

- Press `Ctrl` + `Alt` + `;` (`Cmd` + `Alt` + `;` on macOS).
- Run **Auric Theme: Open Theme Studio** from the Command Palette.
- Select the status bar item.
- In the **Auric Theme** view in the Activity Bar, select **Open Theme Studio**.

Theme Studio opens as an editor tab called **Auric Theme Studio**.

---

4. Tour the Studio

From top to bottom, the Studio shows:

Area	What it shows
Top bar	The Auric Artisan brand, a link to auricartisan.com, the **Auto · English · हिन्दी** language switch and an **Applied live** chip.
Introduction	"Make it yours." with an **Explore colours** → button that jumps to the Colours tab.
Active theme card	The current theme name, **Applied live**, a **Dark** or **Light** chip and an **As shipped** or **Customised** chip.
Metrics	**Built-in variants**, **Theme dials**, **Colour slots** and **Your changes** (modified dials plus colour overrides).
Tabs	Six numbered tabs: **Looks**, **Dials**, **Colours**, **Audit**, **Share**, **Preview**. The Audit tab shows a badge when pairs fail.
Tab content	The cards for the selected tab.
Footer	**Reset all**, **Randomize**, **Settings** and a count such as "2 overrides · 3 dials changed".

Press the number keys `1` to `6` to jump between tabs, or use the arrow keys, `Home` and `End` while a tab has focus. Press `/` to jump to the colour search on the Colours tab.

If something about the theme cannot be applied, a banner under the tabs lists the problem.

5. Make a first change

1. Select the **Dials** tab.
2. In the **Accent** card, select the **Sapphire** swatch under **Accent colour**.
3. Watch the editor: tab rules, selections, focus rings and every accent-derived colour move to blue at once.
4. In the **Syntax** card, open **Syntax palette** and choose **Cool**.
5. A dot and a reset button appear beside each dial you changed. The footer count now shows two dials changed.

6. Check it is still readable

1. Select the **Audit** tab.
2. The top card shows the worst pair in the theme and either **All clear** or the number of failing pairs.
3. Scroll the **Measured pairs** table. Failing pairs are listed first.

See Check contrast for how to read the table.

7. Save your work

1. Select the **Share** tab.
2. In **Presets**, type a name such as "Blue work" and select **Save** (or press `Enter`).
3. The preset appears under **Saved** with the date and time.

You can now experiment freely. To return, select **Load** beside the preset.

Result

Your editor is using an Auric theme with your chosen accent and syntax palette, the contrast audit has checked every text pair, and your set-up is saved as a named preset.

Next steps

- Choose a theme — explore the families.
- Looks and dials — every dial explained.
- Edit individual colours — fine control over 107 slots.
- Save, share and export — move your theme between machines.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Check contrast

Read the Audit tab and the plain-text contrast report, and control the contrast guard that keeps your theme readable.

Every colour the theme produces is checked against the surface it actually sits on. You can read the results in three places: the **Audit** tab in Theme Studio, the **Contrast** section of the sidebar, and a plain-text contrast report.

Key terms

- **Contrast ratio (WCAG):** a number from 1:1 to 21:1 comparing the luminance of text and background. WCAG 2 AA asks for 4.5:1 for body text and 3:1 for large text and control boundaries; AAA asks for 7:1.
- **APCA Lc:** a perceptual contrast measure from the draft WCAG 3 work. Unlike the ratio, it accounts for polarity (dark on light versus light on dark). Roughly, Lc 60 suits body text, Lc 45 large text, and Lc 30 is the floor for anything you must read. A negative value means light text on a dark background.
- **Minimum:** the threshold a particular pair is held to. Decorative washes are held to less than text.

What is audited

The audit checks 81 pairs per theme: editor text, all 26 syntax roles, the 14 text-bearing ANSI terminal colours, source control decorations, and the text and borders of dropdowns, inputs, checkboxes, radios, buttons, badges, menus and quick picks. Form-control borders are held to 3:1 against the fill they enclose (WCAG 1.4.11).

Use the Audit tab

1. Open Theme Studio and select **Audit** (or press `4`). If any pair fails, the tab shows a badge with the number of failures.
2. The top card shows the worst pair in the theme, its ratio, and either "{n} failing" or **All clear**.
3. The **Measured pairs** table lists every pair, failures first, then by ratio.

Column	Meaning
Pair	An "Aa" sample drawn in the real colours, followed by the pair name.
Ratio	The WCAG contrast ratio, such as 7.12:1.
APCA	The APCA Lc value.
Minimum	The threshold for this pair.
Result	**AAA** (7:1 or more), **AA** (meets its minimum), **Tight**, **Fail** (below minimum) or **Missing** (a colour the pair depends on is missing, so it could not be measured).

A **How to read this** card at the bottom repeats these definitions.

Use the sidebar

In the **Auric Theme** view, the **Contrast** section shows the worst pair, a **Pass** or **Fail** chip and its ratio. Select it (**Open audit ›**) to open Theme Studio on the Audit tab. Before the first audit it reads **Not audited yet**.

Open the plain-text contrast report

1. Run **Auric Theme: Show Contrast Report**, or select the checklist icon in the sidebar's title bar.
2. A read-only document called "Auric contrast report" opens in a preview tab.

The report lists the theme name and polarity, the number of audited and failing pairs, then one row per pair with a result flag (**AAA**, **AA** or **FAIL**), the WCAG ratio, the APCA Lc value, the minimum and the pair name. Failures are sorted first. The footer explains AA, AAA and APCA Lc.

The report updates when the theme is re-applied. You can copy it or save it like any other document.

The contrast guard

When **Enforce contrast** (`auricTheme.a11y.enforceContrast`) is on, the default, every colour the customiser produces is adjusted along its OKLCH lightness until it clears your minimum, keeping its hue and chroma. The guard runs after your own colour overrides: a colour you choose is treated as a strong preference, but not as permission to make text unreadable.

To change the guard:

1. Open the **Dials** tab, **Accessibility** card.
2. Set **Minimum contrast** between 3 and 7 (default 4.5). Use 7 for AAA-level text.
3. Turn **Enforce contrast** off if you want exact colours. The audit still reports failures.
4. Turn **Audit on apply** off if you do not want a warning notification each time failures appear.

When **Audit on apply** is on and a change leaves pairs below the minimum, VS Code shows "Auric Theme: {n} colour pairs no longer meet your contrast minimum." Select **Show report** to open the contrast report.

Result

You know which pairs pass, which are tight and which fail, and you can adjust the dials or overrides until the Audit tab shows **All clear**.

> **Note:** The audit measures the colour pairs the theme defines. It is guidance, not a guarantee that every possible customisation meets accessibility requirements. See Limits and accuracy.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Choose a theme

The six surface families, 54 variants and two high-contrast builds, and every way to switch between them.

Auric Artisan Theme ships 54 built-in variants. A variant is one complete theme: a surface family (the neutral backgrounds and text) combined with an accent colour.

The families

Family	Surface	Polarity	Variants
Obsidian	Warm charcoal	Dark	10
Midnight	Near-black	Dark	10
Parchment	Warm cream	Light	10
Porcelain	Cool white	Light	10
Dusk	Violet-tinted dark	Dark	6
Forest	Green-tinted dark	Dark	6
High contrast	Gold, dark and light	One dark, one light	2

Accents per family

Family	Accents
Obsidian, Midnight, Parchment, Porcelain	Gold, Sapphire, Rose, Amethyst, Patina, Emerald, Ember, Crimson, Iris, Graphite
Dusk	Gold, Sapphire, Rose, Amethyst, Iris, Graphite
Forest	Gold, Patina, Emerald, Ember, Rose, Graphite
High contrast	Obsidian Gold (High Contrast) is dark; Porcelain Gold (High Contrast) is light

Every theme appears in VS Code as **Auric Artisan — Family Accent**, for example **Auric Artisan — Parchment Emerald**. The two high-contrast builds are **Auric Artisan — Obsidian Gold (High Contrast)** and **Auric Artisan — Porcelain Gold (High Contrast)**. See the reference for every theme and its ID.

> **Tip:** The variant's accent is only its starting point. You can re-key any variant onto any of 17 accents, or a custom colour, with the **Accent colour** dial.

Ways to switch

From the Command Palette

1. Run **Auric Theme: Switch Variant...**.
2. Type part of a name, such as "forest", to filter the list.
3. Select a theme. Each row shows its polarity and canvas colour, and the active theme is marked **current**.

Cycle through all variants

Press `Ctrl` + `Alt` + `'` (`Cmd` + `Alt` + `'` on macOS), or run **Auric Theme: Cycle Variant**. Each press moves to the next of the 54 variants, in the order shown in the reference. If a non-Auric theme is active, cycling starts at Obsidian Gold.

From the Auric Theme sidebar

1. Select **Auric Theme** in the Activity Bar.
2. Under **Variant**, open the searchable dropdown and pick a theme, or
3. Expand **Browse all swatches** to see compact family galleries. Each family is a collapsible group with a count and small colour-strip tiles. Select a tile to switch.

The sidebar remembers which families you expanded.

From VS Code's own picker

Run **Preferences: Color Theme** (`Ctrl` + `K`, then `Ctrl` + `T`) and choose any **Auric Artisan —** entry.

What happens to your customisations

- Dials (such as accent, surfaces and syntax palette) are stored in your VS Code user settings and follow you to every Auric variant.
- Colour slot overrides are stored per variant. Overrides you make on Obsidian Gold do not appear on Midnight Sapphire.
- When you switch to an Auric variant, your customisation is re-applied to it automatically.

- When you switch to a non-Auric theme, your customisations stop showing, because they are scoped to the Auric theme names. They come back when you switch back.

Result

Your editor uses the Auric variant you picked, and the status bar item shows its short name.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Compare and preview

Use the Preview tab to compare the shipped palette with your current colours and copy the palette as CSS custom properties.

The **Preview** tab shows the shipped palette of the active variant next to your current palette, so you can see exactly what your changes did.

Compare palettes

1. Open Theme Studio and select **Preview** (or press `6`).
2. Open the **Comparison view** dropdown and choose:
   - **Side by side** — both palettes (the default).
   - **As shipped** — only the variant's original palette.
   - **Your palette** — only your current colours.
3. Each specimen shows a title bar, a short syntax colour sample (a comment, keyword, variable, properties, a string, a number, a constant and a function call) and six palette chips: `bg.base`, `fg.default`, `accent.base`, `syntax.string`, `syntax.keyword` and `syntax.fn`, each with its hex value.
4. Below the specimens, a line reads "{count} colour slots differ from the shipped variant. Theme edits apply live."

The specimen is an illustration drawn from the theme engine's colours. It is not a full VS Code editor, and it is not a "try before you apply" mode: every edit you make on other tabs is already live in your editor.

Copy the palette as CSS

1. On the Preview tab, select **Copy palette as CSS**.
2. The full resolved palette is copied to the clipboard as CSS custom properties inside a `:root` block. Each slot becomes a property named `--auric-` followed by the slot path with dots replaced by hyphens.

Example: for an uncustomised Obsidian Gold, the copied block includes lines like these.

```
:root {
  --auric-bg-base: #0C0C0E;
  --auric-fg-default: #E8E4DA;
  --auric-accent-base: #D3AF37;
  --auric-syntax-string: #C3D9A5;
}
```

Your copy contains the current values of all 107 slots.

This exports the palette. It does not convert VS Code's interface into CSS.

Result

You can see which slots changed, compare them against the shipped variant and reuse the palette in a web project.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Edit individual colours

Override any of the 107 colour slots on the Colours tab using the custom colour picker, and revert them.

The theme is built from 107 **colour slots**. A slot is one named colour, such as **Canvas** (`bg.base`) or **Keyword** (`syntax.keyword`), that many parts of the workbench or syntax highlighting draw from. On the **Colours** tab you can override any slot. An override wins over whatever the dials produced.

Open the Colours tab

1. Open Theme Studio (`Ctrl` + `Alt` + `;`).
2. Select the **Colours** tab, press `3`, or select **Explore colours** → in the introduction.

Find a slot

Slots are grouped into 12 collapsible cards:

Group	Contents
Surfaces	The planes the workbench is built from, deepest to nearest (8 slots).
Text	Foreground ramp, from body copy to disabled (6).
Lines	Every border, divider and focus ring (7).
Accent	The accent fills, emphasis and washes derived from it (7).
States	Hover, active, selected and drop feedback (5).
Selection	Editor selection, find, word and range highlights (10).
Semantics	Error, warning, info and success, quiet and loud (8).
Semantic washes	Tinted backdrops behind semantic messages (4).
Terminal	The 16 ANSI colours (16).
Source control	File decoration colours in source control views (9).
Syntax	Token colours shared by semantic and TextMate highlighting (26).
Effects	The shadow colour (1).

- Select a group heading to expand or collapse it. The Studio remembers which groups are open.
- Each group shows a count. When you have overrides in a group, the count reads like "2/26".
- Type in **Filter by name or path — press / to jump here** to filter across all groups. Matching groups open automatically. Press `Escape` in the filter to clear it.
- If nothing matches, the tab shows "No slots match that filter."

Override a slot

1. Find the slot row. It shows the slot label, its path (for example `syntax.string`) and a colour field.
2. Type a hex value into the field, or select the swatch to open the colour picker.
3. The editor updates live as you drag in the picker, and again when you commit the value.

The colour field accepts `#rgb`, `#rrggbb` or `#rrggbbaa`. Any other value shows "Needs #rgb, #rrggbb or #rrggbbaa".

The colour picker

Control	What it does
Hex field	Type an exact value.
Saturation and brightness plane	Drag to set saturation and brightness. Arrow keys nudge the value.
Hue slider	Rotate the hue.
Opacity slider	Shown for slots that allow transparency (washes, selections, states).
RGB channels	Numeric **Red**, **Green** and **Blue** inputs, plus opacity where allowed. Values are range-checked.
Recent colours	Up to eight colours you picked in this panel session. They are not saved after the panel closes.
Clear / inherit	Removes the value so the slot follows the dials again.

Slots are one of three kinds: solid (no transparency), alpha (transparency allowed) or either.

Revert an override

When a slot is overridden, its row shows **Shipped** with the original colour and hex value, and a revert button. Select the revert button to return the slot to the shipped colour.

To remove every override and dial change at once, use **Reset all** in the footer. It asks for confirmation: the button changes to **Reset everything?** and you must select it again within four seconds.

Things to know

- Overrides are stored per variant. An override on Obsidian Gold does not affect Parchment Gold.
- The contrast guard still runs after your overrides. If a colour you chose would fail your minimum contrast, the guard adjusts its lightness. Turn off **Enforce contrast** in the Accessibility dials if you want your exact value, and check the Audit tab afterwards.
- Changing a slot triggers a full recompute of the theme, so dragging in the picker sends updates at a limited rate.

Result

The overridden slots show your colours in the editor, each row shows the shipped colour for comparison, and the group counts show how many overrides each group holds.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Interface language (English and हिन्दी)

Switch Theme Studio and the Theme sidebar between English and Hindi, and understand what follows VS Code's own display language.

Theme Studio and the Theme sidebar can be shown in English or Hindi. You do not need a VS Code language pack for this.

Choose a language

Use any of these:

- In Theme Studio's top bar, select **Auto**, **English** or **हिन्दी**.
- In the sidebar's language bar, select **Auto**, **EN** or **हिं**.
- Run **Auric Theme: Choose Interface Language...** and pick **Auto (VS Code)**, **English** or **Hindi**. The current choice is marked **current**.
- Set `auricTheme.interfaceLanguage` in Settings to `auto`, `en` or `hi`.

Value	Behaviour
`auto` (default)	Follows the VS Code display language. Hindi locales use Hindi; every other locale uses English.
`en`	Always English in Theme Studio and the sidebar.
`hi`	Always Hindi in Theme Studio and the sidebar.

The choice is saved in your user settings and shared by both panels. The switch rebuilds the labels immediately. Your current tab, filters, expanded groups, scroll position and any text you had typed in the Share tab are kept.

What changes and what does not

Part of VS Code	Language used
Theme Studio and the Theme sidebar	Your `auricTheme.interfaceLanguage` choice.
Theme names in the pickers and status bar	VS Code's display language. With Hindi as the display language, all 54 names appear in Hindi, for example "ऑरिक आर्टिजन — ऑब्सिडियन गोल्ड".
Command Palette titles, Settings descriptions, notifications and the walkthrough	VS Code's display language. The extension ships Hindi versions of all of them.
Setting IDs, slot paths and hex values	Always English and unchanged.

To change VS Code's own language, run **Configure Display Language** and install or pick a language pack.

Hindi text in the panels uses Devanagari-friendly fonts and line height.

Result

The Studio and sidebar appear in your chosen language, while VS Code's own menus follow its display language.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Looks and dials

Start from one of 11 looks, then tune the 37 theme dials on the Dials tab, with search, filters and resets.

The theme is generated from a small number of decisions called dials. A **dial** is one setting, such as the accent colour or how strongly panes separate. Change a dial and every colour that depends on it is recomputed, with the contrast guard applied on top. A **look** is a named combination of dial values that you can apply in one click.

Both are available in Theme Studio. Dials are also ordinary VS Code settings under `auricTheme.*`, so you can change them in the Settings editor or in `settings.json`.

Apply a look

1. Open Theme Studio (`Ctrl` + `Alt` + `;`).
2. Select the **Looks** tab (or press `1`).
3. Under **The set**, select a look.

Look	What it does
Signature	The theme exactly as shipped. Applying it clears every dial back to its default.
Gallery	Everything turned up: layered surfaces, gold edges, visible controls. Built for demos and projectors.
Atelier	Warm, soft and unhurried. Expressive italics, quiet chrome, pastel tokens.
Foundry	Monochrome tokens, flat chrome, no ornament. Lightness does all the work.
Bullion	One hue, top to bottom.
Nocturne	Deeper canvas, dimmer chrome, quieter comments. For working late.
Vitrine	Maximum colour separation: vivid tokens, rainbow brackets and guides.
Clarity	Accessibility first: maximum text contrast, strong control edges, AAA enforced everywhere.
Patina	Aged copper instead of gold, with a cool syntax set to match.
Ember	Hot orange accent over a warm syntax set.
Sapphire	The classic blue editor, built on Auric surfaces.

A look is a starting point, not a lock. Everything on the Dials tab layers on top of it, so you can pick a look and keep adjusting. Selecting the same look again puts you back where that look began.

You can also run **Auric Theme: Choose Look…** from the Command Palette, or use the **Look** dropdown in the sidebar.

Tune dials on the Dials tab

1. Select the **Dials** tab (or press `2`).
2. Dials are grouped into ten cards: **Look**, **Accent**, **Surfaces**, **Text**, **Syntax**, **Editor**, **Workbench**, **Terminal**, **Effects** and **Accessibility**.
3. Change a control. The editor updates while you drag a slider and again when you release it.

Controls come in four kinds:

- **Swatches** for the accent colour: 17 named accents plus **Custom…**.
- **Dropdowns** for choices. Long lists can be searched by typing.
- **Sliders** for numbers. Use the arrow keys for small steps, `Shift` for bigger steps, `Page Up` and `Page Down`, `Home` and `End`, or double-click to reset.
- **Switches** for on and off.

Find a dial

- Type in **Search dials by name or setting…**. The search matches the dial label, its description and its setting ID, such as `editor.cursor`.
- Use the **Dial visibility** dropdown to switch between **All dials** and **Modified only**.
- A live count, such as "12 dials shown", sits beside the filters.

See and undo changes

- A dot and a reset button appear beside every dial you have changed. Select the reset button to return that dial to its default.
- Each card has a reset button in its header that resets all dials in that group.
- The footer shows the number of dials and overrides you have changed.

What each group controls

A summary follows. Every value and default is listed in the reference.

Accent

- **Accent colour** re-keys the whole theme onto one hue. The rotation happens in OKLCH, so each accent keeps the same visual weight. **Inherit** keeps the variant's own accent.
- **Custom accent** is any hex colour, used only when Accent colour is **Custom**. On light themes it is darkened automatically until it clears your minimum contrast.
- **Accent intensity** (0 to 200) scales every accent-tinted wash, border and selection.
- **Accent the chrome** fills the status bar, activity bar, both, or all three bars with the accent. Labels are recomputed so they stay legible.

Surfaces

- **Depth** (−4 to 4) sinks every surface towards black (or white on light themes) or lifts them.
- **Warmth** (−100 to 100) tints neutral surfaces towards amber or slate. Text and syntax are not affected.
- **Separation** sets how far apart the editor, sidebar, panel and bars sit tonally.
- **Borders** sets divider strength. Form-control edges are excluded: they always keep their 3:1 minimum.

Text

- **Text contrast** moves the whole foreground ramp at once.
- **Comment emphasis** makes comments quieter or louder.

Syntax

- **Syntax palette** reassigns token hues while keeping each role's lightness and chroma, so a palette swap cannot break the contrast audit. Nine palettes are available.
- **Saturation** (0 to 200): 0 gives greyscale tokens.
- **Italics** and **Bold** set how much of the code uses each style. If your font has no true italic, turn italics off.

Editor

Cursor, **Cursor colour**, **Selection strength**, **Current line**, **Bracket pairs**, **Indent guides**, **Whitespace marks** (only visible when VS Code's `editor.renderWhitespace` is on), **Inlay hints** and **Sticky scroll**.

Workbench

Active tab style, **Status bar**, **Activity bar**, **Title bar** (only visible with the custom title bar style), **Side bar** and **Dim unfocused**.

Terminal

ANSI palette (five palettes; all 14 text-bearing ANSI colours stay contrast-audited) and **Terminal background**.

Effects

Shadows and **Focus ring**. There is deliberately no "off" option for the focus ring.

Accessibility

- **Enforce contrast** (on by default) pushes every colour the customiser produces along its OKLCH lightness until it clears the minimum.
- **Minimum contrast** (3 to 7, default 4.5): 4.5 is WCAG AA for body text; 7 is AAA. It applies to text and syntax, not decorative washes.
- **Audit on apply** (on by default) runs the full contrast audit every time customisations are applied and warns you about failures.

Quick commands for common dials

Command	What it does
Auric Theme: Choose Accent Colour...	Pick from the 17 named accents, each shown with its hex value and a hint.
Auric Theme: Choose Look...	Apply one of the 11 looks.
Auric Theme: Choose Syntax Palette...	Pick one of the 9 syntax palettes.
Auric Theme: Toggle Italics	Switch italics between **off** and **standard**.
Auric Theme: Surprise Me	Clears your dials, then picks a random accent, syntax palette, warmth, separation, bracket style and tab style from curated choices.

Edit dials as settings

Dials are standard settings, so you can also set them in `settings.json`:

```
{
  "auricTheme.look": "signature",
  "auricTheme.accent.preset": "patina",
  "auricTheme.syntax.palette": "cool",
  "auricTheme.a11y.enforceContrast": true
}
```

The theme re-applies automatically when you save. In the Studio footer, **Settings** opens the Settings editor filtered to this extension.

Result

Your chosen look is active, your dial changes are visible in the editor, and each changed dial is marked with a dot so you can reset it individually.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Save, share and export

Keep your customisation as named presets, move it between machines as JSON, and export a standalone VS Code colour theme file.

Your customisation is small: the dials you changed plus any colour slot overrides. The extension gives you three ways to keep and move it.

Method	Where it lives	Best for
Preset	Inside the extension, on this machine	Switching between several set-ups quickly.
Customisation JSON	Clipboard or any text file	Moving a set-up to another machine or sharing it.
Theme file	A `.json` colour theme file	Using the result without this extension, or packaging it as your own theme.

Important: Edits apply live and there is no general undo history. Save a preset or export JSON before large experiments.

Presets

Save a preset

1. Open Theme Studio and select **Share** (or press `5`).
2. In the **Presets** card, type a name in **Name this preset…** (up to 64 characters).
3. Select **Save** or press `Enter`.

If you leave the name empty, the Studio shows "Give the preset a name first." Saving again with the same name updates that preset. Names that differ only by Hindi characters or punctuation are kept as separate presets.

You can also run **Auric Theme: Save Current as Preset…** from the Command Palette or the sidebar's **…** menu. It asks you to name the preset.

Load or delete a preset

- In the **Saved** list, select **Load** beside a preset. Each entry shows the date and time it was saved.
- Select the bin icon beside a preset to delete it.
- From the Command Palette, run **Auric Theme: Load Preset…**. Each preset shows the theme it was saved on. If there are none, you see "Auric Theme: no saved presets yet."

Loading a preset replaces your current dials and the colour overrides of the active variant.

Customisation JSON

The customisation is a small JSON document. It contains a format version, the theme it came from, your changed dials and your colour overrides. Default values are left out.

```
{
  "$schema": "https://auricartisan.com/schema/theme-customization-1.json",
  "version": 1,
  "generator": "auric-artisan-theme",
  "name": "Auric Artisan — Obsidian Gold",
  "theme": "obsidian-gold",
  "knobs": {
    "accent.preset": "sapphire",
    "syntax.palette": "cool"
  },
  "overrides": {
    "syntax.string": "#A0D0B0"
  }
}
```

Export from the Studio

1. On the **Share** tab, the **Export** card shows your current customisation as read-only JSON.
2. Select **Copy** to copy it, or **Refresh** to regenerate it after more changes.

Copy from the Command Palette

Run **Auric Theme: Copy Customisation to Clipboard**. VS Code confirms with "Auric Theme: customisation copied to the clipboard."

Import

1. On the **Share** tab, paste JSON into **Paste a customisation JSON here…** in the **Import** card.
2. Select **Import**. Select **Clear** to empty the box.

Or copy the JSON and run **Auric Theme: Apply Customisation from Clipboard**.

Importing replaces your dials and the overrides of the active variant. Export first if you want a way back. The import is checked first; you may see:

Message	Meaning
Nothing to import — paste some JSON first.	The Import box is empty.
Auric Theme: the clipboard is empty.	The clipboard command found no text.
That is not valid JSON (…).	The text could not be read as JSON.
This looks like it came from "…", not Auric Artisan Theme.	The JSON was produced by another tool.
This customisation needs a newer version of the extension (format v…).	Update the extension, then import again.
That customisation is empty — no dials and no colours.	There is nothing to apply.

Export a standalone theme file

1. Make sure an Auric theme is active. Otherwise you see "Auric Theme: switch to an Auric Artisan theme first."
2. Run **Auric Theme: Export as Theme File…** from the Command Palette, or choose it from the sidebar's **…** menu.
3. A save dialog opens with a suggested name such as `obsidian-gold-custom-color-theme.json`. Choose a location and select **Export theme**.
4. VS Code confirms, for example "Auric Theme: exported obsidian-gold-custom-color-theme.json." If some pairs fail your contrast minimum, the message includes the number of contrast warnings.

The file is a complete VS Code colour theme built from your current palette. You can use it in a theme extension of your own. It does not update when you change the Studio later; export again after changes.

Result

Your set-up is saved as a preset on this machine, available as portable JSON for other machines, and optionally exported as a standalone theme file.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

The Theme sidebar

Every part of the Auric Theme view in the Activity Bar, from the variant galleries to the contrast status and title-bar actions.

The extension adds an **Auric Theme** icon to the Activity Bar. It opens a compact view called **Theme** that covers the most common tasks without opening Theme Studio.

Open the sidebar

Select the **Auric Theme** icon in the Activity Bar. If you do not see it, right-click the Activity Bar and make sure **Auric Theme** is ticked.

Sections, top to bottom

Section	What it does
Header	Shows the active theme name and a summary such as "3 dials changed · 1 slot override", or **Shipped defaults**.
Language	**Auto**, **EN** and **हिं** buttons switch the language of the sidebar and Theme Studio.
Active theme card	Shows the current theme with an **Open Theme Studio** button.
Metrics	**Built-in variants** and **Your changes**.
Variant	A searchable dropdown of all variants, and **Browse all swatches** for the family galleries.
Accent	Swatches for the named accents, plus **Custom** with a colour picker for any hex colour.
Look	A dropdown of the 11 looks.
Contrast	The worst pair, a **Pass** or **Fail** chip and its ratio. Select it to open the Audit tab.
Actions	**Reset** and **Export**, and a link to auricartisan.com.

Variant galleries

Expand **Browse all swatches** to see one collapsible group per family: Obsidian, Midnight, Parchment, Porcelain, Dusk, Forest and High contrast. Each group shows its variant count and a tile per accent. Each tile has a small colour strip showing the canvas, surface, accent, strings and keywords. The active variant has a tick.

- Select a tile to switch theme.
- Use the arrow keys to move between tiles in a group.
- The sidebar remembers which groups you expanded.

Custom accent

Under **Accent**, open **Custom** and choose a colour in the picker. The theme switches the accent to **Custom** and uses your colour. Clearing the colour returns the accent to **Inherit**.

Reset

Select **Reset** once: the button changes to **Reset everything?**. Select it again within four seconds to clear every dial and colour override and return to the shipped theme. If you wait, the button returns to normal.

Export

The **Export** button prepares your customisation JSON. When Theme Studio is open, the JSON appears in the Share tab's Export box. To write a theme file, use **Export as Theme File…** in the title-bar **…** menu instead (see below).

Title-bar actions

The view's title bar has two icons and a **…** menu.

Location	Action
Icon	**Open Theme Studio**
Icon	**Show Contrast Report**
… menu	**Surprise Me**, **Save Current as Preset…**, **Load Preset…**
… menu	**Export as Theme File…**, **Copy Customisation to Clipboard**, **Apply Customisation from Clipboard**
… menu	**Reset Everything** (asks for confirmation in a dialog)

Result

You can switch variant, accent and look, check contrast and reach every sharing action from the sidebar, and open Theme Studio when you need the full controls.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Auric Artisan Theme reference

Every command, keybinding, setting, theme, look, accent, palette, colour slot and Theme Studio control in Auric Artisan Theme 0.2.0.

This reference describes version 0.2.0 as published on the Visual Studio Marketplace.

Commands

All commands appear in the Command Palette under the category **Auric Theme**.

Command	ID	What it does
Open Theme Studio	`auricTheme.openStudio`	Opens (or reveals) the Auric Theme Studio editor tab.
Switch Variant…	`auricTheme.selectTheme`	Quick Pick of all 54 themes with polarity and canvas colour; sets VS Code's colour theme.
Cycle Variant	`auricTheme.cycleVariant`	Switches to the next of the 54 variants.
Choose Accent Colour…	`auricTheme.pickAccent`	Quick Pick of the 17 named accents; sets `auricTheme.accent.preset`.
Choose Look…	`auricTheme.pickLook`	Quick Pick of the 11 looks; applies the look.
Choose Syntax Palette…	`auricTheme.pickPalette`	Quick Pick of the 9 syntax palettes; sets `auricTheme.syntax.palette`.
Choose Interface Language…	`auricTheme.chooseInterfaceLanguage`	Auto (VS Code), English or Hindi for the Studio and sidebar.
Toggle Italics	`auricTheme.toggleItalics`	Switches `auricTheme.syntax.italics` between `off` and `standard`.
Surprise Me	`auricTheme.randomize`	Clears your dials and applies a random curated accent, palette, warmth, separation, bracket and tab style.
Reset Everything	`auricTheme.resetAll`	After a confirmation dialog, clears every dial and the active variant's colour overrides.
Show Contrast Report	`auricTheme.showAudit`	Opens the read-only "Auric contrast report" text document.
Copy Customisation to Clipboard	`auricTheme.copyCustomization`	Copies the customisation JSON.
Apply Customisation from Clipboard	`auricTheme.pasteCustomization`	Imports customisation JSON from the clipboard.
Export as Theme File…	`auricTheme.exportThemeFile`	Saves a standalone colour theme JSON file. Needs an Auric theme active.
Save Current as Preset…	`auricTheme.savePreset`	Asks for a name and saves a preset.
Load Preset…	`auricTheme.loadPreset`	Quick Pick of saved presets; loads the chosen one.
Apply Now	`auricTheme.applyNow`	Re-applies the customisation immediately. Hidden from the Command Palette; you can bind it to a key.

Commands that change the theme only work while an Auric theme is active. Otherwise VS Code shows "Auric Theme customisations apply to the Auric Artisan themes. Pick one first." with a **Choose theme** button.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command
`Ctrl` + `Alt` + `;`	`Cmd` + `Alt` + `;`	Open Theme Studio
`Ctrl` + `Alt` + `'`	`Cmd` + `Alt` + `'`	Cycle Variant

Both work anywhere in VS Code. Change them in **Keyboard Shortcuts** (`Ctrl` + `K`, `Ctrl` + `S`).

Keys inside Theme Studio

Key	Action
`1` to `6`	Select Looks, Dials, Colours, Audit, Share or Preview.
Arrow keys, `Home`, `End`	Move between tabs when a tab has focus.
`/`	Go to the Colours tab and focus the colour filter.
`Escape`	Close an open dropdown or picker; in the colour filter, clear the filter.
`Enter`	In the preset name box, save the preset.

Number keys and `/` are ignored while you type in a field or while a dropdown or picker is open.

Keys in controls

Control	Keys
Dropdowns	Arrow keys, typeahead, `Enter` to choose, `Escape` or a click outside to close. Long lists have a search box.
Sliders	Arrow keys; `Shift` for larger steps; `Page Up`, `Page Down`, `Home`, `End`; double-click resets.
Colour picker plane	Arrow keys adjust saturation and brightness; `Shift` for larger steps.
Swatch grids	Arrow keys move between swatches.

Settings

All settings are application-wide (they apply to every window). Dials are written to your user settings.

General

Setting	Type	Values	Default	What it does
`auricTheme.interfaceLanguage`	string	`auto`, `en`, `hi`	`auto`	Language of Theme Studio and the sidebar.

Theme dials

Setting	Type	Values or range	Default	What it does
`auricTheme.look`	string	`signature`, `gallery`, `atelier`, `foundry`, `bullion`, `nocturne`, `vitrine`, `clarity`, `patina`, `ember`, `sapphire`	`signature`	Named starting point; other dials layer on top.
`auricTheme.accent.preset`	string	`inherit`, 17 accents (see below), `custom`	`inherit`	Accent colour the theme is re-keyed onto.
`auricTheme.accent.custom`	string	Hex colour	empty	Used only when the accent is `custom`.
`auricTheme.accent.intensity`	number	0 to 200, step 5	100	Scales accent-tinted washes, borders and selections.
`auricTheme.accent.applyToChrome`	string	`off`, `statusBar`, `activityBar`, `both`, `full`	`off`	Fills workbench bars with the accent. `full` adds the title bar.
`auricTheme.surface.depth`	number	−4 to 4, step 1	0	Sinks or lifts every surface.

Setting	Type	Values or range	Default	What it does
`auricTheme.surface.warmth`	number	−100 to 100, step 5	0	Tints neutral surfaces towards amber or slate.
`auricTheme.surface.separation`	string	`flat`, `subtle`, `standard`, `layered`	`standard`	Tonal distance between editor, sidebar, panel and bars.
`auricTheme.surface.borders`	string	`none`, `hairline`, `visible`, `strong`, `gold`	`hairline`	Divider strength; `gold` uses accent-tinted dividers.
`auricTheme.text.contrast`	string	`soft`, `standard`, `high`, `maximum`	`standard`	Moves the whole foreground ramp.
`auricTheme.text.comments`	string	`whisper`, `muted`, `standard`, `prominent`	`standard`	Comment emphasis.
`auricTheme.syntax.palette`	string	`jewel`, `vivid`, `pastel`, `mono`, `bullion`, `classic`, `warm`, `cool`, `duotone`	`jewel`	Token hue assignment.
`auricTheme.syntax.saturation`	number	0 to 200, step 5	100	Token saturation; 0 is greyscale.
`auricTheme.syntax.italics`	string	`off`, `comments`, `standard`, `expressive`	`standard`	How much code is italic.
`auricTheme.syntax.bold`	string	`off`, `declarations`, `standard`, `expressive`	`standard`	How much code is bold.
`auricTheme.editor.cursor`	string	`accent`, `contrast`, `syntax`, `custom`	`accent`	Caret, multi-cursor and terminal cursor colour.
`auricTheme.editor.cursorColor`	string	Hex colour	empty	Used only when Cursor is `custom`.
`auricTheme.editor.selection`	number	20 to 200, step 5	100	Scales selection, occurrence, word and find highlights together.
`auricTheme.editor.currentLine`	string	`off`, `subtle`, `standard`, `bordered`, `accent`	`standard`	How the caret's line is marked.
`auricTheme.editor.bracketPairs`	string	`off`, `subtle`, `accent`, `rainbow`, `vivid`	`rainbow`	Six bracket colour levels and their guides.
`auricTheme.editor.indentGuides`	string	`off`, `subtle`, `standard`, `rainbow`	`rainbow`	Indent guide colours.
`auricTheme.editor.whitespace`	string	`faint`, `subtle`, `visible`	`subtle`	Whitespace marks (when `editor.renderWhitespace` is on).
`auricTheme.editor.inlayHints`	string	`ghost`, `subtle`, `chip`	`chip`	Inlay hint backdrop.
`auricTheme.editor.stickyScroll`	string	`flush`, `raised`, `bordered`	`raised`	Sticky headers in editor, panel, sidebar and terminal.
`auricTheme.chrome.tabStyle`	string	`underline`, `overline`, `block`, `accent-block`, `minimal`	`overline`	How the active editor tab is marked.
`auricTheme.chrome.statusBar`	string	`chrome`, `deep`, `accent`, `transparent`	`chrome`	Status bar fill.
`auricTheme.chrome.activityBar`	string	`deep`, `chrome`, `accent`, `transparent`	`deep`	Activity bar fill.
`auricTheme.chrome.titleBar`	string	`deep`, `chrome`, `accent`	`deep`	Title bar fill (custom title bar style only).
`auricTheme.chrome.sidebar`	string	`raised`, `flush`, `deep`	`raised`	Side bar fill.
`auricTheme.chrome.dimInactive`	boolean	`true`, `false`	`false`	Pushes unfocused tabs, editors and selections back.

Setting	Type	Values or range	Default	What it does
`auricTheme.terminal.palette`	string	`auric`, `classic`, `vivid`, `muted`, `monochrome`	`auric`	ANSI terminal palette.
`auricTheme.terminal.background`	string	`editor`, `panel`, `deep`	`editor`	Terminal backdrop.
`auricTheme.effects.shadows`	string	`off`, `subtle`, `standard`, `deep`	`standard`	Shadows under widgets and overlays.
`auricTheme.effects.focusRing`	string	`subtle`, `standard`, `strong`	`standard`	Keyboard focus ring strength.
`auricTheme.a11y.enforceContrast`	boolean	`true`, `false`	`true`	Contrast guard on or off.
`auricTheme.a11y.minContrast`	number	3 to 7, step 0.5	4.5	Minimum ratio for text and syntax.
`auricTheme.a11y.auditOnApply`	boolean	`true`, `false`	`true`	Audit and warn on every apply.

Option meanings

Setting	Option	Meaning
Separation	`flat` / `subtle` / `standard` / `layered`	One value for all panes / barely-there steps / shipped hierarchy / exaggerated steps.
Borders	`none` / `hairline` / `visible` / `strong` / `gold`	No dividers / shipped 8% divider / clearly drawn / hard edges / accent-tinted.
Text contrast	`soft` / `standard` / `high` / `maximum`	Lower (still AA) / shipped / pushes body and muted apart / every foreground to the edge.
Comment emphasis	`whisper` / `muted` / `standard` / `prominent`	As quiet as the floor permits / quieter / shipped / first-class text.
Italics	`off` / `comments` / `standard` / `expressive`	None / comments and doc comments / plus parameters, interfaces, decorators, attributes and `this` / plus keywords, storage modifiers and types.
Bold	`off` / `declarations` / `standard` / `expressive`	None / definition sites only / declarations, headings and invalid tokens / plus keywords and types.
Cursor	`accent` / `contrast` / `syntax` / `custom`	Accent colour / white or black, whichever reads harder / the function-name colour / Cursor colour.
Current line	`off` / `subtle` / `standard` / `bordered` / `accent`	None / faint fill / shipped fill and edge / edge only / accent-tinted fill.
Bracket pairs	`off` / `subtle` / `accent` / `rainbow` / `vivid`	Punctuation colour / low chroma / stepped along the accent / shipped six colours / maximum separation.
Indent guides	`off` / `subtle` / `standard` / `rainbow`	None / one neutral / neutral with accent active guide / shipped six levels.
Whitespace marks	`faint` / `subtle` / `visible`	Barely visible / shipped / clearly readable.
Inlay hints	`ghost` / `subtle` / `chip`	Text only / faint backdrop / shipped tinted chip.
Sticky scroll	`flush` / `raised` / `bordered`	Canvas colour / slightly lifted / lifted with a hard bottom edge.
Active tab	`underline` / `overline` / `block` / `accent-block` / `minimal`	Accent rule at bottom / at top (shipped) / tone only / accent fill / foreground only.
Status bar	`chrome` / `deep` / `accent` / `transparent`	Shipped neutral / matches activity bar / accent fill / canvas colour.

Setting	Option	Meaning
Activity bar	`deep` / `chrome` / `accent` / `transparent`	Shipped darkest / matches status bar / accent fill / merges into sidebar.
Title bar	`deep` / `chrome` / `accent`	Shipped darkest / matches status bar / accent fill.
Side bar	`raised` / `flush` / `deep`	Shipped, lifted / same as editor / matches activity bar.
Terminal background	`editor` / `panel` / `deep`	Shipped / merges into panel / darker than editor.
Shadows	`off` / `subtle` / `standard` / `deep`	None / barely there / shipped / heavy.
Focus ring	`subtle` / `standard` / `strong`	Quieter but visible / accent at full strength / maximum contrast.

Looks

`signature` Signature, `gallery` Gallery, `atelier` Atelier, `foundry` Foundry, `bullion` Bullion, `nocturne` Nocturne, `vitrine` Vitrine, `clarity` Clarity, `patina` Patina, `ember` Ember, `sapphire` Sapphire. See Looks and dials for descriptions.

Accents

ID	Name	Hex	Hint
`gold`	Auric Gold	`#D3AF37`	The brand. Warm, metallic, the default.
`champagne`	Champagne	`#E4CE8F`	Gold, lifted and softened.
`brass`	Brass	`#C9A227`	Deeper and greener than the brand gold.
`bronze`	Bronze	`#C08A4A`	Warm metal, less yellow.
`copper`	Copper	`#CA7A4E`	Orange-forward metal.
`ember`	Ember	`#E0764A`	Hot orange.
`rose`	Rose	`#E08AA0`	Soft pink-red.
`crimson`	Crimson	`#E06A6A`	Red without the alarm.
`amethyst`	Amethyst	`#B48AE6`	Violet jewel tone.
`iris`	Iris	`#8E9BF0`	Blue-violet.
`sapphire`	Sapphire	`#6BA3E8`	Classic editor blue.
`azure`	Azure	`#4FB6E0`	Bright cyan-blue.
`patina`	Patina	`#5CC9B4`	Aged copper. Cool and calm.
`emerald`	Emerald	`#5CC489`	Green jewel tone.
`moss`	Moss	`#A3C46A`	Yellow-green.
`platinum`	Platinum	`#C5C8CE`	Cool neutral metal.
`graphite`	Graphite	`#9A9A9A`	No hue at all. Fully monochrome UI.

The listed hex values are the accent keys. On light themes the accent is darkened automatically until it clears your minimum contrast.

Syntax palettes

ID	Name	Description
`jewel`	Jewel	The shipped palette: gold and champagne callables, amber constants, orchid keywords, patina types, sage strings.
`vivid`	Vivid	The same hues pushed to the edge of the gamut.
`pastel`	Pastel	Low chroma, lifted lightness.
`mono`	Monochrome	Almost no hue; lightness carries meaning.
`bullion`	Bullion	Every token on the accent hue, separated by lightness and chroma.
`classic`	Classic	Blue keywords, green strings, teal types.
`warm`	Warm	Gold, amber, copper and rose; no cool tokens.
`cool`	Cool	Patina, sapphire and amethyst; gold only in the chrome.
`duotone`	Duotone	The accent and its complement only.

Terminal palettes

ID	Name	Description
`auric`	Auric	Matches the syntax palette; yellow is the accent.
`classic`	Classic	Conventional ANSI hues, tuned for the canvas.
`vivid`	Vivid	Higher chroma across all sixteen.
`muted`	Muted	Lower chroma.
`monochrome`	Monochrome	Greyscale, separated by lightness only.

All 54 themes

The theme ID is what VS Code stores in `workbench.colorTheme`.

Family	Theme IDs (display name: Auric Artisan — Family Accent)
Obsidian	`obsidian-gold`, `obsidian-sapphire`, `obsidian-rose`, `obsidian-amethyst`, `obsidian-patina`
Obsidian (cont.)	`obsidian-emerald`, `obsidian-ember`, `obsidian-crimson`, `obsidian-iris`, `obsidian-graphite`
Midnight	`midnight-gold`, `midnight-sapphire`, `midnight-rose`, `midnight-iris`, `midnight-amethyst`
Midnight (cont.)	`midnight-patina`, `midnight-emerald`, `midnight-ember`, `midnight-crimson`, `midnight-graphite`
Parchment	`parchment-gold`, `parchment-rose`, `parchment-ember`, `parchment-emerald`, `parchment-sapphire`
Parchment (cont.)	`parchment-amethyst`, `parchment-patina`, `parchment-crimson`, `parchment-iris`, `parchment-graphite`
Porcelain	`porcelain-sapphire`, `porcelain-amethyst`, `porcelain-patina`, `porcelain-rose`, `porcelain-iris`
Porcelain (cont.)	`porcelain-graphite`, `porcelain-gold`, `porcelain-emerald`, `porcelain-ember`, `porcelain-crimson`
Dusk	`dusk-gold`, `dusk-sapphire`, `dusk-rose`, `dusk-amethyst`, `dusk-iris`, `dusk-graphite`
Forest	`forest-gold`, `forest-patina`, `forest-emerald`, `forest-ember`, `forest-rose`, `forest-graphite`

Family	Theme IDs (display name: Auric Artisan — Family Accent)
High contrast	`obsidian-gold-high-contrast` (dark), `porcelain-gold-high-contrast` (light)

The order in this table is the order used by **Cycle Variant**. Obsidian, Midnight, Dusk and Forest are dark; Parchment and Porcelain are light.

Colour slots

107 slots in 12 groups. Kind: **solid** (no transparency), **alpha** (transparency allowed) or **either**.

Group	Slots (path — label)
Surfaces (solid)	`bg.deepest` Deepest, `bg.base` Canvas, `bg.raised` Raised, `bg.chrome` Chrome
Surfaces (cont.)	`bg.overlay` Overlay, `bg.overlayRaised` Overlay raised, `bg.inset` Inset, `bg.tint` Warm tint
Text (solid)	`fg.default` Body, `fg.muted` Muted, `fg.subtle` Subtle, `fg.faint` Faint, `fg.onAccent` On accent
Text (cont.)	`fg.inverse` Inverse
Lines	`line.subtle` Hairline, `line.default` Divider, `line.strong` Strong, `line.control` Control edge (either)
Lines (cont.)	`line.gold` Gold hairline, `line.goldStrong` Gold edge (alpha); `line.focus` Focus ring (solid)
Accent	`accent.base` Accent, `accent.bright` Emphasis, `accent.deep` Deep, `accent.onAccent` On accent (solid)
Accent (cont.)	`accent.soft` Soft wash, `accent.softer` Softer wash, `accent.wash` Faintest wash (alpha)
States (alpha)	`state.hover` Hover, `state.active` Active, `state.selected` Selected
States (cont.)	`state.selectedInactive` Selected (blurred), `state.drop` Drop target
Selection (alpha)	`sel.text` Selection, `sel.textInactive` Selection (blurred), `sel.highlight` Occurrences
Selection (cont.)	`sel.word` Word (read), `sel.wordWrite` Word (write), `sel.find` Find match, `sel.findOther` Find others
Selection (cont.)	`sel.range` Range, `sel.line` Current line, `sel.lineBorder` Current line edge
Semantics (solid)	`sem.error` Error, `sem.errorStrong` Error (loud), `sem.warn` Warning, `sem.warnStrong` Warning (loud)
Semantics (cont.)	`sem.info` Info, `sem.infoStrong` Info (loud), `sem.ok` Success, `sem.okStrong` Success (loud)
Semantic washes (alpha)	`soft.error` Error wash, `soft.warn` Warning wash, `soft.info` Info wash, `soft.ok` Success wash
Terminal (solid)	`ansi.black`, `ansi.red`, `ansi.green`, `ansi.yellow`, `ansi.blue`, `ansi.magenta`, `ansi.cyan`, `ansi.white`
Terminal (cont.)	`ansi.brightBlack` to `ansi.brightWhite`: the eight bright versions of the colours above
Source control (solid)	`git.added` Added, `git.modified` Modified, `git.deleted` Deleted, `git.untracked` Untracked
Source control (cont.)	`git.ignored` Ignored, `git.conflict` Conflicting, `git.submodule` Submodule
Source control (cont.)	`git.stageDeleted` Staged delete, `git.stageModified` Staged modify
Syntax (solid)	`syntax.comment` Comment, `syntax.punctuation` Punctuation, `syntax.operator` Operator
Syntax (cont.)	`syntax.variable` Variable, `syntax.property` Property, `syntax.parameter` Parameter
Syntax (cont.)	`syntax.string` String, `syntax.stringAlt` String (alt), `syntax.escape` Escape, `syntax.number` Number
Syntax (cont.)	`syntax.constant` Constant, `syntax.keyword` Keyword, `syntax.control` Control flow, `syntax.storage` Storage
Syntax (cont.)	`syntax.fn` Function, `syntax.method` Method, `syntax.type` Type, `syntax.interface` Interface
Syntax (cont.)	`syntax.tag` Tag, `syntax.attribute` Attribute, `syntax.regexp` Regexp, `syntax.invalid` Invalid

Group	Slots (path — label)
Syntax (cont.)	`syntax.deprecated` Deprecated, `syntax.heading` Heading, `syntax.link` Link, `syntax.quote` Quote
Effects (alpha)	`shadow` Shadow

Lines `line.subtle`, `line.default`, `line.strong` and `line.control` accept either kind.

Theme Studio controls by tab

Tab	Controls
Looks	**The set**: 11 look swatches with a count. Selecting a look applies it; selecting the active look again restarts from it.
Dials	**Search dials by name or setting…**, **Dial visibility** (**All dials**, **Modified only**), match count, ten group cards with group reset, per-dial modified dot and reset.
Colours	**Filter by name or path**, 12 collapsible groups with counts, a colour field per slot, **Shipped** comparison and revert per overridden slot.
Audit	Worst-pair card with **All clear** or "{n} failing", **Measured pairs** table (Pair, Ratio, APCA, Minimum, Result), **How to read this**.
Share	**Export** (JSON, **Copy**, **Refresh**), **Import** (paste box, **Import**, **Clear**), **Presets** (name box, **Save**, **Saved** list with **Load** and delete).
Preview	**Comparison view** (**Side by side**, **As shipped**, **Your palette**), **Copy palette as CSS**, specimens and changed-slot count.

Studio-wide: language switch, **Explore colours** →, metrics, **Reset all** (asks **Reset everything?**, confirm within four seconds), **Randomize**, **Settings**.

Other surfaces

Surface	Details
Status bar item	Right side. Paint-can icon and short theme name; `*` when customised. Tooltip shows dials and colours changed. Opens Theme Studio. Hidden when a non-Auric theme is active.
Activity Bar view	**Auric Theme** container with the **Theme** view. See The Theme sidebar.
Contrast report	Read-only text document "Auric contrast report".
Walkthrough	**Make it yours**: **Pick a variant**, **Change the gold**, **Open the Studio**, **Keep it readable**.

Messages you may see

Message	Meaning
Auric Theme customisations apply to the Auric Artisan themes. Pick one first.	A non-Auric theme is active. Select **Choose theme**.
Auric Theme: {n} colour pairs no longer meet your contrast minimum.	Audit on apply found failures. Select **Show report**.
Reset every Auric Theme dial and colour override?	Confirmation for **Reset Everything**. Select **Reset**.
Auric Theme: could not save auricTheme.... — ...	VS Code refused to write a setting, for example because `settings.json` has a syntax error.

Message	Meaning
VS Code rejected a settings key: …	VS Code refused a colour customisation key.
Bad hex colour: …	A colour value is not valid hex.
Auric Theme: no saved presets yet.	**Load Preset…** found no presets.
Auric Theme: switch to an Auric Artisan theme first.	**Export as Theme File…** needs an Auric theme active.

Files and formats

Item	Format
Customisation	JSON with `$schema`, `version`, `generator`, `name`, `theme`, `knobs` and `overrides`.
Theme file	VS Code colour theme JSON; default name `<theme-id>-custom-color-theme.json`.
CSS palette	A `:root` block of `--auric-<slot-path>` custom properties.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Artisan Theme and Theme Studio.

Symptom	Why it happens	What to do
Dials and colour changes have no effect.	A non-Auric colour theme is active. Customisations are scoped to the Auric theme names.	Run **Auric Theme: Switch Variant…** and pick an Auric theme.
A message says "customisations apply to the Auric Artisan themes. Pick one first."	Same as above.	Select **Choose theme** in the message.
The status bar item is missing.	It only shows while an Auric theme is active.	Switch to an Auric theme.
My colour override is not exactly the colour I typed.	The contrast guard adjusted it to meet your minimum contrast.	Lower **Minimum contrast** or turn off **Enforce contrast** in the Accessibility dials. Check the Audit tab afterwards.
Overrides disappeared after switching theme.	Colour overrides are stored per variant.	Switch back to the variant where you made them, or save a preset and load it on the new variant.
A warning keeps saying colour pairs no longer meet the minimum.	**Audit on apply** reports failures after each change.	Fix the failing pairs (see the Audit tab), or turn off **Audit on apply**.
"Auric Theme: could not save auricTheme…."	VS Code could not write your user settings, often because `settings.json` has a syntax error or is read-only.	Open **Preferences: Open User Settings (JSON)**, fix any error, then try again.
Import fails with "That is not valid JSON".	The pasted text is incomplete or not JSON.	Copy the whole block again, including the braces.
Import says the customisation "came from" another tool.	The JSON was not produced by Auric Artisan Theme.	Only import files exported by this extension.
Import says a newer version is needed.	The JSON uses a newer format.	Update the extension.

Symptom	Why it happens	What to do
The sidebar's **Export** button seems to do nothing.	It prepares customisation JSON for the Studio's Share tab; it does not open a save dialog.	Use **Export as Theme File…** from the sidebar's **…** menu, or **Copy Customisation to Clipboard**.
Italics look slanted and blurry.	Your editor font has no true italic, so it is synthesised.	Set **Italics** to **Off**, or run **Auric Theme: Toggle Italics**.
Whitespace marks setting has no visible effect.	Marks only show when VS Code draws whitespace.	Turn on `editor.renderWhitespace`.
Title bar dial has no visible effect.	The title bar colour only applies with VS Code's custom title bar style.	Set `window.titleBarStyle` to `custom`.
A preset I saved in an older version is missing.	In versions before 0.2.0, presets with distinct Hindi or punctuation-only names could overwrite each other.	This was fixed in 0.2.0, but presets already overwritten cannot be recovered.
Recent colours in the picker are gone.	Recent colours last only for the current panel session.	Note important colours as presets or overrides.
The Preview tab does not look like my editor.	Preview is an illustrative specimen of the palette, not a full editor.	Look at a real file to judge the result; edits are already live.
The Studio shows a banner listing problems.	The theme engine reported something it could not apply.	Read the listed problem; resetting the affected dial or slot usually clears it.
Hindi labels do not appear in the Command Palette.	Command titles follow VS Code's display language, not the Studio switch.	Run **Configure Display Language** and choose Hindi, or keep the Studio switch for the panels only.

Start again from the shipped theme

1. Run **Auric Theme: Reset Everything** and confirm with **Reset**, or
2. In Theme Studio, select **Reset all**, then **Reset everything?** within four seconds.

This clears every dial and the active variant's colour overrides. Presets are not deleted.

Still stuck

Collect your VS Code version, the extension version (0.2.0), the active theme and the steps that reproduce the problem, then contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Frequently asked questions

Short answers to common questions about Auric Artisan Theme and Theme Studio.

Is the theme free?

Yes. The extension is free to install and use.

How many themes are there?

54: ten accents each for Obsidian, Midnight, Parchment and Porcelain, six each for Dusk and Forest, and two high-contrast builds. See Choose a theme.

Do I need Theme Studio to use the themes?

No. The themes work like any other VS Code colour theme. Theme Studio is optional.

Does Theme Studio have a preview mode?

No. Every edit applies live. The Preview tab only compares palettes. Save a preset or export JSON before large experiments.

Is there undo?

There is no undo history. Use the per-dial and per-slot reset buttons, **Reset all**, or load a saved preset.

Where are my changes stored?

Dials are VS Code user settings under `auricTheme.*`. Colour overrides and presets are stored by the extension on this machine. The extension also writes theme-scoped entries into VS Code's colour customisation settings so the changes show. See Privacy.

Will my customisation sync to other machines?

Dials are ordinary settings, so they follow VS Code Settings Sync if you use it. Presets and colour overrides are local; move them with customisation JSON (see Save, share and export).

Why did my accent colour get darker on a light theme?

On light themes the accent is darkened until it clears your minimum contrast. The brand gold, for example, is too light to read as text on the cream Parchment canvas.

Does the theme affect other colour themes?

No. Customisations are scoped to the Auric theme names and do nothing when another theme is active. Unrelated theme blocks in your settings are left alone.

What does "Surprise Me" change?

It clears your dials, then picks a random accent, syntax palette, surface warmth, separation, bracket style and tab style from curated choices. Your colour overrides are kept. Reset if you do not like the result.

Can I use my customised theme without the extension?

Yes. Run **Export as Theme File…** to save a standalone VS Code colour theme JSON file.

Can the interface be in Hindi?

Yes. Use the **Auto · English · हिन्दी** switch in Theme Studio or the sidebar. See Interface language.

Does it work in remote, untrusted or virtual workspaces?

Yes. The extension declares support for untrusted and virtual workspaces.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Auric Artisan Theme: other information

Contents of the supporting pages for Auric Artisan Theme: glossary, shortcuts, limits, related products, release notes and privacy.

This folder holds supporting pages for the Auric Artisan Theme extension. They explain terms, list shortcuts, describe what the contrast checks can and cannot tell you, and record the release history and privacy behaviour.

File	What it covers
glossary.md	Terms used in the theme and Theme Studio.
shortcuts.md	Keybindings, Studio keys and every command.
limits-and-accuracy.md	What the contrast audit measures and what the extension does not do.
related.md	Other Auric Artisan extensions and website tools that pair with the theme.
release-notes.md	User-facing history of each version.
privacy.md	What is stored, where, and what uses the network.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Shortcuts and commands

Every keybinding, Theme Studio key and Command Palette command in Auric Artisan Theme.

Global keybindings

Action	Windows and Linux	macOS
Open Theme Studio	`Ctrl` + `Alt` + `;`	`Cmd` + `Alt` + `;`
Cycle Variant	`Ctrl` + `Alt` + `'`	`Cmd` + `Alt` + `'`

To change them, open **Preferences: Open Keyboard Shortcuts** and search for "Auric Theme".

Keys inside Theme Studio

Key	Action
`1`	Looks tab
`2`	Dials tab
`3`	Colours tab
`4`	Audit tab
`5`	Share tab
`6`	Preview tab
`/`	Colours tab, focus the filter
`Escape`	Close a dropdown or picker; clear the colour filter
Arrow keys, `Home`, `End`	Move between tabs, dropdown options, swatches; adjust sliders

Key	Action
`Shift` + arrow keys	Larger slider and picker steps
`Page Up`, `Page Down`	Larger slider steps
Double-click a slider	Reset it
`Enter` in the preset name box	Save the preset

Commands

Open the Command Palette (`Ctrl` + `Shift` + `P`) and type "Auric Theme".

Command	Notes
Auric Theme: Open Theme Studio	Also on the status bar item and the sidebar.
Auric Theme: Switch Variant…	All 54 themes.
Auric Theme: Cycle Variant	Next variant.
Auric Theme: Choose Accent Colour…	17 named accents.
Auric Theme: Choose Look…	11 looks.
Auric Theme: Choose Syntax Palette…	9 palettes.
Auric Theme: Choose Interface Language…	Auto, English or Hindi.
Auric Theme: Toggle Italics	Off or standard.
Auric Theme: Surprise Me	Random curated combination.
Auric Theme: Reset Everything	Asks for confirmation.
Auric Theme: Show Contrast Report	Opens the text report.
Auric Theme: Copy Customisation to Clipboard	JSON.
Auric Theme: Apply Customisation from Clipboard	JSON import.
Auric Theme: Export as Theme File…	Standalone theme JSON.
Auric Theme: Save Current as Preset…	Named preset.
Auric Theme: Load Preset…	Choose a saved preset.

Apply Now (`auricTheme.applyNow`) exists but is hidden from the Command Palette. Bind it to a key if you want to force an immediate re-apply.

With VS Code's display language set to Hindi, these titles appear in Hindi under the category "ऑरिक थीम".

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Glossary

Definitions of the terms used in Auric Artisan Theme and Theme Studio.

Term	Meaning
Accent	The single colour the whole theme is keyed to. Washes, borders, selections, the focus ring and the cursor are derived from it.

Term	Meaning
Alpha	Transparency. Alpha slots accept an opacity value, written as the last two digits of `#rrggbbaa`.
APCA Lc	A perceptual contrast value from the draft WCAG 3 work. It accounts for polarity; a negative value means light text on a dark background.
Audit	The contrast check of 81 foreground and background pairs in the active theme.
Canvas	The editor background, slot `bg.base`.
Chrome	The workbench bars and frames around the editor: status bar, activity bar, title bar, side bar and tabs.
Colour slot	One named colour the theme is built from, such as `syntax.keyword`. There are 107.
Contrast guard	The step that nudges colours along OKLCH lightness until they meet your minimum contrast. Controlled by **Enforce contrast**.
Contrast ratio	The WCAG measure of luminance difference, from 1:1 to 21:1.
Customisation JSON	The portable document holding your changed dials and colour overrides.
Dial	One theme setting, such as **Syntax palette** or **Depth**. There are 37, all under `auricTheme.*`.
Family	A set of variants sharing the same neutral surfaces, such as Obsidian or Porcelain.
High contrast	The two builds designed for VS Code's high-contrast modes, one dark and one light.
Look	A named combination of dial values, such as **Clarity**, that you apply in one click.
OKLCH	A colour model with Lightness, Chroma (colourfulness) and Hue that matches human perception closely, so changing hue keeps colours looking equally strong.
Override	A colour you set for a slot by hand. It wins over the dials and is stored per variant.
Polarity	Whether a theme is dark (light text on dark) or light (dark text on light).
Preset	A named, saved copy of your dials and overrides, kept on this machine.
Semantic highlighting	Colouring based on what a language server knows about a symbol, in addition to TextMate grammar colouring.
Shipped	The colour a slot has in the variant as published, before your changes.
TextMate token	A piece of code classified by a grammar, such as a string or keyword, and coloured by the theme.
Theme Studio	The editor-tab interface for customising the theme, with six tabs.
Variant	One of the 54 complete themes: a family plus an accent.
WCAG AA / AAA	Web Content Accessibility Guidelines levels. AA needs 4.5:1 for body text (3:1 for large text and control edges); AAA needs 7:1.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Limits and accuracy

What the contrast audit measures, how precise it is, and what Auric Artisan Theme does not do.

What the contrast audit measures

- The audit covers 81 defined foreground and background pairs per theme: editor text, 26 syntax roles, 14 text-bearing ANSI colours, source control decorations, and text and borders of common controls.

- Ratios use the WCAG 2 relative-luminance formula on sRGB colours. Transparent colours are composited onto the surface they sit on before measuring.
- APCA Lc values are shown for guidance. APCA is part of draft WCAG 3 work and is not yet a conformance requirement.
- Each pair has its own minimum. Text is held to your **Minimum contrast** setting; control borders are held to 3:1; decorative washes are held to less than text.

What the audit does not tell you

- It does not check every colour VS Code can draw, or colours added by other extensions.
- It does not check your font, font size, font weight or display.
- It does not replace testing with real users. Contrast checks are guidance, not a guarantee that every possible customisation meets accessibility requirements.
- A pair marked **Missing** could not be measured because a colour it depends on is missing.

How the contrast guard changes colours

The guard moves a colour along its OKLCH lightness until the pair clears the minimum, keeping hue and chroma. Very saturated or unusual colours may shift visibly. The guard applies to text and syntax colours, not decorative washes.

What the extension does not do

- Theme edits apply live. There is no dry-run or "apply later" mode, and the Preview tab is an illustrative specimen, not a full editor.
- There is no undo history, cloud preset gallery or multi-device preset service.
- It does not extract palettes from images.
- Recent colours in the picker last only for the current panel session.
- Settings change colours only. They cannot change VS Code's layout.
- Customisations apply only while an Auric Artisan theme is active.
- Colour overrides are per variant and are not shared between variants automatically.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Privacy

What Auric Artisan Theme stores, where it stores it and what, if anything, uses the network.

Auric Artisan Theme works entirely inside VS Code. It has no account, no telemetry and no runtime package dependencies, and it does not contact a server.

What is stored and where

Data	Where it is stored
Dial values and interface language	Your VS Code user settings, under `auricTheme.*`.
Theme-scoped colour customisations	Your VS Code user settings, in the colour, token and semantic-token customisation blocks for the Auric theme names. Unrelated blocks are left alone. An untouched set-up writes nothing.
Colour slot overrides (per variant) and presets	The extension's local storage in VS Code on this machine.

Data	Where it is stored
Studio view state (tab, scroll, open groups, searches)	The panel's own saved state in VS Code.
Recent colours in the picker	Memory only, for the current panel session.

"Local" does not mean read-only: customising the theme writes to your user settings.

What uses the network

Nothing, automatically. The links to auricartisan.com in the Studio and sidebar open your browser only when you select them.

Clipboard and files

- The clipboard is read only when you run **Apply Customisation from Clipboard**, and written only when you copy.
- Files are written only when you run **Export as Theme File…** and choose a location.

Security

The Studio and sidebar load only files packaged with the extension and use a restrictive content security policy. Imported JSON is validated before it is applied and is never executed.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Release notes

User-facing history of Auric Artisan Theme, from 0.1.0 to 0.2.0.

0.2.0 — 8 September 2026

The current Marketplace version.

Added

- 54 built-in variants, up from 24. All existing theme IDs still work.
- Two new dark families, **Dusk** and **Forest**, with six accents each.
- Ten accent variants each for Obsidian, Midnight, Parchment and Porcelain.
- Compact collapsible family galleries in the sidebar, with small accent tiles, counts and keyboard navigation; the sidebar remembers which families are open.
- A searchable variant dropdown.
- The **Preview** tab: side-by-side palette comparison and **Copy palette as CSS**.
- Dial search and a **Modified only** filter.
- A direct custom accent picker in the sidebar.
- Numeric RGB and opacity channels in the colour picker, and up to eight recent colours per session.

Improved

- Theme Studio and the sidebar share a new workspace layout: introduction, active theme card, live counts and numbered tabs.
- Matching Hindi theme names in VS Code's pickers and in both panels.
- Devanagari-friendly fonts and line height.

Fixed

- Dropdowns no longer jump to the top or scroll the page behind them while you scroll; background updates keep the list position.
- Presets with distinct Hindi or punctuation-only names no longer overwrite each other. Presets already overwritten in earlier versions cannot be recovered.
- Popups no longer exceed the available height in very short windows.
- The sidebar **Reset** button now needs a second click to confirm.

0.1.1 — 10 August 2026

- **Auto**, **English** and **हिन्दी** interface selection for Theme Studio and the sidebar, saved in settings.
- Localised commands, settings, notifications and walkthroughs.
- Theme Studio keeps its state when you change language.

0.1.0 — 23 July 2026

- Initial collection of 24 themes, including dark and light high-contrast builds.
- Theme Studio and sidebar with dials, colour slot overrides, contrast audits and presets.
- Portable customisation JSON and standalone theme file export.

VS CODE EXTENSIONS › AURIC ARTISAN THEME

Related extensions and tools

Other Auric Artisan VS Code extensions, website tools and guides that work well with Auric Artisan Theme.

Other Auric Artisan VS Code extensions

Extension	How it relates
Auric Artisan File Icons	File and folder icons for the Explorer, in Facet, Playful and Modern styles. Pairs with the theme for a complete look.
Auric Artisan Icon Studio	Search and insert icons and emoji into code.
Auric Artisan Emoji	Emoji Studio with the same English and Hindi interface switch.
Auric Artisan Studio	Colour, accessibility and code-health tools for your project files.
Auric Artisan Live	Live server and preview; Theme Studio's layout follows the same workspace design.

For installing and managing any of these, see Install and set up.

Website tools

Tool	Why it is useful with the theme
Contrast Checker	Check a single colour pair in detail, including WCAG and APCA values.
Design Engine	Build a design system palette in OKLCH that you can mirror in your editor.
Colour Science Lab	Explore colour models such as OKLCH in depth.

Learn more

The Learn library on the website has articles on colour contrast, WCAG and colour models.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Auric Artisan File Icons

What the Auric Artisan File Icons extension is, who it is for, how to install it and where to find each guide.

Auric Artisan File Icons gives the VS Code Explorer a consistent set of file and folder icons in three styles: **Facet**, **Playful** and **Modern**. Each style is a separate VS Code file icon theme, so you choose one for your Explorer and can switch at any time. Together they contain 1,464 SVG assets.

The extension adds **File Icon Studio**, available as a sidebar and as an editor tab. In its **Library** you search the artwork of all three styles and insert an icon into your code as inline SVG, JSX, an HTML image or a CSS background. In **Appearance** you recolour files and folders, adjust saturation, brightness and opacity, hide the Explorer arrows and choose a framework pack for Modern. In **Workspace** you manage smart detection, profiles, code hovers and settings imports.

Smart detection adds a small badge, a category colour and an explanation to recognised folders and files, for example a React project folder or a generated file. You can also assign any icon to a specific file or folder from the Explorer's right-click menu, and insert icons into code by hovering over tokens such as `:icon-file-typescript:`.

Who it is for

- Developers who want a distinctive, consistent Explorer and control over its colours.
- Teams who want project folders marked by type (framework, language, infrastructure, tests, docs).
- Front-end developers who want to reuse file-type artwork in their own code.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-file-icon
Extension ID	`auric-artisan.auric-artisan-file-icon`
Current version	0.1.1
Requires	Visual Studio Code 1.85 or later
Price	Free to install and use; no account required

```
code --install-extension auric-artisan.auric-artisan-file-icon
```

Quick start

1. Install the extension.
2. Run **Auric Artisan File Icons: Open File Icon Studio**, or select **Auric File Icons** in the Activity Bar.
3. In **Artwork style**, choose **Facet**, **Playful** or **Modern**. The Library shows that style's artwork.
4. Select **Use Facet in Explorer** (the button names the style you chose). Your Explorer switches to that theme.
5. Open the **Appearance** tab, pick a **Folder color**, and select **Apply appearance**. Reload the window if asked.
6. Right-click a folder in the Explorer, open **Auric File Icons** and choose **Set File or Folder Icon** to give it a specific icon.
7. To use an icon in code, open a writable file, search in **Library**, choose a **Code format** and select **Insert** on a card.

What you can do

- Choose one of three Explorer themes: **Auric Artisan File Icons: Facet**, **Playful** or **Modern**.
- Recolour file and folder icons independently and adjust saturation, brightness and opacity.
- Apply a ready-made profile (Classic Facet, Vivid Studio, Focused Explorer, Graphite Mono, High Visibility, Minimal) or save your own.
- Map extensions, filenames, folder names, parent-scoped names and workspace roots to icons.
- See smart badges, colours and explanations on recognised project folders and files, and turn a detected folder type into a permanent icon.
- Insert or copy artwork as inline SVG, JSX, an HTML image or a CSS background, from the Studio, a native picker or a code hover.
- Choose a Modern framework pack: Angular, NestJS, React or Vue.
- Import settings from the older standalone Playful extension or from Auric Icons: Modern.

In this folder

docs/

- Contents of docs/
- Getting started
- Choose a style
- Customise the Explorer
- Assign icons to files and folders
- Smart detection
- Profiles
- Insert icons into code
- Modern and older Playful settings
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Auric Artisan File Icons guides

Contents of the Auric Artisan File Icons guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Artisan File Icons. Start with Getting started, then use the guide for the job you want to do.

File	What it covers
getting-started.md	Install, open File Icon Studio, tour the screen and activate a style.
choose-a-style.md	Facet, Playful and Modern compared, and every way to switch.
customise-the-explorer.md	Colours, saturation, brightness, opacity, arrows and the Modern framework pack.
assign-icons.md	Give a file, folder or pattern its own icon, and clear it.
smart-detection.md	Badges, colours and explanations; built-in rules; custom rules; workspace scans.
profiles.md	Built-in and saved profiles, export and import.
insert-icons-into-code.md	Library cards, the inspector, the native picker and code hovers.
modern-and-legacy.md	Using File Icons with Auric Icons: Modern and moving from the old Playful extension.
reference.md	Every command, setting, menu, colour and format.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Getting started with Auric Artisan File Icons

Install File Icons, open File Icon Studio, tour its three tabs and activate a style in the Explorer.

Before you begin

- You need Visual Studio Code 1.85 or later. No other icon extension is needed.
- No account is needed. The extension does not contact a server and never runs code from your workspace.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Artisan File Icons** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-artisan-file-icon
```

For an official downloaded `.vsix` file, run **Extensions: Install from VSIX...** and choose the file.

2. Open File Icon Studio

- **Sidebar:** select **Auric File Icons** in the Activity Bar. The view is called **File Icon Studio**.
- **Editor tab:** run **Auric Artisan File Icons: Open File Icon Studio**, or select the ↗ button (**Open in editor tab**) in the sidebar.

Both have the same tools; the sidebar is compact.

3. Tour the Studio

Area	Contents
Header	"Auric Artisan / File Icon Studio", an **auricartisan.com** ↗ link, ↗ (sidebar only) and ⚙ (**All extension settings**).
Tabs	**Library**, **Appearance**, **Workspace**.
Target line	"Insert target · {file}" when a writable editor is open, otherwise "No editor target · open a writable file before inserting. Copy is always available."

Library

"Your files, with character." A search box, the **Artwork style**, **Show** and **Code format** pickers, an **Open folder artwork** check box, the **Use … in Explorer** button, a count badge and the icon grid.

Appearance

"One palette. Three styles." The **Modern framework pack** picker and a form with **File color**, **Folder color**, **Saturation**, **Brightness**, **Opacity**, **Hide Explorer arrows**, **Apply appearance** and **Restore defaults…**.

Workspace

"Connected to your project." Four panels: **Smart Explorer matches**, **Profiles**, **Insert from a code hover** and **Import existing file settings**.

The footer reads "Local artwork · no account · no telemetry" and shows which Auric Explorer style is active.

4. Activate a style

1. On **Library**, open **Artwork style** and choose **Facet**, **Playful** or **Modern**. Browsing a style only previews its artwork.
2. Select **Use Facet in Explorer** (the label follows your choice).
3. Your Explorer changes to that theme.

You can also run **Preferences: File Icon Theme** and choose **Auric Artisan File Icons: Facet**, **Auric Artisan File Icons: Playful** or **Auric Artisan File Icons: Modern**.

5. Try a colour

1. Open **Appearance**.
2. Set **Folder color** with the swatch or type a hex value such as `#D8A33C`.
3. Select **Apply appearance**. If VS Code asks, select **Reload**.

Result

Your Explorer uses an Auric style in your colours. Next, see Assign icons to files and folders and Insert icons into code.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Assign icons to files and folders

Give a file, folder, extension or pattern its own icon from the Explorer or in settings, and clear it again.

An **association** tells the icon theme which icon to show for a name. You can create associations from the Explorer or write them in settings.

From the Explorer

The Explorer actions appear when one of the Auric Artisan File Icons themes is active.

1. Right-click a file or folder in the Explorer.
2. Open **Auric File Icons** and select **Set File or Folder Icon**.
3. Choose the scope:

Resource	Scope options
A file, for example `app.config.ts`	**Exact filename**, **All *.config.ts files**, **Only parent/*.config.ts**, **Only parent/app.config.ts**
A file with no extension	**Exact filename**, **Only parent/name**
A folder	**All folders named "..."**, **Only "parent/folder"**
A workspace root folder	No choice: it is set as a workspace-root association.

4. Choose an icon from the list (titled **Set icon for ...**). Search by name or ID. The list shows icons for the Modern style when Modern is active, otherwise Facet's icons.
5. VS Code confirms, for example "TypeScript assigned to All *.config.ts files." Select **Reload** when asked.

Associations made this way are saved in the workspace settings, so they apply to this project only.

Clear an assignment

1. Right-click the same file or folder.
2. Open **Auric File Icons** › **Clear Custom File or Folder Icon**.
3. Choose the same scope you used. If there is nothing to clear for that scope, you see "No custom icon is assigned to ...".

In settings

Values are shipped icon IDs. The Explorer picker is the easiest way to find a valid ID.

```
{
  "auricFileIcons.files.associations": {
    "*.feature": "test",
    "server/*.log": "log",
    "PROJECT_NOTES": "markdown"
  },
  "auricFileIcons.folders.associations": {
    "domain": "models",
    "app/domain": "database"
  },
  "auricFileIcons.folders.rootAssociations": {
    "my-monorepo": "packages"
  }
}
```

Setting	Keys accepted
`auricFileIcons.files.associations`	`*.ext`, `parent/*.ext`, an exact filename, or `parent/name`.
`auricFileIcons.folders.associations`	A folder name, or `parent/folder`. Closed and open icons are applied together.
`auricFileIcons.folders.rootAssociations`	The name of a workspace root folder.

- Use the value `none` to remove a built-in match.
- A parent pattern matches the **direct** parent only, not any folder higher up.

Turn a detected project type into an icon

If smart detection recognises a folder (for example a Rust project), you can make that permanent:

1. Right-click the folder, open **Auric File Icons** and select **Apply Detected Folder Icon**.
2. The matching folder icon is assigned. If nothing is detected, you see "No project type was detected in this folder."

To do this for many folders, see Scan the workspace.

Result

After a reload, the chosen files and folders show the icons you assigned in the active Auric style.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Choose a style

Compare the Facet, Playful and Modern Explorer styles and switch between them.

File Icons contains three complete Explorer styles. Each is a separate VS Code file icon theme, and only one can be active at a time.

Style	Theme name in VS Code	Artwork	What you get
Facet	Auric Artisan File Icons: Facet	500 SVGs	Faceted artwork: 200 file designs and 150 closed and open folder pairs.
Playful	Auric Artisan File Icons: Playful	500 SVGs	A distinct playful palette and artwork covering the same file and folder catalog as Facet.
Modern	Auric Artisan File Icons: Modern	464 SVGs	219 file designs, 114 closed and open folder pairs, light alternatives and state overlays.

The totals include open-folder states, framework artwork and alternatives; they are not counts of unique file types.

Facet and Playful share the same mappings: 270 file extensions, 453 exact filenames, 84 language IDs and 357 folder aliases. Modern uses its own mappings and supports four framework packs (Angular, NestJS, React, Vue).

Switch style

From File Icon Studio

1. Open the **Library** tab.
2. Choose a style in **Artwork style**. This only previews the artwork.

3. Select **Use Facet in Explorer**, **Use Playful in Explorer** or **Use Modern in Explorer**.

From the Command Palette

Command	Result
Auric Artisan File Icons: Activate Facet Icon Theme	Facet becomes the file icon theme. VS Code confirms "Auric Artisan File Icons: Facet is active."
Auric Artisan File Icons: Activate Playful Icon Theme	Playful becomes the file icon theme.
Auric Artisan File Icons: Activate Modern Icon Theme	Modern becomes the file icon theme.

From VS Code

Run **Preferences: File Icon Theme** and choose one of the three **Auric Artisan File Icons** entries.

What carries across styles

- Appearance settings (colours, saturation, brightness, opacity, arrows) apply to all three styles at once.
- Your associations and smart detection work with all three.
- The Explorer right-click actions (**Set File or Folder Icon** and others) are available whenever any of the three is active.
- The Modern framework pack only affects Modern.

Live filename previews

While you type a new file or folder name in the Explorer, VS Code already shows the icon the active theme maps to that name, before you press `Enter`. This is VS Code's own theme rendering: no temporary file is created and no keystrokes are intercepted.

Result

Your Explorer uses the style you picked, with your appearance settings applied.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Customise the Explorer

Set file and folder colours, saturation, brightness, opacity, Explorer arrows and the Modern framework pack.

Appearance settings recolour the Explorer artwork while keeping each icon's faceted light and dark structure. They apply to Facet, Playful and Modern together. They do not change the artwork you insert into code.

Use the Appearance tab

1. Open File Icon Studio and select **Appearance**.
2. Set any of these:

Control	What it does	Range
File color	Recolours every file icon. Use the swatch, type a hex value, or **Reset** to `default`.	`default`, `#rgb` or `#rrggbb`
Folder color	Recolours every folder pair.	`default`, `#rgb` or `#rrggbb`
Saturation	Colour intensity. 0 is greyscale.	0 to 2, steps of 0.05
Brightness	Overall lightness.	0.5 to 1.5, steps of 0.05
Opacity	Transparency.	0.2 to 1, steps of 0.05
Hide Explorer arrows	Hides the expand and collapse chevrons.	On or off

3. Select **Apply appearance**.
4. If the artwork changed, VS Code shows "Auric file icons updated. Reload the window to apply every icon change." Select **Reload**. If nothing needed rebuilding, it says "No theme reload is needed."

Restore defaults... runs **Restore Icon Defaults** (see below).

Use the Configure menu

Run **Auric Artisan File Icons: Open File Icon Studio Settings**. A menu titled **Configure Auric Artisan File Icons** lists:

Item	What it does
File color	Choose from presets or a custom hex colour.
Folder color	Same, for folders.
Saturation	Choose 0, 0.5, 0.75, 1, 1.25, 1.5 or 2.
Brightness	Choose 0.6, 0.75, 0.9, 1, 1.1, 1.25 or 1.4.
Opacity	Choose 0.4, 0.55, 0.7, 0.85 or 1.
Explorer arrows	Toggle between visible and hidden.
Smart detection	Opens the smart detection menu.
Icon profiles	Opens **Apply Icon Profile**.
Scan workspace folders	Runs the workspace folder scan.
Diagnostics	Shows diagnostics.
Open all settings	Opens Settings filtered to this extension.
Restore defaults	Runs **Restore Icon Defaults**.

Colour presets

Auric Artisan File Icons: Choose Folder Color (and the colour items above) offer: **Default gold** (`default`), **Amber** `#D8A33C`, **Ocean** `#4AA3DF`, **Mint** `#43BFA5`, **Violet** `#8D73D8`, **Rose** `#D56782`, **Slate** `#718096`, and **Custom hex color...** for any three- or six-digit value.

Modern framework pack

On **Appearance**, choose **Modern framework pack**: **No framework pack**, **Angular**, **NestJS**, **React** or **Vue**. The setting is `auricFileIcons.modernPack`.

A pack changes Modern's automatic Explorer associations only. All framework artwork stays searchable in the Library whatever you choose.

Toggle arrows quickly

Run **Auric Artisan File Icons: Toggle Explorer Arrows**.

Restore defaults

Run **Auric Artisan File Icons: Restore Icon Defaults**, or select **Restore defaults…** on **Appearance**. After you confirm with **Restore**, it clears:

- the appearance settings (file and folder colour, saturation, brightness, opacity, arrows and Modern pack) in your user settings;
- the file, folder and root associations and the smart detection settings in the current workspace.

Settings files

The Studio writes appearance settings for you. You can also set them in `settings.json`:

```
{
  "auricFileIcons.folderColor": "#D8A33C",
  "auricFileIcons.fileColor": "default",
  "auricFileIcons.saturation": 1.15,
  "auricFileIcons.brightness": 1.05,
  "auricFileIcons.opacity": 0.9,
  "auricFileIcons.hidesExplorerArrows": false,
  "auricFileIcons.modernPack": "none"
}
```

Workspace or folder settings override user settings. If a change seems to have no effect, check for a workspace value.

Result

After a reload, the Explorer shows your colours and settings in whichever Auric style is active.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Insert icons into code

Insert or copy File Icons artwork as inline SVG, JSX, an HTML image or a CSS background from the Studio, a native picker or a code hover.

File Icon Studio lets you reuse the shipped file and folder artwork in your own code.

Output formats

Format	Setting value	Output
Inline SVG	`svg`	A self-contained `<svg>` element, 24 × 24 by default.

Format	Setting value	Output
JSX	`jsx`	The SVG with JSX-compatible attribute names.
HTML image	`img`	A 24 × 24 `<img>` with the SVG embedded as a data URI.
CSS background	`css`	A `background-image` declaration with the SVG embedded as a data URI.

`auricFileIcons.insertFormat` sets the default format (`svg`) for the native picker and the Studio.

> **Note:** Code output always uses the shipped artwork of the chosen style. Your Explorer colour, saturation, brightness and opacity settings do not change exported code. Add meaningful alternative text when an icon conveys information.

From the Library tab

1. Open a writable text file and place the cursor. The Studio's target line shows "Insert target · {file}".
2. In File Icon Studio, open **Library**.
3. Set **Artwork style** (**Facet**, **Playful** or **Modern**) and **Show** (**Files & folders**, **Files** or **Folders**).
4. Type in **Search icons** (placeholder "TypeScript, React, src…", up to 150 characters). Press `/` to jump to the search box.
5. Choose a **Code format**: **Inline SVG**, **JSX**, **HTML image** or **CSS background**.
6. Tick **Open folder artwork** to show folders in their open state.
7. Hover over or focus a card, then choose **Insert**. Choose **Copy** to use the clipboard instead.

Insertion replaces selected text, works at multiple cursors and is a single undo step. It does not save the file and adds no surrounding quotes. **Copy** works without an open editor.

The grid loads more cards as you scroll; use **Load more icons** if it appears.

The inspector

Open a card's inspector to see the exact output before using it. The **Icon inspector** dialog shows a large preview, the card's hover token, the style, format and kind (for example "Facet · SVG · File icon"), and the generated code.

Button	Action
Insert into code	Inserts the code at your selections.
Copy code	Copies the code.
Copy hover token	Copies a token such as `:icon-file-typescript:` for use with code hovers.

From the Command Palette or editor menu

1. In a writable editor, run **Auric Artisan File Icons: Insert Icon into Code…**, or right-click in the editor and choose it.
2. A picker titled **Insert a file or folder icon** lists icons of the active Auric style (Facet if another theme is active), with their tokens.
3. Choose one. It is inserted in your `insertFormat`.

From a code hover

1. Type a complete token such as `:icon-file-typescript:` or `:icon-folder-src:`. Copy valid tokens from the inspector.
2. Hover over the token.
3. Choose **Insert SVG**, **Insert JSX** or **Insert IMG**.

Only that token is replaced, in one undo step. Hovering alone never changes your file.

- Hovers prefer the active Auric Explorer style, and use Facet when another theme is active. If the icon is not in that style, an available style is used.
- If the document changes, closes or becomes hidden before you choose, hover again for a fresh action.
- Turn hovers on or off with **Enable code hovers** on the Workspace tab or `auricFileIcons.hover.enabled`. VS Code's own editor hovers must also be enabled.

Result

The artwork is in your code, in the format you chose, taken from the shipped style you selected.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Modern and older Playful settings

Use File Icons alongside Auric Icons: Modern, import Modern's file settings, and move from the old standalone Playful file icon extension.

File Icons brings together Explorer styles that used to live in separate extensions. This page explains how to move your settings and avoid duplicates.

Using File Icons with Auric Icons: Modern

File Icons includes Modern's Explorer artwork (464 assets) and keeps its theme ID, `auric-artisan-modern-icon`. The Auric Icons: Modern extension also has a general-purpose 720-icon library, which File Icons does not include.

- Keep **Auric Icons: Modern** installed if you use its icon library.
- The Modern version currently on the Marketplace (0.12.1) still includes its own **Auric Icons: Modern** file icon theme under the same theme ID. With both installed, the **File Icon Theme** picker can show two Modern entries, and the two extensions use different settings (`auricIcons.*` for Modern, `auricFileIcons.*` for File Icons).
- The File Icons documentation states that an updated Modern release no longer registers a file theme. Until you have that release, choose one extension to manage Modern's Explorer icons and use its settings.

Import Modern's file settings

1. Open File Icon Studio › **Workspace** › **Import existing file settings**.
2. Select **Import Modern file settings**, or run **Auric Artisan File Icons: Import Modern File Icon Settings...**.
3. Confirm the import.

Only missing file-theme settings are copied, at their original user, workspace or folder scope. Your existing File Icons settings, Modern's library preferences, likes, recent items and emoji are not changed.

Moving from the standalone Playful extension

Facet and Playful now live in File Icons. Their theme IDs (`auric-artisan-file-icons-facet` and `auric-artisan-file-icons-playful`) and all 500 original Playful SVGs are kept.

1. Before uninstalling the old Playful extension, export any saved profiles from it.
2. Install File Icons and run **Auric Artisan File Icons: Import Legacy Playful Settings…** (or **Import legacy settings…** on the Workspace tab). Missing `auricPlayfulIcons` settings are copied into `auricFileIcons` at their original scope. Existing values are not overwritten.
3. Import exported profiles through **Workspace** › **Profiles** › **Import…**.
4. Disable or uninstall the old Playful extension to avoid duplicate themes and decorations.

File Icons cannot read another extension's private profile storage, so step 1 matters.

Not the same Playful

The Playful Explorer style in File Icons is a different set of artwork from the **Auric Icons: Playful** extension, which is a 1,225-icon library for inserting into code and has no Explorer theme.

Result

Your earlier settings are in File Icons, and you see each Explorer theme once.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Profiles

Apply a built-in icon profile, save your own, and move profiles between machines as JSON.

A **profile** is a named, portable set of appearance settings, associations and smart detection settings.

Apply a profile

1. Run **Auric Artisan File Icons: Apply Icon Profile**, or **Apply profile…** on the **Workspace** tab.
2. Choose a profile. Built-in profiles are marked **Built-in**; your saved profiles follow.
3. Select **Reload** when asked.

Built-in profile	Description	Key values
Classic Facet	Balanced original colours and full smart decorations.	Default colours, 1 / 1 / 1, badge and colour
Vivid Studio	Brighter, richer colour for large project explorers.	Saturation 1.35, brightness 1.08
Focused Explorer	Blue folders, softer files, hidden arrows and badge-only smart matches.	Files `#718096`, folders `#4AA3DF`, opacity 0.88
Graphite Mono	Near-monochrome files and folders with semantic smart colours.	`#7B8796`, saturation 0.18, colour only
High Visibility	Maximum opacity, extra brightness and propagated badges.	Folders `#E0A82E`, saturation 1.2, brightness 1.18
Minimal	Muted monochrome artwork with tooltip-only smart explanations.	`#8993A1`, saturation 0, opacity 0.72, arrows hidden

Built-in profiles keep your custom associations and smart rules. Applying a saved or imported profile replaces them with the profile's own.

Save your current set-up

1. Run **Save Current Icon Profile**, or **Save current...** on the Workspace tab.
2. Enter a name (up to 80 characters).
3. VS Code confirms "Saved icon profile "..."."

The Workspace tab's **Profiles** panel shows how many profiles are available and their names.

Delete a saved profile

Run **Delete Saved Icon Profile** (or **Delete saved...**), choose the profile and confirm with **Delete**. Built-in profiles cannot be deleted.

Export and import

- **Export Icon Profile to Clipboard** (or **Export**) copies your current settings as profile JSON. VS Code confirms "Current icon profile copied to the clipboard."
- **Import Icon Profile from Clipboard** (or **Import...**) reads profile JSON from the clipboard, then asks "Import "..."?" with **Apply** or **Apply and Save**.

Imports are checked first:

Message	Meaning
The clipboard does not contain an icon profile.	The clipboard is empty.
The clipboard profile is larger than the 256 KiB safety limit.	Profiles must be under 256 KiB.
The clipboard does not contain valid JSON.	The text is not JSON.
Unsupported icon profile schema: ...	The profile came from an unknown source.
Invalid icon profile: ...	A value failed validation.

Profiles exported from the older standalone Playful extension can be imported. Values are bounded and sanitised before they are written, and icon IDs that do not exist are dropped.

Result

You can switch between complete Explorer looks in two steps and carry them to other machines.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Smart detection

Content-aware badges, colours and explanations in the Explorer, the built-in rules, custom rules and workspace scans.

Smart detection looks at a folder's direct contents, its package dependencies, or the start of a small text file, and adds an Explorer **decoration**: a badge of up to two characters, a category colour and a hover explanation. For example, a folder containing `Cargo.toml` gets the badge **RS** and the explanation "Rust project".

Decorations sit on top of the base icon; they do not replace its SVG. They run only while one of the Auric Artisan File Icons themes is active.

Turn it on or off

- On the **Workspace** tab, tick or clear **Enable smart detection**.
- Run **Auric Artisan File Icons: Toggle Smart Icon Detection**. VS Code confirms "Smart icon detection enabled" or "disabled".
- Set `auricFileIcons.smart.enabled`.

Folder detection and file detection can be switched separately with `auricFileIcons.smart.folders` and `auricFileIcons.smart.files`.

Choose how matches appear

Run **Auric Artisan File Icons: Configure Smart Icon Detection** (or **Configure** on the Workspace tab). The menu offers:

Item	Effect
Enable / Disable smart detection	Toggles detection for this workspace.
Badge and color	Badge plus category colour (default).
Badge only	Badge without colour.
Color only	Category colour without a badge.
Tooltip only	Only the hover explanation.
Refresh smart matches	Clears cached results.
Edit custom rules	Opens `settings.json` at `auricFileIcons.smart.rules`.

Choices made in this menu are saved in the workspace settings.

Explain, refresh and apply

Action	How	Result
Explain a match	Right-click › **Auric File Icons** › **Explain Smart Icon Match**, or the command	A message naming the rule that matched.
Refresh	**Refresh Smart Icon Matches**, or **Refresh** on the Workspace tab	Clears the cache: "Smart icon matches refreshed."
Apply a detected icon	Right-click a folder › **Apply Detected Folder Icon**	Saves the matching folder icon as an association.

Built-in folder rules

Checked highest priority first. A folder matches when it contains one of the listed names, or when its `package.json` lists one of the dependencies.

Badge	Explanation	Recognised by
AL	Auric Live project	`.auric-live`, `auric.live.json`, `.auricignore-live`
AS	Auric Studio project	`.auric-artisan`
AF	Auric Formatter project	`.auricformatrc*`, `.auricformatignore`, `auric.format.json`
AU	Auric workspace	`.auric`, `.auricignore`
NG	Angular project	`angular.json` or `@angular/core`
NX	Next.js project	`next.config.*` or `next`
NU	Nuxt project	`nuxt.config.*` or `nuxt`
SV	Svelte project	`svelte.config.*`, `svelte` or `@sveltejs/kit`
VU	Vue project	`vue`
RE	React project	`react` or `react-dom`
NS	NestJS project	`@nestjs/core`
VT	Vite project	`vite.config.*` or `vite`
RM	Remix project	`remix.config.*` or `@remix-run/*`
QW	Qwik project	`qwik.config.*` or `@builder.io/qwik`
SO	SolidJS project	`solid-js` or `@solidjs/start`
DN	Deno project	`deno.json`, `deno.jsonc`, `deno.lock`
BN	Bun project	`bun.lock`, `bun.lockb`, `bunfig.toml`
NT	.NET project	`*.sln`, `*.csproj`, `*.fsproj`, `*.vbproj`
FL	Flutter or Dart project	`pubspec.yaml`, `pubspec.yml`
RS	Rust project	`Cargo.toml`
GO	Go project	`go.mod`
PY	Python project	`pyproject.toml`, `requirements*.txt`, `setup.py`, `Pipfile`
JV	Java or JVM project	`pom.xml`, `build.gradle`, `build.gradle.kts`, `settings.gradle`, `settings.gradle.kts`
BZ	Bazel workspace	`WORKSPACE`, `WORKSPACE.bazel`, `MODULE.bazel`, `BUILD.bazel`
PM	Pulumi project	`Pulumi.yaml`
SS	Serverless project	`serverless.yml`, `serverless.yaml`
DC	Dev Container workspace	`.devcontainer`, `devcontainer.json`, `.devcontainer.json`
PR	Prisma data project	`prisma`, or `prisma` or `@prisma/client`
DZ	Drizzle data project	`drizzle`, or `drizzle-orm` or `drizzle-kit`
SB	Supabase project	`supabase` or `@supabase/*`
TW	Tailwind CSS project	`tailwind.config.*` or `tailwindcss`
MR	Monorepo workspace	`pnpm-workspace.yaml`, `lerna.json`, `nx.json`, `turbo.json`
TS	TypeScript project	`tsconfig.json`
ND	JavaScript or Node project	`package.json`

Badge	Explanation	Recognised by
HM	Helm chart	`Chart.yaml`
TF	Terraform project	`*.tf`
DK	Container project	`Dockerfile`, `docker-compose.*`, `compose.yml`, `compose.yaml`
T	Test workspace	`jest.config.*`, `vitest.config.*`, `playwright.config.*`, `pytest.ini`
DC	Documentation workspace	`README*` together with `mkdocs.yml`, `docusaurus.config.*` or `docs`
DB	Database workspace	`prisma`, `migrations`, `schema.sql`

Built-in file rules

Badge	Explanation	Recognised by
!	Sensitive key material	A private key header in the content
G	Generated file	`@generated`, `DO NOT EDIT` or `generated by` in the content
NG	Angular component	`@Component(` or `@NgModule(`
VU	Vue component	A `.vue` file with `<template` or `defineComponent(`
RE	React module	An import from `react`, `React.createElement(` or `use client`
T	Test source	`*.test.*`, `*.spec.*`, `test_*`, `*_test.*`
SC	Schema-bearing document	A `"$schema"` key
OA	OpenAPI document	`openapi` or `swagger` keys
MG	Database migration	A `.sql` file with `CREATE TABLE`, `ALTER TABLE` or `CREATE INDEX`
IN	Localization resource	`"translations"`, `"locale"`, `msgid` or `msgstr`
#!	Executable script	Content starting with `#!`

Category colours

Each rule has a category colour. You can change them in `workbench.colorCustomizations`.

Colour ID	Category	Dark default	Light default
`auricFileIcons.smart.auric`	Auric projects	`#F0C15A`	`#946414`
`auricFileIcons.smart.framework`	Frameworks	`#70C7FF`	`#176E9F`
`auricFileIcons.smart.language`	Languages	`#B097FF`	`#6245B2`
`auricFileIcons.smart.infrastructure`	Infrastructure	`#63D8C4`	`#217B70`
`auricFileIcons.smart.test`	Tests	`#7DD58A`	`#2F7838`
`auricFileIcons.smart.docs`	Documentation	`#E28EE8`	`#873C95`
`auricFileIcons.smart.data`	Data	`#F2A665`	`#99521C`
`auricFileIcons.smart.generated`	Generated files	`#A9B4C2`	`#586473`
`auricFileIcons.smart.security`	Security	`#F2768D`	`#A3243E`

Colour ID	Category	Dark default	Light default
`auricFileIcons.smart.custom`	Custom rules (default)	`#F0C15A`	`#946414`

High-contrast themes use lighter variants of each.

Custom rules

Custom rules run before the built-in ones, highest priority first. Add them to `auricFileIcons.smart.rules`:

```
{
  "auricFileIcons.smart.rules": [
    {
      "id": "design-system",
      "label": "Design system package",
      "target": "folder",
      "badge": "DS",
      "color": "custom",
      "priority": 950,
      "whenAll": ["package.json", "tokens"],
      "dependenciesAny": ["storybook", "@storybook/*"]
    }
  ]
}
```

Field	Meaning
`badge`	Required. At most two characters (a VS Code limit).
`id`, `label`	Identify the rule; the label is shown in explanations.
`target`	`folder` (default) or `file`.
`color`	`auric`, `framework`, `language`, `infrastructure`, `test`, `docs`, `data`, `generated`, `security` or `custom`.
`priority`	Higher runs first.
`whenAll`, `whenAny`	Child names that must all, or any, be present (folders).
`dependenciesAny`	Package dependencies, any of which match (folders).
`fileNames`, `contentAny`, `contentAll`, `contentPrefix`	Name patterns and content checks (files).

Patterns support `*` and `?` only, never regular expressions.

Scan the workspace for folder icons

1. Run **Auric Artisan File Icons: Scan Workspace for Folder Icons**, or **Scan folders…** on the Workspace tab.
2. A progress notification ("Scanning project folders for Facet icons") inspects first-level folders, up to `auricFileIcons.smart.scanLimit` (80 by default). You can cancel it.
3. Detected folders get icon associations in the workspace settings. Existing manual associations are preserved. The result reads, for example, "Applied 4 detected folder icon(s); preserved 2 existing assignment(s)."

Bounds and exclusions

Setting	Default	Range	Meaning
`auricFileIcons.smart.maxFileSize`	65536 bytes (64 KiB)	1,024 to 262,144	Largest text file inspected.
`auricFileIcons.smart.maxFolderEntries`	250	25 to 2,000	Direct child names considered per folder.
`auricFileIcons.smart.cacheSeconds`	15	1 to 3,600	How long a match is cached.
`auricFileIcons.smart.scanLimit`	80	1 to 250	Folders inspected by the workspace scan.
`auricFileIcons.smart.exclusions`	See below	List	Folder names or simple wildcards never inspected.
`auricFileIcons.smart.propagate`	false	On or off	Show child decorations on parent folders.
`auricFileIcons.smart.watch`	true	On or off	Watch project marker files and refresh matching folders.

Default exclusions: `.git`, `node_modules`, `vendor`, `dist`, `build`, `out`, `coverage`, `.next`, `.nuxt`, `.cache`, `.turbo`, `target`, `__pycache__`. Binary, oversized and unavailable files are skipped.

Inspection uses VS Code's file system access, so it works in local, remote and virtual workspaces where the provider allows reading. It never runs project code.

Result

Recognised folders and files carry badges, colours and explanations, and you can make any detected folder type a permanent icon.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Auric Artisan File Icons reference

Every command, setting, menu, view, theme, colour and Studio control in Auric Artisan File Icons 0.1.1.

This reference describes version 0.1.1 as published on the Visual Studio Marketplace.

File icon themes

Theme name	Theme ID	Artwork
Auric Artisan File Icons: Facet	`auric-artisan-file-icons-facet`	500 SVGs
Auric Artisan File Icons: Playful	`auric-artisan-file-icons-playful`	500 SVGs
Auric Artisan File Icons: Modern	`auric-artisan-modern-icon`	464 SVGs

Commands

Category in the Command Palette: **Auric Artisan File Icons**.

Command	ID	What it does
Activate Facet Icon Theme	`auricFileIcons.activate`	Sets Facet as the file icon theme.

Command	ID	What it does
Activate Playful Icon Theme	`auricFileIcons.activatePlayful`	Sets Playful as the file icon theme.
Activate Modern Icon Theme	`auricFileIcons.activateModern`	Sets Modern as the file icon theme.
Open File Icon Studio	`auricFileIcons.openStudio`	Opens the Studio editor tab beside the editor.
Open File Icon Studio Settings	`auricFileIcons.configure`	The **Configure Auric Artisan File Icons** menu.
Choose Folder Color	`auricFileIcons.pickFolderColor`	Folder colour presets or a custom hex.
Toggle Explorer Arrows	`auricFileIcons.toggleExplorerArrows`	Shows or hides the Explorer chevrons.
Insert Icon into Code…	`auricFileIcons.quickInsert`	Picker of icons in the active style; inserts in `insertFormat`.
Set File or Folder Icon	`auricFileIcons.assignResourceIcon`	Assigns an icon to the selected Explorer resource.
Clear Custom File or Folder Icon	`auricFileIcons.clearResourceIcon`	Removes that assignment.
Configure Smart Icon Detection	`auricFileIcons.smart.configure`	Smart detection menu.
Refresh Smart Icon Matches	`auricFileIcons.smart.refresh`	Clears cached matches.
Explain Smart Icon Match	`auricFileIcons.smart.inspect`	Explains the match for the selected resource.
Toggle Smart Icon Detection	`auricFileIcons.smart.toggle`	Turns smart detection on or off.
Apply Detected Folder Icon	`auricFileIcons.smart.applyDetectedFolderIcon`	Saves the detected folder icon as an association.
Scan Workspace for Folder Icons	`auricFileIcons.smart.scanWorkspace`	Scans first-level folders and assigns detected icons.
Restore Icon Defaults	`auricFileIcons.restoreDefaults`	Confirms, then clears appearance, associations and smart settings.
Apply Icon Profile	`auricFileIcons.profiles.apply`	Applies a built-in or saved profile.
Save Current Icon Profile	`auricFileIcons.profiles.save`	Saves your current set-up as a profile.
Delete Saved Icon Profile	`auricFileIcons.profiles.delete`	Deletes a saved profile.
Export Icon Profile to Clipboard	`auricFileIcons.profiles.export`	Copies the current profile JSON.
Import Icon Profile from Clipboard	`auricFileIcons.profiles.import`	Imports profile JSON.
Show Icon Theme Diagnostics	`auricFileIcons.diagnostics`	Writes a report to the **Auric Artisan File Icons** output channel.
Import Legacy Playful Settings…	`auricFileIcons.migratePlayful`	Copies missing settings from the old Playful extension.
Import Modern File Icon Settings…	`auricFileIcons.migrateModern`	Copies missing Modern file-theme settings.

There are no default keybindings. Assign your own in **Keyboard Shortcuts**.

Menus

Menu	Item	When shown
Explorer right-click › **Auric File Icons**	**Open File Icon Studio**	Always
Explorer right-click › **Auric File Icons**	**Explain Smart Icon Match**	An Auric file icon theme is active
Explorer right-click › **Auric File Icons**	**Apply Detected Folder Icon**	On folders, with an Auric theme active
Explorer right-click › **Auric File Icons**	**Set File or Folder Icon**	An Auric file icon theme is active
Explorer right-click › **Auric File Icons**	**Clear Custom File or Folder Icon**	An Auric file icon theme is active
Editor right-click	**Insert Icon into Code…**	A writable editor has focus

Views

Place	Contents
Activity Bar	**Auric File Icons** container with the **File Icon Studio** view.
Editor tab	**File Icon Studio**, opened beside the active editor.
Output	**Auric Artisan File Icons** channel for diagnostics.

Settings

Setting	Type	Values or range	Default	What it does
`auricFileIcons.fileColor`	string	`default`, `#rgb`, `#rrggbb`	`default`	Recolours every file icon.
`auricFileIcons.folderColor`	string	`default`, `#rgb`, `#rrggbb`	`default`	Recolours every folder pair.
`auricFileIcons.saturation`	number	0 to 2	1	Saturation multiplier; 0 is greyscale.
`auricFileIcons.brightness`	number	0.5 to 1.5	1	Brightness multiplier.
`auricFileIcons.opacity`	number	0.2 to 1	1	Icon opacity.
`auricFileIcons.hidesExplorerArrows`	boolean	`true`, `false`	`false`	Hides Explorer chevrons.
`auricFileIcons.files.associations`	object	`*.ext`, `parent/*.ext`, filename or `parent/name` to icon ID	`{}`	File associations; `none` removes a match.
`auricFileIcons.folders.associations`	object	folder or `parent/folder` to icon ID	`{}`	Folder associations.
`auricFileIcons.folders.rootAssociations`	object	root name to icon ID	`{}`	Workspace-root associations.
`auricFileIcons.smart.enabled`	boolean	`true`, `false`	`true`	Smart decorations on or off.
`auricFileIcons.smart.folders`	boolean	`true`, `false`	`true`	Classify folders.
`auricFileIcons.smart.files`	boolean	`true`, `false`	`true`	Classify small text files.
`auricFileIcons.smart.style`	string	`badgeAndColor`, `badge`, `color`, `tooltip`	`badgeAndColor`	Decoration style.

Setting	Type	Values or range	Default	What it does
`auricFileIcons.smart.propagate`	boolean	`true`, `false`	`false`	Propagate child decorations to parents.
`auricFileIcons.smart.watch`	boolean	`true`, `false`	`true`	Watch project markers and refresh.
`auricFileIcons.smart.maxFileSize`	integer	1024 to 262144	65536	Largest file inspected, in bytes.
`auricFileIcons.smart.maxFolderEntries`	integer	25 to 2000	250	Child names considered per folder.
`auricFileIcons.smart.cacheSeconds`	integer	1 to 3600	15	Cache duration.
`auricFileIcons.smart.scanLimit`	integer	1 to 250	80	Folders checked by the workspace scan.
`auricFileIcons.smart.exclusions`	array	folder names or wildcards	13 common folders	Never inspected.
`auricFileIcons.smart.rules`	array	rule objects	`[]`	Custom rules, run before built-ins.
`auricFileIcons.hover.enabled`	boolean	`true`, `false`	`true`	Code hovers for icon tokens.
`auricFileIcons.insertFormat`	string	`svg`, `jsx`, `img`, `css`	`svg`	Default code format.
`auricFileIcons.modernPack`	string	`none`, `angular`, `nest`, `react`, `vue`	`none`	Modern framework associations.

Default exclusions: `.git`, `node_modules`, `vendor`, `dist`, `build`, `out`, `coverage`, `.next`, `.nuxt`, `.cache`, `.turbo`, `target`, `__pycache__`.

Theme colours

Ten colours for smart decorations can be changed in `workbench.colorCustomizations`: `auricFileIcons.smart.auric`, `.framework`, `.language`, `.infrastructure`, `.test`, `.docs`, `.data`, `.generated`, `.security` and `.custom`. See Smart detection for their defaults.

File Icon Studio controls

Tab	Control	Options or action
Library	**Search icons**	Free text, up to 150 characters.
Library	**Artwork style**	Facet, Playful, Modern.
Library	**Show**	Files & folders, Files, Folders.
Library	**Code format**	Inline SVG, JSX, HTML image, CSS background.
Library	**Open folder artwork**	Shows folders in their open state.
Library	**Use ... in Explorer**	Activates the chosen style.
Library	Card	**Insert**, **Copy**, inspector; **Load more icons**.
Appearance	**Modern framework pack**	No framework pack, Angular, NestJS, React, Vue.

Tab	Control	Options or action
Appearance	**File color**, **Folder color**	Swatch, hex field, **Reset**.
Appearance	**Saturation**, **Brightness**, **Opacity**	Sliders.
Appearance	**Hide Explorer arrows**	Check box.
Appearance	Buttons	**Apply appearance**, **Restore defaults...**.
Workspace	Smart Explorer matches	**Enable smart detection**, **Configure**, **Refresh**, **Scan folders...**.
Workspace	Profiles	**Apply profile...**, **Save current...**, **Export**, **Import...**, **Delete saved...**.
Workspace	Insert from a code hover	**Enable code hovers**.
Workspace	Import existing file settings	**Import Modern file settings**, **Import legacy settings...**, **Diagnostics**.
Inspector	Buttons	**Insert into code**, **Copy code**, **Copy hover token**.
Header	Buttons	**auricartisan.com** ↗, ↗ (open in editor tab), ⚙ (all extension settings).

Keys: `/` focuses search; dropdown pickers support arrow keys, `Home`, `End` and `Escape`.

Artwork and mappings

Item	Facet	Playful	Modern
SVG assets	500	500	464
File designs	200	200	219
Folder pairs	150	150	114
Mappings	270 extensions, 453 filenames, 84 language IDs, 357 folder aliases	Same as Facet	Its own mappings and 4 framework packs

Diagnostics report

Show Icon Theme Diagnostics lists the version, active theme, workspace trust, remote host, folder count, asset counts, your appearance values, association counts, smart detection settings and limits, the number of custom rules and saved profiles. It omits filenames and file contents.

Limits

Item	Limit
Search text	150 characters
Smart badge	2 characters
Profile import	256 KiB
Profile name	80 characters

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Artisan File Icons.

Symptom	Why it happens	What to do
The Explorer still uses another icon set.	Browsing a style in the Studio only previews it.	Select **Use ... in Explorer**, or choose the theme in **Preferences: File Icon Theme**.
Colours or associations have not changed.	VS Code caches icon themes until a reload.	Accept the reload prompt, or run **Developer: Reload Window**.
Still no change after reloading.	Workspace or folder settings override user settings.	Check `.vscode/settings.json` for `auricFileIcons.*` values.
Duplicate Auric themes or actions.	The old standalone Playful extension is still installed, or Auric Icons: Modern also provides a Modern theme.	Disable the old Playful extension. See Modern and older Playful settings.
The **Auric File Icons** Explorer submenu has only **Open File Icon Studio**.	The other actions need an Auric file icon theme to be active.	Activate Facet, Playful or Modern.
No **Insert** action, or "No editor target".	No writable text editor is open.	Open and click into a writable file first; **Copy** works without one.
A token hover does not appear.	The token is incomplete, hovers are off, or VS Code's editor hovers are disabled.	Use a complete token copied from the inspector, and enable `auricFileIcons.hover.enabled` and editor hovers.
A hover action no longer inserts.	The document changed after the hover appeared.	Hover again for a fresh action.
An unexpected smart badge appears.	A built-in or custom rule matched.	Run **Explain Smart Icon Match**, then refresh or adjust smart settings.
Smart badges do not appear in a remote or virtual workspace.	The file system provider may not allow reading, or the folder is excluded.	Check `auricFileIcons.smart.exclusions` and the provider's support.
Exported code does not match my Explorer colours.	Code output always uses shipped artwork.	This is by design; recolour the code yourself if needed.
Import of a profile fails.	The clipboard is empty, too large, not JSON, or from an unknown source.	See the messages in Profiles.
Old Playful profiles are missing after switching.	File Icons cannot read another extension's private storage.	Export profiles from the old extension first, then import them.

Diagnostics

Run **Auric Artisan File Icons: Show Icon Theme Diagnostics**. The report opens in the **Auric Artisan File Icons** output channel and omits filenames and file contents. Include it, with your VS Code version, the extension version, the active style and the steps to reproduce, when you contact support at https://auricartisan.com/company/contact/. Remove private information from screenshots.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Frequently asked questions

Short answers to common questions about Auric Artisan File Icons.

Do I need another icon extension?

No. All three styles and their artwork are included.

Can I use two styles at once?

No. VS Code allows one file icon theme at a time. Switch whenever you like.

Why do changes need a reload?

VS Code caches icon themes. The extension rebuilds the theme when you change appearance or associations; a reload shows the new icons.

Does smart detection change the icon itself?

No. It adds a badge, colour and explanation on top. Use **Apply Detected Folder Icon** or **Scan Workspace for Folder Icons** to make a detected icon permanent.

Does it run my project's code?

No. Smart detection reads a bounded number of names and the start of small text files. It never executes anything.

Can I use the icons in my own projects?

You can insert them into code from the Studio. The artwork is covered by the Auric Artisan Free Use Licence, which restricts redistribution, resale, republishing and extracting bundled artwork. Review it before reusing artwork outside the extension.

Is the Playful style the same as Auric Icons: Playful?

No. They are different artwork. See Modern and older Playful settings.

How is this different from Icon Studio?

File Icons changes your Explorer and offers its own file-type artwork for code. Auric Artisan Icon Studio is a general icon and emoji browser and does not change the Explorer. See the comparison.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Auric Artisan File Icons: other information

Contents of the supporting pages for Auric Artisan File Icons.

This folder holds supporting pages for Auric Artisan File Icons: terms, commands, limits, a comparison with the other Auric icon extensions, the release history and privacy behaviour.

File	What it covers
glossary.md	Terms used in File Icons.
shortcuts.md	Commands, menus and in-Studio keys.
limits-and-accuracy.md	What the themes, smart detection and code output can and cannot do.
related.md	Comparison with Icon Studio, Icon Hub, the Auric collections, Emoji and Icon Pack.
release-notes.md	User-facing history.
privacy.md	What is read, stored and never sent.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Shortcuts and commands

Commands, context menus and in-Studio keys in Auric Artisan File Icons.

File Icons has no default keybindings. Open **Preferences: Open Keyboard Shortcuts**, search for "Auric Artisan File Icons" and assign keys to the commands you use most, such as **Insert Icon into Code...** or **Open File Icon Studio**.

Keys inside File Icon Studio

Key	Action
`/`	Focus the icon search
`Arrow Up` / `Arrow Down`	Move through picker options
`Home` / `End`	First or last option
`Escape`	Close an open picker or the inspector

Commands

Type "Auric Artisan File Icons" in the Command Palette.

Group	Commands
Themes	Activate Facet Icon Theme; Activate Playful Icon Theme; Activate Modern Icon Theme
Studio	Open File Icon Studio; Open File Icon Studio Settings; Insert Icon into Code...
Appearance	Choose Folder Color; Toggle Explorer Arrows; Restore Icon Defaults
Associations	Set File or Folder Icon; Clear Custom File or Folder Icon
Smart detection	Configure Smart Icon Detection; Refresh Smart Icon Matches; Explain Smart Icon Match; Toggle Smart Icon Detection; Apply Detected Folder Icon; Scan Workspace for Folder Icons

Group	Commands
Profiles	Apply Icon Profile; Save Current Icon Profile; Delete Saved Icon Profile; Export Icon Profile to Clipboard; Import Icon Profile from Clipboard
Other	Show Icon Theme Diagnostics; Import Legacy Playful Settings…; Import Modern File Icon Settings…

Context menus

- **Explorer:** right-click › **Auric File Icons** submenu.
- **Editor:** right-click › **Insert Icon into Code…**.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Glossary

Definitions of the terms used in Auric Artisan File Icons.

Term	Meaning
Association	A rule mapping an extension, filename, folder name or pattern to an icon ID.
Badge	Up to two characters shown beside a resource in the Explorer by smart detection.
Decoration	A badge, colour or tooltip VS Code adds to an Explorer item.
Direct parent	The folder immediately containing a file or folder; `parent/name` patterns only match this.
Exclusion	A folder name or pattern that smart detection never inspects.
Facet	The faceted Explorer style with 500 SVGs.
File icon theme	The set of icons VS Code shows beside files and folders. One is active at a time.
Folder pair	The closed and open versions of one folder icon.
Framework pack	Extra Modern associations for Angular, NestJS, React or Vue.
Hover token	Text such as `:icon-file-typescript:` that a code hover can replace with artwork.
Icon ID	The name of a shipped icon, such as `test` or `database`, used as an association value.
Inspector	The dialog showing a card's exact code output.
Modern	The Explorer style with 464 SVGs, shared with Auric Icons: Modern's theme ID.
Playful	The Explorer style with 500 SVGs and its own palette. Not the Auric Icons: Playful library.
Profile	A named, portable set of appearance, association and smart detection settings.
Root association	An icon for a workspace root folder, by name.
Smart detection	Content-aware classification of folders and small files, shown as decorations.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Limits and accuracy

What the File Icons themes, smart detection and code output can and cannot do.

Themes

- Only one file icon theme can be active in VS Code.
- Appearance and association changes need a window reload.
- Parent patterns match the direct parent only, not a recursive path.
- The asset totals include open-folder states, framework artwork and alternatives; they are not counts of unique file types.

Smart detection

- Detection is a heuristic based on names, dependencies and the start of small files. It can miss projects or match unexpectedly; use **Explain Smart Icon Match** to see why.
- By default it considers up to 250 direct entries per folder and text files up to 64 KiB, cached for 15 seconds. The workspace scan checks up to 80 first-level folders.
- Decorations supplement the icon; they do not replace the artwork dynamically.
- VS Code limits badges to two characters.
- Availability in remote and virtual workspaces depends on the file system provider.

Code output

- Output uses shipped artwork; Explorer appearance settings are not applied to it.
- Insertion adds no surrounding quotes and does not save the file.
- Check the insertion context and add meaningful alternative text where needed.

Licence

Free to install and use under the Auric Artisan Free Use Licence. It is not an open-source or unrestricted asset licence: redistribution, resale, republishing and extraction of bundled artwork are restricted. Inserting code does not change those terms.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Privacy

What Auric Artisan File Icons reads, stores and writes, and confirmation that it has no telemetry.

- **No account or telemetry.** The extension does not automatically contact a server, run shell commands or execute workspace code. Website links open only when you select them.
- **Bounded inspection.** Smart detection reads up to 250 direct folder entries and text files up to 64 KiB by default, caches results for 15 seconds, and skips excluded, binary, oversized and unavailable files.
- **Remote-aware access.** Inspection goes through VS Code's file system interface, which covers local, remote and virtual resources. What can be read or written depends on the provider.
- **Explicit edits.** Code is inserted only after your action, and hover actions check that the document has not changed.

What is stored

Data	Where
Appearance settings	Your VS Code settings under `auricFileIcons.*` (user level when set from the Studio).
Associations from the Explorer, smart style choices and scan results	The workspace settings.
Saved profiles	VS Code's extension storage on this machine.
Rebuilt theme files after customisation	Inside the extension's own folder; the shipped artwork is not modified.

Diagnostics

Show Icon Theme Diagnostics writes counts and settings to a local output channel. It omits filenames and file contents and sends nothing anywhere.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Release notes

User-facing history of Auric Artisan File Icons.

0.1.1 — 22 September 2026

The current Marketplace version.

Modern joins File Icon Studio

- Modern added as a third Explorer style, keeping its theme ID, its 464 SVG assets, light alternatives and four framework packs.
- 219 Modern file designs and 114 folder pairs in the Studio, sidebar, search, code inspector and hover insertion.
- Shared appearance controls now cover Modern, including three-digit hex colours.
- A confirmed, scope-preserving import of Modern's file settings that does not change its general library.

Usability

- A new README with a quick start, style comparison, screenshots, insertion guides, migration steps and troubleshooting.
- The Explorer activation button now shows **Modern** when Modern artwork is selected.

0.1.0 — 19 September 2026

The unified File Icon Studio, starting a new version series.

- The separate Playful project joined this extension, keeping the Facet and Playful theme IDs and all 500 Playful SVGs.
- Shared appearance settings, associations, smart decorations and profile tools across styles.
- A confirmed, non-overwriting import of old Playful settings and exported Playful profiles.
- Matching editor-tab and sidebar workspaces with **Library**, **Appearance** and **Workspace** sections, file and folder filters, open-folder previews and progressive loading.

- Card actions, a source inspector, clipboard export and multi-cursor insertion as SVG, JSX, HTML image and CSS background.
- Code hover actions for complete `:icon-file-…:` and `:icon-folder-…:` tokens.
- Explorer actions grouped in one **Auric File Icons** submenu, a native quick insert and a website link.

Earlier Facet releases

Facet 1.1.0 — 2 August 2026

- Facet expanded to 500 SVGs: 200 file icons and 150 folder pairs.
- Coverage for 270 extensions, 453 filenames, 84 language IDs and 356 folder aliases.
- Independent file and folder tinting and brightness.
- Six built-in profiles and saved personal profiles, with clipboard export and import.
- One-command detected folder icons, a cancellable workspace scan, diagnostics and more smart classifiers.

Facet 1.0.0 — 1 August 2026

- The original Facet artwork (361 SVGs), workspace-root icons, dedicated Auric Artisan workspace mappings, live filename previews, associations, folder tint, saturation, opacity, arrows and cached smart decorations with custom rules.

VS CODE EXTENSIONS › AURIC ARTISAN FILE ICONS

Related extensions and comparison

How File Icons compares with Icon Studio, Icon Hub, the Auric icon collections, Emoji and the Icon Pack.

At a glance

Extension	Main job	Explorer file icon theme	Artwork	Network
File Icons	Explorer file and folder icons in three styles, plus File Icon Studio.	Yes: Facet, Playful and Modern	1,464 SVG assets	None
Auric Icons: Modern	Modern icon library plus the Modern file and folder icon theme.	Yes: **Auric Icons: Modern**	720 library icons; 408 theme icons	None
Auric Icons: Drawn	Warm Drawn icon library.	No	480 icons	None
Auric Icons: Playful	Emoji-style icon library.	No	1,225 icons	None
Icon Studio	One offline studio for every Auric collection, emoji and seven open-source sets.	No	17,234 entries in 11 collections	None
Icon Hub	Seven open-source sets with colour and stroke controls, sprite and ZIP export, and CDN insertion.	No	12,895 icons	Optional metadata checks; CDN output loads from the web
Emoji	Emoji Studio.	No	1,914 emoji	None
Icon Pack	Installs Modern, Playful, Drawn and Emoji together.	Through Modern	None of its own	Needs the Marketplace to install

Which should I install?

- **Change the Explorer's file and folder icons:** File Icons (three styles, smart detection, profiles) or Auric Icons: Modern (the Modern theme with its library).
- **Insert general-purpose icons into code:** Icon Studio, Icon Hub, or one of the Auric collections.
- **Insert file-type artwork into code:** File Icons' Library.

Icon Studio and the Icon Pack do not include File Icons, and installing them does not change your Explorer.

How the Auric collections merge

Auric Icons: Modern, Drawn and Playful share one library view when co-installed, with a **Library** filter and collection badges. Icon Studio hosts all three by default. This merging does not involve File Icons. See The merged Icon Studio.

Other related products

- Auric Artisan Theme — colour themes that pair with the icons.
- Install and set up — installing and managing Auric extensions.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Auric Artisan Icon Pack

What the Auric Artisan Icon Pack is, what it installs, how to get it and where to find each guide.

Auric Artisan Icon Pack is a standard VS Code **extension pack**: a single Marketplace entry that installs four Auric Artisan extensions together. It has no interface, commands or settings of its own.

Installing the pack adds **Auric Icons: Modern**, **Auric Icons: Playful**, **Auric Artisan Emoji** and **Auric Icons: Drawn**. Once installed, Modern, Drawn and Playful share one icon library view, and Emoji keeps its own Emoji Studio. Each member keeps its own version, settings, licence and update cycle, and you can manage, disable or remove each one separately in the Extensions view.

The pack does not include **Auric Artisan Icon Studio** (the all-in-one catalog) or **Auric Artisan File Icons** (Explorer file and folder themes).

Who it is for

- Anyone who wants the four Auric creative libraries in one step.
- Teams who want to recommend one install to colleagues.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-icon-pack
Extension ID	`auric-artisan.auric-artisan-icon-pack`
Current version	0.1.0
Requires	Visual Studio Code 1.85 or later, and access to the extension marketplace
Price	Free

```
code --install-extension auric-artisan.auric-artisan-icon-pack
```

Quick start

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Search for **Auric Artisan Icon Pack** and select **Install**.
3. Let VS Code install the four included extensions.
4. Select an Auric icon library view or **Auric Emoji** in the Activity Bar, or type "Auric" in the Command Palette.
5. Search for artwork and use the chosen extension's insert or copy actions.

What you can do

- Install Auric Icons: Modern, Playful and Drawn and Auric Artisan Emoji together.
- Browse Modern, Drawn and Playful in one merged library.
- Manage each included extension individually afterwards.

In this folder

docs/

- Contents of docs/
- Getting started
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Auric Artisan Icon Pack guides

Contents of the Auric Artisan Icon Pack guides folder.

The Icon Pack only installs other extensions, so its guides are short. For how to use each library, follow the links to the included extensions.

File	What it covers
getting-started.md	Install the pack, check the four members and open them.
reference.md	The included extensions, requirements and what the pack does and does not contain.
troubleshooting.md	Members that did not install, alternative marketplaces and local builds.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Getting started with the Icon Pack

Install the Icon Pack, confirm its four member extensions and start using them.

Before you begin

- You need Visual Studio Code 1.85 or later.
- You need access to the extension marketplace. The pack downloads its members from it.

1. Install the pack

1. Open the Extensions view (`Ctrl` + `Shift` + `X`).
2. Search for **Auric Artisan Icon Pack** and select **Install**.

Or run `code --install-extension auric-artisan.auric-artisan-icon-pack` in a terminal, or use **Extensions: Install from VSIX…** with an official pack download.

2. Check the members

1. Select the pack in the Extensions view.
2. Open its **Extension Pack** section. It lists:
   - **Auric Icons: Modern** (`auric-artisan.auric-artisan-modern-icon`)
   - **Auric Icons: Playful** (`auric-artisan.auric-icons-playful`)
   - **Auric Artisan Emoji** (`auric-artisan.auric-artisan-emoji`)
   - **Auric Icons: Drawn** (`auric-artisan.auric-icons-drawn`)
3. Each should show as installed.

3. Start using them

To	Do this
Browse Modern, Drawn and Playful icons	Select the Auric icon library in the Activity Bar: one merged library with a **Library** filter.
Insert an icon from the keyboard	Press `Ctrl` + `Alt` + `I` in a text editor.
Insert an emoji	Press `Ctrl` + `Alt` + `J`, or open **Auric Emoji** in the Activity Bar.
Change your Explorer icons	Run **Preferences: File Icon Theme** and choose **Auric Icons: Modern**.

How the three icon libraries share one view is explained in The merged Icon Studio.

Result

The four Auric creative libraries are installed and ready. Each member's own documentation explains its features:

- Auric Icons: Modern
- Auric Icons: Playful
- Auric Artisan Emoji
- Auric Icons: Drawn

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Icon Pack reference

The extensions included in Auric Artisan Icon Pack 0.1.0, its requirements and what it does and does not contain.

This reference describes version 0.1.0 as published on the Visual Studio Marketplace.

Included extensions

Extension	ID	What it adds
Auric Icons: Modern	`auric-artisan.auric-artisan-modern-icon`	720 Modern and Sharp library icons, plus the Modern Explorer file icon theme.
Auric Icons: Playful	`auric-artisan.auric-icons-playful`	1,225 emoji-style icons in Color, Outline, Filled and Small.
Auric Artisan Emoji	`auric-artisan.auric-artisan-emoji`	Emoji Studio: 1,914 emoji, skin tones, shortcodes and editor insertion.
Auric Icons: Drawn	`auric-artisan.auric-icons-drawn`	480 hand-drawn icons in Color, Mono, Outline and Duotone.

The pack uses the current publishing IDs. It does not install the retired `auric-artisan-icons` or `auric-artisan-svg-icons` extensions.

What the pack itself contains

Item	Present
Commands	None
Settings	None

Item	Present
Views or keybindings	None
Background service or installer	None
Telemetry or workspace edits	None
Embedded copies of member extensions	No; members are downloaded from the marketplace

Requirements

Requirement	Value
VS Code	1.85 or later
Network	Marketplace access to download members
Category	Extension Packs

Not included

Extension	Why you might want it
Auric Artisan Icon Studio	The separate all-in-one catalog: the three Auric collections, emoji and seven open-source sets.
Auric Artisan File Icons	Explorer file and folder themes in Facet, Playful and Modern styles.
Auric Artisan Icon Hub	Seven open-source sets with colour controls and export.

Licence

The pack is covered by the Auric Artisan Free Use Licence. Each included extension and its artwork keep their own licence and attribution. Installing the pack grants no extra rights to redistribute artwork.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Troubleshooting

What to do when a pack member did not install, or when using alternative marketplaces or local builds.

Symptom	Why it happens	What to do
An included extension did not install.	A network problem, an organisation policy, or an older VS Code version blocked it.	Open the pack's **Extension Pack** section, find the missing member and check its installation error. Verify marketplace access and that VS Code is 1.85 or later.
I installed the pack VSIX offline and the members are missing.	A pack VSIX is not an offline bundle; it only lists the members.	Connect to the marketplace, or install each member's own VSIX.
I use an alternative marketplace and members are missing.	Each member must be published there under the same ID.	Check whether each member is available in your marketplace. Installing the pack does not move extensions between registries.

Symptom	Why it happens	What to do
I installed local development builds and the pack reinstalls members.	VS Code identifies extensions by publisher and name, not folder name.	Confirm your builds use the exact IDs listed in the reference.
I only see one icon library view, not three.	Modern, Drawn and Playful merge into one library when co-installed.	This is expected. Use the **Library** filter to pick a collection.
An icon library view disappeared after installing Icon Studio.	Icon Studio hosts the Auric collections by default.	Set `auricIconStudio.hostFamilyLibraries` to `false` to show their views.
I want to remove one member.	Members are ordinary extensions.	Uninstall or disable it in the Extensions view.

For problems inside a member extension, see that extension's troubleshooting page.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Frequently asked questions

Short answers to common questions about Auric Artisan Icon Pack.

Is the pack another icon browser?

No. It only tells VS Code which four extensions to install. It adds no commands, views or settings.

Does it work offline?

The members work offline once installed. Installing the pack needs marketplace access.

Does it include Icon Studio or File Icons?

No. Install them separately if you need them. See the reference.

Do the members update with the pack?

Each member updates on its own schedule through the marketplace.

Can I uninstall one member?

Yes. Manage each member individually in the Extensions view.

Why are Modern, Drawn and Playful shown as one library?

They are designed to merge into one library when installed together. See The merged Icon Studio.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Icon Pack: other information

Contents of the supporting pages for Auric Artisan Icon Pack.

Supporting pages for the Auric Artisan Icon Pack. The pack has no commands or keyboard shortcuts, so there is no shortcuts page; see each member extension for its shortcuts.

File	What it covers
glossary.md	Terms used on these pages.
limits-and-accuracy.md	What the pack is and is not.
related.md	Comparison of all Auric icon extensions.
release-notes.md	User-facing history.
privacy.md	What the pack does with your data.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Glossary

Terms used in the Auric Artisan Icon Pack documentation.

Term	Meaning
Extension pack	A VS Code extension whose only job is to install a list of other extensions.
Member	One of the extensions an extension pack installs.
Extension ID	The unique publisher-and-name identifier of an extension, such as `auric-artisan.auric-icons-drawn`.
Marketplace	The online registry VS Code installs extensions from.
VSIX	A packaged extension file that can be installed with **Extensions: Install from VSIX...**.
Merged library	The single icon library view shown when Modern, Drawn and Playful are installed together.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Limits and accuracy

What the Auric Artisan Icon Pack is and what it does not do.

- The pack is metadata only. It does not embed its members' VSIX files, duplicate their artwork or run an installer.
- A pack VSIX is not an offline bundle; installing needs marketplace access.
- If a member is unavailable in your marketplace or blocked by policy, the pack cannot install it.
- The exact interface you get depends on the member versions VS Code installs.
- Icon Studio and File Icons are not included.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Privacy

What the Auric Artisan Icon Pack does with your data.

The pack itself runs no code. It adds no telemetry, background service or workspace edits, and stores nothing.

VS Code contacts the extension marketplace to download the four members, as it does for any extension.

Each member has its own privacy behaviour:

- Auric Icons: Modern — Privacy
- Auric Icons: Playful — Privacy
- Auric Artisan Emoji — Privacy
- Auric Icons: Drawn — Privacy

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Release notes

User-facing history of Auric Artisan Icon Pack.

0.1.0 — 22 September 2026

The first Auric Artisan Icon Pack, and the current Marketplace version.

- Groups Auric Icons: Modern, Auric Icons: Playful, Auric Artisan Emoji and Auric Icons: Drawn into one standard VS Code extension pack.
- Uses the active Playful and Drawn extension IDs instead of retired ones.
- Adds a gold Auric bundle icon, installation guidance and marketplace troubleshooting.
- Contains metadata only: no runtime code, background service or custom downloader.

VS CODE EXTENSIONS › AURIC ARTISAN ICON PACK

Related extensions and comparison

How the Icon Pack's members compare with Icon Studio, Icon Hub and File Icons.

Extension	In the pack	Main job	Explorer file icon theme	Artwork	Network
Auric Icons: Modern	Yes	Modern icon library plus the Modern file icon theme.	Yes	720 library icons; 408 theme icons	None
Auric Icons: Playful	Yes	Emoji-style icon library.	No	1,225 icons	None
Auric Icons: Drawn	Yes	Warm Drawn icon library.	No	480 icons	None
Auric Artisan Emoji	Yes	Emoji Studio.	No	1,914 emoji	None
Icon Studio	No	One offline studio for every Auric collection, emoji and seven open-source sets.	No	17,234 entries	None
Icon Hub	No	Seven open-source sets with colour controls, sprite and ZIP export, CDN insertion.	No	12,895 icons	Optional metadata checks
File Icons	No	Explorer file and folder icons in three styles.	Yes	1,464 SVG assets	None

How the pack's icon libraries merge

With Modern, Drawn and Playful installed together, one of them hosts a single library with a **Library** filter and collection badges; the others hide their views. If you later install Icon Studio, it hosts all three by default. See The merged Icon Studio.

See also Install and set up.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Auric Icons: Modern

What the Auric Icons: Modern extension is, who it is for, how to install it and where to find each guide.

Auric Icons: Modern is two things in one VS Code extension. First, it is an original library of 720 modern SVG icons in two families, **Simple Modern** and **Sharp Modern**, each available in **Color**, **Mono**, **Outline** and **Duotone** variants. You search the library in an **Icon Library** view and insert icons into your code in a format that suits the file.

Second, it is a file and folder icon theme for the VS Code Explorer, also called **Auric Icons: Modern**. The theme has 408 icons with a distinct file silhouette, true open and closed folder pairs, semantic colours, four framework packs (Angular, NestJS, React and Redux, Vue and Pinia) and custom associations. You can re-tint folders, adjust saturation and opacity, and hide the Explorer arrows.

Modern can be installed alone or alongside its sibling extensions **Auric Icons: Drawn** and **Auric Icons: Playful**. When they are installed together they show one shared library, labelled **Icon Studio**, with a **Library** filter and a badge on each card showing where it comes from.

Who it is for

- Developers who want a crisp, consistent Explorer icon theme that works at the 16-pixel Explorer size.
- Front-end developers and designers who insert SVG icons into components, pages, stylesheets and documentation.
- Teams using Angular, NestJS, React or Vue who want framework-specific file icons.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-modern-icon
Extension ID	`auric-artisan.auric-artisan-modern-icon`
Current version	0.12.1
Requires	Visual Studio Code 1.85 or later, or a compatible editor built on it
Price	Free to install and use

```
code --install-extension auric-artisan.auric-artisan-modern-icon
```

Modern is also installed by the Auric Artisan Icon Pack.

Quick start

1. Install the extension.
2. Run **Preferences: File Icon Theme** and choose **Auric Icons: Modern**, or run **Auric Icons: Modern: Activate Icon Theme**. Your Explorer now uses Modern icons.
3. Run **Auric Icons: Modern: Configure…** to choose a folder colour or a framework pack.
4. Open a file you are working on and place the cursor where an icon should go.
5. Select **Auric Icons: Modern** in the Activity Bar to open the **Icon Library**.
6. Search for an icon, for example `chart`.

7. Select **Insert** on a card, or press `Ctrl` + `Alt` + `I` in the editor to pick an icon from a list.

What you can do

- Use a 408-icon Explorer theme covering 265 file extensions, 462 exact filenames and 265 folder names.
- Re-tint folders with a preset or any hex colour, without changing the semantic colours of file icons.
- Layer one framework pack over the base icons: Angular, NestJS, React and Redux, or Vue and Pinia.
- Map your own file extensions, filenames and folder names to icons.
- Search 720 library icons by name, concept or synonym, filter by category and style, and preview on light or dark surfaces.
- Insert with language-aware output, copy in nine formats, save an SVG file or open its source.
- Keep likes and recent icons, and see recommendations for the file you are editing.
- Hover over an icon ID in code to preview it.
- Merge with Drawn and Playful into one library when co-installed.

In this folder

docs/

- Contents of docs/
- Getting started
- Use the file icon theme
- Browse the Icon Library
- Insert, copy and save icons
- The merged Icon Studio
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Auric Icons: Modern guides

Contents of the Auric Icons: Modern guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Icons: Modern. The extension has two parts, the Explorer file icon theme and the Icon Library, and each has its own guide.

File	What it covers
getting-started.md	Install, turn on the file icon theme, open the Icon Library and insert a first icon.
file-icon-theme.md	Folder colour, framework packs, saturation, opacity, arrows, associations and workspace detection.
icon-library.md	Search, filters, variants, preview surfaces, the smart guide and recommendations.
insert-copy-and-save.md	Insert and smart insert, nine copy formats, save, open source and hover previews.
merged-icon-studio.md	How Modern, Drawn, Playful and Icon Studio share one library.
reference.md	Every command, setting, keybinding, icon ID and control.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Getting started with Auric Icons: Modern

Install Auric Icons: Modern, turn on its file icon theme, open the Icon Library and insert your first icon.

Before you begin

- You need Visual Studio Code 1.85 or later, or an editor that keeps VS Code's extension and icon theme features.
- Everything works offline. The extension has no telemetry and makes no network requests.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Icons: Modern** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-artisan-modern-icon
```

2. Turn on the file icon theme

1. Open the Command Palette (`Ctrl` + `Shift` + `P`).
2. Run **Preferences: File Icon Theme**.
3. Choose **Auric Icons: Modern**.

Or run **Auric Icons: Modern: Activate Icon Theme**. The Explorer now shows Modern file and folder icons. Folders use the Auric gold by default, and open folders have their own open artwork.

> **Note:** If you also have Auric Artisan File Icons installed, it offers a Modern style under the same theme ID. See Troubleshooting.

3. Personalise the theme

1. Run **Auric Icons: Modern: Configure...**.
2. Choose **Folder colour** and pick a colour, for example **Teal**.
3. VS Code shows "Auric Icons: Modern updated — … Reload to apply?". Select **Reload Window**.

See Use the file icon theme for every option.

4. Open the Icon Library

1. Select **Auric Icons: Modern** in the Activity Bar. The **Icon Library** view opens.
2. Or run **Auric Icons: Modern: Open Icon Library** to open the larger **Icon Studio — Library** editor tab.

If Auric Artisan Icon Studio, or another Auric collection that is acting as host, is installed, this view may be hidden. See The merged Icon Studio.

5. Insert your first icon

1. Open a file, such as a React component, and place the cursor.
2. In the Icon Library, type `chart` in **Search**.
3. Select **Insert** on a card. With smart insert on (the default), the icon arrives in a format that suits the file, such as a JSX element in React.

Or stay in the editor: press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS), type to search, and choose an icon.

Result

Your Explorer uses the Modern icon theme in your colour, and you have inserted a Modern icon into your code.

Next steps

- Browse the Icon Library
- Insert, copy and save icons

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Use the file icon theme

Activate the Modern Explorer icon theme and set folder colour, framework packs, saturation, opacity, arrows, custom associations and workspace detection.

A **file icon theme** decides which icon VS Code shows beside each file and folder in the Explorer, tabs and other lists. The Modern theme has 408 icons designed for the 16-pixel Explorer size. It maps 265 file extensions, 462 exact filenames and 265 folder names, with separate open and closed artwork for folders and light-theme alternatives for monochrome marks.

Activate the theme

- Run **Preferences: File Icon Theme** and choose **Auric Icons: Modern**, or
- Run **Auric Icons: Modern: Activate Icon Theme**.

How changes are applied

VS Code reads an icon theme once and caches it. When you change a Modern setting, the extension rebuilds its theme files. If the theme is active, you see "Auric Icons: Modern updated — {summary}. Reload to apply?". Select **Reload Window** to see the change, or **Later** to keep working. The original artwork is never modified, so any change can be undone by restoring defaults.

A default install writes nothing and asks for no reload. Settings changed while VS Code was closed, or synced from another machine, are applied at start-up.

The Configure menu

Run **Auric Icons: Modern: Configure...**. The menu shows the current value of each item.

Item	What it does
Folder colour	Opens the folder colour list (below).
Icon pack	Opens the framework pack list (below).
Explorer arrows	Toggles between **visible** and **hidden**.
Saturation	Choose 0, 0.25, 0.5, 0.75, 1, 1.25 or 1.5.
Opacity	Choose 0.4, 0.5, 0.6, 0.7, 0.8, 0.9 or 1.
Detect Auric Artisan workspace	Reports whether this workspace is an Auric Artisan workspace.
Restore defaults	Resets every Modern theme setting.

Folder colour

Run **Auric Icons: Modern: Select Folder Colour** (or choose **Folder colour** in Configure).

Option	Value
Auric gold	`default` (the house accent)
Blue	`#3178C6`
Teal	`#2E9E8F`
Green	`#4C9A55`
Violet	`#7B62D4`
Magenta	`#B45FC4`
Crimson	`#B8405A`
Slate	`#6B7684` (neutral)

Option	Value
Custom hex…	Enter your own, such as `#D8A33C`. Three- or six-digit hex.

Folder colour re-tints **folders only**. File icons keep the semantic colours that identify them.

Framework packs

Run **Auric Icons: Modern: Select Icon Pack**. Only one pack can be active at a time; it layers over the base set.

Pack	Adds icons for	Icons
None	Base icon set only	0
Angular	component, module, service, pipe, directive, guard, resolver, interceptor	8
NestJS	controller, service, module, guard, middleware, decorator, DTO, entity, filter	9
React / Redux	hook, context, provider, reducer, slice, store	6
Vue / Pinia	composable, store, plugin	3

The setting is `auricIcons.activeIconPack` (`none`, `angular`, `nest`, `react`, `vue`).

Saturation, opacity and arrows

Setting	Range	Default	Effect
`auricIcons.saturation`	0 to 2	1	0 is greyscale; 1 is unmodified. Hue and lightness are kept, so icons stay distinguishable by shape.
`auricIcons.opacity`	0.1 to 1	1	Lower values let icons recede behind filenames.
`auricIcons.hidesExplorerArrows`	true or false	false	Hides the expand and collapse chevrons. Also toggled by **Toggle Explorer Arrows**.

The Configure menu offers preset steps; type any value in the range into Settings for finer control.

Custom associations

Map your own names to Modern icons in `settings.json`.

```
{
  "auricIcons.files.associations": {
    "*.myext": "typescript",
    "config.local": "ini",
    "*.log": "none"
  },
  "auricIcons.folders.associations": {
    "internal": "src",
    "shared": "lib"
  }
}
```

- File keys are either `*.ext` (an extension) or an exact filename.
- Folder keys are folder names. The open and closed icons are applied together.

- Values are icon IDs. The value `none` removes an association the theme would otherwise apply.
- Your associations take precedence over the theme and any pack.

The full lists of file and folder icon IDs are in the reference.

Auric Artisan workspace icons

Files and folders used by Auric Artisan tools, such as `.auric-artisan/`, `swatches/`, `.auricignore`, `scan-cache.json`, `health.json`, `health-history.json`, `*.sarif`, `ai-remediation-plan.*` and `project-health.*`, have dedicated icons. They are matched by name and need no set-up. `.auric-artisan/settings.json` deliberately gets no special icon, so `.vscode/settings.json` is not affected.

Workspace detection

With `auricIcons.autoDetect` on (the default), opening a local workspace that contains `.auric-artisan/` or `.auricignore` while another icon theme is active shows a one-time offer: **Use Auric Icons: Modern**, **Not now** or **Don't ask again**. It asks at most once per workspace and never when Modern is already active.

Run **Auric Icons: Modern: Detect Auric Artisan Workspace** to check manually. It reports where it found the marker, or that none was found.

Restore defaults

Run **Auric Icons: Modern: Restore Default Settings**. It clears folder colour, saturation, opacity, arrows, icon pack, workspace detection and both association settings. VS Code confirms "settings restored to defaults".

Result

Your Explorer shows Modern icons with your folder colour, pack and associations after a window reload.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Browse the Icon Library

Search the 720-icon Modern library, filter by category, style and library, switch variants and preview surfaces, and use the smart guide and recommendations.

The Icon Library holds 720 original icons: 420 **Simple Modern** and 300 **Sharp Modern**, across 33 categories such as Coding and development, Data and databases, Cybersecurity, Accessibility and Space and Astronomy. Every icon comes in four variants: **Color**, **Mono**, **Outline** and **Duotone**.

Open the library

- **Sidebar:** select **Auric Icons: Modern** in the Activity Bar to open the **Icon Library** view.
- **Editor tab:** run **Auric Icons: Modern: Open Icon Library**, select the ↗ button (**Open full explorer**) in the view, or select the open-preview icon in the view's title bar. The tab is called **Icon Studio — Library**.

The screen

Area	What it does
Header	"Icon Studio", **Icon Library**, a count line such as "720 icons · 72 loaded", and the ↗ button.
Smart guide	Up to four tip cards, such as "TypeScript detected" or "Like icons you reuse". Dismiss a tip with its × button. Turn the guide off with `auricIcons.library.showGuide`.
Search	Placeholder "Search 720 icons — try “empty state” or “auth”".
Library	Only shown when several collections are merged. **All libraries** or one collection.
Category	**All categories** or one category.
Style	**All styles**, **Modern** or **Sharp**.
Variant	**Auto**, **Color**, **Mono**, **Outline**, **Duotone**.
Preview surface	**Auto**, **Light**, **Dark**.
Chips	**All**, ✦ **For you**, ♥ **Likes** (count) and ↺ **Recent** (count).
Recommended rail	✦ **Recommended for this file**, with **Refresh**.
Grid	Icon cards that load as you scroll.

Search

Type a name, concept or synonym; results are ranked as you type. For example, `delete` finds trash icons. Press `/` to focus the search from anywhere in the view, and `Escape` to clear it. Search is also personalised by what you like and use.

Variants and Auto

The **Variant** buttons change the artwork shown on every card and used for insert, copy and save.

Auto follows the file you are editing. It shows the resolved variant, for example "Auto · outline".

Editing in	Auto variant
JavaScript, TypeScript, React JSX and TSX	Outline
HTML, PHP, Handlebars, Vue, Svelte, Astro, Markdown, MDX, XML, SVG	Color
CSS, SCSS, Less, JSON, YAML, Python	Mono
Anything else	Color

The default variant for Command Palette actions is set by `auricIcons.library.defaultVariant` (`auto`, `color`, `mono`, `outline` or `duotone`).

Preview surface

Light and **Dark** show every card on a light or dark backdrop, so you can judge the artwork for either kind of page. **Auto** follows your VS Code theme. The view remembers your choice.

The views

Chip	Shows
All	Every icon that matches the search and filters.
✦ For you	Icons recommended for the file you are editing and your likes and use.
♥ Likes	Icons you liked with the heart.
↺ Recent	Icons you inserted, copied, saved or opened, newest first (up to 60).

Recommendations

The **✦ Recommended for this file** rail shows icons that match words in the file you are editing, icons you like or used recently, and icons that fit your style. Each card shows why, for example **Matches this file**, **In your likes**, **Used recently**, **Fits your style** or **Popular in your catalog**. Selecting a rail card inserts it. Select **Refresh** to recompute.

Setting	Default	Effect
`auricIcons.library.recommendations.enabled`	true	Show recommendations.
`auricIcons.library.recommendations.limit`	12	How many icons the rail shows (4 to 24).

Cards

Each card shows the artwork, a heart, the name and the category (and a collection badge when libraries are merged).

Control	Action
Artwork	Opens the SVG source in an editor tab.
Heart	Adds to or removes from **Likes**.
Insert	Inserts into the editor.
Copy	Copies raw SVG.
Save	Saves an `.svg` file through a dialog.
JSX	Copies as a JSX element.
URI	Copies as a data URI.
Source	Opens the SVG source.

Density and batch size

Setting	Values	Default
`auricIcons.library.density`	`comfortable`, `compact`	`comfortable`
`auricIcons.library.pageSize`	24 to 240	72

The grid streams more cards as you scroll, with a line such as "144 of 720 — keep scrolling".

Result

You can find any Modern icon quickly, preview it in the right variant on the right background, and act on it from its card.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Insert, copy and save icons

Insert Modern icons with language-aware output, copy them in nine formats, save SVG files, open the source and preview icon IDs on hover.

Insert from the Icon Library

1. Open a file in a text editor and place one or more cursors, or select text to replace.
2. In the Icon Library, select **Insert** on a card.
3. The icon is inserted at every cursor. Multi-line output keeps the indentation of the cursor's line.

If no text editor is open, you see "Open a text editor before inserting an SVG." When you work in the full editor-tab library, the icon goes into the first visible file editor.

Smart insert

With `auricIcons.library.smartInsert` on (the default), **Insert** adapts to the language of the file:

Editing in	Output
JavaScript, TypeScript, React JSX and TSX, MDX	JSX element
HTML, PHP, Handlebars, Vue, Svelte, Astro	HTML markup
Markdown	Markdown image
CSS, SCSS, Less	CSS background
JSON, JSONC, YAML, Python	Data URI
XML, SVG and any other language	Raw SVG

With the setting off, **Insert** always inserts raw SVG. The **Smart Insert Icon** command always adapts, whatever the setting.

Smart insert from the keyboard

1. In a text editor, press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS), or run **Auric Icons: Modern: Smart Insert Icon (matches active language)**.
2. A list opens. With no search text, it starts with **Recommended for this file**, then **All icons**. Each row shows the category and tags (and the collection when several are merged).
3. Type to search and choose an icon. VS Code confirms, for example "… inserted for the active language."

Copy in a chosen format

Run **Auric Icons: Modern: Copy Icon As… (JSX, CSS, Markdown, Data URI, Image tag)**, choose an icon, then choose a format:

Format	What you get
Raw SVG	The SVG markup.
JSX element	The SVG with JSX attribute names.
React component	`export function IconName(props) { return (<svg {...props} …>); }`
Markdown image	`![Icon name](data:image/svg+xml,…)`
CSS background	`background-image: url("data:image/svg+xml,…");` plus `background-repeat: no-repeat;` and `background-size: contain;`
Data URI	`data:image/svg+xml,…`
Image tag (self-contained)	`<img src="data:image/svg+xml,…" alt="Icon name" width="24" height="24">`
Icon name	The icon ID, such as `analytics-bar-chart-0042`.
Catalog path	The icon's relative path in the catalog.

Smart insert can also produce **HTML markup**, which is the SVG markup itself.

On a card, **Copy** copies raw SVG, **JSX** copies a JSX element and **URI** copies a data URI.

Save an SVG file

1. Select **Save** on a card, or run **Save SVG As…** and choose an icon.
2. A save dialog opens in your first workspace folder (or your home folder) with the icon's file name.
3. Choose a location and select **Save icon**.

The saved file is the artwork in the selected variant.

Open the SVG source

Select the artwork on a card, **Source** on a card, or run **Open SVG Source**. The SVG file opens in a preview tab so you can read or copy its markup.

Other Command Palette actions

Command	What it does
Search Icons	Choose an icon, then an action: **Smart Insert (matches this file)**, **Insert SVG at Cursor**, **Copy SVG**, **Save SVG As…**, **Open SVG Source** or **Toggle Like**.
Insert SVG at Cursor	Choose an icon and insert it (smart insert applies when the setting is on).
Copy SVG	Choose an icon and copy raw SVG.
Insert from Likes	Choose from your liked icons and insert.
Insert from Recent	Choose from recent icons and insert. Also `Ctrl` + `Alt` + `R`.
Toggle Like for an Icon	Choose an icon and like or unlike it.
Clear Likes, Recents and Learned Recommendations	Asks for confirmation (**Clear**), then resets.

Hover previews

Type or find a Modern icon ID in code, such as `analytics-bar-chart-0042`, and hover over it. The hover shows a 48-pixel colour preview, the name and description, the ID, category and tags, and **Open in Icon Library**.

Hovers work in HTML, CSS, SCSS, Less, Vue, Svelte, Astro, JavaScript, TypeScript, React JSX and TSX, JSON, JSONC, Markdown, PHP, Handlebars and XML. Turn them off with `auricIcons.library.hoverPreview`. When Drawn or Playful is also installed, each extension previews only its own icons, so you never see two hovers.

Result

The icon is in your file, on your clipboard or saved as an `.svg` file, and it appears under **Recent**.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

The merged Icon Studio

How Auric Icons: Modern, Drawn and Playful merge into one Icon Studio library when co-installed, and how Auric Artisan Icon Studio takes over hosting.

Auric Icons: Modern, **Auric Icons: Drawn** and **Auric Icons: Playful** are three extensions built on the same library. Each works on its own. When you install more than one, they cooperate so you get one library instead of duplicate sidebars.

What you see with two or three collections installed

1. One extension automatically becomes the **host**. The choice is made the same way every time, so it does not change from one start-up to the next unless you install, update or remove a collection.
2. The host shows one **Icon Library** view (headed "Icon Studio") that contains every installed collection.
3. A **Library** filter appears, with **All libraries** and one entry per collection, each with its icon count.
4. Every card shows a badge naming its collection, for example **Drawn**, with the tooltip "From the Drawn library".
5. The **Variant** buttons offer every variant any installed collection provides: Color, Mono, Outline and Duotone from Modern and Drawn, plus Filled and Small from Playful. If a collection does not have the selected variant, its cards fall back to Color.
6. The other extensions hide their Activity Bar view, their Command Palette entries and their `Ctrl` + `Alt` + `I` and `Ctrl` + `Alt` + `R` shortcuts. If you run one of their commands (for example from a keybinding), it is passed to the host.

The merge updates by itself when you install, enable, disable or remove a collection. No reload is needed.

Likes, recents and recommendations

- Likes and recent icons belong to the extension that owns each icon. Liking a Drawn icon in the merged view stores it with Drawn.
- The merged view shows everyone's likes and recents together.
- If you uninstall one collection, its icons and their history leave the merged view; the other collections keep theirs.
- **Clear Likes, Recents and Learned Recommendations** in the merged view clears the history of every merged collection.

Hover previews

Each extension previews only its own icon IDs, so hovering over an ID always shows exactly one preview.

When Auric Artisan Icon Studio is installed

Auric Artisan Icon Studio bundles all three collections plus emoji and open-source sets. By default it hosts them: the Modern, Drawn and Playful library views, palette entries and shortcuts are hidden and you use Icon Studio instead.

- Modern's Explorer file icon theme is not affected.
- To bring back the collections' own library views, set `auricIconStudio.hostFamilyLibraries` to `false`. Icon Studio's shortcuts then step aside for the collections' shortcuts.
- Icon Studio keeps its own separate likes and recents.

Which extension is the host?

You do not need to know, because every command works the same whichever extension hosts. The host is the extension with the newest shared library version; if they are equal, the choice falls to a fixed alphabetical order of extension IDs. With the current Marketplace versions (all 0.12.1), the host is Auric Icons: Modern when it is installed, otherwise Auric Icons: Drawn.

Result

With several Auric collections installed, you browse all of them in one library, filter by collection when you need to, and see each icon once.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Auric Icons: Modern reference

Every command, keybinding, setting, view, icon ID, pack, category and library control in Auric Icons: Modern 0.12.1.

This reference describes version 0.12.1 as published on the Visual Studio Marketplace.

Commands

Category in the Command Palette: **Auric Icons: Modern**.

File icon theme

Command	ID	What it does
Configure…	`auricIcons.configure`	Menu for folder colour, icon pack, arrows, saturation, opacity, detection and restore.
Select Icon Pack	`auricIcons.selectIconPack`	Choose None, Angular, NestJS, React / Redux or Vue / Pinia.
Select Folder Colour	`auricIcons.selectFolderColor`	Choose a preset or custom hex folder colour.
Toggle Explorer Arrows	`auricIcons.toggleExplorerArrows`	Show or hide the Explorer chevrons.

Command	ID	What it does
Detect Auric Artisan Workspace	`auricIcons.detectWorkspace`	Report whether the workspace has `.auric-artisan/` or `.auricignore`.
Restore Default Settings	`auricIcons.restoreDefaults`	Clear all Modern theme settings.
Activate Icon Theme	`auricIcons.activate`	Set **Auric Icons: Modern** as the file icon theme.

Icon Library

Command	ID	What it does
Open Icon Library	`auricIcons.library.openExplorer`	Opens the **Icon Studio — Library** editor tab.
Search Icons	`auricIcons.library.searchIcons`	Choose an icon, then an action.
Insert SVG at Cursor	`auricIcons.library.insertSvg`	Choose an icon and insert it.
Smart Insert Icon (matches active language)	`auricIcons.library.smartInsert`	Choose an icon and insert it with language-aware output.
Copy Icon As… (JSX, CSS, Markdown, Data URI, Image tag)	`auricIcons.library.copyAs`	Choose an icon and one of nine formats.
Insert from Likes	`auricIcons.library.showFavorites`	Choose a liked icon and insert it.
Insert from Recent	`auricIcons.library.showRecents`	Choose a recent icon and insert it.
Toggle Like for an Icon	`auricIcons.library.toggleFavorite`	Like or unlike an icon.
Clear Likes, Recents and Learned Recommendations	`auricIcons.library.resetPersonalization`	Asks for confirmation, then resets personalisation.
Copy SVG	`auricIcons.library.copySvg`	Choose an icon and copy raw SVG.
Save SVG As…	`auricIcons.library.saveSvg`	Choose an icon and save it as a file.
Open SVG Source	`auricIcons.library.openSource`	Choose an icon and open its SVG.

Icon Library commands are hidden from the Command Palette while another extension hosts the shared library. They still work from keybindings and forward to the host.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command	When
`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`	Smart Insert Icon	Editor focused and Modern hosts the library
`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`	Insert from Recent	Editor focused and Modern hosts the library

In the Icon Library view: `/` focuses search; `Escape` clears it; in dropdowns, arrow keys move and `Escape` closes.

Settings

File icon theme

Setting	Type	Values or range	Default	What it does
`auricIcons.folderColor`	string	`default` or a hex colour	`default`	Folder colour. `default` is the Auric gold. Folders only.
`auricIcons.activeIconPack`	string	`none`, `angular`, `nest`, `react`, `vue`	`none`	Framework pack layered over the base set.
`auricIcons.autoDetect`	boolean	`true`, `false`	`true`	Offer the theme once in Auric Artisan workspaces.
`auricIcons.hidesExplorerArrows`	boolean	`true`, `false`	`false`	Hide Explorer chevrons.
`auricIcons.saturation`	number	0 to 2	1	Colour saturation multiplier; 0 is greyscale.
`auricIcons.opacity`	number	0.1 to 1	1	Icon opacity.
`auricIcons.files.associations`	object	`{ "*.ext" or filename: icon ID }`	`{}`	Map files to icons; `none` removes a mapping.
`auricIcons.folders.associations`	object	`{ folder name: icon ID }`	`{}`	Map folders to icons; open and closed applied together.

Icon Library

Setting	Type	Values or range	Default	What it does
`auricIcons.library.defaultVariant`	string	`auto`, `color`, `mono`, `outline`, `duotone`	`auto`	Variant for Command Palette actions; `auto` follows the file.
`auricIcons.library.pageSize`	number	24 to 240	72	Cards streamed per batch.
`auricIcons.library.smartInsert`	boolean	`true`, `false`	`true`	Language-aware output for Insert.
`auricIcons.library.recommendations.enabled`	boolean	`true`, `false`	`true`	Recommendations for the file you are editing.
`auricIcons.library.recommendations.limit`	number	4 to 24	12	Icons in the recommendation rail.
`auricIcons.library.showGuide`	boolean	`true`, `false`	`true`	Smart guide tips at the top of the library.
`auricIcons.library.density`	string	`comfortable`, `compact`	`comfortable`	Card size.
`auricIcons.library.hoverPreview`	boolean	`true`, `false`	`true`	Hover previews for icon IDs in code.

Values set under the retired `auricArtisanIcons.*` names are still honoured as fallbacks for library settings.

Views and menus

Place	Contents
Activity Bar	**Auric Icons: Modern** container with the **Icon Library** view (hidden when another extension hosts).
View title bar	**Open Icon Library**, **Smart Insert Icon**, **Insert from Likes**; **…** menu: **Clear Likes, Recents and Learned Recommendations**.
Editor tab	**Icon Studio — Library**.

Place	Contents
File icon theme	**Auric Icons: Modern** (theme ID `auric-artisan-modern-icon`).

File icon theme contents

Item	Count
Theme icons	408
File extensions mapped	265
Exact filenames mapped	462
Folder names mapped	265
Framework pack icons	26 (Angular 8, NestJS 9, React / Redux 6, Vue / Pinia 3)

Icon IDs for associations

Use these values in `auricIcons.files.associations` and `auricIcons.folders.associations`.

File icon IDs

Languages: `javascript`, `typescript`, `python`, `java`, `csharp`, `cpp`, `c`, `go`, `rust`, `php`, `ruby`, `swift`, `kotlin`, `shell`, `powershell`, `sql`, `dart`, `scala`, `elixir`, `erlang`, `haskell`, `clojure`, `fsharp`, `ocaml`, `lua`, `r`, `julia`, `perl`, `groovy`, `objectivec`, `zig`, `nim`, `crystal`, `solidity`, `assembly`, `fortran`, `vlang`, `haxe`, `prolog`, `racket`, `scheme`, `vala`, `coffeescript`, `vbnet`, `pascal`, `cobol`, `matlab`, `batch`.

Web and markup: `html`, `css`, `sass`, `less`, `stylus`, `jsx`, `tsx`, `vue`, `svelte`, `astro`, `handlebars`, `ejs`, `pug`, `liquid`, `wasm`.

Data and config: `json`, `jsonc`, `json5`, `yaml`, `toml`, `xml`, `csv`, `tsv`, `graphql`, `protobuf`, `avro`, `parquet`, `geojson`, `ini`, `env`, `tsconfig`, `jsconfig`, `package-json`, `package-lock`, `eslint`, `prettier`, `editorconfig`, `biome`, `stylelint`, `babel`, `npmrc`.

Build and packages: `webpack`, `vite`, `rollup`, `parcel`, `esbuild`, `tailwind`, `postcss`, `makefile`, `cmake`, `gradle`, `maven`, `cargo`, `gemfile`, `pyproject`, `gomod`, `composer`, `nuget`, `podfile`, `swiftpm`.

Frameworks and platforms: `nextjs`, `nuxt`, `angular`, `remix`, `gatsby`, `svelte-config`, `capacitor`, `electron`, `tauri`, `flutter`, `docker`, `docker-compose`, `kubernetes`, `helm`, `terraform`, `ansible`, `pulumi`, `vagrant`, `jenkins`, `github-actions`, `gitlab-ci`, `circleci`, `travis`, `procfile`, `firebase`, `vercel`, `netlify`, `cloudflare`, `supabase`, `aws`.

Testing and docs: `test`, `jest`, `cypress`, `playwright`, `storybook`, `coverage`, `markdown`, `mdx`, `readme`, `license`, `changelog`, `contributing`, `codeowners`, `text`, `pdf`.

Other: `git`, `sqlite`, `prisma`, `migration`, `image`, `svg`, `video`, `audio`, `font`, `model3d`, `design`, `archive`, `binary`, `executable`, `library`, `certificate`, `key`, `default`, `code`.

Auric Artisan: `auric-ignore`, `auric-scan-cache`, `auric-health`, `sarif`, `auric-history`, `auric-remediation`, `auric-report`.

Folder icon IDs

`src`, `app`, `lib`, `packages`, `modules`, `node-modules`, `components`, `pages`, `routes`, `views`, `layouts`, `templates`, `controllers`, `models`, `services`, `repositories`, `middleware`, `hooks`, `store`, `types`, `schemas`, `graphql`, `api`, `server`, `client`, `utils`, `assets`, `public`, `images`, `icons`, `fonts`, `audio`, `video`, `styles`, `scripts`, `config`, `env`, `database`, `migrations`, `seeds`, `test`, `mocks`, `coverage`, `storybook`, `benchmark`, `docs`, `examples`, `build`, `dist`, `cache`, `temp`, `logs`, `generated`, `git`, `github`, `gitlab`, `ci`, `docker`, `kubernetes`, `helm`, `terraform`, `cloud`, `aws`, `azure`, `gcp`, `firebase`, `vercel`, `netlify`, `auric-artisan`, `swatches`, `security`, `certificates`, `i18n`, `mobile`, `android`, `ios`, `desktop`, `electron`, `javascript`, `typescript`, `python`, `rust`, `java`, `go`, `react`, `vue`, `angular`, `svelte`, `next`, `nuxt`, `wasm`, `default`, `root`.

Icon Library contents

Item	Value
Icons	720 (420 Simple Modern, 300 Sharp Modern)
Variants	Color, Mono, Outline, Duotone
Categories	33
Unique concepts	520

Categories: Files and document types; Folders and organization; Coding and development; Artificial intelligence; Data and databases; Cloud and hosting; Cybersecurity; Communication; Social interaction; User accounts and profiles; Business and finance; E-commerce; Marketing; Gaming and Entertainment; Sports and Fitness; Shopping and Products; Devices and Hardware; Internet and Networking; Settings and Controls; Alerts and Notifications; Actions and Gestures; Status and Feedback; Accessibility; Time and History; Search and Discovery; Relationships and Community; Legal and Administration; Logistics and Delivery; Agriculture and Gardening; Space and Astronomy; Holidays and Celebrations; Abstract Concepts; Experimental and Decorative Symbols.

Library controls

Control	Options
Search	Free text, synonyms, fuzzy ranking.
Library	**All libraries** or one collection (only when merged).
Category	**All categories** or one of 33.
Style	**All styles**, **Modern**, **Sharp** (plus other collections' styles when merged).
Variant	**Auto**, **Color**, **Mono**, **Outline**, **Duotone** (plus **Filled** and **Small** when Playful is merged).
Preview surface	**Auto**, **Light**, **Dark**.
Chips	**All**, ✦ **For you**, ♥ **Likes**, ↺ **Recent**.
Card buttons	Artwork (open source), heart, **Insert**, **Copy**, **Save**, **JSX**, **URI**, **Source**.
Rail	✦ **Recommended for this file** with **Refresh**; selecting a rail card inserts it.

Limits

Item	Limit
Recent icons	60
Liked icons	500
Quick Pick results	200 with a search term; 400 without
Smart guide tips shown	Up to 4

Messages you may see

Message	Meaning
Auric Icons: Modern updated — … Reload to apply?	Theme settings changed; reload to see them.
Auric Icons: Modern: could not apply customisation — … The default icons remain active.	A setting could not be applied; the original icons stay in place.
Open a text editor before inserting an SVG.	No text editor was available for insertion.
… like an icon first — open the library and press the heart.	**Insert from Likes** found no likes.
… no recent icons yet.	**Insert from Recent** found none.
This workspace uses Auric Artisan (found …).	Workspace detection offer.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Troubleshooting

Symptoms, causes and fixes for common problems with the Modern file icon theme and Icon Library.

File icon theme

Symptom	Why it happens	What to do
A setting change does not show in the Explorer.	VS Code caches icon themes until the window reloads.	Select **Reload Window** in the prompt, or run **Developer: Reload Window**.
No reload prompt appears.	The prompt only appears when Modern is the active file icon theme.	Activate the theme first with **Activate Icon Theme**.
"could not apply customisation — … The default icons remain active."	A value could not be applied, for example an invalid colour.	Check the value in Settings; folder colour must be `default` or a three- or six-digit hex colour.
A custom association shows a blank icon.	The value is not one of the theme's icon IDs.	Use an ID from the reference.
A workspace setting does not seem to apply.	The extension's own commands write user-level settings; a workspace value may override them.	Check the workspace `settings.json` for `auricIcons.*` values.
The file icon theme picker lists two Modern entries.	Auric Artisan File Icons also contributes a Modern style (**Auric Artisan File Icons: Modern**) with the same theme ID.	Choose one extension to manage Modern and use its settings. See the note below.

Symptom	Why it happens	What to do
The workspace detection offer never appears.	It only checks local folders, asks once per workspace, and does not ask when Modern is already active or `auricIcons.autoDetect` is off.	Run **Detect Auric Artisan Workspace**, or activate the theme directly.

Note: With both Auric Icons: Modern and Auric Artisan File Icons installed, use either `auricIcons.*` settings (this extension) or File Icon Studio (`auricFileIcons.*`) consistently. If icons look inconsistent, disable one of the two sources of the Modern theme. See Modern and older Playful settings.

Icon Library

Symptom	Why it happens	What to do
The **Auric Icons: Modern** Activity Bar view is missing.	Another extension hosts the shared library: Auric Artisan Icon Studio, or a sibling collection.	Use the host's view, or set `auricIconStudio.hostFamilyLibraries` to `false` if Icon Studio is the host. See The merged Icon Studio.
Modern commands are missing from the Command Palette.	Same as above: they hide while another extension hosts.	Use the host's commands; keybindings still work.
`Ctrl` + `Alt` + `I` does nothing.	The editor is not focused, another extension hosts the library, or the keys are bound elsewhere.	Focus a text editor and check **Keyboard Shortcuts**.
"Open a text editor before inserting an SVG."	No text editor was available.	Open a file, click into it, then insert again.
The inserted format is not what I expected.	Smart insert chose a format for the file's language.	Use **Copy Icon As...** for an explicit format, or turn off `auricIcons.library.smartInsert`.
Likes from the retired "Auric Artisan Drawing Icon" extension are missing.	Each extension keeps its own storage; history does not move between different extensions.	Like the icons again in Modern.
A command error mentions that another extension owns the same command.	The retired "Auric Artisan Drawing Icon" or "Auric Artisan Icons" extension is still installed.	Uninstall the retired extensions; Modern supersedes them.
The library says the icon catalog is not bundled.	The installed files are incomplete.	Reinstall the extension from the Marketplace.

Still stuck

Note your VS Code version, the extension version (0.12.1), which Auric icon extensions are installed, and the steps that reproduce the problem, then contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Frequently asked questions

Short answers to common questions about Auric Icons: Modern.

Is it free?

Yes. It is free to install and use for any purpose, personal or commercial. Redistribution, republishing and resale are not permitted.

Do I have to use both the theme and the library?

No. The file icon theme and the Icon Library are independent. Use either or both.

Why does changing a setting need a reload?

VS Code reads icon themes once and caches them. The extension rebuilds the theme files when you change a setting, and a window reload shows the result.

Does folder colour change file icons too?

No. It re-tints folders only. File icons keep the semantic colours that identify them. Use saturation and opacity to tone down all icons.

Can I use two framework packs at once?

No. One pack is active at a time.

What is the difference between Modern and Sharp?

They are the two style families in the 720-icon library: 420 Simple Modern icons and 300 Sharp Modern icons. Filter by **Style** to see one family.

How is this different from Auric Artisan File Icons?

File Icons offers three Explorer styles (Facet, Playful and Modern) with its own studio, smart decorations and profiles. This extension offers the Modern Explorer theme plus the 720-icon library. See the comparison.

Does Modern need the internet?

No. It has no network access and no telemetry at runtime.

Where did the hand-drawn icons go?

Since version 0.4.0 the hand-drawn (Warm Drawn) icons live in **Auric Icons: Drawn**. Install it alongside Modern to browse both in one library.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Auric Icons: Modern: other information

Contents of the supporting pages for Auric Icons: Modern.

This folder holds supporting pages for Auric Icons: Modern: the terms it uses, its shortcuts, what it can and cannot do, how it compares with the other Auric icon extensions, its release history and its privacy behaviour.

File	What it covers
glossary.md	Terms used in the theme and library.
shortcuts.md	Keybindings and every command.
limits-and-accuracy.md	What the theme and library can and cannot do.
related.md	Comparison with Icon Studio, Icon Hub, File Icons and the other collections.
release-notes.md	User-facing history.
privacy.md	What is stored and what uses the network.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Shortcuts and commands

Keybindings, in-view keys and every command in Auric Icons: Modern.

Keybindings

Action	Windows and Linux	macOS
Smart Insert Icon	`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`
Insert from Recent	`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`

Both need a focused text editor, and are only active while Modern hosts the library (they step aside when Auric Artisan Icon Studio or a sibling collection hosts).

Keys in the Icon Library

Key	Action
`/`	Focus search
`Escape`	Clear search; close a dropdown
`Arrow Up` / `Arrow Down`	Move between dropdown options

Commands

Type "Auric Icons: Modern" in the Command Palette.

Command	Area
Configure…	Theme

Command	Area
Select Icon Pack	Theme
Select Folder Colour	Theme
Toggle Explorer Arrows	Theme
Detect Auric Artisan Workspace	Theme
Restore Default Settings	Theme
Activate Icon Theme	Theme
Open Icon Library	Library
Search Icons	Library
Insert SVG at Cursor	Library
Smart Insert Icon (matches active language)	Library
Copy Icon As… (JSX, CSS, Markdown, Data URI, Image tag)	Library
Insert from Likes	Library
Insert from Recent	Library
Toggle Like for an Icon	Library
Clear Likes, Recents and Learned Recommendations	Library
Copy SVG	Library
Save SVG As…	Library
Open SVG Source	Library

Older keybindings that use `auricArtisanIcons.*` command IDs still work as aliases for the Library commands.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Glossary

Definitions of the terms used in Auric Icons: Modern.

Term	Meaning
Association	A rule that maps a file extension, filename or folder name to an icon.
Auto variant	A variant choice that follows the language of the file you are editing.
Catalog path	The relative location of an icon file inside the bundled catalog.
Data URI	Text that embeds a file directly, starting `data:image/svg+xml,`.
Duotone	A variant drawn in two tones.
Explorer arrows	The chevrons beside folders in the Explorer that expand and collapse them.
File icon theme	The set of icons VS Code shows beside files and folders. Only one is active at a time.
Framework pack	Extra file icons for one framework's file types, such as Angular components or Vue composables.
Host	The one extension that shows the shared library when several Auric collections are installed.

Term	Meaning
Icon ID	The unique name of an icon. Library IDs end in a four-digit number, such as `analytics-bar-chart-0042`; theme IDs are short names such as `typescript`.
Icon Studio	The heading of the shared library view, and also the name of the separate Auric Artisan Icon Studio extension.
Mono	A single-colour variant.
Preview surface	The light, dark or automatic backdrop behind library cards.
Semantic colour	A colour that identifies a file type, such as the TypeScript blue.
Sharp Modern	One of the two style families in the library (300 icons), shown as **Sharp** in the Style filter.
Simple Modern	The other style family (420 icons), shown as **Modern** in the Style filter.
Smart guide	Tip cards at the top of the library that react to your file and usage.
Smart insert	Choosing the output format from the language of the file you are editing.
Variant	One rendering of an icon: Color, Mono, Outline or Duotone.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Limits and accuracy

What the Modern file icon theme and Icon Library can and cannot do.

File icon theme

- Icon themes match file and folder names without path context. A name such as `settings.json` cannot be given a special icon for one folder only, which is why `.auric-artisan/settings.json` has no dedicated icon.
- Association keys are either `*.ext` or an exact name. Wildcards in the middle of a name and folder paths are not supported.
- Changes need a window reload to appear.
- Workspace detection checks only the top of local workspace folders; remote and virtual folders are skipped.
- Only one framework pack can be active.
- Colour is never the only cue: each icon also has a distinct shape, so icons stay distinguishable at low saturation.

Icon Library

- The library is fixed to the installed version: 720 icons. New icons come with updates.
- Smart insert picks a format by language only, not by the exact cursor position. Check quotes and surrounding code.
- Recommendations are local rankings from the file you are editing and your likes and use, not an AI service.
- Hover previews recognise only Modern library IDs (a hyphenated name ending in four digits).
- There are no size, colour or stroke controls for output; icons are inserted as drawn. (Icon Studio and Icon Hub offer output controls.)

Licence

Free to use for any purpose. Redistribution, republishing to any marketplace, resale and derivative releases are not permitted. Letterforms in some icons are outlines derived from Inter under the SIL Open Font Licence 1.1.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Privacy

What Auric Icons: Modern stores, where, and confirmation that it makes no network requests.

Auric Icons: Modern runs entirely inside VS Code. It has no telemetry, no account and no network access at runtime.

What is stored

Data	Where
Theme and library settings	Your VS Code settings under `auricIcons.*`.
A rebuilt copy of the theme when you customise it	Inside the extension's own installation folder. The original artwork is never changed.
Likes (up to 500), recent icons (up to 60), learned preferences and dismissed guide tips	VS Code's extension storage on this machine.
"Don't ask again" for workspace detection	VS Code's storage for that workspace.
Library search, filters, variant and preview surface	The view's saved state in VS Code.

What the extension reads

- To offer the theme in Auric Artisan workspaces, it checks whether `.auric-artisan` or `.auricignore` exists at the top of each local workspace folder. It does not read their contents or walk your folders.
- For recommendations and smart insert, it reads the language, file name and up to about 60 lines around the cursor in the active editor. This stays in memory.

What the extension writes

- Inserting changes your editor content. Copying writes to the clipboard. Saving writes a file only where you choose in a save dialog.
- Its own commands update your user settings.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Release notes

User-facing history of Auric Icons: Modern, from its first release to 0.12.1.

0.12.1

The current Marketplace version.

- Now covered by the Auric Artisan Free Use Licence 2.0 instead of MIT. The extension stays free to install and use for any purpose, personal or commercial. Redistribution, republishing to any marketplace, resale and derivative releases are no longer permitted.

0.12.0

- Marketplace README images now display correctly on the Marketplace page and in VS Code's extension view.
- Future Auric icon collections can join the shared library without every extension being updated.
- The library view, palette entries and shared shortcuts are disabled while another icon collection or Auric Artisan Icon Studio hosts the library.

0.11.0

- New Marketplace artwork.
- The shared library gained keyboard and pointer enlargement and a remembered **Auto**, **Light** or **Dark** preview surface.
- Verified the file theme alone and in every co-install combination.

0.4.0

- Display name changed to **Auric Icons: Modern**. Extension ID, command IDs, settings and theme ID are unchanged.
- The library now carries the 720 modern icons (Simple Modern and Sharp Modern). The 480 hand-drawn icons moved to **Auric Icons: Drawn**.
- Co-install cooperation: with Drawn or Playful installed, one library appears, with a **Library** filter and source badges. Likes and recents stay with the extension that owns each icon, and the merge updates without a reload.
- Each collection now declares its own variants, and the variant switcher shows the union, falling back to Color where a collection lacks the selected variant.
- **Auric Icons: Playful** joined the family.
- Fixed a start-up crash when the retired "Auric Artisan Drawing Icon" extension was still installed.

0.3.0

- The extension combined the file and folder icon theme with the Icon Library, absorbing the retired "Auric Artisan Drawing Icon" and "Auric Artisan Icons" extensions.
- Added hover previews for icon IDs, the self-contained image tag, icon name and catalog path copy formats, and multi-cursor insert.
- Old `auricArtisanIcons.*` commands and settings keep working as fallbacks.

0.2.0

- Renamed the extension; the theme ID became `auric-artisan-modern-icon`, so anyone using the earlier ID had to pick the theme again.
- Added Auric Artisan workspace icons, workspace auto-detection, the four framework packs (26 icons), runtime folder colour, saturation, opacity, arrows and custom associations, and seven new glyphs.
- Fixed the missing extension icon.
- Improved legibility of folder badges and the darkest slate colour.

0.1.0

- Initial release: the chamfered-plate icon language, complete open and closed folder pairs, light-theme variants for monochrome marks and a built-in contrast check.

VS CODE EXTENSIONS › AURIC ICONS: MODERN

Related extensions and comparison

How Auric Icons: Modern compares with Drawn, Playful, Icon Studio, Icon Hub, File Icons, Emoji and the Icon Pack.

At a glance

Extension	Main job	Explorer file icon theme	Artwork	Network
Auric Icons: Modern	Modern icon library plus the Modern file and folder icon theme.	Yes: **Auric Icons: Modern**	720 library icons; 408 theme icons	None
Auric Icons: Drawn	Warm Drawn icon library.	No	480 icons	None
Auric Icons: Playful	Emoji-style icon library.	No	1,225 icons	None
Icon Studio	One offline studio for every Auric collection, emoji and seven open-source sets.	No	17,234 entries in 11 collections	None
Icon Hub	Seven open-source sets with colour and stroke controls, sprite and ZIP export, and CDN insertion.	No	12,895 icons	Optional metadata checks; CDN output loads from the web
File Icons	Explorer file and folder icons in three styles, plus File Icon Studio.	Yes: Facet, Playful and Modern	1,464 SVG assets	None
Emoji	Emoji Studio.	No	1,914 emoji	None
Icon Pack	Installs Modern, Playful, Drawn and Emoji together.	Through Modern	None of its own	Needs the Marketplace to install

How the three collections merge

- Install Modern with Drawn and/or Playful and one of them becomes the host of a single merged library, with a **Library** filter, collection badges on cards and every collection's variants in one switcher. The others hide their views and forward their commands. See The merged Icon Studio.
- Install Auric Artisan Icon Studio and, by default, it hosts all three instead. Modern's file icon theme keeps working.
- Likes and recent icons stay with the extension that owns each icon.

Modern in File Icons

Auric Artisan File Icons also includes a Modern Explorer style, with 464 SVG assets (219 file designs, 114 folder pairs, light alternatives and state overlays) and the same four framework packs. It uses the same theme ID as this extension's file icon theme and its own `auricFileIcons.*` settings. If you install both, see Troubleshooting. The File Icons documentation describes importing Modern's file settings in Modern and older Playful settings.

Retired extensions

Modern supersedes two unpublished preview extensions, "Auric Artisan Drawing Icon" and "Auric Artisan Icons". If either is installed, uninstall it: they share keybindings and command IDs with Modern.

Other related pages

- Install and set up
- Icon Library on the website

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Auric Icons: Drawn

What the Auric Icons: Drawn extension is, who it is for, how to install it and where to find each guide.

Auric Icons: Drawn is a library of 480 original hand-drawn SVG icons for VS Code, in a style called **Warm Drawn**: organic geometry with friendly imperfections. Every icon comes in **Color**, **Mono**, **Outline** and **Duotone** variants, across 30 categories from Coding and development to Music and Audio.

You browse the icons in an **Icon Library** view, search them by concept, description, tag or synonym, and insert them into your code in a format that suits the file: a JSX element in React, markup in HTML, a CSS background in stylesheets, an image in Markdown. You can also copy icons in nine formats, save SVG files and preview icon IDs by hovering over them in code.

Drawn is a library only. It deliberately does not include a file icon theme for the Explorer, because its 64-pixel illustration style does not suit 16-pixel Explorer icons. Install **Auric Icons: Modern** alongside it for the family's Explorer theme; the two libraries then merge into one view.

Who it is for

- Front-end developers and designers who want warm, illustrative icons for interfaces, documentation and empty states.
- Anyone who already uses Auric Icons: Modern or Playful and wants a third artwork style in the same library.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-icons-drawn
Extension ID	`auric-artisan.auric-icons-drawn`
Current version	0.12.1
Requires	Visual Studio Code 1.85 or later, or a compatible editor built on it
Price	Free to install and use

```
code --install-extension auric-artisan.auric-icons-drawn
```

Drawn is also installed by the Auric Artisan Icon Pack, and its artwork is included in Auric Artisan Icon Studio.

Quick start

1. Install the extension.
2. Open a file and place the cursor where an icon should go.
3. Select **Auric Icons: Drawn** in the Activity Bar to open the **Icon Library**.
4. Type a concept in **Search**, for example `calendar`.
5. Choose a variant: **Auto**, **Color**, **Mono**, **Outline** or **Duotone**.
6. Select **Insert** on a card, or use **Copy Icon As...** for a specific format.

What you can do

- Search 480 icons with synonym-aware, fuzzy and personalised ranking.
- Filter by category and preview on light or dark surfaces.
- Insert with language-aware output at multiple cursors.
- Copy in nine formats: raw SVG, JSX element, React component, Markdown image, CSS background, data URI, self-contained image tag, icon name and catalog path.
- Save SVG files and open the SVG source.
- Keep likes and recent icons, and get recommendations for the file you are editing.
- Preview icon IDs on hover.
- Merge with Modern and Playful into one library when co-installed.

In this folder

docs/

- Contents of docs/
- Getting started
- Browse the Icon Library
- Insert, copy and save icons
- Use Drawn with Modern and Playful
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Auric Icons: Drawn guides

Contents of the Auric Icons: Drawn guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Icons: Drawn, a library-only extension of 480 hand-drawn icons.

File	What it covers
getting-started.md	Install, open the Icon Library and insert a first icon.
icon-library.md	Search, filters, variants, preview surfaces, the smart guide and recommendations.
insert-copy-and-save.md	Insert and smart insert, nine copy formats, save, open source and hover previews.
merged-icon-studio.md	How Drawn shares one library with Modern, Playful and Icon Studio.
reference.md	Every command, setting, keybinding, category and control.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Getting started with Auric Icons: Drawn

Install Auric Icons: Drawn, open the Icon Library and insert your first hand-drawn icon.

Before you begin

- You need Visual Studio Code 1.85 or later, or an editor that keeps VS Code's extension features.
- Everything works offline. There is no telemetry and no network access.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Icons: Drawn** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-icons-drawn
```

2. Open the Icon Library

- Select **Auric Icons: Drawn** in the Activity Bar to open the **Icon Library** view, or
- Run **Auric Icons: Drawn: Open Icon Library** to open the larger **Icon Studio — Library** editor tab.

If Auric Artisan Icon Studio or Auric Icons: Modern is also installed, the Drawn view may be hidden because the other extension shows a merged library that already contains Drawn. See Use Drawn with Modern and Playful.

3. Look around

The view shows a header with the icon count, up to four smart guide tips, a **Search** box, **Category** and **Style** filters, **Variant** buttons (**Auto**, **Color**, **Mono**, **Outline**, **Duotone**), **Preview surface** buttons (**Auto**, **Light**, **Dark**), the **All**, **✦ For you**, **♥ Likes**

and ↺ **Recent** chips, a ✦ **Recommended for this file** rail and the icon grid.

4. Insert your first icon

1. Open a file and place the cursor.
2. Type `calendar` in **Search**.
3. Select **Insert** on a card. With smart insert on (the default), a React file receives a JSX element, an HTML file receives SVG markup and a stylesheet receives a CSS background.

Or press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS) in the editor, search, and choose an icon.

5. Like the icons you reuse

Select the heart on a card. Liked icons appear under ♥ **Likes** and in **Insert from Likes**, and they improve your recommendations.

Result

You have inserted a Warm Drawn icon into your file in a format that suits it.

Next steps

- Browse the Icon Library
- Insert, copy and save icons

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Browse the Icon Library

Search the 480 Warm Drawn icons, filter by category, switch variants and preview surfaces, and use the smart guide and recommendations.

The Drawn library holds 480 hand-drawn icons in 30 categories. Each icon is available in **Color**, **Mono**, **Outline** and **Duotone**.

Open the library

- **Sidebar:** select **Auric Icons: Drawn** in the Activity Bar.
- **Editor tab:** run **Auric Icons: Drawn: Open Icon Library**, select the ↗ button (**Open full explorer**), or the open-preview icon in the view's title bar. The tab is called **Icon Studio — Library**.

The screen

Area	What it does
Header	"Icon Studio", **Icon Library**, a count line such as "480 icons · 72 loaded", and ↗.
Smart guide	Up to four dismissible tips, such as "Markdown detected" or "Like icons you reuse". Turn off with `auricIconsDrawn.library.showGuide`.

Area	What it does
Search	Placeholder "Search 480 icons — try ”empty state” or ”auth””.
Library	Only shown when several collections are merged.
Category	**All categories** or one of 30.
Style	**All styles** or **Drawn** (plus other collections' styles when merged).
Variant	**Auto**, **Color**, **Mono**, **Outline**, **Duotone**.
Preview surface	**Auto**, **Light**, **Dark**.
Chips	**All**, ✦ **For you**, ♥ **Likes** (count), ↺ **Recent** (count).
Recommended rail	✦ **Recommended for this file**, with **Refresh**.
Grid	Cards that load as you scroll.

Search

Search by concept, description, tag or synonym; results are ranked as you type and personalised by what you like and use. For example, `delete` finds trash icons. Press `/` to focus search and `Escape` to clear it.

Variants and Auto

Auto follows the file you are editing and shows the resolved variant, for example "Auto · mono".

Editing in	Auto variant
JavaScript, TypeScript, React JSX and TSX	Outline
HTML, PHP, Handlebars, Vue, Svelte, Astro, Markdown, MDX, XML, SVG	Color
CSS, SCSS, Less, JSON, YAML, Python	Mono
Anything else	Color

Command Palette actions use `auricIconsDrawn.library.defaultVariant` (`auto`, `color`, `mono`, `outline`, `duotone`).

Preview surface

Light and **Dark** place every card on a light or dark backdrop; **Auto** follows your VS Code theme. The choice is remembered.

The views

Chip	Shows
All	Every icon that matches the search and filters.
✦ **For you**	Icons recommended for the file you are editing and your likes and use.
♥ **Likes**	Liked icons.
↺ **Recent**	Icons you inserted, copied, saved or opened, newest first (up to 60).

Recommendations

The ✦ **Recommended for this file** rail shows icons that match the file you are editing or your habits, each with a reason: **Matches this file**, **In your likes**, **Used recently**, **Fits your style** or **Popular in your catalog**. Selecting a rail card inserts it; **Refresh** recomputes the rail.

Setting	Default	Effect
`auricIconsDrawn.library.recommendations.enabled`	true	Show recommendations.
`auricIconsDrawn.library.recommendations.limit`	12	Icons in the rail (4 to 24).

Cards

Control	Action
Artwork	Opens the SVG source in an editor tab.
Heart	Adds to or removes from **Likes**.
Insert	Inserts into the editor.
Copy	Copies raw SVG.
Save	Saves an `.svg` file through a dialog.
JSX	Copies a JSX element.
URI	Copies a data URI.
Source	Opens the SVG source.

Density and batch size

Setting	Values	Default
`auricIconsDrawn.library.density`	`comfortable`, `compact`	`comfortable`
`auricIconsDrawn.library.pageSize`	24 to 240	72

Result

You can find any Drawn icon quickly, preview it in the right variant and background, and act on it from its card.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Insert, copy and save icons

Insert Drawn icons with language-aware output, copy them in nine formats, save SVG files, open the source and preview icon IDs on hover.

Insert from the library

1. Open a file and place one or more cursors, or select text to replace.

2. Select **Insert** on a card.
3. The icon is inserted at every cursor; multi-line output keeps the cursor line's indentation.

Without an open text editor you see "Open a text editor before inserting an SVG."

Smart insert

With `auricIconsDrawn.library.smartInsert` on (the default), **Insert** adapts to the file:

Editing in	Output
JavaScript, TypeScript, React JSX and TSX, MDX	JSX element
HTML, PHP, Handlebars, Vue, Svelte, Astro	HTML markup (the SVG)
Markdown	Markdown image
CSS, SCSS, Less	CSS background
JSON, JSONC, YAML, Python	Data URI
XML, SVG and any other language	Raw SVG

With the setting off, **Insert** uses raw SVG. **Smart Insert Icon** always adapts.

From the keyboard: press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS) in a text editor, search, and choose an icon. With no search text the list starts with **Recommended for this file**.

Copy in a chosen format

Run **Auric Icons: Drawn: Copy Icon As... (JSX, CSS, Markdown, Data URI, Image tag)**, choose an icon and a format:

Format	What you get
Raw SVG	The SVG markup.
JSX element	The SVG with JSX attribute names.
React component	`export function IconName(props) { return (<svg {...props} …>); }`
Markdown image	`![Icon name](data:image/svg+xml,…)`
CSS background	`background-image: url("data:image/svg+xml,…");` plus no-repeat and contain sizing
Data URI	`data:image/svg+xml,…`
Image tag (self-contained)	`<img src="data:image/svg+xml,…" alt="Icon name" width="24" height="24">`
Icon name	The icon ID, such as `files-blank-document-drawn-0001`.
Catalog path	The icon's relative path in the catalog.

On a card, **Copy** copies raw SVG, **JSX** a JSX element and **URI** a data URI.

Save and open the source

- **Save** on a card, or **Save SVG As...**, opens a save dialog in your first workspace folder (or home folder) with the icon's file name. Select **Save icon**.

- The artwork, **Source** on a card, or **Open SVG Source** opens the SVG in a preview tab.

Other commands

Command	What it does
Search Icons	Choose an icon, then **Smart Insert (matches this file)**, **Insert SVG at Cursor**, **Copy SVG**, **Save SVG As...**, **Open SVG Source** or **Toggle Like**.
Insert SVG at Cursor	Choose and insert (smart insert applies when on).
Copy SVG	Choose and copy raw SVG.
Insert from Likes	Choose a liked icon and insert.
Insert from Recent	Choose a recent icon and insert. Also `Ctrl` + `Alt` + `R`.
Toggle Like for an Icon	Like or unlike an icon.
Clear Likes, Recents and Learned Recommendations	Confirm with **Clear** to reset.

Hover previews

Hover over a Drawn icon ID, such as `files-blank-document-drawn-0001`, in HTML, CSS, SCSS, Less, Vue, Svelte, Astro, JavaScript, TypeScript, React JSX and TSX, JSON, JSONC, Markdown, PHP, Handlebars or XML. The hover shows a 48-pixel colour preview, the name and description, ID, category and tags, and **Open in Icon Library**. Turn it off with `auricIconsDrawn.library.hoverPreview`.

Result

The icon is in your file, on your clipboard or saved as an `.svg` file, and it appears under **Recent**.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Use Drawn with Modern and Playful

How Drawn merges with Auric Icons: Modern and Playful into one library, and how Auric Artisan Icon Studio takes over hosting.

Drawn works by itself. When **Auric Icons: Modern** or **Auric Icons: Playful** is also installed, the extensions show one library instead of duplicate sidebars.

What happens when they are co-installed

1. One extension becomes the **host** automatically. With the current versions, that is Modern when it is installed; with only Drawn and Playful, Drawn hosts.
2. The host's **Icon Library** view (headed "Icon Studio") contains every installed collection.
3. A **Library** filter appears, and each card shows a badge naming its collection.
4. The **Variant** buttons offer every collection's variants. If a collection lacks the selected variant (for example Drawn has no **Filled** or **Small**), its cards fall back to Color.
5. The other extensions hide their views, Command Palette entries and shortcuts, and forward their commands to the host.

This updates on its own when you install or remove a collection. No reload is needed.

Likes and recents

Likes and recent icons are stored by the extension that owns each icon. The merged view shows them together. If you uninstall Drawn, its icons and their history leave the merged view; the other collections keep theirs.

Hover previews

Drawn previews only Drawn icon IDs, so hovering always shows one preview.

With Auric Artisan Icon Studio

Auric Artisan Icon Studio already contains the Drawn collection. By default it hosts Drawn, Modern and Playful, so their separate views, palette entries and shortcuts are hidden. Set `auricIconStudio.hostFamilyLibraries` to `false` to bring the Drawn view back. Icon Studio keeps its own likes and recents.

See The merged Icon Studio in the Modern documentation for more detail.

A file icon theme for Drawn

Drawn has no Explorer file icon theme. Install Auric Icons: Modern for the family's file and folder theme; both libraries still merge.

Result

With several Auric collections installed, you see Drawn alongside them in one library and can filter to Drawn with the **Library** filter.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Auric Icons: Drawn reference

Every command, keybinding, setting, view, category and library control in Auric Icons: Drawn 0.12.1.

This reference describes version 0.12.1 as published on the Visual Studio Marketplace.

Commands

Category in the Command Palette: **Auric Icons: Drawn**.

Command	ID	What it does
Open Icon Library	`auricIconsDrawn.library.openExplorer`	Opens the **Icon Studio — Library** editor tab.
Search Icons	`auricIconsDrawn.library.searchIcons`	Choose an icon, then an action.
Insert SVG at Cursor	`auricIconsDrawn.library.insertSvg`	Choose an icon and insert it.

Command	ID	What it does
Smart Insert Icon (matches active language)	`auricIconsDrawn.library.smartInsert`	Choose an icon and insert with language-aware output.
Copy Icon As… (JSX, CSS, Markdown, Data URI, Image tag)	`auricIconsDrawn.library.copyAs`	Choose an icon and one of nine formats.
Insert from Likes	`auricIconsDrawn.library.showFavorites`	Choose a liked icon and insert.
Insert from Recent	`auricIconsDrawn.library.showRecents`	Choose a recent icon and insert.
Toggle Like for an Icon	`auricIconsDrawn.library.toggleFavorite`	Like or unlike an icon.
Clear Likes, Recents and Learned Recommendations	`auricIconsDrawn.library.resetPersonalization`	Confirm, then reset personalisation.
Copy SVG	`auricIconsDrawn.library.copySvg`	Choose an icon and copy raw SVG.
Save SVG As…	`auricIconsDrawn.library.saveSvg`	Choose an icon and save it as a file.
Open SVG Source	`auricIconsDrawn.library.openSource`	Choose an icon and open its SVG.

These commands are hidden from the Command Palette while another extension hosts the shared library; from keybindings they forward to the host.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command	When
`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`	Smart Insert Icon	Editor focused and Drawn hosts the library
`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`	Insert from Recent	Editor focused and Drawn hosts the library

In the view: `/` focuses search, `Escape` clears it or closes a dropdown, arrow keys move between dropdown options.

Settings

Setting	Type	Values or range	Default	What it does
`auricIconsDrawn.library.defaultVariant`	string	`auto`, `color`, `mono`, `outline`, `duotone`	`auto`	Variant for Command Palette actions; `auto` follows the file.
`auricIconsDrawn.library.pageSize`	number	24 to 240	72	Cards streamed per batch.
`auricIconsDrawn.library.smartInsert`	boolean	`true`, `false`	`true`	Language-aware output for Insert.
`auricIconsDrawn.library.recommendations.enabled`	boolean	`true`, `false`	`true`	Recommendations for the file you are editing.
`auricIconsDrawn.library.recommendations.limit`	number	4 to 24	12	Icons in the recommendation rail.
`auricIconsDrawn.library.showGuide`	boolean	`true`, `false`	`true`	Smart guide tips.
`auricIconsDrawn.library.density`	string	`comfortable`, `compact`	`comfortable`	Card size.
`auricIconsDrawn.library.hoverPreview`	boolean	`true`, `false`	`true`	Hover previews for icon IDs.

Views and menus

Place	Contents
Activity Bar	**Auric Icons: Drawn** container with the **Icon Library** view (hidden while another extension hosts).
View title bar	**Open Icon Library**, **Smart Insert Icon**, **Insert from Likes**; **…** menu: **Clear Likes, Recents and Learned Recommendations**.
Editor tab	**Icon Studio — Library**.

Drawn contributes no file icon theme.

Library contents

Item	Value
Icons	480
Style	Warm Drawn (shown as **Drawn** in the Style filter)
Variants	Color, Mono, Outline, Duotone
Categories	30

Categories: Files and document types; Folders and organization; Coding and development; Artificial intelligence; Data and databases; Cloud and hosting; Cybersecurity; Communication; Social interaction; User accounts and profiles; Business and finance; E-commerce; Marketing; Analytics and reporting; Productivity; Project management; Calendar and scheduling; Education and learning; Science and laboratory; Healthcare and wellness; Food and cooking; Travel and transportation; Maps and navigation; Weather and environment; Nature and sustainability; Home and Furniture; Tools and Construction; Art and Design; Photography and Video; Music and Audio.

Library controls

Control	Options
Search	Free text, synonyms, fuzzy ranking.
Library	**All libraries** or one collection (only when merged).
Category	**All categories** or one of 30.
Style	**All styles** or **Drawn**.
Variant	**Auto**, **Color**, **Mono**, **Outline**, **Duotone**.
Preview surface	**Auto**, **Light**, **Dark**.
Chips	**All**, ✦ **For you**, ♥ **Likes**, ↺ **Recent**.
Card buttons	Artwork (open source), heart, **Insert**, **Copy**, **Save**, **JSX**, **URI**, **Source**.

Limits

Item	Limit
Recent icons	60

Item	Limit
Liked icons	500
Quick Pick results	200 with a search term; 400 without
Smart guide tips	Up to 4

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Icons: Drawn.

Symptom	Why it happens	What to do
The **Auric Icons: Drawn** view is missing from the Activity Bar.	Auric Artisan Icon Studio or Auric Icons: Modern hosts the shared library, which already includes Drawn.	Use the host's library and filter by **Library** › **Drawn**, or set `auricIconStudio.hostFamilyLibraries` to `false` if Icon Studio is the host.
Drawn commands are missing from the Command Palette.	Same reason: they hide while another extension hosts.	Use the host's commands; keybindings still work.
`Ctrl` + `Alt` + `I` does nothing.	The editor is not focused, another extension hosts, or the keys are bound elsewhere.	Focus a text editor and check **Keyboard Shortcuts**.
"Open a text editor before inserting an SVG."	No text editor is available.	Open a file, click into it and insert again.
I cannot find Drawn icons in the Explorer file icon picker.	Drawn has no file icon theme.	Install Auric Icons: Modern or Auric Artisan File Icons for Explorer icons.
A card looks like Color after choosing Filled or Small.	Filled and Small are Playful variants; Drawn falls back to Color.	Choose Color, Mono, Outline or Duotone for Drawn icons.
The format inserted is not what I expected.	Smart insert chose a format for the file's language.	Use **Copy Icon As...**, or turn off `auricIconsDrawn.library.smartInsert`.
The library says the icon catalog is not bundled.	The installed files are incomplete.	Reinstall from the Marketplace.

If the problem continues, note your VS Code version, the extension version (0.12.1), the other Auric extensions installed and the steps to reproduce, then contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Frequently asked questions

Short answers to common questions about Auric Icons: Drawn.

Why is there no file icon theme?

Drawn's icons are 64-pixel illustrations. They do not have the paired 16-pixel folder shapes and file associations a good Explorer theme needs. Install Auric Icons: Modern for the family's Explorer theme.

Do I need Modern or Playful too?

No. Drawn works alone. Installing them together gives you one merged library.

Is Drawn included in Icon Studio?

Yes. Auric Artisan Icon Studio bundles the Drawn collection, so you do not need this extension to use Drawn artwork there.

Is it free?

Yes, free to use for any purpose. Redistribution and resale are not permitted.

Does it work offline?

Yes. There is no network access or telemetry.

Can I change icon size or colour on insert?

Not in this extension; icons are inserted as drawn. Auric Artisan Icon Studio has size and colour output settings.

Where are my likes kept?

In VS Code's storage for this extension on your machine. They stay with Drawn even when another extension hosts the merged library.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Auric Icons: Drawn: other information

Contents of the supporting pages for Auric Icons: Drawn.

This folder holds supporting pages for Auric Icons: Drawn: terms, shortcuts, limits, a comparison with the other Auric icon extensions, the release history and privacy behaviour.

File	What it covers
glossary.md	Terms used in the library.
shortcuts.md	Keybindings and every command.
limits-and-accuracy.md	What the library can and cannot do.
related.md	Comparison with Modern, Playful, Icon Studio, Icon Hub, File Icons, Emoji and Icon Pack.
release-notes.md	User-facing history.
privacy.md	What is stored and what uses the network.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Shortcuts and commands

Keybindings, in-view keys and every command in Auric Icons: Drawn.

Keybindings

Action	Windows and Linux	macOS
Smart Insert Icon	`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`
Insert from Recent	`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`

Both need a focused text editor and are active only while Drawn hosts the library.

Keys in the Icon Library

Key	Action
`/`	Focus search
`Escape`	Clear search; close a dropdown
`Arrow Up` / `Arrow Down`	Move between dropdown options

Commands

Type "Auric Icons: Drawn" in the Command Palette: **Open Icon Library**, **Search Icons**, **Insert SVG at Cursor**, **Smart Insert Icon (matches active language)**, **Copy Icon As... (JSX, CSS, Markdown, Data URI, Image tag)**, **Insert from Likes**, **Insert from Recent**, **Toggle Like for an Icon**, **Clear Likes, Recents and Learned Recommendations**, **Copy SVG**, **Save SVG As...**, **Open SVG Source**.

See the reference for IDs and details.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Glossary

Definitions of the terms used in Auric Icons: Drawn.

Term	Meaning
Auto variant	A variant choice that follows the language of the file you are editing.
Catalog path	The relative location of an icon file inside the bundled catalog.
Data URI	Text that embeds a file directly, starting `data:image/svg+xml,`.
Duotone	A variant drawn in two tones.
Host	The one extension that shows the shared library when several Auric collections are installed.
Icon ID	The unique name of an icon, such as `files-blank-document-drawn-0001`.
Icon Studio	The heading of the shared library view, and also the name of the separate Auric Artisan Icon Studio extension.
Mono	A single-colour variant.
Preview surface	The light, dark or automatic backdrop behind library cards.
Smart guide	Tip cards at the top of the library that react to your file and usage.
Smart insert	Choosing the output format from the language of the file you are editing.
Variant	One rendering of an icon: Color, Mono, Outline or Duotone.
Warm Drawn	The hand-drawn style of this collection: organic geometry with friendly imperfections.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Limits and accuracy

What Auric Icons: Drawn can and cannot do.

- The library contains 480 icons at the installed version; new icons come with updates.
- There is no Explorer file icon theme.
- Icons are inserted as drawn: there are no size, colour or stroke controls for output.
- Smart insert picks a format from the file's language, not the exact cursor position. Check quotes and surrounding code.
- Recommendations are local rankings from the file you are editing and your likes and use, not an AI service.
- Hover previews recognise only Drawn icon IDs (a hyphenated name ending in four digits).
- Personalisation built up in the retired "Auric Artisan Drawing Icon" preview extension does not transfer; each extension keeps its own storage.
- Licence: free to use for any purpose; redistribution, republishing, resale and derivative releases are not permitted.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Privacy

What Auric Icons: Drawn stores, where, and confirmation that it makes no network requests.

Auric Icons: Drawn runs entirely inside VS Code. It has no telemetry, no account and no network access at runtime.

Data	Where
Settings	Your VS Code settings under `auricIconsDrawn.library.*`.

Data	Where
Likes (up to 500), recent icons (up to 60), learned preferences, dismissed tips	VS Code's extension storage on this machine.
Search, filters, variant and preview surface	The view's saved state in VS Code.

For recommendations and smart insert, the extension reads the language, file name and up to about 60 lines around the cursor of the active editor. This stays in memory and is never stored or sent.

Inserting changes your editor content, copying writes to the clipboard, and saving writes a file only where you choose in a save dialog.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Release notes

User-facing history of Auric Icons: Drawn.

0.12.1

The current Marketplace version.

- Now covered by the Auric Artisan Free Use Licence 2.0 instead of MIT. It stays free to install and use for any purpose; redistribution, republishing to any marketplace, resale and derivative releases are no longer permitted.

0.12.0

- Marketplace README images now display correctly.
- Future Auric icon collections can join the shared library without every extension being updated.
- The library view, palette entries and shared shortcuts are disabled while another icon collection or Auric Artisan Icon Studio hosts the library.

0.11.0

- New Marketplace artwork.
- The shared library gained keyboard and pointer enlargement and a remembered **Auto**, **Light** or **Dark** preview surface.
- Verified alone and co-installed in VS Code.

0.1.0

- Initial release: 480 hand-drawn icons in four variants and 50 categories, with the Icon Library, smart search with synonyms and personalisation, smart insert (`Ctrl` + `Alt` + `I`), nine copy formats, hover previews, likes and recents (`Ctrl` + `Alt` + `R`) and per-file recommendations.
- Cooperates with Auric Icons: Modern and Playful: one merged library with collections badged and filterable, each shown in its own variants, and personalisation kept per extension.
- Succeeds the retired, never-published "Auric Artisan Drawing Icon" preview extension.

The current catalog groups the 480 icons into 30 categories.

VS CODE EXTENSIONS › AURIC ICONS: DRAWN

Related extensions and comparison

How Auric Icons: Drawn compares with Modern, Playful, Icon Studio, Icon Hub, File Icons, Emoji and the Icon Pack.

At a glance

Extension	Main job	Explorer file icon theme	Artwork	Network
Auric Icons: Drawn	Warm Drawn icon library.	No	480 icons	None
Auric Icons: Modern	Modern icon library plus the Modern file and folder icon theme.	Yes: **Auric Icons: Modern**	720 library icons; 408 theme icons	None
Auric Icons: Playful	Emoji-style icon library.	No	1,225 icons	None
Icon Studio	One offline studio for every Auric collection, emoji and seven open-source sets.	No	17,234 entries in 11 collections	None
Icon Hub	Seven open-source sets with colour and stroke controls, sprite and ZIP export, and CDN insertion.	No	12,895 icons	Optional metadata checks; CDN output loads from the web
File Icons	Explorer file and folder icons in three styles, plus File Icon Studio.	Yes: Facet, Playful and Modern	1,464 SVG assets	None
Emoji	Emoji Studio.	No	1,914 emoji	None
Icon Pack	Installs Modern, Playful, Drawn and Emoji together.	Through Modern	None of its own	Needs the Marketplace to install

How the three collections merge

- Install Drawn with Modern and/or Playful and one extension hosts a single merged library, with a **Library** filter, collection badges and every collection's variants. See Use Drawn with Modern and Playful.
- Install Auric Artisan Icon Studio and, by default, it hosts all three instead.
- Likes and recent icons stay with the extension that owns each icon.

Other related pages

- Install and set up
- Icon Library on the website

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Auric Icons: Playful

What the Auric Icons: Playful extension is, who it is for, how to install it and where to find each guide.

Auric Icons: Playful is a library of 1,225 emoji-style SVG icons for VS Code: warm, expressive artwork in 24 categories, from Reactions and Status to Dashboard and Form Elements. Every icon comes in four variants: **Color**, **Outline**, **Filled** and **Small**. The Small variant is a redrawing with simplified detail for use at 16 to 24 pixels, not a scaled-down copy.

You browse the icons in an **Icon Library** view, search by name, concept, description or synonym, and insert them into your code in a format that suits the file. You can also copy icons in nine formats, save SVG files and preview icon IDs by hovering over them in code.

Playful is a library only; it has no Explorer file icon theme. This collection is different from the **Playful** Explorer style in Auric Artisan File Icons, which is a separate set of 500 file and folder icons.

Who it is for

- Designers and front-end developers building friendly, expressive interfaces, dashboards and admin panels.
- Anyone who wants emoji-like artwork as real SVG they can style and ship.
- Users of Auric Icons: Modern or Drawn who want a third style in the same library.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-icons-playful
Extension ID	`auric-artisan.auric-icons-playful`
Current version	0.12.1
Requires	Visual Studio Code 1.85 or later, or a compatible editor built on it
Price	Free to install and use

```
code --install-extension auric-artisan.auric-icons-playful
```

Playful is also installed by the Auric Artisan Icon Pack, and its artwork is included in Auric Artisan Icon Studio.

Quick start

1. Install the extension.
2. Open a file and place the cursor where an icon should go.
3. Select **Auric Icons: Playful** in the Activity Bar to open the **Icon Library**.
4. Search, for example `thumbs up`, or choose a category.
5. Choose a variant: **Auto**, **Color**, **Outline**, **Filled** or **Small**.
6. Select **Insert** on a card.

What you can do

- Search 1,225 icons in 24 categories with synonym-aware, fuzzy and personalised ranking.
- Preview each icon in four variants on light or dark surfaces.
- Insert with language-aware output at multiple cursors.
- Copy in nine formats: raw SVG, JSX element, React component, Markdown image, CSS background, data URI, self-contained image tag, icon name and catalog path.
- Save SVG files and open the SVG source.
- Keep likes and recent icons, and get recommendations for the file you are editing.
- Preview icon IDs on hover.
- Merge with Modern and Drawn into one library when co-installed.

In this folder

docs/

- Contents of docs/
- Getting started
- Browse the Icon Library
- Insert, copy and save icons
- Use Playful with Modern and Drawn
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Auric Icons: Playful guides

Contents of the Auric Icons: Playful guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Icons: Playful, a library-only extension of 1,225 emoji-style icons.

File	What it covers
getting-started.md	Install, open the Icon Library and insert a first icon.
icon-library.md	Search, filters, the four variants, preview surfaces, the smart guide and recommendations.
insert-copy-and-save.md	Insert and smart insert, nine copy formats, save, open source and hover previews.
merged-icon-studio.md	How Playful shares one library with Modern, Drawn and Icon Studio.
reference.md	Every command, setting, keybinding, category and control.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Getting started with Auric Icons: Playful

Install Auric Icons: Playful, open the Icon Library and insert your first emoji-style icon.

Before you begin

- You need Visual Studio Code 1.85 or later, or an editor that keeps VS Code's extension features.
- Everything works offline. There is no telemetry and no network access.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Icons: Playful** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-icons-playful
```

2. Open the Icon Library

- Select **Auric Icons: Playful** in the Activity Bar to open the **Icon Library** view, or
- Run **Auric Icons: Playful: Open Icon Library** to open the larger **Icon Studio — Library** editor tab.

If Auric Artisan Icon Studio, Auric Icons: Modern or Auric Icons: Drawn is also installed, the Playful view may be hidden because another extension shows a merged library that already contains Playful. See Use Playful with Modern and Drawn.

3. Look around

The view shows the icon count, up to four smart guide tips, **Search**, **Category** and **Style** filters, the **Variant** buttons (**Auto**, **Color**, **Outline**, **Filled**, **Small**), **Preview surface** (**Auto**, **Light**, **Dark**), the **All**, ✦ **For you**, ♥ **Likes** and ↺ **Recent** chips, a ✦

Recommended for this file rail and the grid.

4. Insert your first icon

1. Open a file and place the cursor.
2. Choose **Category** › **Reactions**, or search for a word such as `celebrate`.
3. Select **Small** if the icon will be shown at 16 to 24 pixels.
4. Select **Insert** on a card. With smart insert on, the output suits your file: a JSX element in React, SVG markup in HTML, a CSS background in stylesheets.

Or press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS) in the editor, search and choose.

Result

You have inserted a Playful icon, in the variant you chose, in a format that suits your file.

Next steps

- Browse the Icon Library
- Insert, copy and save icons

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Browse the Icon Library

Search the 1,225 Playful icons, filter by category, switch between Color, Outline, Filled and Small, and use previews, the smart guide and recommendations.

The Playful library holds 1,225 icons in 24 categories. Each icon has four dedicated variants:

Variant	Use
Color	Full-colour artwork.
Outline	Line drawing.
Filled	Solid shapes.
Small	A simplified redrawing for 16 to 24 pixels.

There is no Mono or Duotone variant in this collection.

Open the library

- **Sidebar:** select **Auric Icons: Playful** in the Activity Bar.
- **Editor tab:** run **Auric Icons: Playful: Open Icon Library**, select ↗ (**Open full explorer**), or the open-preview icon in the view's title bar. The tab is called **Icon Studio — Library**.

The screen

Area	What it does
Header	"Icon Studio", **Icon Library**, a count line such as "1,225 icons · 72 loaded", and ↗.
Smart guide	Up to four dismissible tips. Turn off with `auricIconsPlayful.library.showGuide`.
Search	Placeholder "Search 1,225 icons — try “empty state” or “auth””.
Library	Only shown when several collections are merged.
Category	**All categories** or one of 24.
Style	**All styles** or **Playful**.
Variant	**Auto**, **Color**, **Outline**, **Filled**, **Small**.
Preview surface	**Auto**, **Light**, **Dark**.
Chips	**All**, ✦ **For you**, ♥ **Likes** (count), ↺ **Recent** (count).
Recommended rail	✦ **Recommended for this file**, with **Refresh**.
Grid	Cards that load as you scroll.

Search

Search by name, concept, description, tag or synonym; results are ranked as you type and personalised by your likes and use. Press `/` to focus search and `Escape` to clear it.

Variants and Auto

Auto follows the file you are editing and shows what it resolved to, such as "Auto · outline".

Editing in	Auto variant
JavaScript, TypeScript, React JSX and TSX	Outline
Every other language	Color

Command Palette actions use `auricIconsPlayful.library.defaultVariant` (`auto`, `color`, `outline`, `filled`, `small`).

Preview surface

Light and **Dark** place every card on a light or dark backdrop; **Auto** follows your VS Code theme. The choice is remembered.

The views

Chip	Shows
All	Every icon matching the search and filters.
✦ **For you**	Icons recommended for the file you are editing and your likes and use.
♥ **Likes**	Liked icons.

Chip	Shows
↺ **Recent**	Icons you inserted, copied, saved or opened, newest first (up to 60).

Recommendations

The ✦ **Recommended for this file** rail shows icons that match the file you are editing or your habits, each with a reason: **Matches this file**, **In your likes**, **Used recently**, **Fits your style** or **Popular in your catalog**. Selecting a rail card inserts it; **Refresh** recomputes.

Setting	Default	Effect
`auricIconsPlayful.library.recommendations.enabled`	true	Show recommendations.
`auricIconsPlayful.library.recommendations.limit`	12	Icons in the rail (4 to 24).

Cards

Control	Action
Artwork	Opens the SVG source in an editor tab.
Heart	Adds to or removes from **Likes**.
Insert	Inserts into the editor.
Copy	Copies raw SVG.
Save	Saves an `.svg` file through a dialog.
JSX	Copies a JSX element.
URI	Copies a data URI.
Source	Opens the SVG source.

Density and batch size

Setting	Values	Default
`auricIconsPlayful.library.density`	`comfortable`, `compact`	`comfortable`
`auricIconsPlayful.library.pageSize`	24 to 240	72

Result

You can find any Playful icon, compare its four variants on either background, and act on it from its card.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Insert, copy and save icons

Insert Playful icons with language-aware output, copy them in nine formats, save SVG files, open the source and preview icon IDs on hover.

Insert from the library

1. Open a file and place one or more cursors, or select text to replace.
2. Select **Insert** on a card.
3. The icon is inserted at every cursor; multi-line output keeps the cursor line's indentation.

Without an open text editor you see "Open a text editor before inserting an SVG."

Smart insert

With `auricIconsPlayful.library.smartInsert` on (the default), **Insert** adapts to the file:

Editing in	Output
JavaScript, TypeScript, React JSX and TSX, MDX	JSX element
HTML, PHP, Handlebars, Vue, Svelte, Astro	HTML markup (the SVG)
Markdown	Markdown image
CSS, SCSS, Less	CSS background
JSON, JSONC, YAML, Python	Data URI
XML, SVG and any other language	Raw SVG

With the setting off, **Insert** uses raw SVG. **Smart Insert Icon** always adapts. From the keyboard: press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS) in a text editor, search and choose.

Copy in a chosen format

Run **Auric Icons: Playful: Copy Icon As... (JSX, CSS, Markdown, Data URI, Image tag)**, choose an icon and a format:

Format	What you get
Raw SVG	The SVG markup.
JSX element	The SVG with JSX attribute names.
React component	`export function IconName(props) { return (<svg {...props} …>); }`
Markdown image	`![Icon name](data:image/svg+xml,…)`
CSS background	`background-image: url("data:image/svg+xml,…");` plus no-repeat and contain sizing
Data URI	`data:image/svg+xml,…`
Image tag (self-contained)	`<img src="data:image/svg+xml,…" alt="Icon name" width="24" height="24">`
Icon name	The icon ID, such as `abstract-balance-0001`.
Catalog path	The icon's relative path in the catalog.

On a card, **Copy** copies raw SVG, **JSX** a JSX element and **URI** a data URI.

Save and open the source

- **Save** on a card, or **Save SVG As…**, opens a save dialog in your first workspace folder (or home folder) with the icon's file name. Select **Save icon**.
- The artwork, **Source** on a card, or **Open SVG Source** opens the SVG in a preview tab.

Other commands

Command	What it does
Search Icons	Choose an icon, then **Smart Insert (matches this file)**, **Insert SVG at Cursor**, **Copy SVG**, **Save SVG As…**, **Open SVG Source** or **Toggle Like**.
Insert SVG at Cursor	Choose and insert (smart insert applies when on).
Copy SVG	Choose and copy raw SVG.
Insert from Likes	Choose a liked icon and insert.
Insert from Recent	Choose a recent icon and insert. Also `Ctrl` + `Alt` + `R`.
Toggle Like for an Icon	Like or unlike an icon.
Clear Likes, Recents and Learned Recommendations	Confirm with **Clear** to reset.

Hover previews

Hover over a Playful icon ID, such as `abstract-balance-0001`, in HTML, CSS, SCSS, Less, Vue, Svelte, Astro, JavaScript, TypeScript, React JSX and TSX, JSON, JSONC, Markdown, PHP, Handlebars or XML. The hover shows a 48-pixel colour preview, the name and description, ID, category and tags, and **Open in Icon Library**. Turn it off with `auricIconsPlayful.library.hoverPreview`.

Result

The icon is in your file, on your clipboard or saved as an `.svg` file, and it appears under **Recent**.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Use Playful with Modern and Drawn

How Playful merges with Auric Icons: Modern and Drawn into one library, and how Auric Artisan Icon Studio takes over hosting.

Playful works by itself or beside **Auric Icons: Modern** and **Auric Icons: Drawn**. Installed together, they show one library.

What happens when they are co-installed

1. One extension becomes the **host** automatically. With the current versions, Modern hosts when it is installed; otherwise Drawn hosts. Playful hosts only when it is the only collection installed.
2. The host's **Icon Library** view (headed "Icon Studio") contains every installed collection.
3. A **Library** filter appears, and each card shows a badge naming its collection.

4. The **Variant** switcher offers every collection's variants. Playful's **Filled** and **Small** join Color, Mono, Outline and Duotone. Where a collection lacks the selected variant, its cards fall back to Color.
5. The other extensions hide their views, Command Palette entries and shortcuts, and forward their commands to the host.

This updates by itself when you install or remove a collection.

Likes and recents

They stay with the extension that owns each icon. Liking a Playful icon in the merged view stores it with Playful. If you uninstall Playful, its icons and history leave the merged view.

Hover previews

Playful previews only Playful icon IDs, so you never see two hovers.

With Auric Artisan Icon Studio

Auric Artisan Icon Studio already contains the Playful collection and, by default, hosts Playful, Modern and Drawn. Set `auricIconStudio.hostFamilyLibraries` to `false` to bring the Playful view back. Icon Studio keeps its own likes and recents.

See The merged Icon Studio in the Modern documentation for more detail.

Result

With several Auric collections installed, you browse Playful with the others in one library and can filter to it with **Library** › **Playful**.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Auric Icons: Playful reference

Every command, keybinding, setting, view, category and library control in Auric Icons: Playful 0.12.1.

This reference describes version 0.12.1 as published on the Visual Studio Marketplace.

Commands

Category in the Command Palette: **Auric Icons: Playful**.

Command	ID	What it does
Open Icon Library	`auricIconsPlayful.library.openExplorer`	Opens the **Icon Studio — Library** editor tab.
Search Icons	`auricIconsPlayful.library.searchIcons`	Choose an icon, then an action.
Insert SVG at Cursor	`auricIconsPlayful.library.insertSvg`	Choose an icon and insert it.
Smart Insert Icon (matches active language)	`auricIconsPlayful.library.smartInsert`	Choose an icon and insert with language-aware output.

Command	ID	What it does
Copy Icon As… (JSX, CSS, Markdown, Data URI, Image tag)	`auricIconsPlayful.library.copyAs`	Choose an icon and one of nine formats.
Insert from Likes	`auricIconsPlayful.library.showFavorites`	Choose a liked icon and insert.
Insert from Recent	`auricIconsPlayful.library.showRecents`	Choose a recent icon and insert.
Toggle Like for an Icon	`auricIconsPlayful.library.toggleFavorite`	Like or unlike an icon.
Clear Likes, Recents and Learned Recommendations	`auricIconsPlayful.library.resetPersonalization`	Confirm, then reset personalisation.
Copy SVG	`auricIconsPlayful.library.copySvg`	Choose an icon and copy raw SVG.
Save SVG As…	`auricIconsPlayful.library.saveSvg`	Choose an icon and save it as a file.
Open SVG Source	`auricIconsPlayful.library.openSource`	Choose an icon and open its SVG.

These commands are hidden from the Command Palette while another extension hosts the shared library; from keybindings they forward to the host.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command	When
`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`	Smart Insert Icon	Editor focused and Playful hosts the library
`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`	Insert from Recent	Editor focused and Playful hosts the library

In the view: `/` focuses search, `Escape` clears it or closes a dropdown, arrow keys move between dropdown options.

Settings

Setting	Type	Values or range	Default	What it does
`auricIconsPlayful.library.defaultVariant`	string	`auto`, `color`, `outline`, `filled`, `small`	`auto`	Variant for Command Palette actions; `auto` follows the file.
`auricIconsPlayful.library.pageSize`	number	24 to 240	72	Cards streamed per batch.
`auricIconsPlayful.library.smartInsert`	boolean	`true`, `false`	`true`	Language-aware output for Insert.
`auricIconsPlayful.library.recommendations.enabled`	boolean	`true`, `false`	`true`	Recommendations for the file you are editing.
`auricIconsPlayful.library.recommendations.limit`	number	4 to 24	12	Icons in the recommendation rail.
`auricIconsPlayful.library.showGuide`	boolean	`true`, `false`	`true`	Smart guide tips.
`auricIconsPlayful.library.density`	string	`comfortable`, `compact`	`comfortable`	Card size.
`auricIconsPlayful.library.hoverPreview`	boolean	`true`, `false`	`true`	Hover previews for icon IDs.

Views and menus

Place	Contents
Activity Bar	**Auric Icons: Playful** container with the **Icon Library** view (hidden while another extension hosts).
View title bar	**Open Icon Library**, **Smart Insert Icon**, **Insert from Likes**; **…** menu: **Clear Likes, Recents and Learned Recommendations**.
Editor tab	**Icon Studio — Library**.

Playful contributes no file icon theme.

Library contents

Item	Value
Icons	1,225
Variants	Color, Outline, Filled, Small
Categories	24

Categories: Lifestyle; Accessibility; Accounts; Actions; Admin Panel; Artificial Intelligence; Blog and Publishing; Business; Calendar and Time; Communication; Dashboard; Developer and Technology; Devices and Hardware; Navigation; E-commerce; Files; Folders; Form Elements; Location and Travel; Media; Reactions; Security and Privacy; Settings; Status.

Library controls

Control	Options
Search	Free text, synonyms, fuzzy ranking.
Library	**All libraries** or one collection (only when merged).
Category	**All categories** or one of 24.
Style	**All styles** or **Playful**.
Variant	**Auto**, **Color**, **Outline**, **Filled**, **Small**.
Preview surface	**Auto**, **Light**, **Dark**.
Chips	**All**, ✦ **For you**, ♥ **Likes**, ↺ **Recent**.
Card buttons	Artwork (open source), heart, **Insert**, **Copy**, **Save**, **JSX**, **URI**, **Source**.

Limits

Item	Limit
Recent icons	60
Liked icons	500
Quick Pick results	200 with a search term; 400 without
Smart guide tips	Up to 4

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Icons: Playful.

Symptom	Why it happens	What to do
The **Auric Icons: Playful** view is missing.	Auric Artisan Icon Studio, Modern or Drawn hosts the shared library, which already includes Playful.	Use the host's library and filter by **Library** › **Playful**, or set `auricIconStudio.hostFamilyLibraries` to `false` if Icon Studio is the host.
Playful commands are missing from the Command Palette.	Same reason.	Use the host's commands; keybindings still work.
`Ctrl` + `Alt` + `I` does nothing.	The editor is not focused, another extension hosts, or the keys are bound elsewhere.	Focus a text editor and check **Keyboard Shortcuts**.
"Open a text editor before inserting an SVG."	No text editor is available.	Open a file, click into it, insert again.
I chose Mono or Duotone and Playful cards look like Color.	Playful has no Mono or Duotone; it falls back to Color in a merged library.	Use Color, Outline, Filled or Small.
Auto shows Color in a CSS file.	Playful has no Mono variant, which Auto would use for stylesheets and data files.	Choose a variant explicitly if you prefer Outline or Filled.
I expected Playful Explorer file icons.	Those belong to Auric Artisan File Icons (the **Playful** Explorer style), a different set of artwork.	Install File Icons and choose **Playful** there.
The inserted format is not what I expected.	Smart insert chose a format for the file's language.	Use **Copy Icon As...**, or turn off `auricIconsPlayful.library.smartInsert`.
The library says the icon catalog is not bundled.	The installed files are incomplete.	Reinstall from the Marketplace.

If the problem continues, note your VS Code version, the extension version (0.12.1), the other Auric extensions installed and the steps to reproduce, then contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Frequently asked questions

Short answers to common questions about Auric Icons: Playful.

Are these emoji?

No. They are emoji-style SVG icons: real vector artwork you can insert, style and ship, not Unicode emoji characters. For Unicode emoji, see Auric Artisan Emoji.

What is the Small variant for?

Small is a separate, simplified drawing for 16 to 24 pixels. Details that would blur at small sizes are removed.

Is this the same as the Playful style in File Icons?

No. File Icons' Playful style is a set of 500 Explorer file and folder icons. This extension is a 1,225-icon library for your code and has no Explorer theme.

Why is there no file icon theme?

Playful's earlier gallery theme did not meet the 16-pixel silhouette, folder-pair and mapping standards of Auric Icons: Modern, which is the family theme. Install Modern alongside Playful for the Explorer theme.

Do I need Modern or Drawn too?

No. Playful works alone. Installing them together gives you one merged library.

Is it free and offline?

Yes. Free to use for any purpose (redistribution and resale are not permitted), with no network access or telemetry.

Can I change icon size or colour on insert?

Not in this extension. Auric Artisan Icon Studio has size and colour output settings and includes the Playful collection.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Auric Icons: Playful: other information

Contents of the supporting pages for Auric Icons: Playful.

This folder holds supporting pages for Auric Icons: Playful: terms, shortcuts, limits, a comparison with the other Auric icon extensions, the release history and privacy behaviour.

File	What it covers
glossary.md	Terms used in the library.
shortcuts.md	Keybindings and every command.
limits-and-accuracy.md	What the library can and cannot do.
related.md	Comparison with Modern, Drawn, Icon Studio, Icon Hub, File Icons, Emoji and Icon Pack.
release-notes.md	User-facing history.
privacy.md	What is stored and what uses the network.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Shortcuts and commands

Keybindings, in-view keys and every command in Auric Icons: Playful.

Keybindings

Action	Windows and Linux	macOS
Smart Insert Icon	`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`
Insert from Recent	`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`

Both need a focused text editor and are active only while Playful hosts the library.

Keys in the Icon Library

Key	Action
`/`	Focus search
`Escape`	Clear search; close a dropdown
`Arrow Up` / `Arrow Down`	Move between dropdown options

Commands

Type "Auric Icons: Playful" in the Command Palette: **Open Icon Library**, **Search Icons**, **Insert SVG at Cursor**, **Smart Insert Icon (matches active language)**, **Copy Icon As... (JSX, CSS, Markdown, Data URI, Image tag)**, **Insert from Likes**, **Insert from Recent**, **Toggle Like for an Icon**, **Clear Likes, Recents and Learned Recommendations**, **Copy SVG**, **Save SVG As...**, **Open SVG Source**.

See the reference for IDs and details.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Glossary

Definitions of the terms used in Auric Icons: Playful.

Term	Meaning
Auto variant	A variant choice that follows the language of the file you are editing.
Catalog path	The relative location of an icon file inside the bundled catalog.
Data URI	Text that embeds a file directly, starting `data:image/svg+xml,`.
Emoji-style icon	Expressive SVG artwork in the spirit of emoji. Not a Unicode emoji character.
Filled	A variant drawn with solid shapes.
Host	The one extension that shows the shared library when several Auric collections are installed.
Icon ID	The unique name of an icon, such as `abstract-balance-0001`.
Icon Studio	The heading of the shared library view, and also the name of the separate Auric Artisan Icon Studio extension.
Outline	A variant drawn in lines.
Preview surface	The light, dark or automatic backdrop behind library cards.
Small	A simplified redrawing for 16 to 24 pixels.
Smart guide	Tip cards at the top of the library that react to your file and usage.
Smart insert	Choosing the output format from the language of the file you are editing.
Variant	One rendering of an icon: Color, Outline, Filled or Small.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Limits and accuracy

What Auric Icons: Playful can and cannot do.

- The library contains 1,225 validated icons at the installed version; new icons come with updates.
- There is no Mono or Duotone variant, and no Explorer file icon theme.
- Icons are inserted as drawn: there are no size, colour or stroke controls for output.
- Smart insert picks a format from the file's language, not the exact cursor position. Check quotes and surrounding code.
- Recommendations are local rankings from the file you are editing and your likes and use, not an AI service.
- Hover previews recognise only Playful icon IDs (a hyphenated name ending in four digits).
- Some copy formats from the retired "Auric Artisan Icons" gallery (Vue single-file component, web component, CSS class and sprite) are not offered, because they depend on runtime files an extension cannot ship.
- Licence: free to use for any purpose; redistribution, republishing, resale and derivative releases are not permitted.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Privacy

What Auric Icons: Playful stores, where, and confirmation that it makes no network requests.

Auric Icons: Playful runs entirely inside VS Code. It has no telemetry, no account and no network access at runtime.

Data	Where
Settings	Your VS Code settings under `auricIconsPlayful.library.*`.
Likes (up to 500), recent icons (up to 60), learned preferences, dismissed tips	VS Code's extension storage on this machine.
Search, filters, variant and preview surface	The view's saved state in VS Code.

For recommendations and smart insert, the extension reads the language, file name and up to about 60 lines around the cursor of the active editor. This stays in memory and is never stored or sent.

Inserting changes your editor content, copying writes to the clipboard, and saving writes a file only where you choose in a save dialog.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Release notes

User-facing history of Auric Icons: Playful.

0.12.1

The current Marketplace version.

- Now covered by the Auric Artisan Free Use Licence 2.0 instead of MIT. It stays free to install and use for any purpose; redistribution, republishing to any marketplace, resale and derivative releases are no longer permitted.

0.12.0

- Marketplace README images now display correctly.
- Future Auric icon collections can join the shared library without every extension being updated.
- The library view, palette entries and shared shortcuts are disabled while another icon collection or Auric Artisan Icon Studio hosts the library.

0.11.0

- The catalog grew from 1,220 to all 1,225 icons after five pieces of artwork were repaired.
- New Marketplace artwork.
- The shared library gained keyboard and pointer enlargement and a remembered **Auto**, **Light** or **Dark** preview surface.

0.1.0

- Initial release: 1,220 icons in four variants (Color, Outline, Filled, Small) and 24 categories, with the Icon Library, smart search, smart insert (`Ctrl` + `Alt` + `I`), nine copy formats, hover previews, likes and recents (`Ctrl` + `Alt` + `R`) and per-file recommendations.
- Cooperates with Auric Icons: Modern and Drawn in one merged library.
- Succeeds the retired, never-published "Auric Artisan Icons" gallery extension. Its file icon theme is replaced by Auric Icons: Modern's.

VS CODE EXTENSIONS › AURIC ICONS: PLAYFUL

Related extensions and comparison

How Auric Icons: Playful compares with Modern, Drawn, Icon Studio, Icon Hub, File Icons, Emoji and the Icon Pack.

At a glance

Extension	Main job	Explorer file icon theme	Artwork	Network
Auric Icons: Playful	Emoji-style icon library.	No	1,225 icons	None
Auric Icons: Modern	Modern icon library plus the Modern file and folder icon theme.	Yes: **Auric Icons: Modern**	720 library icons; 408 theme icons	None
Auric Icons: Drawn	Warm Drawn icon library.	No	480 icons	None
Icon Studio	One offline studio for every Auric collection, emoji and seven open-source sets.	No	17,234 entries in 11 collections	None
Icon Hub	Seven open-source sets with colour and stroke controls, sprite and ZIP export, and CDN insertion.	No	12,895 icons	Optional metadata checks; CDN output loads from the web
File Icons	Explorer file and folder icons in three styles, plus File Icon Studio.	Yes: Facet, Playful and Modern	1,464 SVG assets	None
Emoji	Emoji Studio for Unicode emoji.	No	1,914 emoji	None
Icon Pack	Installs Modern, Playful, Drawn and Emoji together.	Through Modern	None of its own	Needs the Marketplace to install

Two different "Playful"s

- **Auric Icons: Playful** (this extension): 1,225 emoji-style icons for inserting into code.
- **Playful style in Auric Artisan File Icons**: 500 SVG file and folder icons for the Explorer.

They share a name but not artwork.

How the three collections merge

- Install Playful with Modern and/or Drawn and one extension hosts a single merged library, with a **Library** filter, collection badges and every collection's variants. See Use Playful with Modern and Drawn.
- Install Auric Artisan Icon Studio and, by default, it hosts all three instead.
- Likes and recent icons stay with the extension that owns each icon.

Other related pages

- Install and set up
- Icon Library on the website

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Auric Artisan Emoji

What the Auric Artisan Emoji extension is, who it is for, how to install it and where to find each guide.

Auric Artisan Emoji is a local emoji workspace for Visual Studio Code. It lets you search 1,914 emoji and put them straight into your code: from a searchable Quick Pick, from **Emoji Studio** in the sidebar or an editor tab, from shortcode suggestions as you type, or from a hover over a shortcode such as `:rocket:`.

Emoji Studio has four views (**Library**, **For you**, **Favorites** and **Recent**), nine categories, six skin-tone choices, quick filters and grid or list layouts. An inspector shows each emoji's shortcode, code points, emoji version, keywords and related emoji, and previews its text in six output formats: the emoji itself, a shortcode, Unicode code points, HTML entities, a JavaScript escape or a CSS escape. You can also collect several emoji into a sequence and insert them together.

The Studio can be shown in English or Hindi, and Hindi search words such as `दिल` or `धन्यवाद` find emoji. Everything is bundled and stays on your machine.

Who it is for

- Developers writing commit messages, documentation, UI strings, tests or chat bots that use emoji.
- Front-end developers who need the right escape for HTML, JavaScript or CSS.
- Hindi-speaking users who want a Hindi Emoji Studio.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-emoji
Extension ID	`auric-artisan.auric-artisan-emoji`
Current version	0.2.0
Requires	Visual Studio Code 1.85 or later
Price	Free to install and use; no account required

```
code --install-extension auric-artisan.auric-artisan-emoji
```

Emoji is also installed by the Auric Artisan Icon Pack.

Quick start

1. Install the extension.
2. Open a text file and place the cursor where the emoji should go.
3. Press `Ctrl` + `Alt` + `J` (`Cmd` + `Alt` + `J` on macOS) for **Quick Insert Emoji**.
4. Type a word, such as `rocket`, and press `Enter`. The emoji is inserted.
5. Type `:spa` in your file and accept the `:sparkles:` suggestion (the sparkles emoji).
6. Press `Ctrl` + `Alt` + `E` (`Cmd` + `Alt` + `E`) to open Emoji Studio beside your code.
7. In the Studio, select a skin tone, then select **Insert** on a card.

What you can do

- Search by name, keyword, exact emoji, shortcode, code point (such as `U+1F680`) or common Hindi words.
- Browse nine categories, and filter to curated, skin-tone-capable or newest emoji.
- Insert at every cursor in one undo step, with automatic spacing.
- Copy in six formats and set your default copy format.
- Choose a default skin tone (330 catalog entries support tones).
- Keep favorites and recent emoji, and get **For you** suggestions.
- Compose a sequence of up to 50 emoji and insert or copy it at once.
- Replace `:shortcodes:` in a selection with emoji, get completions after `:`, and insert from a code hover.
- Switch the Studio between English and Hindi.

In this folder

docs/

- Contents of docs/
- Getting started
- Insert from the editor
- Use Emoji Studio
- Inspector, skin tones and output formats
- Compose a sequence
- Interface language
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Auric Artisan Emoji guides

Contents of the Auric Artisan Emoji guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Artisan Emoji. Start with Getting started, then use the guide for how you prefer to work: from the editor, or from Emoji Studio.

File	What it covers
getting-started.md	Install, insert a first emoji, open Emoji Studio and tour it.
insert-from-the-editor.md	Quick Insert, shortcode completions, code hovers, replacing shortcodes, recent and favorite pickers.
emoji-studio.md	Search, views, categories, filters, layouts, cards, suggestions and Workspace overview.
inspector-tones-and-formats.md	The inspector, skin tones and the six output formats.
sequences.md	Collect several emoji and insert or copy them together.
interface-language.md	English and Hindi in the Studio.
reference.md	Every command, keybinding, setting, key, category and message.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Getting started with Auric Artisan Emoji

Install Auric Artisan Emoji, insert a first emoji, open Emoji Studio and tour its screen.

Before you begin

- You need Visual Studio Code 1.85 or later.
- The extension works offline and in untrusted and virtual workspaces. It never runs workspace code.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Artisan Emoji** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-artisan-emoji
```

For a local `.vsix`, use **Extensions** › **...** › **Install from VSIX...**.

2. Insert your first emoji

1. Open a text file and place the cursor.
2. Press `Ctrl` + `Alt` + `J` (`Cmd` + `Alt` + `J` on macOS), or right-click and choose **Quick Insert Emoji**.
3. Type `rocket` and press `Enter`. The rocket emoji is inserted.

3. Open Emoji Studio

- **Sidebar:** select **Auric Emoji** in the Activity Bar (view name **Emoji Studio**).
- **Editor tab:** press `Ctrl` + `Alt` + `E` (`Cmd` + `Alt` + `E`), run **Auric Artisan Emoji: Open Emoji Studio**, or select the **Emoji** item in the status bar.

The editor tab opens beside your code. The sidebar is compact, with visible card actions.

4. Tour the Studio

Area	Contents
Brand bar	"Auric Artisan / Emoji Studio", the website link, **Language** (**Auto**, **EN**, **हिं**) and **Offline · private**.
Brand bar buttons	**Customize Emoji Studio** (gear) and, in the sidebar, **Open Emoji Studio in an editor tab**.
Tabs	**01 Library**, **02 For you**, **03 Favorites**, **04 Recent**, with counts.
Workspace overview	Collapsed by default. Quick tools (**Quick insert**, **Replace shortcodes**, **Keyboard map** →), the **Active file** card and library statistics.
Section heading	For example "01 / Library — Find the feeling fast." with a hint.
Filters	**Search**, **Category** chips, **Skin tone**, and in the full Studio **Quick filter** and **Density**.
Results	A count, **Clear filters**, the **Compose** switch, the **Grid** / **List** layout and the emoji cards.
Suggested for you	A rail of suggestions with **Refresh**.
Footer	The website link, "Offline · no telemetry · Unicode Emoji …", **Keyboard map** and **Settings**.

5. Insert from the Studio

1. Keep your file open with the cursor in place.
2. In the Studio, type `party` in **Search**.
3. Hover over or focus a card and select **Insert**. The status line confirms, for example, "Inserted … party popper.", showing the emoji and its name.

If no editor is available, the Studio copies the emoji instead and tells you.

Result

You have inserted emoji from the Quick Pick and from Emoji Studio.

Next steps

- Insert from the editor
- Use Emoji Studio

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Use Emoji Studio

Search, browse and insert with Emoji Studio: views, categories, skin tones, quick filters, layouts, cards, suggestions and the Workspace overview.

Search

Type in **Search** (placeholder "Search 1,914 emoji — name, mood, :shortcode: or U+1F680"). Press `/` or `Ctrl` + `K` (`Cmd` + `K`) to focus it.

You can search by:

- name or keyword, such as `developer` or `thanks`;
- an exact emoji;
- a shortcode, such as `:rocket:`;
- a code point, such as `U+1F680`;
- common Hindi words, such as `दिल`, `रॉकेट` or `धन्यवाद`.

Views

Tab	Shows
01 Library	Every emoji matching the search and filters.
02 For you	Favorites first, then the emoji you use most, then curated picks.
03 Favorites	Emoji you starred. Press `F` or the heart on a card.
04 Recent	Everything you inserted or copied, most recent first.

In the narrow sidebar, **Favorites** and **Recent** buttons at the bottom switch views, and **Back to library** returns.

Filters

Filter	Options
Category	**All** or one of nine: Smileys & Emotion, People & Body, Animals & Nature, Food & Drink, Travel & Places, Activities, Objects, Symbols, Flags. **More** / **Fewer** expands the chip list.
Skin tone	**Yellow** (standard) or **Light**, **Medium light**, **Medium**, **Medium dark**, **Dark**. A note shows how many emoji it applies to.
Quick filter (full Studio)	**All**, **Curated**, **Skin tone** (tone-capable emoji only), **Newest**.
Density (full Studio)	**Compact**, **Comfortable**, **Showcase**.

Clear filters resets search and filters. The results line reads, for example, "42 emoji for "heart"".

Layout

In the full Studio, choose **Grid** or **List**. Cards load as you scroll ("144 of 1,914 loaded — keep scrolling"); use **Load more** if it appears. `auricEmoji.showNames` shows emoji names on cards when there is room.

Cards

Each card shows the emoji and, space permitting, its name. On hover or keyboard focus (always visible in the sidebar and on touch screens):

Action	What it does
Insert	Inserts into the active or last active editor, even if your click action is Copy.
Copy	Copies in your copy format.
…	Opens the inspector.
Heart	Adds to or removes from favorites.

Selecting the emoji itself (or pressing `Enter` on it) runs your **click action** (`auricEmoji.clickAction`): **Insert** (default), **Copy**, or **Insert and copy**. `Shift` + `Enter` copies.

Keyboard in the grid

Key	Action
Arrow keys	Move between cards
`Enter`	Click action
`Shift` + `Enter`	Copy
`Space`	Add to the sequence
`Backspace` (outside text boxes)	Remove the last sequence item
`F`	Favorite or unfavorite
`I`	Open or close the inspector
`Escape`	Close the open panel, then clear the sequence
`?`	Open the keyboard map

`Tab` reaches each card's buttons. Text boxes keep their normal editing keys.

Suggested for you

The **Suggested for you** rail (headed "For {file}" when a file is open) lists emoji based on the file and your recent use, each with a reason such as **In favorites**, **Used 3×** or **Curated**. Select **Refresh** for new suggestions.

Workspace overview

Expand **Workspace overview · active file & quick tools**:

Part	Contents
Hero	"Every emoji. Exactly where you type." with **Quick insert**, **Replace shortcodes** and **Keyboard map** →.
Active file	The target file and language, line and cursor count, skin tone, completion state and click action.

Part	Contents
Active file buttons	**Insert {emoji} again**, **Replace :shortcodes: in selection**, **Clear history**.
Metrics	**Emoji** (with category count), **Skin-tone ready**, **Favorites** (kept on this machine only), **Recent** (ranked by use and recency).

Clear history removes recent emoji and last-emoji recall. Favorites are kept.

Studio settings

- **Customize Emoji Studio** (gear) and **Settings** in the footer open Settings filtered to this extension.
- `auricEmoji.closeStudioAfterInsert` closes the full Studio after a successful insertion.
- `auricEmoji.gridDensity` sets the default density.
- `auricEmoji.statusBar.enabled` shows or hides the **Emoji** status bar button.

Result

You can find any emoji quickly, in your tone, and insert or copy it with one click or key.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Insert from the editor

Insert emoji without opening the Studio, using Quick Insert, shortcode completions, code hovers, shortcode replacement and the recent and favorite pickers.

Quick Insert

1. Place the cursor (or several cursors) in a text file.
2. Press `Ctrl` + `Alt` + `J` (`Cmd` + `Alt` + `J`), run **Auric Artisan Emoji: Quick Insert Emoji**, or right-click › **Quick Insert Emoji**.
3. Search names, shortcodes, categories or Unicode, and choose an emoji.

Quick Insert uses your preferred skin tone and spacing.

Shortcode completions

Type `:` followed by part of a name. Suggestions appear:

```
:spa      → sparkles     :sparkles:
:rock     → rocket       :rocket:
:thinking → thinking face :thinking_face:
```

Accept a suggestion to insert the emoji. Turn completions off with `auricEmoji.completions.enabled`.

Hover over a shortcode

1. Write a complete shortcode with both colons, such as `:rocket:` or `:waving_hand:`.
2. Hover over it. The hover shows a preview.

3. Choose **Insert emoji**, **JavaScript** or **HTML**.

```
// Before: hover over :rocket: inside the string
const launch = ":rocket:";

// After choosing Insert emoji
const launch = "(the rocket emoji character)";
```

- Only the hovered shortcode is replaced, in one undo step. Quotes, other cursors and surrounding code are untouched, and no whitespace is added.
- The emoji uses the skin tone shown in the preview.
- If the document changes before you click, hover again for a fresh action. An action cannot be reused after it has inserted.
- Unlike Studio insertion, a failed hover edit does not fall back to the clipboard.
- Needs VS Code's editor hovers and `auricEmoji.hover.enabled` to be on.

Replace several shortcodes

1. Select text containing shortcodes. With no selection, the current line is used.
2. Run **Auric Artisan Emoji: Replace Emoji Shortcodes in Selection**, or right-click › **Replace Emoji Shortcodes in Selection** (shown when text is selected).

```
Before: Ready to ship :rocket: :sparkles:
After:  Ready to ship (rocket emoji) (sparkles emoji)
```

Unknown tokens are left unchanged. If none are recognised, you see "No recognized :emoji_shortcodes: were found." Without an editor you see "Open a text editor before replacing emoji shortcodes."

Recent, favorite and last emoji

Command	What it does
Insert Recent Emoji	Pick from your recent emoji and insert.
Insert Favorite Emoji	Pick from your favorites and insert.
Copy Last Emoji Again	Copy the last emoji you used. If none, "Use an emoji first, then Copy Last Emoji Again."

If a list is empty you see "…: no saved emoji yet."

Spacing

`auricEmoji.insertSpacing` controls editor insertion:

Value	Behaviour
`auto` (default)	Adds a space only when adjacent text would touch the emoji.
`space`	Appends one space.
`none`	Inserts only the emoji, for exact string contents.

Hover replacement leaves surrounding whitespace unchanged whatever this setting says.

Result

You can add emoji anywhere you type without leaving the editor.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Inspector, skin tones and output formats

Inspect an emoji, preview and set skin tones, and choose among the six output formats.

Open the inspector

Select a card's **…** button, or focus an emoji and press `I`. In the narrow sidebar it opens as a sheet. Press `I` or `Escape`, or select **Close inspector**, to close it.

The inspector shows:

Part	Contents
Stage	A large preview.
Name and shortcode	With **Copy shortcode**.
Pills	Emoji version (for example "Emoji 13.0"), **Skin tone** if tone-capable, **Favorite** if starred.
Skin tone	Previews of each tone.
Copy as	The six formats with their exact output.
Keywords	Search keywords.
Actions	**Insert**, **Copy**, **Favorite** / **Remove favorite**.
Also try	Related emoji.

Skin tones

Choose between **Yellow** (the standard appearance) and five tones: **Light**, **Medium light**, **Medium**, **Medium dark** and **Dark**. 330 catalog entries support tone changes.

- Set the default with the **Skin tone** filter, or `auricEmoji.preferredSkinTone` (`none`, `medium`, `light`, `mediumLight`, `mediumDark`, `dark`).
- In the inspector, select a tone to preview it ("Previewing {tone}. Your default is {default}."), then **Make {tone} my default**.
- Emoji that do not change with tone say "This emoji does not change with skin tone."

Your tone applies to card previews, insertion, Quick Insert, completions and shortcode replacement. Favorites and recents store the emoji, not the tone, so changing tone does not create duplicates.

The six output formats

Format	Setting value	Rocket example	Intended use
Emoji	`emoji`	The rocket character itself	Text and string contents
Shortcode	`shortcode`	`:rocket:`	Systems that understand emoji shortcodes
Unicode	`unicode`	`U+1F680`	Code-point references and documentation
HTML entities	`html`	`🚀`	HTML source
JavaScript escape	`javascript`	`\u{1F680}`	JavaScript and TypeScript string contents
CSS escape	`css`	`\1F680`	CSS strings, such as `content`

In the **Copy as** list you can:

- copy a single format (**Copy as {format}**);
- use a format for this inspector's **Insert** and **Copy** (**Use {format} for Insert and Copy**);
- make a format your default copy format (**Make {format} my copy format**), which sets `auricEmoji.copyFormat`.

These are text representations, not complete snippets. Add the surrounding syntax your language needs; JavaScript code-point escapes, for example, are not valid JSON escapes.

The ledger also explains your insert spacing, for example "Insert spacing is Auto — a space is added only when text would touch the emoji."

Result

You know exactly what text will be inserted or copied, in the tone and format you want.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Interface language (English and हिन्दी)

Show Emoji Studio in English or Hindi and search with Hindi words.

Choose a language

- In Emoji Studio's brand bar, use the **Language** switch: **Auto**, **EN** or **हिं**.
- Run **Auric Artisan Emoji: Choose Interface Language…** (also in the sidebar's title-bar **…** menu).
- Set `auricEmoji.interfaceLanguage` to `auto`, `en` or `hi`.

Value	Behaviour
`auto` (default)	Follows VS Code's display language: Hindi locales use Hindi; others use English.
`en`	Always English in Emoji Studio.
`hi`	Always Hindi in Emoji Studio.

Changing the language keeps your search, filters and sequence.

What changes

Part	Language
Emoji Studio controls, categories, tones, formats and announcements	Your `auricEmoji.interfaceLanguage` choice.
Command Palette titles, Settings descriptions and notifications	VS Code's display language (Hindi versions are included).
Emoji names, shortcodes and code points	Their catalog spelling, always.

Hindi search

Common Hindi words find emoji whatever the interface language, for example `दिल` (heart), `रॉकेट` (rocket) and `धन्यवाद` (thanks).

Result

Emoji Studio appears in your chosen language, and Hindi words help you find emoji.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Compose a sequence

Collect up to 50 emoji in Emoji Studio and insert or copy them together.

A **sequence** is a short list of emoji you collect and then insert or copy in one go, for example rocket, sparkles and party popper.

Build a sequence

Use any of these:

- Focus an emoji and press `Space`.
- `Shift`-click an emoji's preview.
- In the full Studio, turn on **Compose**. The switch shows "Composing · {count}", and selecting emoji adds them.

The **Sequence** tray appears with the count and the emoji. Cards show "In the sequence {n}×". A sequence holds up to 50 emoji, repeats included; beyond that you see "A sequence holds up to 50 emoji."

Edit it

- `Backspace` (outside text boxes) removes the last emoji.
- Select an emoji in the tray to remove it.
- `Escape` clears the sequence (after closing any open panel), or select **Clear**.

Use it

Button	Result
Insert sequence	Inserts the emoji characters at your cursors. "Inserted {n} emoji: …"

Button	Result
Copy sequence	Copies in your copy format. "Copied {n} emoji: …"

When copying, shortcodes, Unicode labels and CSS escapes are separated by spaces; emoji, HTML entities and JavaScript escapes are joined directly. If no editor is active, **Insert sequence** copies instead.

The sequence is restored if the Studio reloads, and changing the interface language keeps it.

Result

Your collected emoji are in your file or on your clipboard in one step.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Auric Artisan Emoji reference

Every command, keybinding, setting, key, view, category, format and message in Auric Artisan Emoji 0.2.0.

This reference describes version 0.2.0 as published on the Visual Studio Marketplace.

Commands

Category in the Command Palette: **Auric Artisan Emoji**.

Command	ID	What it does
Open Emoji Studio	`auricEmoji.openStudio`	Opens the full Studio in an editor tab.
Quick Insert Emoji	`auricEmoji.quickPick`	Searchable Quick Pick; inserts the chosen emoji.
Insert Recent Emoji	`auricEmoji.insertRecent`	Quick Pick of recent emoji.
Insert Favorite Emoji	`auricEmoji.insertFavorite`	Quick Pick of favorites.
Copy Last Emoji Again	`auricEmoji.copyLast`	Copies the last emoji used.
Replace Emoji Shortcodes in Selection	`auricEmoji.replaceShortcodes`	Replaces known `:shortcodes:` in the selection or current line.
Clear Emoji History	`auricEmoji.clearHistory`	Clears recents and last-emoji recall; keeps favorites.
Focus Emoji Search	`auricEmoji.focusSearch`	Focuses the Studio search box.
Choose Interface Language…	`auricEmoji.chooseInterfaceLanguage`	Auto, English or Hindi for the Studio.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command
`Ctrl` + `Alt` + `E`	`Cmd` + `Alt` + `E`	Open Emoji Studio
`Ctrl` + `Alt` + `J`	`Cmd` + `Alt` + `J`	Quick Insert Emoji

Both work anywhere in VS Code.

Keys inside Emoji Studio

Key	Action
`/` or `Ctrl` + `K` (`Cmd` + `K`)	Focus search
Arrow keys on a preview	Move through the grid
`Enter` on a preview	Click action
`Shift` + `Enter` on a preview	Copy
`Space` on a preview	Add to the sequence
`Backspace` outside text boxes	Remove the last sequence item
`F` on a preview	Toggle favorite
`I` on a preview	Open or close the inspector
`Escape`	Close the open panel, or clear the sequence
`?`	Open the keyboard map

Settings

Setting	Type	Values or range	Default	What it does
`auricEmoji.interfaceLanguage`	string	`auto`, `en`, `hi`	`auto`	Emoji Studio language.
`auricEmoji.clickAction`	string	`insert`, `copy`, `insertAndCopy`	`insert`	Primary action when an emoji is chosen.
`auricEmoji.insertSpacing`	string	`none`, `space`, `auto`	`auto`	Spacing added on editor insertion.
`auricEmoji.copyFormat`	string	`emoji`, `shortcode`, `unicode`, `html`, `javascript`, `css`	`emoji`	Format used by copy actions.
`auricEmoji.preferredSkinTone`	string	`none`, `medium`, `light`, `mediumLight`, `mediumDark`, `dark`	`none`	Default tone; `none` is standard yellow.
`auricEmoji.gridDensity`	string	`compact`, `comfortable`, `showcase`	`comfortable`	Card density.
`auricEmoji.showNames`	boolean	`true`, `false`	`true`	Show names on cards when space allows.
`auricEmoji.closeStudioAfterInsert`	boolean	`true`, `false`	`false`	Close the full Studio after inserting.
`auricEmoji.maxRecents`	integer	8 to 200	48	Recent emoji kept.
`auricEmoji.hover.enabled`	boolean	`true`, `false`	`true`	Preview and insert actions on `:shortcode:` hovers.
`auricEmoji.completions.enabled`	boolean	`true`, `false`	`true`	Completions after typing `:`.
`auricEmoji.statusBar.enabled`	boolean	`true`, `false`	`true`	The **Emoji** status bar button.

Views and menus

Place	Contents
Activity Bar	**Auric Emoji** container with the **Emoji Studio** view.
View title bar	**Open Emoji Studio**, **Focus Emoji Search**; **...** menu: **Choose Interface Language...**.
Editor right-click	**Quick Insert Emoji**; **Replace Emoji Shortcodes in Selection** (with a selection).
Status bar	**Emoji** (smiley icon), tooltip "Open Auric Emoji Studio".
Editor tab	**Emoji Studio**, beside the active editor.

Studio controls

Control	Options
Tabs	**01 Library**, **02 For you**, **03 Favorites**, **04 Recent**
Category	**All**, Smileys & Emotion, People & Body, Animals & Nature, Food & Drink, Travel & Places, Activities, Objects, Symbols, Flags
Skin tone	Yellow, Light, Medium light, Medium, Medium dark, Dark
Quick filter	All, Curated, Skin tone, Newest
Density	Compact, Comfortable, Showcase
Layout	Grid, List
Compose	On or off
Card	Insert, Copy, ... (inspector), heart
Sequence tray	**Copy sequence**, **Insert sequence**, **Clear**
Inspector	Tones, **Make {tone} my default**, **Copy as** ledger, **Make {format} my copy format**, **Insert**, **Copy**, **Favorite**, **Also try**
Workspace overview	**Quick insert**, **Replace shortcodes**, **Keyboard map →**, **Insert {emoji} again**, **Clear history**
Footer	**Keyboard map**, **Settings**

Catalog

Item	Value
Emoji	1,914
Categories	9
Tone-capable entries	330
Sequence length	Up to 50

Messages you may see

Message	Meaning
No editor was active, so … was copied.	Studio insertion fell back to the clipboard.
… copied because no writable editor was active.	A command's insertion fell back to the clipboard.
…: no saved emoji yet.	The recent or favorite list is empty.
Use an emoji first, then Copy Last Emoji Again.	Nothing to copy yet.
Open a text editor before replacing emoji shortcodes.	No editor for shortcode replacement.
No recognized :emoji_shortcodes: were found.	Nothing to replace.
A sequence holds up to 50 emoji.	The sequence is full.
Recent emoji cleared.	History was cleared.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Artisan Emoji.

Symptom	Why it happens	What to do
No hover action on a shortcode.	The shortcode is incomplete or not in the catalog, or hovers are off.	Use a complete shortcode with both colons, such as `:rocket:`. Check VS Code's editor hover setting and `auricEmoji.hover.enabled`.
A hover no longer inserts.	The document changed, or the action was already used.	Keep the editor visible and writable, then hover again.
The Studio copied instead of inserting.	No writable editor was available, or your click action is Copy.	Open a writable editor and place the cursor, then return to the Studio. Check **Workspace overview** for the target, and `auricEmoji.clickAction`.
No suggestions after typing `:`.	Completions are off, or VS Code's suggestions are disabled for this language.	Turn on `auricEmoji.completions.enabled`.
Emoji look different or appear as boxes.	Emoji are drawn by your system's emoji font; older fonts lack newer emoji.	Update your operating system's emoji font, or choose an older emoji.
Extra spaces appear around inserted emoji.	`insertSpacing` is `auto` or `space`.	Set `auricEmoji.insertSpacing` to `none`.
A JavaScript escape breaks my JSON.	`\u{…}` code-point escapes are JavaScript syntax, not JSON.	Use the emoji character or a JSON-compatible escape.
Command titles are English although the Studio is Hindi.	Native VS Code labels follow VS Code's display language.	Run **Configure Display Language** if you want Hindi commands.
Remote or virtual file cannot be edited.	Editing depends on the file system provider and the document being writable.	Check that the file is writable; use Copy instead.
The **Emoji** status bar button is missing.	It is turned off.	Turn on `auricEmoji.statusBar.enabled`.

If the problem continues, note your VS Code version, the extension version (0.2.0) and the steps to reproduce, then contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Frequently asked questions

Short answers to common questions about Auric Artisan Emoji.

Does it need the internet?

No. The catalog and Studio are bundled. There is no telemetry and no account.

Why do emoji look different on another computer?

Emoji are drawn by each system's emoji font. The characters are the same; the artwork differs.

Does clearing history remove favorites?

No. **Clear Emoji History** removes recent emoji and last-emoji recall only.

Can I change the default skin tone?

Yes: the **Skin tone** filter, **Make {tone} my default** in the inspector, or `auricEmoji.preferredSkinTone`.

What is the difference between Insert and the click action?

The card's **Insert** button always inserts. Selecting the emoji itself (or pressing `Enter`) runs your click action, which can be Insert, Copy, or Insert and copy.

Is this the same as the emoji in Icon Studio?

Icon Studio also includes 1,914 emoji for browsing alongside icons. This extension adds the full emoji workflow: sequences, completions, shortcode replacement, code hovers, six copy formats and a Hindi interface.

Are Auric Icons: Playful icons emoji?

No. Playful is a library of emoji-style SVG artwork, not Unicode emoji characters.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Auric Artisan Emoji: other information

Contents of the supporting pages for Auric Artisan Emoji.

Supporting pages for Auric Artisan Emoji: terms, shortcuts, limits, a comparison with other Auric extensions, the release history and privacy behaviour.

File	What it covers
glossary.md	Terms used in Emoji Studio.
shortcuts.md	Keybindings, Studio keys and commands.
limits-and-accuracy.md	What the extension can and cannot do.
related.md	Comparison with Icon Studio, the Auric icon collections and other extensions.
release-notes.md	User-facing history.
privacy.md	What is stored and what uses the network.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Shortcuts and commands

Keybindings, Emoji Studio keys and every command in Auric Artisan Emoji.

Keybindings

Action	Windows and Linux	macOS
Open Emoji Studio	`Ctrl` + `Alt` + `E`	`Cmd` + `Alt` + `E`
Quick Insert Emoji	`Ctrl` + `Alt` + `J`	`Cmd` + `Alt` + `J`

Keys in Emoji Studio

Key	Action
`/` or `Ctrl` + `K` (`Cmd` + `K`)	Focus search
Arrow keys	Move between cards
`Enter`	Click action
`Shift` + `Enter`	Copy
`Space`	Add to sequence
`Backspace`	Remove last sequence item
`F`	Toggle favorite
`I`	Toggle inspector
`Escape`	Close panel, then clear sequence
`?`	Keyboard map

Commands

Type "Auric Artisan Emoji" in the Command Palette: **Open Emoji Studio**, **Quick Insert Emoji**, **Insert Recent Emoji**, **Insert Favorite Emoji**, **Copy Last Emoji Again**, **Replace Emoji Shortcodes in Selection**, **Clear Emoji History**, **Focus Emoji Search**, **Choose Interface Language...**.

With VS Code's display language set to Hindi, these titles appear in Hindi.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Glossary

Definitions of the terms used in Auric Artisan Emoji.

Term	Meaning
Click action	What happens when you select an emoji or press `Enter` on it: insert, copy, or both.
Code point	The Unicode number of a character, written like `U+1F680`.
Copy format	The text form used when copying: emoji, shortcode, Unicode, HTML, JavaScript or CSS.
CSS escape	A backslash form such as `\1F680`, valid inside CSS strings.
Curated	A hand-picked selection of commonly useful emoji.
HTML entity	A form such as `🚀` that HTML turns into the character.
Inspector	The panel showing an emoji's details and every output format.
JavaScript escape	A form such as `\u{1F680}`, valid inside JavaScript and TypeScript strings.
Sequence	A list of up to 50 emoji collected to insert or copy together.
Shortcode	A text name between colons, such as `:rocket:`.
Skin tone	A modifier changing the skin colour of emoji that support it.
Yellow	The standard, unmodified appearance of tone-capable emoji.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Limits and accuracy

What Auric Artisan Emoji can and cannot do.

- The catalog has 1,914 entries in nine categories at the installed version; new emoji arrive with updates.
- Glyph appearance depends on your operating system's emoji fonts. Newer emoji may not display with older fonts.
- 330 entries support skin tones; others ignore the tone setting.
- Output formats are text representations, not complete code snippets. JavaScript code-point escapes are not JSON escapes.
- Hover actions are single-use and expire when the document changes; a failed hover edit does not fall back to the clipboard.
- Shortcode replacement leaves unknown tokens unchanged.
- Hindi search terms help you find emoji; names, shortcodes and code points stay in their catalog spelling.
- Editing remote or virtual files depends on the file system provider.
- Recent emoji are limited by `auricEmoji.maxRecents` (8 to 200, default 48). A sequence holds up to 50 emoji.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Privacy

What Auric Artisan Emoji stores, where, and confirmation that it makes no network requests.

The emoji catalog and Studio are bundled. The extension has no telemetry, no account requirement and no runtime package dependencies. It does not run workspace code or fetch remote content for its interface. Website links open in your browser only when you select them.

What is stored

Data	Where
Favorites, recents and last emoji	VS Code's extension storage on this machine.
Preferences	Your VS Code settings under `auricEmoji.*`.
Studio interface state and sequences	The Studio's saved state in VS Code.

Document contents are not stored as history.

Hover actions

When you hover over a shortcode, the extension briefly keeps that shortcode and its document position in memory so it can check your explicit edit. Hover links allow only the extension's insertion command, with single-use tickets and document version checks. Nothing is written to disk or sent anywhere.

Security

The Studio uses a restrictive content security policy and builds its interface without executing imported content.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Release notes

User-facing history of Auric Artisan Emoji.

0.2.0 — 18 September 2026

The current Marketplace version: a connected workspace from search to code.

Redesigned workspace

- Compact brand bar, sticky tabs and consistent cards in the Studio and sidebar.
- Search and the library first; a collapsible **Workspace overview** holds active-file details, quick tools and statistics.
- **Library**, **For you**, **Favorites** and **Recent** views, category chips, tone swatches, quick filters, grid and list layouts and density controls.
- A compact sidebar with visible card actions; hover and focus actions in the full Studio.
- Native scrolling, visible focus, light, dark and high-contrast support, and reduced motion.

Direct editor insertion

- Hover over a complete shortcode such as `:rocket:` to insert the emoji, a JavaScript escape or an HTML entity, in one undo step.
- New setting `auricEmoji.hover.enabled`.
- Workspace overview shows the insertion target, cursor details and preferences, with last-emoji insertion, shortcode replacement and history tools.

Inspector and sequences

- A rebuilt inspector with a large preview, tone variants, a default-tone control, keywords and related emoji.
- JavaScript and CSS escapes added, making six formats; the ledger previews exact output and can set the default copy format.
- Sequences: add with `Space`, `Shift`-click or Compose mode; insert or copy together; restored after a reload.
- A keyboard map, search shortcuts, grid navigation and favorite and inspector keys.

English and Hindi

- **Auto / English / हिन्दी** controls, `auricEmoji.interfaceLanguage` and **Choose Interface Language...**.
- Localised Studio, Hindi manifest, settings and notifications.
- Hindi search terms such as `दिल`, `रॉकेट` and `धन्यवाद`.
- Language changes keep your search, filters and sequence.

0.1.0 — 8 August 2026

- Preview release numbering set to 0.1.0.
- Corrected preview images and added an animated product preview.

Earlier recorded versions

These labels predate the move to 0.1.0 and are kept as history.

1.1.0 — 2 August 2026

- Studio and sidebar refresh with labelled category, tone and density selectors.
- All, For you, Favorites and Recent scopes, refreshable suggestions and explicit card actions.
- The inspector as an on-demand sheet in narrow sidebars.
- Yellow restored as the default tone.

1.0.0 — 2 August 2026

- The first Activity Bar gallery and full Studio with 1,914 emoji in nine categories.
- Favorites, ranked recents, last-emoji recall and six tone choices.
- Insertion, clipboard formats, multi-cursor edits, automatic spacing, Quick Pick and status bar access.
- Colon-triggered completion and shortcode replacement.

VS CODE EXTENSIONS › AURIC ARTISAN EMOJI

Related extensions and comparison

How Auric Artisan Emoji compares with Icon Studio, the Auric icon collections and other Auric extensions.

Extension	Main job	Emoji	Network
Auric Artisan Emoji	The full emoji workflow: Studio, sequences, completions, hovers, shortcode replacement, six formats, Hindi interface.	1,914	None
Icon Studio	One offline studio for Auric icons, emoji and seven open-source sets. Emoji copy as character, shortcode, code points or HTML entities.	1,914	None
Auric Icons: Playful	1,225 emoji-style SVG icons (artwork, not Unicode emoji).	None	None
Icon Pack	Installs Emoji with Modern, Playful and Drawn.	Through Emoji	Marketplace to install

Emoji keeps its own view and commands even when Icon Studio hosts the Auric icon collections.

For the full comparison of all Auric icon extensions, see Icon Studio: Related extensions and comparison.

Other related products

- Auric Artisan Theme — shares the same Auto, English and Hindi interface switch.
- Install and set up

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Auric Artisan Icon Studio

What Auric Artisan Icon Studio is, who it is for, how to install it and where to find each guide.

Auric Artisan Icon Studio is one offline workspace in VS Code for finding icons and emoji and putting them into your code. It bundles every Auric Artisan icon collection (Modern, Drawn and Playful), 1,914 emoji and seven open-source icon sets: Phosphor, Tabler, Lucide, Remix, Heroicons, Feather and Material. In total it searches 17,234 catalog entries in 11 collections.

You can browse in a compact sidebar or open a larger editor-tab Studio. Search understands names, tags and synonyms (for example, `delete` finds trash icons), and Hindi search aliases. **Smart insert** chooses an output format from the file you are editing: a JSX element in React, a string constant in JavaScript, markup in HTML, a CSS background in stylesheets, a Markdown image in Markdown, and a data URI in JSON, YAML or Python. You can also copy an icon in an explicit format or save SVG artwork to a file.

Icon Studio does not change your Explorer file and folder icons; that is the job of Auric Artisan File Icons. It also includes all the Auric artwork itself, so you do not need to install the individual Auric collection extensions.

Who it is for

- Front-end and full-stack developers who insert icons into components, pages, stylesheets or documentation.
- Designers who want to compare icon styles across many sets in one place.
- Anyone who wants emoji with skin tones, shortcodes and code points without leaving the editor.
- Teams that need a tool that works fully offline, with no account or telemetry.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-icon-studio
Extension ID	`auric-artisan.auric-artisan-icon-studio`
Current version	0.1.0
Requires	Visual Studio Code 1.85 or later
Price	Free to install and use

```
code --install-extension auric-artisan.auric-artisan-icon-studio
```

Quick start

1. Install the extension.
2. Open the file you want to add an icon to and place the cursor where it should go.
3. Select **Icon Studio** in the Activity Bar.
4. Type a word into **Search your library**, for example `rocket`.
5. Optionally expand **Refine library** and pick a **Collection**, **Category** or **Style**.
6. Select **Insert** on a card. The icon is inserted in a format that suits the file.
7. Press the heart on icons you reuse so they appear under **Likes**.

What you can do

- Search 17,234 entries across 11 collections, with synonyms and Hindi aliases.
- Filter by collection, category, style and emoji skin tone.
- Keep **Likes** and **Recent** icons, and get **For you** recommendations based on the file you are editing.
- Insert at multiple cursors, with continuation lines indented to match.
- Copy SVG artwork as SVG markup, JSX, a React component, a JavaScript string constant, a self-contained image tag, a Markdown image, CSS, a data URI or the icon ID.
- Copy Material glyphs as HTML or JSX spans, a React component, a string constant, CSS or the ligature name.
- Copy emoji as the character, a shortcode, Unicode code points or HTML entities.
- Save SVG artwork to a file.
- Preview Auric icon IDs by hovering over them in code.
- Insert a specific icon from a keybinding or macro.
- Show one Icon Studio instead of separate libraries when the Auric Modern, Drawn or Playful extensions are also installed.

In this folder

docs/

- Contents of docs/
- Getting started
- Search and browse
- Insert, copy and save
- Use the Command Palette and keybindings
- Work with the Auric collection extensions
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Icon Studio guides

Contents of the Icon Studio guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Artisan Icon Studio. Start with Getting started, then use the task guides. The reference lists every command, setting, keybinding, collection, format and control.

File	What it covers
getting-started.md	Install, open the sidebar or editor tab, tour the screen and insert a first icon.
search-and-browse.md	Search, the four views, filters, styles, skin tones and infinite scrolling.
insert-copy-and-save.md	Smart insert, explicit formats, saving SVG files and hover previews.
commands-and-keybindings.md	Quick Pick workflows, shortcuts and inserting a fixed icon from a keybinding.
auric-collections.md	How Icon Studio works alongside Auric Icons: Modern, Drawn and Playful.
reference.md	Every command, setting, keybinding, collection, style and output format.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Getting started with Icon Studio

Install Icon Studio, open it in the sidebar or an editor tab, tour the screen and insert your first icon.

Before you begin

- You need Visual Studio Code 1.85 or later.
- Icon Studio works offline and in untrusted and virtual workspaces. Files are only written when you save an icon through a save dialog.
- All collections are bundled. You do not need the individual Auric collection extensions.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Artisan Icon Studio** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-artisan-icon-studio
```

2. Open Icon Studio

- **Sidebar:** select **Icon Studio** in the Activity Bar. The view is compact and starts with the filters collapsed, leaving room for artwork.
- **Editor tab:** run **Auric Icon Studio: Open Icon Studio**, or select the ↗ button (**Open Icon Studio in an editor tab**) in the sidebar header. The tab opens beside your editor with the filters expanded.

Opening the editor tab again brings the existing tab forward and focuses the search box.

3. Tour the screen

Area	What it contains
Header	The Auric Artisan mark, "Icon Studio / Creative workspace", an **Offline library** badge and, in the sidebar, the ↗ button.
View chips	**Library**, **For you**, **Likes** and **Recent**. Likes and Recent show a count.
Introduction	"Small details. Distinct character."
Search	**Search your library** with a `/` hint. The placeholder shows the number of entries.
Refine library	Collapsible: **Collection**, **Category**, and when relevant **Style** and **Skin tone**.
Finder footer	"Choose a card to insert. Copy or As… for more formats." and **Reset filters**.
Results	**Explore the collection**, a count such as "Showing 96 of 17,234 icons", and the card grid.
Footer	"Local artwork · No account · No telemetry".

Each card shows the artwork, a heart button, a collection badge, the name, the category, three main buttons (**Insert**, **Copy**, **As…**) and three quick format buttons.

4. Insert your first icon

1. Open a writable file, for example a `.tsx` component, and place the cursor where the icon should go.
2. In Icon Studio, press `/` to focus the search and type `rocket`.
3. Select **Insert** on a card, or select the artwork itself.
4. The icon is inserted at every cursor. In a React file it arrives as a JSX element; in CSS as a background; in Markdown as an image.
5. A short status message confirms, for example "Rocket inserted as JSX."

If no editor is open, Icon Studio copies the output instead and tells you: "copied — open an editor to insert it."

5. Keep useful icons close

- Select the heart on a card to add it to **Likes**. Select it again to remove it.
- Every icon you insert, copy or save is added to **Recent**.
- **For you** ranks icons for the file you are editing and the icons you like and use.

Result

You have inserted an icon in the right format for your file, and Icon Studio has started learning which icons you use.

Next steps

- Search and browse
- Insert, copy and save
- Commands and keybindings

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Work with the Auric collection extensions

How Icon Studio, Auric Icons: Modern, Drawn and Playful share one library interface, and how to keep their separate views if you prefer.

The Auric artwork is also published as three separate extensions: **Auric Icons: Modern**, **Auric Icons: Drawn** and **Auric Icons: Playful**. Each has its own Icon Library view. Icon Studio already contains all three collections, so it is designed to avoid showing the same icons twice.

Default: Icon Studio hosts the collections

With `auricIconStudio.hostFamilyLibraries` set to `true` (the default):

- The separate Icon Library views of Modern, Drawn and Playful are hidden, together with their Command Palette entries and their `Ctrl` + `Alt` + `I` and `Ctrl` + `Alt` + `R` shortcuts.
- Icon Studio's own view, commands and shortcuts are active.
- Modern's Explorer file icon theme is not affected. It keeps working.
- Hovering over an Auric icon ID is still answered by the collection extension that owns it, so you see one preview, not two.

Keep the separate collection views

1. Open Settings and search for `auricIconStudio.hostFamilyLibraries`.
2. Turn it off (`false`).

The collection extensions show their own Icon Library views, commands and shortcuts again. Icon Studio's view and commands stay available, but its `Ctrl` + `Alt` + `I` and `Ctrl` + `Alt` + `R` shortcuts step aside so the shortcuts are not claimed twice. If none of the collection extensions are installed, Icon Studio's shortcuts stay active either way.

The change takes effect immediately. If Icon Studio is disabled or uninstalled, the collection views come back.

When the collections are installed without Icon Studio

If two or three of Modern, Drawn and Playful are installed (and Icon Studio is not hosting them), they cooperate on their own: one of them automatically becomes the host and shows a single merged Icon Library. The others hide their views and forward their commands to it. See Auric Icons: Modern for details.

Likes and recent icons

- Icon Studio keeps its own likes, recent icons and learned recommendations.
- Each collection extension keeps its own likes and recents for its own icons.
- These are separate stores. Liking an icon in Icon Studio does not add it to the Modern extension's likes, and clearing one does not clear the other.

Other Auric icon extensions

- **Auric Artisan Emoji** keeps its own Emoji Studio view and commands.
- **Auric Artisan Icon Hub** keeps its own view and settings.

- **Auric Artisan File Icons** remains the extension for Explorer file and folder themes. Installing Icon Studio does not replace it.

See the comparison for which extension does what.

Result

You see each Auric icon once, in the interface you prefer, and your shortcuts do not clash.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Commands and keybindings

Use Icon Studio from the Command Palette and keyboard, including Quick Pick insertion and a fixed icon bound to a key.

You can use Icon Studio without opening its panel. Open the Command Palette (`Ctrl` + `Shift` + `P`) and type "Auric Icon Studio".

Search everything and choose an action

1. Run **Auric Icon Studio: Search All Icons...** (also the search icon in the sidebar title bar).
2. A Quick Pick opens. With an empty search and recommendations on, it starts with **Recommended for this file**, then **All icons**. Each row shows the icon, its collection, category and tags.
3. Type to search. Up to 200 results are listed.
4. Choose an icon, then choose an action:
   - **Smart Insert (matches this file)**
   - **Insert**
   - **Copy**
   - **Copy As...**
   - **Save as SVG...** (SVG artwork only)
   - **Toggle Like**

Smart insert with one shortcut

1. Place the cursor in a text editor.
2. Press `Ctrl` + `Alt` + `I` (`Cmd` + `Alt` + `I` on macOS), or run **Auric Icon Studio: Smart Insert Icon (matches active language)**. It is also on the editor's right-click menu.
3. Choose an icon. It is inserted in the format that suits the file.

Insert from Recent or Likes

- Press `Ctrl` + `Alt` + `R` (`Cmd` + `Alt` + `R` on macOS) or run **Insert from Recent**.
- Run **Insert from Likes** (also the heart icon in the sidebar title bar).

Both run the action set in `auricIconStudio.clickAction`: **insert** (default), **copy**, or **insert and copy**. If the list is empty, you see "Icon Studio: no recent icons yet." or a prompt to like an icon first.

Other commands

Command	What it does
Open Icon Studio	Opens the editor-tab Studio beside your editor.
Copy Icon As...	Choose an icon, then a format, and copy.
Save Icon as SVG...	Choose an SVG icon and save it through a dialog.
Clear Likes, Recents and Learned Recommendations	After confirmation (**Clear**), resets Icon Studio's personalisation.

Keyboard shortcuts need editor focus. When you have chosen to keep the separate Auric collection libraries, Icon Studio's shortcuts step aside for theirs (see Work with the Auric collection extensions).

Insert a specific icon from a keybinding

The command `auricIconStudio.insertIcon` inserts one icon by its catalog ID. It is hidden from the Command Palette and meant for keybindings and macros.

1. Open **Preferences: Open Keyboard Shortcuts (JSON)**.
2. Add an entry such as:

```
{
  "key": "ctrl+alt+shift+r",
  "command": "auricIconStudio.insertIcon",
  "args": {
    "id": "tabler-rocket",
    "format": "jsx"
  }
}
```

Argument	Required	Meaning
`id`	Yes	The catalog ID. You can also pass the ID as a plain string instead of an object.
`variant`	No	A style ID for that collection, such as `outline`.
`format`	No	An output format ID. If omitted, smart insert rules apply.

Format IDs: `raw`, `jsx`, `jsxComponent`, `js`, `img`, `markdown`, `css`, `dataUri`, `name` (SVG artwork); `html`, `jsx`, `jsxComponent`, `js`, `css`, `name` (Material); `emoji`, `shortcode`, `unicode`, `html` (emoji).

To find an icon's ID, copy it with **As...** › **Icon ID** rather than guessing. An unknown ID shows "Icon Studio: unknown icon id ...".

Result

You can search, insert, copy and save icons entirely from the keyboard, and bind a favourite icon to its own shortcut.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Insert, copy and save

Insert icons with smart, language-aware output, copy them in an explicit format, save SVG files and preview Auric icon IDs on hover.

Every card has three main actions and three quick format buttons.

Button	What it does
Insert (or select the artwork)	Inserts into the editor. With smart insert on, the format follows the file's language.
Copy	Copies the plain format: SVG markup for SVG artwork, an HTML span for Material, the character for emoji.
As…	Opens a list of every format for that icon; the plain format is marked **default**.
Quick buttons	SVG artwork: **JSX**, **URI** (data URI), **CSS**. Material: **JSX**, **CSS**, **HTML**. Emoji: **Code** (shortcode), **U+** (code points), **HTML** (entities).

After each action, a status message confirms what happened, for example "Rocket copied as data uri."

Insert into your file

1. Open a writable file and place one or more cursors, or select text to replace.
2. In Icon Studio, select **Insert** on a card.
3. The output is inserted at every cursor. Selected text is replaced. If the output has several lines, each continuation line gets the same indentation as the cursor's line.

Icon Studio inserts into the active editor or, if you clicked into the Studio, the last text editor you used. If no editor is available, the output is copied instead.

Smart insert

Smart insert is on by default (`auricIconStudio.smartInsert`). It picks the format from the editor's language:

Editing in	SVG artwork becomes
React JSX or TSX, MDX	A JSX element
JavaScript, TypeScript	A JavaScript string constant
HTML, Vue, Svelte, Astro, PHP, Handlebars, Razor, ERB, Django templates, Jinja, Blade, XML, SVG	SVG markup
CSS, SCSS, Sass, Less, Stylus, PostCSS	A CSS background
Markdown	A Markdown image
JSON, JSONC, YAML, TOML, Python	A data URI
Any other language	SVG markup

- **Material** glyphs become a JSX span in React, a string constant in JavaScript or TypeScript, CSS declarations in stylesheets, and an HTML span elsewhere.
- **Emoji** are always inserted as the character.

Turn smart insert off to always insert the plain format. The **Smart Insert Icon** command always uses smart output.

> **Note:** Smart insert chooses a format. It does not configure your application or guarantee valid syntax at every cursor position. Check quotes, surrounding code and accessibility labels.

Copy in an explicit format

1. Select **As...** on a card, or run **Auric Icon Studio: Copy Icon As...**.
2. Choose a format from the list.

Icon type	Formats
SVG artwork	SVG markup, JSX element, React component, JavaScript string constant, Image tag (self-contained), Markdown image, CSS background, Data URI, Icon ID
Material glyph	HTML span, JSX span, React component, JavaScript string constant, CSS declarations, Ligature name
Emoji	Emoji character, Shortcode, Unicode code points, HTML entities

Material icons never export as SVG. Material markup needs the matching Material font in your application; the Studio's preview font is not added to your project.

Control the output

These settings shape SVG and font output:

Setting	Effect
`auricIconStudio.size`	Width and height written into the output. `0` (default) keeps the native size: 64 for Auric artwork, 24 for most open-source sets. Range 0 to 512.
`auricIconStudio.color`	`currentColor` (default, follows the text colour where it is used) or a six-digit hex colour for monochrome output.
`auricIconStudio.strokeWidth`	Stroke width for stroke-based sets (Tabler, Lucide, Feather). Auric artwork keeps its drawn strokes. Range 0.5 to 4, default 2.
`auricIconStudio.emojiSkinTone`	Default skin tone for emoji that support one.

SVG output gets `role="img"` and an `aria-label` with the icon name when the artwork has no accessible name of its own. Review the label and change it if the icon is decorative or needs a better description.

Save an SVG file

1. Run **Auric Icon Studio: Save Icon as SVG...** and choose an icon, or choose **Save as SVG...** in the **Search All Icons...** action list.
2. A save dialog opens with the name `<icon-id>.svg` in your first workspace folder (or your home folder).
3. Choose a location and select **Save icon**.

Only SVG artwork can be saved. Emoji are characters and Material icons are font glyphs, so they have no SVG file to save.

Hover previews in code

Hover over a recognised Auric icon ID, such as `files-blank-document-drawn-0001`, in any file. The hover shows a 48-pixel preview, the icon name and description, its ID, collection and category, and an **Open in Icon Studio** link.

- If the Auric extension that owns that collection (Modern, Drawn or Playful) is installed, Icon Studio leaves the hover to it, so you never see two previews.
- A hover only previews; it never changes your file.
- Turn previews off with `auricIconStudio.hoverPreview`.

Result

The icon is in your file (or on your clipboard, or saved as an `.svg` file) in the format you need, and it appears under **Recent**.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Search and browse

Search 17,234 entries, switch between Library, For you, Likes and Recent, and refine by collection, category, style and skin tone.

Search

1. Select **Search your library**, or press `/` anywhere in Icon Studio.
2. Type a name, tag or concept. The results update as you type.
3. Press `Escape` in the search box to clear it.

Search is synonym-aware and ranks results. For example, `delete` finds trash icons. Hindi search aliases are supported for many entries. The search also takes account of the icons you like and use.

The four views

The chips at the top switch what the grid shows.

View	Shows	Empty message
Library	Every entry that matches your search and filters.	"No icons found. Try another word, or choose All collections."
For you	Up to 96 icons ranked for the file you are editing and your likes and use. Hidden when recommendations are turned off.	"No recommendations yet. Open a file, or like and use a few icons to teach Icon Studio your style."
Likes	Icons you liked with the heart button.	"No liked icons yet. Press the heart on any card to keep it here."
Recent	Icons you inserted, copied or saved, newest first (up to 60).	"No recent icons yet. Icons you insert, copy or save appear here."

Search and filters apply inside every view.

Refine the results

Expand **Refine library** (subtitle "Collection, category & style").

Control	Options
Collection	**All collections**, or one collection. Each option shows its group and count, for example "Auric Artisan · Drawn (480)" or "Open source · Tabler (5,093)".
Category	**All categories**, or a category from the chosen collection.
Style	Appears when you choose one collection that has more than one style. **Default** uses the collection's default style.
Skin tone	Appears when **All collections** or **Emoji** is chosen: **Default tone**, **Light**, **Medium-light**, **Medium**, **Medium-dark**, **Dark**. It changes emoji that support tones.

When you pick a single collection, a credit line under the heading shows the collection name, its licence and its author.

Select **Reset filters** to clear the search, collection, category, style and tone. You stay in the current view (Library, Likes and so on).

Dropdowns support the arrow keys, `Home`, `End`, `Enter`, `Escape` and `Tab`. Scroll a long menu with the mouse wheel.

Collections and styles

Collection	Entries	Styles
Auric Modern	720	Color, Mono, Outline, Duotone
Auric Drawn	480	Color, Mono, Outline, Duotone
Auric Playful	1,225	Color, Outline, Filled, Small
Emoji	1,914	Default tone and five skin tones where supported
Phosphor	1,512	Thin, Light, Regular, Bold, Fill, Duotone
Tabler	5,093	Outline, Filled
Lucide	1,714	Regular
Remix	1,689	Line, Fill
Heroicons	324	24 Outline, 24 Solid, 20 Solid, 16 Solid
Feather	287	Regular
Material	2,276	Outlined, Round, Sharp, Regular, Two-tone

Counts are catalog entries, not style variants. Emoji look as your operating system draws them. Material icons are drawn from a bundled font, not SVG.

Scrolling

The grid loads cards in batches (96 by default; see `auricIconStudio.pageSize`) and adds more as you scroll. A status line reads, for example, "192 of 1,512 loaded — scroll for more", then "All 1,512 loaded". The sidebar and editor tab remember your search, filters and view when you come back.

Card size

Set `auricIconStudio.density` to `compact` for smaller cards or `comfortable` (default) for larger ones.

Result

The grid shows exactly the icons you need, in the style you want to preview.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Icon Studio reference

Every command, keybinding, setting, collection, style, output format, view and control in Auric Artisan Icon Studio 0.1.0.

This reference describes version 0.1.0 as published on the Visual Studio Marketplace.

Commands

Category in the Command Palette: **Auric Icon Studio**.

Command	ID	What it does
Open Icon Studio	`auricIconStudio.openStudio`	Opens the editor-tab Studio beside the active editor, or reveals it and focuses search.
Search All Icons…	`auricIconStudio.searchIcons`	Quick Pick over the whole catalog, then an action list.
Smart Insert Icon (matches active language)	`auricIconStudio.smartInsert`	Quick Pick, then insert with smart output.
Copy Icon As…	`auricIconStudio.copyAs`	Quick Pick, then a format list; copies.
Save Icon as SVG…	`auricIconStudio.saveSvg`	Quick Pick of SVG icons; saves through a dialog.
Insert from Likes	`auricIconStudio.insertFavorite`	Quick Pick of liked icons; runs the click action.
Insert from Recent	`auricIconStudio.insertRecent`	Quick Pick of recent icons; runs the click action.
Clear Likes, Recents and Learned Recommendations	`auricIconStudio.clearPersonalization`	Asks for confirmation, then resets personalisation.
Insert Icon by ID	`auricIconStudio.insertIcon`	Inserts a given icon; for keybindings and macros. Hidden from the Command Palette.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command	When
`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`	Smart Insert Icon	Editor has focus and Icon Studio's shortcuts are active.
`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`	Insert from Recent	Editor has focus and Icon Studio's shortcuts are active.

Icon Studio's shortcuts are active when it hosts the Auric collections (the default) or when none of Modern, Drawn and Playful is installed.

Keys inside Icon Studio

Key	Action
`/`	Focus and select the search box.
`Escape`	In search: clear it. In an open dropdown: close it and return focus.
`Arrow Up`, `Arrow Down`	Open a dropdown from its button; move between options.
`Home`, `End`	First or last dropdown option.
`Tab`	Closes open dropdowns.

Settings

Setting	Type	Values or range	Default	What it does
`auricIconStudio.clickAction`	string	`insert`, `copy`, `insertAndCopy`	`insert`	Action when you choose an icon from a Quick Pick (Likes, Recent).
`auricIconStudio.smartInsert`	boolean	`true`, `false`	`true`	Language-aware output for Insert.
`auricIconStudio.size`	number	0 to 512	0	Width and height in SVG and font output; 0 keeps the native size.
`auricIconStudio.color`	string	`currentColor` or `#rrggbb`	`currentColor`	Colour for monochrome output.
`auricIconStudio.strokeWidth`	number	0.5 to 4	2	Stroke width for Tabler, Lucide and Feather.
`auricIconStudio.emojiSkinTone`	string	`none`, `light`, `mediumLight`, `medium`, `mediumDark`, `dark`	`none`	Default emoji skin tone.
`auricIconStudio.density`	string	`comfortable`, `compact`	`comfortable`	Card size in the grid.
`auricIconStudio.pageSize`	number	24 to 240	96	Cards loaded per batch as you scroll.
`auricIconStudio.recommendations.enabled`	boolean	`true`, `false`	`true`	The **For you** view and Quick Pick recommendations.
`auricIconStudio.hoverPreview`	boolean	`true`, `false`	`true`	Previews for Auric icon IDs in code.
`auricIconStudio.hostFamilyLibraries`	boolean	`true`, `false`	`true`	Hide the separate Modern, Drawn and Playful library views while Icon Studio is installed.

Search Settings for `@ext:auric-artisan.auric-artisan-icon-studio` to see them all.

Views and menus

Place	Contents
Activity Bar	**Icon Studio** container with the **Icon Studio** view.
View title bar	**Open Icon Studio**, **Search All Icons...**, **Insert from Likes**; in the **...** menu, **Clear Likes, Recents and Learned Recommendations**.

Place	Contents
Editor right-click menu	**Smart Insert Icon (matches active language)** (writable editors only).
Editor tab	"Icon Studio", opened beside the active editor.

Studio controls

Control	Options or behaviour
View chips	**Library**, **For you**, **Likes** (count), **Recent** (count).
Search your library	Synonym-aware search; `/` to focus; `Escape` to clear.
Refine library	Collapsed in the sidebar, open in the editor tab.
Collection	**All collections** or one of 11.
Category	**All categories** or one category.
Style	Shown for a single multi-style collection; **Default** or a style.
Skin tone	Shown for all collections or Emoji; six tones.
Reset filters	Clears search and refinements; keeps the current view.
Card	Artwork (inserts), heart (like), badge, name, category, **Insert**, **Copy**, **As...**, quick formats.
Load more	Shown only if automatic loading is unavailable.

Collections

ID	Collection	Group	Entries	Styles (ID: label)	Default style	Licence
`modern`	Modern	Auric Artisan	720	color, mono, outline, duotone	color	Auric Artisan Free Use Licence
`drawn`	Drawn	Auric Artisan	480	color, mono, outline, duotone	color	Auric Artisan Free Use Licence
`playful`	Playful	Auric Artisan	1,225	color, outline, filled, small	color	Auric Artisan Free Use Licence
`emoji`	Emoji	Emoji	1,914	none plus five tones	none	MIT (data); glyphs drawn by your system
`phosphor`	Phosphor	Open source	1,512	thin, light, regular, bold, fill, duotone	regular	MIT
`tabler`	Tabler	Open source	5,093	outline, filled	outline	MIT
`lucide`	Lucide	Open source	1,714	regular	regular	ISC
`remix`	Remix	Open source	1,689	line, fill	line	Remix v1.0
`heroicons`	Heroicons	Open source	324	24/outline, 24/solid, 20/solid, 16/solid	24/outline	MIT

ID	Collection	Group	Entries	Styles (ID: label)	Default style	Licence
`feather`	Feather	Open source	287	regular	regular	MIT
`material`	Material	Open source	2,276	outlined, round, sharp, regular, twotone	outlined	Apache-2.0

Total: 17,234 entries.

Output formats

Kind	Format ID	Label
SVG	`raw`	SVG markup (plain format)
SVG	`jsx`	JSX element
SVG	`jsxComponent`	React component
SVG	`js`	JavaScript string constant
SVG	`img`	Image tag (self-contained)
SVG	`markdown`	Markdown image
SVG	`css`	CSS background
SVG	`dataUri`	Data URI
SVG	`name`	Icon ID
Material	`html`	HTML span (plain format)
Material	`jsx`	JSX span
Material	`jsxComponent`	React component
Material	`js`	JavaScript string constant
Material	`css`	CSS declarations
Material	`name`	Ligature name
Emoji	`emoji`	Emoji character (plain format)
Emoji	`shortcode`	Shortcode
Emoji	`unicode`	Unicode code points
Emoji	`html`	HTML entities

Smart insert by language

Language IDs	SVG output	Material output
`javascriptreact`, `typescriptreact`, `mdx`	JSX element	JSX span
`javascript`, `typescript`	JavaScript string constant	JavaScript string constant

Language IDs	SVG output	Material output
`html`, `vue`, `svelte`, `astro`, `php`, `handlebars`, `razor`, `erb`, `django-html`, `jinja`, `blade`, `xml`, `svg`	SVG markup	HTML span
`markdown`	Markdown image	HTML span
`css`, `scss`, `sass`, `less`, `stylus`, `postcss`	CSS background	CSS declarations
`json`, `jsonc`, `yaml`, `toml`, `python`	Data URI	HTML span
Anything else	SVG markup	HTML span

Emoji always insert as the character.

Limits

Item	Limit
Recent icons kept	60
Liked icons kept	500
For you view	Up to 96 icons
Quick Pick results	200 with a search term; 300 without
Hover preview size	48 pixels

Messages you may see

Message	Meaning
… copied — open an editor to insert it.	No editor was available, so the output was copied.
… is an emoji character and has no SVG file to save.	Only SVG artwork can be saved.
… is a font glyph and has no SVG file to save.	Material icons cannot be saved as SVG.
Icon Studio: like an icon first — press the heart on any card.	**Insert from Likes** found no likes.
Icon Studio: no recent icons yet.	**Insert from Recent** found no recent icons.
Icon Studio: unknown icon id …	**Insert Icon by ID** received an ID that is not in the catalog.
VS Code could not apply the edit.	The editor rejected the insertion, for example because it is read-only.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Artisan Icon Studio.

Symptom	Why it happens	What to do
No search results.	A collection or category filter is narrowing the search, or you are in Likes or Recent.	Select **Reset filters**, switch to **Library**, and try a simpler word or synonym.
A style or skin tone option is missing.	Styles differ by collection; the Style control only appears when one collection with several styles is chosen. Only some emoji support skin tones.	Choose a single collection to see its styles.

Symptom	Why it happens	What to do
For you chip is missing.	Recommendations are turned off.	Turn on `auricIconStudio.recommendations.enabled`.
For you is empty.	There is no editor context and no likes or usage yet.	Open a file, and like or use a few icons.
Insert copied the icon instead.	No editor was available to insert into.	Open and focus a writable text editor, then insert again.
"VS Code could not apply the edit."	The editor is read-only or rejected the change.	Use **Copy**, or open a writable file.
A Material icon shows as a word in my app.	Material output needs the Material font; the Studio's preview font is not installed in your project.	Load the matching Material Icons font and styles in your application.
I cannot save a Material icon or emoji as SVG.	They are font glyphs and characters, not SVG files.	Use **Copy** or **As...** for a text format.
The output needs editing.	Smart insert picks a format; it does not integrate the icon into your code for you.	Use **As...** for a different format and check the insertion point.
The Modern, Drawn or Playful library view disappeared.	Icon Studio is hosting those collections.	Turn off `auricIconStudio.hostFamilyLibraries` to show their views again.
`Ctrl` + `Alt` + `I` does not run Icon Studio.	The editor is not focused, another extension uses the same keys, or Icon Studio's shortcuts stepped aside because you kept the separate collection views.	Focus the editor, check **Keyboard Shortcuts** for conflicts, and review `hostFamilyLibraries`.
Hovering an Auric icon ID shows no Icon Studio preview.	The owning collection extension is installed and shows its own preview, or previews are off.	This is expected; or turn on `auricIconStudio.hoverPreview`.
Emoji look different from other screenshots.	Emoji are drawn by your operating system's emoji font.	This is expected; newer emoji may not appear with older fonts.
An error says the icon catalog failed to load.	The installed extension files are damaged or incomplete.	Reinstall the extension from the Marketplace.

Reset personalisation

Run **Auric Icon Studio: Clear Likes, Recents and Learned Recommendations** and confirm with **Clear**.

Still stuck

Note your VS Code version, the Icon Studio version, the collection and icon ID, and the steps that reproduce the problem. Remove private code and paths from screenshots, then contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Frequently asked questions

Short answers to common questions about Auric Artisan Icon Studio.

Do I need Auric Icons: Modern, Drawn or Playful as well?

No. Icon Studio bundles all three collections. Install the separate extensions only if you want their own interfaces, or Modern's Explorer file icon theme.

Does Icon Studio change my Explorer file icons?

No. It is an artwork browser and insertion tool. For Explorer file and folder icons, use Auric Artisan File Icons.

Does it need an internet connection?

No. The catalog, artwork and Material preview fonts are bundled. Icon Studio makes no network requests.

Is it the same as Icon Hub?

No. Icon Hub covers the seven open-source sets only, adds colour and stroke controls, sprite and ZIP export and CDN insertion, and can check the website for catalog updates. Icon Studio adds the Auric artwork and emoji and works entirely offline. See the comparison.

Can I change the size or colour of inserted icons?

Yes, through settings: `auricIconStudio.size`, `auricIconStudio.color` and `auricIconStudio.strokeWidth`. See Insert, copy and save.

Can I search in Hindi?

Many entries have Hindi search aliases. The interface itself is in English.

Why do Material icons not export as SVG?

Material icons in this catalog are glyphs in a font, not SVG files. Icon Studio offers HTML, JSX, React, JavaScript, CSS and ligature-name output instead.

Can I use the artwork in my own projects?

Auric artwork is covered by the Auric Artisan Free Use Licence, which restricts extracting and redistributing artwork. Third-party icons keep their own licences (MIT, ISC, Remix v1.0, Apache-2.0). Check the licence for your use.

Where are my likes stored?

In VS Code's extension storage on this machine. See Privacy.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Icon Studio: other information

Contents of the supporting pages for Icon Studio: glossary, shortcuts, limits, comparison, release notes and privacy.

This folder holds supporting pages for Auric Artisan Icon Studio. It includes the comparison of all Auric icon extensions, the terms used, the keyboard shortcuts, what the tool can and cannot do, its release history and its privacy behaviour.

File	What it covers
glossary.md	Terms used in Icon Studio.
shortcuts.md	Keybindings, in-Studio keys and every command.
limits-and-accuracy.md	What smart insert and search can and cannot do.
related.md	Comparison of Icon Studio, Icon Hub, the collections, File Icons, Emoji and Icon Pack.
release-notes.md	User-facing history.
privacy.md	What is stored and what uses the network.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Shortcuts and commands

Keybindings, keys inside the Studio and every Command Palette command in Auric Artisan Icon Studio.

Keybindings

Action	Windows and Linux	macOS
Smart Insert Icon	`Ctrl` + `Alt` + `I`	`Cmd` + `Alt` + `I`
Insert from Recent	`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`

Both need editor focus. They are active when Icon Studio hosts the Auric collections (the default) or when no collection extension is installed. The Auric collection extensions use the same keys, so only one extension claims them at a time.

Keys inside Icon Studio

Key	Action
`/`	Focus search
`Escape`	Clear search, or close an open dropdown
`Arrow Up` / `Arrow Down`	Open a dropdown; move between its options
`Home` / `End`	First or last option
`Enter`	Choose the focused option or button
`Tab`	Move on and close open dropdowns

Commands

Command	Default key
Auric Icon Studio: Open Icon Studio	None
Auric Icon Studio: Search All Icons…	None
Auric Icon Studio: Smart Insert Icon (matches active language)	`Ctrl` + `Alt` + `I`
Auric Icon Studio: Copy Icon As…	None
Auric Icon Studio: Save Icon as SVG…	None
Auric Icon Studio: Insert from Likes	None
Auric Icon Studio: Insert from Recent	`Ctrl` + `Alt` + `R`
Auric Icon Studio: Clear Likes, Recents and Learned Recommendations	None

`auricIconStudio.insertIcon` (Insert Icon by ID) is available for keybindings and macros only. See Commands and keybindings.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Glossary

Definitions of the terms used in Auric Artisan Icon Studio.

Term	Meaning
Catalog entry	One icon or emoji in the bundled catalog. Its styles or tones are not counted separately.
Collection	A set of icons from one source, such as Auric Drawn or Tabler. Icon Studio has 11.
`currentColor`	A CSS keyword meaning "use the surrounding text colour", so an icon follows light and dark themes.
Data URI	A way to embed a file's content directly in text, starting `data:image/svg+xml,`.
Font-backed icon	An icon drawn from a font glyph (Material), not an SVG file. It needs the font loaded to display.
For you	A view of icons ranked for the file you are editing and your likes and use.
Host (collections)	The one extension that shows the shared library when several Auric collection extensions, or Icon Studio, are installed.
Icon ID	The unique catalog name of an icon, such as `tabler-rocket` or `files-blank-document-drawn-0001`.
JSX	The HTML-like syntax used in React components.
Ligature name	The word that a Material icon font turns into a glyph, such as `home`.
Likes	Icons you marked with the heart.
Quick Pick	VS Code's searchable pop-up list at the top of the window.
Recent	Icons you inserted, copied or saved, newest first.
Shortcode	A text name for an emoji between colons, such as `:rocket:`.
Skin tone	A modifier that changes the skin colour of emoji that support it.
Smart insert	Choosing the output format from the language of the file you are editing.
Style	A visual variant of an icon, such as Outline or Filled. Also called a variant.

Term	Meaning
SVG	Scalable Vector Graphics, a text format for vector images.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Limits and accuracy

What Icon Studio's search, recommendations and smart insert can and cannot do.

Catalog

- The catalog is fixed at the version you installed: 17,234 entries in 11 collections. New icons arrive with extension updates.
- Counts are catalog entries, not the number of style or tone variants.
- Emoji are drawn by your operating system's emoji font, so their look varies and very new emoji may not display on older systems.
- Material icons are font glyphs and cannot be exported as SVG.

Search and recommendations

- Search matches names, concepts, tags and synonyms, and uses Hindi aliases where available. It is not a full translation of the catalog.
- **For you** and Quick Pick recommendations are ranked locally from the text around your cursor (about 60 lines either side), the file type, and your likes and use. They are suggestions, not an AI service.

Smart insert

- Smart insert picks a format by language. It does not look at the exact cursor position, so it cannot tell whether you are inside a string, a tag or a comment.
- It does not add imports, fonts or build configuration to your project.
- Inserted SVG includes an accessible name based on the icon name when the artwork has none. Decide whether each icon is decorative or meaningful and adjust the label.

Hover previews

- Only Auric icon IDs (a hyphenated name ending in a four-digit number) are previewed. Open-source icon IDs are not.
- When the owning collection extension is installed, it shows the preview instead of Icon Studio.

Licences

Auric artwork is covered by the Auric Artisan Free Use Licence, which is not an unrestricted asset licence. Open-source sets keep their own licences, shown in the credit line when you choose a single collection.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Privacy

What Icon Studio stores, where, and confirmation that it makes no network requests.

Icon Studio is offline by design. The catalog, SVG artwork and Material preview fonts ship with the extension. It makes no network requests, has no account and collects no telemetry.

What is stored

Data	Where
Likes (up to 500), recent icons (up to 60) and learned preferences for tags and categories	VS Code's extension storage on this machine.
Search text, filters and current view	The panel's saved state in VS Code.
Settings	Your VS Code settings under `auricIconStudio.*`.

What Icon Studio reads

To rank recommendations and choose smart-insert formats, it reads the language, file name and up to about 60 lines around the cursor of the file you are editing. This stays in memory and is not stored or sent anywhere.

What Icon Studio writes

- Inserting changes the editor content at your cursors or selections.
- Copying writes to your clipboard.
- Saving an SVG writes a file only through a save dialog you confirm.

Review what you insert or export, especially with multiple cursors or a selection.

Clear your data

Run **Auric Icon Studio: Clear Likes, Recents and Learned Recommendations** and confirm with **Clear**.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Release notes

User-facing history of Auric Artisan Icon Studio.

0.1.0

The first unified Icon Studio, and the current Marketplace version: Auric artwork, third-party icon collections and emoji in one offline workspace.

Browse and discover

- Search 17,234 bundled entries across 11 collections: Auric Modern, Drawn and Playful, Emoji, Phosphor, Tabler, Lucide, Remix, Heroicons, Feather and Material.
- Work in a compact sidebar or a full editor-tab Studio, with a sticky header, **Library**, **For you**, **Likes** and **Recent** chips, and a collapsible **Refine library** section with **Reset filters**.
- Filter by collection, category, style and supported emoji skin tone.
- Search by synonyms, including Hindi aliases, with progressive loading as you scroll.
- Keep likes and recent items, with locally ranked **For you** recommendations.

Insert and export

- Language-aware output for JSX, JavaScript, markup, CSS, Markdown and data URIs.
- Insert at multiple cursors, keeping continuation-line indentation.
- Explicit copy formats and saving SVG artwork through a file dialog.
- Emoji as characters, shortcodes, Unicode code points or HTML entities.
- Font-backed Material glyphs through markup and CSS; no SVG output for them.

Editor workflow

- Hover previews for Auric icon IDs, deferring to an installed owning collection extension.
- Smart Insert with `Ctrl` + `Alt` + `I` and Insert from Recent with `Ctrl` + `Alt` + `R`.
- `auricIconStudio.insertIcon` for a specific icon in a keybinding or macro.
- Hosting of the Modern, Drawn and Playful libraries through `auricIconStudio.hostFamilyLibraries`, with the option to keep their separate views.

Offline and licensing

- Catalog, artwork and preview fonts bundled for offline use, with no network requests or telemetry.
- Third-party attribution and licence texts included with their collections.

VS CODE EXTENSIONS › AURIC ARTISAN ICON STUDIO

Related extensions and comparison

How Icon Studio compares with Icon Hub, the Auric icon collections, File Icons, Emoji and the Icon Pack, and which to install for each job.

Auric Artisan publishes several icon extensions for VS Code. They overlap on purpose, so this page explains which one does what.

At a glance

Extension	Main job	Explorer file icon theme	Artwork	Network
Icon Studio	One offline studio to search and insert every Auric collection, emoji and seven open-source sets.	No	17,234 entries in 11 collections	None
Icon Hub	Seven open-source sets with size, colour and stroke controls, sprite and ZIP export, and CDN insertion.	No	12,895 icons	Optional catalog metadata checks; CDN output loads from the web
Auric Icons: Modern	Modern icon library plus the Modern file and folder icon theme.	Yes: **Auric Icons: Modern**	720 library icons; 408 theme icons	None
Auric Icons: Drawn	Warm Drawn icon library.	No	480 icons	None
Auric Icons: Playful	Emoji-style icon library.	No	1,225 icons	None
File Icons	Explorer file and folder icons, plus a File Icon Studio to insert that artwork.	Yes: Facet, Playful and Modern	1,464 SVG assets	None

Extension	Main job	Explorer file icon theme	Artwork	Network
Emoji	Emoji Studio with shortcodes, sequences, completions and six copy formats.	No	1,914 emoji	None
Icon Pack	Installs Modern, Playful, Drawn and Emoji together.	Through Modern	None of its own	Needs the Marketplace to install

Which should I install?

- **I want icons and emoji in my code, offline, in one place:** Icon Studio.
- **I want to recolour, restroke and export open-source icons, or load them from a CDN on my site:** Icon Hub.
- **I want my Explorer's file and folder icons to change:** File Icons (or Modern's own file theme).
- **I only want one Auric artwork style with its own interface:** Modern, Drawn or Playful.
- **I want the full emoji workflow (sequences, shortcode completion, replace in selection):** Emoji.
- **I want the four creative libraries installed together in one step:** Icon Pack.

How the three collections merge

Modern, Drawn and Playful are built on the same library core and know about each other.

- **Without Icon Studio:** when two or three are installed, one becomes the host automatically and shows a single merged Icon Library. It adds a **Library** filter and a badge on each card showing which collection it comes from, and offers every collection's variants in one switcher. The other extensions hide their views and forward their commands to the host. This re-runs when you install or remove one, without a reload.
- **With Icon Studio:** by default Icon Studio hosts all three, so their separate views and shortcuts are hidden and you use Icon Studio instead. Set `auricIconStudio.hostFamilyLibraries` to `false` to get their own interfaces back.
- **Personalisation stays with its owner.** Likes and recent icons belong to the extension that owns the icon, so uninstalling one extension never removes another's history. Icon Studio keeps its own separate likes and recents.

See Work with the Auric collection extensions for the settings.

Name clashes to be aware of

- **"Playful"** means two different things. In File Icons it is an Explorer file icon style with 500 file and folder SVGs. In **Auric Icons: Playful** it is a 1,225-icon emoji-style library. They are different artwork.
- **"Modern"** file icons exist in both File Icons and Auric Icons: Modern, under the same theme ID. See File Icons for how to avoid duplicates.
- **"Icon Studio"** is the name of this extension and also the heading of the merged library view inside the Modern, Drawn and Playful extensions.

Other related products

- Install and set up — installing and managing Auric extensions.
- Icon Library on the website — browse open-source icons in the browser.
- Icon CDN — the icon delivery service used by Icon Hub's CDN output.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Auric Artisan Icon Hub

What the Auric Artisan Icon Hub extension is, who it is for, how to install it and where to find each guide.

Auric Artisan Icon Hub is an icon workspace for Visual Studio Code, connected to the Auric icon tool at https://icons.auricartisan.com. It searches 12,895 icons (32,802 indexed style variants) across seven open-source collections: Phosphor, Tabler, Lucide, Remix, Heroicons, Feather and Material. All artwork and preview fonts are bundled, so browsing, previewing and exporting work offline.

You can refine results by set, style, category and licence, shape the output with size, colour and stroke controls (including a full custom colour picker with contrast tools), and insert code that suits your file. In HTML and web-framework files, **Insert** uses the Auric icon CDN and connects a small loader file for you. You can also copy or save an icon in SVG, JSX, React component, HTML, data URI, CSS or CDN form, and export a selection of icons as an SVG sprite or a ZIP with licence notices.

Icon Hub can check the website for updated catalog metadata in the background. This is optional, limited to five public JSON files, and never sends your code, searches or usage.

Who it is for

- Front-end developers who use popular open-source icon sets and want them tuned and inserted from the editor.
- Designers who want to recolour and restroke icons, then export sprites or ZIPs with correct licences.
- Teams that want pages to load icons from the Auric CDN.

Where to find it

Item	Value
Marketplace listing	https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-icon-lib
Extension ID	`auric-artisan.auric-artisan-icon-lib` (displayed as **Auric Artisan Icon Hub**)
Current version	0.1.0
Requires	Visual Studio Code 1.85 or later
Price	Free to install and use

```
code --install-extension auric-artisan.auric-artisan-icon-lib
```

Quick start

1. Install the extension.
2. Select **Auric Icon Hub** in the Activity Bar, or run **Auric Icon Hub: Open Icon Hub** for a full panel.
3. Open a writable file and place the cursor.
4. Type in **Search your library**, for example `arrow`.
5. Expand **Refine library** to choose a set, style, category or licence.
6. Expand **Shape the output** to set size, colour and stroke.
7. Select **Insert** on a card, or select the preview to open the inspector and choose another format.

What you can do

- Search 12,895 icons in seven collections, and refine by set, style, category and licence, sorted by relevance, name or set.
- Keep **Favorites**, **Recent** icons and a **Selected** working set.
- Set output size, colour (with contrast checks, shade ramps and harmonies) and stroke width.
- Insert code that suits the editor language, or copy and save in seven formats.
- Connect pages to the Auric icon CDN automatically on Insert.
- Export selected icons as an SVG sprite or a ZIP with licence notices.
- Use the native Quick Search with `Ctrl` + `Alt` + `Shift` + `I`.
- Keep catalog metadata up to date from the website, or turn that off.

In this folder

docs/

- Contents of docs/
- Getting started
- Browse and refine
- Shape the output
- Insert, copy and save
- Connect the icon CDN
- Collect and export
- Catalog metadata updates
- Reference
- Troubleshooting
- FAQ

others/

- Contents of others/
- Glossary
- Shortcuts and commands
- Limits and accuracy
- Related extensions and comparison
- Release notes
- Privacy

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Icon Hub guides

Contents of the Icon Hub guides folder, with one line on each guide.

This folder holds the task guides and the complete reference for Auric Artisan Icon Hub.

File	What it covers
getting-started.md	Install, open the Hub, tour the screen and insert a first icon.
browse-and-refine.md	Search, views, refinements, sorting, cards and the Smart Finder.
shape-the-output.md	Size, colour, stroke and the custom colour picker.
insert-copy-and-save.md	Insert by language, the inspector, formats, Quick Search and saving.
cdn-insertion.md	How Insert connects the Auric icon CDN, and what your site needs.
collect-and-export.md	Select icons and export an SVG sprite or a ZIP.
catalog-updates.md	Automatic and manual metadata checks, and turning them off.
reference.md	Every command, setting, keybinding, collection, format and message.
troubleshooting.md	Symptoms, causes and fixes.
faq.md	Short answers to common questions.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Getting started with Icon Hub

Install Icon Hub, open it in the sidebar or a full panel, tour the screen and insert your first icon.

Before you begin

- You need Visual Studio Code 1.85 or later.
- Browsing, copying and self-contained SVG insertion work in untrusted workspaces. Connecting the CDN on Insert needs a trusted workspace.

1. Install

Open the Extensions view (`Ctrl` + `Shift` + `X`), search for **Auric Artisan Icon Hub** and select **Install**, or run:

```
code --install-extension auric-artisan.auric-artisan-icon-lib
```

You can also install an official VSIX with **Extensions: Install from VSIX...**.

2. Open the Hub

- **Sidebar:** select **Auric Icon Hub** in the Activity Bar (view name **Icon Hub**). Refinement controls start collapsed to leave room for artwork.
- **Full panel:** run **Auric Icon Hub: Open Icon Hub**, select the Hub's expand button (**Open Icon Hub in an editor tab**), or select the **Icons** item in the status bar.

3. Tour the screen

Area	Contents
Header	"Auric Artisan / ICON HUB / CREATIVE WORKSPACE", **Refresh** (check for metadata updates), **Website ↗** and the expand button.
Views	**Library**, **Favorites**, **Recent**, **Selected**.
Introduction	"Find your symbol. Make it your own." and a catalog status badge such as "Bundled catalog · offline ready".
Search	**Search your library** (`Ctrl` / `Cmd` `K`), placeholder "Try arrow, rocket or navigation…".
Refine library	Collection, style, category, licence and sort.
Shape the output	Size, colour and stroke.
Finder footer	"Insert adapts to your active file. Inspect for every format." and **Reset filters**.
Smart Finder	"Recently used & suggested", with **Shuffle**.
Results	**Explore the collection** and the icon grid, with **Load more**.
Selection tray	"{n} selected", **Copy sprite**, **Export ZIP**, **Clear**.

4. Insert your first icon

1. Open a writable file and place the cursor. The Hub remembers the last text editor you used, even after you click into the Hub.
2. Search for `rocket`.
3. Select **Insert** on a card.

What you get depends on the file: see Insert, copy and save. In an HTML or web-framework file, the first Insert also creates a small loader file next to your source; see Connect the icon CDN.

If no editor is available, the output is copied instead and VS Code says "No writable editor selected; icon copied to the clipboard."

Result

You have inserted an icon from one of seven open-source collections, shaped to your settings.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Browse and refine

Search Icon Hub, switch between Library, Favorites, Recent and Selected, refine by set, style, category and licence, and use the Smart Finder.

Search

Type in **Search your library**. Press `Ctrl` + `K` (`Cmd` + `K`) anywhere in the Hub to jump there. Every word you type must appear in the icon's name, ID, category, search terms or set.

Views

View	Shows
Library	Every icon in the catalog.
Favorites	Only icons you starred.
Recent	Icons you used lately (up to `auricIconLib.maxRecents`, default 40).
Selected	Only the current selection.

Refine library

Expand **Refine library** ("Collection, style, category and license").

Control	Options
Collection	**All sets**, Phosphor, Tabler, Lucide, Remix, Heroicons, Feather, Material
Style	**All styles**, or a style such as Regular, Bold, Fill, Outline, Filled, Line, 24/outline, Round, Two-tone
Category	**All categories** or one category
License	**All licenses**, MIT, ISC, Remix v1.0, Apache-2.0
Sort	**Relevance**, **Name**, **Set**

Reset filters clears the search and refinements. It keeps your selection and the current view.

The first time you open the Hub, the collection and style start from `auricIconLib.defaultSet` and `auricIconLib.defaultStyle`.

Collections

Collection	Icons	Styles	Default style	Licence
Phosphor	1,512	Thin, Light, Regular, Bold, Fill, Duotone	Regular	MIT
Tabler	5,093	Outline, Filled	Outline	MIT
Lucide	1,714	Regular	Regular	ISC
Remix	1,689	Line, Fill	Line	Remix v1.0
Heroicons	324	24 Outline, 24 Solid, 20 Solid, 16 Solid	24 Outline	MIT
Feather	287	Regular	Regular	MIT
Material	2,276	Outlined, Round, Sharp, Regular, Two-tone	Outlined	Apache-2.0

The count is logical icons; style variants (32,802 in total) are indexed separately. The combined Remix sprite is left out because it is not a standalone icon. Material is font-backed and never exported as SVG.

Cards

Control	Action
Preview	Opens the inspector.
Star	**Star this icon** (favorite).
Check	**Add to the selection**.
Insert	**Insert at the cursor**.
Copy	**Copy to the clipboard** in your copy format.

Keyboard: arrow keys move between cards, `Enter` opens the inspector and `F` stars the focused card. Focus stays on the card when the grid updates.

Smart Finder

Expand **SMART FINDER — Recently used & suggested**. It shows your recent icons and a rotating sample of the catalog under "Keep useful icons close." Select **Shuffle** to reshuffle. It is a local convenience, not a remote AI service. It is hidden when `auricIconLib.recommendations.enabled` is off.

Density and batch size

Setting	Values	Default
`auricIconLib.density`	`compact`, `comfortable`, `showcase`	`comfortable`
`auricIconLib.pageSize`	24 to 240	72

Select **Load more** to show the next batch.

Result

The grid shows the icons you want, from the right collection and style, ready to shape and use.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Catalog metadata updates

How Icon Hub checks icons.auricartisan.com for updated catalog metadata, how to refresh manually, and how to turn checks off.

Icon Hub ships with a complete catalog. It can also check https://icons.auricartisan.com for compatible updates to the catalog's **metadata** (search terms, counts and similar data).

What is checked

- Only five public JSON files under `/data/`: `meta.json`, `sets.json`, `stats.json`, `index.json` and `material.json`.
- Requests are time-limited (15 seconds), size-limited, send no credentials and reject redirects.
- Downloaded metadata must match the bundled icon IDs, sets, styles and font mappings before it replaces the cache.

- Updates can improve search metadata. They **never** download new artwork or add icon sets; new artwork arrives with extension updates.

Automatic checks

With `auricIconLib.remoteRefresh.onStartup` on (the default), Icon Hub checks when it activates, at most once per `auricIconLib.remoteRefresh.intervalHours` (24 hours by default, 1 to 168). Failed attempts also wait for the interval. There is no continuous timer.

Background failures never show pop-ups. The Hub's badge shows the state:

Badge	Meaning
Bundled catalog · offline ready	Using the catalog shipped with the extension.
Checking metadata…	A check is running.
Metadata up to date	Compatible metadata was downloaded.
Offline · using local catalog	The check failed; the local catalog is in use.

Check now

Select **Refresh** in the Hub header, or run **Auric Icon Hub: Refresh Icon Catalog**. This ignores the interval. You see "Catalog metadata updated. Shipped artwork remains available offline." or "Using offline catalog: …". Your search and selection are kept.

Return to the bundled catalog

Run **Auric Icon Hub: Clear Downloaded Icon Catalog**. You see "Cache cleared; bundled catalog restored."

Turn checks off

Stop automatic checks only:

```
{
  "auricIconLib.remoteRefresh.onStartup": false
}
```

Disable all catalog network checks, including **Refresh**:

```
{
  "auricIconLib.remoteRefresh.enabled": false
}
```

With checks disabled, **Refresh** says "Remote catalog refresh is disabled."

Result

Your catalog stays current when you want it to, and everything keeps working offline when you do not.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Connect the icon CDN

How Icon Hub's Insert connects your HTML and web-framework files to the Auric icon CDN, what file it creates, and what your site needs.

In HTML and supported web-framework files, **Insert** places an icon placeholder that is drawn at runtime by the Auric icon CDN script, `https://icons.auricartisan.com/icon-cdn.js`. This works for all seven collections, including Material, whose glyphs load their matching font from the CDN.

A **CDN** (content delivery network) is a web service that serves files, here icon data and fonts, to your pages.

What happens on your first Insert

1. Icon Hub checks the target file. It must be saved, local or in a VS Code remote workspace, and the workspace must be trusted.
2. It creates a small file called `icons.auricartisan.com.js` next to your source file and saves it. This **bridge** contains loader logic only: no fonts, no embedded font data and no icon catalog.
3. It connects the bridge to your file:

File type	Connection added
HTML	`<script src="./icons.auricartisan.com.js" defer data-auric-icons="cdn"></script>` after `<head>`
JavaScript, TypeScript, React JSX and TSX	`import "./icons.auricartisan.com.js";` at the top, after any directives such as `"use client"`
Vue, Svelte	An import inside the component's `<script>` block (a block is added if there is none)
Astro	A browser-side `<script>` with the import at the end of the file (not a server-only frontmatter import)

4. It inserts the icon placeholder at your cursors. The whole change is one edit labelled "Insert Auric icon and local CDN support".
5. The message ends "CDN bridge connected."

On later inserts, an existing matching bridge and connection are reused. If a different file already uses the bridge name, a second name with a short hash is used; existing files are never overwritten.

Only the bridge is saved automatically. Save your source file yourself.

At runtime

The bridge loads the CDN script once, renders every icon placeholder on the page, and also renders icons added later or whose attributes change.

Requirements and limits

- Pages need network access to show CDN icons.
- Your site's content security policy must allow the CDN for scripts, data requests and fonts, and the loader's generated font styles.
- Serve the bridge file with your application.

- JavaScript imports need module-aware tooling or a module script. Server-rendered frameworks need the bridge in a browser (client) entry.
- An HTML file that uses a `<base>` URL, or a Vue or Svelte component that uses an external script, is refused with an explanation; connect the bridge yourself or use Copy.
- If a selection overlaps the place where the loader connection goes, Insert asks you to place the cursor inside the page content.
- If the editor changes while Insert prepares, it asks you to try again.
- Browsing, searching, previewing, **Copy** and **Save** never create project files.

> **Note:** At the time of the 0.1.0 release, the published README noted that cross-origin access to the CDN's catalog files and fonts had not yet been enabled for other sites. Until it is, pages served from other origins may show blank icons even with a correct bridge. For fully self-contained artwork, use **Copy** as SVG or export SVG files. Material is font-backed and has no SVG form.

Messages

Message	Meaning
Save this file locally or in your remote workspace before inserting CDN icons. Copy is still available.	The file is untitled or on an unsupported file system.
Trust this workspace before generating CDN support files. SVG insertion and Copy are still available.	The workspace is in Restricted Mode.
CDN insertion requires an HTML or web-framework source file. Use Copy for other output formats.	The file type is not supported for CDN insertion.
The CDN support filenames are already used by different files. No files were overwritten.	Both bridge names are taken by other content.
Icon inserted, but its new CDN bridge could not be saved. Save the support file before previewing the page.	Save the bridge file manually.

Result

Your page references the Auric icon CDN through a small local bridge, and each inserted placeholder renders as the chosen icon when the page loads.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Collect and export

Build a selection of icons in Icon Hub and export it as an SVG sprite or a ZIP with licence notices.

Build a selection

1. Select the check button (**Add to the selection**) on each icon you want.
2. The selection tray at the bottom shows "{n} selected".
3. Switch to the **Selected** view to see only your selection.

Selections are kept when you search, refine or reset filters, and when catalog metadata updates arrive.

Export

Button	Result
Copy sprite	Copies the selected SVG icons as one SVG symbol sprite sheet.
Export ZIP	Saves a ZIP of the selected artwork with licence notices.
Clear	Empties the selection. Favorites are not changed.

What the ZIP contains

- One SVG file per selected SVG icon, named after its set and name.
- `MATERIAL_ICONS.txt`, if you selected Material icons: they are font-backed, so they are listed by name and style instead of being exported as SVG.
- `README.txt` explaining that each icon remains under its upstream licence.
- `THIRD_PARTY_LICENSES.md` and a licence file for each selected set.

Limits

- Each export handles up to 200 icons.
- Output uses your current **Shape the output** size, colour and stroke.
- Each collection keeps its own licence (MIT, ISC, Remix v1.0 or Apache-2.0). Follow the attribution requirements when you use exported artwork.

Result

You have one sprite or ZIP containing exactly the icons you need, with their licences.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Insert, copy and save

How Icon Hub's Insert adapts to your file, the inspector and its formats, Quick Search and the Command Palette, and saving icons to files.

Insert

Select **Insert** on a card, or in the inspector. Icon Hub inserts at every selection in the active editor or, if you clicked into the Hub, the last text editor you used.

Editing in	Insert produces
HTML, Vue, Svelte, Astro	A CDN icon placeholder element, and the CDN loader is connected.
React JSX or TSX	The CDN placeholder as JSX (or a React component if you chose one in the inspector), and the loader is connected.
JavaScript, TypeScript	A named, quoted markup constant, and the loader is connected.
CSS, SCSS, Less	A self-contained CSS declaration.

Editing in	Insert produces
SVG, XML and other languages	Self-contained SVG markup.

See Connect the icon CDN for what the loader is and what it needs.

- Material icons can only be inserted into HTML or web-framework files: "Insert Material icons into HTML or a web-framework file to connect your CDN. Use Copy for other formats."
- Read-only editors are refused: "The selected editor is read-only. Use Copy instead."
- A rejected edit is reported, not recorded as a success.
- With no editor, the output is copied and VS Code says "No writable editor selected; icon copied to the clipboard."

The inspector

Select a card's preview to open the inspector. It shows a large preview, the set, name, category and licence, the available styles, a **Copy format** dropdown, a CDN snippet preview, and **Insert**, **Copy** and **Save**.

- Select a style button to switch the icon's style.
- The **Copy format** applies to **Copy** and **Save**. **Insert** always follows the editor language.
- `Escape` closes the inspector; `Tab` stays inside it.

Formats

Format	Setting value	SVG icons	Material icons
SVG	`svg`	Yes	No (HTML is used instead)
JSX	`jsx`	Yes	Yes
React component	`reactComponent`	Yes	Yes
HTML	`html`	Yes	Yes
Data URI	`dataUri`	Yes	No (HTML is used instead)
CSS	`css`	Yes	Yes
CDN snippet	`cdn`	Yes	Yes

The CDN snippet looks like this:

```
<i data-icon="set:name" data-style="…" data-size="24" data-color="currentColor" data-stroke="2" aria-hidden="true"></i>
<script src="https://icons.auricartisan.com/icon-cdn.js" defer></script>
```

Standalone SVG, data URI and CSS exports include the SVG namespace, so they work as separate files. SVG is sanitised before preview and export.

Set the default for **Copy** with `auricIconLib.copyFormat` (default `svg`), and what a card choice does with `auricIconLib.clickAction` (`insert`, `copy`, `insertAndCopy`; default `insert`).

Save

Select **Save** in the inspector, or run **Auric Icon Hub: Save Icon As...**. A save dialog opens in your first workspace folder with a file name based on the icon. The file type follows the format:

Format	Extension
SVG	`.svg`
CSS	`.css`
JSX, React component	`.jsx`
HTML, CDN snippet	`.html`
Data URI	`.txt`

Save Icon As... from the Command Palette uses your `copyFormat` setting.

From the Command Palette

Command	Flow
Quick Search Icons (`Ctrl` + `Alt` + `Shift` + `I`)	Type a search ("Search Auric Icons"), pick from the matches, and your click action runs.
Smart Insert Icon	The same flow; also on the editor right-click menu.
Show Favorite Icons / **Show Recent Icons**	The same flow, limited to favorites or recent icons.
Copy Icon As...	Pick from all icons, then a format; copies. Also on the editor right-click menu.
Save Icon As...	Pick an icon, then save.

Result

The icon is in your file, on the clipboard or saved as a file, in the form you need, and it appears under **Recent**.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Shape the output

Set icon size, colour and stroke width in Icon Hub, and use the custom colour picker with contrast tools.

Expand **Shape the output** ("Size, color and stroke"). The controls apply to the previews and to everything you insert, copy, save or export.

Control	Effect	Range
Size	Width and height of the output, in pixels.	8 to 512
Color	The icon colour. Until you choose one, artwork uses `currentColor` and follows the editor's text colour.	Any colour
Stroke	Stroke width for artwork drawn with strokes.	0.5 to 4

Stroke only affects stroke-based artwork. It does not turn filled glyphs into outline icons.

These controls are for the current session. To change the starting values permanently, set `auricIconLib.size` (default 24), `auricIconLib.color` (default `currentColor`, or a six-digit hex colour) and `auricIconLib.strokeWidth` (default 2).

The colour picker

Select the **Color** swatch to open the Auric colour picker. It includes:

- a colour spectrum;
- colour-space inputs;
- named colours;
- recent choices;
- shade ramps;
- harmonies;
- WCAG and APCA contrast checks against a background you choose, with tools to adjust towards AA or AAA;
- an eyedropper, where the host supports it.

The swatch tooltip reads "Icon colour {hex} — contrast, ramps and harmonies included".

Result

Previews and output use your size, colour and stroke, so what you insert matches what you saw.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Icon Hub reference

Every command, keybinding, setting, view, collection, format and message in Auric Artisan Icon Hub 0.1.0.

This reference describes version 0.1.0 as published on the Visual Studio Marketplace.

Commands

Category in the Command Palette: **Auric Icon Hub**.

Command	ID	What it does
Open Icon Hub	`auricIconLib.openStudio`	Opens the full Hub panel.
Quick Search Icons	`auricIconLib.quickSearch`	Search box, then a Quick Pick of matches; runs your click action.
Smart Insert Icon	`auricIconLib.smartInsert`	Same flow as Quick Search; insertion follows the editor language.
Copy Icon As…	`auricIconLib.copyAs`	Pick an icon and a format; copies.
Save Icon As…	`auricIconLib.saveIcon`	Pick an icon; saves in your copy format.
Show Favorite Icons	`auricIconLib.showFavorites`	Quick Search limited to favorites.
Show Recent Icons	`auricIconLib.showRecents`	Quick Search limited to recent icons.
Refresh Icon Catalog	`auricIconLib.refreshCatalog`	Checks for compatible metadata now.
Clear Downloaded Icon Catalog	`auricIconLib.clearCache`	Returns to the bundled metadata.
Open Auric Icon Tool	`auricIconLib.openWebsite`	Opens https://icons.auricartisan.com in your browser.
Clear Icon Favorites and Recents	`auricIconLib.clearPersonalization`	Clears favorites and recents immediately, without a confirmation.
Export Selected Icons	`auricIconLib.exportSelected`	Opens the Hub for exporting; hidden from the Command Palette.

Keybindings

Keys (Windows and Linux)	Keys (macOS)	Command
`Ctrl` + `Alt` + `Shift` + `I`	`Cmd` + `Alt` + `Shift` + `I`	Quick Search Icons

Keys inside the Hub

Key	Action
`Ctrl` + `K` / `Cmd` + `K`	Focus search
Arrow keys	Move between cards; move in dropdowns
`Enter`	Open the inspector for the focused card; choose a dropdown option
`F`	Star the focused card
`Home`, `End`	First or last dropdown option
`Escape`	Close a dropdown or the inspector
`Tab`	Stays inside the inspector while it is open

Settings

Setting	Type	Values or range	Default	What it does
`auricIconLib.clickAction`	string	`insert`, `copy`, `insertAndCopy`	`insert`	Action when an icon is chosen.
`auricIconLib.copyFormat`	string	`svg`, `jsx`, `reactComponent`, `html`, `dataUri`, `css`, `cdn`	`svg`	Default format for Copy and Save.
`auricIconLib.defaultSet`	string	`all`, `phosphor`, `tabler`, `lucide`, `remix`, `heroicons`, `feather`, `material`	`all`	Initial collection filter.
`auricIconLib.defaultStyle`	string	`auto` or a style ID	`auto`	Initial style filter.
`auricIconLib.size`	number	8 to 512	24	Output size in pixels.
`auricIconLib.color`	string	`currentColor` or `#rrggbb`	`currentColor`	Preview and output colour.
`auricIconLib.strokeWidth`	number	0.5 to 4	2	Stroke width.
`auricIconLib.density`	string	`compact`, `comfortable`, `showcase`	`comfortable`	Card density.
`auricIconLib.pageSize`	number	24 to 240	72	Cards per batch.
`auricIconLib.recommendations.enabled`	boolean	`true`, `false`	`true`	Shows the Smart Finder.
`auricIconLib.statusBar.enabled`	boolean	`true`, `false`	`true`	Shows the status bar item.
`auricIconLib.maxRecents`	number	5 to 200	40	Recent icons kept.
`auricIconLib.remoteRefresh.enabled`	boolean	`true`, `false`	`true`	Allow metadata checks from https://icons.auricartisan.com.

Setting	Type	Values or range	Default	What it does
`auricIconLib.remoteRefresh.onStartup`	boolean	`true`, `false`	`true`	Check in the background when the Hub activates.
`auricIconLib.remoteRefresh.intervalHours`	number	1 to 168	24	Minimum hours between automatic attempts.

Style IDs: Phosphor `thin`, `light`, `regular`, `bold`, `fill`, `duotone`; Tabler `outline`, `filled`; Lucide and Feather `regular`; Remix `line`, `fill`; Heroicons `24/outline`, `24/solid`, `20/solid`, `16/solid`; Material `outlined`, `round`, `sharp`, `regular`, `twotone`.

Views and menus

Place	Contents
Activity Bar	**Auric Icon Hub** container with the **Icon Hub** view.
View title bar	**Quick Search Icons**, **Refresh Icon Catalog**, **Open Icon Hub**.
Editor right-click	**Smart Insert Icon**, **Copy Icon As...**.
Status bar	Icon and "{count} Icons"; tooltip "Auric Icon Hub · {catalog source}"; opens the Hub.

Hub controls

Area	Controls
Header	**Refresh**, **Website** ↗, open in editor tab
Views	**Library**, **Favorites**, **Recent**, **Selected**
Refine library	Set, style, category, licence, sort (**Relevance**, **Name**, **Set**)
Shape the output	**Size**, **Color** (picker), **Stroke**
Finder footer	**Reset filters**
Smart Finder	**Shuffle**
Card	Preview (inspector), star, check (select), **Insert**, **Copy**
Inspector	Style buttons, **Copy format**, CDN preview, **Insert**, **Copy**, **Save**
Selection tray	**Copy sprite**, **Export ZIP**, **Clear**
Grid	**Load more**

Collections

ID	Name	Icons	Licence
`phosphor`	Phosphor	1,512	MIT
`tabler`	Tabler	5,093	MIT
`lucide`	Lucide	1,714	ISC
`remix`	Remix	1,689	Remix v1.0

ID	Name	Icons	Licence
`heroicons`	Heroicons	324	MIT
`feather`	Feather	287	MIT
`material`	Material	2,276	Apache-2.0

Total: 12,895 icons, 32,802 indexed variants.

Insertion by language

Language IDs	Insert output
`html`, `vue`, `svelte`, `astro`	CDN placeholder; bridge connected
`javascriptreact`, `typescriptreact`	CDN placeholder (or React component); bridge connected
`javascript`, `typescript`	Named markup constant; bridge connected
`css`, `scss`, `less`	CSS (not available for Material)
`svg`, `xml` and others	SVG (not available for Material)

Limits

Item	Limit
Icons per export	200
Metadata request timeout	15 seconds
Metadata files	5, under `/data/`

Messages you may see

Message	Meaning
No writable editor selected; icon copied to the clipboard.	Insert fell back to Copy.
Material Icons are font-backed; generated HTML markup instead of SVG.	You asked for SVG or data URI for a Material icon.
The selected editor is read-only. Use Copy instead.	Insert was refused.
The editor could not accept this insertion. Use Copy instead.	The edit was rejected.
Insert Material icons into HTML or a web-framework file to connect your CDN. Use Copy for other formats.	Material insertion needs a web file.
Catalog metadata updated. Shipped artwork remains available offline.	Manual refresh succeeded.
Using offline catalog: …	Refresh failed; the local catalog is used.
Remote catalog refresh is disabled.	`remoteRefresh.enabled` is off.
Cache cleared; bundled catalog restored.	Downloaded metadata removed.
Select one or more icons	Export was started with an empty selection.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Troubleshooting

Symptoms, causes and fixes for common problems with Auric Artisan Icon Hub.

Symptom	Why it happens	What to do
The catalog gained no new icons after **Refresh**.	Refresh updates compatible metadata only.	Install an extension update for new artwork.
The badge says **Offline · using local catalog**.	The metadata check could not reach or validate the website.	Local browsing still works. Check your connection or try **Refresh** later; automatic retries wait for the interval.
Material names such as `4k_plus` appear as text on my page.	The Material font is not loaded, often from an older insertion that used a local font.	Reinsert the icon with the current version so it uses a CDN placeholder, save the source, and reload the page with network access. Old spans are not rewritten automatically.
CDN icons appear blank on my site.	The page has no network access, your content security policy blocks the CDN, the bridge file is not served, or the CDN does not yet allow cross-origin access to its catalog and fonts.	Check network access, CSP and that the bridge is deployed. For self-contained artwork, use **Copy** as SVG or **Export ZIP**.
Insert says the workspace must be trusted.	Creating the CDN bridge needs a trusted workspace.	Trust the workspace, or use **Copy**.
Insert says to save the file first.	CDN insertion needs a saved local or remote file.	Save the file, then insert again.
Insert refuses because of a `<base>` URL or an external component script.	The bridge path could be wrong in those files.	Connect the bridge yourself, or use **Copy**.
Insert cannot use the editor.	No writable editor, or the editor is read-only.	Focus a writable text file, then return to the Hub. **Copy** is always available.
A Material icon cannot be inserted into my file.	Material insertion needs HTML or a web-framework file.	Use **Copy** with HTML, JSX, React component, CSS or CDN format.
Stroke has no effect.	The icon is filled, not stroke-based.	This is expected; choose an outline style.
Export ZIP includes a text list instead of Material SVGs.	Material icons are font-backed.	This is expected; they are listed in `MATERIAL_ICONS.txt`.
I also have Icon Studio. Which should I use?	They are separate products with separate settings and views.	Icon Hub: seven open-source sets, colour tools, sprite and ZIP export, CDN insertion. Icon Studio: Auric artwork, emoji and open-source sets, fully offline.

If the problem continues, note your VS Code version, the Hub version (0.1.0), the icon ID and the steps to reproduce, remove private code from screenshots, and contact support at https://auricartisan.com/company/contact/.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Frequently asked questions

Short answers to common questions about Auric Artisan Icon Hub.

Why is the extension ID "icon-lib" when it is called Icon Hub?

The extension keeps its original ID, `auric-artisan.auric-artisan-icon-lib`, so existing `auricIconLib.*` settings, favorites and recent items carry over. Its display name is **Auric Artisan Icon Hub**.

Does Icon Hub work offline?

Yes, for browsing, searching, previewing, copying, saving and exporting. The optional metadata checks and pages that use the CDN need a network connection.

Does it upload my code or searches?

No. Metadata checks request five public JSON files and include no source code, search terms, favorites or usage.

Why does Insert create a file in my project?

In HTML and web-framework files, Insert connects the Auric icon CDN through a small loader file, `icons.auricartisan.com.js`. It is only created on an explicit Insert. Copy and Save never create files. See Connect the icon CDN.

How do I get self-contained SVG instead?

Use **Copy** or **Save** with the SVG format, export a ZIP, or insert into a CSS, SVG or other non-web file.

Does it include Auric artwork or emoji?

No. Icon Hub covers the seven open-source collections. Auric Artisan Icon Studio adds the Auric Modern, Drawn and Playful artwork and emoji.

Can I use exported icons commercially?

Each collection keeps its own licence: MIT (Phosphor, Tabler, Heroicons, Feather), ISC (Lucide), Remix v1.0 (Remix) and Apache-2.0 (Material). The ZIP export includes the licence notices.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Icon Hub: other information

Contents of the supporting pages for Auric Artisan Icon Hub.

Supporting pages for Auric Artisan Icon Hub: terms, shortcuts, limits, a comparison with the other Auric icon extensions, the release history and its network and privacy behaviour.

File	What it covers
glossary.md	Terms used in Icon Hub.
shortcuts.md	Keybindings, in-Hub keys and commands.
limits-and-accuracy.md	What Icon Hub can and cannot do.
related.md	Comparison with Icon Studio, the Auric collections, File Icons, Emoji and Icon Pack.
release-notes.md	User-facing history.
privacy.md	What is stored and exactly what uses the network.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Shortcuts and commands

Keybindings, keys inside the Hub and every command in Auric Artisan Icon Hub.

Keybindings

Action	Windows and Linux	macOS
Quick Search Icons	`Ctrl` + `Alt` + `Shift` + `I`	`Cmd` + `Alt` + `Shift` + `I`

Keys inside the Hub

Key	Action
`Ctrl` + `K` / `Cmd` + `K`	Focus search
Arrow keys	Move between cards or dropdown options
`Enter`	Open the inspector for the focused card
`F`	Star the focused card
`Home` / `End`	First or last dropdown option
`Escape`	Close a dropdown or the inspector

Commands

Type "Auric Icon Hub" in the Command Palette: **Open Icon Hub**, **Quick Search Icons**, **Smart Insert Icon**, **Copy Icon As...**, **Save Icon As...**, **Show Favorite Icons**, **Show Recent Icons**, **Refresh Icon Catalog**, **Clear Downloaded Icon Catalog**, **Open Auric Icon Tool**, **Clear Icon Favorites and Recents**.

The editor right-click menu has **Smart Insert Icon** and **Copy Icon As...**.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Glossary

Definitions of the terms used in Auric Artisan Icon Hub.

Term	Meaning
Bridge	The small `icons.auricartisan.com.js` loader file Icon Hub creates next to your source on the first CDN Insert.
Catalog metadata	The data describing icons (names, terms, counts), separate from the artwork itself.
CDN	A content delivery network: a web service that serves icon data and fonts to your pages.
CDN placeholder	An element such as `<span data-icon="set:name" …>` that the CDN script turns into an icon.
Content security policy	A website rule listing which sources may load scripts, fonts and data.
`currentColor`	A CSS keyword meaning "use the surrounding text colour".
Data URI	Text that embeds a file directly, starting `data:`.
Font-backed icon	An icon drawn from a font glyph (Material), not an SVG file.
Indexed variant	One style of one icon. Icon Hub indexes 32,802.
Selection	Icons you ticked for export. Up to 200 per export.
Smart Finder	A local panel of recent icons and a rotating sample of the catalog.
SVG sprite	One SVG file containing several icons as reusable symbols.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Limits and accuracy

What Auric Artisan Icon Hub can and cannot do.

Catalog

- 12,895 icons and 32,802 indexed variants in seven collections, bundled with the extension.
- Metadata updates never add artwork or sets; new artwork arrives with extension updates.
- The combined Remix sprite is excluded because it is not a standalone icon.
- Material is font-backed and never exported as SVG.

Output

- Stroke controls only affect stroke-based artwork.
- The eyedropper in the colour picker depends on the host.
- Contrast checks in the colour picker are guidance for choosing a colour against one background.
- Exports are capped at 200 icons each.

CDN insertion

- CDN pages need network access and a content security policy that allows the CDN.
- The bridge is created only for saved files in trusted workspaces; untitled files, untrusted workspaces and unsupported file types are refused.

- JavaScript imports need module-aware tooling; server-rendered frameworks need the bridge in a client entry.
- At release, the published README noted that cross-origin access to the CDN's catalog files and fonts had not yet been enabled, so pages on other origins could show blank icons until it is.
- Previously generated local font stylesheets are not deleted automatically, and old local-font spans are not rewritten.

Search

Search matches every word you type against the name, ID, category, search terms and set. It does not use synonyms beyond those search terms.

Licences

Each collection keeps its upstream licence. The extension itself is under the Auric Artisan Free Use Licence.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Privacy

What Icon Hub stores, exactly what uses the network, and what is never sent.

No account, telemetry or search upload is required.

What uses the network

Feature	What is contacted	When
Metadata checks	Five public JSON files under https://icons.auricartisan.com/data/	On activation (at most once per interval) and when you select **Refresh**. Can be turned off.
Open Auric Icon Tool and **Website ↗**	https://icons.auricartisan.com in your browser	Only when you select them.
Pages using CDN output	The CDN script, catalog and fonts	When a visitor loads your page, not from VS Code.

Metadata requests send no credentials and include none of your source code, search terms, favorites or usage history. Like any web request, they reveal ordinary connection information such as your IP address to the host.

To stop all catalog network checks, set `auricIconLib.remoteRefresh.enabled` to `false`.

What is stored

Data	Where
Favorites and recent icons	VS Code's extension storage on this machine.
Downloaded metadata	The extension's private storage folder; cleared by **Clear Downloaded Icon Catalog**.
Search, filters, selection and colour history	The Hub's saved view state.
Settings	Your VS Code settings under `auricIconLib.*`.

What is written to your project

Only the CDN bridge file, and only on an explicit Insert into a supported file in a trusted workspace. Copy, Save and browsing never create project files.

Security

The Hub's webview uses a content security policy with per-session nonces, validates every message, sanitises SVG before preview and export, and never runs the CDN loader inside VS Code.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Release notes

User-facing history of Auric Artisan Icon Hub.

0.1.0 — 23 September 2026

The first release of **Auric Artisan Icon Hub**, and the current Marketplace version.

Library and workspace

- 12,895 icons and 32,802 indexed variants across Phosphor, Tabler, Lucide, Remix, Heroicons, Feather and Material.
- A full editor panel and a compact Activity Bar sidebar, with sticky navigation and responsive cards.
- Search and refine by collection, style, category and licence, with sorting and a one-click filter reset.
- Favorites, recent icons and a working selection.
- Collapsible filter and appearance sections, compact by default in the sidebar.

Controls and accessibility

- Custom dropdowns for filters and the inspector's format chooser, with full keyboard support.
- The Auric colour picker: spectrum, colour-space inputs, eyedropper where supported, named colours, recent choices, shade ramps, harmonies, and WCAG and APCA contrast checks with AA and AAA adjustment.
- `currentColor` kept until you pick a colour.
- Keyboard and hover tooltips, announced actions, focus kept on cards during updates, and support for narrow layouts, light themes, high contrast and reduced motion.

Insertion and export

- CDN insertion for HTML and supported frameworks across all seven collections, including Material fonts, through a small reusable bridge created only on explicit Insert.
- Language-appropriate insertion, including JSX for React and named markup constants for plain JavaScript and TypeScript.
- Multi-selection insertion, read-only and rejected-edit reporting, and a clipboard fallback with a notification.
- SVG, JSX, React component, HTML, data URI, CSS and CDN output; SVG sprite and ZIP export with upstream licences.

Website and metadata

- **Open Auric Icon Tool** opens https://icons.auricartisan.com.

- Automatic metadata checks on activation with a configurable 24-hour cooldown, manual refresh and a command to clear downloaded metadata.
- Strictly limited, validated requests; the local catalog is kept when a check fails.

VS CODE EXTENSIONS › AURIC ARTISAN ICON HUB

Related extensions and comparison

How Icon Hub compares with Icon Studio, the Auric icon collections, File Icons, Emoji and the Icon Pack.

At a glance

Extension	Main job	Explorer file icon theme	Artwork	Network
Icon Hub	Seven open-source sets with colour and stroke controls, sprite and ZIP export, and CDN insertion.	No	12,895 icons	Optional metadata checks; CDN output loads from the web
Icon Studio	One offline studio for every Auric collection, emoji and the same seven open-source sets.	No	17,234 entries in 11 collections	None
Auric Icons: Modern	Modern icon library plus the Modern file and folder icon theme.	Yes	720 library icons; 408 theme icons	None
Auric Icons: Drawn	Warm Drawn icon library.	No	480 icons	None
Auric Icons: Playful	Emoji-style icon library.	No	1,225 icons	None
File Icons	Explorer file and folder icons in three styles, plus File Icon Studio.	Yes: Facet, Playful and Modern	1,464 SVG assets	None
Emoji	Emoji Studio.	No	1,914 emoji	None
Icon Pack	Installs Modern, Playful, Drawn and Emoji together.	Through Modern	None of its own	Needs the Marketplace to install

Icon Hub or Icon Studio?

You want to	Use
Recolour, restroke and resize icons with a full colour picker	Icon Hub
Export an SVG sprite or a ZIP with licences	Icon Hub
Load icons on your site from the Auric CDN	Icon Hub
Browse Auric Modern, Drawn and Playful artwork and emoji too	Icon Studio
Work with no network access at all	Icon Studio (Icon Hub also works offline if you turn off metadata checks and avoid CDN output)

The two keep separate settings, views, favorites and recents. Icon Hub keeps its own view even when Icon Studio hosts the Auric collections.

Other related products

- Icon Library on the website — the same seven collections in your browser.
- Icon CDN — the icon delivery service used by CDN output.
- Install and set up

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Auric Artisan Live

What Auric Artisan Live does, who it is for, how to install it and where to find every guide.

Auric Artisan Live (full name **Auric Artisan Live — Live Server & Preview**) is a local development server inside VS Code. Run **Go Live** and it serves your project folder over a real HTTP server on your own machine, opens it in your browser or in a preview beside your code, and updates connected pages when you save. It works for plain HTML, CSS and JavaScript, and it can also run or connect to a backend: PHP, Python, Ruby, Go, Java, Node.js and more.

Around the server it gives you the tools you normally need a second terminal or a browser extension for: HTTPS with a generated certificate, an API proxy so your page and your backend share one origin, mock endpoints and a JSON-file REST database for APIs that do not exist yet, a request inspector with curl and fetch copies and HAR export, slow-network simulation, a phone QR code for testing on a real device, and an in-page developer dashboard (`Alt` + `Shift` + `D`) showing page health, network, storage and server state.

Everything runs locally. The extension has no runtime npm dependencies and no telemetry. It is a development server, not production hosting.

Who it is for

- Front-end developers who want live reload and a quick preview while editing HTML, CSS and JavaScript.
- Full-stack developers who run a framework dev server or a backend and want one address for the page and its API.
- PHP developers who want `.php` pages to run instead of download.
- Anyone testing a site on a phone, over HTTPS, or on a slow connection.
- Developers debugging requests: which rule answered, how long it took, what was sent.

Where to find it

- Marketplace ID: `auric-artisan.auric-artisan-live`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-live
- Install from a terminal:

```
code --install-extension auric-artisan.auric-artisan-live
```

- Requires VS Code 1.95 or newer and a folder on disk. Version 0.2.1 at the time of writing.
- Interface in English and Hindi. See Use an extension in Hindi.

Quick start

1. Install the extension and open your website folder in VS Code (**File** › **Open Folder...**).
2. Run **Auric Live: Go Live** from the Command Palette, press `Ctrl` + `Alt` + `L` (`Cmd` + `Alt` + `L` on macOS), or select **Go Live** in the status bar.
3. Your default browser opens at `http://localhost:5500` (or the next free port). A notification offers **Open**, **Preview** and **Phone QR**.
4. Edit an HTML, CSS or image file and save it. Connected pages update: stylesheets and images swap in place, HTML reloads.

5. Press `Ctrl` + `Alt` + `K` to open a preview beside your editor, and pick a device size.
6. Open the **Auric Live** icon in the Activity Bar to see the **Servers**, **Requests** and **Routes and Mocks** views.
7. Press `Ctrl` + `Alt` + `Shift` + `I` to open the control panel for requests, run targets, routing and session settings.
8. Stop with **Auric Live: Stop Server**, or stop every server with `Ctrl` + `Alt` + `Shift` + `L`.

What you can do

- Serve a folder, a subfolder or a build output (`dist`, `public`) with live reload on save.
- Swap changed CSS and images without a full reload, and restore scroll position after one.
- Preview beside your code at responsive, phone, tablet, laptop and desktop sizes, with rotation.
- Open the site on your phone with a locally generated QR code.
- Serve over HTTPS with a generated or imported certificate.
- Run a detected backend or framework dev server (PHP, Python, Ruby, Go, Java, Node.js, Deno, Bun, .NET, Rust, Elixir, Hugo, Jekyll and more), or your own command, and forward to it.
- Connect to a backend that is already running, and wait for it while it starts.
- Proxy API paths to a backend, including WebSockets.
- Answer mock endpoints and a full REST API from a JSON file.
- Run PHP through `php-cgi`.
- Inspect, filter, sort, replay and export requests as HAR; copy any request as curl or fetch.
- Simulate slow 3G, 3G, 4G, cable or custom networks for every client, including a phone.
- Open an in-page developer dashboard with health, network, server, storage, environment, request and console tabs.
- Keep the setup with the repository in `auric.live.json` and `.auricignore-live`.

In this folder

- docs/ — task guides and the full reference.
  - Getting started
  - Serve a folder with live reload
  - Preview, devices and phone testing
  - Run a backend or framework dev server
  - Run PHP sites
  - Proxy an API, mock endpoints and a REST database
  - Inspect requests
  - Use the in-page browser dashboard
  - Serve over HTTPS
  - Simulate a slow network
  - Project configuration
  - Reference: commands, keys and settings
  - Troubleshooting
  - FAQ
- others/ — supporting material.
  - Glossary
  - Shortcuts
  - Limits and safeguards
  - Privacy

- Related
- Release notes

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Auric Artisan Live guides

Contents of the Auric Artisan Live task guides and reference.

Start with the getting-started guide for a tour of every surface. Each task guide covers one workflow from start to result. The reference lists all 54 commands, the seven keyboard shortcuts and all 103 settings.

File	What it covers
getting-started.md	Install, the screen tour (status bar, sidebar views, control panel, preview, dashboard) and a first live-reload session
serve-and-live-reload.md	Go Live, choose the folder and port, reload triggers, hot swap, reload-storm protection, exclusions
preview-and-devices.md	The preview beside the editor, device presets, rotation, network address and phone QR code
run-targets-and-backends.md	Detected runtimes and framework dev servers, custom commands, URL-only backends, readiness
php-and-cgi.md	Run `.php` pages through `php-cgi` or the PHP built-in server
api-proxy-and-mocks.md	Proxy rules, mock routes with placeholders, the JSON REST database
inspect-requests.md	The Requests view and panel section: filter, sort, details, replay, curl/fetch, HAR
browser-dashboard.md	The in-page dashboard's seven tabs, console capture and the Auric cursor
https-and-certificates.md	Generated and imported certificates, trust on computers and phones
network-simulation.md	Network profiles, custom latency and bandwidth, what shaping covers
project-configuration.md	`auric.live.json`, precedence, session overrides, `.auricignore-live`, session records
reference.md	Every command, keybinding, menu entry and setting
troubleshooting.md	Symptom, cause and fix
faq.md	Short answers to common questions

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Getting started with Auric Artisan Live

Install Auric Artisan Live, learn where each control lives, and run a first live-reload session.

This guide installs the extension, tours every surface it adds to VS Code, and ends with a page that updates when you save.

Install

1. Check that VS Code is version 1.95 or newer (**Help** › **About**).
2. In the Extensions view (`Ctrl` + `Shift` + `X`), search for `@id:auric-artisan.auric-artisan-live` and select **Install**. Or run:

   ```
   code --install-extension auric-artisan.auric-artisan-live
   ```

3. Open the folder that contains your website with **File** › **Open Folder...**. Auric Live serves folders; with no folder open, its view shows an **Open Folder** button instead.

Backend runtimes (PHP, Python, Node.js and so on) are not included. Install the ones your project needs separately.

The screen tour

Status bar

On the right of the status bar:

- **Go Live** (broadcast icon) when nothing is running. Select it to start a server.
- The **port number** (radio-tower icon) while a server runs, with `+1`, `+2` and so on when several workspace folders are live. Hover for a summary: run target, served root, live reload and connected pages, proxy rules and upstream state, mock routes, network profile and request count. Select it to open the **Auric Live** menu.
- A **spinning icon with a warning background** when the server is up but a proxied backend is not answering yet.

Hide the item with `auricLive.showStatusBar`.

The Auric Live sidebar

Select the **Auric Live** icon in the Activity Bar. It has three views.

Servers — one entry per workspace folder, showing its address or state. Expand a running server to see:

Row	What it shows	Select it to
Root	The folder being served	—
Run target	**Static files**, a detected runtime or your own command	Change the run target
Dev server	A supervised process, its state, port and restart count	Show its output
Upstream ready / not answering	Each proxied backend's state	Find it (when not answering)
Local	The `localhost` address	Open it in your browser
Network	Up to three addresses other devices can use	Show the phone QR code
Live reload	How many pages are connected, or **off**	—
Requests	Handled and failed counts	Open the request inspector
Certificate	**generated** or **provided** (HTTPS only)	Reveal the certificate
Config file	The project configuration file, if one exists	Open it

The view's title bar has **Go Live**, **Select Run Target...**, **Stop All Servers** and **Open Control Panel**, and its **...** menu has **Refresh**, **Open Settings** and **Show Log**. Hover a running server for inline **Open in Default Browser** and **Stop Server** buttons; right-click it for **Restart Server**, **Open Preview Beside**, **Copy Local URL**, **Copy Network URL** and **Show Phone QR Code**.

Requests — the most recent requests, each shown as method and path with status, which handler answered and the time taken. Select one for its details; right-click for **Replay Request**, **Copy Request as curl** and **Copy Request as fetch**. The title bar has **Clear Request Log** and **Export Requests as HAR**.

Routes and Mocks — every proxy rule (`/api → target`), active mock route (method, path, status and hit count), REST collection (address and record count) and CGI handler (`*.php › php-cgi`). The title bar has **Add API Proxy Route...**, **Add Mock API Route...** and **Find a Running Backend**; its **...** menu has **Open Mock Database**, **Open Project Config File**, **Restart Framework Dev Server** and **Show Dev Server Output**.

The control panel

Run **Auric Live: Open Control Panel** (`Ctrl` + `Alt` + `Shift` + `I`). It opens as an editor tab with five sections:

Section	Use it to
01 / Overview	Start, stop or restart; choose the active workspace; open in the browser or preview beside; copy addresses; show the phone QR code; control reload and network shaping
02 / Requests	Search, filter and sort captured requests, pause the display, open details, copy curl or fetch, replay, export HAR
03 / Runtime	Choose a detected or custom run target, check readiness and prerequisites, see its output, connect to an existing backend
04 / Routing	See and add proxy rules, mock endpoints, REST collections and CGI handlers
05 / Settings	Change session settings, open the project configuration or VS Code settings, import an HTTPS certificate

The header has the language switch (**Auto**, **English**, **हिन्दी**) and a link to auricartisan.com. The panel follows your light, dark or high-contrast theme and adapts to narrow editor splits. Its drop-down menus work with the keyboard: arrow keys, type-ahead and `Escape`.

Changes you make in the panel are *session overrides*: they apply until you close or reload the VS Code window, and they do not rewrite your settings or project file. To keep a choice, put it in `auric.live.json` or VS Code settings. See Project configuration.

The Auric Live menu

Select the port number in the status bar, run **Auric Live: Open Live Menu**, or press `Ctrl` + `Alt` + `Shift` + `K`. It lists every common action with its current state, such as the network profile, whether HTTPS and live reload are on, the port and the served folder.

Editor and Explorer entry points

- Right-click a folder in the Explorer for **Serve This Folder** and the **Auric Live** submenu.
- Right-click an `.html`, `.htm`, `.xhtml`, `.php` or `.md` file in the Explorer for **Open with Auric Live**. For HTML, XHTML and PHP files the same entry is on the editor's context menu and the editor tab's context menu.
- The editor title bar's **Run** menu has an **Auric Live** submenu: **Go Live**, **Select Run Target...**, **Add Custom Run Command...**, **Open Preview Beside**, **Open in Default Browser**, **Show Phone QR Code**, **Restart Framework Dev Server**, **Stop Server** and **Open Live Menu**.

Your first live-reload session

1. Open a folder with an `index.html` that links a stylesheet.
2. Press `Ctrl` + `Alt` + `L` (or run **Auric Live: Go Live**). If you have several workspace folders, choose which one to serve.
3. Your browser opens at `http://localhost:5500`. A notification says Auric Live is serving the folder and offers **Open**, **Preview** and **Phone QR**.
4. If the project turns out to be a framework with its own dev server (such as Vite or Next.js) or a PHP site, Auric Live offers to set that up once. See Run a backend or framework dev server.
5. Change a colour in the stylesheet and save. The page updates without a reload, keeping its scroll position.
6. Change the HTML and save. The page reloads and returns to the same scroll position.
7. Press `Alt` + `Shift` + `D` in the browser to open the developer dashboard over the page.

You now have a served page that follows your saves. Next, read Serve a folder with live reload for every option, or Preview, devices and phone testing.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Proxy an API, mock endpoints and a REST database

Forward API paths to a backend, answer endpoints that do not exist yet, and serve a full REST API from a JSON file with Auric Live.

Auric Live can answer a request in several ways. For each request it tries, in order: a **mock route**, the **REST database**, a **proxy rule**, then **static files** (or a CGI handler). So you can forward a whole API to a real backend and still override one endpoint with a mock.

Proxy an API path to a backend

A *proxy* forwards requests for a path to another server and passes the answer back. With `/api` proxied to your backend, the page and its API share one origin: no CORS workaround, no hard-coded backend address in your front-end code, and cookies behave as they will in production.

Add a rule from VS Code

1. Run **Auric Live: Add API Proxy Route...**, or select the arrows button in the **Routes and Mocks** view title bar.
2. **Proxy route (1 of 2):** enter the path prefix to forward, for example `/api`. It must start with `/`.
3. **Proxy route (2 of 2):** enter where it should go, for example `http://localhost:3000`. It must start with `http://` or `https://`.
4. Auric Live says **Auric Live now forwards /api to http://localhost:3000. This lasts for the session.** Choose **Save to config file** to keep it in `auric.live.json`.

Rules added this way forward WebSockets and rewrite the `Host` header to the target.

Write rules in the project file

The short form maps a prefix to a target:

```
{
  "proxy": { "/api": "http://localhost:3000" }
}
```

The full form is a list of rules:

```
{
  "proxy": [
    {
      "path": "/api",
      "target": "http://localhost:3000",
      "changeOrigin": true,
      "ws": true
    }
  ]
}
```

Field	Default	Meaning
`path`	—	Path prefix to forward (also accepted as `context`)
`target`	—	Backend address; must be `http://` or `https://`
`pathRewrite`	none	Rewrites as pattern–replacement pairs, for example `{ "^/api": "" }`; each pattern is a regular expression
`stripPrefix`	`false`	Remove the matched prefix before forwarding
`changeOrigin`	`true`	Send the target's host as the `Host` header
`secure`	`false`	Verify the target's HTTPS certificate
`ws`	`true`	Forward WebSocket connections
`headers`	none	Extra request headers to send to the target
`cookieDomainRewrite`	`true`	Rewrite cookie domains so cookies from the backend work on the Auric Live address
`healthPath`	none	An absolute path that must answer 200–399 before the target counts as ready
`timeoutMs`	`30000`	How long to wait for the target (100 to 600000)

The **Routes and Mocks** view lists each rule as `/api →` with its target. Rules can also be set in the `auricLive.proxy` setting. Proxy rules need a trusted workspace.

While the backend starts, requests wait instead of failing. See "Waiting for a backend" in Run a backend or framework dev server.

A few details that keep a proxied site working:

- When rules overlap, the longest matching prefix wins.
- A redirect from the backend is pulled back to the Auric Live address, so a login redirect does not send you to the backend's own port.
- Cookies lose their `Domain` and, over plain HTTP, their `Secure` flag, so the browser keeps the session cookie.
- HTML pages that come through the proxy get the Auric script injected like static pages, so the developer dashboard works on a proxied site too.

Mock endpoints

A *mock route* is an endpoint Auric Live answers itself, with the status, headers, delay and body you declare. Use it to build a front end against an API that does not exist yet.

Add one from VS Code

1. Run **Auric Live: Add Mock API Route…**, or select the beaker button in the **Routes and Mocks** view title bar.
2. **Mock route (1 of 3):** choose the method: `GET`, `POST`, `PUT`, `PATCH` or `DELETE`.
3. **Mock route (2 of 3):** enter the path. Named parameters (`:id`) and wildcards (`*`) are supported.
4. **Mock route (3 of 3):** enter the JSON body. It must be valid JSON.
5. Auric Live turns mocks on and says **Auric Live now answers *GET /api/example*.** Choose **Save to config file** to keep it.

The status is 201 for `POST` and 200 otherwise.

Write routes in the project file

```
{
  "mock": {
    "enabled": true,
    "delayMs": 0,
    "routes": [
      {
        "method": "GET",
        "path": "/api/health",
        "status": 200,
        "body": { "ok": true }
      },
      {
        "method": "POST",
        "path": "/api/session",
        "status": 201,
        "delayMs": 250,
        "body": { "id": "{{uuid}}", "user": "{{body.user}}", "issued": "{{now}}" }
      }
    ]
  }
}
```

A short form maps `"METHOD /path"` to a body: `{ "GET /api/users": [ … ] }`. Without a method, `GET` is assumed.

Field	Default	Meaning
`method`	`GET`	HTTP method, or `*` for any
`path`	—	Route pattern; must start with `/`
`status`	`200`	Status code, 100–599
`headers`	none	Response headers
`body`	—	Response body (JSON or text)
`file`	—	A file to send as the body instead (needs a trusted workspace)
`delayMs`	`0`	Delay before answering, up to 60000; also accepted as `delay`
`id`	`route-N`	A name for the route
`description`	empty	Shown as the tooltip in the **Routes and Mocks** view
`disabled`	`false`	Keep the route but do not answer it

A route needs at least a `body`, a `file` or a `status`.

Route patterns

Pattern	Matches
`/users/:id`	`/users/42`, captured as `params.id`
`/files/:name?`	`/files` and `/files/report`, with an optional segment
`/assets/*`	one path segment, captured as `params.0`
`/docs/**`	the rest of the path, including slashes

A pattern without a wildcard matches only that exact path: `/api` does not answer `/api/users` unless you write `/api/**`. This keeps a mock from swallowing requests meant for the proxy.

Placeholders in bodies

Placeholder	Replaced with
`{{params.id}}`	A route parameter
`{{query.page}}`	A query-string value
`{{body.email}}`	A field of the parsed request body
`{{headers.origin}}`	A request header
`{{uuid}}`	A fresh random identifier
`{{now}}`	The current time in ISO 8601
`{{timestamp}}`	The current time in milliseconds since 1970
`{{method}}`, `{{path}}`	The request's method and path
`{{counter}}`	A number that goes up by one per response
`{{random}}`	A random number from 0 up to 1

A string that is exactly one placeholder keeps the value's type, so `"id": "{{counter}}"` gives a number. Placeholders are substitution only; no code runs.

Turn mocks on and off

- **Auric Live: Toggle Mock API** switches mock answering on or off for the session.
- The control panel's **Overview** has **Answer mock routes**.
- `auricLive.mock.delayMs` adds latency to every mock and REST response, on top of each route's own delay.

A full REST API from a JSON file

The *REST database* turns a JSON file into a working REST backend: lists, reads, creates, replaces, updates and deletes, with filtering, sorting, pagination and relationships. Forms submit, lists paginate and deleted rows stay deleted.

Set it up

1. Run **Auric Live: Open Mock Database**, or choose **Open mock database** in the **Routes and Mocks** view's **…** menu.
2. If `db.json` does not exist, Auric Live offers **Create**. It writes a sample with `users`, `posts` and a `profile` object, and turns on mocks and the REST database for the session.
3. Edit the file. Each top-level key holding an array is a *collection*; a key holding an object is a single resource.

```
{
  "users": [ { "id": 1, "name": "Ada Lovelace" } ],
  "posts": [ { "id": 1, "userId": 1, "title": "First post" } ],
  "profile": { "theme": "dark" }
}
```

To keep it on, put this in `auric.live.json`:

```
{ "mock": { "enabled": true, "rest": { "enabled": true, "base": "/api", "database": "db.json" } } }
```

Setting	Default	Meaning
`auricLive.mock.rest.enabled`	`false`	Serve the REST database (mocks must also be enabled)
`auricLive.mock.rest.base`	`/api`	Prefix the collections are mounted under
`auricLive.mock.rest.database`	`db.json`	The file, relative to the served root
`auricLive.mock.rest.persist`	`false`	Write changes back to the file. Off, a restart returns to the original data

Changes to unrelated settings keep the in-memory data. The **Routes and Mocks** view lists each collection with its record count.

Routes

For a collection `users` under `/api`:

Request	What it does
`GET /api/users`	List records
`GET /api/users/1`	Read one record
`POST /api/users`	Create a record
`PUT /api/users/1`	Replace a record
`PATCH /api/users/1`	Merge fields into a record
`DELETE /api/users/1`	Delete a record

Query options

Query	Effect
`?_page=2&_limit=20`	Pagination, with `X-Total-Count` and `Link` headers
`?_start=10&_end=20`	Slice the list
`?_sort=name&_order=desc`	Sort; separate several keys with commas
`?_q=ada`	Search every field
`?status=active`	Exact match on a field
`?views_gte=100`	Operators: `_ne`, `_lt`, `_lte`, `_gt`, `_gte`, `_like`, `_in`, `_nin`
`?_embed=posts`	Include child records that point at this one
`?_expand=user`	Include the parent record this one points at

Relationships follow the singular-name convention: a post with `userId` belongs to a user.

A path outside the base, or a collection the file does not contain, falls through to the proxy and then to static files.

When this is working, the **Routes and Mocks** view lists your proxy rules, mock routes with hit counts, and REST collections, and the **Requests** view shows `proxy`, `mock` or `rest` as the handler for each answered request.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Use the in-page browser dashboard

Open the Auric Live developer dashboard over a served page to see page health, network delivery, server state, storage, environment, requests and console messages.

Auric Live can place a *developer dashboard* over any page it serves. It shows how the page is doing and what the server is doing, without opening browser developer tools, and it works on a phone too.

Open and close it

- In a served page, press `Alt` + `Shift` + `D`. Change the keys with `auricLive.dashboard.shortcut` (modifier names joined with `+`, for example `Ctrl+Shift+.`).
- Close it with the × button, or press `Escape` (an open drop-down closes first).

The dashboard is on by default (`auricLive.dashboard.enabled`). It is added to the page along with the live-reload script. If a framework's own dev server handles reloading, Auric Live still adds the dashboard on its own (`auricLive.dashboard.independent`).

The dashboard's styles and menus are kept separate from your page (in a Shadow DOM), so it does not restyle your site and your site does not restyle it. It makes no requests for external fonts or branding.

The header

Control	What it does
auricartisan.com ↗	Open the Auric Artisan website in a new tab
Shortcut badge	Shows the key combination that opens the dashboard
Use site cursor / **Use Auric cursor**	Switch between the Auric pointer and the page's own cursor
Pause metrics / **Resume metrics**	Stop or restart sampling
↻ (Refresh dashboard data)	Re-read storage and server data now
× (Close developer dashboard)	Close the dashboard

The subtitle shows **LOCAL DEVELOPMENT /** and the page address.

The seven tabs

Move between tabs with the mouse, or with the arrow keys, `Home` and `End` when a tab has focus.

Overview

- **Development health** — a summary status (**All systems nominal**, or findings marked **Critical**, **Warning** or **Review**), such as the live-reload connection being unavailable or the frame rate falling below 30 FPS.
- **Live page health** — frame rate, first contentful paint, largest contentful paint, interaction latency, layout shift, long tasks and DOM node count.
- **Runtime findings** and a **Delivery summary**: requests, reloads and the current reload trigger, time to first byte, DOM ready, window load and connected pages.

Network

Transfer overview and resource analysis for the page: resources by type, bytes transferred and decoded, the largest and slowest resources, and a request timeline. Cached responses are labelled either **Likely cached** (the browser's estimate) or **Confirmed** (the server answered from cache or with 304), so estimates are never presented as facts.

Server

The Auric server's state: connection, uptime, HTTP or HTTPS, bytes served, compression and cache policy, network profile, live reload and its trigger, proxy rules, mock routes, REST collections and CGI, and routing and project controls such as the ignore file and its patterns.

Storage

A summary of the page's storage: origin usage and quota, local storage, session storage, Cache Storage, IndexedDB, cookies and whether storage is persistent or best effort. It counts and sizes storage; it does not read stored values. Storage is only sampled while this tab is open or when you refresh.

Environment

Page context (address, viewport, device pixel ratio, DOM nodes, visibility, service worker), **Security and capabilities** (secure context, cross-origin isolation, online status, cookies, reduced motion, forced colours), **Device and browser** (language, logical processors, device memory, touch points, JavaScript heap) and **Connection** (type, round-trip time, downlink, data saver).

Requests

The server's recent requests for this page, with method, path, status, time, size and source.

Console

A copy of the page's browser console messages, when console capture is on (see below). Filter by level, **Copy console** (copies the sanitised messages) or **Clear console**.

Measurements the browser does not support show as **Unavailable** rather than a guess.

Console capture

Console capture is **off by default**. When on, the page's console messages are mirrored into the dashboard's **Console** tab and into the **Auric Artisan Live** output channel in VS Code.

1. Turn it on with **Auric Live: Toggle Browser Console Capture**, **Capture browser console** in the control panel, or `auricLive.console.enabled`.
2. Reload any page that was already open.
3. Open the dashboard and choose **Console**.

Setting	Default	Range	What it does
`auricLive.console.levels`	`error`, `warn`, `info`, `log`, `debug`	—	Levels to capture
`auricLive.console.maxEntries`	200	20–2000	Messages kept per page
`auricLive.console.maxMessageLength`	2000	100–10000	Characters kept from one console call
`auricLive.console.rateLimit`	20	1–100	Messages sent per second from each page

Sensitive fields are redacted and output is rate-limited. The page's own console keeps working normally, and the original console behaviour is restored when the page leaves. Captured messages are not added to session records.

The Auric cursor

While a page is served, Auric Live replaces the page's cursor with the Auric pointer, which has link, button and text states. Touch screens, coarse pointers and forced-colour (high-contrast) modes keep the native cursor. Switch it off at once with **Use site cursor** in the dashboard, or permanently with `auricLive.dashboard.customCursor`.

Scrolling and keyboard

- Scroll the dashboard with the mouse wheel or trackpad, even on pages that block scrolling.
- With the content focused, use the arrow keys, `Page Up`, `Page Down`, `Home` and `End`; `Space` and `Shift` + `Space` page up and down without triggering buttons.
- Scrolling inside an open drop-down stays in the drop-down.

Performance of the dashboard itself

Sampling is on demand. It pauses when the tab is hidden, runs local measurements during browser idle time, and asks the server for its state every `auricLive.dashboard.sampleIntervalMs` (default 1000 ms) only while the dashboard is open. Its own requests are excluded from the page's figures.

The dashboard and the website being served are not translated when you choose Hindi in the Auric Live panel.

When it is working, pressing `Alt` + `Shift` + `D` on a served page opens the dashboard on its **Overview** tab with live figures.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Serve over HTTPS

Serve your site over HTTPS with a generated or imported certificate, and make browsers and phones trust it.

Browsers only allow some features in a *secure context*: service workers, `navigator.clipboard`, the camera, microphone and geolocation. On `localhost` plain HTTP counts as secure, but on any other address, such as your computer's network address on a phone, it does not. Serving over HTTPS makes those features testable on a phone.

Two things are separate:

- **Encryption.** Auric Live encrypts the connection as soon as HTTPS is on.
- **Trust.** Whether the browser accepts the certificate without a warning depends on whether that device trusts it. Auric Live does not install trust on your computer or phone.

Turn HTTPS on with a generated certificate

1. Run **Auric Live: Toggle HTTPS**, or use **Toggle HTTPS** in the Auric Live menu.
2. Confirm **Enable HTTPS**. The message explains that the server will use a self-signed certificate and that the browser will warn once.
3. The server restarts on `https://`. Addresses you copy and the phone QR code now use `https://`.

4. The first time you open the site, the browser warns that the certificate is not trusted. Accept it for this address to continue.

To make it permanent, set `auricLive.https.enabled` to `true` in settings or `"https": { "enabled": true }` in `auric.live.json`.

The generated certificate is self-signed, is reused between sessions, and covers `localhost`, the IPv4 and IPv6 loopback addresses and your computer's current network addresses. **Auric Live: Reveal Development Certificate** shows the certificate file in your operating system's file manager, so you can install it as trusted on a device if you choose. If none has been generated yet, it says **Enable HTTPS first**.

Use your own certificate

For no browser warning at all, use a certificate your devices already trust, for example one made with a local certificate tool such as `mkcert`.

Import it with the command

1. Trust the workspace; certificate files need workspace trust.
2. Run **Auric Live: Use HTTPS Certificate and Key**, or **Import an HTTPS certificate** in the Auric Live menu, or **Configure HTTPS certificate** in the control panel's **Settings** section.
3. **Choose HTTPS certificate (PEM)** — pick a `.pem`, `.crt` or `.cer` file.
4. **Choose matching private key** — pick a `.pem` or `.key` file.
5. If the key is encrypted, enter its passphrase. It is kept for this session only and never saved.
6. Auric Live checks that the key matches the certificate and that today falls within the certificate's validity dates, then turns HTTPS on. The message says **HTTPS certificate configured until *date***, and reminds you whether each browser or device still needs to trust it.

Imported certificates apply for the session, like other panel changes.

Set it permanently

```
{
  "https": {
    "enabled": true,
    "certPath": "certs/localhost.pem",
    "keyPath": "certs/localhost-key.pem"
  }
}
```

The same keys exist as settings: `auricLive.https.enabled`, `auricLive.https.certPath` and `auricLive.https.keyPath`. Certificate paths need a trusted workspace.

Trust on a phone

1. Get the certificate file (**Reveal Development Certificate** for the generated one, or your own).
2. Transfer it to the phone and install it following the phone's own instructions for installing a certificate, then mark it as trusted where the phone asks.
3. Open the site from the QR code. If the browser still warns, the certificate is not trusted on that phone yet.

The control panel's phone guidance reminds you that the phone must also trust the HTTPS certificate.

Limits

- Auric Live never adds a certificate authority to your operating system, browser or phone.
- A certificate only matches the addresses it lists. If a device reaches your computer at an address the certificate does not cover, the browser warns.
- HTTPS here is for development. Auric Live is not a production server.

When HTTPS is on, the **Servers** view shows a **Certificate** row (**generated** or **provided**) and the local address begins with `https://`.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Inspect requests

See every request the Auric Live server handles, find slow or failing ones, and copy, replay or export them.

The *request inspector* records each request the Auric Live server handles: method, path, status, which handler answered, time taken, size, and (by default) headers and bodies. Credentials are masked before anything is stored.

You can inspect requests in three places:

- the **Requests** view in the Auric Live sidebar;
- the **02 / Requests** section of the control panel (**Auric Live: Open Control Panel** or **Auric Live: Open Request Inspector**);
- the **Requests** tab of the in-page browser dashboard.

The Requests view

Each row reads `METHOD /path` with `status · handler · time` beside it, for example `GET /api/users` — `200 · proxy · 34ms`. Select a row to open **Show Request Detail**. Right-click a row for:

- **Replay Request**
- **Copy Request as curl**
- **Copy Request as fetch**

The title bar has **Clear Request Log** and **Export Requests as HAR**.

The control panel's Requests section

The section searches the latest 200 captured requests.

Filter and sort

Control	Options
Search	Type part of a path, method, status or handler. `Ctrl` + `F` (`Cmd` + `F` on macOS) focuses it
Status	**All statuses**, **2xx**, **3xx**, **4xx**, **5xx**
Method	**All methods**, `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, `OPTIONS`
Handler	**All handlers**, `static`, `proxy`, `mock`, `rest`, `cgi`, `listing`, `redirect`, `error`, `client`, `live-reload`
Sort	**Newest first**, **Slowest first**, **Largest first**

Filters combine. Results are paged with **Previous** and **Next**. **Reset filters** clears them. The filters and your place in the list are kept while new requests arrive, and they survive switching the panel's language.

Pause the display

Pause feed freezes the list so you can read it; the server keeps capturing in the background. **Resume feed** catches up.

Totals

The section shows **Captured requests**, **Failed responses**, **Response bytes recorded** and a **p95 latency**. The p95 is the time within which 95 per cent of the *recent* requests shown were answered. It is a sample of the latest requests, not a figure for the whole session.

Request details

Open a path to see:

- the method, address, status, handler and timing;
- **Request headers**, **Request body**, **Response headers** and **Response body**, as far as they were captured;
- **Copy curl** and **Copy fetch**;
- **Replay request**.

`Escape` closes the details and returns focus to the list. Details load on demand, so a request that has since dropped out of the inspector shows **This request has expired or was cleared. Select a newer request.**

Copy as curl or fetch

Copy curl gives a command you can paste into a terminal; **Copy fetch** gives a JavaScript `fetch()` call you can paste into a browser console or code. Values that were redacted or not captured are missing from the copy; fill them in yourself.

Replay a request

Replay request sends the captured request to the server again.

- Methods other than `GET` and `HEAD` ask for confirmation first.
- A capture whose data was redacted, truncated, omitted or binary cannot be replayed safely. Auric Live refuses with **This capture was redacted, truncated, or contains a binary body. Copy as fetch or curl and supply the missing values before sending it.**

Export HAR

A *HAR file* (HTTP Archive) is a standard JSON format for a list of requests and responses that browsers and many tools can open.

1. Run **Auric Live: Export Requests as HAR**, or select **Export HAR**.
2. Choose where to save the file.
3. Auric Live reports **Exported *N* requests.** and offers **Open**.

HAR exports leave out request and response bodies unless you turn on `auricLive.inspector.harIncludeBodies`, because bodies often hold personal or application data. Review a HAR file before sharing it.

Privacy and limits

Safeguard	Default	Setting
Requests kept	500; the panel shows the newest 200	`auricLive.inspector.maxEntries`
Bodies captured	On, up to 256 KiB per body	`auricLive.inspector.captureBodies`, `auricLive.inspector.maxBodyBytes`
Credential headers	Always masked (`Authorization`, `Cookie`, `Set-Cookie` and similar)	—
Sensitive query and body fields	Redacted	`auricLive.inspector.redactSensitive`
Extra field names to redact	None	`auricLive.inspector.sensitiveFields` (case insensitive)
Bodies in HAR	Off	`auricLive.inspector.harIncludeBodies`
Recording	On	`auricLive.inspector.enabled`

Incomplete JSON is left out rather than risk exposing part of a credential. Redaction is not a general personal-data detector: names, addresses and free text can still be sensitive.

Traffic is kept separate per workspace folder. With several servers running, **Auric Live: Select Active Server** chooses which one the inspector shows.

Auric Live: Clear Request Log (or **Clear history** in the panel) empties the inspector.

Output channel logging

Separately from the inspector, `auricLive.requestLogging` writes one line per request to the **Auric Artisan Live** output channel (at debug level for successes, higher for failures). `auricLive.logLevel` sets how much reaches the channel: `off`, `error`, `warn`, `info` (default) or `debug`. Credential-like values are masked and paths are written relative to the served root.

When you load a page with the inspector open, its requests appear within a moment, each labelled with the handler that answered it.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Simulate a slow network

Slow down every client of the Auric Live server, including a phone, to test how your site behaves on slow connections.

Network shaping makes the server answer as if the connection were slower: it adds latency (a delay before each response) and limits bandwidth (how fast bytes flow). Because it happens in the server, it applies to every client, including a phone on your network where browser developer tools are not available.

Choose a profile

1. Run **Auric Live: Simulate Network Profile...**, choose **Simulate a network profile** in the Auric Live menu, or use **Network shaping** in the control panel's **Overview**.
2. Pick a profile. The current one is ticked.

Profile	Latency	Download	Upload
`off`	none	unlimited	unlimited
`slow-3g`	400 ms	400 kbps	400 kbps
`3g`	150 ms	1600 kbps	750 kbps
`4g`	40 ms	9000 kbps	3000 kbps
`cable`	8 ms	30000 kbps	10000 kbps
`custom`	your value	your value	your value

Rates are in kilobits per second. The choice lasts for the session. To keep it, set `auricLive.throttle.profile` in settings or `throttle.profile` in `auric.live.json`.

A custom profile

Set the profile to `custom` and choose the numbers:

Setting	Default	Range	Meaning
`auricLive.throttle.latencyMs`	0	0–60000	Delay added before each response
`auricLive.throttle.downloadKbps`	0	0 or more	Download ceiling; `0` means unlimited
`auricLive.throttle.uploadKbps`	0	0–1000000	Upload ceiling; `0` means unlimited

In the project file:

```
{
  "throttle": {
    "profile": "custom",
    "latencyMs": 150,
    "downloadKbps": 1600,
    "uploadKbps": 750
  }
}
```

What is shaped

- Static files, proxied responses, injected HTML, mock endpoints and REST responses are slowed down.
- Uploads (request bodies) are paced as well as downloads.
- Auric Live's own dashboard and status requests stay responsive.

What it is not

- It shapes each transfer separately. It is not a shared bandwidth budget across all connections.
- It does not drop packets and is not a packet-level network emulator.
- WebSocket messages are not bandwidth-shaped.
- The time a request spends in simulated latency does not count towards the request-body timeout (`auricLive.requestBodyTimeoutMs`).

When a profile is active, pages load visibly slower, the status bar tooltip lists the throttle, and the dashboard's **Server** tab shows the profile.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Run PHP sites

Run .php pages in Auric Live through php-cgi or the PHP built-in server, and run other CGI scripts.

A plain static server sends a `.php` file to the browser as a download. Auric Live can run it instead, in one of two ways.

Way	How it works	Best for
PHP through CGI	Auric Live runs each `.php` request through `php-cgi` and serves the result itself	Classic PHP sites made of `.php` pages, with live reload and the dashboard
PHP built-in server, Laravel or Symfony	Auric Live starts PHP's own server (or `php artisan serve`, or `symfony server:start`) and forwards to it	Front-controller applications and frameworks

CGI (Common Gateway Interface) is a standard way for a web server to run a program for each request and send back what it prints. With `php-cgi`, `$_GET`, `$_POST`, headers, cookies, sessions and `header()` calls all work.

Both ways need PHP installed on your computer, and both need a trusted workspace.

Run PHP through CGI

1. Install PHP so that `php-cgi` is on your `PATH`, or note its full path.
2. Open the folder and run **Auric Live: Go Live**. The first time, Auric Live notices the PHP files and offers **Enable PHP**. Choose it.

   Or choose **PHP through CGI** in **Auric Live: Select Run Target...**, or set `auricLive.cgi.enabled` to `true`.
3. Open a `.php` page. It runs, and its output is served.

When you choose **Enable PHP**, the index list becomes `index.php`, then `index.html`, so a folder opens its `index.php`.

CGI settings

Setting	Default	What it does
`auricLive.cgi.enabled`	`false`	Run matching files through a handler instead of serving them
`auricLive.cgi.handlers`	`{ ".php": "php-cgi" }`	The command for each file extension, for example `{ ".php": "php-cgi", ".cgi": "perl" }`
`auricLive.cgi.indexes`	`["index.php"]`	Extra index file names tried for a folder when CGI is on

The handler must be on your `PATH` or given as a full path, such as `C:\php\php-cgi.exe`. In the project file you can also set `cgi.timeoutMs` (default 30000) for how long a script may run before the request fails.

The **Routes and Mocks** view lists each handler, for example `*.php` › `php-cgi`. If a script fails, the page shows an error overlay (`auricLive.liveReload.overlay`) and the output channel has the details.

Run the PHP built-in server or a framework

In **Auric Live: Select Run Target...**, choose:

- **Laravel** when `artisan` is present: runs `php artisan serve --port=8000`.
- **Symfony** when `symfony.lock` is present: runs `symfony server:start --port=8000 --no-tls`.
- **PHP built-in server** when the folder has `composer.json`, `index.php` or `.php` files: runs `php -S 127.0.0.1:8000 -t <docroot>`, where the document root is `public` if `public/index.php` exists.

Auric Live starts the process, waits for it, and forwards requests to it. See Run a backend or framework dev server for supervision, port adoption and restarts.

In an untrusted workspace

CGI settings (`auricLive.cgi.enabled`, `auricLive.cgi.handlers`, `auricLive.cgi.indexes`) are ignored until you trust the folder, even when a project file sets them. Choosing a PHP run target shows **Trust this workspace before running CGI, commands, or proxy profiles.**

When PHP is running, opening a `.php` page shows the rendered page, not its source, and the **Requests** view shows `cgi` as the handler.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Preview, devices and phone testing

Preview your served site beside the editor at device sizes, and open it on a real phone with a locally generated QR code.

The preview beside the editor

The preview is a browser frame inside VS Code, opened next to your code. It shows the served site and follows live reload exactly as a browser does.

Open it

- **Auric Live: Open Preview Beside** — `Ctrl` + `Alt` + `K` (`Cmd` + `Alt` + `K` on macOS).
- **Preview** in the notification that appears when the server starts.
- **Preview beside** in the control panel's **Overview**.
- Set `auricLive.open` to `preview` to open it every time the server starts.

If no server is running, opening the preview starts one first. There is one preview tab; opening it again moves it to the new address.

The toolbar

Control	What it does
Reload	Reload this frame
Address box	Type an address and press `Enter`, or select **Go**
Device size	Choose a size from the list below
Rotate	Swap width and height
Reload all pages	Reload every connected browser, not only this frame
Open in browser	Open the current address in your default browser
Size readout	The frame's width × height, and the scale when it is shrunk to fit

Control	What it does
Auric Artisan ↗	Open auricartisan.com in your browser

Device sizes

Device	Width × height (CSS pixels)
Responsive	Fills the panel
iPhone SE	375 × 667
iPhone 15	393 × 852
Pixel 8	412 × 915
iPad	820 × 1180
Laptop	1280 × 800
Desktop	1600 × 900

When a size is larger than the panel, the frame is scaled down to fit and the readout shows the percentage.

What the preview is and is not

- It simulates a viewport size. It does not emulate a device's browser, touch input, pixel density or performance. Use a real device for those (see below).
- Page state lives inside the frame. Developer tools and the network tab belong in a real browser.
- The page cannot reach VS Code itself: the frame is isolated from the editor.
- The preview toolbar uses the language chosen in the Auric Live panel. Reopen an existing preview to change its language.

Test on a real phone

What you need

- The phone and the computer on the **same network**.
- The server bound to your network. This is the default in a trusted workspace (`auricLive.host` is `0.0.0.0`). If you set `127.0.0.1`, a phone cannot reach it.
- Your firewall allowing VS Code to accept connections. It may ask the first time.

Show the QR code

1. Start the server.
2. Run **Auric Live: Show Phone QR Code**, choose **Phone QR** in the start notification, select a **Network** row in the **Servers** view, or open **Phone QR** in the control panel's **Overview**.
3. The QR code appears inside the Auric Live panel, with the address and connection guidance.
4. Scan it with the phone's camera, or use **Copy phone address** and send the address to the phone.

If the server is bound to this machine only, Auric Live says a phone cannot reach it and offers **Bind to the network** (switches the host for this session) or **Show anyway**.

The QR code is generated on your computer; no address is sent to a QR service. It clears itself when the server stops or its address changes, so a stale code cannot point at the wrong place.

Several network adapters

Auric Live: Copy Network URL lists every address when there is more than one. Addresses from virtual adapters (such as virtual machines or VPNs) are marked as probably unreachable from a phone. The **Servers** view shows up to three network addresses.

HTTPS on a phone

Features such as service workers, the clipboard, camera, microphone and geolocation need a secure connection on any address other than `localhost`. Turn on HTTPS to test them on a phone, and make the phone trust the certificate. See Serve over HTTPS.

Slow connections on a phone

Network simulation applies to every client, including a phone, where browser developer tools are not available. See Simulate a slow network.

With the phone on the same network, scanning the code opens the same served site, and it updates when you save, just like the desktop browser.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Project configuration

Keep an Auric Live setup with your repository in auric.live.json, control exclusions with .auricignore-live, and understand how settings, project files and session changes combine.

Where settings come from

Auric Live combines four layers. A later layer wins:

1. **Built-in defaults.**
2. **VS Code settings** — user, workspace and folder settings with the `auricLive.` prefix.
3. **The project file** — `auric.live.json` in the workspace folder.
4. **Session overrides** — changes you make with commands, the Auric Live menu or the control panel. They last until you close or reload the VS Code window.

So a repository can carry its own proxy, mock and CGI setup, and you can still try something for a moment without editing a file.

The project file: auric.live.json

Create it

1. Run **Auric Live: Create Project Config File**, or select **Create a config file** in the **Servers** view when nothing is running.
2. Auric Live writes a starter `auric.live.json` in the workspace folder and opens it.

The starter file:

```
{
  // Auric Artisan Live — project configuration.
  // Every key is optional and overrides the matching auricLive.* setting.
  "port": 5500,
  "spa": { "enabled": false, "fallback": "/index.html" },
  "cleanUrls": false,

  // Send /api to a backend you already run, so the front end talks to the
  // real thing without a CORS shim.
  "proxy": [
    {
      "path": "/api",
      "target": "http://localhost:3000",
      "changeOrigin": true,
      "ws": true
    }
  ],

  // Or answer /api yourself while that backend does not exist yet.
  "mock": {
    "enabled": false,
    "routes": [
      {
        "method": "GET",
        "path": "/api/health",
        "status": 200,
        "body": { "ok": true, "service": "auric-live" }
      },
      {
        "method": "POST",
        "path": "/api/session",
        "status": 201,
        "delayMs": 250,
        "body": { "id": "{{uuid}}", "user": "{{body.user}}", "issued": "{{now}}" }
      }
    ],
    "rest": {
      "enabled": false,
      "base": "/api",
      "database": "db.json"
    }
  },

  "headers": [
    { "match": ["**/*.html"], "headers": { "X-Frame-Options": "SAMEORIGIN" } }
  ],

  "liveReload": { "enabled": true, "reloadOnSave": "save", "cssHotSwap": true, "overlay": true },

  // Alt+Shift+D opens the in-page developer dashboard. Diagnostics are saved
  // locally under .auric-live/ without headers, bodies, queries or values.
  "dashboard": { "enabled": true, "customCursor": true },
  "records": { "enabled": true, "maxRequests": 200 }
}
```

Rules for the file

- Every key is optional. Keys are the setting names **without** the `auricLive.` prefix, nested as objects: the setting `auricLive.liveReload.cssHotSwap` is `"liveReload": { "cssHotSwap": … }`.
- Comments (`//` and `/* … */`) and trailing commas are allowed.

- A `$schema` key is ignored.
- The file is looked for in the workspace folder first, then in `.auric/`.
- Change the file name with the `auricLive.configFile` setting.
- If the file is not valid JSON, Auric Live shows **Auric Live: auric.live.json is not valid JSON** with the error, and ignores the file until it is fixed.

Open and reload

- **Auric Live: Open Project Config File** opens it. If there is none, it offers **Create it**.
- Changes apply as soon as you save, while `auricLive.autoRestart.onConfigChange` is on (the default). **Auric Live: Reload Configuration** re-reads everything by hand.
- Several commands offer **Save to config file** after a session change: adding a proxy route or a mock route, and the **Write config** choice when a framework is detected.

A fuller example

```
{
  "host": "127.0.0.1",
  "port": 5500,
  "open": "none",
  "liveReload": {
    "enabled": true,
    "reloadOnSave": "save",
    "cssHotSwap": true,
    "preserveScroll": true
  },
  "dashboard": {
    "enabled": true,
    "independent": false
  },
  "proxy": [
    { "path": "/api", "target": "http://localhost:3000" }
  ],
  "inspector": {
    "sensitiveFields": ["customerPin"],
    "harIncludeBodies": false
  },
  "console": {
    "enabled": true,
    "levels": ["error", "warn", "info"]
  },
  "throttle": {
    "profile": "custom",
    "latencyMs": 150,
    "downloadKbps": 1600,
    "uploadKbps": 750
  },
  "maxRequestBodyBytes": 16777216,
  "requestBodyTimeoutMs": 30000
}
```

Additional project-file keys

These keys have no VS Code setting of their own but are accepted in `auric.live.json`:

Key	Default	Meaning
`mount`	`/`	Serve the site under a sub-path, such as `/app`
`trailingSlash`	`preserve`	`preserve`, `add` or `remove` a trailing slash, by redirect
`liveReload.fullReload`	`[]`	Globs whose changes always cause a full reload instead of a hot swap
`watch.extraPaths`	`[]`	Additional absolute paths to watch
`cors.methods`	`GET`, `HEAD`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`	Methods allowed on cross-origin requests
`cors.headers`	`*`	Request headers allowed on cross-origin requests
`cors.exposeHeaders`	`[]`	Response headers exposed to cross-origin scripts
`cors.maxAge`	`600`	Seconds a browser may cache a preflight answer
`auth.realm`	`Auric Live`	Name shown in the browser's login prompt
`cgi.timeoutMs`	`30000`	How long a CGI script may run
`mock.rest.pageSize`	`10` when unset	Page size when `_page` is given without `_limit`
`maxPortAttempts`	`40`	How many ports `increment` tries before giving up

Workspace trust and the project file

In an untrusted workspace, a project file cannot switch on anything that runs code or opens the server to other devices. These keys are ignored until you trust the folder: `cgi.enabled`, `cgi.handlers`, `cgi.indexes`, `proxy`, `host`, `auth.username`, `auth.password`, `https.certPath`, `https.keyPath` and `mock.rest.persist`. Run commands, proxying, CGI, file-backed mocks, custom certificate files and listening on your network all need trust. Static serving, live reload and request inspection keep working on this machine.

Exclusions: .auricignore-live

`.auricignore-live` lists paths Auric Live must leave completely alone. A matched path is never served, listed, watched, hot-swapped, reloaded or recorded.

1. Run **Auric Live: Open .auricignore-live**. If the file does not exist, a commented starter is created in the served folder.
2. Add one pattern per line, relative to the served folder.
3. Save. The patterns apply straight away.

```
# Auric Artisan Live project exclusions
# One glob per line, relative to the served root. Later !rules re-include a path.
private/**
generated/**
!generated/preview/**
```

The syntax is a small, predictable part of the `.gitignore` syntax:

- blank lines and lines starting with `#` are ignored;
- `*`, `**` and `?` are wildcards;
- a pattern ending in `/` matches a folder and everything inside it;
- a pattern starting with `/` is anchored to the served root;

- a later line starting with `!` re-includes something an earlier line excluded.

Choosing **Ignore *folder/*** after a reload storm adds a line here for you.

Session records: .auric-live/

Auric Live keeps a small local record of each development session under `.auric-live/` in the workspace folder: sanitised request summaries and the page's performance summary. It never stores query strings, headers, bodies, cookies, IP addresses, user agents or storage values, and the folder is never served.

Setting	Default	Range	Meaning
`auricLive.records.enabled`	`true`	—	Keep session records
`auricLive.records.maxRequests`	200	20–5000	Request summaries per session
`auricLive.records.maxSessionBytes`	5242880 (about 5 MiB)	65536–104857600	Size of one session record; oldest summaries go first
`auricLive.records.maxSessions`	12	1–100	Recent sessions kept under `.auric-live/sessions/`
`auricLive.records.flushIntervalMs`	1000	250–60000	Minimum time between writes
`auricLive.records.telemetryIntervalMs`	5000	1000–60000	How often a served page sends its performance summary to the record

Auric Live: Open Recorded Live Session opens `.auric-live/latest.json`. If nothing has been recorded yet, it says to start the server and open a served page first. Browser console messages are never added to session records.

You may want to add `.auric-live/` to your `.gitignore`.

Multi-root workspaces

Each workspace folder has its own server, settings, project file and ignore file. Settings with resource scope (almost all Auric Live settings) can differ per folder. Unrelated configuration changes keep API routes, session settings and in-memory REST data.

When your configuration is in place, the **Servers** view shows a **Config file** row for the running server, and the settings apply every time you go live.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Run a backend or framework dev server

Let Auric Live run your project's backend or framework dev server (PHP, Python, Ruby, Go, Java, Node.js and more), or connect to one already running, and serve it through one address.

A *run target* is what actually serves your site. By default it is **Static files**: Auric Live serves the folder itself. For a project with a backend or a framework dev server, you can choose a different run target. Auric Live then starts that process, waits for it to be ready, and forwards requests to it, so you still use one address with the dashboard, request inspector, network simulation and phone QR code.

Auric Live does not install runtimes or packages. PHP, Python, Ruby, Go, Java, Node.js and the rest must already be installed and on your `PATH`, and the project's dependencies must already be installed.

Running a command needs a trusted workspace. In Restricted Mode only static files are served.

Choose a run target

1. Run **Auric Live: Select Run Target...** (`Ctrl` + `Alt` + `J`), select the **Run target** row in the **Servers** view, or use the control panel's **03 / Runtime** section.
2. The list, titled **Run target — *folder name***, shows in this order:
   - your saved commands (from `auricLive.runProfiles`);
   - the detected framework dev server, if any;
   - detected language runtimes;
   - **Static files**, which is always available.

   Each entry shows its command and why it was detected, for example `manage.py is present`. The current target has a tick.
3. The list also offers **Add a custom command...**, **Edit saved commands...** and **Find a running backend...**.
4. Pick a target. If the server was not running, it starts.

What happens next depends on the kind of target:

Kind	What Auric Live does
Static files	Serves the folder itself, with Auric live reload
A server command	Starts the command, forwards `/` to its port and waits while it starts
PHP through CGI	Runs `.php` files per request with `php-cgi`. See Run PHP sites
An existing server (URL only)	Forwards to a server you already run, without starting or stopping it

With a server command, Auric live reload is switched off for that session so it does not compete with the framework's hot reload, and the developer dashboard is still added to pages.

Auric Live: Show Detected Run Targets writes every target it can offer for the folder, with the reason for each, to the output channel.

What is detected

Detection is based on evidence in the folder: a file that exists, a dependency that is declared, an entry point that is present. It suggests a target; it does not guarantee every framework configuration works.

JavaScript framework dev servers

Detected from `package.json` dependencies. The command uses your package manager, chosen from the lockfile: `pnpm run`, `yarn`, `bun run` or `npm run`. Auric Live prefers the script the project declares (`dev`, `start` or `serve`).

Framework	Usual script	Usual port
Next.js	`dev`	3000
Nuxt	`dev`	3000
SvelteKit	`dev`	5173
Astro	`dev`	4321

Framework	Usual script	Usual port
Remix	`dev`	3000
Angular	`start`	4200
Gatsby	`develop`	8000
Create React App	`start`	3000
Vue CLI	`serve`	8080
Eleventy	`start`	8080
Vite	`dev`	5173
Express	`start`	3000
Any project with a `dev`, `start` or `serve` script	that script	3000

Language runtimes

Language	Target	Detected when	Command	Port
PHP	Laravel	`artisan` is present	`php artisan serve --port=<port>`	8000
PHP	Symfony	`symfony.lock` is present	`symfony server:start --port=<port> --no-tls`	8000
PHP	PHP built-in server	`composer.json`, `index.php` or any `.php` file	`php -S 127.0.0.1:<port> -t <docroot>` (docroot `public` when `public/index.php` exists)	8000
PHP	PHP through CGI	the folder contains `.php` files	runs per request	—
Python	Django	`manage.py` is present	`python manage.py runserver <port>`	8000
Python	FastAPI	`fastapi` in `requirements.txt` or `pyproject.toml`, or `main.py` creates a FastAPI app	`uvicorn main:app --reload --port <port>`	8000
Python	Flask	`flask` declared, or `app.py` or `wsgi.py` creates a Flask app	`flask run --port <port>`	5000
Python	Python http.server	the folder contains `.py` files	`python -m http.server <port>`	8000
Ruby	Ruby on Rails	`bin/rails` is present	`ruby bin/rails server -p <port>`	3000
Ruby	Jekyll	`_config.yml` is present	`bundle exec jekyll serve --port <port>`	4000
Ruby	Sinatra	`sinatra` in the `Gemfile`	`ruby app.rb -p <port>`	4567
TypeScript	Deno	`deno.json` or `deno.jsonc`	`deno task start`	8000
TypeScript	Bun	a Bun lockfile	`bun run dev`	3000
JavaScript	Node entry point	`package.json` `main`, or `server.js`, `app.js`, `index.js`, `main.js`, `src/server.js`, `src/index.js`	`node <entry>`	3000
C#	ASP.NET Core	a `.csproj` or `.sln` file	`dotnet watch run --urls http://localhost:<port>`	5000
Java	Spring Boot (Maven)	`pom.xml` declares Spring Boot	`mvnw spring-boot:run` (or `mvn`)	8080

Language	Target	Detected when	Command	Port
Java	Spring Boot (Gradle)	the Gradle build declares Spring Boot	`gradlew bootRun` (or `gradle`)	8080
Go	Go	`go.mod` is present	`go run .`	8080
Rust	Rust	`Cargo.toml` is present	`cargo run`	8080
Elixir	Phoenix	`mix.exs` declares Phoenix	`mix phx.server`	4000
Go	Hugo	`hugo.toml`, `hugo.yaml` or `config.toml`	`hugo server --port <port>`	1313

Only one generic fallback is offered per language: a Django project is not also offered `python -m http.server`. For Maven and Gradle, the project's wrapper script (`mvnw`, `gradlew`) is used when it exists.

The first-run suggestion

The first time you go live in a folder that is clearly not a plain static site, Auric Live says so once:

- For a framework: **This looks like a *Vite* project, which has its own dev server on port *5173*.** Choose **Start it and proxy**, **Write config** (saves a suggested setup to `auric.live.json`) or **Not now**.
- For PHP: **This looks like a PHP project.** Choose **Enable PHP** or **Not now**.

Serving a Vite or Next.js project as plain files shows an unbuilt page full of module errors; this prompt prevents that. Turn it off with `auricLive.suggestFrameworkSetup`. To skip the question and always start the detected dev server, set `auricLive.devServer.autoStart` to `true`.

Framework dev server commands

Command	What it does
Auric Live: Start Framework Dev Server...	Detect the framework, confirm the command (**Run: *command***), start it and forward to it
Auric Live: Stop Framework Dev Server	Stop it, and remove the forwarding it added
Auric Live: Restart Framework Dev Server	Restart it
Auric Live: Show Dev Server Output	Show its output (the **Auric Live: Run Target** output channel)

When it starts: **Auric Live started *Next.js* and is forwarding to it. Requests wait while it compiles, and pages reload when it restarts.**

Supervision

By default the dev server runs as a supervised process, which gives you three things:

- **Port adoption.** If the framework moves to another port because yours was taken, Auric Live reads the port it prints and forwards there (`auricLive.upstream.autoDetectPort`).
- **Restart on crash.** If it exits unexpectedly, it restarts with a growing delay (`auricLive.devServer.autoRestart`), up to `auricLive.devServer.maxRestarts` times (default 5). A clean exit is not restarted. After repeated failures you see **keeps failing** with **Show output** and **Try again**.
- **Reload on recovery.** Pages reload when the backend comes back (`auricLive.upstream.reloadOnRecover`).

If your command needs typed input, set `auricLive.devServer.useTerminal` to run it in a VS Code terminal instead. You lose port adoption, restart supervision and reload on recovery.

To override the detected command, set `auricLive.devServer.command`, for example `npm run dev -- --host`.

Add your own command

1. Run **Auric Live: Add Custom Run Command…**, or choose **Add a custom command…** in the run-target list.
2. **Run command (1 of 3):** paste the command that starts your site, exactly as you would type it. Environment variables written in front of it are read and applied to the process, in any of these shell styles:

   ```
   $env:API_MODE=mock; node server.js
   API_MODE=mock node server.js
   set API_MODE=mock && node server.js
   ```

   So a command copied from a PowerShell prompt also works when VS Code uses bash, and the other way round. A line that contains only assignments adds those variables to the currently selected run target.
3. **Run command (2 of 3):** give it a name.
4. **Run command (3 of 3):** enter the port it listens on, or leave it empty to read the port from the command's output when it starts.
5. Choose **Run it now** or **Not yet**.

The command is saved to `auricLive.runProfiles` in your workspace settings (per folder in a multi-root workspace), so it survives a restart. **Auric Live: Edit Saved Run Commands** opens that setting.

Run profile fields

You can also write profiles by hand, in settings or in `auric.live.json`. A profile can be just a command string, or an object:

Field	Meaning
`label`	Name shown in the list
`id`	Stable identifier; derived from the label if omitted
`description`	Extra text shown in the list
`command`	The command line; leading environment assignments are read
`env`	Environment variables as name–value pairs
`cwd`	Working directory, relative to the served root
`port`	Port the process listens on; `0` or omitted reads it from the output
`url`	An existing server to forward to (`http://` or `https://`, no embedded credentials). Without `command`, nothing is started
`kind`	`server`, `static` or `cgi`
`mount`	Path the target is forwarded under; default `/`
`healthPath`	An absolute path that must answer 200–399 before the target counts as ready
`autoStart`	`true` to apply this profile automatically on **Go Live**
`adoptPort`	`false` to stop Auric Live following the port the process prints
`shell`	A specific shell executable to run the command with

Connect to a backend you already run

A *URL-only* profile forwards to a server you started elsewhere. Auric Live never starts or stops it.

```
{
  "runProfiles": [
    {
      "id": "existing-api",
      "label": "Existing API",
      "url": "http://localhost:8000",
      "mount": "/api",
      "healthPath": "/health",
      "autoStart": true
    }
  ]
}
```

Find a running backend

Auric Live: Find a Running Backend scans the ports these tools usually use (or `auricLive.upstream.candidatePorts` if you set it), skipping Auric Live's own port.

- If something answers: **Something is listening on port *N*. Forward / to it?** Choose **Forward it**.
- If nothing answers: **Auric Live checked *N* common ports and nothing was listening.** Choose **Start the dev server** or **Show ports**.

The **Upstream not answering** row in the **Servers** view runs the same scan.

Waiting for a backend

When a proxied backend refuses the connection, it is usually still compiling. With `auricLive.upstream.wait` on (the default), a request waits up to `auricLive.upstream.waitTimeoutMs` (default 25 seconds) instead of failing with 502. After that, the page shows a waiting screen that reloads itself until the backend answers. Auric Live checks a silent backend every `auricLive.upstream.pollIntervalMs` (default 400 ms), then less often once it is up.

Readiness is a TCP connection by default. Give a profile or proxy rule a `healthPath` to require an HTTP answer from 200 to 399 instead. Readiness checks do not follow redirects.

Check prerequisites

Auric Live: Diagnose Project and Runtime opens a local report of the configuration, the run targets and whether each command's executable (and any configured shell) is found on your `PATH` or in the project. Credential fields are redacted. It inspects; it never runs or installs anything. The control panel's **03 / Runtime** section has **Run diagnostics** and shows the same readiness information.

When a run target is working, the **Servers** view shows **Dev server: *name*** as running with its port, **Upstream ready**, and the status bar shows the port without a warning background.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Serve a folder with live reload

Start the Auric Live server, choose what it serves and where, and control how and when connected pages update.

Live reload means the pages you have open in a browser update by themselves when you change a file. Auric Live adds a small script to every HTML page it serves; the script listens to the server and refreshes the page, or swaps just the changed stylesheet or image, when you save.

Start the server

Use whichever is closest:

- **Auric Live: Go Live** — `Ctrl` + `Alt` + `L` (`Cmd` + `Alt` + `L` on macOS), the **Go Live** status-bar item, the **Go Live** button in the **Servers** view, or the editor's **Run** menu › **Auric Live**.
- **Serve This Folder** — right-click a folder in the Explorer. That folder becomes the served root for this session.
- **Open with Auric Live** — right-click an HTML, XHTML, PHP or Markdown file. The server starts if needed and opens that file's address. A file outside the served root cannot be opened this way.

With several workspace folders, **Go Live** asks **Which folder should Auric Live serve?**. Each workspace folder gets its own server and port. **Auric Live: Select Active Server** chooses which one the panel and commands act on; **Auric Live: Stop All Servers** (`Ctrl` + `Alt` + `Shift` + `L`) stops every one.

When the server starts:

1. What opens depends on `auricLive.open`: `default` opens your default browser, `preview` opens the preview beside the editor, `none` opens nothing. `auricLive.openPath` adds a path, such as `/docs/index.html`, for sites whose entry page is not the root.
2. A notification says what is being served and where, with **Open**, **Preview** and (when a network address exists) **Phone QR**.
3. If the project looks like a framework with its own dev server, or a PHP site, Auric Live offers the matching setup once per folder. Turn this off with `auricLive.suggestFrameworkSetup`.

To start automatically whenever you open the folder, set `auricLive.startOnOpen` to `true`.

If the server cannot start, an error offers **Change Port** and **Show Log**.

Choose what is served

- **Served folder.** By default, the workspace folder itself. Set `auricLive.root` to a subfolder such as `dist` or `public` for a build output, or run **Auric Live: Change Served Folder...** and pick one for this session.
- **Index files.** A request for a folder tries `auricLive.index` in order (`index.html`, then `index.htm`). A folder without one shows a browsable listing unless you turn off `auricLive.directoryListing`.
- **Clean URLs.** With `auricLive.cleanUrls` on, `/about` serves `about.html`.
- **Single-page apps.** Turn on `auricLive.spa.enabled` so unknown paths return the fallback document (`auricLive.spa.fallback`, default `/index.html`), keeping client-side routes working on a deep link or hard refresh. Paths in `auricLive.spa.ignore` (by default `/api/**` and anything with a file extension) still return a real 404, so a missing script is not answered with HTML.
- **Custom 404.** Point `auricLive.notFoundPage` at a page in the served folder.
- **Dot-files.** `auricLive.dotfiles` is `ignore` by default, which hides paths such as `.env` and `.git`. `allow` serves them; `deny` refuses them.
- **Exclusions.** Paths listed in `.auricignore-live` are never served, listed, watched, hot-swapped, reloaded or recorded. See Project configuration.

Links that leave the served folder through a symbolic link or junction are blocked.

Choose the address

- **Port.** `auricLive.port` is `5500` by default. `0` lets the operating system choose a free port, useful for several projects at once. Change it for this session with **Auric Live: Change Port...**; ports below 1024 usually need administrator rights.
- **When the port is taken.** `auricLive.portStrategy`: `increment` walks upward to the next free port (default), `fail` reports the conflict (for a port your OAuth callback or code depends on), `random` picks a free port between 20000 and 40000.
- **Host.** `auricLive.host` is `0.0.0.0` in a trusted workspace, which also answers other devices on your network so you can test on a phone; your firewall may ask the first time. Set `127.0.0.1` to keep the server on this machine only. In an untrusted workspace the server stays on this machine.
- **Password.** `auricLive.auth.enabled` with `auricLive.auth.username` and `auricLive.auth.password` requires a login before anything is served. The password is kept in plain settings; it is a convenience for shared networks, not a secret store.

Copy addresses with **Auric Live: Copy Local URL** and **Auric Live: Copy Network URL**. If your computer has several network adapters, you choose one; virtual adapters are marked as probably unreachable from a phone.

Control how pages update

The reload trigger

`auricLive.liveReload.reloadOnSave` decides what may refresh the page:

Value	Behaviour	Use it when
`save` (default)	Only an explicit save in VS Code updates pages, once per save	Normal editing. Generated files, sync tools, formatters and caches cannot keep refreshing the page
`change`	Any change in the served folder triggers an update	An external compiler or build tool writes the files the browser shows
`off`	Nothing reloads automatically	You want the page completely still; use **Reload Connected Pages** when ready

In the control panel's **Overview**, the same choice appears as **VS Code saves**, **Filesystem changes** and **Manual only**.

What an update does

- **Stylesheets** swap in place without a reload (`auricLive.liveReload.cssHotSwap`), so the page keeps its scroll position, open dialogs and filled-in forms.
- **Images** refresh in place (`auricLive.liveReload.imageHotSwap`).
- **Everything else** reloads the page. After a full reload, the scroll position and open disclosure elements are restored (`auricLive.liveReload.preserveScroll`).
- **Failures** in a proxied request or CGI handler show as an overlay on the page (`auricLive.liveReload.overlay`) instead of only in the log.

The reload script goes at the end of `body` by default. Set `auricLive.liveReload.injectTo` to `head` if a page fails while loading and you need the script attached earlier.

Manual controls

Command	Keys	What it does
Auric Live: Reload Connected Pages	`Ctrl` + `Alt` + `U`	Reload every connected page now
Auric Live: Toggle Live Reload	—	Turn live reload on or off for this session
Auric Live: Resume Live Reload	—	Resume after reload-storm protection paused it

Reload-storm protection

If files keep changing, for example while a package installs or a sync tool rewrites files, reloading every time would make the page unusable. Auric Live allows `auricLive.liveReload.stormLimit` updates (default 3) within `auricLive.liveReload.stormWindowMs` (default 10 seconds). Beyond that it pauses live reload and shows **Auric Live paused live reload — files kept changing** with the files responsible. The page stays live. Choose:

- **Ignore *folder/*** — adds that folder to `.auricignore-live` and resumes;
- **Resume** — resumes as it was;
- **Show log** — opens the output channel.

What never triggers a reload

`auricLive.liveReload.ignore` lists paths that never trigger an update. The default covers `node_modules`, `dist`, `out`, `build`, `coverage`, `target`, `vendor`, `bin`, `obj`, `__pycache__`, `venv`, `tmp`, `temp`, log, temporary, swap, compiled and database files, and the lock and thumbnail files left by Office, OneDrive, Dropbox, Windows and macOS. Dot-folders such as `.next/`, `.vite/` and `.git/` are always excluded in addition.

In `change` mode, `auricLive.liveReload.debounceMs` (default 350 ms) waits for a quiet moment so a build writing many files produces one update.

Watching on network shares and virtual machines

Some file systems never report changes. Set `auricLive.watch.pollIntervalMs` to a number of milliseconds to poll instead. `auricLive.watch.enabled` set to `false` stops watching entirely and leaves updates to **Reload Connected Pages**.

Responses and caching

- **Compression** (`auricLive.compression`): `auto`, `gzip`, `brotli` or `off`, for text responses over one kilobyte. Turn it off to measure uncompressed sizes.
- **Caching** (`auricLive.cache`): `no-store` (default; a stale file never survives a reload), `no-cache` (keep but revalidate), `short` (a short freshness window, for measuring repeat visits), `immutable-assets` (fonts and images cached for a year, everything else uncached).
- **Validators** (`auricLive.etag`): unchanged files answer `304 Not Modified`.
- **Range requests** are supported, so audio and video can seek.
- **Content types**: add or override types with `auricLive.mimeTypes`, for example `{ ".data": "application/json" }`.
- **Extra headers**: `auricLive.headers` is a list of rules, each with `match` globs and the `headers` to add:

```
[
  { "match": ["**/*.html"], "headers": { "X-Frame-Options": "SAMEORIGIN" } }
]
```

- **Cross-origin requests** are answered by default (`auricLive.cors.enabled`, origin `*`). Set a specific `auricLive.cors.origin` and turn on `auricLive.cors.credentials` if the calling page sends cookies.
- **Cross-origin isolation** (`auricLive.crossOriginIsolation`) sends the headers that `SharedArrayBuffer`, WebAssembly threads and precise timers need. It also blocks cross-origin resources that do not opt in.

Changes while running

With `auricLive.autoRestart.onConfigChange` on (the default), changes to settings or the project file apply to the running server straight away. Only a change of port, host, served root, mount point or HTTPS settings restarts the listener; other changes apply without dropping connections. Connected pages reload afterwards unless you turn off `auricLive.autoRestart.reloadBrowsers`.

Stop

- **Auric Live: Stop Server** stops the active server.
- **Auric Live: Stop All Servers** (`Ctrl` + `Alt` + `Shift` + `L`) stops every one.
- **Auric Live: Restart Server** restarts the active one.

When you save a file with the server running, connected pages update within a moment, and the **Servers** view shows how many pages are connected.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Reference: commands, keys and settings

Every Auric Artisan Live command, keyboard shortcut, menu entry, view and setting, with types, defaults and ranges.

This page lists everything Auric Artisan Live 0.2.1 contributes to VS Code: 54 commands, 7 keyboard shortcuts, 3 sidebar views and 103 settings. Command titles appear in the Command Palette with the **Auric Live:** prefix.

Commands

Command	ID	Keys	What it does
Auric Live: Select Active Server	`auricLive.selectServer`	—	Choose which workspace folder's server the panel and commands act on
Auric Live: Toggle Browser Console Capture	`auricLive.toggleConsole`	—	Turn browser console capture on or off for this session
Auric Live: Use HTTPS Certificate and Key	`auricLive.configureCertificate`	—	Import and validate a PEM certificate and matching private key, then turn HTTPS on
Auric Live: Diagnose Project and Runtime	`auricLive.diagnose`	—	Open a local report on configuration, run targets and prerequisites, with credentials redacted

Command	ID	Keys	What it does
Auric Live: Go Live	`auricLive.goLive`	`Ctrl` + `Alt` + `L` (macOS `Cmd` + `Alt` + `L`)	Start serving the workspace folder, or reopen a server that is already running
Auric Live: Stop Server	`auricLive.stop`	—	Stop the active server
Auric Live: Stop All Servers	`auricLive.stopAll`	`Ctrl` + `Alt` + `Shift` + `L` (macOS `Cmd` + `Alt` + `Shift` + `L`)	Stop every Auric Live server
Auric Live: Restart Server	`auricLive.restart`	—	Restart the active server
Auric Live: Serve This Folder	`auricLive.startHere`	—	Serve the folder you right-clicked (Explorer only); not in the Command Palette
Auric Live: Open with Auric Live	`auricLive.openWithLive`	—	Start the server if needed and open this file's address
Auric Live: Open in Default Browser	`auricLive.openInBrowser`	—	Open the server's address in your default browser
Auric Live: Open Preview Beside	`auricLive.openPreview`	`Ctrl` + `Alt` + `K` (macOS `Cmd` + `Alt` + `K`)	Open the preview beside the editor
Auric Live: Copy Local URL	`auricLive.copyUrl`	—	Copy the localhost address
Auric Live: Copy Network URL	`auricLive.copyNetworkUrl`	—	Copy an address other devices on your network can use
Auric Live: Show Phone QR Code	`auricLive.showQrCode`	—	Show a locally generated QR code for opening the site on a phone
Auric Live: Open Control Panel	`auricLive.openControlPanel`	`Ctrl` + `Alt` + `Shift` + `I` (macOS `Cmd` + `Alt` + `Shift` + `I`)	Open the Auric Live control panel
Auric Live: Open Request Inspector	`auricLive.openInspector`	—	Open the control panel at its Requests section
Auric Live: Clear Request Log	`auricLive.clearRequests`	—	Empty the request inspector
Auric Live: Export Requests as HAR	`auricLive.exportHar`	—	Save captured requests as a HAR file
Auric Live: Copy Request as curl	`auricLive.copyAsCurl`	—	Copy a captured request as a curl command (Requests view); not in the Command Palette
Auric Live: Copy Request as fetch	`auricLive.copyAsFetch`	—	Copy a captured request as a fetch() call (Requests view); not in the Command Palette
Auric Live: Replay Request	`auricLive.replayRequest`	—	Send a captured request again (Requests view); not in the Command Palette
Auric Live: Show Request Detail	`auricLive.showRequestDetail`	—	Show a captured request's details (Requests view); not in the Command Palette
Auric Live: Create Project Config File	`auricLive.createConfigFile`	—	Write a starter auric.live.json and open it
Auric Live: Open Project Config File	`auricLive.openConfigFile`	—	Open auric.live.json, or offer to create it

Command	ID	Keys	What it does
Auric Live: Open .auricignore-live	`auricLive.openIgnoreFile`	—	Open .auricignore-live, creating a starter if needed
Auric Live: Open Recorded Live Session	`auricLive.openRuntimeRecords`	—	Open the latest session record, .auric-live/latest.json
Auric Live: Reload Configuration	`auricLive.reloadConfig`	—	Re-read settings and the project file
Auric Live: Change Port...	`auricLive.changePort`	—	Set the port for this session (0 chooses a free one)
Auric Live: Change Served Folder...	`auricLive.setRoot`	—	Pick the folder to serve for this session
Auric Live: Toggle Live Reload	`auricLive.toggleLiveReload`	—	Turn live reload on or off for this session
Auric Live: Toggle Mock API	`auricLive.toggleMocks`	—	Turn mock answering on or off for this session
Auric Live: Toggle HTTPS	`auricLive.toggleHttps`	—	Turn HTTPS on (with a generated certificate) or off for this session
Auric Live: Simulate Network Profile...	`auricLive.setThrottle`	—	Choose a network profile for this session
Auric Live: Reveal Development Certificate	`auricLive.revealCertificate`	—	Show the generated certificate file in your file manager
Auric Live: Open Settings	`auricLive.openSettings`	—	Open the Settings editor filtered to Auric Live
Auric Live: Show Log	`auricLive.showLog`	—	Show the Auric Artisan Live output channel
Auric Live: Reload Connected Pages	`auricLive.reloadBrowsers`	`Ctrl` + `Alt` + `U` (macOS `Cmd` + `Alt` + `U`)	Reload every connected page now
Auric Live: Resume Live Reload	`auricLive.resumeLiveReload`	—	Resume live reload after reload-storm protection paused it
Auric Live: Start Framework Dev Server...	`auricLive.startDevServer`	—	Detect the framework dev server, start it and forward to it; trusted workspaces only
Auric Live: Stop Framework Dev Server	`auricLive.stopDevServer`	—	Stop the dev server and remove the forwarding it added; trusted workspaces only
Auric Live: Restart Framework Dev Server	`auricLive.restartDevServer`	—	Restart the dev server or run target; trusted workspaces only
Auric Live: Show Dev Server Output	`auricLive.showDevServerLog`	—	Show the run target's output
Auric Live: Find a Running Backend	`auricLive.detectBackendPort`	—	Scan common ports for a running backend and offer to forward to it
Auric Live: Select Run Target...	`auricLive.selectRunTarget`	`Ctrl` + `Alt` + `J` (macOS `Cmd` + `Alt` + `J`)	Choose what serves the site: static files, a detected runtime or your own command
Auric Live: Add Custom Run Command...	`auricLive.addRunProfile`	—	Save a command (with any leading environment variables) as a run target
Auric Live: Edit Saved Run Commands	`auricLive.editRunProfiles`	—	Open the auricLive.runProfiles setting

Command	ID	Keys	What it does
Auric Live: Show Detected Run Targets	`auricLive.showRunTargets`	—	List every run target for the folder, with the reason for each, in the log
Auric Live: Add API Proxy Route...	`auricLive.addProxyRoute`	—	Forward a path prefix to a backend for this session
Auric Live: Add Mock API Route...	`auricLive.addMockRoute`	—	Answer a method and path with a JSON body for this session
Auric Live: Open Mock Database	`auricLive.openMockDatabase`	—	Open the REST database file, or create a sample
Auric Live: Refresh	`auricLive.refresh`	—	Refresh the Servers view (view menu only); not in the Command Palette
Auric Live: Open Live Menu	`auricLive.openMenu`	`Ctrl` + `Alt` + `Shift` + `K` (macOS `Cmd` + `Alt` + `Shift` + `K`)	Open the Auric Live quick menu
Auric Live: Focus Auric Live	`auricLive.focus`	—	Show the Auric Live sidebar

Commands marked "not in the Command Palette" run from the context menus of the **Servers** and **Requests** views, the Explorer or a view's title bar. Most other commands appear in the Command Palette only when a folder is open; **Open with Auric Live** appears when an editor is open.

Keyboard shortcuts

All seven shortcuts work whenever a folder is open. On macOS, `Cmd` replaces `Ctrl`.

Keys (Windows and Linux)	Keys (macOS)	Command
`Ctrl` + `Alt` + `L`	`Cmd` + `Alt` + `L`	**Go Live**
`Ctrl` + `Alt` + `Shift` + `L`	`Cmd` + `Alt` + `Shift` + `L`	**Stop All Servers**
`Ctrl` + `Alt` + `K`	`Cmd` + `Alt` + `K`	**Open Preview Beside**
`Ctrl` + `Alt` + `Shift` + `K`	`Cmd` + `Alt` + `Shift` + `K`	**Open Live Menu**
`Ctrl` + `Alt` + `J`	`Cmd` + `Alt` + `J`	**Select Run Target...**
`Ctrl` + `Alt` + `U`	`Cmd` + `Alt` + `U`	**Reload Connected Pages**
`Ctrl` + `Alt` + `Shift` + `I`	`Cmd` + `Alt` + `Shift` + `I`	**Open Control Panel**

Inside other surfaces:

Where	Keys	What they do
A served page	`Alt` + `Shift` + `D` (set by `auricLive.dashboard.shortcut`)	Open the in-page dashboard
Control panel, Requests section	`Ctrl` + `F` (`Cmd` + `F` on macOS)	Focus the request search
Control panel	`Escape`	Close request details or an open menu
Dashboard tabs	Arrow keys, `Home`, `End`	Move between tabs

Where	Keys	What they do
Dashboard content	Arrow keys, `Page Up`, `Page Down`, `Home`, `End`, `Space`, `Shift` + `Space`	Scroll

Where commands appear

- Status bar: **Go Live** when stopped; the port (opens the Live menu) when running
- **Servers** view title: **Go Live**, **Select Run Target...**, **Stop All Servers**, **Open Control Panel**; **...** menu: **Refresh**, **Open Settings**, **Show Log**
- A running server in **Servers**: Inline: **Open in Default Browser**, **Stop Server**. Right-click: **Restart Server**, **Open Preview Beside**, **Copy Local URL**, **Copy Network URL**, **Show Phone QR Code**
- A stopped server in **Servers**: Inline: **Go Live**
- **Requests** view title: **Clear Request Log**, **Export Requests as HAR**
- A request in **Requests**: Inline: **Show Request Detail**. Right-click: **Replay Request**, **Copy Request as curl**, **Copy Request as fetch**
- **Routes and Mocks** view title: **Add API Proxy Route...**, **Add Mock API Route...**, **Find a Running Backend**; **...** menu: **Open Mock Database**, **Open Project Config File**, **Restart Framework Dev Server**, **Show Dev Server Output**
- Explorer, on a folder: **Serve This Folder**, and the **Auric Live** submenu
- Explorer, on `.html`, `.htm`, `.php`, `.xhtml` or `.md`: **Open with Auric Live**
- Editor and editor tab, for `.html`, `.htm`, `.php` or `.xhtml`: **Open with Auric Live**
- Editor title **Run** menu › **Auric Live**: **Go Live**, **Select Run Target...**, **Add Custom Run Command...**, **Open Preview Beside**, **Open in Default Browser**, **Show Phone QR Code**, **Restart Framework Dev Server**, **Stop Server**, **Open Live Menu**

The Auric Live menu

Open Live Menu lists, when a server is running: **Open in browser**, **Open preview beside**, **Show phone QR code**, **Reload connected pages**, **Open control panel**, **Restart server** and **Stop server**. When stopped: **Go Live** and **Open control panel**. Then always: **Add an API proxy route**, **Add a mock API route**, **Open the mock database**, **Change the run target...**, **Add a custom command...**, the run target's **Restart the run target**, **Show its output** and **Stop it** (or **Start the framework dev server** when none is running), **Find a backend that is already running**, **Simulate a network profile**, **Select active server**, **Toggle browser console capture**, **Diagnose configuration and runtimes**, **Import an HTTPS certificate**, **Toggle HTTPS**, **Toggle live reload**, **Change the port**, **Change the served folder**, **Open the project config file**, **Open .auricignore-live**, **Open recorded session**, **Export requests as HAR**, **Open settings** and **Show log**. Each shows its current value where it has one.

Views

View	Where	Contents
Servers	Auric Live sidebar	One entry per workspace folder; expand for root, run target, dev server, upstreams, addresses, live reload, requests, certificate and config file
Requests	Auric Live sidebar	Recent requests with status, handler and time
Routes and Mocks	Auric Live sidebar	Proxy rules, mock routes, REST collections and CGI handlers
Control panel	Editor tab	Overview, Requests, Runtime, Routing, Settings

View	Where	Contents
Auric Live Preview	Editor tab beside the code	The served site at a chosen device size

Output channels: **Auric Artisan Live** (server log) and **Auric Live: Run Target** (run target output).

Settings

All 103 settings, grouped by area. A project file sets the same values with the names shown here minus the `auricLive.` prefix. "Values or range" lists the allowed values, or the minimum and maximum for numbers.

Server and address

Setting	Type	Default	Values or range	What it does
`auricLive.root`	string	empty		Folder to serve, relative to the workspace folder. Empty serves the workspace folder; use `dist` or `public` for a build output.
`auricLive.port`	number	`5500`	0–65535	Port to listen on. Set 0 to let the operating system pick a free one, which is useful when several projects run at once.
`auricLive.portStrategy`	string	`increment`	`increment`, `fail`, `random`	What to do when the chosen port is already in use.
`auricLive.host`	string	`0.0.0.0`		Address to bind. `0.0.0.0` also answers devices on your network (phone testing); `127.0.0.1` keeps it on this machine.
`auricLive.open`	string	`default`	`default`, `preview`, `none`	What to open once the server starts.
`auricLive.openPath`	string	empty		Path appended to the URL that is opened, for a site whose entry point is not the root — /docs/index.html, for example.
`auricLive.startOnOpen`	boolean	`false`		Start the server automatically when a folder is opened.
`auricLive.showStatusBar`	boolean	`true`		Show the Go Live control in the status bar.
`auricLive.suggestFrameworkSetup`	boolean	`true`		Once per folder, offer the matching setup when the project is a framework with its own dev server, or a PHP site.
`auricLive.configFile`	string	`auric.live.json`		Name of the project file, looked for in the workspace folder, then `.auric/`. It overrides these settings.
`auricLive.interfaceLanguage`	string	`auto`	`auto`, `en`, `hi`	Language of the Auric Live panel and supported messages. `auto` follows VS Code. Changing it does not restart servers.

HTTPS

Setting	Type	Default	Values or range	What it does
`auricLive.https.enabled`	boolean	`false`		Serve over HTTPS, which service workers, clipboard, camera, microphone and geolocation need on a phone.
`auricLive.https.certPath`	string	empty		Certificate file. Empty generates and reuses a self-signed one for `localhost` and your network addresses.
`auricLive.https.keyPath`	string	empty		Private key matching `auricLive.https.certPath`.

Static files and responses

Setting	Type	Default	Values or range	What it does
`auricLive.index`	array	`["index.html","index.htm"]`		File names tried, in order, when a directory is requested.
`auricLive.directoryListing`	boolean	`true`		Render a browsable index for a directory that has no index file.
`auricLive.cleanUrls`	boolean	`false`		Serve /about from about.html, so links can drop the extension the way most static hosts do.
`auricLive.dotfiles`	string	`ignore`	`ignore`, `allow`, `deny`	How paths with a dot-file or dot-folder are treated. The default hides them, keeping `.env` and `.git` unserved.
`auricLive.notFoundPage`	string	empty		Page served for a missing file, relative to the served root. Leave empty for the built-in 404.
`auricLive.spa.enabled`	boolean	`false`		Answer unknown paths with the fallback document, so a client-side router keeps working on a deep link and a hard refresh.
`auricLive.spa.fallback`	string	`/index.html`		Document served for unknown paths when the single-page fallback is on.
`auricLive.spa.ignore`	array	`["/api/**","**/*.*"]`		Paths the fallback must not answer. The default keeps `/api` and anything with a file extension returning honest 404s.
`auricLive.compression`	string	`auto`	`auto`, `gzip`, `brotli`, `off`	Compression for text responses over one kilobyte. Turn it off when measuring uncompressed transfer sizes.
`auricLive.cache`	string	`no-store`	`no-store`, `no-cache`, `short`, `immutable-assets`	Caching policy sent with served files.
`auricLive.etag`	boolean	`true`		Send a validator so an unchanged file can answer 304 instead of being sent again.
`auricLive.mimeTypes`	object	`{}`		Extra content types, keyed by extension — `{ ".data": "application/json" }`. Overrides the built-in table.
`auricLive.headers`	array	`[]`		Extra response headers, applied to the paths each rule matches.

Cross-origin and access

Setting	Type	Default	Values or range	What it does
`auricLive.cors.enabled`	boolean	`true`		Answer cross-origin requests, so a page served from another port can call this one during development.
`auricLive.cors.origin`	string	`*`		Origin allowed by the CORS headers. Use a specific origin when credentials are involved.
`auricLive.cors.credentials`	boolean	`false`		Allow cookies and authorization headers on cross-origin requests.
`auricLive.crossOriginIsolation`	boolean	`false`		Send COOP and COEP headers, needed for `SharedArrayBuffer`, WebAssembly threads and precise timers.
`auricLive.auth.enabled`	boolean	`false`		Require a username and password before anything is served. Useful when bound to a shared network.
`auricLive.auth.username`	string	empty		Username for basic authentication.
`auricLive.auth.password`	string	empty		Password for basic authentication. Stored in plain settings; a development convenience, not a secret store.

Request limits

Setting	Type	Default	Values or range	What it does
`auricLive.maxRequestBodyBytes`	number	`8388608`	0–1073741824	Maximum buffered request size in bytes. Larger bodies receive HTTP 413. Zero rejects all non-empty bodies.
`auricLive.requestBodyTimeoutMs`	number	`30000`	0–3600000	Deadline for receiving a request body; 0 disables it. Simulated transfer time is excluded.

Network simulation

Setting	Type	Default	Values or range	What it does
`auricLive.throttle.profile`	string	`off`	`off`, `slow-3g`, `3g`, `4g`, `cable`, `custom`	Simulate a slower network for every client, including a phone on your LAN where browser devtools are not available.
`auricLive.throttle.latencyMs`	number	`0`	0–60000	Delay added before each response when the custom profile is selected.
`auricLive.throttle.downloadKbps`	number	`0`	0–no maximum	Download ceiling in kilobits per second when the custom profile is selected. 0 means unlimited.
`auricLive.throttle.uploadKbps`	number	`0`	0–1000000	Upload bandwidth in kilobits per second; 0 uses unlimited bandwidth.

Live reload and watching

Setting	Type	Default	Values or range	What it does
`auricLive.liveReload.enabled`	boolean	`true`		Inject the reload client into served HTML and update open pages when the configured save/change trigger fires.

Setting	Type	Default	Values or range	What it does
`auricLive.liveReload.reloadOnSave`	string	`save`	`save`, `change`, `off`	What may refresh the page: `save` (explicit VS Code saves), `change` (any file change) or `off` (manual only).
`auricLive.liveReload.injectTo`	string	`body`	`body`, `head`	Where the reload client is inserted: end of `body` (out of the critical path) or `head` (earlier).
`auricLive.liveReload.cssHotSwap`	boolean	`true`		Swap a changed stylesheet in place, keeping scroll position, open dialogs and filled forms.
`auricLive.liveReload.imageHotSwap`	boolean	`true`		Refresh a changed image in place instead of reloading the page.
`auricLive.liveReload.preserveScroll`	boolean	`true`		Restore scroll position and open disclosure elements after a full reload.
`auricLive.liveReload.overlay`	boolean	`true`		Show an overlay in the page when a proxied request or a CGI handler fails, instead of leaving the failure in the log.
`auricLive.liveReload.ignore`	array	see "Default reload exclusions"		Paths that never trigger a reload. Dot-paths such as `.next/`, `.vite/` and `.git/` are always excluded as well.
`auricLive.liveReload.stormLimit`	number	`3`	2–200	Reloads allowed within the storm window before Auric Live stops reloading and reports the files responsible.
`auricLive.liveReload.stormWindowMs`	number	`10000`	1000–120000	The window the reload limit is measured over.
`auricLive.liveReload.debounceMs`	number	`350`	0–5000	Quiet period after the last file change before pages are told, so a multi-file build causes one update.
`auricLive.watch.enabled`	boolean	`true`		Watch the served folder for changes. Turning this off leaves live reload to the Reload Connected Pages command.
`auricLive.watch.pollIntervalMs`	number	`0`	0–60000	Poll for changes at this interval instead of file system events (for network shares and VMs). `0` uses events.

Browser dashboard and console

Setting	Type	Default	Values or range	What it does
`auricLive.dashboard.enabled`	boolean	`true`		Inject the in-page developer dashboard with live health, performance, cache, network, storage and server diagnostics.
`auricLive.dashboard.independent`	boolean	`false`		Inject the dashboard even when Auric live reload is off. Framework run targets turn this on.

Setting	Type	Default	Values or range	What it does
`auricLive.dashboard.shortcut`	string	`Alt+Shift+D`		Keys inside a served page that open the dashboard, joined with `+`, for example `Alt+Shift+D`.
`auricLive.dashboard.customCursor`	boolean	`true`		Show the Auric pointer on served pages. Touch, coarse-pointer and forced-colour setups keep the native cursor.
`auricLive.dashboard.sampleIntervalMs`	number	`1000`	500–10000	How often an open dashboard asks for server state, in milliseconds.
`auricLive.console.enabled`	boolean	`false`		Capture browser console messages into the dashboard and the extension log, redacted and rate-limited.
`auricLive.console.levels`	array	`["error","warn","info","log","debug"]`		Console levels to capture.
`auricLive.console.maxEntries`	number	`200`	20–2000	Maximum console entries retained in each page.
`auricLive.console.maxMessageLength`	number	`2000`	100–10000	Maximum characters captured from one console call.
`auricLive.console.rateLimit`	number	`20`	1–100	Maximum console messages sent per second from each page.

Session records

Setting	Type	Default	Values or range	What it does
`auricLive.records.enabled`	boolean	`true`		Keep a bounded local session record under `.auric-live/`. It never stores queries, headers, bodies or cookies.
`auricLive.records.maxRequests`	number	`200`	20–5000	Maximum sanitized request summaries retained in one recorded session.
`auricLive.records.maxSessionBytes`	number	`5242880`	65536–104857600	Maximum approximate size in bytes of one `.auric-live` session snapshot. Older request summaries are removed first.
`auricLive.records.maxSessions`	number	`12`	1–100	Number of recent recorded sessions retained under `.auric-live/sessions/`.
`auricLive.records.flushIntervalMs`	number	`1000`	250–60000	Minimum interval between atomic diagnostic snapshot writes.
`auricLive.records.telemetryIntervalMs`	number	`5000`	1000–60000	How often a served page sends its bounded performance summary to the local session record.

Proxy and backend readiness

Setting	Type	Default	Values or range	What it does
`auricLive.proxy`	array	`[]`		Paths forwarded to another server, so the page and its API share one origin. Each rule has a `path` and a `target`.
`auricLive.upstream.wait`	boolean	`true`		Hold a request while a proxied backend is still starting, instead of answering 502 at once.
`auricLive.upstream.waitTimeoutMs`	number	`25000`	0–600000	How long a request waits for the backend before the waiting page is shown; that page reloads until it answers.
`auricLive.upstream.pollIntervalMs`	number	`400`	50–10000	How often a backend that is not answering is checked. Once it is up, checks drop to one every few seconds.
`auricLive.upstream.reloadOnRecover`	boolean	`true`		Reload every connected page when a backend comes back.
`auricLive.upstream.autoDetectPort`	boolean	`true`		Adopt the port a supervised dev server prints when it differs from the configured one.
`auricLive.upstream.candidatePorts`	array	`[]`		Ports searched by Find a Running Backend. Leave empty to use the built-in list of ports these tools normally choose.

Run targets and dev servers

Setting	Type	Default	Values or range	What it does
`auricLive.runProfiles`	array	`[]`		Named ways to run the project, shown in the run-target list. Environment assignments before a command are read.
`auricLive.devServer.autoStart`	boolean	`false`		Start the detected framework dev server automatically when Go Live runs on a project that has one, instead of offering it.
`auricLive.devServer.autoRestart`	boolean	`true`		Restart a supervised dev server when it exits unexpectedly, with a growing delay between attempts.
`auricLive.devServer.maxRestarts`	number	`5`	0–50	How many times a dev server may be restarted before Auric Live stops and reports it.
`auricLive.devServer.command`	string	empty		Command used to start the dev server. Leave empty to use the one detected from the project, such as npm run dev.
`auricLive.devServer.useTerminal`	boolean	`false`		Run the dev server in a VS Code terminal. Allows typed input, but loses port adoption and restart supervision.
`auricLive.autoRestart.onConfigChange`	boolean	`true`		Apply setting and project-file changes to a running server. Only port, host, root, mount or TLS changes rebind it.
`auricLive.autoRestart.reloadBrowsers`	boolean	`true`		Reload connected pages after a configuration change is applied.

Mock API and REST database

Setting	Type	Default	Values or range	What it does
`auricLive.mock.enabled`	boolean	`false`		Answer the declared mock routes. They are matched before the proxy, so one endpoint can be overridden.

Setting	Type	Default	Values or range	What it does
`auricLive.mock.routes`	array	`[]`		Endpoints answered by the server itself. Bodies can use placeholders such as `{{params.id}}` and `{{uuid}}`.
`auricLive.mock.delayMs`	number	`0`	0–60000	Latency applied to every mock and REST response, on top of any per-route delay.
`auricLive.mock.rest.enabled`	boolean	`false`		Serve a full REST backend from a JSON file, with filtering, sorting, pagination and relationships.
`auricLive.mock.rest.base`	string	`/api`		Prefix the REST collections are mounted under. Other paths fall through to the proxy, then to static files.
`auricLive.mock.rest.database`	string	`db.json`		JSON file holding the collections, relative to the served root, shaped like `{ "users": [ … ] }`.
`auricLive.mock.rest.persist`	boolean	`false`		Write changes back to the database file. Off by default so a restart returns to a known fixture.

PHP and CGI

Setting	Type	Default	Values or range	What it does
`auricLive.cgi.enabled`	boolean	`false`		Run matching files through a CGI handler instead of serving them as downloads. With `php-cgi`, PHP sites run.
`auricLive.cgi.handlers`	object	`{".php":"php-cgi"}`		Handler command per extension, such as `{ ".php": "php-cgi" }`. It must be on your PATH or a full path.
`auricLive.cgi.indexes`	array	`["index.php"]`		Additional index file names tried for a directory when CGI is enabled.

Request inspector

Setting	Type	Default	Values or range	What it does
`auricLive.inspector.enabled`	boolean	`true`		Record requests so the Requests view shows which rule answered each one.
`auricLive.inspector.maxEntries`	number	`500`	20–10000	How many requests are kept before the oldest are dropped.
`auricLive.inspector.captureBodies`	boolean	`true`		Keep request and response bodies for inspection and HAR export. Credential headers are always masked.
`auricLive.inspector.maxBodyBytes`	number	`262144`	0–8388608	Maximum bytes captured per request or response body.
`auricLive.inspector.redactSensitive`	boolean	`true`		Redact common credential fields in captured query strings and JSON/form bodies. Credential headers are always redacted.
`auricLive.inspector.sensitiveFields`	array	`[]`		Additional field and header names to redact, case insensitive.
`auricLive.inspector.harIncludeBodies`	boolean	`false`		Include captured bodies in HAR exports. Disabled by default because arbitrary application data can be sensitive.

Logging

Setting	Type	Default	Values or range	What it does
`auricLive.logLevel`	string	`info`	`off`, `error`, `warn`, `info`, `debug`	How much detail reaches the Auric Artisan Live output channel. Credential-like values are masked.
`auricLive.requestLogging`	boolean	`true`		Write one line per request to the output channel, at debug level for successes and higher for failures.

Default reload exclusions

The default value of `auricLive.liveReload.ignore`:

```
[
  "**/node_modules/**", "**/dist/**", "**/out/**", "**/build/**", "**/coverage/**",
  "**/target/**", "**/vendor/**", "**/bin/**", "**/obj/**", "**/__pycache__/**",
  "**/venv/**", "**/tmp/**", "**/temp/**", "**/*.log", "**/*.tmp", "**/*.temp",
  "**/*.swp", "**/*.swx", "**/*~", "**/*.pyc", "**/*.class", "**/*.o", "**/*.obj",
  "**/*.pdb", "**/*.crdownload", "**/*.partial", "**/*-journal", "**/*.sqlite*",
  "**/~$*", "**/.~lock.*", "**/Thumbs.db", "**/desktop.ini", "**/.DS_Store"
]
```

Enumerated values explained

Setting	Value	Meaning
`auricLive.portStrategy`	`increment`	Walk upward from the chosen port until one is free
`auricLive.portStrategy`	`fail`	Report the conflict instead of moving
`auricLive.portStrategy`	`random`	Choose a free port between 20000 and 40000
`auricLive.open`	`default`	Open the default browser when the server starts
`auricLive.open`	`preview`	Open the preview beside the editor
`auricLive.open`	`none`	Open nothing
`auricLive.dotfiles`	`ignore`, `allow`, `deny`	Hide, serve or refuse paths with a dot-file or dot-folder
`auricLive.cache`	`no-store`	Never cache, so a stale file never survives a reload
`auricLive.cache`	`no-cache`	Keep a copy but check it on every request
`auricLive.cache`	`short`	A short freshness window, for measuring repeat visits
`auricLive.cache`	`immutable-assets`	Fonts and images immutable for a year; everything else uncached
`auricLive.throttle.profile`	`off`, `slow-3g`, `3g`, `4g`, `cable`, `custom`	See Simulate a slow network
`auricLive.liveReload.reloadOnSave`	`save`	Reload once per explicit VS Code save
`auricLive.liveReload.reloadOnSave`	`change`	Watch the whole served folder
`auricLive.liveReload.reloadOnSave`	`off`	Never reload automatically
`auricLive.liveReload.injectTo`	`body`, `head`	Where the reload script is inserted
`auricLive.logLevel`	`off`, `error`, `warn`, `info`, `debug`	Detail written to the output channel

Setting	Value	Meaning
`auricLive.interfaceLanguage`	`auto`, `en`, `hi`	Follow VS Code, English, or Hindi

Settings restricted in untrusted workspaces

`auricLive.cgi.enabled`, `auricLive.cgi.handlers`, `auricLive.cgi.indexes`, `auricLive.proxy`, `auricLive.host`, `auricLive.auth.username`, `auricLive.auth.password`, `auricLive.https.certPath`, `auricLive.https.keyPath` and `auricLive.mock.rest.persist` are ignored until you trust the workspace, whether they come from settings or the project file.

Setting scope

Most settings have **resource** scope, so each folder of a multi-root workspace can differ. `auricLive.showStatusBar`, `auricLive.etag`, `auricLive.inspector.enabled`, `auricLive.inspector.maxEntries`, `auricLive.inspector.captureBodies`, `auricLive.logLevel`, `auricLive.requestLogging` and `auricLive.interfaceLanguage` have **window** scope: they can be set for the user or the workspace, but not per folder.

Files Auric Live reads and writes

File	Written by	Purpose
`auric.live.json` (or the name in `auricLive.configFile`), in the workspace folder or `.auric/`	You, **Create Project Config File** or **Save to config file**	Project configuration
`.auricignore-live`, in the served folder	You, **Open .auricignore-live** or **Ignore *folder/***	Paths never served, watched or recorded
`.auric-live/latest.json` and `.auric-live/sessions/`	Auric Live	Local session records
`db.json` (or `auricLive.mock.rest.database`)	You, or **Open Mock Database** › **Create**	REST database; written back only with `mock.rest.persist`
A `.har` file	**Export Requests as HAR**	Request export
Generated certificate	Auric Live, in its own extension storage	Self-signed HTTPS certificate
Workspace settings	**Add Custom Run Command...**	Saves `auricLive.runProfiles`

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Troubleshooting

Symptoms, causes and fixes for common Auric Artisan Live problems, from pages that do not update to phones that cannot connect.

For anything not listed, run **Auric Live: Diagnose Project and Runtime** and **Auric Live: Show Log**, then see Get support.

Starting

"Auric Live needs an open folder to serve."

- **Why:** no folder is open. Auric Live serves folders, not single files.
- **Fix:** open the folder with **File** › **Open Folder...**, then go live.

"Auric Live could not start"

- **Why:** usually the port is taken and `auricLive.portStrategy` is `fail`, or the address cannot be bound.
- **Fix:** choose **Change Port** in the message, set the port to `0` for a free one, or set the strategy back to `increment`. **Show Log** has the exact reason.

The status bar has no Go Live item

- **Why:** `auricLive.showStatusBar` is off, or no folder is open.
- **Fix:** turn the setting on, or use **Auric Live: Go Live** from the Command Palette.

Commands for dev servers are missing from the Command Palette

- **Why: Start**, **Stop** and **Restart Framework Dev Server** only appear in trusted workspaces.
- **Fix:** run **Workspaces: Manage Workspace Trust** and trust the folder.

Live reload

Saving does not update the page

- **Why:** live reload is off, the trigger does not match how you work, the file is excluded, or the page was opened before the server started.
- **Fix:**
  1. Check the **Servers** view: **Live reload** should show a number of connected pages, not **off**.
  2. If files are written by a build tool rather than saved in VS Code, set `auricLive.liveReload.reloadOnSave` to `change`.
  3. Check `auricLive.liveReload.ignore` and `.auricignore-live` for a pattern matching the file.
  4. Reload the page once by hand, or run **Reload Connected Pages** (`Ctrl` + `Alt` + `U`).
  5. On a network share or in a virtual machine, set `auricLive.watch.pollIntervalMs`.

The page reloads over and over

- **Why:** something keeps writing files in the served folder.
- **Fix:** keep the default `save` trigger. When **Auric Live paused live reload — files kept changing** appears, choose **Ignore *folder/*** to exclude the folder responsible.

Live reload stopped and did not come back

- **Why:** reload-storm protection paused it.
- **Fix:** run **Auric Live: Resume Live Reload**, or choose **Resume** in the message.

A framework project shows a blank page and module errors

- **Why:** the project needs its own dev server (for example Vite or Next.js) but is being served as plain files.
- **Fix:** choose the framework's dev server in **Select Run Target...**, or run **Start Framework Dev Server...**. See Run a backend or framework dev server.

A session change reverted

- **Why:** changes made in the panel or by commands last only until the window closes or reloads.
- **Fix:** put the value in `auric.live.json` or VS Code settings.

Backends and proxies

502 or the waiting page for a proxied path

- **Why:** the backend is not running, is still starting, or is on a different port.
- **Fix:** check the **Upstream** row in the **Servers** view; select **Find it** to scan for the backend; run **Show Dev Server Output**. If the backend has a health endpoint, set `healthPath`. Readiness checks do not follow redirects, so point `healthPath` at a path that answers directly.

"keeps failing" after starting a run target

- **Why:** the process exits straight away, often from a missing dependency or a syntax error. Auric Live stops restarting it after `auricLive.devServer.maxRestarts` attempts.
- **Fix:** choose **Show output**, fix the error, then **Try again**.

"*command* was not found on PATH or in the project"

- **Why:** the runtime is not installed or not on your `PATH`. Auric Live never installs runtimes.
- **Fix:** install it, or use the full path to the executable in the run profile.

Nothing was found by Find a Running Backend

- **Why:** the backend uses a port that is not in the list Auric Live scans.
- **Fix:** set `auricLive.upstream.candidatePorts` to the ports you use, or add a URL-only run profile.

"Trust this workspace before running CGI, commands, or proxy profiles."

- **Why:** the folder is in Restricted Mode.
- **Fix:** trust the folder if you trust its contents.

PHP

`.php` pages download instead of running

- **Why:** CGI is off, or `php-cgi` is not found.
- **Fix:** choose **Enable PHP** when offered, or choose **PHP through CGI** as the run target. Make sure `php-cgi` is on your `PATH`, or set its full path in `auricLive.cgi.handlers`.

A PHP page times out

- **Why:** the script ran longer than the CGI time limit (30 seconds by default).
- **Fix:** set `cgi.timeoutMs` in `auric.live.json`.

Phones and other devices

The phone cannot connect

- **Why:** the server is bound to this machine only, the devices are on different networks, the firewall blocks VS Code, or the address belongs to a virtual adapter.

- **Fix:** keep `auricLive.host` at `0.0.0.0` (or choose **Bind to the network**), put both devices on the same network, allow VS Code through the firewall, and pick a physical adapter's address.

The phone warns about the certificate

- **Why:** HTTPS is encrypted but the phone does not trust the certificate.
- **Fix:** install and trust the certificate on the phone, or use one it already trusts. See Serve over HTTPS.

"that URL is too long to encode"

- **Why:** the address is too long for a QR code.
- **Fix:** use **Copy Network URL** and send the address to the phone another way.

Requests

Request details are empty or expired

- **Why:** the request has dropped out of the inspector (500 kept by default), body capture is off, or the body was larger than the capture limit.
- **Fix:** make the request again. Raise `auricLive.inspector.maxEntries` or `auricLive.inspector.maxBodyBytes` if needed.

Replay is refused

- **Why:** the capture was redacted, truncated, omitted or binary, so replaying it would send something different.
- **Fix:** use **Copy curl** or **Copy fetch** and fill in the missing values.

Uploads fail with 413 or 408

- **Why:** the request body is larger than `auricLive.maxRequestBodyBytes` (8 MiB by default; `0` rejects every non-empty body), or it arrived slower than `auricLive.requestBodyTimeoutMs` (30 seconds).
- **Fix:** raise the limit or the timeout for your test.

Preview and dashboard

The preview toolbar is in the wrong language

- **Why:** an open preview keeps the language it opened with.
- **Fix:** close and reopen the preview.

The dashboard does not open

- **Why:** the dashboard is off, the page was not served by Auric Live, or another script uses the same keys.
- **Fix:** check `auricLive.dashboard.enabled`, open the page through the Auric Live address, or change `auricLive.dashboard.shortcut`.

The console tab is empty

- **Why:** console capture is off by default, or the page was open before you turned it on.
- **Fix:** run **Toggle Browser Console Capture**, then reload the page.

Some dashboard figures say Unavailable

- **Why:** the browser does not provide that measurement.
- **Fix:** none needed; the dashboard shows only what the browser reports.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

FAQ

Short answers to common questions about Auric Artisan Live.

Is Auric Live a production web server?

No. It is a development server for your own machine. Do not use it to host a public site.

Does it need Node.js or other software?

Serving static files needs nothing extra. Running a backend needs that backend's runtime (PHP, Python, Node.js and so on) and the project's dependencies, installed by you. The extension itself has no runtime npm dependencies.

What is the default address?

`http://localhost:5500`, or the next free port. Change it with `auricLive.port`.

Can other people on my network see my site?

In a trusted workspace the default host `0.0.0.0` lets devices on your local network reach the server, subject to your firewall. Set `auricLive.host` to `127.0.0.1` to keep it on your machine, or turn on `auricLive.auth.enabled` to require a password.

Why does the page not reload for files my build tool writes?

The default trigger reacts only to saves in VS Code, so generated and synchronised files cannot keep refreshing the page. Set `auricLive.liveReload.reloadOnSave` to `change` for build tools.

Can I use it with Vite, Next.js, Angular or another framework?

Yes. Choose the framework's dev server as the run target. Auric Live starts it, forwards to it and keeps the framework's own hot reload in charge, while still adding its dashboard.

Can I run PHP?

Yes, through `php-cgi` or the PHP built-in server, with PHP installed. See Run PHP sites.

Can I serve several projects at once?

Yes. Each folder of a multi-root workspace gets its own server and port. Use **Select Active Server** to choose which one the panel shows.

Does it work in an untrusted folder?

Partly. Static serving, live reload and request inspection work on this machine only. Running commands, PHP, proxying, file-backed mocks, custom certificates and network access need trust.

Does it work over Remote SSH, in containers or in virtual workspaces?

It needs a real file system and a port. Virtual workspaces are not supported. In remote setups it runs next to the files; forward the port to reach it from your computer.

Does HTTPS remove the browser warning?

Only if the browser trusts the certificate. The generated certificate is self-signed, so each browser or device warns until you trust it. A certificate your devices already trust removes the warning.

Is my request data sent anywhere?

No. Requests are recorded locally, credentials are masked, and HAR exports leave out bodies by default. There is no telemetry.

Can I use it in Hindi?

Yes. Choose **हिन्दी** in the control panel header, or set `auricLive.interfaceLanguage` to `hi`. The browser dashboard and your website are not translated.

How is it different from other live servers?

Besides live reload, it runs and waits for backends, proxies and mocks APIs, serves a REST API from a JSON file, inspects and replays requests, simulates slow networks for every client, and adds an in-page dashboard.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Supporting material

Contents of the supporting pages for Auric Artisan Live.

These pages support the Auric Artisan Live guides: terms, shortcuts, built-in limits and safeguards, privacy, related products and the version history.

File	What it covers
glossary.md	Terms used in the Auric Live documentation
shortcuts.md	Every keyboard shortcut
limits-and-accuracy.md	Built-in limits, safeguards and what the measurements mean
privacy.md	What stays on your machine and what uses the network
related.md	Related extensions, tools and pages
release-notes.md	What changed in each version

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Shortcuts

Every keyboard shortcut Auric Artisan Live provides in VS Code, the control panel, the preview and the in-page dashboard.

VS Code

These work whenever a folder is open. Change them in **Preferences: Open Keyboard Shortcuts** by searching for `auricLive`.

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `L`	`Cmd` + `Alt` + `L`	**Auric Live: Go Live**
`Ctrl` + `Alt` + `Shift` + `L`	`Cmd` + `Alt` + `Shift` + `L`	**Auric Live: Stop All Servers**
`Ctrl` + `Alt` + `K`	`Cmd` + `Alt` + `K`	**Auric Live: Open Preview Beside**
`Ctrl` + `Alt` + `Shift` + `K`	`Cmd` + `Alt` + `Shift` + `K`	**Auric Live: Open Live Menu**
`Ctrl` + `Alt` + `J`	`Cmd` + `Alt` + `J`	**Auric Live: Select Run Target...**
`Ctrl` + `Alt` + `U`	`Cmd` + `Alt` + `U`	**Auric Live: Reload Connected Pages**
`Ctrl` + `Alt` + `Shift` + `I`	`Cmd` + `Alt` + `Shift` + `I`	**Auric Live: Open Control Panel**

The other 47 commands have no default shortcut. Assign one in the Keyboard Shortcuts editor if you use them often.

Control panel

Keys	What they do
`Ctrl` + `F` (`Cmd` + `F`)	Focus the search box in the Requests section
`Escape`	Close request details and return focus, or close an open menu
Arrow keys, type-ahead, `Enter`	Move through and choose from drop-down menus

Preview

Keys	What they do
`Enter` in the address box	Load the typed address

In a served page

Keys	What they do
`Alt` + `Shift` + `D`	Open the developer dashboard (change with `auricLive.dashboard.shortcut`)
`Escape`	Close an open dashboard menu, then the dashboard
Left and right arrows, `Home`, `End` (tab focused)	Move between dashboard tabs
Up and down arrows, `Page Up`, `Page Down`, `Home`, `End`	Scroll the dashboard
`Space`, `Shift` + `Space` (content focused)	Page down and up

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Glossary

Plain-language definitions of the terms used in the Auric Artisan Live documentation.

Backend — the part of a site that runs on a server: an API, a PHP application, a framework dev server.

CGI (Common Gateway Interface) — a standard way for a web server to run a program for each request and send back what it prints. Auric Live uses it to run PHP through `php-cgi`.

CORS (cross-origin resource sharing) — the browser rules that decide whether a page from one address may call another. A proxy avoids CORS problems by putting the page and its API on one address.

Cross-origin isolation — a pair of response headers that unlock features such as `SharedArrayBuffer`, at the cost of blocking cross-origin resources that do not opt in.

Dev server — a framework's own development server, such as the one `npm run dev` starts for Vite or Next.js.

Developer dashboard — the panel Auric Live places over a served page, opened with `Alt` + `Shift` + `D`.

Handler — the part of Auric Live that answered a request: `static`, `proxy`, `mock`, `rest`, `cgi`, `listing`, `redirect`, `error`, `client` or `live-reload`.

HAR (HTTP Archive) — a standard JSON file format for a list of requests and responses.

Hot swap — replacing a changed stylesheet or image in a page without reloading it.

Live reload — updating open pages automatically when files change.

Loopback — addresses that only reach the same computer, such as `127.0.0.1` and `localhost`.

Mock route — an endpoint Auric Live answers itself with a response you declare.

Network profile — a preset latency and bandwidth used to simulate a slower connection.

p95 latency — the time within which 95 per cent of the sampled requests were answered.

Project file — `auric.live.json`, the configuration kept with your repository.

Proxy — forwarding requests for a path to another server and passing its answers back.

Reload storm — many reloads in a short time, usually caused by a tool writing files repeatedly. Auric Live pauses live reload when it detects one.

REST database — a JSON file served as a full REST API with list, read, create, replace, update and delete.

Run profile / run target — a named way to run the project: static files, a detected runtime or dev server, your own command, or an existing server's URL.

Secure context — a page served over HTTPS (or from `localhost`), which browsers require for features such as service workers and the camera.

Self-signed certificate — an HTTPS certificate not issued by an authority a device already trusts, so the device warns until you trust it.

Session override — a change made with a command or the control panel that lasts until the VS Code window closes or reloads.

SPA fallback — answering unknown paths with the app's main page so a client-side router can handle them.

Upstream — the backend a proxy rule forwards to.

URL-only profile — a run profile that forwards to a server you already run, without starting or stopping it.

Workspace trust — your decision about whether VS Code and extensions may run code from a folder. Untrusted folders run in Restricted Mode.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Limits and safeguards

The built-in limits of Auric Artisan Live, why they exist, which ones you can change, and how to read its measurements.

Auric Live keeps memory bounded and protects your data by design. Most limits can be changed; the safeguards around privacy and workspace trust are deliberate.

Built-in limits

Limit	Default	Change it with
Requests kept by the inspector	500	`auricLive.inspector.maxEntries` (20–10000)
Requests shown in the control panel	Newest 200	—
Body captured per request or response	256 KiB	`auricLive.inspector.maxBodyBytes` (0–8388608)
Incoming request body	8 MiB; larger bodies get 413; `0` rejects all non-empty bodies	`auricLive.maxRequestBodyBytes`
Time to receive a request body	30 seconds; stalled bodies get 408; `0` disables it	`auricLive.requestBodyTimeoutMs`

Limit	Default	Change it with
Console capture	Off; when on, 200 entries, 2,000 characters per message, 20 messages per second	`auricLive.console.*`
Session records	On; 200 request summaries, 12 sessions, about 5 MiB per session	`auricLive.records.*`
Reloads before storm protection	3 in 10 seconds	`auricLive.liveReload.stormLimit`, `auricLive.liveReload.stormWindowMs`
Dev server restarts	5	`auricLive.devServer.maxRestarts` (0–50)
Wait for a starting backend	25 seconds	`auricLive.upstream.waitTimeoutMs`
Proxy request timeout	30 seconds per rule	`timeoutMs` in a proxy rule
CGI script run time	30 seconds	`cgi.timeoutMs` in the project file

Deliberate boundaries

- **Development only.** Auric Live is not public or production hosting.
- **Virtual workspaces are not supported.** It needs files on disk and a local port.
- **Runtimes are not installed for you.** Executables and framework packages must already be installed. Diagnostics report what is missing without installing or running anything.
- **Detection suggests, it does not certify.** A detected run target is based on files in the folder. It does not guarantee every framework configuration will work.
- **Workspace trust.** Untrusted project files cannot turn on running commands, proxying, CGI, file-backed mocks, custom certificates or listening on your network.
- **Symbolic links.** Links or junctions that lead outside the served folder are not followed.
- **Certificates.** Generating or importing a certificate never installs trust on a device.
- **Platform coverage.** Automated tests do not certify every operating system, framework or VS Code installation.

Network simulation accuracy

- Latency and bandwidth are applied per transfer, not as a shared budget across all connections.
- There is no packet loss and no packet-level emulation.
- WebSocket messages are not bandwidth-shaped.
- Treat results as an approximation of a slow connection, not a measurement of a real network.

Reading the measurements

- **p95 latency** in the control panel is calculated from the recent requests shown (up to 200), not from the whole session. Inspector totals count everything captured since the inspector was cleared.
- **Dashboard metrics** come from browser APIs. A browser that does not provide one shows **Unavailable**, not an estimate.
- **Cache results** in the dashboard are labelled either as the browser's estimate (**Likely cached**) or as confirmed by the server (304 or cached responses).
- **Storage figures** are sizes and counts reported by the browser; stored values are never read.
- **Frame rate and timings** are sampled while the dashboard is open and the tab is visible, so they describe that moment, not the page's lifetime.

Redaction

Credential headers are always masked. Common sensitive query parameters and JSON or form fields are redacted by default, and you can add your own field names. Redaction is not a general personal-data detector: names, addresses and free text in bodies may still be sensitive. Review exports before sharing them.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Privacy

What Auric Artisan Live keeps on your machine, what can reach the network, and how captured data is protected.

What stays on your machine

- **No telemetry.** Auric Live sends no usage data anywhere.
- **The server runs locally.** Your files are served from your computer.
- **Requests are recorded locally** in the inspector, in memory, and optionally as local session records under `.auric-live/`.
- **The phone QR code is generated locally.** No address goes to a QR service.
- **The translation into Hindi is bundled.** No translation service receives your data.
- **The dashboard makes no external requests** for fonts or branding.

What can reach the network

- **Other devices on your network.** With the default host `0.0.0.0` in a trusted workspace, devices on your local network can open the served site, subject to your firewall. Set `auricLive.host` to `127.0.0.1` to prevent this, or require a login with `auricLive.auth.enabled`.
- **Proxy targets.** Requests for proxied paths go to the targets you configure.
- **Run targets.** Commands you run as run targets do whatever they are written to do.
- **Links you choose.** Links to auricartisan.com in the panel, preview and dashboard open in your browser.

How captured data is protected

Data	Protection
Credential headers (`Authorization`, `Cookie`, `Set-Cookie` and similar)	Always masked before storage
Sensitive query parameters and JSON or form fields	Redacted by default (`auricLive.inspector.redactSensitive`); add names with `auricLive.inspector.sensitiveFields`
Incomplete JSON bodies	Left out rather than partly exposed
Request and response bodies in HAR exports	Left out unless `auricLive.inspector.harIncludeBodies` is on
Session records	Never store query strings, headers, bodies, cookies, IP addresses, user agents or storage values
Browser console capture	Off by default; redacted and rate-limited when on; not saved to session records
Dashboard storage summary	Counts and sizes only; stored values are never read
Log output	Credential-like values masked; paths written relative to the served root

Data	Protection
Environment variables in run commands	Values of sensitive-looking names are redacted in displays and diagnostics
Certificate passphrases	Kept for the session only
Basic-authentication password	Stored in plain settings; do not use a real password

Redaction cannot recognise every kind of personal data. Review any HAR file, diagnostic report or session record before sharing it.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Release notes

What changed in each published version of Auric Artisan Live.

Dates are when each version appeared on the Visual Studio Marketplace.

0.2.1 — English and Hindi (11 September 2026)

Hindi is built into the same extension; there is nothing extra to install and no translation service.

- Choose **Auto**, **English** or **हिन्दी** in the Auric Live panel header. The choice is saved in `auricLive.interfaceLanguage`. **Auto** follows VS Code's display language, including regional Hindi locales.
- The control panel, quick settings, request controls, inline Phone QR guidance and common messages are translated.
- All 54 command titles and the three sidebar view names have Hindi versions that follow VS Code's display language.
- New preview tabs use the selected language; reopen an existing preview to update it.
- Switching language keeps the current section and request filters, and does not restart the server.
- Technical messages without a translation stay in English. Served websites, request data, commands and the browser dashboard are not translated.
- HAR exports now record the installed extension version.

0.2.0 — A more connected development workspace (8 September 2026)

Added

- **Development workspace.** The control panel's Overview, Requests, Runtime, Routing and Settings sections bring server controls, backend setup and diagnostics into one tab.
- **Inline phone QR.** Show a locally generated QR code inside the panel, copy the address and read network and HTTPS guidance without another tab.
- **Request exploration.** Combine search and status filters, sort by time, latency or size, page through results, pause the display and open details on demand.
- **Runtime diagnostics.** Check prerequisites and backend readiness, use optional HTTP health paths, or connect to an existing server with a URL-only run profile.
- **Browser console.** Opt-in console mirroring with level filters, bounded history, credential redaction, rate limiting and copy and clear controls.
- **Certificate import checks.** Imported certificates are checked for matching keys and validity dates. Generated certificates also cover IPv6 loopback.
- New commands: **Select Active Server**, **Toggle Browser Console Capture**, **Use HTTPS Certificate and Key** and **Diagnose Project and Runtime**.

Improved

- A clearer Marketplace title, **Auric Artisan Live — Live Server & Preview**. The extension ID is unchanged.
- The Auric visual design across the panel and the browser dashboard.
- Keyboard-friendly custom drop-down menus throughout the panel, request filters, workspace selection, preview and browser console.
- Layouts that work in narrow editor splits and on phones.
- A responsive request feed that keeps your search focus, selection and display preferences.
- Inspector totals shown separately from a clearly labelled recent-request p95.
- Upload pacing and consistent download shaping across static files, proxied traffic, injected HTML, mocks and REST responses.
- Changing run targets keeps unrelated session settings and API routes; unrelated configuration changes keep in-memory REST data.
- Links to auricartisan.com in the panel, preview and dashboard.

Fixed

- Mouse-wheel, trackpad and keyboard scrolling in the browser dashboard, including on pages that block scrolling.
- Clipped console drop-downs, doubled borders and menus running off screen.
- `Escape` closing the dashboard before an open drop-down.
- Stale phone QR codes after the server stopped or its address changed.
- Request updates from one workspace appearing in another.
- Unnecessary restarts after a clean exit, and duplicate restart attempts.
- Stalled or oversized uploads and abandoned transfers holding resources too long.

Privacy and safeguards

- Sensitive query and body fields are redacted before storage or export.
- HAR bodies need explicit opt-in; incomplete or redacted captures cannot be replayed.
- Workspace trust is enforced for project-supplied runtime, proxy and CGI settings.
- Static files cannot be reached through links that lead outside the served folder.
- Console capture is off by default.

Compatibility

- Requires VS Code 1.95 or newer. Backend runtimes are installed separately.
- Self-signed certificates still need trust on each browser or device.

0.1.0 — First release (31 July 2026)

- A local HTTP and HTTPS server with clean URLs, single-page-app fallback, directory listings, custom headers, basic authentication and safe dot-file handling.
- Live reload on explicit saves by default, optional file watching, CSS and image hot swap, scroll restoration and reconnection.
- Reload-storm protection and generated-file exclusions.
- Compression, range requests, ETags and conditional responses.
- Framework and language detection, custom launch commands, port discovery, backend monitoring and bounded crash recovery.

- HTTP and WebSocket proxying, mock routes, JSON REST collections and PHP through CGI in trusted workspaces.
- Network simulation, a responsive editor preview, network access and phone QR codes.
- The **Servers**, **Requests** and **Routes** views, request replay, curl and fetch copying and redacted HAR export.
- The in-page browser dashboard with health, performance, network, storage, server and environment tabs.
- The custom cursor with link, button and text states and a native-cursor switch.
- Multi-root workspaces, `auric.live.json`, project exclusions and local session records.
- No telemetry and no runtime npm dependencies.

VS CODE EXTENSIONS › AURIC ARTISAN LIVE

Related

Other Auric Artisan extensions and pages that work well alongside Auric Artisan Live.

Other Auric Artisan VS Code extensions

- Auric Artisan Formatter — format the HTML, CSS and JavaScript you are previewing, with a diff preview first.
- Auric Artisan File System — create project structures, batch rename files and find related files in the Explorer.
- Auric Artisan Font Hub — add web fonts to the pages you are serving.
- Auric Artisan Studio — colour, accessibility and code-health checks for the same project.

Setup

- Install and set up the Auric Artisan extensions
- Use an extension in Hindi
- Settings, sync and workspace trust

Auric Artisan pages

- VS Code extensions: https://auricartisan.com/vscode/
- Marketplace listing: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-live
- Contact and support: https://auricartisan.com/company/contact/

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Auric Artisan File System

What Auric Artisan File System does, who it is for, how to install it and where to find every guide.

Auric Artisan File System adds a file workspace to the VS Code Explorer. It keeps VS Code's own Explorer, with its New File, New Folder, Rename and drag-and-drop, and adds the file work the Explorer does not do: creating whole project structures in one line, templates and framework presets, batch renaming with a preview, finding a file's test, style or story, tags, groups and profiles, a graphical **Properties** view, duplicate review, and a workspace trash you can restore from.

Its commands appear in the **Auric Files** section of the Explorer, in an editor-sized workspace panel, in an **Auric Files** submenu on Explorer and editor context menus, and in the Command Palette under **Auric Files**. There are 81 commands in seven groups: **Create**, **Inspect & navigate**, **Edit & batch**, **Organise**, **Find & filter**, **History & recovery** and **Workspace**.

Operations are guarded: paths are checked, protected files cannot be renamed, moved or deleted, collisions are planned before anything changes, bulk operations ask first, and every Auric operation is recorded so you can undo it. Everything runs locally; no account is needed.

Who it is for

- Developers who create the same file patterns again and again: a component with its test, style and story.
- Anyone renaming or reorganising many files at once and wanting to see the result before it happens.
- People who work in large repositories and want tags, working groups and saved filters.
- Anyone who wants a safety net for deletes inside VS Code.

Where to find it

- Marketplace ID: `auric-artisan.auric-artisan-file-system`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-file-system
- Install from a terminal:

```
code --install-extension auric-artisan.auric-artisan-file-system
```

- Requires VS Code 1.95 or newer. Version 0.1.0 at the time of writing. The interface is in English.

Quick start

1. Install the extension and open a workspace folder.
2. In the Explorer, expand the **Auric Files** section, or run **Auric Files: Open File Workspace** for the larger editor panel.
3. Select a file in the Explorer or open one in the editor. The panel shows it as the **Selected resource**.
4. Type in **Find an action** (for example `rename` or `trash`), or open a group and choose an action.
5. Right-click a file in the Explorer and choose **File Properties** (or press `Alt` + `Enter` with the Explorer focused) to see its details.
6. Right-click and open the **Auric Files** submenu for grouped operations: **Create**, **Edit and Batch**, **Navigate and Paths**, **Organize**, **Safety and History** and **Sort**.
7. For a batch operation or a comparison, select several files in the Explorer first, then choose the action.
8. If something went wrong, run **Auric Files: Undo Last File Operation**.

What you can do

- Create files and folders, nested paths, and whole structures with braces and ranges: `src/{Button,Card}/{index.ts,styles.css}`.
- Create from templates with variables, filters and conditions, or from ten framework presets: React, Vue, Angular, Svelte, Node.js, Python, Java, Go, Rust and Generic.
- Jump between a file and its test, style, story, component or documentation, and generate missing companions.
- Rename, move, copy, duplicate and convert extensions, one file or many, with a before-and-after preview.
- Compare two files, merge in VS Code's merge editor, and export a JSON manifest of a selection.
- Tag files, collect them into groups and virtual folders, mark favourites, pins and highlights, and save workspace profiles.
- Search with glob, regular expression or fuzzy matching, and filter by kind, tag, extension, path, size, date, flags and Git state.
- Find duplicate and empty files and move the ones you choose to the trash.
- Undo and redo Auric operations, review the history, and restore deleted items from the workspace trash.
- See a file's properties: name, path, type, size, dates, tags, groups, related files, permissions, Git status and history.

In this folder

- docs/ — task guides and the full reference.
  - Getting started
  - Create files and structures
  - Templates and generators
  - Rename, move and copy
  - Related files
  - Properties, compare and export
  - Tags, groups and profiles
  - Search, filters and duplicates
  - History, undo and the workspace trash
  - Reference: commands, keys and settings
  - Troubleshooting
  - FAQ
- others/ — supporting material.
  - Glossary
  - Shortcuts
  - Limits and accuracy
  - Privacy
  - Related
  - Release notes

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Auric Artisan File System guides

Contents of the Auric Artisan File System task guides and reference.

Start with the getting-started guide for a tour of every surface. The task guides follow the extension's own groups. The reference lists all 81 commands, the 69 keyboard shortcut declarations and all 48 settings.

File	What it covers
getting-started.md	Install, the workspace panel and sidebar, menus, Quick Actions, a first task
create-files-and-structures.md	New file and folder, sibling and child files, batch creation with braces and ranges
templates-and-generators.md	Templates, their file format and language, framework presets, custom generators
rename-move-and-copy.md	Rename, batch rename modes, move, copy, duplicate, convert extension, previews and conflicts
related-files.md	Navigate to tests, styles, stories, components and docs; generate missing companions; custom rules
properties-compare-export.md	The Properties view, compact hovers, compare, merge, path copying and selection export
organise-tags-groups-profiles.md	Tags, groups, virtual folders, favourites, pins, highlights, exclusions, focus mode, profiles
search-filter-and-duplicates.md	Search, filters, saved filters, sorting, grouping, duplicate and empty-file review
history-and-workspace-trash.md	Undo, redo, operation history, delete, the workspace trash, protected paths
reference.md	Every command, keybinding, menu and setting
troubleshooting.md	Symptom, cause and fix
faq.md	Short answers to common questions

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Getting started with Auric Artisan File System

Install Auric Artisan File System, learn where its controls live, and complete a first task.

Install

1. Check that VS Code is version 1.95 or newer (**Help** › **About**).
2. In the Extensions view (`Ctrl` + `Shift` + `X`), search for `@id:auric-artisan.auric-artisan-file-system` and select **Install**. Or run:

   ```
   code --install-extension auric-artisan.auric-artisan-file-system
   ```

3. Open a workspace folder. File operations need a trusted workspace; in Restricted Mode you can browse, search, filter and navigate, but not create, rename, move, duplicate or delete.

The screen tour

The Auric Files section in the Explorer

In the Explorer, below your file tree, expand **Auric Files**. It is a compact companion to the native Explorer, not a second file tree. It shows the same tools as the full workspace panel in a narrow layout. Its title bar has an **Open File Workspace**

button for the larger panel.

The workspace panel

Run **Auric Files: Open File Workspace**, select **Open workspace ↗** in the sidebar, or use the button in the Explorer title bar. The panel opens as an editor tab called **Auric Files**. From top to bottom:

Area	What it shows
Header	The Auric Artisan mark and a link to auricartisan.com
Active workspace	The workspace name, whether it is trusted and whether the index is ready, with **Open Explorer** → and **Open workspace ↗**
Summary	**Indexed files**, **Folders**, **Saved tags** and **Operations** (Auric operations recorded)
Sections	**01 Actions** and **02 Properties**
Selected resource	The file or folder the actions will use, with its path, and **Choose…** to pick another
Status line	Messages about the last action

The selected resource follows the file you open in the editor. When you open the panel from the Explorer with several files selected, the whole selection is kept for batch actions.

01 Actions

- **Find an action** — type to filter every action across the groups (for example `rename`, `tags` or `trash`). `Escape` clears the search.
- The seven groups, each a collapsible section. Opening one closes the others unless you are searching. **Collapse all** closes every group.

Group	Covers
Create	Files, folders and complete structures
Inspect & navigate	Properties, comparisons and companion files
Edit & batch	Rename, duplicate and transform a selection
Organise	Tags, groups and personal workspace views
Find & filter	Search, duplicates and a quieter Explorer
History & recovery	Review operations and recover workspace trash
Workspace	Native Explorer and extension preferences

Actions that change or delete files are marked as such. An action that needs a target shows **Choose a resource inside the open workspace first** if there is none.

- **Icon tools** — when the active file icon theme is **Auric Artisan File Icons: Facet** or **Playful**, this section offers **Explain smart match**, **Set icon…** and **Clear custom icon** for the selected resource, handled by that icon theme. With another icon theme it says **No compatible theme active**. The icon themes are a separate, optional extension.

02 Properties

The file-system facts and Auric organisation data for the selected resource, with **Refresh**, **Copy path**, **Edit tags…** and **Add to group…**. See Properties, compare and export.

Context menus

- **Explorer:** right-click any file or folder for **File Properties** at the top, and an **Auric Files** submenu with **File Properties**, six sub-menus (**Create**, **Edit and Batch**, **Navigate and Paths**, **Organize**, **Safety and History**, **Sort**), **Open File Workspace** and **Open Settings**.
- **Editor and editor tab:** the same **Auric Files** submenu.

Explorer title bar

The native Explorer's title bar gains **Batch Structure...**, a **Sort** menu (name, type, date modified, ascending, descending) and **Open File Workspace**. Its **...** menu has **Quick Actions...**.

Quick Actions

Auric Files: Quick Actions... opens a keyboard-first list of the most used actions, grouped under **VS Code Explorer**, **Create**, **Find and transform**, **Organise** and more. Use it if you prefer the keyboard to the panel.

Explorer badges and hovers

Items you mark show a small badge in the Explorer: ★ favourite, **P** pinned, ● highlighted, **H** hidden, the first two letters of a tag, or **W** for a member of a group. Hovering an item shows a compact summary (name and size, and a pointer to Properties). **Auric Files: Toggle File Metadata** (`Ctrl` + `Shift` + `M` with the Explorer focused) switches these summaries off and on.

A first task: create a component with its companions

1. Right-click the `src` folder and choose **Auric Files** › **Create** › **Batch Create Files...**.
2. Confirm or change the destination folder.
3. Type:

   ```
   components/{Button,Card}/{index.ts,styles.css}
   ```

4. Choose the content for the new files: **Empty files**, your configured default template, or a template from the list.
5. Review the preview: **4 items planned · 0 conflicts**, with every path ticked. Untick anything you want to skip and press `Enter`.
6. Confirm if asked (bulk operations ask once they reach `auricFiles.safety.confirmBulk` paths, 5 by default). Auric Files creates the folders and files and opens the first one.
7. Open `components/Button/index.ts` and run **Auric Files: Go to Related File...** to move between companions.
8. If you made a mistake, run **Auric Files: Undo Last File Operation** (`Ctrl` + `Alt` + `Z` once you enable the shortcut deck).

You now have a nested structure created in one step, recorded in the operation history.

Keyboard shortcuts at a glance

These work when the Explorer has focus:

Keys	Command
`Ctrl` + `N`	**New File...** (Auric's version)
`Ctrl` + `Shift` + `N`	**New Folder...**
`Alt` + `Enter`	**File Properties**
`Ctrl` + `Shift` + `M`	**Toggle File Metadata**
`Alt` + `S`	**Sort** (cycle name, type, date modified)
`Ctrl` + `Alt` + `B`	**Batch Structure...**

About 60 more shortcuts are available once you turn on `auricFiles.keyboard.enableDefaultShortcuts`. See Shortcuts.

Next steps

- Create files and structures
- Rename, move and copy
- History, undo and the workspace trash

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Create files and structures

Create single files and folders, sibling and child files, and whole nested structures in one line with braces and ranges.

All creation commands are in the **Create** group of the workspace panel, the **Auric Files** › **Create** submenu and the Command Palette. They need a trusted workspace.

Command	Use it to
New File...	Create one file (or a folder, with a trailing `/`)
New Folder...	Create one folder, including nested paths
New Sibling File	Create a file next to the selected file
New Child File	Create a file inside the selected folder
Batch Create Files...	Create many files from comma-separated paths
Batch Create Folders...	Create many folders from comma-separated paths
Batch Structure...	Type a mixed structure of files and folders
New File from Template...	Create from a template. See Templates and generators
New from Framework Preset...	Create from a preset or custom generator. See Templates and generators
Create Related Files...	Create a file's test, style, story and other companions. See Related files

For creation directly in the root of a single-folder workspace, use the native Explorer's **New File** and **New Folder**; the panel's target actions need a file or subfolder as their target.

New file or folder

1. Run **Auric Files: New File...** (`Ctrl` + `N` with the Explorer focused) or **New Folder...** (`Ctrl` + `Shift` + `N`).

2. **Choose the destination folder.** The first entry is the current target; the rest are folders in the workspace.
3. Type the name. For **New File...**, a trailing `/` creates a folder. Nested paths such as `components/ui/Button.tsx` create the missing folders too (turn this off with `auricFiles.create.createIntermediateDirs`).
4. If `auricFiles.create.defaultTemplate` names a template, the new file is filled from it; otherwise it is empty.
5. The file opens in the editor (`auricFiles.create.openAfterCreate`), with the cursor at the template's cursor marker if it has one.

These are Auric's versions of the Explorer shortcuts while the Explorer has focus. VS Code's own **New File** and **New Folder** buttons in the Explorer title bar are unchanged.

Sibling and child files

- **New Sibling File** creates a file in the same folder as the selected file. It is offered for files.
- **New Child File** creates a file inside the selected folder. It is offered for folders.

Batch creation

Batch creation turns one line of text into many paths.

1. Run **Batch Create Files...**, **Batch Create Folders...** or **Batch Structure...** (`Ctrl` + `Alt` + `B` with the Explorer focused).
2. Choose the destination folder.
3. Type the paths (see the syntax below). Mistakes are reported as you type.
4. For files, **Choose the content used for every new file**: **Empty files**, the configured default template, or any template.
5. Review the preview. It lists every path as **Ready to create** or **Already exists — the configured conflict policy will apply**, with a summary such as **6 items planned · 1 conflict**. Untick anything to skip and press `Enter`.
6. If the operation touches at least `auricFiles.safety.confirmBulk` paths (5 by default), confirm **Create *N* items?**.

Syntax

You type	You get
`a.ts, b.ts`	Two files; commas separate targets
`src/lib/util.ts`	A nested path; missing folders are created
`docs/`	A folder (trailing slash)
`folder:assets`	A folder, stated explicitly
`file:LICENSE`	A file, stated explicitly
`src/{Button,Card}/index.ts`	`src/Button/index.ts` and `src/Card/index.ts`
`src/{Button,Card}/{index.ts,styles.css}`	Four files: every combination
`tests/case-{1..3}.ts`	`case-1.ts`, `case-2.ts`, `case-3.ts`
`tests/case-{01..03}.ts`	`case-01.ts`, `case-02.ts`, `case-03.ts` (leading zeros keep the width)
`step-{0..10..5}.md`	`step-0.md`, `step-5.md`, `step-10.md` (a step size)
`section-{a..c}.md`	`section-a.md`, `section-b.md`, `section-c.md` (letters)
`a\{1\}.ts`	A file literally named `a{1}.ts`; a backslash before a brace or comma makes it literal
`src\components\Button.tsx`	Backslashes elsewhere are treated as folder separators, so Windows paths work

Each line can hold several comma-separated targets, and you can type several lines. Braces that are neither a list nor a range, such as `{png}`, are kept literally.

To take the input literally, with no brace expansion, turn off `auricFiles.create.expandBraces`.

Limits

- One target may expand to at most 1,000 paths; beyond that it is rejected with **expands to more than 1000 paths**.
- One batch creates at most 2,000 paths; it stops with **Stopped after 2000 paths; narrow the input and try again**.
- Duplicate paths in the input are created once.
- Invalid names (for example characters Windows does not allow) are reported and skipped.

What happens to existing files

A path that already exists is a *conflict*. The preview marks it, and the configured conflict policy decides the outcome; nothing is overwritten silently. Paths matching `auricFiles.safety.protectedPaths` cannot be touched at all.

Every creation is recorded in the operation history, so **Undo Last File Operation** can reverse it.

When a batch finishes, the new folders and files appear in the Explorer and the first new file opens in the editor.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

History, undo and the workspace trash

Undo and redo Auric file operations, review the history, delete safely into the workspace trash, restore items and control the safety settings.

These commands are in the **History & recovery** group of the workspace panel and the **Auric Files** › **Safety and History** submenu.

Undo and redo

Every operation Auric Files performs (create, rename, move, copy, duplicate, delete, convert) is recorded as one step, however many files it touched.

Command	What it does	Deck shortcut
Undo Last File Operation	Reverse the most recent Auric operation	`Ctrl` + `Alt` + `Z`
Redo File Operation	Redo the last undone operation	`Ctrl` + `Alt` + `Y` (macOS `Cmd` + `Alt` + `Shift` + `Z`)
Show Operation History	List recent operations; choose one to undo back to it	`Ctrl` + `Alt` + `A`, then `H`

When it works you see **Undid *operation*.** or **Redid *operation*.** If an undo cannot be completed, both the undo and redo lists stay as they were rather than pretending.

- The history keeps the last `auricFiles.safety.historyLimit` operations (default 200, range 10–2000).
- It is saved in VS Code's storage for the workspace, so it survives a restart.
- It records **Auric** operations only. Edits you make in the editor, VS Code's own Explorer actions (rename with `F2`, drag-and-drop) and changes made by other tools are not in it, and it does not keep old copies of file contents. Keep using version control and backups.

With no history yet: **No file operations have been recorded yet.**

Delete into the workspace trash

The *workspace trash* is a folder inside your workspace, `.auric/trash`, where Auric Files moves deleted items so you can restore them.

1. Select items and run **Delete (Workspace Trash)** from the panel, the context menu or the deck (`Ctrl` + `Alt` + `Shift` + `Backspace`).
2. Confirm **Move *N* items to the workspace trash?**. The dialog lists what goes and says you can **Restore from the Trash surface or with Undo**.
3. The items move to `.auric/trash`.

Each deleted item is kept in its own folder in `.auric/trash`, with a small record of where it came from, so the trash can always be rebuilt even if its index file is lost.

Delete permanently

Delete Permanently (`Ctrl` + `Alt` + `A`, then `Shift` + `Backspace` with the deck) asks **Permanently delete *N* items?** with **This cannot be undone.** and removes the items without using the workspace trash. If you turn off `auricFiles.safety.useWorkspaceTrash`, every delete is permanent.

Delete (Workspace Trash) and **Delete Permanently** act on the selection and are not listed in the Command Palette.

Browse and restore

- **Open Workspace Trash** lists every entry by its original name. Choose one to restore it, or choose **Empty Workspace Trash…** at the end of the list. If the trash is empty: **The workspace trash is empty.**
- **Restore from Trash…** restores a chosen entry to its original place. If something already exists there, it is restored under a free name instead of overwriting. The message says **Restored *path*.**

Empty the trash

Empty Workspace Trash asks **Permanently delete *N* items from the workspace trash?**, shows how much space will be freed and says **This cannot be undone.** Choose **Empty Trash** to confirm.

The trash also cleans itself:

Setting	Default	What it does
`auricFiles.safety.useWorkspaceTrash`	`true`	Use the workspace trash for deletes
`auricFiles.safety.trashRetentionDays`	14 (1–365)	Days an entry is kept before it is purged automatically
`auricFiles.safety.trashMaxBytes`	268435456 (256 MiB)	Size limit; the oldest entries are purged when a delete would exceed it

A single item larger than the whole trash limit is refused with an explanation rather than half-stored.

The workspace trash is left out of Auric's index, search and duplicate review. You may want to add `.auric/` to your `.gitignore`. In virtual workspaces the operating system's trash is not available.

Safety settings

Setting	Default	What it does
`auricFiles.safety.confirmDelete`	`true`	Ask before anything is deleted, listing what goes and where
`auricFiles.safety.confirmBulk`	5	Ask once an operation touches this many paths; `0` asks every time
`auricFiles.safety.protectedPaths`	see below	Globs no Auric operation may rename, move or delete
`auricFiles.safety.warnOnGitTracked`	`true`	Warn when moving or deleting a file Git tracks
`auricFiles.safety.warnOnUncommittedChanges`	`true`	Warn when moving or deleting a file with uncommitted edits
`auricFiles.safety.previewBeforeApply`	`true`	Show and require approval of a batch rename or move preview
`auricFiles.safety.historyLimit`	200	Operations kept for undo

Default protected paths:

```
[
  "**/.git/**", "**/node_modules/**", "**/.env", "**/.env.*",
  "**/*.pem", "**/*.key", "**/package-lock.json", "**/yarn.lock", "**/pnpm-lock.yaml"
]
```

A plan that touches a protected path is blocked before any change is made. The safety settings are workspace configuration: in an untrusted workspace they come only from your user settings, and file-changing operations are disabled until you trust the folder.

After deleting into the workspace trash, **Open Workspace Trash** lists the item, and restoring it puts it back where it was.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Tags, groups and profiles

Organise files without moving them, with tags, groups, virtual folders, favourites, pins, highlights, Explorer exclusions, focus mode and workspace profiles.

Auric Files lets you organise files by meaning rather than by folder. None of these tools moves or changes a file: they store labels about files. The labels follow a file when you rename or move it with Auric Files.

The commands are in the **Organise** group of the workspace panel and the **Auric Files** › **Organize** submenu.

Where the organisation is stored

Tags, groups, virtual folders, flags, saved filters and profiles are kept in VS Code's storage for this workspace, on your machine. They are not written into your project, are not committed with it, and are not copied by Settings Sync. The one exception is **Toggle Native Explorer Exclusion**, which writes to the workspace's `files.exclude` setting.

Tags

A *tag* is a named, coloured label you can put on any number of files.

Create tags

1. Run **Auric Files: Manage Tags...**.
2. Choose **New tag...**.
3. Type the **Tag name**.
4. **Choose a colour**: gold, amber, orange, red, pink, purple, blue, teal, green or gray.

In **Manage Tags...** you can also choose an existing tag to **Rename** or **Delete** it.

Tag files

1. Select files in the Explorer.
2. Run **Tag Files...** (`Ctrl` + `Alt` + `A`, then `G` with the deck), or choose **Edit tags...** in Properties.
3. Tick the tags to apply and untick any to remove. With no tags yet, the list says **No tags yet — create one from Manage Tags**.

A tagged file shows the first two letters of its first tag as a badge in the Explorer, and its hover names the tag. Filters can restrict results to tags; see Search, filters and duplicates.

Groups

A *group* is a named set of files and folders from anywhere in the workspace: a working set for a feature, a bug or a review.

- **Add to Group...** (`Ctrl` + `Alt` + `A`, then `K` with the deck) — add the selection to an existing group, or choose **New group...** and type a **Group name**.
- **Create Workspace from Selection** — create a group straight from the selection, named after the first item (with a number added if the name is taken). The message says **Created "*name*" with *N* linked items. Hover a member to see the workspace association.**
- **Manage Groups...** — choose a group, then **Rename**, **Delete** or **Open a member**.

Members of a group show a **W** badge in the Explorer, and Properties lists their groups.

Virtual folders

A *virtual folder* gathers items under a name, and can sit inside another virtual folder, without moving anything on disk.

1. Run **New Virtual Folder...** from the panel's **Organise** group.
2. Type the **Folder name**.
3. Choose a parent: **(top level)** or an existing virtual folder.

Manage them in **Manage Groups...**, under **Virtual folders**.

Favourites, pins, highlights and exclusions

Command	Explorer badge	Deck shortcut
Toggle Favorite	★	`Ctrl` + `Alt` + `A`, then `1`
Toggle Pinned	**P**	`Ctrl` + `Alt` + `A`, then `2`
Toggle Native Explorer Exclusion	**H**	`Ctrl` + `Alt` + `A`, then `3`

Command	Explorer badge	Deck shortcut
Toggle Highlight	•	`Ctrl` + `Alt` + `A`, then `4`

Each toggles the flag on the selected items and reports, for example, **Added favorite on 3 items.**

Toggle Native Explorer Exclusion hides the item from VS Code's Explorer by adding it to the workspace's `files.exclude` setting (per folder in a multi-root workspace). Toggling again removes only the entries Auric Files added. To see hidden items again in Auric's own search and filters, turn on `auricFiles.explorer.showHiddenFiles`.

Focus mode

Focus mode narrows Auric's indexed file lists to the paths you are working on.

1. Run **Auric Files: Toggle Focus Mode**.
2. **Focus Mode — choose paths**: pick the folders or files to focus on.
3. The indexed file list opens, limited to those paths: **Focused on *N* paths.**

Run the command again to switch it off (**Focus mode off.**). Focus mode affects Auric's lists; it does not change VS Code's own Explorer tree.

Workspace profiles

A *profile* is a saved way of looking at the workspace: the sort, grouping, active filter, which folders are open, focus mode and the hidden list. It stores no file contents and no absolute paths, and switching profile never changes a file.

- **Save Workspace Profile…** — type a **Profile name**. **Saved the "*name*" profile.**
- **Switch Workspace Profile…** — choose a profile; the active one is marked **active**. With none saved: **No workspace profiles saved yet.**

Remove organisation

- Remove a tag from files with **Tag Files…** (untick it), or delete the tag in **Manage Tags…**.
- Delete a group or virtual folder in **Manage Groups…**; the files themselves are untouched.
- Toggle a flag again to clear it.

When you tag or flag files, the Explorer badges update straight away, and Properties shows the tags and groups.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Properties, compare and export

Inspect a file's properties, compare and merge files, copy paths, and export a JSON manifest of a selection.

These commands are in the **Inspect & navigate** group of the workspace panel and the **Auric Files** › **Navigate and Paths** submenu.

Properties

Properties shows the facts about a file or folder in one place: what the file system reports, plus what Auric Files knows about it. It reads metadata only, never the file's contents.

Open it

- Right-click a file or folder in the Explorer and choose **File Properties** (the first entry in the menu).
- Press `Alt` + `Enter` with the Explorer focused.
- Run **Auric Files: File Properties**. If VS Code does not pass a selected file (some keyboard routes do not), a resource picker opens.
- In the workspace panel or the **Auric Files** sidebar, choose **02 Properties**.

Properties opens inside the existing workspace panel or sidebar rather than a new tab.

What it shows

General

Row	Meaning
Name	The file or folder name
Full path	The complete path (or URI for remote and virtual files)
Type	The extension in capitals (for example `TSX`), **Folder** or **Symbolic link**
Size	The reported size; folders show **Folder** (this is not a recursive disk-usage total)
Created	Creation time, where the file system records one; otherwise **Unavailable**
Modified	Last modification time

Organisation

Row	Meaning
Tags	Tags assigned to the item, or **None**
Group / workspace	Groups and virtual folders it belongs to
Related files	Up to six existing companions, such as its test or stylesheet. Before the index is built: **Not indexed yet**

Access & history

Row	Meaning
Owner / creator	**Not exposed by the VS Code file-system provider**; never guessed
Permissions	**Read-only** if the file system reports it, otherwise **No read-only flag reported**; plus **protected by Auric** for protected paths
Git status	The state from the last index, labelled as such, or **Unavailable**
Version	How many Auric operations touched the item and the latest one, or **No Auric operation history**

Buttons

- **Refresh** — re-read the file-system facts. (To update related files and Git state, refresh the workspace index with **Auric Files: Refresh**.)
- **Copy path** — copy the full path.
- **Edit tags…** — assign or remove tags.
- **Add to group…** — add the item to a group.

A resource outside the open workspace can be inspected, but file actions on it are blocked: **This resource is outside the workspace. Properties are available; choose an in-workspace resource to use file actions.**

Compact hovers in the Explorer

Hovering an item in the Explorer shows a short summary: its name and size, and a reminder that Properties has the rest. Tagged, favourite, pinned, highlighted and hidden items say so in the hover.

Switch the summaries off and on with **Auric Files: Toggle File Metadata** (`Ctrl` + `Shift` + `M` with the Explorer focused). When off, the hover says **Metadata is hidden. Press Ctrl+Shift+M to show it.**

Compare two files

1. Select exactly two files in the Explorer.
2. Run **Auric Files: Compare Selected Files** (`Ctrl` + `Alt` + `Shift` + `D` with the deck).
3. VS Code's diff editor opens with the first file on the left and the second on the right.

Selecting anything other than two files shows **Select exactly two files in VS Code Explorer to compare.**

Merge files

1. Select two files, or three: a base file followed by two inputs.
2. Run **Auric Files: Merge Selected Files**.
3. VS Code's merge editor opens. With three files, the first is the base and the result; with two, the first file is both base and result.
4. Resolve the changes and save the merge editor. Nothing is written until you save.

This needs a trusted workspace. Some VS Code-based editors do not provide the merge editor; you then see **This VS Code-compatible editor does not expose the merge editor.**

Copy paths and reveal

Command	What it copies or does	Deck shortcut
Copy Relative Path	The path from the workspace folder	`Ctrl` + `Alt` + `Shift` + `C`
Copy Absolute Path	The full path on disk	`Ctrl` + `Alt` + `Shift` + `A`
Copy File URI	The file's URI (`file:///…`)	`Ctrl` + `Alt` + `A`, then `U`
Reveal in File Manager	Shows the file in your operating system's file manager	`Ctrl` + `Alt` + `A`, then `V`
Reveal in VS Code Explorer	Selects the file in the Explorer	`Ctrl` + `Alt` + `A`, then `Shift` + `V`

Reveal in File Manager is not available in virtual workspaces.

Export a selection

Export Selected Items… saves a JSON *manifest*: a list describing the selected items and their metadata. It is not an archive of the files' contents.

1. Select files and folders in the Explorer.
2. Run **Auric Files: Export Selected Items...** (`Ctrl` + `Alt` + `A`, then `Shift` + `X` with the deck).
3. Choose where to save. The default name is `auric-selection.json`.

The manifest looks like this:

```
{
  "format": "auric-artisan-selection",
  "version": 1,
  "workspace": "my-project",
  "exportedAt": "2026-09-25T10:00:00.000Z",
  "items": [
    {
      "name": "Button.tsx",
      "path": "src/components/Button.tsx",
      "uri": "file:///…/src/components/Button.tsx",
      "type": "file",
      "extension": ".tsx",
      "size": 1834,
      "created": "2026-09-20T08:12:00.000Z",
      "modified": "2026-09-24T16:40:00.000Z",
      "owner": null,
      "tags": ["UI"],
      "groups": ["Checkout"],
      "related": ["src/components/Button.test.tsx"],
      "versions": 2,
      "permissions": "writable",
      "gitStatus": "clean"
    }
  ]
}
```

`owner` is always `null` because the file system does not report it. Values that are not available are `null`.

When the export finishes, a message confirms how many items were exported and where.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Related files

Jump between a file and its test, style, story, component and documentation, create missing companions, and teach Auric Files your own conventions.

Most source files come with *companions*: a component has a test, a stylesheet and a story; a module has documentation and type declarations. Auric Files knows the usual conventions for each framework preset, so it can open a file's companion in one step, show which ones are missing and create them.

Go to a companion

With a file open in the editor:

Command	Opens	Deck shortcut
Auric Files: Go to Related File...	A list of every companion that exists	`Ctrl` + `Alt` + `R`
Auric Files: Open Matching Test	The file's test	`Ctrl` + `Alt` + `A`, then `Q`
Auric Files: Open Matching Style	The file's stylesheet	`Ctrl` + `Alt` + `A`, then `S`

Command	Opens	Deck shortcut
Auric Files: Open Matching Story	The file's story	`Ctrl` + `Alt` + `A`, then `W`
Auric Files: Open Matching Component	The component a test, style or story belongs to	`Ctrl` + `Alt` + `A`, then `I`
Auric Files: Open Matching Documentation	The file's documentation	`Ctrl` + `Alt` + `A`, then `Shift` + `D`

The shortcuts need `auricFiles.keyboard.enableDefaultShortcuts`. The same commands are in **Auric Files** › **Navigate and Paths** on a file's context menu, and in the **Inspect & navigate** group of the panel.

Navigation works in both directions: from `Button.tsx` to `Button.test.tsx`, and from the test back to the component.

See the whole group

Show Related File Group lists every companion the rules expect for the file, marking which exist and which are missing. It runs from the panel's **Inspect & navigate** group.

The Properties view also lists up to six related files that exist. See Properties, compare and export.

Create missing companions

- **Auric Files: Create Related Files...** (`Ctrl` + `Alt` + `A`, then `R` with the deck) — choose which companions to create from a list.
- **Auric Files: Generate Missing Companion Files** — create every missing companion at once. If none is missing: **Every companion file already exists.**

New companions are created from the rule's template, using the preset's style language and test framework (`auricFiles.create.componentStyleLanguage`, `auricFiles.create.testFramework`).

Relationship kinds

The built-in rules use these kinds: **Component**, **Test**, **Style**, **Story**, **Template**, **Types**, **Snapshot**, **Schema**, **Documentation**, **Barrel** (an index file that re-exports) and **Config**. Which ones apply depends on the active preset.

Settings

Setting	Default	What it does
`auricFiles.related.enabled`	`true`	Resolve companions for the active file
`auricFiles.related.testDirectories`	`__tests__`, `tests`, `test`, `spec`	Folders searched when tests live apart from the source
`auricFiles.related.storyExtensions`	`.stories.tsx`, `.stories.jsx`, `.stories.ts`, `.stories.js`, `.stories.svelte`, `.stories.mdx`	Story file suffixes, tried in order
`auricFiles.related.rules`	none	Your own relationships (below)

Add your own rules

A rule connects a source pattern to one or more target patterns. Add rules for conventions the presets do not know, such as a repository layer or a generated client.

```
{
  "auricFiles.related.rules": [
    {
      "id": "api-client",
      "kind": "client",
      "label": "Generated API client",
      "from": "{dir}/{name}.service.ts",
      "to": ["{dir}/generated/{name}.client.ts"],
      "presets": []
    }
  ]
}
```

Field	Required	Meaning
`id`	Yes	Identifier; a built-in rule with the same id is replaced
`kind`	Yes	The relationship name shown beside the companion: one of the kinds above, or your own
`from`	Yes	Pattern matching the source file
`to`	Yes	Candidate targets in priority order. Navigation opens the first that exists; generation writes the first
`label`	No	Wording in related-file lists
`presets`	No	Presets the rule belongs to; empty applies everywhere
`templateId`	No	Template used when the companion is generated

Pattern tokens

Token	Meaning
`{dir}`	Zero or more leading folders, captured
`{name}`	The file name without extensions, so `Button.test.tsx` is read as `Button`
`{ext}`	The extension without its dot; defaults to the source file's own
`{testDir}`	Each configured test folder, in order
`{storyExt}`	Each configured story extension, in order
`{mirrorDir}`	The source folder relocated under a test root (`src/ui` becomes `tests/ui`), and back again in reverse
`{a,b}`	A brace set, expanded into alternatives

Rules are read backwards too: a file matching a `to` pattern resolves back to its `from` file. Relation rules come from workspace settings, so they only apply in a trusted workspace.

Index

Related files are found through the workspace index. Until the index is built, Properties says **Not indexed yet**; run **Auric Files: Refresh** to build it.

With a component open, **Go to Related File...** lists its companions, and choosing one opens it.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Rename, move and copy

Rename, move, copy and duplicate files one at a time or in batches, convert extensions, and review every change before it happens.

These commands are in the **Edit & batch** group of the workspace panel and the **Auric Files** › **Edit and Batch** submenu. They need a trusted workspace. VS Code's own **Rename** (`F2`) and drag-and-drop in the Explorer keep working as usual; the Auric versions add collision planning, protection, confirmations and undo.

Command	Use it to
Rename...	Rename the selected item
Rename Current File	Rename the file open in the editor
Move... / **Move Current File**	Move items, or the current file, to another folder
Copy To...	Copy items to another folder
Duplicate / **Duplicate Current File**	Make a copy beside the original
Batch Rename...	Rename many files by a rule, with a preview
Batch Move...	Move many items to one folder
Convert Extension...	Change the extension of many files

Rename..., **Move...**, **Copy To...** and **Duplicate** act on the selection and run from the panel, the context menus or the shortcut deck; they are not listed in the Command Palette. The "Current File" versions act on the editor's file.

Before anything changes

Every operation is planned first:

- **Protected paths.** Anything matching `auricFiles.safety.protectedPaths` cannot be renamed, moved or deleted. By default that covers `.git`, `node_modules`, `.env` files, `.pem` and `.key` files, and the npm, Yarn and pnpm lockfiles. A plan that touches one is blocked before any byte is written.
- **Collisions.** A target that already exists is a conflict and is reported. Nothing is overwritten silently.
- **Case-only renames** (`button.ts` to `Button.ts`) on a case-insensitive file system are applied as two renames, and the preview says so.
- **Git warnings.** Moving or deleting a file tracked by Git (`auricFiles.safety.warnOnGitTracked`) or with uncommitted edits (`auricFiles.safety.warnOnUncommittedChanges`) shows a warning first.
- **Bulk confirmation.** An operation touching at least `auricFiles.safety.confirmBulk` paths (default 5) asks first. Set it to `0` to be asked every time.
- **History.** The operation is recorded so **Undo Last File Operation** can reverse it.

Rename one file

1. Right-click the file and choose **Auric Files** › **Edit and Batch** › **Rename Current File**, or run **Auric Files: Rename Current File** with the file open.
2. Type the new name.

3. The file is renamed; tags, groups and other Auric metadata follow it.

Batch rename

1. Select the files in the Explorer (hold `Ctrl` or `Shift` while clicking).
2. Run **Auric Files: Batch Rename...**.
3. Answer **How should *N* files be renamed?**:

Mode	What it does	You then enter
Find and replace	Replace literal text	**Find**, then **Replace with**
Regular expression	Replace by pattern; use `$1`, `$2` for capture groups	**Find** (a regular expression), then **Replace with**
Pattern	Build each name from tokens	**Pattern**, for example `{parent}-{name}{ext}`
Sequence	Number the files	A pattern; the default is `{name}-{n}{ext}`
Change case	Change the letter case	A style: `kebab`, `camel`, `pascal`, `snake`, `upper`, `lower`, `title`
Change extension	Swap one extension for another	The extension to replace (blank for any), then the new one

4. Review **Batch Rename — preview**: *N* files would change, with any problems counted. Each row shows the old name and, with an arrow, the new one; rows with a problem show a warning and the reason.
5. Choose **Apply** to rename, or press `Escape` to cancel.

With `auricFiles.safety.previewBeforeApply` on (the default), the preview is always shown and needs your approval.

Pattern tokens

Token	Replaced with
`{name}`	The file name without its extension (compound extensions such as `.test.ts` count as one)
`{ext}`	The extension, including the dot
`{n}`	A running number, starting at 1
`{index}`	The file's position in the selection, starting at 0
`{parent}`	The parent folder's name
`{date}`	Today's date as YYYY-MM-DD
`{rand}`	Six random letters and digits

Regular expressions

The preview updates as you type. A pattern that is too slow to preview stops the batch with **Pattern is too slow to preview; simplify it.** Patterns are limited to 512 characters.

Move or copy

1. Select the items and run **Move...**, **Batch Move...** or **Copy To...** (or **Move Current File** for the editor's file).
2. **Choose the destination** from the folders in the workspace, or choose **Browse...** to pick a folder on disk.
3. Confirm if asked.

Moved items keep their tags, groups and other Auric metadata.

Duplicate

Duplicate (or **Duplicate Current File**) makes a copy beside the original under a name that does not clash with an existing file.

Convert extensions

1. Select the files and run **Auric Files: Convert Extension...**.
2. **Which extension should change?** Choose one of the extensions in the selection, or **Any extension**.
3. Type the **New extension**, for example `.ts`.
4. Review the preview and choose **Apply**.

Converting renames files only; it does not change their contents.

Undo

Every rename, move, copy and duplicate made through Auric Files can be undone with **Auric Files: Undo Last File Operation**. See History, undo and the workspace trash.

After a batch rename, the Explorer shows the new names, and **Show Operation History** lists the rename as one operation.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Search, filters and duplicates

Search the workspace index, build and save filters, sort and group files, and review duplicate and empty files.

These commands are in the **Find & filter** group of the workspace panel. They work on the *workspace index*: a list of the workspace's files and folders that Auric Files builds the first time you need it and keeps up to date. Searching and filtering work in untrusted workspaces too.

The workspace index

- It is built on demand, the first time a command needs it. The panel shows **Index builds when needed** until then, and **Workspace index ready** after.
- Folders in `auricFiles.explorer.excludePatterns` (by default `node_modules`, `.git`, `dist`, `out`, `.next`, `__pycache__`, `target` and `vendor`) and anything your `.gitignore` excludes (`auricFiles.explorer.respectGitignore`) are never indexed, searched, compared or batch-processed.
- One scan holds at most `auricFiles.explorer.maxNodes` entries (default 20,000). A larger workspace is marked as truncated, and counts say so.
- After the first scan, changes are followed automatically (`auricFiles.performance.watchWorkspace`). Run **Auric Files: Refresh** to rebuild it by hand.

Search

1. Run **Auric Files: Search Files...**, or **Quick Open in Auric Files...** (`Ctrl` + `Alt` + `O` with the deck), which focuses the Explorer first.

2. Type part of a name or path. The title shows the default search mode.
3. Choose a file to open it.

The list holds up to `auricFiles.search.maxResults` files (default 2,000). With `auricFiles.explorer.showFileSize` on, each result shows its size.

Filters

A *filter* narrows the indexed file list by several conditions at once.

Build a filter

1. Run **Auric Files: Apply Filter...**.
2. If you have saved filters, start from **Current filter** or one of the **Saved filters**.
3. Answer each step. Every step is optional; press `Enter` to skip it.

Step	What you enter
Search text	A glob, a regular expression or a fuzzy fragment
How should the text be matched?	`glob` wildcards (`src/**/*.tsx`), a `regex` regular expression, or `fuzzy` loose matching
Restrict to kinds	`file`, `folder`, `symlink`
Restrict to tags	Any of your tags
Extensions	Comma-separated, such as `.ts, .tsx, .vue`
Path globs	Include patterns, then `!exclude` patterns, such as `src/**, packages/**, !**/generated/**`
Size range	Bytes as `min..max`, either side may be blank, such as `1024..1048576`
Modified-date range	`YYYY-MM-DD..YYYY-MM-DD`, such as `2026-01-01..2026-12-31`
Flags	**Favorite**, **Pinned**, **Hidden**, **Highlighted**, **Empty files and folders only**, **Case-sensitive text**
Git states	`modified`, `added`, `deleted`, `untracked`, `ignored`, `conflict`, `clean`

4. The results open as **Filtered Files — *summary of the filter***. Choose one to open it. If nothing matches: **No files match the selected filter.**

The default match mode is `auricFiles.search.defaultMode` (`glob`). Case is ignored unless you tick **Case-sensitive text** or turn on `auricFiles.search.caseSensitive`. Dot-files and hidden items are left out unless `auricFiles.explorer.showHiddenFiles` is on.

Save and clear

- **Save Current Filter...** — type a **Name for this filter**. It then appears under **Saved filters** in **Apply Filter...**.
- **Clear Filter** — remove the active filter (**Filter cleared.**).

Sorting

VS Code's Explorer

Auric Files can set the sort order of VS Code's own Explorer. It writes the `explorer.sortOrder` setting, and its direction, for this workspace.

Command	Effect
Sort (`Alt` + `S` with the Explorer focused)	Cycle between name, type and date modified
Sort by Name	Alphabetical
Sort by Type	By file type
Sort by Date Modified	By last change
Sort Ascending / **Sort Descending**	Direction

The same commands are in the Explorer title bar's **Sort** menu and in **Auric Files** › **Sort**. VS Code's Explorer only supports these three orders.

Auric's indexed lists

The indexed pickers can sort by more: `auricFiles.explorer.sort` is `name` (natural order, so `file10` follows `file9`), `path`, `type`, `size`, `modified`, `created`, `extension`, `status` (conflicts and changed Git files first) or `custom`. Add a tie-breaker with `auricFiles.explorer.secondarySort`, set directions with `auricFiles.explorer.sortDirection` and `auricFiles.explorer.secondarySortDirection`, and keep folders on top with `auricFiles.explorer.foldersFirst`.

Grouping

Group Indexed Files... (in the panel's **Find & filter** group) chooses the headings for the indexed file list:

Choice	Headings
No grouping	One sorted list
Files and folders	Files and folders separately
Extension	One heading per extension
Parent folder	One heading per containing folder
Auric tag	One heading per first tag; untagged items last
Modified time	Today, this week, this month and older
Git status	Changed, untracked, conflicted, clean and ignored

The setting is `auricFiles.explorer.grouping`.

Collapse and expand

Collapse All and **Expand All** fold or unfold VS Code's Explorer tree.

Duplicate files

1. Run **Auric Files: Find Duplicate Files**. A progress notification shows the scan; you can cancel it.
2. The **Duplicate Files** list shows each group of matching files. In each group, the first file is kept and the other copies are ticked. Each row says why it matched and how much space the group would free.
3. Adjust the ticks and press `Enter`.
4. The ticked copies are moved to the workspace trash (with the usual confirmation), so you can restore them.

What counts as a duplicate depends on `auricFiles.search.duplicateStrategy`:

Value	Meaning
`content` (default)	Same bytes. Files are grouped by size first, so files with a unique size are never read
`name`	The same file name in different folders
`size`	The same length; the fastest and loosest

In a virtual workspace, content hashing is not available, and Auric Files falls back to comparing by size with a warning.

Nothing is deleted automatically: you choose every file.

Empty files and folders

Auric Files: Find Empty Files lists empty files and folders, all ticked. Untick any to keep, press `Enter`, and the rest move to the workspace trash. If there are none: **Auric Files found no empty files or folders.**

After a filter, the results list shows only the files that meet every condition you set.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Templates and generators

Create files from templates, use the ten framework presets, and write your own templates and generators with variables, filters and conditions.

A *template* is a file pattern: a file name and a body with placeholders that are filled in when you create a file. A *framework preset* is a ready-made set of templates and file relationships for one ecosystem. A *generator* is a template that can also create companion files in the same step.

Create from a template

1. Run **Auric Files: New File from Template...** (**Create** group, or `Ctrl` + `Alt` + `A`, then `T` with the shortcut deck).
2. **Choose a template.** The list includes the active preset's built-in templates, your generators and any templates in your template folders.
3. **Choose the destination folder.**
4. Answer the template's questions, such as the name.
5. The file (and any companions the template defines) is created, and the first file opens with the cursor where the template placed it.

Create from a framework preset

1. Run **Auric Files: New from Framework Preset...** (`Ctrl` + `Alt` + `A`, then `P` with the deck).
2. **Choose a framework preset or custom generator.**
3. **Choose what to create** from that preset's templates.
4. **Choose the destination folder** and answer its questions.

The ten presets

Preset	What it covers
React	Components, hooks, contexts, tests, styles and stories
Vue	Single file components, composables, Pinia stores and specs
Angular	Standalone components with their template, style and spec
Svelte	Runes components, stores, tests and stories
Node.js	CommonJS and ES modules, CLI entry points, tests and declarations
Python	Dataclass modules, pytest suites, package markers and CLI entry points
Java	Classes, records, interfaces and JUnit 5 tests with derived packages
Go	Package files, table tests, command entry points and doc comments
Rust	Modules with inline test blocks, crate roots and error types
Generic	Plain modules, documents and data files for any workspace

The templates in each preset:

- **React:** React Component, React Component (JavaScript), React Hook, React Context Provider, React Component Test, Component Stylesheet, Storybook Story, Barrel Index, Component Types.
- **Vue:** Vue Single File Component, Vue Composable, Pinia Store, Vue Component Test, Component Stylesheet, Storybook Story, Barrel Index, Component Types.
- **Angular:** Angular Component, Component Template, Component Stylesheet, Component Spec, Angular Service, Barrel Index, Model.
- **Svelte:** Svelte Component, Svelte Store, Svelte Component Test, Component Stylesheet, Storybook Story, Barrel Index, Component Types.
- **Node.js:** CommonJS Module, ES Module, TypeScript Module, Module Test, CLI Entry Point, Barrel Index, Type Declarations.
- **Python:** Python Module, Pytest Test, Package Init, Python CLI, Python Types.
- **Java:** Java Class, Java Record, Java Interface, JUnit Test, Package Info.
- **Go:** Go Source File, Go Test, Go Main, Package Documentation, Go Interfaces.
- **Rust:** Rust Module, Rust Integration Test, Module Index, Crate Root, Error Type.
- **Generic:** JavaScript Module, Module Test, Stylesheet, Barrel Index, Type Declarations, Markdown Document, Documentation Page, JSON File, YAML File, Text File.

Choosing the preset

`auricFiles.create.preset` is `auto` by default: Auric Files reads the workspace's dependencies and configuration files and picks the best match. Set it to `react`, `vue`, `angular`, `svelte`, `node`, `python`, `java`, `go`, `rust` or `generic` to fix it.

Three settings shape what the presets generate:

Setting	Default	Values
`auricFiles.create.componentStyleLanguage`	`css`	`css`, `scss`, `less`, `styled`, `module-css`, `module-scss`
`auricFiles.create.testFramework`	`auto`	`auto`, `jest`, `vitest`, `mocha`, `node`, `pytest`, `junit`, `gotest`, `cargo`
`auricFiles.create.indent`	`space2`	`space2`, `space4`, `tab`

Java and Kotlin templates derive the package from the folder, recognising source roots such as `src/main/java`.

Write your own templates

Where templates live

Scope	Where	Setting
Workspace	`.auric/templates` and `.vscode/templates` in each workspace folder	`auricFiles.templates.directories`
Global	Absolute folders you choose, shared across all workspaces	`auricFiles.templates.global`
Generator	Defined in settings	`auricFiles.create.generatorProfiles`
Built-in	The active preset	`auricFiles.create.preset`

Every file in a template folder is read as one template. When two templates share an id, a workspace template replaces a global one, which replaces a generator, which replaces a built-in template. So you can override a built-in template by giving yours the same id.

Template folders are workspace configuration, so they only apply in a trusted workspace.

The file format

A template file has optional *front matter* between two `---` lines, followed by the body:

```
---
id: react-card
label: Card component
description: A card with an optional stylesheet
filename: "{{name|pascal}}/{{name|pascal}}.tsx"
variables: [{"name": "withStyles", "type": "boolean", "default": true, "label": "Include styles"}]
companions: [{"when": "{{withStyles}}", "filename": "{{name|pascal}}/{{name|pascal}}.css", "body": ".{{name|kebab}} {}\n"}]
---
export function {{name|pascal}}() {
  return <div className="{{name|kebab}}">$CURSOR</div>;
}
```

Key	Required	Meaning
`id`	Defaults to the file name	Letters and numbers, with `.`, `_` or `-` between them
`label`	Defaults from the id	Name shown in the template list
`description`	No	Extra text in the list
`filename`	Defaults to `{{name}}`	Output path, with placeholders; may include folders
`variables`	No	Questions to ask, as inline JSON (see below)
`companions`	No	Extra files, as inline JSON: `when`, `filename`, `body`
`match`	No	Globs that narrow where the template is offered
`preset`	No	The ecosystem the template belongs to

Front matter lines are `key: value`. A key with a list below it uses `- item` lines. Anything nested (`variables`, `companions`) is written as inline JSON on one line. The whole front matter can also be a single JSON object.

Variables you define

Each variable has a `name` and optionally:

Field	Meaning
`label`	The question shown
`description`	Extra help
`type`	`string` (default), `boolean`, `number` or `choice`
`choices`	The options for a `choice` variable
`default`	The pre-filled answer
`required`	`true` to insist on an answer

`auricFiles.templates.variables` supplies values to every template, such as an author or a licence header. An answer you type wins over it.

Built-in variables

Variable	Value for `src/components/Button.test.tsx`
`name`, `basename`	`Button` (compound extensions such as `.test.tsx` removed)
`fileName`	`Button.test.tsx`
`stem`	the file name without its last extension
`ext`	the extension
`dir`	`src/components`
`parentDir`	`components`
`relativePath`	`src/components/Button.test.tsx`
`absolutePath`	the full path on disk
`workspace`	the workspace name
`date`, `time`, `year`, `isoDate`	the current local date, time, year and ISO timestamp
`uuid`	a new random identifier
`className`, `pascalName`	`Button`
`camelName`, `kebabName`, `snakeName`, `upperName`	`button`, `button`, `button`, `BUTTON`
`indent`	the indentation from `auricFiles.create.indent`
`javaPackage`, `pythonPackage`	the package derived from the folder
`goPackage`	the package name from the parent folder, or `main`

Template language

Syntax	Meaning
`{{name}}`	Insert a value; an unknown name inserts nothing
`{{#if withStyles}} … {{else}} … {{/if}}`	Include text when a value is true (non-empty, not `false`, not `0`)

Syntax	Meaning
`{{#unless flag}} … {{/unless}}`	Include text when a value is false
`{{#each items}} … {{/each}}`	Repeat for each item; inside, use `{{this}}`, `{{@index}}` and `{{@key}}`
`{{> other-template-id}}`	Insert another template (a partial), up to 8 levels deep
`$CURSOR`	Where the cursor goes when the file opens

To apply a *filter*, write a vertical bar and the filter name after the value inside the braces: `{{name|pascal}}` turns `my button` into `MyButton`. Filters can be chained, as in `{{name|kebab|upper}}`.

`{{#each}}` also accepts a comma-separated string.

Filters: `upper`, `lower`, `kebab`, `camel`, `pascal`, `snake`, `title`, `trim`, `basename`, `dirname`, `stem`, `ext`, `plural`, `singular`, `date`, `isoDate`, `year`. An unknown filter does nothing, and template validation reports it.

The language is deliberately small: there is no code execution. A broken template renders nothing rather than running anything.

Generator profiles

A generator is a template defined in settings, shown beside the framework presets. Add them to `auricFiles.create.generatorProfiles`:

```
{
  "auricFiles.create.generatorProfiles": [
    {
      "id": "repository",
      "label": "Repository class",
      "description": "Data access class with its test",
      "match": ["src/**"],
      "filename": "src/data/{{name|pascal}}Repository.ts",
      "body": "export class {{name|pascal}}Repository {\n  $CURSOR\n}\n",
      "variables": [{ "name": "name", "label": "Entity name", "required": true }],
      "companions": [
        {
          "filename": "src/data/{{name|pascal}}Repository.test.ts",
          "body": "import { {{name|pascal}}Repository } from './{{name|pascal}}Repository';\n"
        }
      ]
    }
  ]
}
```

Required fields are `id`, `label`, `filename` and `body`. A companion is created only when its `when` expression renders to something true; leave `when` out to always create it.

When a template runs, the files it describes appear in the Explorer, and the first opens with the cursor at `$CURSOR`.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Reference: commands, keys and settings

Every Auric Artisan File System command grouped as the product groups them, every keyboard shortcut, menu and setting, with types, defaults and ranges.

This page covers Auric Artisan File System 0.1.0: 81 commands, 69 keyboard shortcut declarations, one Explorer view and 48 settings. Commands appear in the Command Palette with the **Auric Files:** prefix.

Commands

Commands are grouped as the workspace panel groups them. The **Keys** column gives Windows and Linux keys. **(Explorer)** keys work whenever the Explorer has focus. **(deck)** keys need `auricFiles.keyboard.enableDefaultShortcuts`. On macOS, `Cmd` replaces `Ctrl` in every shortcut, except **Redo File Operation**, which is `Cmd` + `Alt` + `Shift` + `Z`. "Then" means press the first combination, release, then press the second.

Commands marked "not in the Command Palette" act on a selection and run from the panel, context menus or shortcuts.

Create

Files, folders and complete structures.

Command	ID	Keys	What it does
Auric Files: New File...	`auricFiles.newFile`	`Ctrl` + `N` (Explorer); `Ctrl` + `Alt` + `A`, then `Ctrl` + `N` (deck)	Create a file: choose the folder, type a name (a trailing / makes a folder), optionally pick a template
Auric Files: New Folder...	`auricFiles.newFolder`	`Ctrl` + `Shift` + `N` (Explorer); `Ctrl` + `Alt` + `A`, then `Ctrl` + `Shift` + `N` (deck)	Create a folder, including nested paths
Auric Files: New File from Template...	`auricFiles.newFromTemplate`	`Ctrl` + `Alt` + `A`, then `T` (deck)	Create a file from a built-in, workspace or global template, filling in its variables
Auric Files: New from Framework Preset...	`auricFiles.newFromPreset`	`Ctrl` + `Alt` + `A`, then `P` (deck)	Create files from a framework preset (React, Vue, Angular and more) or a custom generator
Auric Files: New Sibling File	`auricFiles.newSibling`	`Ctrl` + `Alt` + `N` (deck); `Ctrl` + `Alt` + `A`, then `N` (deck)	Create a file next to the selected file
Auric Files: New Child File	`auricFiles.newChild`	`Ctrl` + `Alt` + `A`, then `Shift` + `N` (deck)	Create a file inside the selected folder
Auric Files: Create Related Files...	`auricFiles.newRelated`	`Ctrl` + `Alt` + `A`, then `R` (deck)	Create companion files (test, style, story and so on) for the selected file
Auric Files: Batch Structure...	`auricFiles.batchCreate`	`Ctrl` + `Alt` + `B` (Explorer); `Ctrl` + `Alt` + `A`, then `B` (deck)	Type a whole structure of files and folders in one line; braces and ranges expand
Auric Files: Batch Create Files...	`auricFiles.batchCreateFiles`	`Ctrl` + `Alt` + `Shift` + `F` (deck); `Ctrl` + `Alt` + `A`, then `F` (deck)	Create many files at once from comma-separated paths with brace expansion
Auric Files: Batch Create Folders...	`auricFiles.batchCreateFolders`	`Ctrl` + `Alt` + `Shift` + `N` (deck); `Ctrl` + `Alt` + `A`, then `Shift` + `F` (deck)	Create many folders at once from comma-separated paths with brace expansion

Inspect & navigate

Properties, comparisons and companion files.

Command	ID	Keys	What it does
Auric Files: File Properties	`auricFiles.showMetadata`	`Alt` + `Enter` (Explorer)	Open Properties for the selected resource
Auric Files: Quick Open in Auric Files...	`auricFiles.quickOpen`	`Ctrl` + `Alt` + `O` (deck)	Focus the Explorer and search indexed files by name or path
Auric Files: Compare Selected Files	`auricFiles.compareSelected`	`Ctrl` + `Alt` + `Shift` + `D` (deck); `Ctrl` + `Alt` + `A`, then `Shift` + `C` (deck)	Open two selected files in VS Code's diff editor
Auric Files: Merge Selected Files	`auricFiles.mergeSelected`	`Ctrl` + `Alt` + `A`, then `Shift` + `G` (deck)	Open two files, or a base plus two inputs, in VS Code's merge editor
Auric Files: Export Selected Items...	`auricFiles.exportSelection`	`Ctrl` + `Alt` + `A`, then `Shift` + `X` (deck)	Save a JSON manifest of the selected items' paths and metadata
Auric Files: Go to Related File...	`auricFiles.goToRelated`	`Ctrl` + `Alt` + `R` (deck); `Ctrl` + `Alt` + `A`, then `O` (deck)	Pick and open any companion of the current file
Auric Files: Open Matching Test	`auricFiles.goToTest`	`Ctrl` + `Alt` + `A`, then `Q` (deck)	Open the current file's test
Auric Files: Open Matching Style	`auricFiles.goToStyle`	`Ctrl` + `Alt` + `A`, then `S` (deck)	Open the current file's stylesheet
Auric Files: Open Matching Story	`auricFiles.goToStory`	`Ctrl` + `Alt` + `A`, then `W` (deck)	Open the current file's story
Auric Files: Open Matching Component	`auricFiles.goToComponent`	`Ctrl` + `Alt` + `A`, then `I` (deck)	Open the component a test, style or story belongs to
Auric Files: Open Matching Documentation	`auricFiles.goToDocs`	`Ctrl` + `Alt` + `A`, then `Shift` + `D` (deck)	Open the current file's documentation
Auric Files: Show Related File Group	`auricFiles.showRelatedGroup`	—	List the file's related group, including missing companions; not in the Command Palette
Auric Files: Generate Missing Companion Files	`auricFiles.generateMissingRelated`	—	Create the companion files that do not exist yet
Auric Files: Reveal in File Manager	`auricFiles.revealInExplorer`	`Ctrl` + `Alt` + `A`, then `V` (deck)	Show the file in your operating system's file manager
Auric Files: Reveal in VS Code Explorer	`auricFiles.revealInVsCodeExplorer`	`Ctrl` + `Alt` + `A`, then `Shift` + `V` (deck)	Select the file in the VS Code Explorer
Auric Files: Copy Relative Path	`auricFiles.copyRelativePath`	`Ctrl` + `Alt` + `Shift` + `C` (deck); `Ctrl` + `Alt` + `A`, then `L` (deck)	Copy the workspace-relative path
Auric Files: Copy Absolute Path	`auricFiles.copyAbsolutePath`	`Ctrl` + `Alt` + `Shift` + `A` (deck); `Ctrl` + `Alt` + `A`, then `A` (deck)	Copy the full path
Auric Files: Copy File URI	`auricFiles.copyUri`	`Ctrl` + `Alt` + `A`, then `U` (deck)	Copy the file's URI

Edit & batch

Rename, duplicate and transform a selection.

Command	ID	Keys	What it does
Auric Files: Rename...	`auricFiles.rename`	—	Rename the selected item with collision checks and history; not in the Command Palette
Auric Files: Move...	`auricFiles.move`	—	Move the selected items to another folder; not in the Command Palette
Auric Files: Copy To...	`auricFiles.copyTo`	`Ctrl` + `Alt` + `A`, then `C` (deck)	Copy the selected items to another folder; not in the Command Palette
Auric Files: Duplicate	`auricFiles.duplicate`	—	Make a copy beside the original with a non-clashing name; not in the Command Palette
Auric Files: Batch Rename...	`auricFiles.batchRename`	`Ctrl` + `Alt` + `A`, then `Shift` + `R` (deck)	Rename many files by find and replace, regular expression, pattern, sequence, case or extension, with a preview
Auric Files: Batch Move...	`auricFiles.batchMove`	`Ctrl` + `Alt` + `A`, then `Shift` + `M` (deck)	Move many items to one folder, with collision planning
Auric Files: Convert Extension...	`auricFiles.convertExtension`	`Ctrl` + `Alt` + `A`, then `X` (deck)	Change the extension of the selected files, with a preview

Organise

Tags, groups and personal workspace views.

Command	ID	Keys	What it does
Auric Files: Tag Files...	`auricFiles.tagFiles`	`Ctrl` + `Alt` + `A`, then `G` (deck)	Assign or remove tags on the selected items; not in the Command Palette
Auric Files: Manage Tags...	`auricFiles.manageTags`	—	Create, rename or delete tags
Auric Files: Add to Group...	`auricFiles.addToCollection`	`Ctrl` + `Alt` + `A`, then `K` (deck)	Add the selected items to a group, or create a new one; not in the Command Palette
Auric Files: Create Workspace from Selection	`auricFiles.createWorkspace`	—	Create a group (virtual workspace) from the current selection
Auric Files: Manage Groups...	`auricFiles.manageCollections`	—	Rename, delete or open members of groups and virtual folders
Auric Files: New Virtual Folder...	`auricFiles.newVirtualFolder`	—	Create a virtual folder that gathers items without moving them; not in the Command Palette
Auric Files: Toggle Favorite	`auricFiles.toggleFavorite`	`Ctrl` + `Alt` + `A`, then `1` (deck)	Mark or unmark as a favourite (★ in the Explorer); not in the Command Palette
Auric Files: Toggle Pinned	`auricFiles.togglePinned`	`Ctrl` + `Alt` + `A`, then `2` (deck)	Pin or unpin (P in the Explorer); not in the Command Palette
Auric Files: Toggle Native Explorer Exclusion	`auricFiles.toggleHidden`	`Ctrl` + `Alt` + `A`, then `3` (deck)	Add or remove the item from VS Code's files.exclude for this workspace; not in the Command Palette
Auric Files: Toggle Highlight	`auricFiles.toggleHighlight`	`Ctrl` + `Alt` + `A`, then `4` (deck)	Highlight or unhighlight (● in the Explorer); not in the Command Palette
Auric Files: Toggle Focus Mode	`auricFiles.toggleFocusMode`	—	Limit Auric's indexed file lists to chosen paths, or switch focus mode off

Command	ID	Keys	What it does
Auric Files: Switch Workspace Profile...	`auricFiles.switchProfile`	—	Apply a saved workspace profile
Auric Files: Save Workspace Profile...	`auricFiles.saveProfile`	—	Save the current view preferences as a named profile

Find & filter

Search, duplicates and a quieter Explorer.

Command	ID	Keys	What it does
Auric Files: Search Files...	`auricFiles.search`	—	Search indexed files by name or path and open one
Auric Files: Apply Filter...	`auricFiles.applyFilter`	—	Build a filter from text, kinds, tags, extensions, globs, size, dates, flags and Git state
Auric Files: Save Current Filter...	`auricFiles.saveFilter`	—	Save the current filter under a name
Auric Files: Clear Filter	`auricFiles.clearFilter`	—	Remove the active filter
Auric Files: Find Duplicate Files	`auricFiles.findDuplicates`	—	Find duplicate files and choose copies to move to the workspace trash
Auric Files: Find Empty Files	`auricFiles.findEmptyFiles`	—	Find empty files and folders and choose which to move to the workspace trash
Auric Files: Sort	`auricFiles.setSort`	`Alt` + `S` (Explorer)	Cycle the native Explorer sort between name, type and date modified
Auric Files: Sort by Name	`auricFiles.sortName`	—	Sort the native Explorer by name
Auric Files: Sort by Type	`auricFiles.sortType`	—	Sort the native Explorer by type
Auric Files: Sort by Date Modified	`auricFiles.sortModified`	—	Sort the native Explorer by date modified
Auric Files: Sort Ascending	`auricFiles.sortAscending`	—	Sort the native Explorer ascending
Auric Files: Sort Descending	`auricFiles.sortDescending`	—	Sort the native Explorer descending
Auric Files: Group Indexed Files...	`auricFiles.setGrouping`	—	Choose the headings used by the indexed file list; not in the Command Palette
Auric Files: Collapse All	`auricFiles.collapseAll`	—	Collapse every folder in the Explorer
Auric Files: Expand All	`auricFiles.expandAll`	—	Expand the Explorer tree

History & recovery

Review operations and recover workspace trash.

Command	ID	Keys	What it does
Auric Files: Undo Last File Operation	`auricFiles.undo`	`Ctrl` + `Alt` + `Z` (deck); `Ctrl` + `Alt` + `A`, then `Z` (deck)	Undo the last Auric file operation
Auric Files: Redo File Operation	`auricFiles.redo`	`Ctrl` + `Alt` + `Y` (deck); `Ctrl` + `Alt` + `A`, then `Y` (deck)	Redo an undone Auric file operation

Command	ID	Keys	What it does
Auric Files: Show Operation History	`auricFiles.showHistory`	`Ctrl` + `Alt` + `A`, then `H` (deck)	List recent Auric operations and undo back to one
Auric Files: Open Workspace Trash	`auricFiles.openTrash`	`Ctrl` + `Alt` + `A`, then `Shift` + `T` (deck)	Browse the workspace trash; restore an entry or empty it
Auric Files: Restore from Trash...	`auricFiles.restoreFromTrash`	`Ctrl` + `Alt` + `A`, then `Shift` + `U` (deck)	Restore an entry from the workspace trash to its original place
Auric Files: Delete (Workspace Trash)	`auricFiles.delete`	`Ctrl` + `Alt` + `Shift` + `Backspace` (deck); `Ctrl` + `Alt` + `A`, then `Backspace` (deck)	Move the selected items to the workspace trash; not in the Command Palette
Auric Files: Delete Permanently	`auricFiles.deletePermanent`	`Ctrl` + `Alt` + `A`, then `Shift` + `Backspace` (deck)	Delete the selected items permanently, after confirmation; not in the Command Palette
Auric Files: Empty Workspace Trash	`auricFiles.emptyTrash`	`Ctrl` + `Alt` + `A`, then `Shift` + `Delete` (deck)	Permanently delete everything in the workspace trash

Workspace

Native Explorer and extension preferences.

Command	ID	Keys	What it does
Auric Files: Focus VS Code Explorer	`auricFiles.focus`	`Ctrl` + `Alt` + `E` (deck)	Focus the native VS Code Explorer
Auric Files: Refresh	`auricFiles.refresh`	—	Rebuild the workspace index
Auric Files: Open Settings	`auricFiles.openSettings`	`Ctrl` + `Alt` + `A`, then `Shift` + `S` (deck)	Open the Settings editor filtered to Auric Files
Auric Files: Show Log	`auricFiles.showLog`	—	Show the Auric Artisan File System output channel

Editor and global commands

Commands outside the workspace groups.

Command	ID	Keys	What it does
Auric Files: Rename Current File	`auricFiles.renameCurrent`	`Ctrl` + `Alt` + `Shift` + `R` (deck); `Ctrl` + `Alt` + `A`, then `E` (deck)	Rename the file open in the editor
Auric Files: Move Current File	`auricFiles.moveCurrent`	`Ctrl` + `Alt` + `Shift` + `M` (deck); `Ctrl` + `Alt` + `A`, then `M` (deck)	Move the file open in the editor
Auric Files: Duplicate Current File	`auricFiles.duplicateCurrent`	`Ctrl` + `Alt` + `D` (deck); `Ctrl` + `Alt` + `A`, then `D` (deck)	Duplicate the file open in the editor
Auric Files: Toggle File Metadata	`auricFiles.toggleMetadata`	`Ctrl` + `Shift` + `M` (Explorer)	Switch the compact Explorer hover summaries on or off
Auric Files: Open File Workspace	`auricFiles.openMenu`	`Ctrl` + `Alt` + `Shift` + `E` (deck); `Ctrl` + `Alt` + `A`, then `Shift` + `E` (deck)	Open the Auric Files workspace in an editor tab
Auric Files: Quick Actions...	`auricFiles.quickActions`	—	Open a keyboard-first list of file-system actions

Where commands appear

- Explorer title bar: **Batch Structure...**, the **Sort** menu, **Open File Workspace**; **...** menu: **Quick Actions...**
- Explorer context menu: **File Properties** at the top; the **Auric Files** submenu
- Editor and editor tab context menus: The **Auric Files** submenu (not for untitled files)
- **Auric Files** submenu: **File Properties**, **Create**, **Edit and Batch**, **Navigate and Paths**, **Organize**, **Safety and History**, **Sort**, **Open File Workspace**, **Open Settings**
- **Create**: **New File...**, **New Folder...**, **New Sibling File** (files) or **New Child File** (folders), **Batch Create Files...**, **Batch Create Folders...**, **Batch Structure...**, **New File from Template...**, **New from Framework Preset...**, **Create Related Files...** (files)
- **Edit and Batch**: **Rename Current File**, **Duplicate**, **Move Current File**, **Copy To...**, **Batch Rename...**, **Batch Move...**, **Convert Extension...**
- **Navigate and Paths**: **File Properties**, **Compare Selected Files**, **Merge Selected Files**, **Export Selected Items...**, **Go to Related File...**, **Open Matching Test**, **Style**, **Story**, **Component** and **Documentation**, **Reveal in VS Code Explorer**, **Reveal in File Manager**, **Copy Relative Path**, **Copy Absolute Path**, **Copy File URI**
- **Organize**: **Create Workspace from Selection**, **Tag Files...**, **Add to Group...**, **Toggle Favorite**, **Toggle Pinned**, **Toggle Native Explorer Exclusion**, **Toggle Highlight**
- **Safety and History**: **Delete (Workspace Trash)**, **Delete Permanently**, **Undo Last File Operation**, **Redo File Operation**, **Show Operation History**, **Open Workspace Trash**, **Restore from Trash...**, **Empty Workspace Trash**
- **Sort**: **Sort by Name**, **Sort by Type**, **Sort by Date Modified**, **Sort Ascending**, **Sort Descending**
- **Auric Files** view title: **Open File Workspace**

Views and panels

Surface	Where	Contents
Auric Files	A section of the Explorer	The compact workspace: summary, selection, **01 Actions**, **02 Properties**
Auric Files panel	An editor tab	The same workspace at full size

Output channel: **Auric Artisan File System**.

Explorer badges

Badge	Meaning
•	Highlighted
★	Favourite
P	Pinned
H	Hidden (excluded from the native Explorer)
Two letters	The first two letters of the item's first tag
W	A member of a group

When an item has several marks, the first in this list is shown.

Settings

All 48 settings, grouped by area. They can be set in user or workspace settings.

Explorer, sorting and indexing

Setting	Type	Default	Values or range	What it does
`auricFiles.explorer.sort`	string	`name`	`name`, `path`, `type`, `size`, `modified`, `created`, `extension`, `status`, `custom`	Primary order for Auric's indexed pickers. Orders VS Code supports are also applied to its Explorer.
`auricFiles.explorer.sortDirection`	string	`asc`	`asc`, `desc`	Direction the chosen sort runs in — ascending puts A, the smallest and the oldest first.
`auricFiles.explorer.secondarySort`	string	`none`	`none`, `name`, `path`, `type`, `size`, `modified`, `created`, `extension`, `status`, `custom`	Optional second sort key used when two entries have the same primary value.
`auricFiles.explorer.secondarySortDirection`	string	`asc`	`asc`, `desc`	Direction for the secondary sort key.
`auricFiles.explorer.grouping`	string	`none`	`none`, `type`, `extension`, `folder`, `tag`, `modified`, `status`	Headings used by Auric's indexed grouped-file picker.
`auricFiles.explorer.foldersFirst`	boolean	`true`		Keep directories above files no matter which sort is active.
`auricFiles.explorer.showHiddenFiles`	boolean	`false`		Include dot-files and metadata-hidden entries in Auric search and filter results.
`auricFiles.explorer.showFileSize`	boolean	`false`		Show a size beside each file in Auric's file lists. Costs one file check per entry on large trees.
`auricFiles.explorer.showGitStatus`	boolean	`true`		Read Git state for filtering and Git-aware safety warnings.
`auricFiles.explorer.previewOnSingleClick`	boolean	`true`		Open a single click in a reusable preview tab. Off, a double click is needed to open anything.
`auricFiles.explorer.maxNodes`	number	`20000`	500–200000	Most entries held in one scan. Reaching it stops the scan and marks the index as truncated.
`auricFiles.explorer.excludePatterns`	array	see "Long default values"		Globs skipped while scanning. Matches are never indexed, searched, compared or batch-processed.
`auricFiles.explorer.respectGitignore`	boolean	`true`		Read the workspace .gitignore and skip whatever it excludes, on top of the patterns above.

Creating files

Setting	Type	Default	Values or range	What it does
`auricFiles.create.defaultTemplate`	string	empty		Template id applied by New File when you do not pick one. Leave empty to create a blank file.
`auricFiles.create.openAfterCreate`	boolean	`true`		Open a new file in the editor, with the cursor at the template's cursor marker.
`auricFiles.create.createIntermediateDirs`	boolean	`true`		Create missing folders in a path such as `src/lib/util/index.ts` instead of refusing.
`auricFiles.create.expandBraces`	boolean	`true`		Treat braces and ranges as several targets, so `src/{Button,Card}/index.ts` makes two files.
`auricFiles.create.preset`	string	`auto`	`auto`, `react`, `vue`, `angular`, `svelte`, `node`, `python`, `java`, `go`, `rust`, `generic`	Which framework's templates and relation rules are offered. `auto` picks from the workspace's files.
`auricFiles.create.componentStyleLanguage`	string	`css`	`css`, `scss`, `less`, `styled`, `module-css`, `module-scss`	Stylesheet flavour for a component's companion style file, and the extension relation rules look for.
`auricFiles.create.testFramework`	string	`auto`	`auto`, `jest`, `vitest`, `mocha`, `node`, `pytest`, `junit`, `gotest`, `cargo`	Test style and imports for generated test files. `auto` reads the workspace's dependencies and scripts.
`auricFiles.create.indent`	string	`space2`	`space2`, `space4`, `tab`	Indentation written into generated file bodies.
`auricFiles.create.generatorProfiles`	array	`[]`		Custom generators shown beside the framework presets. Each can create any number of companion files.

Safety, trash and history

Setting	Type	Default	Values or range	What it does
`auricFiles.safety.confirmDelete`	boolean	`true`		Ask before anything is deleted, listing what goes and where it lands.
`auricFiles.safety.confirmBulk`	number	`5`	0–no maximum	Ask for confirmation once an operation touches this many paths. `0` asks every time.
`auricFiles.safety.useWorkspaceTrash`	boolean	`true`		Move deleted items to `.auric/trash` in the workspace so they can be restored.
`auricFiles.safety.trashRetentionDays`	number	`14`	1–365	How long a trashed entry survives before it is purged automatically.
`auricFiles.safety.trashMaxBytes`	number	`268435456`	0–no maximum	Size limit for the workspace trash in bytes (256 MiB). The oldest entries are purged first.
`auricFiles.safety.protectedPaths`	array	see "Long default values"		Globs no Auric operation may rename, move or delete. A plan touching one is blocked before any change.
`auricFiles.safety.warnOnGitTracked`	boolean	`true`		Warn when a file about to be deleted or moved is tracked by Git.

Setting	Type	Default	Values or range	What it does
`auricFiles.safety.warnOnUncommittedChanges`	boolean	`true`		Warn when a file about to be deleted or moved has uncommitted edits.
`auricFiles.safety.previewBeforeApply`	boolean	`true`		Show the full before-and-after list of a batch rename or move, with conflicts, and require approval.
`auricFiles.safety.historyLimit`	number	`200`	10–2000	How many past operations stay undoable. Older entries drop off the end of the stack.

Related files

Setting	Type	Default	Values or range	What it does
`auricFiles.related.enabled`	boolean	`true`		Resolve companions for the active file: test, style, story, component, docs and types.
`auricFiles.related.rules`	array	`[]`		Extra file relationships for conventions the built-in presets do not know.
`auricFiles.related.testDirectories`	array	`["__tests__","tests","test","spec"]`		Folder names searched when tests live in a separate folder rather than beside each file.
`auricFiles.related.storyExtensions`	array	see "Long default values"		Suffixes recognised as a component's story file, in the order they are tried.

Templates

Setting	Type	Default	Values or range	What it does
`auricFiles.templates.directories`	array	`[".auric/templates",".vscode/templates"]`		Workspace folders scanned for templates. A workspace template replaces a built-in one with the same id.
`auricFiles.templates.global`	array	`[]`		Absolute folders of templates shared across every workspace, for conventions you carry between projects.
`auricFiles.templates.variables`	object	`{}`		Values available in every template, such as an author or licence header. Prompted values win.

Search and duplicates

Setting	Type	Default	Values or range	What it does
`auricFiles.search.defaultMode`	string	`glob`	`glob`, `regex`, `fuzzy`	How a filter's search text is read: `glob` wildcards, `regex` regular expression, or `fuzzy` matching.

Setting	Type	Default	Values or range	What it does
`auricFiles.search.caseSensitive`	boolean	`false`		Match letter case exactly. Off, a search for button also finds Button.tsx.
`auricFiles.search.maxResults`	number	`2000`	50–50000	Matches kept by one search before it stops and reports a partial result.
`auricFiles.search.duplicateStrategy`	string	`content`	`content`, `name`, `size`	What counts as a duplicate: `content` (same bytes), `name` (same file name) or `size` (same length).

Performance

Setting	Type	Default	Values or range	What it does
`auricFiles.performance.watchWorkspace`	boolean	`true`		After the first indexed command, follow file changes and update the index. Turn off on slow network shares.
`auricFiles.performance.scanConcurrency`	number	`8`	1–64	Folders read at once while indexing. Raise on a fast local disk, lower on a remote file system.
`auricFiles.performance.debounceMs`	number	`120`	0–2000	Quiet period after the last file change before the index updates.

Keyboard and logging

Setting	Type	Default	Values or range	What it does
`auricFiles.keyboard.enableDefaultShortcuts`	boolean	`false`		Turn on the full shortcut deck of `Ctrl` + `Alt` (`Cmd` + `Alt`) combinations. Off by default.
`auricFiles.logLevel`	string	`warn`	`off`, `error`, `warn`, `info`, `debug`	Detail written to the output channel. Paths are logged workspace-relative; file contents never.

Long default values

`auricFiles.explorer.excludePatterns`:

```
["**/node_modules/**", "**/.git/**", "**/dist/**", "**/out/**", "**/.next/**", "**/__pycache__/**", "**/target/**",
"**/vendor/**"]
```

`auricFiles.safety.protectedPaths`:

```
["**/.git/**", "**/node_modules/**", "**/.env", "**/.env.*", "**/*.pem", "**/*.key", "**/package-lock.json", "**/yarn.lock",
"**/pnpm-lock.yaml"]
```

`auricFiles.related.storyExtensions`:

```
[".stories.tsx", ".stories.jsx", ".stories.ts", ".stories.js", ".stories.svelte", ".stories.mdx"]
```

Enumerated values explained

Setting	Value	Meaning
`auricFiles.explorer.sort`	`name`	Alphabetical, natural order, so `file10` follows `file9`
`auricFiles.explorer.sort`	`path`	By the full workspace-relative path
`auricFiles.explorer.sort`	`type`	Folders, files and links in that order, then by name
`auricFiles.explorer.sort`	`size`	Largest first when descending; folders keep name order
`auricFiles.explorer.sort`	`modified`	Most recently written first when descending
`auricFiles.explorer.sort`	`created`	Creation time, where recorded
`auricFiles.explorer.sort`	`extension`	By extension, then by name
`auricFiles.explorer.sort`	`status`	Conflicts and changed Git files first, then clean and ignored
`auricFiles.explorer.sort`	`custom`	A stored custom rank; unranked items follow by name
`auricFiles.explorer.grouping`	`none`	One continuous list
`auricFiles.explorer.grouping`	`type`	Folders, files and links under separate headings
`auricFiles.explorer.grouping`	`extension`	One heading per extension
`auricFiles.explorer.grouping`	`folder`	One heading per parent folder
`auricFiles.explorer.grouping`	`tag`	One heading per tag; untagged last
`auricFiles.explorer.grouping`	`modified`	Today, this week, this month and older
`auricFiles.explorer.grouping`	`status`	Changed, untracked, conflicted, clean and ignored
`auricFiles.search.defaultMode`	`glob`	Wildcards over the path
`auricFiles.search.defaultMode`	`regex`	A regular expression tested against the path
`auricFiles.search.defaultMode`	`fuzzy`	Loose matching ranking the closest paths first
`auricFiles.search.duplicateStrategy`	`content`	Same bytes; files are bucketed by size first
`auricFiles.search.duplicateStrategy`	`name`	Same file name in different folders
`auricFiles.search.duplicateStrategy`	`size`	Same length; fastest and loosest
`auricFiles.logLevel`	`off`, `error`, `warn`, `info`, `debug`	Nothing, failures, skips too, one line per operation, per-step detail

Settings restricted in untrusted workspaces

Workspace values of these settings are ignored until you trust the folder: `auricFiles.safety.confirmDelete`, `auricFiles.safety.confirmBulk`, `auricFiles.safety.useWorkspaceTrash`, `auricFiles.safety.protectedPaths`, `auricFiles.safety.warnOnGitTracked`, `auricFiles.safety.warnOnUncommittedChanges`, `auricFiles.safety.previewBeforeApply`, `auricFiles.related.rules`, `auricFiles.templates.directories`, `auricFiles.templates.global` and `auricFiles.templates.variables`.

Files and settings Auric Files writes

What	Where	Written by
Workspace trash	`.auric/trash/` in the workspace folder, with `index.json`	**Delete (Workspace Trash)**
Workspace templates	`.auric/templates/`, `.vscode/templates/` (read, not written)	You
Selection manifest	Where you choose; default `auric-selection.json`	**Export Selected Items...**
`files.exclude`	Workspace or folder settings	**Toggle Native Explorer Exclusion**
`explorer.sortOrder` and its direction	Workspace settings	The **Sort** commands
Tags, groups, virtual folders, flags, filters, profiles, history	VS Code's storage for this workspace	The **Organise**, **Find & filter** and history commands

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Troubleshooting

Symptoms, causes and fixes for common Auric Artisan File System problems.

For anything not listed, set `auricFiles.logLevel` to `debug`, repeat the action, run **Auric Files: Show Log**, and see Get support.

Actions do nothing or are refused

Create, rename, move and delete are unavailable

- **Why:** the workspace is not trusted. In Restricted Mode, browsing, search, filtering and related-file navigation work, but nothing that changes files.
- **Fix:** run **Workspaces: Manage Workspace Trust** and trust the folder.

"Choose a resource inside the open workspace first."

- **Why:** the action needs a file or folder as its target, and none is selected, or the selected item is outside the workspace.
- **Fix:** select an item in the Explorer or open a file, or use **Choose...** in the panel.

"Choose a file or subfolder for this action. Use the native Explorer to create directly in the workspace root."

- **Why:** the panel's target actions need a file or subfolder; the workspace root itself is not a valid target for them.
- **Fix:** use VS Code's own **New File** or **New Folder** for the root, or choose a subfolder.

"This resource is outside the workspace."

- **Why:** you are inspecting a file that is not in any open workspace folder.
- **Fix:** Properties still work. Open the file's folder as a workspace to use file actions on it.

An operation is blocked

- **Why:** it touches a path in `auricFiles.safety.protectedPaths` (for example `.env`, lockfiles or `node_modules`).
- **Fix:** do it with VS Code's own Explorer if you really mean it, or remove the pattern from the setting.

Properties opens a picker instead of the file

- **Why:** some keyboard routes do not tell the extension which Explorer item is focused.
- **Fix:** pick the file in the list, or right-click the file and choose **File Properties**.

Shortcuts

Most shortcuts do nothing

- **Why:** the full shortcut deck is off by default, so it does not take over `Ctrl` + `Alt` combinations.
- **Fix:** turn on `auricFiles.keyboard.enableDefaultShortcuts`.

`Ctrl` + `N` in the Explorer opens Auric's New File

- **Why:** with the Explorer focused, `Ctrl` + `N`, `Ctrl` + `Shift` + `N`, `Alt` + `Enter`, `Ctrl` + `Shift` + `M`, `Alt` + `S` and `Ctrl` + `Alt` + `B` run Auric commands.
- **Fix:** remove those bindings in **Preferences: Open Keyboard Shortcuts** (search for `auricFiles`) if you prefer VS Code's defaults.

A deck shortcut conflicts with another extension

- **Fix:** open **Preferences: Open Keyboard Shortcuts**, search for the keys, and rebind one of them.

Creation

"expands to more than 1000 paths" or "Stopped after 2000 paths"

- **Why:** the brace or range expression produces too many paths.
- **Fix:** split it into smaller batches.

Braces were created literally

- **Why:** `auricFiles.create.expandBraces` is off, a brace was escaped with a backslash, or the braces are neither a list nor a range (such as `{png}`).
- **Fix:** turn the setting on and write a list (`{a,b}`) or a range (`{1..3}`).

A template does not appear in the list

- **Why:** the template file is not in a template folder, its id is invalid, it uses an unknown filter, a block is not closed, or the workspace is untrusted.
- **Fix:** check `auricFiles.templates.directories`, trust the workspace, and look in the log for **Ignored template** with the reason.

Generated files use the wrong framework

- **Why:** automatic preset detection picked a different framework.

- **Fix:** set `auricFiles.create.preset` explicitly.

Related files, search and Properties

"Not indexed yet" in Properties, or related files are missing

- **Why:** the workspace index has not been built, or the file is excluded from it.
- **Fix:** run **Auric Files: Refresh**. Check `auricFiles.explorer.excludePatterns` and whether `.gitignore` excludes the file (`auricFiles.explorer.respectGitignore`).

Search results stop early

- **Why:** a search keeps at most `auricFiles.search.maxResults` matches, and the index holds at most `auricFiles.explorer.maxNodes` entries.
- **Fix:** raise the limits, or narrow the search with a filter.

Git status looks out of date

- **Why:** Properties shows Git state from the last index and labels it so.
- **Fix:** run **Auric Files: Refresh**.

Owner or creation date says Unavailable

- **Why:** the file system does not report them through VS Code. Auric Files never invents a value.
- **Fix:** none; check in your operating system if you need them.

The index is slow on a network drive

- **Fix:** lower `auricFiles.performance.scanConcurrency`, turn off `auricFiles.performance.watchWorkspace`, and refresh by hand.

Duplicates, trash and history

Duplicate review compares by size

- **Why:** content hashing is not available in this environment (for example a virtual workspace).
- **Fix:** none needed; review the groups carefully before trashing.

A deleted file is not in the workspace trash

- **Why:** it was deleted with **Delete Permanently**, with VS Code's own Explorer, or while `auricFiles.safety.useWorkspaceTrash` was off; or it expired after `auricFiles.safety.trashRetentionDays`, or was purged to stay under `auricFiles.safety.trashMaxBytes`.
- **Fix:** restore from version control or a backup.

Undo did not reverse my change

- **Why:** the history only records Auric operations, not editor edits or native Explorer actions.
- **Fix:** use the editor's undo or VS Code's Explorer undo for those changes.

The merge editor does not open

- **Why:** your VS Code-based editor does not provide the merge editor, or you selected fewer than two or more than three files.
- **Fix:** select two files, or a base and two inputs, in VS Code itself.

Icon tools

"No compatible theme active"

- **Why:** the icon tools need the Auric Artisan File Icons **Facet** or **Playful** icon theme to be active.
- **Fix:** install Auric Artisan File Icons and choose one of those themes with **Preferences: File Icon Theme**, or ignore the section.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

FAQ

Short answers to common questions about Auric Artisan File System.

Does it replace the VS Code Explorer?

No. VS Code's Explorer stays as it is. Auric Files adds a companion section, a workspace panel, menus and commands around it.

Does it change my files when I tag or group them?

No. Tags, groups, virtual folders, favourites, pins, highlights, filters and profiles are stored in VS Code's storage for the workspace. Only file operations (create, rename, move, copy, duplicate, convert, delete) change files, and **Toggle Native Explorer Exclusion** changes the `files.exclude` setting.

Is anything uploaded?

No. Everything runs locally and no account is needed. Links to the Auric Artisan website and support open in your browser only when you choose them.

Can I undo a batch rename?

Yes. **Undo Last File Operation** reverses the whole batch in one step.

Where do deleted files go?

To `.auric/trash` in your workspace, unless you use **Delete Permanently** or turn off the workspace trash. Restore them with **Restore from Trash...** within the retention period (14 days by default).

Should I commit `.auric/`?

The trash lives in `.auric/trash`, and templates you share can live in `.auric/templates`. You may want to commit shared templates and keep the trash out of version control by adding `.auric/trash/` to `.gitignore`.

Do tags follow a file when I rename it?

Yes, when you rename or move it with Auric Files. Changes made outside Auric Files may leave organisation data pointing at the old path.

Does Export Selected Items zip my files?

No. It writes a JSON manifest of paths and metadata, not the files' contents.

Why is folder size shown as "Folder"?

Properties shows what the file system reports. A recursive disk-usage total is not calculated.

Does it work over Remote SSH, in WSL or in virtual workspaces?

It runs next to the workspace files. In virtual workspaces browsing, scaffolding, renaming and moving work, but revealing in the operating system's file manager, content hashing and the native trash do not. What is available depends on the file system provider.

Why do most shortcuts not work?

The full shortcut deck is off by default so it does not claim `Ctrl` + `Alt` combinations. Turn on `auricFiles.keyboard.enableDefaultShortcuts`.

Is the interface available in Hindi?

Not at present; the interface is in English.

Do I need the Auric icon themes?

No. The **Icon tools** section only works with the Auric Artisan File Icons Facet or Playful theme, which is an optional, separate extension.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Supporting material

Contents of the supporting pages for Auric Artisan File System.

These pages support the Auric Artisan File System guides: terms, shortcuts, limits, privacy, related products and the version history.

File	What it covers
glossary.md	Terms used in the File System documentation
shortcuts.md	Every keyboard shortcut, including the optional deck
limits-and-accuracy.md	What Properties can and cannot report, index and batch limits
privacy.md	What stays local and what uses the network
related.md	Related extensions and pages
release-notes.md	What changed in each version

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Shortcuts

Every keyboard shortcut Auric Artisan File System declares, the six that are always on and the optional deck.

Auric Files declares 69 shortcuts. Six are always on while the Explorer has focus. The other 63 form the optional *deck*, which is off by default so the extension does not claim `Ctrl` + `Alt` combinations across VS Code. Turn the deck on with:

```
{ "auricFiles.keyboard.enableDefaultShortcuts": true }
```

Deck shortcuts work when a folder is open; a few (such as **Rename Current File**) need an editor to be open. Every command is also available from menus and the Command Palette, and you can assign or change any shortcut in **Preferences: Open Keyboard Shortcuts** by searching for `auricFiles`.

Some commands have both a direct shortcut and a chord. A *chord* is two key presses in a row: the first combination, then a second key.

Always on (Explorer focused)

Windows and Linux	macOS	Command
`Ctrl` + `N`	`Cmd` + `N`	**New File...**
`Ctrl` + `Shift` + `N`	`Cmd` + `Shift` + `N`	**New Folder...**
`Alt` + `Enter`	`Alt` + `Enter`	**File Properties**
`Ctrl` + `Shift` + `M`	`Cmd` + `Shift` + `M`	**Toggle File Metadata**
`Alt` + `S`	`Alt` + `S`	**Sort**
`Ctrl` + `Alt` + `B`	`Cmd` + `Alt` + `B`	**Batch Structure...**

Deck: direct shortcuts

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `Shift` + `D`	`Cmd` + `Alt` + `Shift` + `D`	**Compare Selected Files**
`Ctrl` + `Alt` + `E`	`Cmd` + `Alt` + `E`	**Focus VS Code Explorer**
`Ctrl` + `Alt` + `O`	`Cmd` + `Alt` + `O`	**Quick Open in Auric Files...**
`Ctrl` + `Alt` + `N`	`Cmd` + `Alt` + `N`	**New Sibling File**
`Ctrl` + `Alt` + `Shift` + `N`	`Cmd` + `Alt` + `Shift` + `N`	**Batch Create Folders...**
`Ctrl` + `Alt` + `D`	`Cmd` + `Alt` + `D`	**Duplicate Current File**
`Ctrl` + `Alt` + `Shift` + `R`	`Cmd` + `Alt` + `Shift` + `R`	**Rename Current File**
`Ctrl` + `Alt` + `Shift` + `M`	`Cmd` + `Alt` + `Shift` + `M`	**Move Current File**
`Ctrl` + `Alt` + `Shift` + `Backspace`	`Cmd` + `Alt` + `Shift` + `Backspace`	**Delete (Workspace Trash)**
`Ctrl` + `Alt` + `Shift` + `C`	`Cmd` + `Alt` + `Shift` + `C`	**Copy Relative Path**
`Ctrl` + `Alt` + `Shift` + `A`	`Cmd` + `Alt` + `Shift` + `A`	**Copy Absolute Path**
`Ctrl` + `Alt` + `R`	`Cmd` + `Alt` + `R`	**Go to Related File...**
`Ctrl` + `Alt` + `Z`	`Cmd` + `Alt` + `Z`	**Undo Last File Operation**
`Ctrl` + `Alt` + `Y`	`Cmd` + `Alt` + `Shift` + `Z`	**Redo File Operation**
`Ctrl` + `Alt` + `Shift` + `E`	`Cmd` + `Alt` + `Shift` + `E`	**Open File Workspace**
`Ctrl` + `Alt` + `Shift` + `F`	`Cmd` + `Alt` + `Shift` + `F`	**Batch Create Files...**

Deck: chords

Press `Ctrl` + `Alt` + `A` (`Cmd` + `Alt` + `A` on macOS), release, then press the second key.

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `A`, then `N`	`Cmd` + `Alt` + `A`, then `N`	**New Sibling File**
`Ctrl` + `Alt` + `A`, then `Shift` + `N`	`Cmd` + `Alt` + `A`, then `Shift` + `N`	**New Child File**
`Ctrl` + `Alt` + `A`, then `F`	`Cmd` + `Alt` + `A`, then `F`	**Batch Create Files...**
`Ctrl` + `Alt` + `A`, then `Shift` + `F`	`Cmd` + `Alt` + `A`, then `Shift` + `F`	**Batch Create Folders...**
`Ctrl` + `Alt` + `A`, then `B`	`Cmd` + `Alt` + `A`, then `B`	**Batch Structure...**
`Ctrl` + `Alt` + `A`, then `Ctrl` + `N`	`Cmd` + `Alt` + `A`, then `Cmd` + `N`	**New File...**
`Ctrl` + `Alt` + `A`, then `Ctrl` + `Shift` + `N`	`Cmd` + `Alt` + `A`, then `Cmd` + `Shift` + `N`	**New Folder...**
`Ctrl` + `Alt` + `A`, then `T`	`Cmd` + `Alt` + `A`, then `T`	**New File from Template...**
`Ctrl` + `Alt` + `A`, then `P`	`Cmd` + `Alt` + `A`, then `P`	**New from Framework Preset...**
`Ctrl` + `Alt` + `A`, then `R`	`Cmd` + `Alt` + `A`, then `R`	**Create Related Files...**
`Ctrl` + `Alt` + `A`, then `E`	`Cmd` + `Alt` + `A`, then `E`	**Rename Current File**
`Ctrl` + `Alt` + `A`, then `D`	`Cmd` + `Alt` + `A`, then `D`	**Duplicate Current File**

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `A`, then `M`	`Cmd` + `Alt` + `A`, then `M`	**Move Current File**
`Ctrl` + `Alt` + `A`, then `C`	`Cmd` + `Alt` + `A`, then `C`	**Copy To...**
`Ctrl` + `Alt` + `A`, then `Shift` + `R`	`Cmd` + `Alt` + `A`, then `Shift` + `R`	**Batch Rename...**
`Ctrl` + `Alt` + `A`, then `Shift` + `M`	`Cmd` + `Alt` + `A`, then `Shift` + `M`	**Batch Move...**
`Ctrl` + `Alt` + `A`, then `X`	`Cmd` + `Alt` + `A`, then `X`	**Convert Extension...**
`Ctrl` + `Alt` + `A`, then `Backspace`	`Cmd` + `Alt` + `A`, then `Backspace`	**Delete (Workspace Trash)**
`Ctrl` + `Alt` + `A`, then `Shift` + `Backspace`	`Cmd` + `Alt` + `A`, then `Shift` + `Backspace`	**Delete Permanently**
`Ctrl` + `Alt` + `A`, then `O`	`Cmd` + `Alt` + `A`, then `O`	**Go to Related File...**
`Ctrl` + `Alt` + `A`, then `V`	`Cmd` + `Alt` + `A`, then `V`	**Reveal in File Manager**
`Ctrl` + `Alt` + `A`, then `L`	`Cmd` + `Alt` + `A`, then `L`	**Copy Relative Path**
`Ctrl` + `Alt` + `A`, then `A`	`Cmd` + `Alt` + `A`, then `A`	**Copy Absolute Path**
`Ctrl` + `Alt` + `A`, then `U`	`Cmd` + `Alt` + `A`, then `U`	**Copy File URI**
`Ctrl` + `Alt` + `A`, then `G`	`Cmd` + `Alt` + `A`, then `G`	**Tag Files...**
`Ctrl` + `Alt` + `A`, then `K`	`Cmd` + `Alt` + `A`, then `K`	**Add to Group...**
`Ctrl` + `Alt` + `A`, then `1`	`Cmd` + `Alt` + `A`, then `1`	**Toggle Favorite**
`Ctrl` + `Alt` + `A`, then `2`	`Cmd` + `Alt` + `A`, then `2`	**Toggle Pinned**
`Ctrl` + `Alt` + `A`, then `3`	`Cmd` + `Alt` + `A`, then `3`	**Toggle Native Explorer Exclusion**
`Ctrl` + `Alt` + `A`, then `4`	`Cmd` + `Alt` + `A`, then `4`	**Toggle Highlight**
`Ctrl` + `Alt` + `A`, then `Q`	`Cmd` + `Alt` + `A`, then `Q`	**Open Matching Test**
`Ctrl` + `Alt` + `A`, then `S`	`Cmd` + `Alt` + `A`, then `S`	**Open Matching Style**
`Ctrl` + `Alt` + `A`, then `W`	`Cmd` + `Alt` + `A`, then `W`	**Open Matching Story**
`Ctrl` + `Alt` + `A`, then `I`	`Cmd` + `Alt` + `A`, then `I`	**Open Matching Component**
`Ctrl` + `Alt` + `A`, then `Shift` + `D`	`Cmd` + `Alt` + `A`, then `Shift` + `D`	**Open Matching Documentation**
`Ctrl` + `Alt` + `A`, then `Shift` + `V`	`Cmd` + `Alt` + `A`, then `Shift` + `V`	**Reveal in VS Code Explorer**
`Ctrl` + `Alt` + `A`, then `Z`	`Cmd` + `Alt` + `A`, then `Z`	**Undo Last File Operation**
`Ctrl` + `Alt` + `A`, then `Y`	`Cmd` + `Alt` + `A`, then `Y`	**Redo File Operation**
`Ctrl` + `Alt` + `A`, then `H`	`Cmd` + `Alt` + `A`, then `H`	**Show Operation History**
`Ctrl` + `Alt` + `A`, then `Shift` + `T`	`Cmd` + `Alt` + `A`, then `Shift` + `T`	**Open Workspace Trash**
`Ctrl` + `Alt` + `A`, then `Shift` + `U`	`Cmd` + `Alt` + `A`, then `Shift` + `U`	**Restore from Trash...**
`Ctrl` + `Alt` + `A`, then `Shift` + `Delete`	`Cmd` + `Alt` + `A`, then `Shift` + `Delete`	**Empty Workspace Trash**
`Ctrl` + `Alt` + `A`, then `Shift` + `C`	`Cmd` + `Alt` + `A`, then `Shift` + `C`	**Compare Selected Files**
`Ctrl` + `Alt` + `A`, then `Shift` + `G`	`Cmd` + `Alt` + `A`, then `Shift` + `G`	**Merge Selected Files**
`Ctrl` + `Alt` + `A`, then `Shift` + `X`	`Cmd` + `Alt` + `A`, then `Shift` + `X`	**Export Selected Items...**
`Ctrl` + `Alt` + `A`, then `Shift` + `E`	`Cmd` + `Alt` + `A`, then `Shift` + `E`	**Open File Workspace**

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `A`, then `Shift` + `S`	`Cmd` + `Alt` + `A`, then `Shift` + `S`	**Open Settings**

In the workspace panel

Keys	What they do
`Escape` in **Find an action**	Clear the search
`Tab`, `Enter`, `Space`	Move between and use buttons and group headings
Mouse wheel, arrow keys	Scroll the panel

Native VS Code keys that still apply

`F2` renames in the Explorer with VS Code's own rename, and drag-and-drop moves files as usual. These native actions are not recorded in Auric's operation history.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Glossary

Plain-language definitions of the terms used in the Auric Artisan File System documentation.

Barrel — an index file that re-exports other modules, such as `index.ts`.

Batch operation — one action applied to many files at once, such as a batch rename.

Brace expansion — writing `{a,b}` or `{1..3}` in a path to produce several paths.

Companion file — a file that belongs with another: its test, stylesheet, story, documentation or types.

Conflict — a planned change whose target already exists.

Deck — the optional set of about 60 keyboard shortcuts turned on by `auricFiles.keyboard.enableDefaultShortcuts`.

Filter — a set of conditions (text, kind, tag, extension, path, size, date, flags, Git state) that narrows the indexed file list.

Focus mode — limiting Auric's indexed file lists to chosen paths.

Framework preset — a ready-made set of templates and relationships for one ecosystem, such as React or Go.

Front matter — the settings block between two `---` lines at the top of a template file.

Generator — a template defined in settings that can create companion files in the same step.

Group — a named set of files and folders from anywhere in the workspace. The panel calls creating one "Create Workspace from Selection".

Guarded operation — a file operation that is planned, checked against protected paths and collisions, confirmed when large, and recorded for undo.

Operation history — the list of Auric file operations you can undo and redo.

Partial — a template included inside another with `{{> id}}`.

Profile — a saved way of viewing the workspace: sort, grouping, filter, open folders, focus mode and hidden list.

Properties — the view that shows a file's facts and Auric organisation data.

Protected path — a path matching `auricFiles.safety.protectedPaths`, which no Auric operation may rename, move or delete.

Relation rule — a pattern linking a source file to its companions.

Restricted Mode — how VS Code runs an untrusted workspace; Auric Files then allows browsing and navigation but no file changes.

Tag — a named, coloured label you can put on files.

Template — a file pattern with a name and a body containing placeholders.

Virtual folder — a named collection of items that exists only as organisation data, not on disk.

Workspace index — Auric's list of the workspace's files and folders, used for search, filters, duplicates and related files.

Workspace trash — the `.auric/trash` folder where deleted items are kept so they can be restored.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Limits and accuracy

What Auric Artisan File System reports and how precisely, its built-in limits, and what it deliberately does not do.

What Properties can report

Properties shows what the file system reports through VS Code, and labels anything it cannot know.

Detail	Accuracy
Name, path, type	Exact
Size	As reported. Folders show **Folder**; a recursive disk-usage total is not calculated
Created	Only where the file system records a creation time; otherwise **Unavailable**
Modified	As reported
Owner / creator	Not available through VS Code; never guessed
Permissions	Only the read-only flag. Its absence does not prove you can write the file. Full access-control lists are not available
Git status	From the last index, and labelled so; refresh the index to update it
Related files	Found through the index; before it is built, shown as not indexed
Version	Counts Auric operations only

Built-in limits

Limit	Default	Setting
Entries per index scan	20,000 (500–200,000)	`auricFiles.explorer.maxNodes`
Matches per search	2,000 (50–50,000)	`auricFiles.search.maxResults`

Limit	Default	Setting
Paths from one brace expression	1,000	—
Paths in one batch creation	2,000	—
Operations kept for undo	200 (10–2,000)	`auricFiles.safety.historyLimit`
Workspace trash size	256 MiB	`auricFiles.safety.trashMaxBytes`
Workspace trash retention	14 days (1–365)	`auricFiles.safety.trashRetentionDays`
Folders read at once while indexing	8 (1–64)	`auricFiles.performance.scanConcurrency`
Regular expression length in batch rename	512 characters	—

When the index reaches its limit, it stops and marks itself as truncated, and counts say so, rather than stalling VS Code.

Deliberate boundaries

- The **Auric Files** sidebar is a companion, not a second file tree. VS Code owns the Explorer's rows, sorting and drag-and-drop.
- VS Code's Explorer supports sorting by name, type and date modified only. Auric's other sort and grouping options apply to its own indexed lists.
- The operation history records Auric operations only. It does not record editor edits or native Explorer actions, and it does not keep old copies of file contents.
- Duplicate review never deletes by itself; you choose every file.
- Selection export is a JSON manifest, not an archive of file contents.
- Template expressions are substitution only; templates cannot run code.
- File-changing operations require a trusted workspace.
- Remote and virtual file systems offer different capabilities. In virtual workspaces, revealing in the operating system's file manager, content hashing and the native trash are unavailable.
- The interface is in English.
- The icon tools only work with the Auric Artisan File Icons Facet or Playful theme. Icon menu entries added by those themes are managed by that extension.

Keeping your work safe

Protected paths, confirmations, previews, history and the workspace trash are safeguards, not a backup. Keep using version control and normal backups.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Privacy

What Auric Artisan File System keeps on your machine and what, if anything, uses the network.

Everything runs locally

- File operations, the index, search, filters, duplicate review and Properties all run on your machine (or next to the workspace, in a remote setup).
- No Auric Artisan account is needed.

- Properties reads file-system metadata only, never file contents. Duplicate review reads contents only to compare files by hash when the `content` strategy is used, and only for files that share a size.

Where data is kept

Data	Where
Tags, groups, virtual folders, flags, saved filters, profiles, operation history	VS Code's storage for this workspace, on your machine. Not written into the project and not copied by Settings Sync
Workspace trash	`.auric/trash/` inside the workspace folder
Native Explorer exclusions	The workspace's `files.exclude` setting
Explorer sort order	The workspace's `explorer.sortOrder` setting
Selection manifests	Wherever you save them

Logging

The **Auric Artisan File System** output channel logs paths relative to the workspace and never writes file contents. `auricFiles.logLevel` controls how much is written (default `warn`).

Network

The extension's own features do not use the network. The workspace panel loads its scripts and styles from the extension itself. Opening the auricartisan.com link, support links or Marketplace images uses your browser and the network.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Release notes

What changed in each published version of Auric Artisan File System.

0.1.0 (11 September 2026)

The first Marketplace release.

A new file workspace

- An editor-sized **Auric Files** workspace and a matching compact **Auric Files** section in the Explorer.
- File tools in seven searchable groups (**Create**, **Inspect & navigate**, **Edit & batch**, **Organise**, **Find & filter**, **History & recovery**, **Workspace**), with keyboard-operable sections that close their siblings, and **Collapse all**.
- Workspace summaries, the selected resource and a resource picker.
- VS Code's own Explorer and the **Quick Actions...** list are kept.

Graphical Properties

- Compact Explorer hovers replace long metadata tooltips.
- A **Properties** view with **General**, **Organisation** and **Access & history** cards, path copying, tag editing and group actions.

- Index-dependent information and cached Git state are labelled; unavailable owner and permission details are never made up.

Cleaner menus and safer targeting

- One **Auric Files** submenu on Explorer and editor context menus, with **File Properties** kept directly in the Explorer menu.
- Multi-selection in the Explorer is kept when opening the workspace.
- A selection outside the workspace can no longer fall back to a different file for file actions.
- Icon tools for the optional Facet and Playful icon themes.

File-system capabilities

- Native Explorer creation, rename, sorting, multi-selection and file badges.
- Structured batch creation, templates and ten framework presets.
- Related-file discovery and generation.
- Guarded move, copy and duplicate, batch rename, extension conversion, compare and merge, and selection export.
- Search, filters, duplicate and empty-file review, tags, groups and profiles.
- Undo and redo of operations, the workspace trash and restoration.

VS CODE EXTENSIONS › AURIC ARTISAN FILE SYSTEM

Related

Other Auric Artisan extensions and pages that work alongside Auric Artisan File System.

Other Auric Artisan VS Code extensions

- Auric Artisan File Icons — the Facet and Playful icon themes that the **Icon tools** section works with.
- Auric Artisan Formatter — format the files you create, with a diff preview first.
- Auric Artisan Live — serve and preview the project you are organising.
- Auric Artisan Studio — colour, accessibility and code-health checks.

Setup

- Install and set up the Auric Artisan extensions
- Settings, sync and workspace trust

Auric Artisan pages

- VS Code extensions: https://auricartisan.com/vscode/
- Marketplace listing: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-file-system
- Contact and support: https://auricartisan.com/company/contact/

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Auric Artisan Font Hub

What Auric Artisan Font Hub does, who it is for, how to install it and where to find every guide.

Auric Artisan Font Hub is a typography workspace inside VS Code. It holds the Auric Artisan font library: 2,023 font families, 547 of them variable, with metadata bundled in the extension so you can browse offline. Search and filter the library, open a family's specimen, glyphs, OpenType features and licence, compare families side by side, and get recommendations for a design intent, an industry or a colour palette.

When you have chosen, Font Hub adds the font to your code from the Auric font CDN, `fonts.auricartisan.com`. **Add to code** adds a stylesheet link and a reusable class such as `.auric-font-inter` to an HTML, CSS or JavaScript file without touching your existing styles. **Preview** shows the change first in VS Code's own diff view, with **Apply** and **Cancel**. For other file types it copies an embed for you to place, and it offers every common format: HTML link, CSS `@import`, `font-family`, `@font-face`, Tailwind configuration, SCSS variables and design tokens. For self-hosting, it exports a ZIP kit with the font files, CSS, a demo page and the required licence notices, where the licence allows it.

Browsing works offline. Previews and downloads fetch from the Auric font CDN and are cached locally. No account is needed, and your searches, code and favourites are not uploaded.

Who it is for

- Front-end developers choosing and adding web fonts.
- Designers checking a family's styles, glyph coverage, features and licence before committing to it.
- Teams building a design system who need tokens, Tailwind or SCSS entries for their fonts.
- Anyone who wants to self-host fonts with the right licence files.

Where to find it

- Marketplace ID: `auric-artisan.auric-artisan-font-hub`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-font-hub
- Install from a terminal:

```
code --install-extension auric-artisan.auric-artisan-font-hub
```

- Requires VS Code 1.85 or newer. Version 0.1.0 at the time of writing.
- The same library on the web: https://auricartisan.com/tool/font/
- Settings and command IDs use the prefix `auricFontLib`, kept from the extension's earlier name, Font Library.

Quick start

1. Install the extension.
2. Select **Auric Font Hub** in the Activity Bar, or run **Auric Artisan Font Hub: Open Font Hub** (`Ctrl` + `Alt` + `F`; `Cmd` + `Alt` + `F` on macOS).
3. Type a family, designer, script or feature in the search box, or narrow the list with **Category**, **Sort** and **More filters**.
4. Select a font card's name to open its details: specimen, waterfall, glyphs, features, styles and licence.

5. Open an HTML, CSS or JavaScript file in the editor, then choose **Preview** on the font card.
6. Review the proposed change in VS Code's diff view, then choose **Apply** (or **Cancel** to leave the file as it was). The edit is left unsaved, so you can undo it.
7. Apply the generated class in your markup, for example `<p class="auric-font-inter">`.
8. For a quick pick without the panel, press `Ctrl` + `Alt` + `T` (**Quick Pick Font**).

What you can do

- Browse 2,023 families by category, weight, variable, italic and monospace properties, OpenType features, scripts, licence and source.
- Edit the specimen text and size, switch card density, and page through results.
- Inspect a family in twelve tabs: **Specimen**, **Waterfall**, **Suitability**, **Pairings**, **Features**, **Develop**, **Glyphs**, **Charset**, **Styles**, **About**, **License** and **Use & Embed**.
- Try variable axes and OpenType features live in the specimen.
- Rank families against a design intent, an industry and a colour palette with **Find your font**.
- Save favourites, return to recent fonts, select up to 12 families and compare up to four.
- Add fonts to HTML, CSS, SCSS, Less, PostCSS, JavaScript and TypeScript, with a native diff preview first.
- Copy or insert ten embed formats.
- Check the CDN and browser font delivery (CORS) with a built-in report.
- Export a licence-aware self-hosting kit as a ZIP.
- Use the shared Auric colour picker for palette editing, contrast checks and ramps.

In this folder

- docs/ — task guides and the full reference.
  - Getting started
  - Find and compare fonts
  - Font details
  - Add a font to your code
  - Check CDN and browser delivery
  - Export a self-hosting kit
  - Offline use, cache and network
  - Moving from Font Library
  - Reference: commands, keys and settings
  - Troubleshooting
  - FAQ
- others/ — supporting material.
  - Glossary
  - Shortcuts
  - Limits and accuracy
  - Privacy
  - Related
  - Release notes

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Auric Artisan Font Hub guides

Contents of the Auric Artisan Font Hub task guides and reference.

Start with the getting-started guide for a tour of the sidebar and the studio. The reference lists all 16 commands, the two keyboard shortcuts and all 13 settings.

File	What it covers
getting-started.md	Install, the sidebar and studio, the screen tour, a first font added to a page
find-and-compare-fonts.md	Search, filters, scopes, sorting, collections, Find your font, favourites, selection, comparison
font-details.md	The twelve detail tabs: specimen, waterfall, suitability, pairings, features, glyphs, licence and more
add-fonts-to-code.md	Add to code, Preview with Apply and Cancel, embed formats, multi-family insertion, Quick Pick
cdn-and-browser-delivery.md	The CDN health check, CORS, content types and Content Security Policy
self-hosting-kit.md	Download a ZIP kit with fonts, CSS, demo and licences
offline-cache-and-network.md	Offline catalogue, previews cache, refresh schedule, switching the network off
moving-from-font-library.md	Replacing the earlier Font Library extension
reference.md	Every command, keybinding, menu, control and setting
troubleshooting.md	Symptom, cause and fix
faq.md	Short answers to common questions

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Getting started with Auric Artisan Font Hub

Install Auric Artisan Font Hub, learn the sidebar and studio layout, and add a first font to a page.

Install

1. Check that VS Code is version 1.85 or newer (**Help** › **About**).
2. In the Extensions view (`Ctrl` + `Shift` + `X`), search for `@id:auric-artisan.auric-artisan-font-hub` and select **Install**. Or run:

   ```
   code --install-extension auric-artisan.auric-artisan-font-hub
   ```

3. If the older **Font Library** extension is installed, disable or uninstall it first. See Moving from Font Library.

Font Hub works in untrusted and virtual workspaces. It only writes files when you choose an explicit save or insert.

Two ways to open it

- **Sidebar.** Select **Auric Font Hub** in the Activity Bar. The **Font Hub** view is a compact version of the studio.
- **Studio.** Run **Auric Artisan Font Hub: Open Font Hub** (`Ctrl` + `Alt` + `F`), select **Open studio ↗** in the sidebar, or select the **Fonts** item in the status bar. The studio opens as an editor tab with more room.

Both share the same layout, controls, favourites and selection. The sidebar's title bar has **Open Font Hub**, **Focus Font Search** and **Refresh Font Catalog from Auric CDN**.

Opening Font Hub creates no files in your project.

The screen tour

From top to bottom:

Area	What it contains
Header	The Auric Artisan mark; **auricartisan.com** ↗; **Open studio** ↗ (in the sidebar); ↻ refresh the catalogue; ↗ open the web library; ⚙ Font Hub settings
Introduction	The catalogue status, such as **Offline catalog ready**
Your font delivery card	The CDN address `fonts.auricartisan.com` and **Check CDN health**
CDN & insertion help	A collapsible explanation, the last check's status and **Run a live check**
Guide	**01 Discover · 02 Compare · 03 Use**, with a hint to try **Preview**. Dismiss it with ×
Controls	**Search**, **Category**, **Sort**, **Preview text**, **Size** (18–96 px) and **More filters**
Advanced filters	**Properties**, **Weights**, **OpenType**, **Scripts**, **License**, **Source** and **Reset filters**
Scopes	**All**, **For you**, **Favorites**, **Recent**, **Variable**, **Selected**
Find your font	The recommender: design intent, industry, colour palette, options and **Rank fonts**
Density	Compact, comfortable and showcase cards
Collections & recommendations	Featured collections and recommended families (collapsible)
Results	The font cards, with **Show more** to load the next batch
Footer	The version, a licence reminder and **CDN diagnostics**

A font card

Each card shows the category, a favourite heart and a select button, the family name set in its own font with your preview text, the designer, whether it is variable and how many weights it has, and three scores out of 100: **A11y** (accessibility), **Read** (readability) and **UI** (UI and product). At the bottom are **Add to code** and **Preview**.

Select the family name to open its details. See Font details.

The selection tray

When you select families, a **Selected** button appears. It opens the **Selected families** tray with **Copy embed**, **Insert**, **Compare** and **Download**.

A first task: add a font to an HTML page

1. Open an HTML file that has a `<head>` in the editor.
2. In Font Hub, search for `Inter`.
3. On the Inter card, choose **Preview**.

4. VS Code opens a diff titled **Font Hub: Inter — proposed changes**. The right side adds a stylesheet link and a small class rule inside `<head>`.
5. In the notification **Inter: add CDN setup to *file*? Existing content stays unchanged.**, choose **Apply**.
6. The file now contains the setup and is left unsaved; `Ctrl` + `Z` undoes it.
7. Use the class in your markup:

```
<p class="auric-font-inter">A considered choice of type.</p>
```

8. Save and open the page in a browser. The paragraph uses Inter.

Keyboard

Keys	Where	What they do
`Ctrl` + `Alt` + `F`	VS Code	Open Font Hub
`Ctrl` + `Alt` + `T`	VS Code	Quick Pick Font
`/` or `Ctrl` + `K`	Font Hub	Focus the search box
`Escape`	Font Hub	Close the comparison, details or tray; otherwise clear the search
Arrow keys	Font cards	Move between cards

On macOS, `Cmd` replaces `Ctrl`.

Next steps

- Find and compare fonts
- Add a font to your code
- Check CDN and browser delivery

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Add a font to your code

Add fonts from the Auric font CDN to HTML, CSS and JavaScript files with a reviewable diff, insert explicit embed formats, and add several families at once.

Font Hub adds fonts from the Auric font CDN. Every embed points to a stylesheet at:

```
https://fonts.auricartisan.com/css/<family-id>.css
```

For example, Inter is `https://fonts.auricartisan.com/css/inter.css`. Font Hub never substitutes another font service.

Automatic insertion

Add to code (on a font card) and **Auric Artisan Font Hub: Add Font to Code** use *Automatic* mode by default. Automatic mode adds two things: the CDN stylesheet, and a reusable class named `.auric-font-<family-id>` that applies the font. It

never replaces selected text, never changes your existing rules and never applies the font to the whole page. You decide where to use the class.

What it adds depends on the active file:

Active file	What Automatic mode does
HTML with a `<head>`	Adds `<link rel="stylesheet" href="…">` and a small `<style>` rule for the class just before `</head>`
CSS, SCSS, Less, PostCSS	Adds `@import url('…');` before your normal rules (after any `@charset`, `@use` or `@forward`) and appends the class rule
JavaScript, TypeScript	Appends a small guarded block that adds the stylesheet and class to the page when it runs in a browser, and does nothing on the server
JSX, TSX, Vue, Svelte, Astro, data files and other languages	Copies an HTML embed to the clipboard for you to place in your layout or global stylesheet
No editor open	Copies the CDN embed to the clipboard

For example, after adding Inter to a page, use:

```
<p class="auric-font-inter">A considered choice of type.</p>
```

Running it again does not duplicate the setup: if the link and class are already present, Font Hub says the family is already configured. An HTML file without a `<head>` is not changed; you get **No HTML head found. Copy the embed into your page head or framework layout.** and the embed is copied.

Automatic insertion works on files up to 2 MiB. It is deliberately cautious and is not a full HTML, CSS or JavaScript parser.

Preview before applying

1. Open the file you want to change.
2. Choose **Preview** on a font card, run **Auric Artisan Font Hub: Preview Font Insertion**, or choose **Preview CDN insertion** in **Quick Pick Font**.
3. VS Code's diff editor opens with your file on the left and the proposed file on the right, titled **Font Hub: *family* — proposed changes**.
4. A notification asks **Add CDN setup to *file*? Existing content stays unchanged.** Choose **Apply** or **Cancel**.
5. **Apply** checks that your file has not changed since the preview, then makes the edit. The file is left unsaved, and one undo removes the change.

If you edited the file after opening the preview: **The document changed. Preview the font insertion again before applying.** Open the preview again.

Explicit formats

To insert a specific format instead of the automatic setup:

- In the details panel, open **Use & Embed** or **Develop**, choose a format and select **Insert**. The snippet is inserted at the cursor, or replaces each selection.
- Or set `auricFontLib.defaultFormat` to a format, and **Add Font to Code** inserts that format at the cursor.

Explicit formats insert text where you are. They do not merge anything into framework configuration files such as `tailwind.config.js`; you paste the entry where it belongs.

If no editor can be written to, the snippet is copied to the clipboard instead, and a message says why.

Copy an embed

- **Auric Artisan Font Hub: Copy Font Embed As...** — choose a family, then **Choose a snippet format**: HTML stylesheet link, CSS @import, CSS font-family, @font-face, AuricFonts loader, Tailwind config, SCSS variable, Design tokens JSON or Family name.
- **Copy** in the details panel copies the current format.
- **Save** writes the current format to a new file you name.

Several families at once

1. Select up to 12 families (the select button on each card).
2. Open the **Selected** tray.
3. Choose **Insert** to add every family's setup to the active file, each with its own class, or **Copy embed** to copy one combined embed.

After inserting, Font Hub reminds you to *Apply the generated .auric-font- classes in your markup. File left unsaved.** A system font in the selection stops the insertion: **Remove system fonts from the CDN selection first.**

From the editor's context menu

Right-click in an editor for the **Auric Fonts** submenu: **Quick Pick Font**, **Add Font to Code**, **Preview Font Insertion**, **Copy Font Embed As...** and **Open Font Hub**.

Using the font in your CSS

To use the family in your own selectors instead of the generated class, copy its `font-family` value (**Copy family** in the specimen, or the **CSS font-family** format) and use it in your stylesheet. The stylesheet link or `@import` must still be present, or the browser has nothing to load.

If the font does not appear in the browser, see Check CDN and browser delivery.

When this works, the file contains the CDN setup, the element with the class shows the new font in a browser, and **Recent** in Font Hub lists the family.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Check CDN and browser delivery

Run Font Hub's CDN health check and fix fonts that preview in VS Code but do not load on your website.

A font that previews in Font Hub can still fail on your website. Previews are fetched by VS Code itself, while your page's fonts are fetched by the browser, which applies extra security rules. The main one is *CORS* (cross-origin resource sharing): a browser only uses a font from another address if that server says the page is allowed to.

Run the check

1. Run **Auric Artisan Font Hub: Check Font CDN and Browser CORS**, or select **Check CDN health** (or **CDN diagnostics**, or **Run a live check**) in Font Hub.
2. Font Hub sends small `HEAD` requests to the CDN for three things:
   - **Catalog** — the catalogue file;
   - **Stylesheet** — a family's stylesheet;
   - **Font file** — that family's preview font.
3. Each request is made as if from the test address `http://localhost:5500`.
4. The report shows, for each item, whether it is available, whether its content type is right, and, for the font file, whether CORS allows the test address.

The overall result is **Browser delivery ready** only if every item is available with the right content type and the font file allows cross-origin use. Otherwise it says **Needs attention**, with **CORS missing for test origin** when that is the problem. **Check again** reruns it.

The check is a sample: one catalogue file, one stylesheet and one font file, from one test address. It is not an audit of every family or of your production address. It needs `auricFontLib.network.enabled`; if the network is switched off you see **Enable network access in Font Hub settings to run CDN checks.**

When fonts do not load on your site

Work through these in your browser's developer tools (Network tab, filtered to fonts and CSS):

1. **Is the stylesheet loaded?** Look for `fonts.auricartisan.com/css/<family-id>.css` with status 200. If it is missing, the link or `@import` is not in the page, or it is in a file the page does not load.
2. **Is the font file loaded?** A successful stylesheet alone is not enough. Look for the font file request.
3. **Is it blocked by CORS?** The browser console reports a CORS error. A public font CDN can answer every site with:

   ```
   Access-Control-Allow-Origin: *
   ```

   A restricted CDN should allow your exact website and development addresses instead.
4. **Is the content type right?** Stylesheets must be served as `text/css` and fonts as a font type.
5. **Is it blocked by your Content Security Policy?** If your site has a Content Security Policy, allow the CDN in `style-src` and `font-src`. Under a strict policy, inline `<style>` rules added by Font Hub may be refused; move the generated class rule into a stylesheet your policy allows.
6. **Is the class applied?** The generated setup only defines `.auric-font-<family-id>`. Add the class to your markup, or use the family in your own CSS.

Font Hub reports what the CDN answers. It cannot change a server's hosting configuration or get around browser security.

When delivery works, the browser's Network tab shows both the stylesheet and the font file loaded with status 200, and the text renders in the chosen family.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Find and compare fonts

Search and filter the Auric font library, get recommendations for a design intent, and keep favourites, recents and a selection to compare.

Search

Type in **Search** (placeholder **Family, designer, script, feature…**). It matches family names, designers, scripts, OpenType features and licence information. Press `/` or `Ctrl` + `K` anywhere in Font Hub to jump to it, and `Escape` to clear it.

Results appear as cards and load in batches of `auricFontLib.pageSize` (48 by default); **Show more** loads the next batch. If nothing matches, **No matching families** offers **Reset filters**.

Category and sort

- **Category** — **All categories** or one of the library's categories: sans-serif, serif, display, handwriting and monospace.
- **Sort** — order the results:

Sort	Order
Alphabetical	By family name
Accessibility score	Highest accessibility score first
Readability score	Highest readability score first
UI score	Highest UI and product score first
Developer score	Highest developer and code score first
Editorial score	Highest editorial score first
Most weights	Families with the most weights first
Most glyphs	Families with the most characters first
Newest	Most recently added first
Variable first	Variable families first

Advanced filters

Select **More filters** to show chips you can combine:

Filter	Chips
Properties	**Variable**, **Italic**, **Monospace**, **4+ weights**
Weights	100, 200, 300, 400, 500, 600, 700, 800, 900
OpenType	The twelve most common feature tags in the library, such as `kern` or `liga`
Scripts	The fourteen most common character ranges, such as Basic Latin, Cyrillic or Devanagari
License	The eight most common licences, such as OFL or Apache 2.0
Source	**Auric library**, and **System snapshot** when system fonts are included

Reset filters clears them. Your search, filters, sort and scope are remembered when you close and reopen Font Hub.

Preview text, size and density

- **Preview text** changes the sample shown on every card (up to 240 characters). The default comes from `auricFontLib.previewText`.
- **Size** sets the sample size from 18 to 96 pixels. The default is `auricFontLib.previewSize` (36).
- The density buttons switch between **Compact**, **Comfortable** and **Showcase** cards. The default is `auricFontLib.density`.

Scopes

Scope	Shows
All	Every family that matches
For you	Recommended families, ranked from role scores, style intent and your local use
Favorites	Families you marked with the heart, with a count
Recent	Families you recently inserted, copied, saved or downloaded, with a count
Variable	Variable families only
Selected	Families in your selection, with a count

Turn recommendations off with `auricFontLib.recommendations.enabled`.

Find your font

Find your font ranks families against what the type has to do. It uses measured traits and suitability scores, not popularity alone.

1. Select **Find your font** above the results.
2. Choose a **Design intent**: **UI & Product**, **Accessibility**, **Developer & Code**, **Editorial**, **Luxury**, **Friendly brand**, **Technical product** or **Bold display**.
3. Optionally choose an **Industry** to bias the ranking: **SaaS / Software**, **Developer / DevTools**, **Startup / Tech**, **E-commerce / Retail**, **Luxury / Fashion**, **Editorial / Publishing**, **Finance / Legal**, **Healthcare**, **Education**, **Food / Hospitality**, **Corporate / B2B** or **Creative / Agency**.
4. Optionally add a **Colour palette**: type up to eight hex colours (such as `#0F172A #F8FAFC #D7A638`) or add swatches with the colour picker. The palette's mood nudges the ranking.
5. Tick or untick **Prioritize accessibility** and **Prefer variable fonts**.
6. Select **Rank fonts**.

The shared Auric colour picker also offers contrast checks, generated ramps and a colour history for the palette.

Collections and recommendations

Collections & recommendations is a collapsible section with featured collections of families and a row of recommendations.

Favourites, recents and selection

- **Favourite** — the heart on a card, or **Favorite** in the details. **Pick from Favorite Fonts** lists them in a Quick Pick.
- **Recents** — Font Hub remembers fonts you use. **Pick from Recent Fonts** lists them.
- **Selection** — the select button on a card, or **Select** in the details, adds a family to your selection. The **Selected** button opens the tray.
- **Clear Favorites, Recents, and Selection** clears all three after confirmation.

Favourites, recents and selection are stored on your computer in VS Code's extension storage. They are not uploaded.

The selection tray

Button	What it does
Copy embed	Copy one combined embed for every selected family
Insert	Add the setup for every selected family to the active file (up to 12)
Compare	Open the comparison for up to four selected families
Download	Download a self-hosting kit for the selection

Compare families

1. Select two to four families.
2. Choose **Compare** in the tray, or run **Auric Artisan Font Hub: Compare Selected Fonts**.
3. The **Compare selected fonts** table shows each family side by side: **Category**, **Weights**, **Variable**, **Accessibility**, **Readability**, **UI & Product**, **Developer**, **Editorial** and **Glyphs**, with a specimen for each.

With fewer than two selected: **Select at least two font families to compare.** Only the first four are compared.

Quick Picks

For keyboard-only use:

- **Quick Pick Font** (`Ctrl` + `Alt` + `T`) — search every family by name, designer, category, script or licence, then choose **Add font to code**, **Preview CDN insertion**, **Copy embed as…**, **Inspect in Font Hub**, **Download self-hosting kit** or **Toggle favorite**.
- **Pick from Favorite Fonts** and **Pick from Recent Fonts** — the same, limited to those lists.

Each entry shows the category, weights, whether it is variable, the designer, its accessibility score and its licence.

System fonts

Turn on `auricFontLib.includeSystemFonts` to include metadata for common system fonts in the results. They are tagged **System snapshot**, their previews only work if the font is installed on your computer, and they cannot be added from the CDN or exported: copy their family name instead.

Scores are guidance

Suitability and accessibility scores are catalogue estimates based on measured traits. They are not guarantees. Check your real text, languages, sizes and contrast in the finished product.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Font details

Inspect a font family in Font Hub's twelve detail tabs, from the live specimen and waterfall to glyphs, features, styles and licence.

Select a family's name on its card to open the details panel. The header shows the family name and a short summary, with **Favorite**, **Select** and a close button (**×**, or `Escape`). Twelve tabs sit below it.

Previews are drawn with the real font, fetched from the Auric font CDN through VS Code and cached. If a preview cannot be loaded, the card and panel say **Preview unavailable** with the reason, and the rest of the details still work from the bundled catalogue.

Specimen

- An editable sample: click it and type your own text.
- **Size** slider, 18 to 120 pixels (48 by default).
- **Variable axes** — for variable families, a slider for each axis (such as weight or width), from its minimum to its maximum, starting at the default.
- **OpenType controls** — tick up to twelve of the family's features (such as ligatures or small capitals) to see them applied live.
- **Copy family** — copy the CSS `font-family` value.
- **Insert embed** — add the font to the active file (see Add a font to your code).

Waterfall

The same text at eleven sizes, from 11 to 88 pixels, so you can see where the design stops working small or large. Variable families with a weight axis also get a **Weight ramp**. A **Paragraph proof** shows a paragraph at 13, 16 and 19 pixels. Change the text with **Preview text** in the toolbar.

Suitability

Five scores out of 100 (**Accessibility**, **Readability**, **UI & Product**, **Developer**, **Editorial**), a **Style fingerprint** and **Why these scores**, which explains the measured traits behind them, such as x-height and how distinct similar letters are.

Pairings

Pairing suggestions for a second family, ranked by category contrast, x-height harmony, shared design traits and role strength.

Features

The family's OpenType features, each with its tag, a plain name (for example `liga` Standard ligatures, `kern` Kerning, `smcp` Small capitals, `onum` old-style figures, `tnum` Tabular figures, `frac` Fractions, `zero` Slashed zero) and a sample. **Copy CSS** copies the `font-feature-settings` line for that feature. Toggle features live on the **Specimen** tab. If there are none: **This family exposes no cataloged OpenType features.**

Develop

Developer exports — framework configuration and design-system tokens. Choose a format (Tailwind configuration, SCSS variable, design tokens JSON, the AuricFonts loader and the others), then **Copy**, **Insert** or **Save** it to a file.

Glyphs

A **Character map** of the family's characters. Select a character to copy it.

Charset

Charset coverage: the Unicode ranges the family covers, with the number of characters and the first glyph in each range. Use it to check a language or script is supported.

Styles

Styles and files: every style with its weight, posture (normal or italic), glyph count, file size and embedding permission, plus whether it is a variable or static family. **Download kit** exports a self-hosting kit.

About

About the family: designer, foundry, version, dates and description where the catalogue has them, and the **Copyright** notice.

License

The licence name and identifier, and a grid of permissions, each marked allowed (✓), not allowed (×) or unknown (?):

- **Commercial use**
- **Modification**
- **Distribution**
- **Private use**
- **Sell font alone**
- **Include license**

Load full license text shows the complete licence. The reminder **Review the original license before redistribution.** applies to every family: each font keeps its own licence.

Use & Embed

Hosted CDN or self-hosting-ready code. Choose a format, then **Copy**, **Insert** or **Save**:

Format	What you get
HTML stylesheet link	A `<link rel="stylesheet">` to the family's CDN stylesheet
CSS @import	An `@import url(…)` line
CSS font-family	A `font-family` declaration with a fallback stack
@font-face	`@font-face` rules for each style, with `font-display: swap` and variable weight ranges preserved
AuricFonts loader	A script snippet that loads the Auric font loader for this family
Tailwind config	A `fontFamily` entry for `tailwind.config.js`
SCSS variable	A `$font-…` variable
Design tokens JSON	A typography token with family, fallback and CSS address
Family name	Just the name

Self-host this family offers **Download kit**. See Export a self-hosting kit.

Copying a `font-family` declaration, Tailwind entry or SCSS variable alone does **not** load the font; you also need the stylesheet link, `@import`, `@font-face` rules or the loader.

Open on the web

Explore Fonts on AuricArtisan.com opens the same library in your browser at https://auricartisan.com/tool/font/.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Moving from Font Library

Replace the earlier Auric Artisan Font Library extension with Font Hub without duplicate commands, and know what carries over.

Font Hub 0.1.0 is the renamed and redesigned successor of an earlier extension called **Font Library**. Font Hub has a new extension ID, `auric-artisan.auric-artisan-font-hub`, so VS Code treats them as two different extensions.

Switch over

1. Open the Extensions view (`Ctrl` + `Shift` + `X`) and find **Font Library**.
2. Select **Disable** or **Uninstall**.
3. Install Font Hub: search for `@id:auric-artisan.auric-artisan-font-hub` and select **Install**.
4. Reload the window if VS Code asks.

If both are enabled at the same time, you see duplicate commands and views.

What carries over

Item	Carries over?
Settings (`auricFontLib.*`)	Yes. Font Hub uses the same setting names
Command IDs (`auricFontLib.*`)	Yes, so your custom keyboard shortcuts keep working
Favourites, recents and selection	No. They were stored with the old extension
Cached previews and font files	No. Font Hub builds its own cache

After switching, the Activity Bar shows **Auric Font Hub** and the Command Palette lists commands under **Auric Artisan Font Hub**.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Offline use, cache and network

What Font Hub can do offline, how it refreshes and caches catalogue data and fonts, and how to switch its network use off.

What works offline

The extension includes a snapshot of the catalogue: every family's details and the licence notices. Without a network connection you can:

- search, filter, sort and browse every family;
- open details, including styles, charset, about and licence information;
- use favourites, recents, selection and the recommender;
- copy and insert embeds (the page that uses them still needs the CDN when it loads).

Font previews and self-hosting downloads need the font files. Previews you have already seen stay available from the cache where possible; others need the network.

What uses the network

Only one address is ever contacted: `https://fonts.auricartisan.com`. Font Hub validates every address before requesting it, and the Font Hub panels themselves cannot make network requests; all fetching goes through VS Code.

Activity	When
Catalogue refresh	On the schedule in `auricFontLib.catalogRefresh`, or when you run **Refresh Font Catalog from Auric CDN**
Font previews	When a card or detail panel needs a font that is not cached
Licence text	When you select **Load full license text** or build a kit
Self-hosting downloads	When you download a kit
CDN check	When you run **Check Font CDN and Browser CORS**

Search queries, your source code, favourites and usage history are not sent. The CDN still receives the ordinary information any web request carries, including your IP address.

Switch the network off

Set `auricFontLib.network.enabled` to `false`. Font Hub then works from the bundled snapshot and the cache only. Previews that are not cached are unavailable, the CDN check is refused, and kits cannot be downloaded.

Catalogue refresh

`auricFontLib.catalogRefresh`	**Behaviour**
`manual`	Only when you run **Refresh Font Catalog from Auric CDN** (or select ↻)
`daily` (default)	Automatically, once the last refresh is older than `auricFontLib.cache.maxAgeHours` (24 hours by default)
`always`	Automatically every time Font Hub checks, without waiting for the cache to age

After a refresh: **Auric Font Hub refreshed *N* families.** If the refresh fails or is rejected, Font Hub keeps what it had: **Auric Font Hub kept cached data: *reason*.** The bundled snapshot always remains available. Refreshed identities and licence data are validated; a family the extension does not recognise may need an extension update.

The status bar **Fonts** item's tooltip shows the number of families and whether the **CDN catalog**, a **Cached catalog** or the **Bundled catalog** is in use. Turn the item off with `auricFontLib.statusBar.enabled`.

Cache

Setting	Default	Range	Meaning
`auricFontLib.cache.maxAgeHours`	24	1–720	How long cached CDN metadata counts as fresh
`auricFontLib.cache.maxMB`	512	64–4096	Disk space for previews, font files and refreshed metadata; the oldest items are removed first

The cache lives in VS Code's storage for the extension, not in your project. **Auric Artisan Font Hub: Clear Downloaded Font Cache** empties it: **Auric Font Hub cache cleared. Bundled catalog data remains available offline.**

System fonts

With `auricFontLib.includeSystemFonts` on, metadata for common system fonts captured when the catalogue was built appears in the list. Their previews depend on the font being installed on your computer.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Export a self-hosting kit

Download fonts from Font Hub as a ZIP kit with CSS, a demo page and licence notices, for hosting on your own server.

Self-hosting means serving font files from your own website instead of the CDN. Font Hub can package families into a ZIP *kit* ready to copy into a project.

Download a kit

Use any of these:

- **Download kit** in a family's **Styles** or **Use & Embed** tab;

- **Download** in the **Selected** tray, for every selected family;
- **Download self-hosting kit** in **Quick Pick Font**;
- **Auric Artisan Font Hub: Download Self-hosting Font Kit**, which lets you tick families in a list (**Choose font families**).

Then:

1. Font Hub checks each family's licence (see below).
2. Choose where to save the ZIP. The default name is `<family>-font-kit.zip` for one family or `auric-font-selection-kit.zip` for several.
3. A progress notification, **Preparing Auric font kit**, shows each family as its files are fetched.
4. **Saved self-hosting kit for *N* families.**

Saving is always an explicit action; nothing is written into your project unless you save it there.

What the kit contains

Path	Contents
`fonts/<family>/`	The font files for every style
`css/<family-id>.css`	`@font-face` rules pointing at those files, with `font-display: swap` and variable weight ranges kept
`licenses/<family>.txt`	The family's licence text, when there is one
`README-<family>.md`	Notes for each family
`index.html`	A demo page showing the families
`README.md`	Instructions: review every licence before redistribution, load the CSS and keep the relative font paths intact

Load a family by linking its CSS file from your page, keeping the `css/` and `fonts/` folders in the same relative positions.

Licence checks

A kit is only created when every family can be redistributed:

- Families whose licence does not allow redistribution, or where permission cannot be established, are refused.
- Families whose licence requires a notice are refused if the notice cannot be loaded.
- System fonts are never exported.

When a family is refused, the whole kit is blocked with **Self-hosting kit blocked.** and the reason for each family. Remove those families and try again.

Each font keeps its own licence. The extension's own licence does not grant rights to any font. Review the files in `licenses/` before you redistribute anything.

Limits

Limit	Value
Families per kit	`auricFontLib.download.maxFamilies`, 6 by default (1–12)
One font file	40 MB
Whole kit	250 MB; larger selections stop with **Selected font files exceed the 250 MB kit limit.**

Downloading needs the network and `auricFontLib.network.enabled`. Files already in the cache are reused.

When the kit is saved, opening its `index.html` in a browser shows each family rendered from the local files.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Reference: commands, keys and settings

Every Auric Artisan Font Hub command, keyboard shortcut, menu entry, panel control and setting, with types, defaults and ranges.

This page covers Auric Artisan Font Hub 0.1.0: 16 commands, 2 keyboard shortcuts, one Activity Bar view and 13 settings. Command and setting IDs use the `auricFontLib` prefix, kept from the extension's earlier name.

Commands

Fifteen commands appear under **Auric Artisan Font Hub:** in the Command Palette; **Show Logs** appears under **Auric Font Hub:**.

Command	ID	Keys	What it does
Open Font Hub	`auricFontLib.openStudio`	`Ctrl` + `Alt` + `F`	Open the full studio in an editor tab
Quick Pick Font	`auricFontLib.quickPick`	`Ctrl` + `Alt` + `T`	Choose a family, then an action
Add Font to Code	`auricFontLib.insertEmbed`	—	Choose a family and add it to the active file in the default format
Copy Font Embed As...	`auricFontLib.copyEmbed`	—	Choose a family and a format, and copy the snippet
Pick from Favorite Fonts	`auricFontLib.showFavorites`	—	Quick Pick limited to favourites
Pick from Recent Fonts	`auricFontLib.showRecents`	—	Quick Pick limited to recent fonts
Compare Selected Fonts	`auricFontLib.compareSelected`	—	Compare up to four selected families
Download Self-hosting Font Kit	`auricFontLib.downloadKit`	—	Export a ZIP kit for chosen families
Refresh Font Catalog from Auric CDN	`auricFontLib.refreshCatalog`	—	Refresh catalogue data now
Clear Favorites, Recents, and Selection	`auricFontLib.clearPersonalization`	—	Clear all three, after confirmation
Clear Downloaded Font Cache	`auricFontLib.clearCache`	—	Empty the cache; the bundled catalogue stays
Explore Fonts on AuricArtisan.com	`auricFontLib.openWeb`	—	Open https://auricartisan.com/tool/font/
Focus Font Search	`auricFontLib.focusSearch`	—	Move focus to the search box
Show Logs	`auricFontLib.showLogs`	—	Open the Font Hub log
Preview Font Insertion	`auricFontLib.previewInsert`	—	Show the proposed change as a diff, with Apply and Cancel
Check Font CDN and Browser CORS	`auricFontLib.checkCdn`	—	Run the CDN delivery check

On macOS the two shortcuts are `Cmd` + `Alt` + `F` and `Cmd` + `Alt` + `T`.

Where commands appear

Place	Entries
Activity Bar	**Auric Font Hub**, with the **Font Hub** view
Font Hub view title bar	**Open Font Hub**, **Focus Font Search**, **Refresh Font Catalog from Auric CDN**
Editor context menu › **Auric Fonts**	**Quick Pick Font**, **Add Font to Code**, **Preview Font Insertion**, **Copy Font Embed As...**, **Open Font Hub**
Status bar	**Fonts** (opens Font Hub); hide with `auricFontLib.statusBar.enabled`

Quick Pick Font actions

After you choose a family: **Add font to code**, **Preview CDN insertion**, **Copy embed as...**, **Inspect in Font Hub**, **Download self-hosting kit**, **Toggle favorite**.

Panel controls

Control	Where	What it does
Search	Controls	Search family, designer, script, feature and licence
Category	Controls	Filter by category
Sort	Controls	Ten orders, from **Alphabetical** to **Variable first**
Preview text	Controls	Sample text on cards (up to 240 characters)
Size	Controls	Sample size, 18–96 px
More filters	Controls	Show **Properties**, **Weights**, **OpenType**, **Scripts**, **License**, **Source** chips
Scopes	Above results	**All**, **For you**, **Favorites**, **Recent**, **Variable**, **Selected**
Find your font	Above results	Design intent, industry, colour palette, **Prioritize accessibility**, **Prefer variable fonts**, **Rank fonts**
Density	Above results	Compact, comfortable, showcase
Collections & recommendations	Above results	Featured collections and recommendations
Show more	Below results	Load the next batch
Check CDN health, **CDN diagnostics**, **Run a live check**	Card, footer, help	Run the CDN check
↻, ↗, ⚙	Header	Refresh catalogue, open the web library, open settings
Open studio ↗	Sidebar header	Open the full studio
Card: heart, select	Each card	Favourite, add to selection
Card: **Add to code**, **Preview**	Each card	Automatic insertion, with or without a diff preview
Tray: **Copy embed**, **Insert**, **Compare**, **Download**	Selected tray	Act on the whole selection
Details: **Favorite**, **Select**	Details header	Favourite or select the open family

Control	Where	What it does
Details tabs	Details	**Specimen**, **Waterfall**, **Suitability**, **Pairings**, **Features**, **Develop**, **Glyphs**, **Charset**, **Styles**, **About**, **License**, **Use & Embed**
Details: **Copy**, **Insert**, **Save**	Develop, Use & Embed	Use the chosen format
Details: **Download kit**	Styles, Use & Embed	Export a kit for this family
Details: **Load full license text**	License	Show the full licence

Keys inside Font Hub

Keys	What they do
`/` (outside a text field) or `Ctrl` + `K`	Focus and select the search box
`Escape`	Close the comparison, details or tray; otherwise clear the search
Arrow keys on a card	Move between cards
`Tab`	Stays within an open details panel, tray or comparison

Settings

Setting	Type	Default	Values or range	What it does
`auricFontLib.defaultFormat`	string	`auto`	`auto`, `link`, `import`, `family`, `fontFace`, `loader`, `tailwind`, `scss`, `tokens`, `name`	Format used by **Add Font to Code**; `auto` adds the CDN stylesheet and a class
`auricFontLib.previewText`	string	`The quick brown fox jumps over the lazy dog`	—	Default sample text on cards
`auricFontLib.previewSize`	number	`36`	18–96	Default sample size in pixels
`auricFontLib.density`	string	`comfortable`	`compact`, `comfortable`, `showcase`	Card density in the sidebar and studio
`auricFontLib.pageSize`	integer	`48`	24–144	Families shown per batch
`auricFontLib.recommendations.enabled`	boolean	`true`	—	Show recommendations from role scores, style intent and local use
`auricFontLib.network.enabled`	boolean	`true`	—	Allow requests to fonts.auricartisan.com for metadata, previews and downloads
`auricFontLib.catalogRefresh`	string	`daily`	`manual`, `daily`, `always`	When to refresh the catalogue from the CDN
`auricFontLib.cache.maxAgeHours`	integer	`24`	1–720	How long cached CDN metadata counts as fresh

Setting	Type	Default	Values or range	What it does
`auricFontLib.cache.maxMB`	integer	`512`	64–4096	Disk space in MiB for the cache; oldest items removed first
`auricFontLib.download.maxFamilies`	integer	`6`	1–12	Most families in one self-hosting kit
`auricFontLib.statusBar.enabled`	boolean	`true`	—	Show the **Fonts** status bar item
`auricFontLib.includeSystemFonts`	boolean	`false`	—	Include system-font metadata; previews depend on installed fonts

Format values

Value	Label	Output
`auto`	Automatic	CDN stylesheet plus `.auric-font-<id>` class, placed by file type
`link`	HTML stylesheet link	`<link rel="stylesheet" href="…/css/<id>.css">`
`import`	CSS @import	`@import url('…/css/<id>.css');`
`family`	CSS font-family	`font-family: '<Family>', <fallback>;`
`fontFace`	@font-face	`@font-face` rules per style
`loader`	AuricFonts loader	A script that loads the Auric font loader for the family
`tailwind`	Tailwind config	A `fontFamily` entry
`scss`	SCSS variable	`$font-<role>: '<Family>', <fallback>;`
`tokens`	Design tokens JSON	A typography token with family, fallback and CSS address
`name`	Family name	The family name only

Files Font Hub writes

File	When
The active document	**Add to code**, **Insert**, **Apply** (left unsaved)
A snippet file you name	**Save** in the details panel
A ZIP kit you name	**Download kit** and related commands
Cache	VS Code's extension storage, not your project

Output channel: **Auric Font Hub**.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Troubleshooting

Symptoms, causes and fixes for common Auric Artisan Font Hub problems.

For anything not listed, run **Auric Font Hub: Show Logs** and see Get support. Include your extension version, the file language and the error message with anything private removed.

Fonts are not visible on your website

- **Why:** the stylesheet is not loaded, the font file is blocked (often by CORS or a Content Security Policy), or the class is not applied.
- **Fix:** run **Check Font CDN and Browser CORS**, then check the browser's Network tab for both the stylesheet and the font file. A successful stylesheet response alone is not enough. See Check CDN and browser delivery.

An embed is present but the text has not changed

- **Why:** the setup only defines the `.auric-font-<id>` class.
- **Fix:** add the class to your markup, or use the family's `font-family` value in your own selector.

Nothing changed after inserting

- **Why:** the setup was already in the file (**... is already configured in this document.**), or the file type needs manual placement and the embed was copied to the clipboard instead.
- **Fix:** read the notification, then paste the embed where it belongs if it was copied.

"No HTML head found."

- **Why:** the HTML file has no `<head>` element, for example a partial or a component template.
- **Fix:** paste the copied embed into your page's head or your framework's layout.

"The document changed. Preview the font insertion again before applying."

- **Why:** the file was edited after the preview opened, so the preview is out of date.
- **Fix:** run **Preview** again and apply the new preview.

"Smart insertion supports documents up to 2 MiB."

- **Why:** the file is larger than automatic insertion handles.
- **Fix:** copy an embed and place it by hand.

"This is a local system font."

- **Why:** system fonts are not on the CDN, so there is nothing to link to.
- **Fix:** copy its family name and rely on it being installed, or choose a library font.

Previews are missing or say "Preview unavailable"

- **Why:** you are offline, `auricFontLib.network.enabled` is off, the font is not cached yet, or the file could not be fetched.
- **Fix:** connect, turn the network setting on, or wait and retry. Details, styles and licences still work from the bundled catalogue.

The catalogue refresh was rejected

- **Why:** the network failed, or the refreshed data did not pass validation.
- **Fix:** nothing is lost: **Auric Font Hub kept cached data** and the bundled snapshot stays available. New families may need an extension update.

"Self-hosting kit blocked."

- **Why:** a family's licence does not allow redistribution, permission could not be established, a required licence notice could not be loaded, or the family is a system font.
- **Fix:** remove the families named in the message and try again.

"Choose at most *N* families for one kit." or the 250 MB limit

- **Why:** the kit is limited by `auricFontLib.download.maxFamilies` (6 by default, up to 12) and to 250 MB in total.
- **Fix:** split the selection into several kits, or raise the setting.

"Select at least two font families to compare."

- **Fix:** select two to four families with the select button on each card, then **Compare**.

Duplicate commands or views

- **Why:** the older Font Library extension is still enabled.
- **Fix:** disable or uninstall it. See Moving from Font Library.

"Enable network access in Font Hub settings to run CDN checks."

- **Fix:** set `auricFontLib.network.enabled` to `true`.

A preview became stale

- **Why:** you edited the source file after opening the diff.
- **Fix:** reopen **Preview** after editing.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

FAQ

Short answers to common questions about Auric Artisan Font Hub.

How many fonts are there?

2,023 library families, 547 of them variable. With `auricFontLib.includeSystemFonts` on, metadata for common system fonts is added.

Does it work offline?

Browsing, searching, details and licences work offline from the bundled snapshot. Font previews and downloads need the network unless they are already cached.

Does it use Google Fonts?

No. Embeds point to `https://fonts.auricartisan.com`. Font Hub does not substitute another font service.

Will it change my existing CSS?

No. Automatic insertion only adds a stylesheet and a new class. It does not replace selected text, edit your rules or apply the font to the whole page. You can preview every change first, and one undo removes it.

Why did it copy an embed instead of inserting?

For JSX, TSX, Vue, Svelte, Astro, data files and other languages, the right place for a font depends on your framework, so Font Hub copies the embed for you to place. It also copies when no editor is open.

Does it create files in my project?

Opening Font Hub creates nothing. Files are only written when you insert into the active document, save a snippet, or save a kit.

Can I use the fonts commercially?

It depends on each font's licence. Check the **License** tab. The extension's own licence does not grant rights to any font.

Are the scores reliable?

They are catalogue estimates from measured traits, useful for shortlisting. Always test with your real text, languages, sizes and colours.

Is anything I do uploaded?

No. Search queries, your code, favourites and history stay on your computer. The CDN sees the ordinary information of each request, such as your IP address. There is no analytics and no account.

Why do settings start with `auricFontLib`?

The extension was previously called Font Library. The prefix is kept so existing settings and shortcuts keep working.

Is the interface available in Hindi?

Not at present; the interface is in English.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Supporting material

Contents of the supporting pages for Auric Artisan Font Hub.

These pages support the Font Hub guides: terms, shortcuts, what the scores and checks mean, privacy, related products and the version history.

File	What it covers
glossary.md	Typography and delivery terms used in the Font Hub documentation
shortcuts.md	Every keyboard shortcut
limits-and-accuracy.md	What scores, checks and insertion can and cannot guarantee; size limits
privacy.md	What stays local and what uses the network
related.md	Related extensions, tools and pages
release-notes.md	What changed in each version

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Shortcuts

The keyboard shortcuts Auric Artisan Font Hub provides in VS Code and inside its panels.

VS Code

Windows and Linux	macOS	Command
`Ctrl` + `Alt` + `F`	`Cmd` + `Alt` + `F`	**Auric Artisan Font Hub: Open Font Hub**
`Ctrl` + `Alt` + `T`	`Cmd` + `Alt` + `T`	**Auric Artisan Font Hub: Quick Pick Font**

Other commands have no default shortcut. Assign one in **Preferences: Open Keyboard Shortcuts** by searching for `auricFontLib`.

Inside the sidebar and studio

Keys	What they do
`/` (when not typing in a field)	Focus and select the search box
`Ctrl` + `K` (`Cmd` + `K` on macOS)	Focus and select the search box
`Escape`	Close the comparison, details panel or tray; otherwise clear the search
Arrow keys on a font card	Move to the next or previous card, or the card above or below
`Tab`, `Shift` + `Tab`	Move through controls; focus stays inside an open details panel, tray or comparison
Arrow keys, type-ahead, `Enter` in drop-downs	Choose a category, sort order, intent or industry

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Glossary

Plain-language definitions of the typography and delivery terms used in the Font Hub documentation.

@font-face — a CSS rule that tells the browser where to download a font file and which weight and style it provides.

Automatic insertion — Font Hub's default way of adding a font: a CDN stylesheet plus a reusable class, placed according to the file type.

CDN (content delivery network) — a server that delivers files such as fonts to websites. Font Hub uses `fonts.auricartisan.com`.

Charset / character coverage — which characters, grouped into Unicode ranges, a font can display.

CORS (cross-origin resource sharing) — the rule that lets a browser use a font from a different address only if that server allows it.

Content Security Policy — a website setting that limits where styles, fonts and scripts may load from.

Design intent — what the type has to do, such as UI and product text or editorial reading, used to rank recommendations.

Design tokens — named values, such as a font family and its fallbacks, shared across a design system.

Fallback stack — the list of fonts the browser tries if the chosen one is not available.

Family — a set of related fonts sharing one design, in several weights and styles.

Glyph — the drawn shape of a character in a font.

OpenType feature — an optional font behaviour switched on with a four-letter tag, such as `liga` for ligatures or `tnum` for tabular figures.

Pairing — two families that work well together, often a heading face and a body face.

Self-hosting — serving font files from your own website instead of a CDN.

Specimen — sample text set in a font to judge it.

Suitability score — a 0–100 catalogue estimate of how well a family suits a role such as accessibility, readability, UI, developer or editorial use.

System font — a font installed on a computer rather than served from the web.

Variable font — one font file with adjustable axes, such as weight, instead of a separate file per style.

Variable axis — one adjustable dimension of a variable font, such as `wght` (weight) or `wdth` (width).

Waterfall — the same text at a range of sizes.

Weight — how thick the strokes are, from 100 (thin) to 900 (black); 400 is regular and 700 is bold.

x-height — the height of lower-case letters such as x, which strongly affects readability at small sizes.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Limits and accuracy

What Font Hub's scores, recommendations, checks and insertion can and cannot guarantee, and its size limits.

Scores and recommendations

- Suitability and accessibility scores are catalogue estimates built from measured traits of the fonts, such as x-height and how distinct similar characters are. They are guidance for shortlisting, not guarantees.
- Categories are assigned automatically and can occasionally be wrong for unusual designs.
- Recommendations blend the chosen intent, industry, palette and your local use. They are a starting point.
- Always check your real text, languages, sizes and colour contrast in the finished product.

Previews

- Previews use the real font files, fetched through VS Code and cached. A family may show **Preview unavailable** when offline or when a file cannot be fetched.
- Variable-axis sliders and OpenType toggles only show what the preview font file supports.
- System-font previews depend on the font being installed on your computer.

Automatic insertion

- Supports HTML files with a `<head>`, CSS, SCSS, Less, PostCSS, JavaScript and TypeScript. Other languages get a copied embed.
- Works on files up to 2 MiB.
- It is deliberately conservative, not a complete HTML, CSS or JavaScript parser. It skips `<head>` tags inside comments, scripts and styles, and it will not add a setup twice.
- Explicit formats insert at the cursor; they do not merge into framework configuration files.
- Up to 12 families can be inserted together.

CDN check

- Checks one catalogue file, one stylesheet and one font file, with `HEAD` requests from the test address `http://localhost:5500`, with a 10-second timeout per request.
- It is a sample, not an audit of every family or of your production address.
- It reports what the CDN answers; it cannot change hosting settings or bypass browser security.

Self-hosting kits

Limit	Value
Families per kit	6 by default, up to 12 (`auricFontLib.download.maxFamilies`)
One font file	40 MB
Whole kit	250 MB

Kits are refused when redistribution permission or a required licence notice cannot be established. System fonts are never exported.

Catalogue

- 2,023 library families, 547 variable, in the bundled snapshot.
- A refresh only accepts validated data; new families may need an extension update.
- The cache is limited to `auricFontLib.cache.maxMB` (512 MiB by default); the oldest items go first.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Privacy

What Auric Artisan Font Hub keeps on your machine and what it sends over the network.

What stays on your machine

- Search queries, your source code, favourites, recents, selection and usage history. Recommendations that use "local use" are calculated on your computer.
- The cache of previews, font files and refreshed metadata, in VS Code's extension storage.
- No account, no analytics integration and no telemetry.

What uses the network

Font Hub contacts one address, `https://fonts.auricartisan.com`, and only for:

- refreshing the catalogue (on the `auricFontLib.catalogRefresh` schedule, or when you ask);
- fetching fonts for previews that are not cached;
- loading full licence texts;
- downloading self-hosting kits;
- the CDN check you run.

Requests go through VS Code, not through the Font Hub panels, and every address is validated first. The CDN receives the ordinary information any web request carries, such as your IP address.

Switch all of this off with `auricFontLib.network.enabled` set to `false`.

Your website

Embeds that Font Hub adds make your visitors' browsers load fonts from `fonts.auricartisan.com`. If you would rather your site made no requests to it, use a self-hosting kit instead.

Links

Explore Fonts on AuricArtisan.com and the **auricartisan.com** ↗ links open your browser.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Release notes

What changed in each published version of Auric Artisan Font Hub.

0.1.0 (24 September 2026)

The first release under the name **Auric Artisan Font Hub**, with the new extension ID `auric-artisan.auric-artisan-font-hub`.

A shared Auric workspace

- Renamed from Font Library to **Auric Artisan Font Hub**, with matching typography icons in the Marketplace and the Activity Bar.
- A redesigned studio and compact sidebar with editorial typography, gold accents, sticky navigation and responsive cards.
- **Add to code** and **Preview** buttons directly on font cards.
- Editor context-menu actions grouped under **Auric Fonts**.
- Custom drop-downs, the shared colour picker, keyboard navigation, collapsible filters and native scrolling kept.

CDN insertion you can review

- Automatic, non-duplicating CDN setup for HTML, CSS-family files and JavaScript and TypeScript.
- Reusable font classes that do not change existing styles or replace selected text.
- A native diff preview with **Apply** and **Cancel**, stale-document checks, and undoable, unsaved edits.
- A clipboard fallback for framework and other file types.
- Safe insertion of several families at once.
- Live CDN diagnostics for availability, content types and browser font CORS.
- Variable weight ranges kept in generated `@font-face` snippets.

Typography tools

- 2,023 library families, 547 variable, with bundled metadata.
- Search, filters, specimens, waterfalls, axes, features, glyphs, charset and licence inspection.
- Four-family comparison, recommendations, favourites, recents and selection.
- Manual embed formats, design tokens and licence-aware self-hosting kits.
- A bounded cache, network controls, validated HTTPS CDN access and offline metadata.

Compatibility

- Requires VS Code 1.85 or newer.
- Existing `auricFontLib.*` settings and command IDs are kept.
- Disable the old Font Library extension before enabling Font Hub. Its favourites and cache are not moved automatically.
- Browser delivery needs correct CORS headers from the CDN; the extension does not change hosting settings or bypass browser restrictions.

VS CODE EXTENSIONS › AURIC ARTISAN FONT HUB

Related

Other Auric Artisan extensions, tools and pages that work alongside Font Hub.

On the Auric Artisan website

- The same font library in a browser: https://auricartisan.com/tool/font/ — see also Font Library.

- The font CDN service: Font CDN.

Other Auric Artisan VS Code extensions

- Auric Artisan Icon Hub — the icon counterpart: search open-source icons and insert SVG.
- Auric Artisan Live — serve the page and check the font in a real browser.
- Auric Artisan Studio — colour and contrast checks for the text you set.

Setup

- Install and set up the Auric Artisan extensions

Support

- Marketplace listing: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-font-hub
- Contact and support: https://auricartisan.com/company/contact/

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Auric Artisan Formatter

What Auric Artisan Formatter does, who it is for, how to install it and where to find every guide.

Auric Artisan Formatter formats your code in VS Code. It rebuilds JavaScript, TypeScript, JSON, CSS, HTML, Markdown, YAML and more from a real parse, and tidies the indentation and spacing of hundreds of other languages with a conservative structural engine. Every result passes a *safety gate* before it reaches your file: the output is re-read and compared with the input, and if the formatter cannot prove the change keeps your code's meaning, it keeps your original text.

It is built for adopting a formatter without surprises. **Preview Formatting as a Diff** shows the result beside your source before anything changes. A **workspace scan** formats every file in memory when the window opens and tells you how many would change, without writing anything. **Formatter Studio** lets you try the 391 options against your own code, find the few that matter for a file, and copy only the ones you changed into a configuration file. The same engine runs from a command line for CI checks.

It reads existing Prettier configuration and `.editorconfig`, offers 13 presets (including one byte-compatible with Prettier 3), and has no runtime dependencies.

Who it is for

- Developers who want consistent formatting with a preview before changes land.
- Teams adopting a formatter on an existing codebase who want to see the impact first and keep `git blame` useful.
- Teams moving from Prettier who want the same output on day one.
- Anyone working in less common languages that other formatters do not cover.

Where to find it

- Marketplace ID: `auric-artisan.auric-artisan-formatter`
- Marketplace page: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-formatter
- Install from a terminal:

```
code --install-extension auric-artisan.auric-artisan-formatter
```

- Requires VS Code 1.74 or newer. Version 0.1.0 at the time of writing. The interface is in English.

Quick start

1. Install the extension.
2. Run **Auric Formatter: Toggle Format on Save**. It makes Auric the default formatter and turns on format on save.
3. Open a file and run **Auric Formatter: Preview Formatting as a Diff** (`Ctrl` + `Alt` + `Shift` + `F`). Review the read-only result beside your source.
4. Select **Apply formatting** in the editor title or the CodeLens, or **Discard preview**.
5. Open **Auric Formatter: Open Formatter Studio** (`Ctrl` + `Alt` + `Shift` + `P`) to tune options against your own file, then **Copy config**.
6. Paste the configuration into a `.auricformatrc` file at the root of your project.

7. Before formatting a whole existing repository, check the status bar count from the workspace scan, or run **Auric Formatter: Scan the Workspace Without Writing**.
8. Use **Show Files That Need Formatting** (`Ctrl` + `Alt` + `Shift` + `U`) to review and format files a folder at a time.

What you can do

- Format a document, a selection or every file that needs it; format on save, or only the lines you changed.
- Preview formatting as a read-only document beside your source, with Apply, Discard, Refresh and Compare.
- Tune 391 options live in Formatter Studio, see the diff and an explanation of each decision, and copy only what you changed.
- Scan the workspace in memory, see which files would change and by how much, and format selected files explicitly.
- Check formatting without writing, reporting into the Problems panel, and run the same check in CI from the command line.
- Start from one of 13 presets, import Prettier settings, generate a configuration file.
- Configure per language and per file pattern, with `.editorconfig` and inline directives.
- See why a file was formatted as it was with **Explain Formatting Decisions**.
- Format 253 languages: 31 natively, the rest with the structural engine.

In this folder

- docs/ — task guides and the full reference.
  - Getting started
  - Format files and format on save
  - Preview formatting as a diff
  - Formatter Studio
  - Scan the workspace
  - Configuration
  - Presets
  - The safety gate and formatting tiers
  - Supported languages
  - Command line and CI
  - Reference: commands, keys and settings
  - Options reference
  - Troubleshooting
  - FAQ
- others/ — supporting material.
  - Glossary
  - Shortcuts
  - Limits and accuracy
  - Privacy
  - Related
  - Release notes

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Auric Artisan Formatter guides

Contents of the Auric Artisan Formatter task guides and references.

Start with the getting-started guide. The reference lists the 21 commands, 3 keyboard shortcuts and 11 editor settings; the options reference lists all 391 formatting options.

File	What it covers
getting-started.md	Install, make it the default formatter, the screen tour, the setup walkthrough
format-and-format-on-save.md	Format a document or selection, format on save, only modified lines, format on type, language lists
preview-diff.md	The native read-only preview with Apply, Discard, Refresh and Compare
formatter-studio.md	Studio and the Explorer sidebar: scope, options rail, Find what matters, diff, explain, custom rules
workspace-scan.md	The in-memory scan, the Workspace tab, formatting selected or all files, CI mode in Problems
configuration.md	Configuration files, precedence, overrides, `.editorconfig`, Prettier import, directives, ignore files
presets.md	The 13 presets and what each changes
safety-gate-and-tiers.md	The five safety checks, native and universal tiers, Explain Formatting Decisions
supported-languages.md	Every language by tier
command-line.md	The `auric-format` command line, flags and exit codes
reference.md	Commands, keybindings, menus, status bar and editor settings
options-reference.md	All 391 formatting options
troubleshooting.md	Symptom, cause and fix
faq.md	Short answers to common questions

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Getting started with Auric Artisan Formatter

Install Auric Artisan Formatter, make it your formatter, learn where its controls are, and preview a first format.

Install

1. Check that VS Code is version 1.74 or newer (**Help** › **About**).
2. In the Extensions view (`Ctrl` + `Shift` + `X`), search for `@id:auric-artisan.auric-artisan-formatter` and select **Install**. Or run:

   ```
   code --install-extension auric-artisan.auric-artisan-formatter
   ```

The extension starts when VS Code opens and offers formatting for every language it supports.

Make it your formatter

VS Code uses one formatter per language. If you have several installed, it asks which to use every time. Tell it to use Auric:

- Run **Auric Formatter: Toggle Format on Save**. It sets `editor.defaultFormatter` to `auric-artisan.auric-artisan-formatter` and turns on `editor.formatOnSave`, in the workspace settings when a folder is open (otherwise in user settings). Run it again to turn format on save off.
- Or add the settings yourself:

```
{
  "editor.defaultFormatter": "auric-artisan.auric-artisan-formatter",
  "editor.formatOnSave": true
}
```

To use it only for some languages, scope the settings:

```
{
  "[typescript]": {
    "editor.defaultFormatter": "auric-artisan.auric-artisan-formatter",
    "editor.formatOnSave": true
  }
}
```

The setup walkthrough

VS Code's **Welcome** page (**Help** › **Welcome**) includes **Set up Auric Artisan Formatter**, five steps with buttons:

1. **Choose a preset** — **Switch preset**.
2. **Try the Playground** — **Open Playground** (Formatter Studio).
3. **See what the whole repository would do** — **Show what needs formatting**.
4. **Turn on format on save** — **Toggle format on save**.
5. **Understand the safety gate** — **Explain this file**.

The screen tour

Status bar

Two items on the right:

Item	Meaning	Select it to
Auric with a tick	The last format ran; the tooltip names the tier (native or universal) and the time taken, or says the file was already formatted	Open the log
Auric with a shield, warning background	The safety gate reverted a change; your original text was kept	Open the log
Auric with a slash	The file was skipped (for example, it looks minified)	Open the log
Auric with an error, error background	Formatting failed	Open the log
***N* unformatted**, warning background	The workspace scan found *N* files that would change; nothing has been written	Review them in Formatter Studio
Auric with a double tick	Every scanned file is already formatted	Open the list
Auric *scanned/total*, spinning	The scan is running in memory	—

The first item appears for languages Auric supports. Hide it with `auricFormatter.showStatusBar`.

Formatter Studio and the Explorer sidebar

- **Auric Formatter: Open Formatter Studio** (`Ctrl` + `Alt` + `Shift` + `P`) opens the full studio as an editor tab.
- The **Auric Formatter** section in the Explorer is a compact version, with **Open full Studio**.

Both have two tabs: **01 Studio** (try options on your code) and **02 Workspace** (the scan results). See Formatter Studio.

Editor menus

- **Editor title bar: Preview Formatting as a Diff** for supported files. While a preview is open: **Apply Formatting Preview**, **Compare Formatting Changes**, **Refresh Formatting Preview** and **Discard Formatting Preview**.
- **Editor context menu: Format Selection** when text is selected, otherwise **Preview Formatting as a Diff**.
- **Explorer context menu on a folder: Format Every File in the Workspace**.

A first format with a preview

1. Open a JavaScript, TypeScript, CSS or JSON file.
2. Press `Ctrl` + `Alt` + `Shift` + `F` (**Preview Formatting as a Diff**).
3. A read-only tab, **Preview — *file name***, opens beside your source with the formatted result.
4. Use the CodeLens at the top: **Apply formatting**, **Compare changes**, **Discard preview** or **Full file · refresh preview**.
5. Choose **Apply formatting**, then confirm. Your source is updated and left unsaved; `Ctrl` + `Z` undoes it.

You now have a formatted file and have seen exactly what changed before it happened.

Next steps

- Format files and format on save
- Formatter Studio
- Scan the workspace
- Configuration

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Command line and CI

Run the same Auric formatting engine from a terminal with auric-format, check formatting in CI, and use its flags and exit codes.

`auric-format` runs the same engine, options, configuration files and safety gate as the extension, from a terminal. Use it to check formatting in continuous integration (CI) or to format many files in a script.

Where it is

The command line ships inside the installed extension as the script `bin/auric-format.js`. It needs Node.js 16 or newer. It is not published as a separate npm package at the time of writing.

VS Code installs extensions in a folder named after the extension and version, for example:

```
~/.vscode/extensions/auric-artisan.auric-artisan-formatter-0.1.0/bin/auric-format.js
```

On Windows, `~` is your user folder (`%USERPROFILE%`). Run it with Node:

```
node ~/.vscode/extensions/auric-artisan.auric-artisan-formatter-0.1.0/bin/auric-format.js --check .
```

The examples below write `auric-format` for short. To use the bare name, create a shell alias or a small wrapper script that runs the command above. A CI machine needs the extension's files available in the same way; the engine itself has no runtime dependencies beyond Node.js.

Common tasks

```
auric-format --write src             # rewrite files in place
auric-format --check .               # exit 1 if anything is unformatted
auric-format --preset prettier -w .  # start from a preset
cat a.css | auric-format --stdin-filepath a.css
auric-format --explain src/weird.scss # why did it do that?
```

Flags

Flag	What it does
`-w`, `--write`	Rewrite each file in place
`-c`, `--check`	Report which files need formatting, change nothing, exit non-zero if any do
`-l`, `--list-different`	Print the names of files that differ from the formatted output
`--stdin-filepath <path>`	Treat standard input as this path, to pick a language and find configuration
`--config <path>`	Use this configuration file instead of searching
`--no-config`	Do not look for a configuration file
`--ignore-path <path>`	Use this ignore file instead of `.auricformatignore`
`--no-ignore`	Do not read any ignore file
`--no-editorconfig`	Do not read `.editorconfig`
`--preset <name>`	Start from a named preset
`--parser <name>`	Force a parser instead of inferring one
`--explain`	Explain the formatting decisions for each file instead of formatting
`--support-info`	Print the languages, parsers and options this build supports, as JSON
`--list-languages`	Print every supported language with its tier and extensions
`--list-presets`	Print every preset with its description
`--find-config`	Print the configuration file that would be used for the given path
`--loglevel <level>`	`silent`, `error`, `warn`, `log` or `debug` (default `log`)
`--concurrency <n>`	How many files to format at once (default: the number of processors)
`-v`, `--version`	Print the version

Flag	What it does
`-h`, `--help`	Print help

`--check`, `--write`, `--list-different`, `--stdin-filepath`, `--config`, `--ignore-path` and `--no-config` behave as they do in Prettier's command line.

Formatting options as flags

Every one of the 391 options is also a flag, written in kebab case: `printWidth` is `--print-width`. Booleans take a `--no-` form to switch them off. For example:

```
auric-format --print-width 100 --single-quote --trailing-comma es5 --no-semi -w src
```

A mistyped flag suggests the one you probably meant. See the options reference for every option.

Exit codes

Code	Meaning
`0`	Every file is formatted (or was formatted successfully)
`1`	Files need formatting, or one failed
`2`	A usage error, such as an unknown flag

Use it in CI

Add a step that runs the check and fails the build when anything is unformatted:

```
node path/to/auric-format.js --check .
```

Output is deterministic: no clock, no randomness and no locale-dependent ordering, so the same input and configuration always give the same result on every machine.

Inside VS Code, **Auric Formatter: Check Formatting Without Writing (CI Mode)** runs the same check and lists the results in the Problems panel.

When the check passes, the command exits with code `0` and reports nothing to fix.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Configuration

Configure Auric Artisan Formatter with presets, configuration files, per-language and per-file overrides, .editorconfig, Prettier settings, inline directives and ignore files.

The formatter has 391 options (see the options reference). You rarely need more than a handful: start from a preset and set only what you disagree with.

Where options come from

From lowest to highest precedence:

1. **A preset** — `"preset": "prettier"` and twelve others. See Presets.
2. **A configuration file** — see the list below.
3. `.editorconfig` — for the options it can express.
4. **VS Code settings** under `auricFormatter.*`, a command-line flag, or an inline directive in the file itself.

Inside a configuration file, `languages` (by language id) and `overrides` (by file pattern) beat the file's top-level values.

Auric Formatter: Explain Formatting Decisions for This File shows which layer set each option for the current file.

Configuration files

The formatter looks for the first of these files, searching from the file's folder upward. Within a folder, this order is the precedence:

File	Format
`.auricformatrc`	JSON or YAML, detected automatically
`.auricformatrc.json`	JSON
`.auricformatrc.yaml`, `.auricformatrc.yml`	YAML
`.auricformatrc.js`, `.auricformatrc.cjs`	JavaScript (CommonJS)
`.auricformat.config.js`, `.auricformat.config.cjs`	JavaScript (CommonJS)
`auric.format.json`	JSON
`package.json`	Under an `auricFormatter` (or `auric-formatter`) key
`.prettierrc`, `.prettierrc.json`, `.prettierrc.yaml`, `.prettierrc.yml`, `.prettierrc.js`, `prettier.config.js`	Prettier configuration, read for compatibility
`package.json`	Under a `prettier` key

Auric files come first, so adding one to a Prettier project is additive; Prettier files come last, so an existing project keeps working untouched. JSON configuration files get validation and completion in VS Code for `.auricformatrc`, `.auricformatrc.json` and `auric.format.json`.

JavaScript configuration files are not run in an untrusted workspace. A JavaScript file written as an ES module must be renamed to `.cjs`, converted to CommonJS, or replaced with `.auricformatrc.json`.

An example

```
{
  "preset": "prettier",
  "printWidth": 100,
  "languages": {
    "markdown": { "proseWrap": "always" }
  },
  "overrides": [
    { "files": "**/*.test.ts", "options": { "printWidth": 200 } }
  ]
}
```

- `preset` — the starting point.
- Top-level options — apply to every file.
- `languages` — options for one language id.
- `overrides` — options for files matching a glob.

Create a configuration file

- **Auric Formatter: Generate a Configuration File** writes `.auricformatrc` in the first workspace folder, with your current preset and the options you have set in VS Code settings, and opens it.
- **Copy config** in Formatter Studio copies only the options you changed, ready to paste.

Choose a preset

- **Auric Formatter: Switch Formatting Preset** — pick from the 13 presets (**Every option you set yourself still wins over the preset**). It saves `auricFormatter.preset` in the workspace settings (or user settings with no folder open) and re-runs the workspace scan.
- Or write `"preset": "<name>"` in a configuration file; that wins over the VS Code setting.
- Or put `@auric-preset <name>` in a file's leading comment for that file.

Coming from Prettier

If a project only has Prettier configuration, the formatter reads it and adopts the `prettier` preset automatically, so the output matches Prettier 3 at its defaults with every Auric-only enhancement off.

Auric Formatter: Import Settings From Prettier finds the first Prettier configuration in the workspace (outside `node_modules`) and opens an unsaved JSONC document containing the equivalent Auric configuration, starting from the `prettier` preset, with comments. Save it as `.auricformatrc` when you are happy. If there is no Prettier configuration: **No Prettier configuration found in this workspace.**

`.prettierignore` and Prettier's `prettier-ignore` comments are honoured too.

.editorconfig

With `auricFormatter.editorconfig` on (the default), the formatter reads `.editorconfig` and uses `indent_style`, `indent_size`, `end_of_line`, `max_line_length` and the final-newline settings.

VS Code settings

Every option is also a VS Code setting, `auricFormatter.<option>`, with resource scope, so it can be set for the user, the workspace or one folder of a multi-root workspace. Search Settings for `auricFormatter`.

Inline directives

Put directives in a comment, in the file's own comment syntax. File-level directives only count at the top of the file, before any code (a byte order mark, shebang, XML declaration, front matter and earlier comments are allowed before them). A directive must be the whole content of its comment line, optionally followed by a reason, so a comment that merely mentions one does nothing.

Directive	Effect
`auric-format-ignore`	Leave the next node unformatted
`auric-format-ignore-file`	Leave the whole file unformatted
`auric-format-ignore-start` … `auric-format-ignore-end`	Leave everything between unformatted
`auric-format-off` … `auric-format-on`	The same, other spelling
`@auric printWidth=120 singleQuote`	Options for this file only
`@auric-preset compact`	A preset for this file only
`@format`	Marks the file as opted in when `requirePragma` is on

Each directive also works with the short `auric` spelling and the `prettier` spelling (such as `prettier-ignore`). `@auric-format`, `@auric` and `@prettier` also mark a file as opted in.

Related options: `auricFormatter.respectIgnoreComments` (honour ignore comments; on by default), `auricFormatter.requirePragma` (format only files with the pragma), `auricFormatter.insertPragma` (add `@format` to formatted files) and `auricFormatter.pragmaKeyword`.

A malformed option value in a directive becomes a warning; the directive is dropped and the file still formats.

Ignore files

- `.auricformatignore` uses `.gitignore` syntax. Files it matches are not formatted by the editor, the scan or the command line.
- `.prettierignore` is honoured too.
- The command line can use a different file with `--ignore-path`, or none with `--no-ignore`.

Plugins

`auricFormatter.plugins` lists additional formatter plugins by module path or registered name. Plugin languages go through the same option handling and safety gate as the built-ins, and a plugin that fails disables itself with a warning. Plugins are not run in an untrusted workspace.

Clear the cache

The formatter caches configuration it has read. If a change to a configuration file does not seem to apply, run **Auric Formatter: Clear the Configuration Cache** (**Auric Formatter: configuration cache cleared.**).

When your configuration is in place, **Explain Formatting Decisions** for any file lists your file as the source of the options you set.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Format files and format on save

Format a document or selection, format on save (including only changed lines), format as you type, and choose which languages Auric formats.

Format a document

- Run **Auric Formatter: Format Document**. It uses Auric for this format even if another formatter is your default.
- Or use VS Code's own **Format Document** (`Shift` + `Alt` + `F` on Windows) once Auric is your default formatter.

The status bar reports the result: the tier used, whether the file was already formatted, whether the safety gate kept your original text, or whether the file was skipped.

Format a selection

1. Select the code.
2. Run **Auric Formatter: Format Selection**, or right-click and choose **Format Selection**. With nothing selected: **Select some code first.**

A selection may grow to a complete syntax unit (for example a whole statement or block) so the result stays valid.

Format on save

1. Run **Auric Formatter: Toggle Format on Save**. The message says **Auric Formatter is now your default formatter and runs on save.**
2. Save a file. It is formatted first.

Running the command again turns format on save off (**Format on save is off.**).

Only the lines you changed

Reformatting a whole file on its first save in a large codebase makes one huge diff and clutters `git blame`. Set:

```
{ "auricFormatter.formatOnSaveMode": "modifications" }
```

Format on save then formats only the lines changed since the last commit, using Git. The default, `file`, formats the whole file. The `minimal-diff` preset is another way to keep changes small; see Presets.

If another formatter is installed

VS Code does not guess between formatters; it asks on every save. Setting `editor.defaultFormatter` stops that. **Toggle Format on Save** sets it for you.

Format as you type

Set `auricFormatter.formatOnType` to `true` and VS Code's `editor.formatOnType` to `true`. Typing a closing character (`;`, `}`, `)`, `]`) or a new line then reformats the current line and its enclosing block.

Choose languages

Setting	Default	Effect
`auricFormatter.enabledLanguages`	`[]`	Format only these VS Code language ids; empty means every supported language
`auricFormatter.disabledLanguages`	`[]`	Never format these language ids, even when Auric is the default formatter

For example, to leave Markdown alone:

```
{ "auricFormatter.disabledLanguages": ["markdown"] }
```

Format many files

Command	What it does
Format Every File That Needs It	Writes every file the workspace scan found would change
Format Every File in the Workspace	The same; also on a folder's context menu in the Explorer. In this version it formats every pending file in the workspace, not only that folder

Both ask **Format *N* files?** first (while `auricFormatter.confirmBeforeFormattingAll` is on): **This rewrites files on disk. Commit or stash your work first.** Each file is re-read and re-formatted at the moment it is written, so a file you edited since the scan is never overwritten with a stale result, and open files with unsaved changes are written through the editor. These commands have no default shortcut on purpose. See Scan the workspace to review first and format a folder at a time.

Files that are skipped

A file is left alone when:

- it is larger than `auricFormatter.maxFileSizeKb` (4096 KB by default; `0` removes the limit);
- it looks minified, unless `auricFormatter.formatMinified` is on;
- it is listed in `.auricformatignore` or `.prettierignore`;
- it contains an ignore-file directive, or `auricFormatter.requirePragma` is on and it has no `@format` pragma.

The status bar shows **Skipped** with the reason, such as **Skipped — looks minified** or **Skipped — larger than maxFileSizeKb**.

A parse that runs longer than `auricFormatter.parseTimeoutMs` (5000 ms by default) is abandoned.

When formatting works, the status bar shows **Auric** with a tick, and its tooltip names the tier that formatted the file.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Formatter Studio

Use Formatter Studio and the Explorer sidebar to try options on your own code, see the diff and the reasons, and export a configuration.

Formatter Studio (the setup walkthrough calls it the Playground) puts your code on one side, the formatted result on the other, and every option that applies in between. Change an option and the result updates as you type. Nothing you do in Studio touches your file until you explicitly apply it.

Open it

- **Auric Formatter: Open Formatter Studio** — `Ctrl` + `Alt` + `Shift` + `P` (`Cmd` + `Alt` + `Shift` + `P` on macOS). Opens as an editor tab with the active editor's content loaded.
- The **Auric Formatter** section in the Explorer — a compact version of the same tools, with **Open full Studio**.
- **Show Files That Need Formatting** (`Ctrl` + `Alt` + `Shift` + `U`) opens Studio on its **Workspace** tab.

The panel follows your VS Code theme (light, dark and high contrast), respects reduced motion, and remembers its state when you reload.

The layout

Top bar

Control	What it does
Brand line	The number of options and languages, and the version
auricartisan.com ↗	Open the Auric Artisan website
Lang	The language to format the input as; searchable, grouped into **Native printers** and **Universal engine**
Preset	The preset to start from; searchable, each with a description
Commands	The Studio command palette (also `Ctrl` + `K`)
Layout button	Show or hide the options rail

Tabs and actions

Control	What it does
01 Studio	Try options on your code
02 Workspace	The workspace scan results, with a count badge. See Scan the workspace
Explain	Explain how this file would be formatted and which layer set each option
Copy config	Copy the options you changed, with the preset, as `.auricformatrc` content
Yes, apply	Apply the reviewed preview to its original document, after confirmation
No, discard	Discard the preview; no file changes

Workflow row

Control	What it does
Preview beside editor	Open the native read-only preview for the source, using the saved project rules. See Preview formatting as a diff
Formatting scope	**Full document**, or **Editor selection** (select code in VS Code first; the range may expand to a balanced syntax unit)
Formatting preview	On: show the formatted result, diff or explanation. Off: show the original
Reload editor	Load the active editor's content again
Open full Studio	In the sidebar only: open the full Studio tab

Custom rules

Custom rules · JSON configuration is a collapsible editor where you type options as a JSON object, for example:

```
{ "printWidth": 100, "singleQuote": true }
```

- **Validate & preview rules** checks every name and value against the option catalogue (an unknown name is refused with **Unsupported rule**) and previews the result.
- **Load current rules** fills the editor with the options currently set in Studio.

No code runs from the rules editor; it only accepts catalogue options.

The Studio tab

Input

Your source code. It opens with the active editor's content; **Paste code here, or open a file and reopen the Playground.** when empty. The pane has **Clear the input** and **Reload from the active editor**.

Options

The options rail lists every option that applies to the chosen language.

- **Search** the options by name or by what they do.
- **Group** filters by category: Global, Common, JavaScript, TypeScript, CSS, HTML, Markdown, YAML, JSON, GraphQL, Universal, Alignment, Sorting, Comments, Blank Lines, Safety, Experimental.
- Filter chips: **Changed** (only options you moved off the preset), **Affects this file** (after **Find what matters**) and **Experimental** (include options still settling).
- **Find what matters** formats the file once per option and reports the few that actually change *this* file. It can take a moment: **Probing every option against this file…**.
- **Reset every option to the preset default.**
- Each option can be opened in VS Code Settings with **Open this option in Settings**.

Output

Choose the view:

View	Shows
Result	The formatted file

View	Shows
Diff	Only what changes, with added and removed lines counted
Explain	Which language and parser were chosen, the formatting tier, timing, which configuration layer set each option, changed options and safety warnings

In **Diff**: **Inline** or **Side by side**, **Show unchanged lines** (or collapse them), **Show more diff lines** for long diffs, and **Previous change** and **Next change**. The output can be copied, and soft wrap toggled from the command palette.

The status line at the bottom shows what Studio is doing, such as **Formatting…** or **Ready**, and warnings such as a safety revert.

Apply the result to your file

1. Load the file with **Reload editor** (or open Studio from it).
2. Tune the options and review the **Diff**.
3. Choose **Yes, apply**.
4. Confirm **Apply reviewed formatting to *file*?** — **Only this document is edited. Changes remain unsaved and can be undone.**

Apply only works on the document the preview came from, and only if it has not changed since: otherwise **The file changed since this preview. Reload from the editor and review again.** Scratch text you typed yourself, and previews that were truncated, can be copied but not applied.

Save your configuration

1. Choose **Copy config**. Studio copies JSON with your preset and only the options you changed, not all 391: **Configuration copied — paste it into .auricformatrc.**
2. Create `.auricformatrc` at the root of your project and paste.

See Configuration for where the file can live and how it combines with other settings.

The Studio command palette

Press `Ctrl` + `K` (or select **Commands**) and type:

Command	What it does
Format every file that needs it	Write every pending file to disk (asks first)
Scan the workspace	Format every file in memory again
Show the workspace / **Show the playground**	Switch tabs
Find the options that change this file	Run **Find what matters**
Explain this file	Show the explanation
Show the diff / **Show the result**	Switch the output view
Copy the options as a config file	Same as **Copy config**
Apply the output to the editor	Same as **Yes, apply**
Reset every option	Back to the preset defaults

Command	What it does
Toggle the options rail	Show or hide the rail
Toggle soft wrap	Wrap long lines in the panes
Open the formatter log	Show the output channel
Open the formatter settings	Open VS Code Settings

Limits

- Studio previews are limited to 200,000 characters: **Preview is limited to 200,000 characters. Use editor formatting for larger files.** Larger files can still be formatted in the editor, up to `auricFormatter.maxFileSizeKb`.
- Native printers and the universal engine are labelled separately. Supporting a language does not mean every option applies to it.

When you are happy with a result, **Copy config** gives you a configuration that reproduces it for the whole project.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Options reference

All 391 Auric Artisan Formatter formatting options in 17 categories, with type, default, allowed values and effect.

Auric Artisan Formatter has 391 formatting options in 17 categories. This page lists every one. The eleven settings that control how the extension works in VS Code (preset, format on save, the workspace scan and so on) are in the reference.

How to use this page

Each option can be set in four places. The same name works in all of them:

Where	How it is written
VS Code settings	`"auricFormatter.printWidth": 100`
A configuration file such as `.auricformatrc`	`"printWidth": 100` (no prefix)
The command line	`--print-width 100` (the name in kebab case; booleans also take a `--no-` form)
An inline directive in a file	`// @auric printWidth=100`

The defaults shown are the extension's own defaults. A preset changes many of them; `"preset": "prettier"`, for example, switches every Auric-only enhancement off. Anything you set yourself wins over the preset. See Configuration and Presets.

Options that only apply to some languages only affect files in those languages. Formatter Studio's **Affects this file** filter shows which options change the file you are looking at.

Options marked "Experimental" are still settling; Formatter Studio hides them unless you tick **Experimental**.

Global

21 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.byteOrderMark`	string	`preserve`	`preserve`, `add`, `remove`	Decide what happens to a UTF-8 byte order mark at the start of the file.
`auricFormatter.cursorOffset`	integer	`-1`	-1–no maximum	Track this character offset through the reformat and report where it lands.
`auricFormatter.editorconfig`	boolean	`true`		Read .editorconfig and translate indent_style, indent_size, end_of_line, max_line_length and the final-newline keys.
`auricFormatter.embeddedLanguageFormatting`	string	`auto`	`auto`, `off`	Decide whether code embedded inside another language is formatted.
`auricFormatter.endOfLine`	string	`lf`	`lf`, `crlf`, `cr`, `auto`, `preserve`	Choose the line ending written to disk.
`auricFormatter.filepath`	string	empty		Supply the path the text came from so language inference and glob overrides can work on stdin.
`auricFormatter.formatEmbeddedUnknown`	boolean	`false`		Attempt the universal engine on embedded fragments whose language has no native printer.
`auricFormatter.insertFinalNewline`	boolean	`true`		Guarantee the file ends with exactly one line ending.
`auricFormatter.insertPragma`	boolean	`false`		Insert an @format pragma into the leading comment of files that were formatted.
`auricFormatter.parser`	string	empty		Force a specific parser instead of inferring one from the file path or language id.
`auricFormatter.plugins`	array	`[]`		List additional printer plugins to load, by module path or by registered plugin name.
`auricFormatter.pragmaKeyword`	string	`format`		Set the bare word an opt-in pragma comment must contain, without the leading at sign.
`auricFormatter.printWidth`	integer	`80`	0–1000	Wrap lines that would otherwise exceed this many columns wherever the syntax allows a break.
`auricFormatter.rangeEnd`	integer	end of file	0–no maximum	Format only the text before this character offset.
`auricFormatter.rangeStart`	integer	`0`	0–no maximum	Format only the text at or after this character offset.
`auricFormatter.requirePragma`	boolean	`false`		Format only files whose leading comment carries an @format or @auric-format pragma.
`auricFormatter.respectIgnoreComments`	boolean	`true`		Honour auric-ignore, auric-ignore-start and auric-ignore-end comments and leave those regions untouched.
`auricFormatter.tabWidth`	integer	`2`	0–24	Set how many columns one level of indentation occupies.
`auricFormatter.tabWidthByLanguage`	array	`[]`		Override `tabWidth` for individual languages with `languageId=width` entries such as `python=4`.
`auricFormatter.trimTrailingWhitespace`	boolean	`true`		Strip whitespace at the end of every line outside of strings and preserved blocks.
`auricFormatter.useTabs`	boolean	`false`		Indent with hard tab characters instead of spaces.

Notes and language-specific defaults:

- `auricFormatter.printWidth`: `120` for java, kotlin, scala, csharp, swift.
- `auricFormatter.tabWidth`: `4` for python, php, java, kotlin, csharp, c, cpp, objective-c, objective-cpp, rust, swift, groovy, solidity, apex; `4` for go, gomod, gowork, makefile, gotmpl.
- `auricFormatter.useTabs`: `true` for go, gomod, gowork, makefile, gotmpl.

Common

15 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.arrowParens`	string	`always`	`always`, `avoid`	Choose whether a single arrow function parameter is parenthesised.
`auricFormatter.bracketSameLine`	boolean	`false`		Put the closing angle bracket of a multi-line opening tag on the last attribute line.
`auricFormatter.bracketSpacing`	boolean	`true`		Print spaces just inside object literal braces, as in `{ foo: 1 }`.
`auricFormatter.curlyBraces`	string	`preserve`	`preserve`, `all`, `multi-line`	Decide whether single-statement control-flow bodies get braces.
`auricFormatter.hexCase`	string	`lower`	`preserve`, `lower`, `upper`	Choose the letter case of hexadecimal digits in numeric literals and colour values.
`auricFormatter.htmlWhitespaceSensitivity`	string	`css`	`css`, `strict`, `ignore`	Choose how strictly whitespace around HTML elements is preserved.
`auricFormatter.jsxSingleQuote`	boolean	`false`		Prefer single quotes over double quotes inside JSX attributes.
`auricFormatter.objectWrap`	string	`preserve`	`preserve`, `collapse`	Decide whether the author's first newline inside an object forces it to stay expanded.
`auricFormatter.proseWrap`	string	`preserve`	`preserve`, `always`, `never`	Choose how prose paragraphs are wrapped.
`auricFormatter.quoteProps`	string	`as-needed`	`as-needed`, `consistent`, `preserve`, `consistent-as-needed`	Choose when object property keys are quoted.
`auricFormatter.semi`	boolean	`true`		Print a semicolon at the end of every statement.
`auricFormatter.singleAttributePerLine`	boolean	`false`		Put every attribute on its own line as soon as a tag has to break.
`auricFormatter.singleQuote`	boolean	`false`		Prefer single quotes over double quotes for strings.
`auricFormatter.trailingComma`	string	`all`	`all`, `es5`, `none`	Choose where trailing commas are added in multi-line constructs.
`auricFormatter.vueIndentScriptAndStyle`	boolean	`false`		Indent the contents of the top-level `<script>` and `<style>` blocks in a single file component.

JavaScript

60 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.arrayBracketSpacing`	boolean	`false`		Print spaces just inside array brackets, as in `[ 1, 2 ]`.
`auricFormatter.arrayElementsPerLine`	integer	`0`	0–100	Print at most this many array elements per line, or 0 to fit as many as the print width allows.
`auricFormatter.arrowBodyStyle`	string	`preserve`	`preserve`, `as-needed`, `always`	Choose whether arrow functions keep braced bodies or collapse to expressions.
`auricFormatter.arrowChainBreak`	string	`as-needed`	`as-needed`, `always`, `never`	Choose how chains of curried arrow functions are broken.
`auricFormatter.assignmentLayout`	string	`auto`	`auto`, `break-after-operator`, `break-lhs`, `chain`, `never-break-after-operator`	Choose how the two sides of an assignment are split when the line is too long.
`auricFormatter.awaitParens`	string	`as-needed`	`as-needed`, `always`	Choose when the operand of `await` is wrapped in parentheses.
`auricFormatter.breakBeforeBinaryOperators`	boolean	`false`		Start the new line with the operator when a binary expression breaks.
`auricFormatter.breakLongStrings`	boolean	`false`		Split a string literal that overflows the print width into concatenated parts.
`auricFormatter.catchPosition`	string	`same-line`	`same-line`, `new-line`	Choose where `catch` and `finally` sit relative to the preceding closing brace.
`auricFormatter.chainedCallIndent`	string	`indent`	`indent`, `align`, `preserve`	Choose how a broken member chain is indented.
`auricFormatter.chainedCallKeepFirstCall`	boolean	`true`		Keep the first call of a member chain attached to its receiver, as in `expect(x)\n .toBe(y)`.
`auricFormatter.directivePrologueQuotes`	string	`preserve`	`preserve`, `follow`, `double`, `single`	Choose how the quotes around directive prologues are printed.
`auricFormatter.doWhilePosition`	string	`same-line`	`same-line`, `new-line`	Choose where the trailing `while` of a do-while loop sits.
`auricFormatter.elsePosition`	string	`same-line`	`same-line`, `new-line`	Choose where the `else` keyword sits relative to the preceding closing brace.
`auricFormatter.emptyBlockStyle`	string	`collapse`	`collapse`, `preserve`, `expand`	Choose how blocks with no statements are printed.
`auricFormatter.firstArgumentHugging`	boolean	`true`		Let a leading function argument expand in place for call shapes like `describe(function () {}, opts)`.
`auricFormatter.functionParameterNewline`	string	`multiline`	`preserve`, `always`, `never`, `multiline`	Choose how function parameter lists are broken across lines.

Setting	Type	Default	Values or range	What it does
`auricFormatter.generatorStarPosition`	string	`after-function`	`after-function`, `before-name`, `spaced`	Choose where the star of a generator function sits.
`auricFormatter.jsxBracketSameLine`	boolean	`false`		Put the closing angle bracket of a multi-line JSX opening tag on the last attribute line.
`auricFormatter.jsxEmptyExpressionComment`	boolean	`true`		Keep a comment that is the sole contents of a JSX expression container on its own line.
`auricFormatter.jsxSelfCloseSpace`	boolean	`true`		Print a space before the slash of a self-closing JSX element.
`auricFormatter.jsxSingleAttributeSameLine`	boolean	`true`		Keep an element with exactly one short attribute on a single line.
`auricFormatter.jsxSortProps`	boolean	`false`		Sort JSX attributes, keeping `key` and `ref` first and spreads pinned in place.
`auricFormatter.jsxSpaceBraces`	boolean	`false`		Print spaces just inside JSX expression braces, as in `{ value }`.
`auricFormatter.jsxWrapMultiline`	string	`parens`	`parens`, `preserve`, `never`	Choose whether multi-line JSX expressions are wrapped in parentheses.
`auricFormatter.labelIndent`	string	`preserve`	`preserve`, `dedent`, `flush`	Choose how statement labels are indented.
`auricFormatter.lastArgumentHugging`	boolean	`true`		Let a trailing function or object argument expand in place instead of breaking the whole call.
`auricFormatter.memberChainBreakThreshold`	integer	`3`	0–32	Break a member chain across lines once it contains at least this many member accesses.
`auricFormatter.methodChainCallThreshold`	integer	`3`	0–32	Break a member chain across lines once it contains at least this many call expressions.
`auricFormatter.newExpressionParens`	string	`always`	`always`, `preserve`	Choose whether a `new` expression with no arguments keeps its parentheses.
`auricFormatter.normalizeNumericLiterals`	boolean	`true`		Normalise numeric literals by trimming a trailing dot, adding a leading zero and dropping a redundant exponent sign.
`auricFormatter.numericLiteralCase`	string	`lower`	`preserve`, `lower`, `upper`	Choose the letter case of the non-digit parts of numeric literals.
`auricFormatter.numericSeparators`	string	`preserve`	`preserve`, `remove`, `thousands`	Choose how underscore separators inside numeric literals are handled.
`auricFormatter.objectDestructuringNewline`	string	`multiline`	`preserve`, `always`, `never`, `multiline`	Choose how object destructuring patterns are broken across lines.
`auricFormatter.objectExpandThreshold`	integer	`0`	0–100	Force an object with at least this many properties onto multiple lines, or 0 to decide by width alone.

Setting	Type	Default	Values or range	What it does
`auricFormatter.objectPropertyNewline`	string	`preserve`	`preserve`, `always`, `never`	Choose whether every object property gets its own line.
`auricFormatter.preserveArrayFormatting`	boolean	`false`		Keep the author's line breaks in arrays of numbers and short literals so hand-laid tables survive.
`auricFormatter.preserveObjectShape`	boolean	`true`		Keep an object expanded when the author already broke it after the opening brace.
`auricFormatter.preserveParens`	boolean	`false`		Keep every parenthesis the author wrote, even where precedence makes it redundant.
`auricFormatter.quoteStyleForJSXText`	string	`preserve`	`preserve`, `entities`, `curly`	Choose how quotation marks inside JSX text are printed.
`auricFormatter.removeRedundantParens`	boolean	`true`		Drop parentheses that precedence and readability rules make unnecessary.
`auricFormatter.semiColonBeforeIIFE`	boolean	`true`		Start a line with a defensive semicolon when omitting semicolons would let it join the statement above.
`auricFormatter.shortArrayThreshold`	integer	`0`	0–100	Collapse an array onto one line when it holds at most this many short elements, or 0 to decide by width alone.
`auricFormatter.singleLineArrowBody`	boolean	`true`		Keep a short arrow function body on the same line as the arrow.
`auricFormatter.singleLineIfStatement`	string	`fit`	`fit`, `preserve`, `never`	Decide whether a braceless `if` may share a line with its body.
`auricFormatter.spaceAfterKeyword`	boolean	`true`		Print a space between a control-flow keyword and its opening parenthesis, as in `if (x)`.
`auricFormatter.spaceAroundOperators`	boolean	`true`		Print a single space on each side of binary and assignment operators.
`auricFormatter.spaceBeforeBlocks`	boolean	`true`		Print a space between a closing parenthesis or keyword and an opening block brace.
`auricFormatter.spaceBeforeColon`	boolean	`false`		Print a space before the colon in object properties and labels.
`auricFormatter.spaceBeforeFunctionParen`	string	`anonymous`	`never`, `always`, `named`, `anonymous`, `asyncArrow`	Choose which functions get a space between the name and the parameter list.
`auricFormatter.spaceInParens`	boolean	`false`		Print spaces just inside parentheses, as in `foo( a, b )`.
`auricFormatter.spreadSpacing`	boolean	`false`		Print a space between a spread or rest operator and its argument, as in `... rest`.

Setting	Type	Default	Values or range	What it does
`auricFormatter.switchCaseBlankLine`	string	`preserve`	`preserve`, `always`, `never`	Choose whether consecutive switch cases are separated by a blank line.
`auricFormatter.switchCaseBraceSameLine`	boolean	`true`		Keep the opening brace of a braced case body on the case line.
`auricFormatter.switchCaseIndent`	boolean	`true`		Indent `case` clauses one level inside the switch block.
`auricFormatter.taggedTemplateFormat`	string	`auto`	`auto`, `preserve`, `always`	Choose whether the contents of tagged template literals are formatted.
`auricFormatter.templateExpressionSpacing`	boolean	`false`		Print spaces just inside template literal interpolations, as in `${ value }`.
`auricFormatter.templateLiteralIndent`	string	`preserve`	`preserve`, `indent`, `dedent`	Choose whether the interior of a template literal is re-indented.
`auricFormatter.ternaryFormat`	string	`indented`	`inline`, `indented`, `aligned`, `nested-flat`	Choose how conditional expressions are laid out when they break.
`auricFormatter.yieldParens`	string	`as-needed`	`as-needed`, `always`	Choose when the operand of `yield` is wrapped in parentheses.

TypeScript

23 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.abstractMemberBlankLine`	boolean	`false`		Separate abstract members from concrete ones with a blank line.
`auricFormatter.arrowReturnTypeSpacing`	boolean	`true`		Print a space after the colon that introduces a function return type.
`auricFormatter.conditionalTypeFormat`	string	`indented`	`indented`, `aligned`, `chain`, `inline`	Choose how conditional types are laid out when they break.
`auricFormatter.decoratorPosition`	string	`preserve`	`preserve`, `above`, `inline`	Choose where decorators sit relative to the declaration they decorate.
`auricFormatter.decoratorsBeforeExport`	string	`preserve`	`preserve`, `before`, `after`	Choose where decorators sit relative to the `export` keyword.
`auricFormatter.enumMemberTrailing`	string	`follow`	`follow`, `all`, `none`	Choose whether a trailing comma follows the last member of an enum.
`auricFormatter.genericSpacing`	boolean	`false`		Print spaces just inside type argument angle brackets, as in `Array< string >`.
`auricFormatter.importTypeStyle`	string	`preserve`	`preserve`, `separate`, `inline`	Choose how type-only import specifiers are grouped.
`auricFormatter.indexSignatureSpacing`	string	`none`	`none`, `inside`	Choose the spacing inside index signature brackets.
`auricFormatter.intersectionFormat`	string	`auto`	`auto`, `inline`, `expanded`	Choose how intersection types are laid out.

Setting	Type	Default	Values or range	What it does
`auricFormatter.mappedTypeFormat`	string	`auto`	`auto`, `expanded`, `inline`	Choose how mapped types are laid out.
`auricFormatter.parameterDecoratorInline`	boolean	`true`		Keep parameter decorators on the same line as the parameter they decorate.
`auricFormatter.preserveEnumOneLine`	boolean	`false`		Keep a short enum on a single line when the author wrote it that way.
`auricFormatter.preserveTypeParens`	boolean	`false`		Keep redundant parentheses inside type expressions where the author wrote them.
`auricFormatter.readonlyArrayShorthand`	boolean	`false`		Print `readonly T[]` in place of `ReadonlyArray<T>` where the two are interchangeable.
`auricFormatter.satisfiesOperatorBreak`	string	`auto`	`auto`, `never`, `always`	Choose how a `satisfies` expression breaks when the line is too long.
`auricFormatter.typeAnnotationSpacing`	string	`after`	`after`, `before`, `both`, `none`	Choose the spacing around the colon of a type annotation.
`auricFormatter.typeCastParens`	boolean	`false`		Parenthesise an `as` expression whose position would otherwise make the cast ambiguous to a reader.
`auricFormatter.typeLiteralTrailingSeparator`	boolean	`true`		Print a trailing separator after the last member of a multi-line object type.
`auricFormatter.typeMemberSeparator`	string	`semi`	`semi`, `comma`, `none`, `preserve`	Choose the separator printed between members of an object type.
`auricFormatter.typeParameterNewline`	string	`multiline`	`multiline`, `preserve`, `always`, `never`	Choose how type parameter lists are broken across lines.
`auricFormatter.unionFormat`	string	`auto`	`auto`, `inline`, `expanded`, `leading-pipe`	Choose how union types are laid out.
`auricFormatter.unionLeadingPipeWhenBroken`	boolean	`true`		Print a leading pipe on the first member of a union that has been broken across lines.

CSS

39 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.atRuleBlankLine`	boolean	`false`		Print a blank line before a block at-rule such as @media or @supports.
`auricFormatter.atRuleCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of at-rule names.
`auricFormatter.blankLineBetweenRules`	integer	`0`	0–4	Print exactly this many blank lines between sibling rules, or 0 to leave the author's spacing.

Setting	Type	Default	Values or range	What it does
`auricFormatter.colorNotation`	string	`preserve`	`preserve`, `hex`, `rgb`, `hsl`, `oklch`	Choose the notation colour values are rewritten into.
`auricFormatter.cssTrailingSemicolon`	string	`always`	`always`, `never`, `preserve`	Choose whether the last declaration in a block ends with a semicolon.
`auricFormatter.declarationColonSpace`	boolean	`true`		Print a space after the colon that separates a property from its value.
`auricFormatter.declarationOrder`	string	`preserve`	`preserve`, `alphabetical`, `concentric`, `smacss`, `idiomatic`, `grouped`	Choose the order declarations are printed in inside a rule.
`auricFormatter.emptyLineBeforeNested`	boolean	`false`		Print a blank line before the first nested rule inside a block.
`auricFormatter.emptyRuleStyle`	string	`preserve`	`preserve`, `collapse`, `remove`	Choose what happens to rules with no declarations.
`auricFormatter.functionArgSpacing`	boolean	`false`		Print spaces just inside the parentheses of CSS functions, as in `rgb( 1, 2, 3 )`.
`auricFormatter.gridTemplateAlign`	boolean	`true`		Keep each row of a multi-line grid-template-areas value on its own line and align the columns.
`auricFormatter.hexShorthand`	string	`preserve`	`preserve`, `short`, `long`	Choose whether hex colours are written in their short or long form.
`auricFormatter.importantSpacing`	string	`before`	`before`, `none`, `both`	Choose the spacing around an !important flag.
`auricFormatter.keyframeSelectorCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of keyframe selectors.
`auricFormatter.leadingZero`	string	`always`	`always`, `never`, `preserve`	Choose whether decimals below one carry a leading zero.
`auricFormatter.lessGuardFormat`	string	`preserve`	`preserve`, `expanded`, `compact`	Choose how Less mixin guard clauses are laid out.
`auricFormatter.lessMixinSeparator`	string	`preserve`	`preserve`, `semicolon`, `comma`	Choose the separator printed between Less mixin arguments.
`auricFormatter.mathOperatorSpacing`	boolean	`true`		Print spaces around the operators inside calc() and preprocessor arithmetic.
`auricFormatter.mediaQueryFormat`	string	`preserve`	`preserve`, `compact`, `expanded`	Choose how media query feature lists are laid out.
`auricFormatter.nestedRuleBlankLine`	boolean	`false`		Print a blank line before a nested rule that follows a declaration.
`auricFormatter.preserveCustomPropertyValues`	boolean	`true`		Print the value of a custom property exactly as written, since it may be an arbitrary token stream.
`auricFormatter.propertyCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of property names.

Setting	Type	Default	Values or range	What it does
`auricFormatter.propertyValueAlign`	boolean	`false`		Align the values of consecutive declarations into a column.
`auricFormatter.quotesInUrl`	string	`preserve`	`preserve`, `always`, `never`	Choose whether url() arguments are quoted.
`auricFormatter.scssIncludeParens`	string	`preserve`	`preserve`, `always`, `never`	Choose whether an argument-free @include keeps its parentheses.
`auricFormatter.scssMapFormat`	string	`preserve`	`preserve`, `expanded`, `compact`	Choose how SCSS maps are laid out.
`auricFormatter.scssOperatorSpacing`	boolean	`true`		Print spaces around SCSS comparison and arithmetic operators inside @if and @while.
`auricFormatter.selectorAttributeQuotes`	string	`always`	`always`, `preserve`, `never`	Choose whether attribute selector values are quoted.
`auricFormatter.selectorCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of type selectors and pseudo-class names.
`auricFormatter.selectorCombinatorSpacing`	string	`spaced`	`spaced`, `tight`, `preserve`	Choose the spacing around child, sibling and descendant combinators.
`auricFormatter.selectorSeparator`	string	`newline`	`newline`, `space`, `preserve`	Choose how the selectors of a selector list are separated.
`auricFormatter.shorthandExpansion`	string	`preserve`	`preserve`, `collapse`, `expand`	Choose whether shorthand properties are expanded or collapsed.
`auricFormatter.singleLineRuleThreshold`	integer	`0`	0–32	Print a rule on one line when it holds at most this many declarations, or 0 to always break.
`auricFormatter.singleQuoteCss`	boolean	`false`		Prefer single quotes over double quotes in stylesheets.
`auricFormatter.trailingZero`	string	`never`	`never`, `preserve`	Choose whether trailing zeros in numbers are trimmed.
`auricFormatter.unitCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of units.
`auricFormatter.valueKeywordCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of keyword values.
`auricFormatter.vendorPrefixAlign`	boolean	`false`		Align a run of vendor-prefixed variants of the same property into a column.
`auricFormatter.zeroUnits`	string	`preserve`	`preserve`, `strip`, `keep`	Choose whether zero values keep their unit.

HTML

29 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.htmlAttributeCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of attribute names.
`auricFormatter.htmlAttributeOrder`	array	`[]`		List attribute names or glob patterns in the order they should be printed when sorting is on.
`auricFormatter.htmlAttributeWrap`	string	`auto`	`auto`, `preserve`, `always`, `never`	Choose when a tag's attribute list is broken across lines.
`auricFormatter.htmlBlankLineBetweenBlocks`	integer	`0`	0–4	Print exactly this many blank lines between sibling block elements, or 0 to leave the author's spacing.
`auricFormatter.htmlBooleanAttributeStyle`	string	`preserve`	`preserve`, `bare`, `explicit`	Choose how boolean attributes are printed.
`auricFormatter.htmlClassAttributeWrap`	boolean	`false`		Break a long class attribute into one class name per line.
`auricFormatter.htmlDoctypeCase`	string	`lower`	`lower`, `upper`, `preserve`	Choose the letter case of the doctype declaration.
`auricFormatter.htmlEntityStyle`	string	`preserve`	`preserve`, `named`, `numeric`, `minimal`	Choose how character entities are printed.
`auricFormatter.htmlIndentInlineElements`	boolean	`false`		Indent the children of inline elements the way block elements are indented.
`auricFormatter.htmlMinifyWhitespace`	boolean	`false`		Collapse every run of insignificant whitespace between elements to nothing.
`auricFormatter.htmlPreserveTags`	array	`["pre","textarea","listing","plaintext"]`		List tags whose contents are copied through untouched because their whitespace is significant.

Setting	Type	Default	Values or range	What it does
`auricFormatter.htmlQuoteAttributes`	string	`double`	`double`, `single`, `preserve`, `as-needed`	Choose how attribute values are quoted.
`auricFormatter.htmlScriptIndent`	string	`indent`	`indent`, `flush`, `preserve`	Choose how the contents of a `<script>` block are indented.
`auricFormatter.htmlSelfClosing`	string	`preserve`	`preserve`, `always`, `never`	Choose whether childless elements are printed self-closed.
`auricFormatter.htmlSortAttributes`	boolean	`false`		Sort attributes using htmlAttributeOrder, then alphabetically for anything left over.
`auricFormatter.htmlSrcsetWrap`	boolean	`true`		Break a long srcset or sizes attribute after each comma.
`auricFormatter.htmlStyleIndent`	string	`indent`	`indent`, `flush`, `preserve`	Choose how the contents of a `<style>` block are indented.
`auricFormatter.htmlTagCase`	string	`lower`	`preserve`, `lower`, `upper`	Choose the letter case of tag names.
`auricFormatter.htmlVoidElementSlash`	boolean	`true`		Print the trailing slash on void elements, as in ` `.
`auricFormatter.htmlWrapText`	boolean	`true`		Reflow text nodes so that they fit within the print width.
`auricFormatter.svgAttributeOrder`	array	`[]`		List SVG attribute names in the order they should be printed when attribute sorting is on.
`auricFormatter.svgPathPrecision`	integer	`0`	0–12	Round the numbers inside an SVG path to this many decimal places, or 0 to leave them untouched.

Setting	Type	Default	Values or range	What it does
`auricFormatter.svgPathWrap`	string	`preserve`	`preserve`, `command`, `never`	Choose how SVG path data is wrapped.
`auricFormatter.xmlDeclarationQuotes`	string	`double`	`double`, `single`, `preserve`	Choose how the values in the XML declaration are quoted.
`auricFormatter.xmlIndentAttributes`	boolean	`true`		Indent a broken attribute list one level under its tag name.
`auricFormatter.xmlPreserveCdata`	boolean	`true`		Copy CDATA sections through byte for byte.
`auricFormatter.xmlSelfClosingSpace`	boolean	`true`		Print a space before the slash of a self-closing XML tag.
`auricFormatter.xmlSortAttributes`	boolean	`false`		Sort XML attributes alphabetically, leaving namespace declarations first.
`auricFormatter.xmlWhitespaceSensitivity`	string	`strict`	`strict`, `ignore`, `preserve-mixed`	Choose how strictly whitespace inside XML elements is preserved.

Markdown

32 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.mdBlockquoteBlankLineMarker`	boolean	`true`		Print a lone marker on the blank lines inside a blockquote so the quote stays visually joined.
`auricFormatter.mdBlockquoteMarker`	string	`spaced`	`spaced`, `tight`, `preserve`	Choose the spacing after a blockquote marker.
`auricFormatter.mdBulletAlternate`	boolean	`false`		Alternate the bullet character at each nesting level so that levels are visually distinct.
`auricFormatter.mdBulletMarker`	string	`dash`	`dash`, `asterisk`, `plus`, `preserve`	Choose the character used for unordered list bullets.
`auricFormatter.mdCodeFence`	string	`backtick`	`backtick`, `tilde`, `preserve`	Choose the character used to fence code blocks.

Setting	Type	Default	Values or range	What it does
`auricFormatter.mdCodeFenceLength`	integer	`3`	3–10	Print code fences with at least this many characters, lengthening them further when the content demands it.
`auricFormatter.mdEmphasisMarker`	string	`underscore`	`asterisk`, `underscore`, `preserve`	Choose the character used for emphasis.
`auricFormatter.mdEscapeSpecial`	boolean	`true`		Escape characters in text that would otherwise be read as markup.
`auricFormatter.mdFormatCodeBlocks`	boolean	`true`		Format the contents of fenced code blocks whose info string names a language Auric supports.
`auricFormatter.mdFrontmatterFormat`	string	`format`	`format`, `preserve`	Choose whether document frontmatter is formatted.
`auricFormatter.mdHeadingBlankLines`	integer	`1`	0–3	Print exactly this many blank lines before a heading that follows other content.
`auricFormatter.mdHeadingClosingHashes`	boolean	`false`		Print a matching run of hashes at the end of an ATX heading.
`auricFormatter.mdHeadingStyle`	string	`atx`	`atx`, `setext`, `preserve`	Choose the heading style.
`auricFormatter.mdHtmlBlockFormat`	string	`preserve`	`preserve`, `format`	Choose whether raw HTML blocks inside Markdown are formatted.
`auricFormatter.mdLineBreakStyle`	string	`preserve`	`preserve`, `backslash`, `spaces`	Choose how hard line breaks inside a paragraph are written.
`auricFormatter.mdLinkStyle`	string	`preserve`	`preserve`, `inline`, `reference`	Choose whether links are printed inline or as references.
`auricFormatter.mdListIndent`	string	`aligned`	`aligned`, `one`, `tab`, `preserve`	Choose how the continuation lines of a list item are indented.
`auricFormatter.mdListItemBlankLine`	string	`preserve`	`preserve`, `loose`, `tight`	Choose whether list items are separated by blank lines.
`auricFormatter.mdMaxHeadingLevel`	integer	`0`	0–6	Warn when a heading is nested deeper than this level, or 0 to allow any depth.
`auricFormatter.mdNumberedListAlign`	boolean	`true`		Right-align the numbers of an ordered list that reaches double digits.
`auricFormatter.mdOrderedMarker`	string	`period`	`period`, `paren`, `preserve`	Choose the delimiter that follows an ordered list number.
`auricFormatter.mdOrderedNumbering`	string	`preserve`	`preserve`, `one`, `ordinal`	Choose how ordered list items are numbered.
`auricFormatter.mdReferencePlacement`	string	`preserve`	`preserve`, `end`, `section`	Choose where link reference definitions are placed.
`auricFormatter.mdSetextUnderlineFull`	boolean	`true`		Extend a setext heading underline to the full width of the heading text.

Setting	Type	Default	Values or range	What it does
`auricFormatter.mdStrongMarker`	string	`asterisk`	`asterisk`, `underscore`, `preserve`	Choose the character used for strong emphasis.
`auricFormatter.mdTableAlign`	boolean	`true`		Pad table cells so that every pipe lines up into a column.
`auricFormatter.mdTableCompact`	boolean	`false`		Collapse table alignment when the table would otherwise exceed the print width.
`auricFormatter.mdTableMinCellWidth`	integer	`3`	1–40	Pad every aligned table cell to at least this many characters.
`auricFormatter.mdTablePadding`	boolean	`true`		Print one space of padding inside each table cell's pipes.
`auricFormatter.mdThematicBreak`	string	`dashes`	`dashes`, `asterisks`, `underscores`, `preserve`	Choose how thematic breaks are printed.
`auricFormatter.mdTocGenerate`	boolean	`false`		Rebuild the table of contents between the auric-toc markers from the document's headings.
`auricFormatter.mdTocMaxDepth`	integer	`3`	1–6	Include headings down to this level when rebuilding a table of contents.

YAML

15 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.yamlAlignValues`	boolean	`false`		Align the values of consecutive mapping entries into a column.
`auricFormatter.yamlAnchorFormat`	string	`preserve`	`preserve`, `inline`, `above`	Choose where anchors are printed relative to the value they name.
`auricFormatter.yamlBlankLineBetweenTopLevelKeys`	integer	`0`	0–4	Print exactly this many blank lines between top-level keys, or 0 to leave the author's spacing.
`auricFormatter.yamlBooleanStyle`	string	`preserve`	`preserve`, `true-false`, `yes-no`, `on-off`	Choose how boolean values are written.
`auricFormatter.yamlBracketSpacing`	boolean	`true`		Print spaces just inside the braces of flow mappings, as in `{ a: 1 }`.
`auricFormatter.yamlCommentIndent`	string	`align`	`align`, `preserve`, `flush`	Choose how standalone comments are indented.
`auricFormatter.yamlDocumentSeparator`	string	`preserve`	`preserve`, `always`, `never`	Choose whether documents are introduced by a triple-dash separator.
`auricFormatter.yamlExplicitKeys`	boolean	`false`		Write complex mapping keys with the explicit question-mark form.

Setting	Type	Default	Values or range	What it does
`auricFormatter.yamlFlowCollectionWidth`	integer	`0`	0–1000	Break a flow collection into block form once it would exceed this many columns, or 0 to follow the print width.
`auricFormatter.yamlIndent`	integer	`2`	1–12	Indent each nested mapping level by this many spaces.
`auricFormatter.yamlKeySort`	boolean	`false`		Sort mapping keys alphabetically, which is safe because YAML mappings are unordered.
`auricFormatter.yamlNullStyle`	string	`preserve`	`preserve`, `empty`, `tilde`, `null`	Choose how null values are written.
`auricFormatter.yamlPreserveBlockScalars`	boolean	`true`		Copy the body of a literal or folded block scalar through byte for byte.
`auricFormatter.yamlQuoteStyle`	string	`preserve`	`preserve`, `single`, `double`, `minimal`	Choose how scalar values are quoted.
`auricFormatter.yamlSequenceIndent`	string	`indented`	`indented`, `flush`, `preserve`	Choose how sequence items are indented under their parent key.

JSON

22 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.iniAlignValues`	boolean	`false`		Align the values of consecutive properties into a column.
`auricFormatter.iniKeyCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of property names.
`auricFormatter.iniSectionBlankLine`	integer	`1`	0–4	Print exactly this many blank lines before a section header, or 0 to leave the author's spacing.
`auricFormatter.iniSpaceAroundEquals`	boolean	`true`		Print a space on each side of the equals sign.
`auricFormatter.jsonBracketSpacing`	boolean	`true`		Print spaces just inside the braces of a single-line object.
`auricFormatter.jsonEmptyCollectionStyle`	string	`collapse`	`collapse`, `preserve`	Choose how empty objects and arrays are printed.
`auricFormatter.jsonEscapeUnicode`	boolean	`false`		Escape every non-ASCII character in strings as a \u sequence.
`auricFormatter.jsonExpandArrayThreshold`	integer	`0`	0–100	Break an array onto multiple lines once it holds at least this many elements, or 0 to decide by width alone.
`auricFormatter.jsonNormalizeNumbers`	boolean	`false`		Rewrite numbers into their shortest round-trip form.
`auricFormatter.jsonPackageJsonOrder`	boolean	`false`		Print the well-known top-level keys of a package.json in their conventional order.

Setting	Type	Default	Values or range	What it does
`auricFormatter.jsonPreserveComments`	boolean	`true`		Keep comments in JSON dialects that permit them and refuse to format the file if they cannot be placed.
`auricFormatter.jsonQuoteKeys`	string	`preserve`	`preserve`, `always`, `as-needed`	Choose whether object keys are quoted.
`auricFormatter.jsonSingleLineArrays`	boolean	`false`		Keep arrays of primitives on one line whenever they fit.
`auricFormatter.jsonSortKeys`	boolean	`false`		Sort object keys alphabetically throughout the document.
`auricFormatter.jsonTrailingComma`	string	`none`	`none`, `all`, `preserve`	Choose whether broken arrays and objects end with a trailing comma.
`auricFormatter.tomlAlignValues`	boolean	`false`		Align the values of consecutive key-value pairs into a column.
`auricFormatter.tomlArrayWrap`	string	`auto`	`auto`, `always`, `never`, `preserve`	Choose how TOML arrays are wrapped.
`auricFormatter.tomlIndentTables`	boolean	`false`		Indent the body of a table one level under its header.
`auricFormatter.tomlInlineTableSpacing`	boolean	`true`		Print spaces just inside the braces of an inline table.
`auricFormatter.tomlKeyQuoting`	string	`as-needed`	`as-needed`, `always`, `preserve`	Choose whether TOML keys are quoted.
`auricFormatter.tomlSortKeys`	boolean	`false`		Sort the keys inside each table alphabetically.
`auricFormatter.tomlTableBlankLine`	integer	`1`	0–4	Print exactly this many blank lines before a table header, or 0 to leave the author's spacing.

GraphQL

8 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.graphqlArgumentWrap`	string	`auto`	`auto`, `always`, `never`	Choose how field and directive argument lists are wrapped.
`auricFormatter.graphqlBracketSpacing`	boolean	`true`		Print spaces just inside the braces of an inline object value.
`auricFormatter.graphqlCommaSeparators`	string	`remove`	`remove`, `preserve`	Choose whether GraphQL's optional commas are printed.
`auricFormatter.graphqlDescriptionStyle`	string	`preserve`	`preserve`, `block`, `inline`	Choose how schema descriptions are quoted.
`auricFormatter.graphqlFieldAlign`	boolean	`false`		Align the types of consecutive field definitions into a column.
`auricFormatter.graphqlSeparateOperations`	integer	`1`	0–4	Print exactly this many blank lines between top-level definitions.

Setting	Type	Default	Values or range	What it does
`auricFormatter.graphqlSortDefinitions`	boolean	`false`		Sort top-level definitions by kind and then by name.
`auricFormatter.graphqlSortFields`	boolean	`false`		Sort the fields of a type definition alphabetically.

Universal

39 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.cPointerAlignment`	string	`preserve`	`preserve`, `left`, `right`, `middle`	Choose where the star of a pointer declaration sits.
`auricFormatter.dockerfileInstructionCase`	string	`upper`	`preserve`, `lower`, `upper`	Choose the letter case of Dockerfile instructions.
`auricFormatter.goAlignComments`	boolean	`true`		Align the trailing comments of a const or var block into a column, the way gofmt does.
`auricFormatter.goAlignStructTags`	boolean	`true`		Align the field names, types and struct tags of a struct definition into columns.
`auricFormatter.javaAnnotationPosition`	string	`above`	`above`, `inline`, `preserve`	Choose where Java-family annotations are printed.
`auricFormatter.makefileTabGuard`	boolean	`true`		Refuse to convert the leading tab of a recipe line to spaces, because make requires it.
`auricFormatter.pythonAlignDictValues`	boolean	`false`		Align the values of a multi-line dictionary literal into a column.
`auricFormatter.pythonBlankLinesNested`	integer	`1`	0–4	Separate methods and nested functions with exactly this many blank lines.
`auricFormatter.pythonBlankLinesTopLevel`	integer	`2`	0–4	Separate top-level classes and functions with exactly this many blank lines.
`auricFormatter.rustAlignMatchArms`	boolean	`false`		Align the fat arrows of a match block into a column.
`auricFormatter.rustAlignStructFields`	boolean	`false`		Align the types of consecutive struct fields into a column.
`auricFormatter.shellFunctionKeyword`	boolean	`false`		Print the optional `function` keyword before shell function definitions.
`auricFormatter.shellIndentCase`	boolean	`true`		Indent the patterns of a case statement one level inside the case block.
`auricFormatter.shellRedirectSpacing`	boolean	`false`		Print a space between a redirection operator and its target.

Setting	Type	Default	Values or range	What it does
`auricFormatter.sqlAlignSelectColumns`	boolean	`false`		Align the aliases of a select list into a column.
`auricFormatter.sqlCommaPosition`	string	`trailing`	`trailing`, `leading`, `preserve`	Choose whether the commas of a broken list lead or trail.
`auricFormatter.sqlIdentifierCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of unquoted identifiers.
`auricFormatter.sqlIndentJoins`	boolean	`true`		Indent join clauses one level under the FROM clause they belong to.
`auricFormatter.sqlIndentSubqueries`	boolean	`true`		Indent the body of a subquery one level inside its parentheses.
`auricFormatter.sqlKeywordCase`	string	`upper`	`preserve`, `lower`, `upper`	Choose the letter case of SQL keywords.
`auricFormatter.universalAlignAssignments`	boolean	`false`		Align the equals signs of a consecutive run of assignments into a column.
`auricFormatter.universalAlignConsecutive`	boolean	`false`		Align consecutive declarations, struct fields and case arms that share a shape.
`auricFormatter.universalAlignTrailingComments`	boolean	`false`		Align the trailing comments of a consecutive run of lines into a column.
`auricFormatter.universalAttributeIndent`	string	`preserve`	`preserve`, `above`, `inline`	Choose where attributes and annotations are printed relative to their target.
`auricFormatter.universalBraceStyle`	string	`preserve`	`preserve`, `same-line`, `next-line`, `next-line-indented`	Choose where the opening brace of a block is placed.
`auricFormatter.universalCaseIndent`	boolean	`true`		Indent case labels one level inside the switch or match block that owns them.
`auricFormatter.universalContinuationIndent`	integer	`0`	0–24	Indent the continuation lines of a wrapped statement by this many columns, or 0 to use one full level.
`auricFormatter.universalHangingIndent`	boolean	`true`		Align a wrapped argument list under the opening bracket that introduced it.
`auricFormatter.universalIndentSize`	integer	`0`	0–24	Indent universal-engine languages by this many columns, or 0 to follow tabWidth.
`auricFormatter.universalKeywordCase`	string	`preserve`	`preserve`, `lower`, `upper`	Choose the letter case of language keywords.
`auricFormatter.universalLabelIndent`	string	`preserve`	`preserve`, `dedent`, `flush`	Choose how goto labels and their equivalents are indented.
`auricFormatter.universalMaxBlankLines`	integer	`0`	0–8	Collapse runs of blank lines to this many, or 0 to follow maxBlankLines.

Setting	Type	Default	Values or range	What it does
`auricFormatter.universalNormalizeCommaSpacing`	boolean	`true`		Print no space before a comma and exactly one after it.
`auricFormatter.universalNormalizeOperatorSpacing`	boolean	`true`		Print exactly one space on each side of a binary operator.
`auricFormatter.universalNormalizeParenSpacing`	boolean	`true`		Remove the padding just inside parentheses and brackets.
`auricFormatter.universalNormalizeSemicolonSpacing`	boolean	`true`		Print no space before a semicolon and exactly one after it when it separates rather than terminates.
`auricFormatter.universalPreserveAlignmentBlocks`	boolean	`true`		Keep a hand-aligned run of lines intact when its columns already line up.
`auricFormatter.universalPreserveShebang`	boolean	`true`		Keep a shebang line flush with the left margin and free of reformatting.
`auricFormatter.universalTrimBlockEdges`	boolean	`true`		Remove blank lines immediately inside an opening or closing brace.

Alignment

15 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.alignAssignments`	string	`off`	`off`, `consecutive`, `block`	Choose how the equals signs of neighbouring assignments are aligned.
`auricFormatter.alignConsecutiveDeclarations`	boolean	`false`		Align the names of consecutive variable declarations into a column.
`auricFormatter.alignEnumValues`	boolean	`false`		Align the initialisers of consecutive enum members into a column.
`auricFormatter.alignImportSpecifiers`	boolean	`false`		Align the `from` keyword of consecutive import statements into a column.
`auricFormatter.alignObjectValues`	boolean	`false`		Align the values of consecutive object properties into a column.
`auricFormatter.alignTernaryBranches`	boolean	`false`		Align the question mark and colon of a broken conditional expression into a column.
`auricFormatter.alignTrailingComments`	boolean	`false`		Align the trailing comments of consecutive lines into a column.
`auricFormatter.alignTrailingCommentsMaxColumn`	integer	`0`	0–1000	Give up aligning a trailing comment that would start past this column, or 0 to use the print width.
`auricFormatter.alignTrailingCommentsMinSpaces`	integer	`1`	1–8	Leave at least this many spaces between code and an aligned trailing comment.
`auricFormatter.alignTypeMembers`	boolean	`false`		Align the types of consecutive members of an object type into a column.

Setting	Type	Default	Values or range	What it does
`auricFormatter.alignmentBreakOnBlankLine`	boolean	`true`		End an alignment group at a blank line, so that separate paragraphs align separately.
`auricFormatter.alignmentBreakOnComment`	boolean	`false`		End an alignment group at a standalone comment.
`auricFormatter.alignmentMaxPadding`	integer	`24`	1–200	Abandon an alignment that would need more than this many spaces of padding on any line.
`auricFormatter.alignmentMinGroupSize`	integer	`2`	2–20	Align a group only once it contains at least this many lines.
`auricFormatter.alignmentUseSpacesOnly`	boolean	`true`		Pad alignment with spaces even when the file is indented with tabs, so columns survive any tab width.

Sorting

19 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.classMemberOrder`	array	see note		List the class member kinds in the order they should be printed when class sorting is on.
`auricFormatter.exportOrder`	string	`preserve`	`preserve`, `alphabetical`, `grouped`	Choose the order of export declarations.
`auricFormatter.importOrder`	array	`[]`		List regular expressions describing the import groups, in the order those groups should appear.
`auricFormatter.importOrderCaseInsensitive`	boolean	`false`		Ignore letter case when sorting import paths.
`auricFormatter.importOrderGroupNamespaceSpecifiers`	boolean	`false`		Put namespace imports at the top of their group.
`auricFormatter.importOrderSeparation`	boolean	`false`		Separate the import groups with a blank line.
`auricFormatter.importOrderSideEffectsFirst`	boolean	`true`		Keep side-effect-only imports at the top and never move them past another import.
`auricFormatter.importOrderSortSpecifiers`	boolean	`false`		Sort the named specifiers inside each import statement.
`auricFormatter.importOrderTypeImports`	string	`preserve`	`preserve`, `first`, `last`, `separate`	Choose where type-only imports are sorted.
`auricFormatter.sortCaseSensitivity`	string	`insensitive`	`insensitive`, `sensitive`	Choose how letter case is treated when sorting names.
`auricFormatter.sortClassMembers`	boolean	`false`		Sort class members into the groups named by classMemberOrder.

Setting	Type	Default	Values or range	What it does
`auricFormatter.sortIgnorePrefix`	string	empty		Strip this leading prefix from a name before comparing it, so decorated names sort with their plain neighbours.
`auricFormatter.sortImports`	boolean	`false`		Sort import declarations into the groups named by importOrder.
`auricFormatter.sortInterfaceMembers`	boolean	`false`		Sort the members of an interface or object type alphabetically.
`auricFormatter.sortNaturalNumeric`	boolean	`true`		Compare digit runs inside names numerically so that item2 sorts before item10.
`auricFormatter.sortObjectKeys`	boolean	`false`		Sort the keys of object literals that carry a sort-keys comment directive.
`auricFormatter.sortPreserveComments`	boolean	`true`		Carry each entry's leading and trailing comments with it when it moves.
`auricFormatter.sortRespectBlankLineGroups`	boolean	`true`		Treat a blank line as a group boundary and sort only inside each group.
`auricFormatter.sortUnionMembers`	boolean	`false`		Sort the members of a union type, keeping null and undefined last.

Notes and language-specific defaults:

- `auricFormatter.classMemberOrder` default: `["static-field","field","constructor","static-method","method","getter","setter"]`

Comments

18 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.bannerComment`	string	empty		Ensure the file starts with this comment text, inserting it when it is missing.
`auricFormatter.blankLineAfterComment`	string	`preserve`	`preserve`, `always`, `never`	Choose whether standalone comment blocks are followed by a blank line.
`auricFormatter.blankLineBeforeComment`	string	`preserve`	`preserve`, `always`, `never`	Choose whether standalone comments are preceded by a blank line.
`auricFormatter.blockCommentStarAlign`	boolean	`true`		Align the leading stars of a block comment one column in from the opening slash.
`auricFormatter.blockCommentStarPad`	boolean	`false`		Print a space after each leading star of a block comment.

Setting	Type	Default	Values or range	What it does
`auricFormatter.commentAttachment`	string	`smart`	`smart`, `preserve`	Choose how comments are attached to the syntax around them.
`auricFormatter.commentIgnorePattern`	string	empty		Copy through untouched any comment whose text matches this regular expression.
`auricFormatter.commentSpacing`	string	`preserve`	`preserve`, `always`, `never`	Choose the spacing between a line comment marker and its text.
`auricFormatter.commentWrap`	string	`preserve`	`preserve`, `always`, `never`	Choose whether comment text is reflowed.
`auricFormatter.commentWrapWidth`	integer	`0`	0–1000	Wrap comment text at this column, or 0 to use the print width.
`auricFormatter.footerComment`	string	empty		Ensure the file ends with this comment text, inserting it when it is missing.
`auricFormatter.jsdocAlign`	boolean	`false`		Align the type, name and description columns of a documentation block's tags.
`auricFormatter.jsdocBlankLineAfterDescription`	boolean	`false`		Separate a documentation block's description from its first tag with a blank line.
`auricFormatter.jsdocDescriptionIndent`	integer	`0`	0–24	Indent the wrapped continuation of a tag description by this many extra columns.
`auricFormatter.jsdocNormalizeTags`	boolean	`false`		Rewrite documentation tag aliases to their canonical spelling, turning @return into @returns.
`auricFormatter.jsdocTagOrder`	array	`[]`		List documentation tags in the order they should be printed, with unlisted tags keeping their relative order at the end.
`auricFormatter.jsdocWrap`	string	`preserve`	`preserve`, `always`, `never`	Choose whether documentation block descriptions are reflowed.
`auricFormatter.preserveBannerComment`	boolean	`true`		Keep the leading comment block of a file flush with the top and free of reflowing, so licence headers survive.

Blank Lines

15 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.blankLineAfterDirectives`	boolean	`false`		Print a blank line after the directive prologue of a file or function.
`auricFormatter.blankLineAfterImports`	string	`preserve`	`preserve`, `always`, `never`	Choose whether a blank line follows the import block.
`auricFormatter.blankLineBeforeControlFlow`	string	`preserve`	`preserve`, `always`, `never`	Choose whether if, for, while and switch statements are preceded by a blank line.

Setting	Type	Default	Values or range	What it does
`auricFormatter.blankLineBeforeExport`	string	`preserve`	`preserve`, `always`, `never`	Choose whether an export declaration is preceded by a blank line.
`auricFormatter.blankLineBeforeReturn`	string	`preserve`	`preserve`, `always`, `never`	Choose whether a return statement is preceded by a blank line.
`auricFormatter.blankLineBetweenMethods`	string	`preserve`	`preserve`, `always`, `never`	Choose whether class members are separated by a blank line.
`auricFormatter.blankLineBetweenTopLevel`	string	`preserve`	`preserve`, `always`, `never`	Choose whether top-level declarations are separated by a blank line.
`auricFormatter.collapseEmptyBlockBlankLines`	boolean	`true`		Remove the blank lines from a block that contains nothing else.
`auricFormatter.maxBlankLines`	integer	`1`	0–8	Collapse any run of blank lines to at most this many.
`auricFormatter.maxBlankLinesAtEnd`	integer	`0`	0–8	Allow at most this many blank lines at the very end of a file.
`auricFormatter.maxBlankLinesAtStart`	integer	`0`	0–8	Allow at most this many blank lines at the very top of a file.
`auricFormatter.maxBlankLinesInsideBlocks`	integer	`1`	0–8	Collapse any run of blank lines inside a block or function body to at most this many.
`auricFormatter.padBlocks`	boolean	`false`		Print a blank line just inside the braces of every block.
`auricFormatter.padClassBody`	boolean	`false`		Print a blank line just inside the opening and closing braces of a class body.
`auricFormatter.padFunctionBody`	boolean	`false`		Print a blank line just inside the opening and closing braces of a function body.

Notes and language-specific defaults:

- `auricFormatter.maxBlankLines`: `2` for python.

Safety

12 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.formatMinified`	boolean	`false`		Format files that look minified, instead of leaving them alone.
`auricFormatter.maxFileSizeKb`	integer	`4096`	0–1048576	Refuse to format a file larger than this many kilobytes, or 0 to lift the limit.
`auricFormatter.parseTimeoutMs`	integer	`5000`	0–600000	Abandon a parse that has run for this many milliseconds, or 0 to wait indefinitely.
`auricFormatter.safetyCommentPreservation`	boolean	`true`		Require every comment body in the input to appear in the output.
`auricFormatter.safetyIdempotency`	boolean	`false`		Format the output a second time and require the two results to be identical.

Setting	Type	Default	Values or range	What it does
`auricFormatter.safetyIdempotencyPasses`	integer	`1`	1–8	Run this many extra formatting passes when the idempotency check is on.
`auricFormatter.safetyNumericLiterals`	boolean	`true`		Require every numeric literal to keep its value through any rewriting of its spelling.
`auricFormatter.safetyOnFailure`	string	`preserve`	`preserve`, `warn`, `throw`	Choose what happens when a safety check fails.
`auricFormatter.safetyReparse`	boolean	`true`		Re-parse the output with the same parser and require it to succeed.
`auricFormatter.safetyStringLiterals`	boolean	`true`		Require the multiset of string and template literal contents to be identical before and after.
`auricFormatter.safetyTokenCheck`	boolean	`true`		Re-lex the output and require the significant token stream to match the input exactly.
`auricFormatter.safetyTokenCheckComments`	boolean	`true`		Include comment tokens in the token comparison.

Experimental

9 options.

Setting	Type	Default	Values or range	What it does
`auricFormatter.experimentalCommentReflow`	boolean	`false`		Experimental. Reflow comment prose with the Markdown paragraph engine, preserving lists and code spans.
`auricFormatter.experimentalDiffMinimization`	boolean	`false`		Experimental. Prefer the layout closest to the input whenever two layouts are equally valid.
`auricFormatter.experimentalIncrementalCache`	boolean	`false`		Experimental. Reuse printed fragments for subtrees whose text has not changed since the previous run.
`auricFormatter.experimentalJsxTextFill`	boolean	`false`		Experimental. Fill JSX text as prose so that words repack rather than each child taking a line.
`auricFormatter.experimentalMarkdownTableWrap`	boolean	`false`		Experimental. Wrap the contents of an over-wide Markdown table cell instead of letting the row overflow.
`auricFormatter.experimentalOperatorPosition`	string	`end`	`end`, `start`	Experimental. Choose which side of a line break a binary operator lands on.
`auricFormatter.experimentalPreserveSemanticBlocks`	boolean	`false`		Experimental. Treat a blank-line separated run of statements as a paragraph and keep it intact when breaking.
`auricFormatter.experimentalTernaries`	boolean	`false`		Experimental. Use the indented case-and-result layout for nested conditional expressions.
`auricFormatter.experimentalWidthAwareChains`	boolean	`false`		Experimental. Choose member chain layout by measuring every candidate rather than by counting links.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Presets

The 13 Auric Artisan Formatter presets, what each is for and which options each sets.

A *preset* is a named starting point: a coherent set of option values. Choose the one closest to how your team already writes code, then override the few things you disagree with. Anything you set yourself wins over the preset; options you do not mention keep the preset's value. The default preset is `auric`.

Set a preset

- **Auric Formatter: Switch Formatting Preset** — pick from the list; it saves `auricFormatter.preset` and re-runs the workspace scan.
- In a configuration file: `"preset": "prettier"`. This wins over the VS Code setting.
- In VS Code settings: `"auricFormatter.preset": "prettier"`.
- For one file: a leading comment with `@auric-preset compact`.
- On the command line: `--preset prettier`.
- In Formatter Studio: the **Preset** drop-down, to compare presets on your own file before choosing.

```
{
  "preset": "prettier",
  "printWidth": 100
}
```

This gives Prettier's exact behaviour at 100 columns instead of 80.

The 13 presets

Preset	Builds on	Options it sets	Description
`auric`	—	50	The Auric house style: Prettier's layout plus the Auric enhancements.
`prettier`	—	281	Byte-compatible with Prettier 3 at its own defaults, with every Auric-only enhancement switched off.
`standard`	`prettier`	14	StandardJS: no semicolons, single quotes, and a space before every function parameter list.
`google`	`prettier`	18	The Google style guide: single quotes, tight object braces, and maintained documentation blocks.
`airbnb`	`prettier`	19	The Airbnb style guide: single quotes, wider lines, grouped imports, and concise arrow bodies.
`compact`	`auric`	35	Minimise vertical space: wider lines, collapsed collections, and no decorative blank lines.
`expanded`	`auric`	28	Maximise readability and diff granularity: one item per line almost everywhere.
`legacy-tabs`	—	25	Tabs, four columns, CRLF line endings and Allman braces, for codebases that were here first.
`minimal-diff`	—	143	Change as little as possible: preserve the author's layout everywhere it is legal to do so.
`aligned`	`auric`	29	Turn the column alignment engine on everywhere it is safe, for code that reads like a table.
`sorted`	`auric`	30	Turn on every ordering rule the safety gate can verify, from import groups to stylesheet declarations.

Preset	Builds on	Options it sets	Description
`docs`	`auric`	19	Tuned for prose: reflowed paragraphs, generated tables of contents and maintained documentation blocks.
`verified`	`auric`	11	Run every safety check, re-format to prove idempotency, and fail the run rather than emit unverified output.

Which preset

If you	Choose
Want the house style this formatter was designed around	`auric`
Are moving from Prettier and want identical output on day one	`prettier`
Follow StandardJS	`standard`
Follow the Google style guides	`google`
Follow the Airbnb JavaScript style guide	`airbnb`
Want more code on screen	`compact`
Want maximum readability in review, one thing per line	`expanded`
Have a tab-indented codebase that is staying that way	`legacy-tabs`
Are formatting an existing codebase and want the smallest diff	`minimal-diff`
Like code aligned into columns	`aligned`
Want imports, keys and members sorted	`sorted`
Mostly write documentation and comments	`docs`
Want every safety check and a hard failure on anything unverified	`verified`

What each preset sets

auric

The Auric house style: Prettier's layout plus the Auric enhancements.

```
{
  "preset": "auric",
  "printWidth": 100,
  "endOfLine": "lf",
  "insertFinalNewline": true,
  "trimTrailingWhitespace": true,
  "objectWrap": "preserve",
  "preserveObjectShape": true,
  "preserveArrayFormatting": true,
  "singleLineIfStatement": "fit",
  "lastArgumentHugging": true,
  "firstArgumentHugging": true,
  "blankLineAfterImports": "always",
  "blankLineAfterDirectives": true,
  "blankLineBetweenMethods": "always",
  "maxBlankLinesAtStart": 0,
  "maxBlankLinesAtEnd": 0,
  "collapseEmptyBlockBlankLines": true,
  "commentSpacing": "always",
  "blockCommentStarAlign": true,
  "blockCommentStarPad": true,
  "preserveBannerComment": true,
  "jsdocAlign": true,
  "jsdocNormalizeTags": true,
  "jsdocTagOrder": [
    "@module",
    "@description",
    "@template",
    "@typedef",
    "@property",
    "@param",
    "@returns",
    "@yields",
    "@throws",
    "@example",
    "@remarks",
    "@see",
    "@link",
    "@since",
    "@version",
    "@author",
    "@deprecated",
    "@internal"
  ],
  "alignTrailingComments": true,
  "alignmentBreakOnBlankLine": true,
  "alignmentMaxPadding": 24,
  "alignmentUseSpacesOnly": true,
  "blankLineBetweenRules": 1,
  "nestedRuleBlankLine": true,
  "atRuleBlankLine": true,
  "propertyCase": "lower",
  "valueKeywordCase": "lower",
  "unitCase": "lower",
  "atRuleCase": "lower",
  "keyframeSelectorCase": "lower",
  "hexCase": "lower",
  "mdOrderedNumbering": "one",
  "mdEmphasisMarker": "asterisk",
  "mdStrongMarker": "asterisk",
  "mdTableAlign": true,
  "mdFormatCodeBlocks": true,
```

```
  "mdFrontmatterFormat": "format",
  "jsonPackageJsonOrder": true,
  "yamlCommentIndent": "align",
  "safetyTokenCheck": true,
  "safetyReparse": true,
  "safetyCommentPreservation": true,
  "safetyStringLiterals": true,
  "safetyNumericLiterals": true,
  "safetyOnFailure": "preserve"
}
```

prettier

Byte-compatible with Prettier 3 at its own defaults, with every Auric-only enhancement switched off.

It sets 281 options, too many to list usefully here. Choose it in Formatter Studio's **Preset** drop-down and use the **Diff** view to see what it changes in your own files.

standard

StandardJS: no semicolons, single quotes, and a space before every function parameter list. It builds on `prettier`.

```
{
  "preset": "standard",
  "semi": false,
  "singleQuote": true,
  "jsxSingleQuote": true,
  "quoteProps": "as-needed",
  "trailingComma": "none",
  "arrowParens": "avoid",
  "spaceBeforeFunctionParen": "always",
  "spaceBeforeBlocks": true,
  "spaceAfterKeyword": true,
  "semiColonBeforeIIFE": true,
  "curlyBraces": "multi-line",
  "objectWrap": "preserve",
  "printWidth": 80,
  "commentSpacing": "always"
}
```

google

The Google style guide: single quotes, tight object braces, and maintained documentation blocks. It builds on `prettier`.

```
{
  "preset": "google",
  "printWidth": 80,
  "tabWidth": 2,
  "singleQuote": true,
  "quoteProps": "consistent",
  "trailingComma": "all",
  "bracketSpacing": false,
  "arrowParens": "always",
  "semi": true,
  "curlyBraces": "all",
  "blankLineBetweenMethods": "always",
  "jsdocWrap": "always",
  "jsdocNormalizeTags": true,
  "jsdocTagOrder": [
    "@module",
    "@description",
    "@template",
    "@typedef",
    "@property",
    "@param",
    "@returns",
    "@yields",
    "@throws",
    "@example",
    "@remarks",
    "@see",
    "@link",
    "@since",
    "@version",
    "@author",
    "@deprecated",
    "@internal"
  ],
  "commentSpacing": "always",
  "htmlQuoteAttributes": "double",
  "propertyCase": "lower",
  "unitCase": "lower",
  "hexCase": "lower"
}
```

airbnb

The Airbnb style guide: single quotes, wider lines, grouped imports, and concise arrow bodies. It builds on `prettier`.

```
{
  "preset": "airbnb",
  "printWidth": 100,
  "singleQuote": true,
  "semi": true,
  "trailingComma": "all",
  "arrowParens": "always",
  "arrowBodyStyle": "as-needed",
  "bracketSpacing": true,
  "arrayBracketSpacing": false,
  "spaceBeforeBlocks": true,
  "spaceInParens": false,
  "objectWrap": "preserve",
  "maxBlankLines": 1,
  "blankLineAfterImports": "always",
  "sortImports": true,
  "importOrder": [
    "<BUILTIN_MODULES>",
    "<THIRD_PARTY_MODULES>",
    "^@/",
    "^~/",
    "^[.][.]/",
    "^[.]/"
  ],
  "importOrderSeparation": true,
  "importOrderSortSpecifiers": true,
  "importOrderCaseInsensitive": true,
  "commentSpacing": "always"
}
```

compact

Minimise vertical space: wider lines, collapsed collections, and no decorative blank lines. It builds on `auric`.

```
{
  "preset": "compact",
  "printWidth": 120,
  "objectWrap": "collapse",
  "preserveObjectShape": false,
  "preserveArrayFormatting": false,
  "shortArrayThreshold": 8,
  "jsonSingleLineArrays": true,
  "arrowBodyStyle": "as-needed",
  "singleLineArrowBody": true,
  "singleLineIfStatement": "fit",
  "curlyBraces": "multi-line",
  "emptyBlockStyle": "collapse",
  "preserveEnumOneLine": true,
  "maxBlankLines": 1,
  "maxBlankLinesInsideBlocks": 0,
  "blankLineBetweenMethods": "never",
  "blankLineAfterImports": "preserve",
  "blankLineAfterDirectives": false,
  "padClassBody": false,
  "padFunctionBody": false,
  "padBlocks": false,
  "blankLineBetweenRules": 0,
  "nestedRuleBlankLine": false,
  "atRuleBlankLine": false,
  "singleLineRuleThreshold": 1,
  "selectorSeparator": "space",
  "mediaQueryFormat": "compact",
  "scssMapFormat": "compact",
  "htmlBlankLineBetweenBlocks": 0,
  "htmlAttributeWrap": "auto",
  "singleAttributePerLine": false,
  "mdListItemBlankLine": "tight",
  "mdTableCompact": true,
  "graphqlSeparateOperations": 0,
  "tomlTableBlankLine": 0,
  "iniSectionBlankLine": 0
}
```

expanded

Maximise readability and diff granularity: one item per line almost everywhere. It builds on `auric`.

```
{
  "preset": "expanded",
  "printWidth": 80,
  "objectPropertyNewline": "always",
  "objectDestructuringNewline": "always",
  "functionParameterNewline": "always",
  "typeParameterNewline": "always",
  "arrayElementsPerLine": 1,
  "jsonExpandArrayThreshold": 2,
  "jsonSingleLineArrays": false,
  "unionFormat": "expanded",
  "intersectionFormat": "expanded",
  "mappedTypeFormat": "expanded",
  "emptyBlockStyle": "expand",
  "switchCaseBlankLine": "always",
  "blankLineBetweenMethods": "always",
  "blankLineBeforeReturn": "always",
  "blankLineBetweenTopLevel": "always",
  "singleAttributePerLine": true,
  "htmlAttributeWrap": "always",
  "jsxSingleAttributeSameLine": false,
  "selectorSeparator": "newline",
  "blankLineBetweenRules": 1,
  "nestedRuleBlankLine": true,
  "atRuleBlankLine": true,
  "mediaQueryFormat": "expanded",
  "scssMapFormat": "expanded",
  "mdListItemBlankLine": "loose",
  "graphqlArgumentWrap": "always",
  "tomlArrayWrap": "always"
}
```

legacy-tabs

Tabs, four columns, CRLF line endings and Allman braces, for codebases that were here first.

```
{
  "preset": "legacy-tabs",
  "useTabs": true,
  "tabWidth": 4,
  "printWidth": 120,
  "endOfLine": "crlf",
  "insertFinalNewline": true,
  "trimTrailingWhitespace": true,
  "alignmentUseSpacesOnly": false,
  "semi": true,
  "singleQuote": false,
  "trailingComma": "none",
  "bracketSpacing": true,
  "curlyBraces": "all",
  "elsePosition": "new-line",
  "catchPosition": "new-line",
  "doWhilePosition": "new-line",
  "switchCaseIndent": true,
  "universalBraceStyle": "next-line",
  "universalIndentSize": 4,
  "universalContinuationIndent": 4,
  "universalCaseIndent": true,
  "cPointerAlignment": "left",
  "javaAnnotationPosition": "above",
  "maxBlankLines": 2,
  "commentSpacing": "always",
  "blockCommentStarAlign": true
}
```

minimal-diff

Change as little as possible: preserve the author's layout everywhere it is legal to do so.

It sets 143 options, too many to list usefully here. Choose it in Formatter Studio's **Preset** drop-down and use the **Diff** view to see what it changes in your own files.

aligned

Turn the column alignment engine on everywhere it is safe, for code that reads like a table. It builds on `auric`.

```
{
  "preset": "aligned",
  "alignObjectValues": true,
  "alignAssignments": "consecutive",
  "alignConsecutiveDeclarations": true,
  "alignTrailingComments": true,
  "alignTernaryBranches": true,
  "alignEnumValues": true,
  "alignTypeMembers": true,
  "alignImportSpecifiers": true,
  "alignmentMaxPadding": 40,
  "alignmentMinGroupSize": 2,
  "alignmentBreakOnBlankLine": true,
  "alignmentUseSpacesOnly": true,
  "propertyValueAlign": true,
  "vendorPrefixAlign": true,
  "gridTemplateAlign": true,
  "yamlAlignValues": true,
  "tomlAlignValues": true,
  "iniAlignValues": true,
  "graphqlFieldAlign": true,
  "jsdocAlign": true,
  "universalAlignAssignments": true,
  "universalAlignTrailingComments": true,
  "universalAlignConsecutive": true,
  "goAlignStructTags": true,
  "goAlignComments": true,
  "rustAlignMatchArms": true,
  "rustAlignStructFields": true,
  "sqlAlignSelectColumns": true,
  "pythonAlignDictValues": true
}
```

sorted

Turn on every ordering rule the safety gate can verify, from import groups to stylesheet declarations. It builds on `auric`.

```
{
  "preset": "sorted",
  "sortImports": true,
  "importOrder": [
    "<BUILTIN_MODULES>",
    "<THIRD_PARTY_MODULES>",
    "^@/",
    "^~/",
    "^[.][.]/",
    "^[.]/"
  ],
  "importOrderSeparation": true,
  "importOrderSortSpecifiers": true,
  "importOrderCaseInsensitive": true,
  "importOrderGroupNamespaceSpecifiers": true,
  "importOrderSideEffectsFirst": true,
  "importOrderTypeImports": "last",
  "exportOrder": "alphabetical",
  "sortObjectKeys": true,
  "sortUnionMembers": true,
  "sortInterfaceMembers": true,
  "sortClassMembers": true,
  "classMemberOrder": [
    "static-field",
    "field",
    "constructor",
    "static-method",
    "method",
    "getter",
    "setter"
  ],
  "sortCaseSensitivity": "insensitive",
  "sortNaturalNumeric": true,
  "sortPreserveComments": true,
  "sortRespectBlankLineGroups": true,
  "jsxSortProps": true,
  "htmlSortAttributes": true,
  "xmlSortAttributes": true,
  "declarationOrder": "grouped",
  "yamlKeySort": true,
  "jsonSortKeys": true,
  "jsonPackageJsonOrder": true,
  "tomlSortKeys": true,
  "graphqlSortDefinitions": true,
  "graphqlSortFields": true,
  "jsdocTagOrder": [
    "@module",
    "@description",
    "@template",
    "@typedef",
    "@property",
    "@param",
    "@returns",
    "@yields",
    "@throws",
    "@example",
    "@remarks",
    "@see",
    "@link",
    "@since",
    "@version",
    "@author",
```

```
    "@deprecated",
    "@internal"
  ],
  "jsdocNormalizeTags": true
}
```

docs

Tuned for prose: reflowed paragraphs, generated tables of contents and maintained documentation blocks. It builds on `auric`.

```
{
  "preset": "docs",
  "printWidth": 80,
  "proseWrap": "always",
  "commentWrap": "always",
  "commentWrapWidth": 80,
  "commentSpacing": "always",
  "jsdocWrap": "always",
  "jsdocAlign": true,
  "jsdocNormalizeTags": true,
  "jsdocTagOrder": [
    "@module",
    "@description",
    "@template",
    "@typedef",
    "@property",
    "@param",
    "@returns",
    "@yields",
    "@throws",
    "@example",
    "@remarks",
    "@see",
    "@link",
    "@since",
    "@version",
    "@author",
    "@deprecated",
    "@internal"
  ],
  "jsdocBlankLineAfterDescription": true,
  "mdTocGenerate": true,
  "mdTocMaxDepth": 3,
  "mdHeadingBlankLines": 1,
  "mdTableAlign": true,
  "mdLinkStyle": "inline",
  "mdFormatCodeBlocks": true,
  "mdEscapeSpecial": true,
  "mdFrontmatterFormat": "format",
  "experimentalCommentReflow": true
}
```

verified

Run every safety check, re-format to prove idempotency, and fail the run rather than emit unverified output. It builds on `auric`.

```
{
  "preset": "verified",
  "safetyTokenCheck": true,
  "safetyTokenCheckComments": true,
  "safetyIdempotency": true,
  "safetyIdempotencyPasses": 2,
  "safetyReparse": true,
  "safetyCommentPreservation": true,
  "safetyStringLiterals": true,
  "safetyNumericLiterals": true,
  "safetyOnFailure": "throw",
  "maxFileSizeKb": 8192,
  "parseTimeoutMs": 15000
}
```

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Preview formatting as a diff

See the formatted result beside your source in a read-only editor, compare the changes, and apply or discard them.

The native preview shows what formatting would do before your file changes. It opens a read-only copy of the formatted result in a normal VS Code editor beside your source; no Studio panel opens.

Open a preview

1. Focus the source file in the editor.
2. Use any of these:
   - **Auric Formatter: Preview Formatting as a Diff** (`Ctrl` + `Alt` + `Shift` + `F`; `Cmd` + `Alt` + `Shift` + `F` on macOS);
   - the **Preview Formatting as a Diff** button in the editor title bar;
   - right-click with nothing selected and choose **Preview Formatting as a Diff**;
   - **Preview beside editor** in Formatter Studio.
3. A read-only tab named **Preview — *file name*** opens beside the source, with the same language highlighting.

With text selected, only the selection is formatted in the preview (it may grow to a complete syntax unit).

If formatting fails or the safety gate refuses the change, no preview opens and a warning says why, for example **Auric Formatter: the safety gate refused this change**.

Act on the preview

The preview shows CodeLens links at the top and buttons in the editor title bar:

Control	What it does
Apply formatting	Replace the source with the preview, after confirming **Apply formatting to *file*?** (**The source stays unsaved. You can undo this edit.**)
Compare changes	Open VS Code's diff editor, **Formatting changes — *file***, source on the left and result on the right
Discard preview	Close the preview; the source is unchanged
Full file · refresh preview (or **Selection preview · refresh from source**)	Format the source again and update the preview

When the result is the same as the source, the first CodeLens reads **No formatting changes** and opens the comparison instead.

The same actions are commands: **Apply Formatting Preview**, **Compare Formatting Changes**, **Discard Formatting Preview** and **Refresh Formatting Preview**. The title-bar buttons appear only while a preview is focused.

Stale previews

The preview is tied to the exact version of your source. If you edit the source after opening it:

- the CodeLens changes to **Source changed — refresh preview**;
- **Apply** and **Compare** refuse with **The source changed. Refresh the preview before applying.**

Refresh, review again, then apply. A preview can never overwrite a different document or a newer version of the same one.

Limits

- Up to 24 previews can be open at once; beyond that you are asked to close or discard one first.
- Focus the source editor, not a preview, before creating a preview: **focus the source editor to create a preview.**
- If you close the source file, **Refresh** asks you to reopen it.

After **Apply formatting**, the source shows the formatted text, unsaved, and the preview closes.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

The safety gate and formatting tiers

How Auric Artisan Formatter checks every result before it reaches your file, what the native and universal tiers can change, and how to see why a file was formatted as it was.

A formatter rewrites your source. That is only acceptable if it can show it did not change what the source means. Auric Artisan Formatter checks every file before you see the result.

The five checks

Check	What it proves	Option
Token stream	The output re-reads to the same significant tokens as the input: no dropped argument, swallowed operator or mangled string	`safetyTokenCheck` (and `safetyTokenCheckComments` to include comments)
Comments	Every comment in the input appears in the output	`safetyCommentPreservation`
String literals	Every string and template keeps the same content	`safetyStringLiterals`
Reparse	The output parses again with the same parser	`safetyReparse`
Idempotency	Formatting the output again gives identical bytes	`safetyIdempotency` (off by default; `safetyIdempotencyPasses` sets the extra passes)

`safetyNumericLiterals` also requires every number to keep its value when its spelling changes.

The gate knows which changes your options allow. With `singleQuote` on, it expects quotes to change; with `cssTrailingSemicolon` on, it knows the last `;` before `}` is optional in CSS. A change your options do not allow is treated as corruption and reverted.

When a check fails

Your original text is kept — not partly applied, not a best effort. The status bar shows **Auric** with a shield and a warning background (**your original text was kept**), and the log records which check failed and where. Run **Auric Formatter: Show the Formatter Log** to read it.

`auricFormatter.safetyOnFailure` decides what happens:

Value	Behaviour
`preserve` (default)	Return the original text unchanged, with a warning
`warn`	Return the formatted text, with a warning describing the check that failed
`throw`	Fail the whole run with a safety error, which is what CI wants (used by the `verified` preset)

Because `warn` lets unverified output through, keep `preserve` unless you have a specific reason.

You can switch individual checks off with the `safety*` options, for example to save time on very large files, but they are the reason the formatter can safely be aggressive about layout. The `verified` preset turns every check on.

The tiers

Not every language has a full parser, and pretending otherwise would be unsafe. Each language sits in one tier:

Native

A real parser and pretty-printer. The file is rebuilt from its syntax tree, so any layout decision is available: line breaks are chosen, quotes, semicolons, trailing commas and property quotes are normalised, and comments are re-attached. 31 native modes are verified: JavaScript, TypeScript, JSON, JSONC, JSON5, JSON.stringify, YAML, TOML, CSS, SCSS, Less, PostCSS, HTML, XML, SVG, Vue, Svelte, Astro, Markdown, MDX, GraphQL queries and schema, Handlebars, and the YAML- and JSON-based OpenAPI, AsyncAPI, Swagger, RAML, Docker Compose, Pulumi, Ansible and Kubernetes formats.

Universal

A structural engine for 220 other languages, guided by a grammar profile for each. It follows one rule: **one source line in, one source line out**. It only rewrites a line's leading whitespace and the gaps between tokens already on that line, so it cannot reorder, insert or delete a token. How much it does depends on the language:

Depth	Languages	What it changes
Reindent and spacing	104	Indentation and spacing, for languages whose blocks are in the syntax: C, C#, C++, Go, Java, Kotlin, Rust, Swift, Ruby, Lua, shell, PowerShell, Terraform and more
Spacing only	60	Spacing only, where indentation is meaning or layout is sensitive: Python, Haskell, OCaml, SQL, Sass, Pug, Lisp, Erlang and more
Whitespace only	60	Trailing whitespace, blank lines, line endings and the final newline, where leading whitespace is data: Makefiles, CSV, INI, COBOL, templates, prose markup

Some things are deliberately left alone: `<` and `>` are never spaced (only a parser can tell `a > b` from `Vec<T>`), arithmetic spacing is enabled per language (in shell, `--flag` must not become `-- flag`), and brace padding (`{ a: 1 }` versus `{a: 1}`) is left to the author.

The engine declines rather than guesses: a file whose brackets do not balance keeps its indentation and says so, and a file the engine cannot fully read is returned untouched.

The full lists are in Supported languages.

Passthrough

For anything unrecognised, only the changes that cannot alter any language's meaning: trailing whitespace, runs of blank lines, line endings and the final newline.

Falling back

A file whose native parse fails falls back a tier rather than going unformatted.

JSX and TSX. In version 0.1.0, a `.jsx` or `.tsx` file that contains JSX elements falls back to the universal tier: correctly indented and spaced, but not pretty-printed. Plain TypeScript in a `.tsx` file is formatted natively. The parser refuses JSX rather than guess, because a confidently wrong syntax tree is how a formatter corrupts a file.

See why a file was formatted as it was

Auric Formatter: Explain Formatting Decisions for This File opens a Markdown report for the active file with:

- the language and parser chosen;
- the tier used and the time taken;
- which configuration layer (preset, configuration file, `.editorconfig`, VS Code setting or directive) set each option;
- the options changed from their defaults;
- any safety warnings.

Formatter Studio's **Explain** view shows the same for the code in Studio, and the command line has `--explain`.

When the gate passes, the status bar shows **Auric** with a tick and names the tier; when it reverts, your file is exactly as you wrote it.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Supported languages

Every language Auric Artisan Formatter formats, grouped by how much it can change.

The formatter recognises 253 languages by file extension, exact file name, shebang line and VS Code language id. What it does with each depends on its tier; see The safety gate and formatting tiers.

Tier	Languages	What changes
Native	33 registry entries; 31 verified native modes	Anything layout-related: the file is rebuilt from a real parse
Universal: reindent and spacing	104	Indentation and spacing
Universal: spacing only	60	Spacing between tokens; indentation untouched
Universal: whitespace only	60	Trailing whitespace, blank lines, line endings, final newline

The two native entries that are not yet verified are JavaScript React (`jsx`) and TypeScript React (`tsx`): a file containing JSX elements falls back to the universal tier in version 0.1.0.

To see the list yourself, run `auric-format --list-languages` (see Command line and CI). Formatter Studio's **Lang** drop-down groups languages into **Native printers** and **Universal engine**.

Native tier

The "Formatter id" is the name used in the formatter's `languages` configuration.

Language	Formatter id	File extensions or names
JavaScript	`javascript`	`.js` `.cjs` `.mjs` `.es` `.es6` `.pac` and more
JavaScript React	`jsx`	`.jsx`
TypeScript	`typescript`	`.ts` `.mts` `.cts` `.d.ts` `.d.mts` `.d.cts`
TypeScript React	`tsx`	`.tsx` `.mtsx`
JSON	`json`	`.json` `.geojson` `.topojson` `.jsonld` `.importmap` `.har` and more
JSON5	`json5`	`.json5`
JSON with Comments	`jsonc`	`.jsonc`
JSON.stringify	`json-stringify`	`package.json` `package-lock.json` `bower.json` `.babelrc.json`
CSS	`css`	`.css` `.wxss`
SCSS	`scss`	`.scss`
Less	`less`	`.less`
PostCSS	`postcss`	`.pcss` `.postcss` `.sss`
HTML	`html`	`.html` `.htm` `.xhtml` `.xht` `.shtml` `.html.hl` and more
Vue	`vue`	`.vue`
Svelte	`svelte`	`.svelte`
Astro	`astro`	`.astro`
XML	`xml`	`.xml` `.xsd` `.xsl` `.xslt` `.rss` `.atom` and more
SVG	`svg`	`.svg` `.svgz`
Markdown	`markdown`	`.md` `.markdown` `.mdown` `.mkd` `.mkdn` `.mdwn` and more
MDX	`mdx`	`.mdx`
Handlebars	`handlebars`	`.handlebars` `.hbs` `.hbs.html`
YAML	`yaml`	`.yaml` `.yml` `.yaml.tmpl` `.mir` `.reek` `.rviz` and more
TOML	`toml`	`.toml` `.tml`
GraphQL	`graphql`	`.graphql` `.gql`
GraphQL SDL	`graphql-schema`	`.graphqls` `.gqls` `.sdl`
OpenAPI	`openapi`	`openapi.yaml` `openapi.yml` `openapi.json` `api.yaml`
AsyncAPI	`asyncapi`	`asyncapi.yaml` `asyncapi.yml` `asyncapi.json`
Swagger 2.0	`swagger`	`swagger.yaml` `swagger.yml` `swagger.json`
RAML	`raml`	`.raml`

Language	Formatter id	File extensions or names
Docker Compose	`docker-compose`	`docker-compose.yml` `docker-compose.yaml` `docker-compose.override.yml` `docker-compose.override.yaml`
Pulumi	`pulumi`	`Pulumi.yaml` `Pulumi.yml`
Ansible	`ansible`	`playbook.yml` `playbook.yaml` `site.yml` `site.yaml`
Kubernetes manifest	`kubernetes`	`kustomization.yaml` `kustomization.yml` `kubeconfig`

Files identified by name

Some formats share an extension with another, and a few files are recognised by name. For example, `docker-compose.yml`, `compose.yaml`, `openapi.yaml`, `swagger.json`, `asyncapi.yaml`, `Pulumi.yaml`, `playbook.yml` and `kustomization.yaml` get their own native handling. Ambiguous extensions are resolved to one language by default: `.h` is C, `.m` is Objective-C, `.pl` is Perl, `.sql` is SQL and `.ts` is TypeScript. Set `auricFormatter.parser` or a `languages` entry if you need a different interpretation.

Universal tier: reindent and spacing (104)

Indentation and spacing are normalised. JavaScript, TypeScript and their React variants appear here because this is where they fall back when a native parse is not possible.

ABAP, Ada, Alloy, ANTLR, Apache Avro IDL, Apache config, Apache Thrift, Apex, AppleScript, ASN.1, AWK, Bash, BASIC, Bicep, C, C#, C++, Caddyfile, Cairo, Cap'n Proto, CMake, Crystal, CUDA, CUE, D, Dart, Delphi, Elixir, Fish, FlatBuffers, Fortran, Gleam, GLSL, GNU Octave, gnuplot, Go, Gradle, Graphviz DOT, Groovy, Hack, Haxe, HCL, Hjson, HLSL, Java, JavaScript, JavaScript React, jq, Jsonnet, Julia, KDL, Kotlin, Lex / Flex, Lua, M4, MATLAB, Metal Shading Language, Move, nginx, Nix, Nushell, Objective-C, Objective-C++, Odin, OpenCL, Pascal, Perl, PHP, PowerShell, Protocol Buffers, Puppet, R, Raku, Reason, ReScript, REXX, Ruby, Rust, Rusty Object Notation, Scala, Shell script, Smalltalk, Solidity, Stan, Stata, Swift, SystemVerilog, Tcl, Terraform, TypeScript, TypeScript React, TypoScript, V, Vala, Verilog, VHDL, Vim script, Visual Basic, Web IDL, WGSL, XQuery, Yacc / Bison, Zig, Zsh.

Universal tier: spacing only (60)

Indentation is left exactly as written, because it carries meaning or the layout is sensitive; spacing between tokens is normalised.

Agda, AutoHotkey, Batch file, Bazel, Clojure, ClojureScript, CoffeeScript, Common Lisp, Coq / Rocq, Cypher, Cython, Dhall, EDN, Elm, Emacs Lisp, Erlang, F#, Fennel, GDScript, Haml, Haskell, HiveQL, Idris, Isabelle, Jade, Janet, Lean, LiveScript, Mercury, Meson, Michelson, MySQL, Nim, OCaml, PL/SQL, PostgreSQL, Prolog, Pug, PureScript, Python, Racket, SAS, Sass (indented), Scheme, Slim, SMT-LIB, Spark SQL, SPARQL, SPSS, SQL, SQLite SQL, Standard ML, Starlark, Stylus, T-SQL, TLA+, Vyper, WebAssembly, WebAssembly Text, Wolfram Mathematica.

Universal tier: whitespace only (60)

Only trailing whitespace, blank-line runs, line endings and the final newline change, because leading whitespace is data in these formats.

Apache Ant, AsciiDoc, ASP.NET Web Forms, Assembly, BibTeX, Blade, COBOL, ColdFusion Markup, Containerfile, Creole, CSV, Diff / patch, Dockerfile, Document Type Definition, Dotenv, EditorConfig, EJS, ERB, Forth, FreeMarker, Gherkin, Git attributes, Git commit message, Git config, Git ignore, Git rebase todo, Go module, Helm chart, INI, Java Properties, JavaServer Pages,

Jinja, JSON Lines, Just, LaTeX, Liquid, LLVM IR, Makefile, MediaWiki, Mermaid, Mustache, Ninja, Nunjucks, Org Mode, PlantUML, Razor, reStructuredText, Robot Framework, RPG (IBM i), sed, Smarty, systemd unit, Texinfo, Textile, Thymeleaf, TSV, Turtle / RDF, Twig, Velocity, XAML.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Scan the workspace

See how many files would change before formatting a repository, review them in the Workspace tab, format them explicitly, or check without writing.

Adopting a formatter on an existing codebase is mostly a question of trust. Format on save quietly rewrites the first file you open; formatting the whole workspace rewrites thousands at once. The workspace scan is a third option: it formats every supported file **in memory**, tells you what would change, and writes nothing.

What the scan does

A few seconds after the window opens (`auricFormatter.scanOnStartup`, on by default), the formatter:

- reads every supported file in the workspace and formats it in memory;
- records whether each file would change, and how many lines it would gain and lose;
- notes files that fail to parse, are skipped, or trip the safety gate.

Nothing is written to disk; your working tree and `git status` stay as they were.

The scan also follows your work: saved and unsaved edits, external changes and created or deleted files update the results, and new files trigger a short, delayed rescan. Changing the preset or saving a configuration file re-runs the scan, because they change what "already formatted" means.

Where the results appear

- **The status bar** shows ***N* unformatted** with a warning background. The tooltip says ***N* of *M* files would change if formatted. Nothing has been written — click to review them.** When everything is formatted, it shows **Auric** with a double tick.
- **The Workspace tab** of Formatter Studio lists every file.
- **Check Formatting Without Writing (CI Mode)** puts the same result in the Problems panel.

Run a scan on demand

Auric Formatter: Scan the Workspace Without Writing runs the scan now, even if the startup scan is off. Afterwards:

- if files would change: ***N* of *M* files would change. Nothing has been written.** with **Review them** and **Format them all**;
- otherwise: **all *M* files are already formatted.**

The Workspace tab

Open it with **Show Files That Need Formatting** (`Ctrl` + `Alt` + `Shift` + `U`), the status bar item, or **02 Workspace** in Studio.

Control	Options
Which files to show	**Needs formatting** (files whose bytes would change), **Every scanned file**, **Already formatted**, **Errors and skips** (parse failures, safety reverts, skipped files)
Sort order	**Largest change first** (what a review will notice), **Path** (alphabetical, grouped by folder), **Language**, **File size** (biggest first)
Filter by path…	Type part of a path
Rescan / **Stop**	Format every file in memory again, or stop a running scan
Tick boxes and **Select every listed file**	Choose files
Format selected	Write the ticked files
Format all files	Write every file that needs formatting
A row's diff button	Open that file with a preview of what would change
A row's write button	Write this one file

Each row shows the path, language, tier and the lines added and removed.

Taking the change

All three ways are explicit:

1. **Format all files** writes every pending file at once.
2. Tick rows, then **Format selected**, to take a folder at a time so each commit stays reviewable.
3. The diff button on a row lets you read the change for one file first.

Writing more than one file asks **Format *N* files?** first, while `auricFormatter.confirmBeforeFormattingAll` is on. Each file is re-read and re-formatted at the moment it is written, so a file you edited since the scan is never overwritten with a stale result; a file open with unsaved changes is written through the editor. Batch formatting saves the files, unlike a single-document **Apply**, which leaves the file unsaved.

The result is reported as **formatted *N* files**, or with a count of failures and **Show log**.

Check without writing (CI mode)

Auric Formatter: Check Formatting Without Writing (CI Mode) runs a fresh scan and adds a Problems entry for every file that is not formatted (**This file is not formatted (*N* lines added, *M* removed). Run Auric Formatter to fix it.**) or could not be formatted. It then reports ***N* of *M* files need formatting. See Problems.** For the same check in a CI pipeline, use the command line: see Command line and CI.

Scan settings

Setting	Default	Effect
`auricFormatter.scanOnStartup`	`true`	Scan when the window opens
`auricFormatter.scanExclude`	empty (built-in list)	Globs the scan skips. Empty uses `node_modules`, `.git`, `dist`, `build`, `out`, `coverage`, `vendor`, `target`, `.next`, `.venv`, minified files and source maps
`auricFormatter.scanMaxFiles`	4000	Stop after this many files, so a very large monorepo is bounded

Setting	Default	Effect
`auricFormatter.confirmBeforeFormattingAll`	`true`	Ask before writing more than one file

Files larger than `auricFormatter.maxFileSizeKb` are skipped before they are read. In a multi-root workspace, each file is identified with its folder name.

The scan is not an unlimited index: exclusions, the file-size limit and `scanMaxFiles` always apply.

When you format selected files, the Workspace tab removes them from **Needs formatting**, and the status bar count goes down.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Reference: commands, keys and settings

Every Auric Artisan Formatter command, keyboard shortcut, menu entry, status bar state, file and editor setting.

This page covers Auric Artisan Formatter 0.1.0: 21 commands, 3 keyboard shortcuts, one Explorer view, a setup walkthrough, 11 editor settings and 391 formatting options (listed in the options reference). Commands appear in the Command Palette with the **Auric Formatter:** prefix.

Commands

Command	ID	Keys	What it does
Format Document	`auricFormatter.formatDocument`	—	Format the active file with Auric
Format Selection	`auricFormatter.formatSelection`	—	Format the selected code (shown only with a selection)
Format Every File in the Workspace	`auricFormatter.formatWorkspace`	—	Write every file the scan found would change (asks first)
Format Every File That Needs It	`auricFormatter.formatAll`	—	The same, from the scan results (asks first)
Scan the Workspace Without Writing	`auricFormatter.scanWorkspace`	—	Format every file in memory and report; offers **Review them** or **Format them all**
Show Files That Need Formatting	`auricFormatter.showPending`	`Ctrl` + `Alt` + `Shift` + `U`	Open Formatter Studio on the **Workspace** tab
Check Formatting Without Writing (CI Mode)	`auricFormatter.checkWorkspace`	—	Scan and list unformatted files in the Problems panel
Open Formatter Studio	`auricFormatter.openPlayground`	`Ctrl` + `Alt` + `Shift` + `P`	Open Formatter Studio
Preview Formatting as a Diff	`auricFormatter.previewDiff`	`Ctrl` + `Alt` + `Shift` + `F` (editor focused)	Open a read-only formatted copy beside the source
Apply Formatting Preview	`auricFormatter.applyPreview`	—	Apply the open preview to its source, after confirmation
Discard Formatting Preview	`auricFormatter.discardPreview`	—	Close the preview without changes
Refresh Formatting Preview	`auricFormatter.refreshPreview`	—	Format the source again and update the preview

Command	ID	Keys	What it does
Compare Formatting Changes	`auricFormatter.comparePreview`	—	Open the diff between source and preview
Explain Formatting Decisions for This File	`auricFormatter.explainFormatting`	—	Open a report of language, parser, tier, option sources and warnings
Switch Formatting Preset	`auricFormatter.selectPreset`	—	Choose one of the 13 presets
Generate a Configuration File	`auricFormatter.generateConfig`	—	Write `.auricformatrc` with your preset and settings
Import Settings From Prettier	`auricFormatter.migrateFromPrettier`	—	Convert a Prettier configuration into an unsaved Auric one
Toggle Format on Save	`auricFormatter.toggleFormatOnSave`	—	Turn format on save on (and make Auric the default formatter) or off
Show the Formatter Log	`auricFormatter.showOutput`	—	Open the **Auric Artisan Formatter** output channel
Report a Formatting Problem	`auricFormatter.reportIssue`	—	Open a Markdown template with version, language, tier and explanation
Clear the Configuration Cache	`auricFormatter.clearCache`	—	Forget cached configuration so files are re-read

On macOS, `Cmd` replaces `Ctrl` in the three shortcuts. **Format Every File That Needs It** and **Format Every File in the Workspace** have no shortcut on purpose.

Where commands appear

- Editor title bar: **Preview Formatting as a Diff** (supported files)
- Editor title bar, in a preview: **Apply Formatting Preview**, **Compare Formatting Changes**, **Refresh Formatting Preview**, **Discard Formatting Preview**
- Preview CodeLens: **Apply formatting** (or **No formatting changes**, or **Source changed — refresh preview**), **Compare changes**, **Discard preview**, **Full file · refresh preview** / **Selection preview · refresh from source**
- Editor context menu: **Format Selection** with a selection; **Preview Formatting as a Diff** without
- Explorer context menu, on a folder: **Format Every File in the Workspace**
- Explorer: The **Auric Formatter** view
- Welcome page: The walkthrough **Set up Auric Artisan Formatter**

Status bar

Text	Meaning
Auric (brush icon)	Ready for this language
Auric with a tick	Formatted, or already formatted; the tooltip names the tier and the time
Auric with a shield	The safety gate kept your original text
Auric with a slash	Skipped, with the reason in the tooltip
Auric with an error	Formatting failed
Auric ***scanned/total*** (spinning)	Scanning in memory

Text	Meaning
Auric writing…	Writing formatted files
***N* unformatted**	Files the scan found would change; select to review
Auric with a double tick	Every scanned file is formatted

The first four open the log when selected.

Editor settings

Setting	Type	Default	Values or range	What it does
`auricFormatter.preset`	string	`auric`	13 presets (see Presets)	The preset to start from; your own options and a configuration file's `preset` win
`auricFormatter.formatOnType`	boolean	`false`	—	Reformat the line and block as you type closing punctuation; needs `editor.formatOnType`
`auricFormatter.formatOnSaveMode`	string	`file`	`file`, `modifications`	Format the whole file on save, or only lines changed since the last commit
`auricFormatter.showStatusBar`	boolean	`true`	—	Show the status bar item for the tier and safety result
`auricFormatter.logLevel`	string	`warn`	`silent`, `error`, `warn`, `log`, `debug`	How much reaches the output channel
`auricFormatter.enabledLanguages`	array	`[]`	Language ids	Format only these languages; empty means all
`auricFormatter.disabledLanguages`	array	`[]`	Language ids	Never format these languages
`auricFormatter.scanOnStartup`	boolean	`true`	—	Scan the workspace in memory when the window opens
`auricFormatter.scanExclude`	array	`[]`	Globs	Globs the scan skips; empty uses the built-in list
`auricFormatter.scanMaxFiles`	integer	`4000`	1 or more	Stop the scan after this many files
`auricFormatter.confirmBeforeFormattingAll`	boolean	`true`	—	Ask before writing more than one file

All settings, including the 391 options, have resource scope: they can be set for the user, the workspace or a single folder.

Files

File	Role
`.auricformatrc` and its variants, `auric.format.json`, `package.json` (`auricFormatter` key)	Configuration; see Configuration
`.prettierrc` and variants, `package.json` (`prettier` key)	Read for compatibility
`.editorconfig`	Read for indentation, line endings, line length and final newline
`.auricformatignore`, `.prettierignore`	Files not to format

Written by the extension:

What	When
`.auricformatrc`	**Generate a Configuration File**
`editor.defaultFormatter`, `editor.formatOnSave`	**Toggle Format on Save** (workspace settings, or user settings with no folder open)
`auricFormatter.preset`	**Switch Formatting Preset**
Your source files	Format commands, **Apply**, format on save, **Format selected** / **Format all files**

Workspace trust and virtual workspaces

- In an untrusted workspace, configuration files are read but JavaScript configuration files and formatter plugins are not run.
- Virtual workspaces are supported.

Setup walkthrough

Set up Auric Artisan Formatter on the Welcome page has five steps: **Choose a preset**, **Try the Playground**, **See what the whole repository would do**, **Turn on format on save** and **Understand the safety gate**.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Troubleshooting

Symptoms, causes and fixes for common Auric Artisan Formatter problems.

For anything not listed, run **Auric Formatter: Report a Formatting Problem**, add what happened and a small sample file, and see Get support. **Show the Formatter Log** has the details of each run; set `auricFormatter.logLevel` to `debug` for more.

Nothing happens when I save

- **Why:** Auric is not the default formatter, format on save is off, or the language is excluded.
- **Fix:** run **Toggle Format on Save**, or check `editor.defaultFormatter` and `editor.formatOnSave` (including language-specific blocks such as `"[typescript]"`). Check `auricFormatter.enabledLanguages` and `auricFormatter.disabledLanguages`.

VS Code asks which formatter to use on every save

- **Why:** several formatters are installed and no default is set.
- **Fix:** run **Toggle Format on Save**, which sets `editor.defaultFormatter`, or set it yourself per language.

The status bar shows a shield: "your original text was kept"

- **Why:** a safety check failed, so the formatter refused to change the file.
- **Fix:** open the log to see which check failed and where. Often the file has a construct the parser does not handle yet; the file is safe as it is. Report it with **Report a Formatting Problem** if you think the change should have been allowed.

The file was skipped

- **Why:** it is larger than `auricFormatter.maxFileSizeKb`, looks minified, is in `.auricformatignore` or `.prettierignore`, has an ignore directive, or `requirePragma` is on and it has no `@format`.
- **Fix:** hover the status bar item for the reason. Raise the size limit, turn on `auricFormatter.formatMinified`, or remove the ignore entry, as appropriate.

JSX or TSX is only indented, not reformatted

- **Why:** in version 0.1.0 a file containing JSX elements falls back to the universal tier.
- **Fix:** none yet; the output is still safe. Plain TypeScript in `.tsx` files is formatted natively.

A language is only lightly formatted

- **Why:** it uses the universal tier, which only changes whitespace, and for some languages only spacing or only trailing whitespace. See Supported languages.
- **Fix:** none needed; this is deliberate so the meaning cannot change.

My configuration file is ignored

- **Why:** another file takes precedence, the workspace is untrusted and the file is JavaScript, the JavaScript file is an ES module, or the cache is stale.
- **Fix:** run **Explain Formatting Decisions** to see which file and layer set each option. Trust the workspace, rename an ES module config to `.cjs` or use `.auricformatrc.json`, and run **Clear the Configuration Cache**.

Output matches Prettier, not the Auric style

- **Why:** the project has only a Prettier configuration, so the `prettier` preset was adopted automatically.
- **Fix:** add an `.auricformatrc` with the preset you want, or run **Import Settings From Prettier** and remove `"preset": "prettier"`.

"Select some code first."

- **Fix:** select text before **Format Selection**.

Preview problems

- **"The source changed. Refresh the preview before applying."** — you edited the source after opening the preview. Refresh, review, then apply.
- **"Close or discard an existing formatting preview before opening another."** — 24 previews are open. Discard some.
- **"focus the source editor to create a preview."** — the preview itself is focused. Click in the source file first.
- **No preview opened, with a warning** — formatting failed or the safety gate refused the change; the warning says which.

Formatter Studio problems

- **"Preview is limited to 200,000 characters."** — use editor formatting or the native preview for larger files.
- **"Unsupported rule: *name*"** — the custom rules JSON contains a name that is not a formatting option (or a range option). Check the spelling in the options reference.
- **"The file changed since this preview. Reload from the editor and review again."** — select **Reload editor** and review again before **Yes, apply**.
- **A blank or scratch preview cannot be applied** — text typed into Studio, or a truncated preview, can only be copied.

Workspace scan problems

- **The status bar shows no count** — the scan is off (`auricFormatter.scanOnStartup`) or no folder is open. Run **Scan the Workspace Without Writing**.
- **Files are missing from the scan** — they match `auricFormatter.scanExclude` or the built-in exclusions, exceed `maxFileSizeKb`, or the scan stopped at `scanMaxFiles`.
- **Format Every File in the Workspace formatted more than one folder** — in this version it formats every pending file in the workspace. Use the Workspace tab's tick boxes and **Format selected** to take one folder at a time.

"No Prettier configuration found in this workspace."

- **Why: Import Settings From Prettier** found no `.prettierrc`, `.prettierrc.json`, `.prettierrc.yaml`, `.prettierrc.yml`, `.prettierrc.js` or `prettier.config.js` outside `node_modules`.
- **Fix:** none needed if you do not use Prettier; create a configuration with **Generate a Configuration File** instead.

The command line is not found

- **Why:** `auric-format` is a script inside the installed extension, not a command on your `PATH`.
- **Fix:** run it with Node from the extension folder, or create an alias. See Command line and CI.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

FAQ

Short answers to common questions about Auric Artisan Formatter.

Will it rewrite my repository when I install it?

No. The startup scan only formats in memory and reports a count. Files are written only when you format them: on save (if you turn it on), with a Format command, with **Apply**, or with **Format selected** or **Format all files**.

Can it produce exactly the same output as Prettier?

The `prettier` preset is byte-compatible with Prettier 3 at its own defaults, with every Auric-only enhancement off. A project with only a Prettier configuration gets that preset automatically.

How do I keep `git blame` useful?

Set `auricFormatter.formatOnSaveMode` to `modifications` so format on save only touches lines you changed, or use the `minimal-diff` preset. When you do format everything, use the Workspace tab to take a folder at a time.

Could it break my code?

Every result passes the safety gate: tokens, comments, strings and a reparse are checked before anything is written, and on failure your original text is kept. The universal engine can only change whitespace. Keep using version control as usual.

Which languages does it support?

253 languages: 31 native modes plus 220 on the universal tier. See Supported languages.

Does it format JSX?

In version 0.1.0, files containing JSX elements are indented and spaced by the universal tier but not pretty-printed.

Where do I put my settings?

In a `.auricformatrc` at the project root, so everyone gets the same result. VS Code settings work too. See Configuration.

How do I find the option I want?

Open Formatter Studio, load your file and choose **Find what matters**. It lists only the options that would change that file.

Can I use it in CI?

Yes. Run the command-line check, which exits with code 1 when files need formatting. See Command line and CI.

Does it send my code anywhere?

No. Formatting runs locally with no network requests and no runtime dependencies.

Is the interface available in Hindi?

Not at present; the interface is in English.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Supporting material

Contents of the supporting pages for Auric Artisan Formatter.

These pages support the Formatter guides: terms, shortcuts, limits and guarantees, privacy, related products and the version history.

File	What it covers
glossary.md	Terms used in the Formatter documentation
shortcuts.md	Every keyboard shortcut
limits-and-accuracy.md	What the safety gate guarantees, known gaps and size limits
privacy.md	What stays local
related.md	Related extensions and pages
release-notes.md	What changed in each version

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Shortcuts

The keyboard shortcuts Auric Artisan Formatter provides in VS Code and inside Formatter Studio.

VS Code

Windows and Linux	macOS	Command	When
`Ctrl` + `Alt` + `Shift` + `F`	`Cmd` + `Alt` + `Shift` + `F`	**Preview Formatting as a Diff**	An editor has focus
`Ctrl` + `Alt` + `Shift` + `P`	`Cmd` + `Alt` + `Shift` + `P`	**Open Formatter Studio**	Any time
`Ctrl` + `Alt` + `Shift` + `U`	`Cmd` + `Alt` + `Shift` + `U`	**Show Files That Need Formatting**	Any time

There is deliberately no shortcut for formatting every file. VS Code's own **Format Document** (`Shift` + `Alt` + `F` on Windows) and **Format Selection** use Auric once it is the default formatter. Assign shortcuts to other commands in **Preferences: Open Keyboard Shortcuts** by searching for `auricFormatter`.

Formatter Studio

Keys	What they do
`Ctrl` + `K`	Open the Studio command palette
Left and right arrows on the tabs	Switch between **01 Studio** and **02 Workspace**
Arrow keys, type-ahead, `Enter` in drop-downs	Choose a language, preset, category or view
`Escape`	Close the command palette or an open drop-down

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Glossary

Plain-language definitions of the terms used in the Auric Artisan Formatter documentation.

CI (continuous integration) — automated checks that run on every change, where `auric-format --check` can fail a build if files are unformatted.

CodeLens — clickable links VS Code shows above code; the preview uses them for Apply, Compare, Discard and Refresh.

Configuration file — a file such as `.auricformatrc` that holds formatting options for a project.

Default formatter — the formatter VS Code uses for a language, set with `editor.defaultFormatter`.

Directive — a comment in a file that talks to the formatter, such as `auric-format-ignore` or `@auric printWidth=120`.

Format on save — formatting a file automatically each time you save it.

Formatter Studio — the panel for trying options on your own code (called the Playground in the walkthrough).

Idempotent — formatting the output again changes nothing.

Native tier — languages with a real parser and printer, rebuilt from their syntax tree.

Option — one formatting choice, such as `printWidth` or `singleQuote`. There are 391.

Parser — the part that reads source code into a structure the formatter can print.

Passthrough — the lightest treatment, for unrecognised files: only whitespace at line ends, blank lines, line endings and the final newline.

Pragma — a marker such as `@format` in a file's leading comment that opts the file in.

Preset — a named starting set of option values, such as `auric` or `prettier`.

Print width — the line length the formatter tries to stay within.

Safety gate — the checks every result passes before it is written; on failure, the original text is kept.

Tier — how much the formatter is allowed to change for a language: native, universal or passthrough.

Token — a meaningful piece of code, such as a keyword, name, operator or string.

Universal tier — a structural engine for languages without a native printer; it changes only whitespace, one line in, one line out.

Workspace scan — formatting every supported file in memory to see what would change, without writing.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Limits and accuracy

What Auric Artisan Formatter's safety gate guarantees, its known gaps in version 0.1.0, and its size and time limits.

What the safety gate guarantees

With the default settings, a result is written only if the output re-reads to the same significant tokens, keeps every comment and every string's content, and parses again. If any check fails, the original text is kept. The idempotency check (formatting twice gives the same bytes) is off by default because it doubles the work; the `verified` preset turns it on.

The gate protects meaning, not taste: a result can pass every check and still not be the layout you want. Use Formatter Studio to tune it.

Known gaps in version 0.1.0

- **JSX and TSX** files containing JSX elements are formatted by the universal tier (indentation and spacing only), not pretty-printed.
- **Universal-tier languages** only change whitespace. Some only change spacing or trailing whitespace. See Supported languages.
- **Format Every File in the Workspace** on a folder's context menu formats every pending file in the workspace, not only that folder.
- **The command line** ships inside the extension and is not a separate npm package.

Limits

Limit	Default	Setting
Largest file formatted	4096 KB (`0` removes the limit)	`auricFormatter.maxFileSizeKb`
Parse time before a parse is abandoned	5000 ms (`0` waits indefinitely)	`auricFormatter.parseTimeoutMs`
Files in one workspace scan	4000	`auricFormatter.scanMaxFiles`
Formatter Studio preview	200,000 characters	—
Open native previews	24	—
Formatting options	391, in 17 categories	—

Deterministic output

The engine uses no clock, no randomness and no locale-dependent ordering, so the same input and configuration give the same output on every machine and in CI. Line width is measured in display columns, so wide characters such as CJK text and emoji break lines where a person would expect.

What it is not

- It is not a linter: it changes layout, not code behaviour, and does not report style errors beyond "needs formatting".
- It is not a backup: the gate prevents corruption, but formatting many files is still a change you should commit or stash first.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Privacy

What Auric Artisan Formatter does with your code and settings.

- **Everything runs locally.** Formatting, the workspace scan, Formatter Studio and the command line run on your machine (or next to the files in a remote setup). The engine has no runtime dependencies and makes no network requests.
- **No telemetry and no account.**
- **The workspace scan only reads.** It formats in memory; nothing is written until you choose a Format action.
- **Configuration** stays in your project files and VS Code settings.
- **Report a Formatting Problem** only opens a local document for you to review; nothing is sent until you choose to share it. It contains the formatter version, VS Code version, language, tier and the explanation for the active file, not the

file itself.

- **In an untrusted workspace**, JavaScript configuration files and formatter plugins are not run.
- **Links** to auricartisan.com open your browser only when you select them.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Release notes

What changed in each published version of Auric Artisan Formatter.

Formatting output is treated as part of the extension's public behaviour: a change that makes an already-formatted file format differently is at least a minor version and is listed under "Formatting changes", so a team can decide when to take the diff.

0.1.0 (23 September 2026)

The first Marketplace release.

Editor

- A native read-only formatting preview beside the source, with **Apply**, **Discard**, **Refresh** and **Compare Changes** in the editor title and CodeLens.
- Format document, selection and every pending file; format on save, optionally only changed lines; format as you type.
- A status bar item reporting the tier and any safety reversion.
- **Explain Formatting Decisions**, **Import Settings From Prettier**, **Generate a Configuration File**, **Switch Formatting Preset** and **Report a Formatting Problem**.
- A setup walkthrough on the Welcome page.

Formatter Studio

- Formatter Studio and a matching Explorer sidebar, with searchable drop-downs, a filterable option rail (category, changed-only, affects-this-file and experimental filters), **Find what matters**, and live **Result**, **Diff** and **Explain** views.
- **Full document** and **Editor selection** scopes, a preview toggle, and explicit **Yes, apply** and **No, discard**. Apply is bound to the original document and refuses stale, discarded, scratch and truncated previews.
- A side-by-side or inline diff with change highlights, change navigation and expandable context.
- A validated JSON rules editor and **Copy config**, which exports only the options you changed.
- A command palette on `Ctrl` + `K`, full light, dark, high-contrast, forced-colours and reduced-motion support, and state that survives a reload.

Workspace

- The in-memory workspace scan: the ***N* unformatted** status bar count and the **Workspace** tab, with sorting, filtering, **Format selected**, **Format all files** and per-file diffs. Nothing is written until you choose.
- Files are re-read and re-formatted at the moment they are written, and open unsaved files are written through the editor.
- Tracking of edits and file changes, with bounded rescans for new files; the preset or a configuration change re-runs the scan.
- **Check Formatting Without Writing (CI Mode)** reports line counts per file in Problems.
- New settings: `scanOnStartup`, `scanExclude`, `scanMaxFiles`, `confirmBeforeFormattingAll`.

Engine

- A zero-dependency engine that measures width in display columns, so CJK text, emoji and combining marks break where a person expects.
- 391 options in 17 categories, each with a type, a validated default and a command-line flag.
- 13 presets, including `prettier`, byte-compatible with Prettier 3 at its defaults.
- Native formatting for 31 modes, including JavaScript and TypeScript, and a universal structural engine for 220 more languages.
- The five-check safety gate, aware of which changes your options allow.
- Tiered fallback: a failed parse falls back to safe whitespace formatting.
- Configuration compatibility with `.auricformatrc`, `package.json`, `.editorconfig` and Prettier configuration, with per-language and per-glob overrides and inline directives.
- The `auric-format` command line with the familiar `--check`, `--write`, `--list-different` and related flags, and typo suggestions.
- A plugin mechanism for additional languages, where a failing plugin disables itself.

Not in this release

- A native JSX front end: `.jsx` and `.tsx` files containing JSX elements are formatted by the universal tier.

VS CODE EXTENSIONS › AURIC ARTISAN FORMATTER

Related

Other Auric Artisan extensions and pages that work alongside the Formatter.

Other Auric Artisan VS Code extensions

- Auric Artisan Studio — Code Health, Accessibility and Color checks that complement formatting.
- Auric Artisan Live — serve and preview the pages you format.
- Auric Artisan File System — create and rename the files you format.
- Auric Artisan Theme — the editor theme the Formatter Studio panel follows.

Setup

- Install and set up the Auric Artisan extensions
- Settings, sync and workspace trust

Auric Artisan pages

- VS Code extensions: https://auricartisan.com/vscode/
- Marketplace listing: https://marketplace.visualstudio.com/items?itemName=auric-artisan.auric-artisan-formatter
- Contact and support: https://auricartisan.com/company/contact/